SWAT+ Documentation

SWAT+ Documentation

docs.swat.tamu.edu

Contents

SWAT+ Documentation **17**
About this site . . . 17

Getting Started **18**
Pages . . . 18
Background . . . 18
Processes simulated . . . 18
Applications . . . 19
Running the model . . . 19
Source and license . . . 19
Spatial structure . . . 19
Input files . . . 20
Management . . . 20
Groundwater . . . 20
Additional processes . . . 20
Outputs . . . 21
Code . . . 21
Migration . . . 21
License . . . 21
Citing the software . . . 21
Citing the documentation . . . 22
Older versions . . . 22
62.0.1 . . . 22
Documentation for 62.0.1 . . . 23
SWAT+ tools . . . 24
Model executable . . . 24
Building from source . . . 25
Next . . . 25
The dataset . . . 26
Before running . . . 26
Running . . . 26
Output . . . 26
Warnings in diagnostics.out . . . 27
Troubleshooting . . . 27
Next . . . 27
The master file . . . 28
Input groups . . . 29
Outputs . . . 30
Log and check files . . . 31
Requirements . . . 31
Step 1: Delineate the watershed (QSWAT+) . . . 31
Step 2: Create HRUs . . . 32
Step 3: Build the input files (SWAT+ Editor) . . . 32
Step 4: Run . . . 32

Step 5: Look at output . . . 32
Step 6: Calibrate . . . 33
Common first-run problems . . . 33
Overview . . . 33
Pages . . . 33
SWAT 2009 processes not in SWAT+ 62.0.1 . . . 34
Page layout . . . 34
Overview . . . 34
Soil water balance . . . 35
Daily order of calculations . . . 35
Switches and parameters . . . 36
Output . . . 36
Related pages . . . 36
Overview . . . 37
Weather inputs . . . 37
Snow . . . 39
Potential evapotranspiration . . . 40
Switches and parameters . . . 41
Related pages . . . 41
Overview . . . 42
Canopy interception . . . 42
Surface runoff . . . 43
Surface runoff lag . . . 45
Soil water routing . . . 46
Channel routing . . . 48
Switches and parameters . . . 48
Related pages . . . 48
Overview . . . 49
Soil layers . . . 49
Actual evapotranspiration . . . 50
Soil temperature . . . 51
Switches and parameters . . . 53
Related pages . . . 53
Overview . . . 53
Peak runoff rate . . . 54
MUSLE . . . 54
Sub-daily erosion . . . 56
Sediment in lateral and tile flow . . . 56
Particle size . . . 57
Sediment lag . . . 57
Urban areas . . . 57
Filter strips, grassed waterways and user BMPs . . . 58
Channel erosion and deposition . . . 59
Switches and parameters . . . 59
Related pages . . . 59
Overview . . . 60
Plant types . . . 60
Heat units . . . 61
Dormancy . . . 61
Light interception and biomass . . . 62
Leaf area and canopy . . . 62
Water uptake . . . 63
Temperature stress . . . 64
Nitrogen and phosphorus . . . 64
Harvest index and yield . . . 65
Land use management . . . 65

Switches and parameters . . . 66
Related pages . . . 67
Overview . . . 68
Soil pools . . . 68
Temperature and water factors . . . 69
Humus mineralisation . . . 69
Residue decomposition . . . 69
Nitrification and ammonia volatilisation . . . 70
Denitrification . . . 70
Atmospheric deposition . . . 70
Nitrate movement . . . 71
Organic N and P with sediment . . . 71
Soluble phosphorus . . . 72
Mineral phosphorus transformations . . . 72
Lag of nutrients in surface runoff . . . 72
Algae, CBOD and dissolved oxygen in runoff . . . 73
Septic systems . . . 73
In-stream processes . . . 73
Switches and parameters . . . 73
Related pages . . . 73
Overview . . . 74
Pools . . . 75
Residue inputs . . . 75
Pool transformations . . . 76
Carbon in water and sediment . . . 77
Initial pools . . . 78
Parameters . . . 78
Related pages . . . 78
Overview . . . 79
Application . . . 79
Washoff . . . 79
Decay . . . 80
Plant uptake . . . 81
Transport in water . . . 81
Transport with sediment . . . 82
Channels . . . 82
Aquifers . . . 82
Parameters . . . 82
Related pages . . . 83
Overview . . . 83
Transport in soil water . . . 84
Salt chemistry . . . 84
Salinity stress on plants . . . 85
Salt uptake by plants . . . 85
Selenium reactions . . . 85
Sorption of constituents . . . 85
Inputs . . . 86
Aquifers, channels and water bodies . . . 86
Pathogens and heavy metals . . . 86
Related pages . . . 87
Overview . . . 87
Rating curve . . . 88
Flow routing . . . 88
Transmission losses and evaporation . . . 89
Sediment . . . 89
Water quality . . . 90

Water temperature . . . 92
Parameters . . . 92
Related pages . . . 93
Overview . . . 93
1D aquifer . . . 94
gwflow grid model . . . 96
Parameters . . . 97
Related pages . . . 97
Overview . . . 98
Reservoir water balance . . . 98
Release rules . . . 99
Reservoir sediment . . . 100
Reservoir nutrients . . . 100
Wetlands and paddies . . . 100
Parameters . . . 101
Related pages . . . 101
Overview . . . 102
Recall objects . . . 102
Export coefficient objects . . . 103
Delivery ratios . . . 103
Related pages . . . 103
Overview . . . 104
Structure . . . 104
Evaluation . . . 105
Related pages . . . 106
Older projects . . . 107
Simulation control . . . 107
Basin parameters . . . 107
Spatial objects and connectivity . . . 107
Climate . . . 107
Soils and land use . . . 108
Channels . . . 108
Aquifers, reservoirs and wetlands . . . 108
Management . . . 108
Initial conditions . . . 108
Structural practices . . . 108
Pollutants . . . 108
Groundwater . . . 108
Salts and constituents . . . 109
Water allocation . . . 109
Purpose . . . 109
Format . . . 109
Categories and default file names . . . 109
Missing and null files . . . 111
Weather data folders . . . 111
Output folder . . . 111
Older file.cio files . . . 111
Example . . . 112
Related files . . . 113
Purpose . . . 113
Format . . . 113
Simulation period . . . 114
Time step . . . 114
Weather outside the observed record . . . 115
Example . . . 115
Related files . . . 115

Purpose . . . 115
Format . . . 116
Output group rows . . . 117
Print timing . . . 118
Average annual values . . . 119
Example . . . 119
Related files . . . 120
Purpose . . . 120
Format . . . 121
Object counts . . . 121
Basin areas . . . 122
gwflow . . . 122
Example . . . 122
Related files . . . 122
Purpose . . . 123
Format . . . 123
PET method . . . 125
Extra input columns . . . 125
Crack flow . . . 125
Peak runoff rate . . . 126
Channel routing . . . 126
USLE C factor . . . 126
Carbon model . . . 126
Lapse rates . . . 127
Unit hydrograph . . . 127
Runoff method . . . 127
In-stream nutrients . . . 127
gwflow module . . . 128
Tillage effect on decomposition . . . 128
Example . . . 128
Related files . . . 128
Purpose . . . 129
Format . . . 129
Zero and missing values . . . 133
Initial soil water . . . 133
Surface runoff lag . . . 133
Peak rate factor . . . 133
Pesticides in aquifers . . . 134
Pesticide decay and temperature . . . 134
Uptake distribution . . . 134
Denitrification . . . 134
Frozen soil . . . 135
Dormancy threshold . . . 135
USLE C factor . . . 135
Sub-daily erosion . . . 135
CO2 . . . 135
Random numbers . . . 135
Calibration . . . 136
Example . . . 136
Related files . . . 136
Purpose . . . 138
Activation and file names . . . 138
Format . . . 138
Initial carbon pools . . . 140
Soluble carbon losses . . . 140
Organic fertilizer and manure . . . 140

Tillage and biological mixing . . . 140
Temperature factor . . . 141
Water factor . . . 141
Residue decomposition . . . 141
Notes on individual columns . . . 142
Calibration . . . 142
Example . . . 142
Related files . . . 142
Purpose . . . 143
Activation and file name . . . 143
Format . . . 143
Use in the carbon model . . . 145
Calibration . . . 145
Example . . . 145
Related files . . . 146
Purpose . . . 146
Format . . . 148
Area, location and elevation . . . 149
Weather station lookup . . . 149
Receiving object codes . . . 149
Hydrograph codes . . . 150
Inflow to HRUs and routing units . . . 150
Flow-control rule . . . 150
Export coefficient objects . . . 151
Groundwater flow (gwflow) models . . . 151
Routing loops . . . 151
Example . . . 151
Related files . . . 151
Purpose . . . 152
Format . . . 152
Name lookup . . . 153
Tile drainage . . . 153
Example . . . 154
Related files . . . 154
Purpose . . . 155
hru-lte.hru . . . 155
Name matching . . . 157
Daily water balance . . . 157
Plant growth . . . 160
Irrigation . . . 161
Erosion . . . 161
Area units . . . 161
Calibration . . . 162
Example . . . 162
Related files . . . 162
Purpose . . . 163
rout_unit.def . . . 164
rout_unit.ele . . . 165
rout_unit.rtu . . . 167
ls_unit.def . . . 168
ls_unit.ele . . . 169
aqu_catunit.ele . . . 170
Columns in earlier documentation . . . 171
Related files . . . 171
Purpose . . . 172
chan-surf.lin . . . 172

aqu_cha.lin . . . 173
Example . . . 175
Related files . . . 175
Purpose . . . 176
hydrology.hyd . . . 176
topography.hyd . . . 180
field.fld . . . 181
shade_factor.shf . . . 181
Calibration . . . 182
Examples . . . 182
Related files . . . 182
Purpose . . . 183
recall.rec . . . 183
Recall data file . . . 184
recall_db.rec . . . 185
pest.com . . . 186
Salt and constituent recall files . . . 186
Region files . . . 186
Output . . . 187
Columns in earlier documentation . . . 187
Example . . . 187
Related files . . . 187
Purpose . . . 188
Common rules . . . 188
weather-sta.cli . . . 189
weather-wgn.cli . . . 190
Observed data lists . . . 193
atmodep.cli . . . 193
Salt and constituent deposition . . . 195
Related files . . . 195
Purpose . . . 196
Format . . . 196
Reading and storage . . . 197
Missing days . . . 197
Sub-daily runs . . . 197
Elevation adjustment . . . 198
Calibration . . . 198
Example . . . 198
Related files . . . 198
Purpose . . . 199
Format . . . 199
Missing days . . . 200
Elevation adjustment . . . 200
Calibration . . . 200
Example . . . 200
Related files . . . 201
Purpose . . . 201
Format . . . 202
Example . . . 202
Related files . . . 202
Purpose . . . 202
Format . . . 202
Snow processes . . . 203
Calibration and decision tables . . . 204
Example . . . 204
Related files . . . 205

Purpose . . . 205
Format . . . 205
Layering . . . 207
Derived layer properties . . . 208
Use of the soil properties . . . 208
Calibration and conditions . . . 209
soils_lte.sol . . . 210
Example . . . 210
Related files . . . 210
Purpose . . . 211
Format . . . 211
Name matching . . . 212
Use at the start of the run . . . 212
Land use change . . . 213
Calibration groups . . . 213
Septic systems . . . 213
Example . . . 213
Related files . . . 214
Purpose . . . 214
cntable.lum . . . 215
cons_practice.lum . . . 217
ovn_table.lum . . . 219
Related files . . . 220
Purpose . . . 221
Files on the channel line . . . 221
initial.cha . . . 221
initial.cha_cs . . . 222
temperature.cha . . . 222
Legacy channel files . . . 225
Example . . . 226
Related files . . . 226
Purpose . . . 227
Links between the files . . . 227
channel-lte.cha . . . 227
hyd-sed-lte.cha . . . 228
sed_nut.cha . . . 230
nutrients.cha . . . 231
Channel geometry and rating curve . . . 234
Flow routing . . . 234
Sediment . . . 235
Nutrients . . . 237
Pesticides . . . 237
Calibration and decision tables . . . 237
Example . . . 237
Related files . . . 238
Purpose . . . 239
aquifer.aqu . . . 240
Aquifer water balance . . . 243
Nitrate and phosphorus . . . 244
Pesticides, salts and constituents . . . 244
Aquifer flow to channels with aqu_cha.lin . . . 244
Calibration and decision tables . . . 244
initial.aqu . . . 245
initial.aqu_cs . . . 245
Example . . . 245
Related files . . . 246

Purpose . . . 247
reservoir.res . . . 247
initial.res . . . 248
hydrology.res . . . 248
Release rules . . . 250
sediment.res . . . 251
nutrients.res . . . 251
weir.res . . . 253
Calibration . . . 253
Example . . . 253
Related files . . . 254
Purpose . . . 255
wetland.wet . . . 255
hydrology.wet . . . 256
Wetland behaviour . . . 257
Management and decision tables . . . 259
Example . . . 259
Related files . . . 259
Purpose . . . 260
Format . . . 260
harv.ops . . . 261
graze.ops . . . 261
irr.ops . . . 262
chem_app.ops . . . 263
fire.ops . . . 263
sweep.ops . . . 263
Example . . . 264
Related files . . . 264
Purpose . . . 265
Format . . . 266
Condition variables . . . 267
Land use and management actions . . . 269
Reservoir release actions . . . 274
Land use scenarios . . . 275
Example . . . 275
Related files . . . 276
Purpose . . . 276
Format . . . 277
Plant types . . . 279
Heat units to maturity . . . 279
Leaf area development . . . 280
Radiation use efficiency . . . 280
Perennials and trees . . . 280
Nutrient uptake . . . 280
Residue lignin . . . 281
Values set by the model . . . 281
Calibration . . . 281
Example . . . 281
Related files . . . 281
Purpose . . . 282
fertilizer.frt . . . 283
manure_db.frt . . . 284
manure_om.frt . . . 284
Related files . . . 285
Purpose . . . 286
Format . . . 286

Tillage mixing . . . 288
Biological mixing . . . 288
Surface roughness . . . 288
Example . . . 288
Related files . . . 289
Replacement files . . . 289
Keyword migration . . . 290
Example . . . 290
Related files . . . 290
Purpose . . . 291
Format . . . 291
Example . . . 291
Related files . . . 291
Purpose . . . 291
Format . . . 291
Example . . . 292
Related files . . . 293
management.sch . . . 293
puddle.ops . . . 296
transplant.ops . . . 297
co2_yr.dat and co2.out . . . 297
object.prt . . . 298
Purpose . . . 299
Initialization files . . . 299
plant.ini . . . 299
soil_plant.ini . . . 301
nutrients.sol . . . 302
om_water.ini . . . 303
Related files . . . 304
Purpose . . . 305
File layout . . . 305
tiledrain.str . . . 305
septic.str . . . 306
septic.sep . . . 308
filterstrip.str . . . 308
grassedww.str . . . 309
bmpuser.str . . . 311
satbuffer.str . . . 311
urban.urb . . . 312
References . . . 312
Related files . . . 313
Purpose . . . 314
pesticide.pes . . . 314
pest_hru.ini . . . 317
pest_water.ini . . . 317
pest_metabolite.pes . . . 318
Related files . . . 318
Purpose . . . 319
pathogens.pth . . . 319
path_hru.ini . . . 320
path_water.ini . . . 321
Behaviour in 62.0.1 . . . 321
Related files . . . 321
Purpose . . . 322
hmet_hru.ini . . . 322
Related files . . . 322

Purpose . . . 323
Activation . . . 323
File overview . . . 323
Format conventions . . . 324
codes.gw . . . 324
zones.gw . . . 325
cells.gw . . . 326
cellcon.gw . . . 326
outputs.gw . . . 327
chancell.gw . . . 327
gwflow.con . . . 328
hrucell.gw . . . 328
lsucell.gw . . . 328
chan_depth.gw . . . 328
pumpex.gw . . . 329
hru_pump.gw . . . 329
tile.gw . . . 329
rescell.gw . . . 330
gwflow.wetland . . . 330
floodplain.gw . . . 330
gwflow_canal.con . . . 330
phreato.gw and phreato_cell.gw . . . 330
tvheads.gw . . . 330
solute.gw . . . 331
cell_sol.gw . . . 331
minerals.gw . . . 331
ponds.gw, pond_cell.gw and pond_div.gw . . . 331
sw_group.gw . . . 332
transit.gw . . . 332
Example . . . 332
Related files . . . 333
Purpose . . . 333
Activation . . . 334
File overview . . . 334
Missing files . . . 334
salt_hru.ini . . . 335
salt_aqu.ini . . . 335
salt_channel.ini . . . 335
salt_res . . . 335
salt_irrigation . . . 336
salt_fertilizer.frt . . . 336
salt_road . . . 336
salt_plants . . . 337
salt_uptake . . . 337
salt_urban . . . 338
Example . . . 338
Related files . . . 338
Purpose . . . 339
constituents.cs . . . 339
File overview . . . 340
Missing files . . . 340
cs_hru.ini . . . 340
cs_aqu.ini . . . 341
cs_channel.ini . . . 341
cs_res . . . 341
cs_irrigation . . . 342

fertilizer.frt_cs . . . 342
cs_reactions . . . 342
cs_uptake . . . 343
cs_plants_boron . . . 343
cs_urban . . . 343
cs_streamobs . . . 343
Example . . . 343
Related files . . . 344
Purpose . . . 345
Files and activation . . . 345
Daily sequence . . . 345
place_of_use.wro . . . 346
point_of_diver.wro . . . 348
water_hru_irr.wal . . . 348
wtps_wuses.wal . . . 349
water_treat.wal . . . 349
water_use.wal . . . 350
outside_src.wal . . . 350
om_treat.wal, om_use.wal and om_osrc.wal . . . 350
water_tower.wal . . . 351
water_canal.wal . . . 351
water_pipe.wal . . . 352
outside_rcv.wal . . . 352
Decision tables . . . 352
Limitations in 62.0.1 . . . 352
Example . . . 353
Outputs . . . 354
Related files . . . 354
Pages . . . 355
Recent changes . . . 355
Filename pattern . . . 355
Output folder . . . 356
File layout . . . 356
Print timing . . . 356
print.prt rows and files . . . 356
Files written every run . . . 358
Related files . . . 358
Purpose . . . 359
Format . . . 359
Aggregation . . . 361
Example . . . 361
Related files . . . 361
Purpose . . . 362
Files at a glance . . . 362
Leading columns . . . 363
channel_sd files . . . 363
channel_sdmorph files . . . 364
sd_chanbud files . . . 365
Sediment budget files . . . 365
Basin files . . . 366
Sub-daily outflow . . . 366
Legacy channel files . . . 367
Example . . . 367
Related files . . . 367
Purpose . . . 368
Aquifer . . . 368

Reservoir . . . 369
Crop yield . . . 371
Plant and weather . . . 372
Related files . . . 373
Purpose . . . 374
Files and switches . . . 374
Nutrient balance columns . . . 374
Losses columns . . . 375
Aggregation . . . 375
Example . . . 376
Related files . . . 376
Purpose . . . 376
Hydrograph columns . . . 377
hydout files . . . 377
hydin files . . . 378
deposition files . . . 378
ru files . . . 378
hydcon.out . . . 379
Constituent hydrographs . . . 379
Print timing . . . 379
Example . . . 379
Related files . . . 379
Purpose . . . 380
Format . . . 380
Values by recall time step . . . 381
Aggregation . . . 381
Example . . . 381
Related files . . . 382
Purpose . . . 382
Files and switches . . . 382
Value types and aggregation . . . 383
Flat HRU files . . . 383
Per-layer HRU files . . . 384
Landscape unit files . . . 387
Layer count file . . . 387
Legacy CSU carbon outputs . . . 387
Example . . . 388
Related files . . . 388
Purpose . . . 389
Output switches . . . 389
Basin water balance . . . 390
Cell water balance . . . 391
Heat balance . . . 391
Observation wells . . . 392
HRU pumping . . . 392
Diagnostic files . . . 392
Hydrograph separation . . . 393
Files always written . . . 393
Related files . . . 393
Purpose . . . 394
simulation.out . . . 394
diagnostics.out . . . 395
checker.out . . . 396
files_out.out . . . 396
success.fin . . . 396
area_calc.out . . . 397

Other files . . . 397
Checks after a run . . . 397
Related files . . . 397
Purpose . . . 397
spcheck.py . . . 398
swat_output_to_csv_files.py . . . 398
Related files . . . 399
Pages . . . 400
Requirements . . . 400
Configure . . . 400
Compile . . . 400
Compiler options . . . 401
Linking . . . 401
Install . . . 401
Required software . . . 402
Starting Visual Studio . . . 402
Cloning the repository . . . 402
Configuring, compiling, running . . . 403
Resources . . . 404
Cost . . . 404
Starting an instance . . . 404
Building . . . 405
Running and debugging . . . 405
Using your own dataset . . . 405
Stopping the instance . . . 406
Deleting the instance . . . 406
Test script . . . 407
Running the test at 62.0.1 . . . 407
Adding a scenario . . . 407
Continuous integration . . . 408
Version numbers . . . 408
Version in the executable and outputs . . . 408
Building a tagged version . . . 408
Making a release . . . 409
Interoperability and portability . . . 409
Readability . . . 410
Reliability . . . 410
Arrays . . . 411
Dynamic memory and pointers . . . 411
Looping . . . 411
Functions and procedures . . . 412
I/O . . . 412
Obsolescent features to avoid . . . 412
Source files . . . 413
References . . . 413

Code Of Conduct **414**
Repositories . . . 414
Reporting a problem . . . 414
Contributing code . . . 414
Contributing to the documentation . . . 415
Pages . . . 415
References . . . 415
Purpose . . . 416
Run sequence . . . 416
Input files . . . 416
Adjustment steps . . . 421

Output files . . . 423
Limits in 62.0.1 . . . 424
Related pages . . . 424
Purpose . . . 425
Files . . . 425
Run sequence . . . 425
Change types . . . 426
Object types . . . 426
Unknown names . . . 427
Related pages . . . 427
Purpose . . . 427
cal_parms.cal . . . 427
calibration.cal . . . 428
Parameter names . . . 429
Related pages . . . 433
Purpose . . . 434
Condition lines . . . 434
Condition variables . . . 434
Regional calibration . . . 435
Scheduled land use updates . . . 435
Related pages . . . 435
Purpose . . . 436
Steps . . . 436
Writing calibration.cal . . . 436
Reading results . . . 437
Related pages . . . 437
Pages . . . 437
file.cio . . . 437
Simulation settings . . . 438
Basin settings . . . 439
Climate . . . 439
HRUs . . . 439
Land use and management . . . 440
Databases and initial conditions . . . 440
Running the model . . . 440
Outputs . . . 441
Changes to try . . . 443
file.cio . . . 443
Simulation settings . . . 444
Basin settings . . . 445
Climate . . . 445
HRU, routing unit, aquifer and channel . . . 446
Land use and management . . . 446
Other files . . . 447
Running the model . . . 447
Outputs . . . 448
Changes to try . . . 450
Files read . . . 451
Switches . . . 451
Ames_sub1 setup . . . 451
Running the notebook . . . 452
Graphs . . . 452
Related pages . . . 452
Simulation period . . . 453
Daily or monthly output . . . 453
PET method . . . 454

Green-Ampt infiltration . . . 454
HRU addition . . . 454
Carbon model . . . 455
This site and the FORD reference . . . 455
Local build . . . 455
Purpose . . . 456
Pages . . . 456
Checks after every run . . . 456
Purpose . . . 456
Message locations . . . 456
Errors that stop the run . . . 457
Messages in diagnostics.out . . . 458
Names matched without a message . . . 460
Files the model creates . . . 460
Old-format inputs . . . 460
Runs that stop without a message . . . 461
Related files . . . 461
Purpose . . . 461
Balance check . . . 461
Precipitation . . . 462
Surface runoff . . . 462
Evapotranspiration . . . 462
Management . . . 462
Percolation . . . 463
Lateral and tile flow . . . 463
Basin totals . . . 463
Related files . . . 463
Purpose . . . 464
One run per processor core . . . 464
Build type . . . 464
Time step . . . 464
Output volume . . . 464
Object count and storage . . . 465
Summary . . . 465

SWAT+ Documentation

An open-source watershed model with plain-text inputs and outputs.

Get started What is SWAT+

- **Getting started**

`Install SWAT+, run the reference dataset, and learn the shape of a project folder.`

- **Model theory**

`Hydrology, soil, plants, channels, aquifers, reservoirs, nutrients, carbon, pesticides, salts.`

- **Input reference**

`Every input file, with its columns, units and defaults.`

- **Output reference**

`Every output file group, with column lists and aggregation rules.`

- **Developer guide**

`Build from source, test, tag releases, contribute.`

- **Calibration**

`Soft and hard calibration workflows.`

- **Tutorials**

`Walkthroughs of the Ames_sub1 and Osu_1hru reference datasets.`

- **FAQ**

`Common runtime errors, water-balance issues, performance tips.`

About this site

This documentation describes SWAT+ revision 62.0.1. Each SWAT+ release has a separate version of the site, chosen with the version selector at the top of the page; `latest` is the newest.

The pages are written for model users. Every statement about model behaviour has been checked against the Fortran source of the release, and the commit is recorded in each page's front matter. Where the source does something unexpected, the page says what the model does. Source files and variable names are listed in a collapsed Sources block at the end of each page.

The model source is at github.com/swat-model/swatplus. Every page can be downloaded as a PDF, alone or with its section (Download as PDF).

Getting Started

This section gets you from a fresh machine to a running SWAT+ simulation.

If you have never run the model before, follow the pages in order:

1. Install. Get the SWAT+ executable on your machine.
2. Quick start: Ames_sub1. Run the small reference dataset shipped with the source repository.
3. Project folder anatomy. Understand what is in a SWAT+ project directory and what each group of files does.
4. Your first watershed. Build a project for a new area using QSWAT+ and run it.

If you only want to read about the model first, skip ahead to Model theory.

SWAT+ is a watershed model. It simulates water, sediment, nutrient, pesticide, and crop yield response to climate and management at scales from a single field up to a large river basin.

This section covers what the model is, how it differs from the original SWAT, how to cite it, and how releases are numbered.

Pages

- What is SWAT+
- SWAT vs SWAT+
- Citing and license
- Release notes

The Soil and Water Assessment Tool Plus (SWAT+) is an open-source watershed model developed jointly by the USDA Agricultural Research Service (USDA-ARS) and Texas A&M AgriLife Research, with contributions from Colorado State University, the USDA-ARS Hydrology and Remote Sensing Laboratory and other institutions.

SWAT+ simulates the quantity and quality of surface water and groundwater, from small watersheds to large river basins. It is used to evaluate the effects of land use, land management, soils and climate change on water resources.

Background

Over more than two decades the original Soil and Water Assessment Tool (SWAT) became one of the most widely used watershed models in the world. Its many applications showed limits in how a watershed could be represented, and the additions made over the years left a code base that was hard to maintain and extend. SWAT+ is a restructured version of SWAT. Most of the process equations are the same, but the code is organized around objects and the input files are relational, which makes the model easier to maintain and lets new science be added as separate modules. SWAT+ also allows a more flexible spatial representation: landscape units, HRUs, aquifers, channels and reservoirs can be connected in any arrangement through connection files. The main differences are listed on SWAT vs SWAT+.

Processes simulated

For each HRU, channel, aquifer, reservoir and wetland, SWAT+ keeps a daily mass balance of:

- water: precipitation, snow, evapotranspiration, surface runoff, lateral flow, percolation, groundwater flow and channel routing
- sediment: detachment, transport and deposition on the landscape and in channels
- nutrients: nitrogen and phosphorus cycling in the soil, plant uptake, transport with runoff and subsurface flow, and in-stream transformations
- carbon: soil organic carbon and residue pools
- pesticides and pathogens: application, decay and transport
- salts and other constituents
- plant growth: biomass, leaf area, yield, and water, nutrient and temperature stress

Management (planting, tillage, fertilizer and manure, irrigation, harvest, grazing) can follow fixed schedules or rules in decision tables. Water can be allocated between sources and users, and structural practices such as tile drains, filter strips and terraces can be represented.

Output can be written daily, monthly, yearly and as average annual values. The Model Theory section describes each process.

Applications

- effects of management practices on water yield and water quality
- soil erosion and non-point source pollution
- crop yield response to climate and management
- watershed planning and total maximum daily load (TMDL) studies
- climate change impacts on water resources
- irrigation and water allocation

Running the model

SWAT+ is a command-line program. It reads a folder of plain-text input files that describe the watershed, climate, soils, land use, management and simulation period, and writes plain-text (and optionally CSV) output files to the same folder.

Most users do not write the input files by hand. They use the supporting tools:

- **QSWAT+**, a QGIS plugin, delineates the watershed from a DEM, creates HRUs from soil, land use and slope maps, and writes the project database.
- **SWAT+ Editor** edits the inputs of a project, writes the input files and runs the model.
- **SWAT+ Toolbox** runs sensitivity analysis, calibration and other tasks on a finished project.

All three can be installed from install.swat.tamu.edu. See Install.

Source and license

The source code is at github.com/swat-model/swatplus and is released under the GNU Lesser General Public License 2.1. See Citing and license.

Sources

- `README.md` and `CITATION.cff` in the source repository at 62.0.1.
- Input file groups: `src/input_file_module.f90`.

SWAT+ is the restructured successor to SWAT (Soil and Water Assessment Tool). Most of the process equations are the same. The code structure, the input files and the way parts of a watershed are connected are different. The SWAT column below refers to SWAT2012, the last release of the original model (`swat-model/swat2012`).

Spatial structure

	SWAT2012	SWAT+
Units	subbasins, each with HRUs and one main channel	landscape units, routing units, HRUs, channels, aquifers, reservoirs, wetlands, point sources, each a separate object
Connections	fixed: HRUs drain to their subbasin's channel, subbasins drain downstream in numbered order	defined in connection files (`*.con`): any object can send any part of its flow (surface runoff, lateral flow, tile flow, recharge) to any other object
Floodplain and upland	not separated	landscape units can separate upland and floodplain areas

The connection files make it possible, for example, to route upland runoff across a floodplain before it reaches the channel, to send tile flow from one HRU to a wetland, or to divide a reservoir's outflow between two channels.

Input files

SWAT2012 used one set of files per subbasin and HRU (`.sub`, `.hru`, `.mgt`, `.gw`, `.rte`, `.sol`, …), each repeating the full parameter list.

SWAT+ uses relational input files grouped by purpose. A parameter set is written once, given a name, and referenced by name from every object that uses it. For example, one row of `hydrology.hyd` can serve hundreds of HRUs. The master file `file.cio` lists every input file by group:

- simulation: `time.sim`, `print.prt`, `object.cnt`
- basin: `codes.bsn`, `parameters.bsn`, `carbon.bsn`
- climate: `weather-sta.cli`, `weather-wgn.cli`, `pcp.cli`, `tmp.cli` and the station files
- connections: `hru.con`, `rout_unit.con`, `aquifer.con`, `chandeg.con`, `reservoir.con`, `recall.con`, …
- HRUs, land use and management: `hru-data.hru`, `landuse.lum`, `management.sch`, `cntable.lum`, decision tables (`*.dtl`)
- hydrology: `hydrology.hyd`, `topography.hyd`, `field.fld`
- soils and plants: `soils.sol`, `nutrients.sol`, `plants.plt`, `plant.ini`
- databases: fertilizer, tillage, pesticide, urban and other parameter tables

The Input Reference describes each file.

Management

SWAT2012 scheduled operations in the `.mgt` file of each HRU, by date or by fraction of heat units.

SWAT+ management schedules (`management.sch`) are shared between HRUs, and can combine dated operations with decision tables (`*.dtl`). A decision table checks conditions every day (soil water, plant growth stage, date, crop rotation year, reservoir level, and so on) and applies an action when they are met. Decision tables also drive reservoir releases, flow control and land use change. See Decision tables.

Groundwater

SWAT2012 had a shallow and a deep aquifer for each HRU, with the shallow aquifer feeding the main channel of its subbasin.

SWAT+ aquifers are separate objects, usually one per landscape unit or group of HRUs, connected to channels through `aquifer.con`. SWAT+ also includes gwflow, a gridded groundwater module that simulates water table heads and exchanges with channels, soils, tile drains, reservoirs and wells cell by cell. See Aquifers.

Additional processes

SWAT+ adds or extends: a carbon module with soil organic matter pools, water allocation between sources and users, salts and other constituents, rice paddies, and a sediment budget for channels. Each is described in the Model

Theory section.

Outputs

SWAT2012 wrote a fixed set of files (`output.hru`, `output.sub`, `output.rch`, `output.std`).

SWAT+ writes one file per object type, process and time step, switched on and off in `print.prt`. Examples from the `Ames_sub1` reference dataset:

```
basin_wb_aa.txt        basin water balance, average annual
hru_wb_aa.txt          HRU water balance, average annual
channel_sd_day.txt     channel flow and loads, daily
basin_aqu_aa.txt       aquifer, average annual
```

The suffix gives the time step: `_day`, `_mon`, `_yr` or `_aa`. Each row of `print.prt` switches the four time steps on or off. See the Output Reference.

Code

	SWAT2012	SWAT+
Language	Fortran, fixed form	Fortran, free form
Organization	shared common blocks	modules grouped by process and object
Source	released as archives	public on GitHub, LGPL-2.1
Build	hand-written makefiles	CMake

Migration

There is no automatic converter from a SWAT2012 project to SWAT+. Rebuild the project in QSWAT+ from the same DEM, soil, land use and climate data, then transfer calibrated parameter values by hand, checking each against the SWAT+ units and meaning.

Sources

- Connection and input file names: `type input_con` and the other `input_*` types in `src/input_file_module.f90`.
- Aquifer objects: `src/aquifer_module.f90`, read by `aqu_read.f90`. Gridded groundwater: `gwflow_*.f90`. Spatial aquifer connections (`aquifer2d.con`): `aqu2d_read.f90`, `aqu2d_init.f90`.
- Build: `CMakeLists.txt` compiles `src/*.f90` free form (`-free` for Intel, `-ffree-line-length-none` for GNU) into one executable.
- SWAT2012: `swat-model/swat2012`.

License

SWAT+ is released under the GNU Lesser General Public License, version 2.1 (LGPL-2.1). The full text is in the `LICENSE` file of the source repository.

Citing the software

Cite the model release used in the study, with its version and DOI. The author list below is the one in the repository's citation file at 62.0.1:

> Arnold, J. G., Čerkasova, N., White, M. J., Bailey, R., Thorp, K., Jeong, J., Zhang, X., Ugraskan, T., van Griensven, A., Rathjens, H., Raj, C., Cai, X., Geter, W. F., David, O., Carlson, J. R., and Chawanda, C. J. Soil and Water Assessment Tool Plus (SWAT+), version 62.0.1. https://doi.org/10.5281/zenodo.14983533

Releases are archived on Zenodo:

DOI	Points to
10.5281/zenodo.14983533	all versions; resolves to the newest archived release
10.5281/zenodo.22919722	release 62.0.0
10.5281/zenodo.14983534	release 61.0.2

When a release has a version DOI, cite that DOI so the exact code can be found again.

GitHub shows a "Cite this repository" button on the repository page, built from the same citation file, with APA and BibTeX forms.

Citing the documentation

When a statement comes from this site, cite the page and the documentation version shown in the version selector at the top of the page, for example "SWAT+ Documentation, revision 62.0.1, Soft calibration, https://docs.swat.tamu.edu/62.0.1/calibration/soft-calibration/".

Older versions

Revisions before the move to GitHub are in the modular_swatplus repository on Bitbucket.

Sources

- `CITATION.cff`, `LICENSE` and `README.md` at 62.0.1.
- Zenodo records 14983533 (concept), 22919722 and 14983534, checked 20 October 2026.

SWAT+ releases are listed on the GitHub releases page, with prebuilt executables and the merged pull requests of each release. Version numbers are explained in Tagging and releases.

This page lists the changes in each release that affect model users: input files, outputs and results. Each version of this documentation describes one release; use the version selector at the top of the page for earlier ones.

62.0.1

Released 19 October 2026.

Input files

- `file.cio`, `water_rights` line: now names two files, the places of use (`place_of_use.wro`) and the points of diversion (`point_of_diver.wro`), in place of the three water rights files. The water allocation input files were reorganized around places of use, points of diversion and return; see Water allocation. A `file.cio` written for 62.0.0 or earlier must be updated; see Older file.cio files.
- `parameters.bsn`: two columns that were unused are now read as the pesticide groundwater mixing factor and a switch for temperature-dependent pesticide decay. SWAT+ Editor still labels them `lin_sed` and `exp_sed`; a value of 1 in the second turns temperature-dependent decay on. See parameters.bsn.
- `pesticide.pes`: three new columns for plant uptake. Files in the older layout are still read, with no plant uptake. See pesticide.pes.
- Recall: the recall file named in `file.cio` is read (instead of a fixed name), and empty recall files are no longer created. See recall.
- `exco.exc` and `exco_om.exc`: the original layout is restored.
- Manure: a manure database (`manure_db.frt`, `manure_om.frt`) and manure allocation inputs were added. See manure allocation.
- Transplanting: transplant data are read from `transplant.ops`; `transplant.plt` is no longer read.

Outputs

- Water allocation outputs were replaced by place-of-use based files.
- New sediment budget outputs for channels, stream orders and the basin, with corrected average annual values.
- Rice paddy output, and paddy water included in the ET reports.
- gwflow outputs now follow their `print.prt` and gwflow switches, and the average annual storage change is computed correctly.
- The legacy carbon output no longer writes duplicate rows for each HRU.

See the Output Reference for the files.

Model changes

- Rice paddies: seepage, weir spill volume and the impoundment switch day were revised, and soil evaporation is skipped while paddy water remains.
- Canopy interception is applied for all runoff methods.
- Pesticides: temperature-dependent decay, plant uptake and mass balance corrections.
- Water allocation and irrigation were reworked; the irrigation days of an unlimited source are reset correctly.

Calibration

- `aquifer_delay` (gwflow) is changed in days.
- New object types `gwf_riv` and `gwf_sgl` for gwflow channel and single-value parameters.
- `nperco_lchtile` changes the right parameter.

See Parameter change files.

Fixes

- A crash in tillage mixing for plant communities with more than 12 plants.
- An endless loop when all operations of a management schedule share one date.
- Root depth of perennials stuck at 25.4 mm at a cold start.
- Release executables built with gfortran no longer stop with a floating-point trap on macOS, and the macOS executable no longer needs Homebrew's gcc libraries.

Documentation for 62.0.1

This version of the documentation was rewritten against the 62.0.1 source. Pages now describe what the model does, including inputs that are read but have no effect and behaviour that differs from the input file labels, so that results can be interpreted correctly. Examples:

- Measured PET is replaced by the Hargreaves estimate within the same day, so `pet` 3 in `codes.bsn` gives Hargreaves results (codes.bsn).
- Name lookups between files fall back to the first entry without a message when a name does not match; check `diagnostics.out` and the names themselves (weather-sta.cli, hru-data.hru).
- Several basin parameters cannot be changed through `calibration.cal` (Parameter change files).
- Manure allocation, street sweeping and the pathogen simulation have no effect, and water allocation behaves as its inputs describe only for a single place of use (Water allocation).

Each page states these points where they apply.

Sources

- SWAT+ 62.0.1 release and `git log 62.0.0..62.0.1` in the source repository.
- Input changes: `type input_water_allocation` in `src/input_file_module.f90` and `read (107,*) name, in_wallo` in `src/readcio_read.f90`; `pestgwfact` and `temp_decay` replacing `spcon` and `spexp` in `src/basin_module.f90`; `src/pesticide_data_module.f90`; `src/recall_read.f90` and `src/recalldb_read.f90`; `src/exco_db_read.f90` and `src/exco_read_om.f90`; `src/plant_transplant_read.f90`.

- Outputs: `src/wallo_pou_output.f90`, `src/basin_sedbud_output.f90`, `src/ch_sedbud_output.f90`, `src/chord_sedbud_output.f90`, `src/gwflow_output.f90`.
- Build: `CMakeLists.txt` (gfortran flags and macOS linking).

SWAT+ has two parts: the model itself, a command-line program, and the tools used to build and edit projects (QSWAT+, SWAT+ Editor and SWAT+ Toolbox). Most users install the tools and let SWAT+ Editor run the model. The model can also be downloaded or built without the tools.

SWAT+ tools

install.swat.tamu.edu has one-line installers for Windows, Linux and macOS. Each installer can be run again at any time to update.

Install all three tools at once:

```
# Linux and macOS
curl -fsSL https://install.swat.tamu.edu/all.sh | bash

# Windows (PowerShell)
irm https://install.swat.tamu.edu/all.ps1 | iex
```

Or one at a time:

Tool	Purpose	Linux and macOS	Windows (PowerShell)
QSWAT+	QGIS plugin that delineates the watershed, creates HRUs and writes the project database. Also installs the SWAT+ support data and TauDEM. Needs QGIS.	`curl -fsSL https://install.swat.tamu.edu/qsp.sh | bash`	`irm https://install.swat.tamu.edu/qsp.ps1 | iex`
SWAT+ Editor	Edits project inputs, writes the input files and runs the model.	`curl -fsSL https://install.swat.tamu.edu/spe.sh | bash`	`irm https://install.swat.tamu.edu/spe.ps1 | iex`
SWAT+ Toolbox	Sensitivity analysis, calibration and other tasks on a finished project.	`curl -fsSL https://install.swat.tamu.edu/spt.sh | bash`	`irm https://install.swat.tamu.edu/spt.ps1 | iex`

On Linux and macOS the tools go to `~/.local/share/SWATPlus/`, and on Windows to `%USERPROFILE%\SWATPlus\`.

Model executable

Every release on the GitHub releases page has prebuilt executables. For 62.0.1:

File	System
`swatplus-62.0.1-gnu-win_amd64-Rel.zip`	Windows, 64-bit (gfortran build)
`swatplus-62.0.1-ifo-win_amd64-Rel.zip`	Windows, 64-bit (Intel Fortran build)
`swatplus-62.0.1-gnu-lin_x86_64-Rel.zip`	Linux, x86-64 (gfortran build)
`swatplus-62.0.1-ifx-lin_x86_64-Rel.zip`	Linux, x86-64 (Intel Fortran build)
`swatplus-62.0.1-gnu-lin_aarch64-Rel.zip`	Linux, ARM64
`swatplus-62.0.1-gnu-mac_arm64-Rel.zip`	macOS, Apple silicon
`swatplus-62.0.1-ifo-mac_x86_64-Rel.zip`	macOS, Intel

Each zip contains a single executable with the same name (plus `.exe` on Windows). The name tells you the version, the compiler (`gnu` for gfortran, `ifo` for Intel Fortran Classic, `ifx` for Intel Fortran), the system and architecture,

and `Rel` for an optimized release build. The gfortran and Intel builds give results that can differ slightly in the last digits.

Unzip the file and put the executable next to your project folder or in a folder on your `PATH`. On Linux and macOS, make it executable:

```
chmod +x swatplus-62.0.1-gnu-lin_x86_64-Rel
```

On macOS the first start of a downloaded program may be blocked; allow it under System Settings, Privacy and Security, or remove the quarantine flag with `xattr -d com.apple.quarantine <file>`.

To check the download, run it in an empty folder:

```
mkdir empty && cd empty
../swatplus-62.0.1-gnu-lin_x86_64-Rel
```

It prints a banner with the revision number and stops because there is no project:

```
                  SWAT+
             Revision 62.0.1
      Soil & Water Assessment Tool
GNU (13.3.0), 2026-10-19 22:51:37, Linux
    Program reading . . . executing

 Cannot find object.cnt input file
```

It also writes a few files (`diagnostics.out`, `simulation.out`, `codes.bsn` and others) into the folder. Run the model from inside a project folder; it reads its inputs from the current folder and writes the outputs there.

Building from source

Build from source to change the model or to use a development version. You need `git`, CMake 3.22 or newer, a Fortran compiler (gfortran, Intel `ifx` or `ifort`) and `make` or `ninja`.

```
git clone https://github.com/swat-model/swatplus.git
cd swatplus
git checkout 62.0.1
cmake -B build -D CMAKE_BUILD_TYPE=Release
cmake --build build -j 8
```

The executable is written to `build/` with a name of the form `swatplus-<version>-<compiler>-<system>_<arch>-Rel` (`-Dbg` for a `Debug` build, no suffix when no build type is given). Builds from a commit between tags have a version such as `62.0.1-3-g1a2b3c4`. The Developer Guide has instructions for Visual Studio, VS Code and Codespaces.

Next

Run the reference dataset to check the executable from start to finish: Quick start: Ames_sub1.

Sources

- Release files: SWAT+ 62.0.1 on GitHub, checked 20 October 2026; the banner above is from the `gnu-lin_x86_64` executable.
- Installers: install.swat.tamu.edu, checked 20 October 2026.
- Executable naming: `set(SWATPLUS_EXE "swatplus-${SWAT_VERSION}-${FFC}-${AR}${TY}")` in `CMakeLists.txt`, with `SWAT_VERSION` from `git describe --tags`.
- Build instructions: `doc/Building.md`.

Run a complete SWAT+ simulation with the small reference dataset that comes with the source code.

The dataset

`Ames_sub1` is a test dataset of 12 HRUs near Ames, Iowa, with no channels, aquifers or reservoirs. It simulates 1 January 1975 to 31 December 2020 on a daily time step. It is used to test the model and has not been calibrated. The folder is in the SWAT+ source repository at `refdata/Ames_sub1`.

Before running

You need:

- the SWAT+ executable (see Install);
- a copy of the `Ames_sub1` folder, either from a clone of the source repository or downloaded from the link above;
- about 10 GB of free disk space. With the shipped `print.prt` the run writes daily carbon output for every HRU, about 9 GB in 62.0.1. To keep the output small, change the `daily` column of the `hru_cb` and `hru_cb_vars` rows in `print.prt` from `l` to `n` before running.

The shipped folder already contains output files from earlier model versions (for example `basin_wb_aa.txt`, written in 2024). A run with the shipped `print.prt` does not rewrite them, so check the date in the first line of an output file, or delete the old outputs (`*_aa.txt`, `*_yr.txt`, `*_day.txt`, `*.out`) from your copy before running.

Running

From a terminal, change into your copy of the folder and run the executable from there:

```
# Linux and macOS
cd path/to/Ames_sub1
path/to/swatplus-62.0.1-gnu-lin_x86_64-Rel
```

```
:: Windows
cd path\to\Ames_sub1
path\to\swatplus-62.0.1-gnu-win_amd64-Rel.exe
```

SWAT+ reads its inputs from the current folder, so always start it from inside the project folder. The run takes a few minutes; on a recent Linux workstation it took 4 minutes 20 seconds, most of it writing the daily carbon output.

Output

While it runs, SWAT+ prints the simulation date to the screen:

```
  Original Simulation            1   14   1994 Yr    20 of    46 Time   12:37:36
```

At the end, the screen and `simulation.out` show:

```
 Execution successfully completed
```

and the same line is written to `success.fin`.

With the shipped `print.prt`, a 62.0.1 run writes:

Files	Content
`hru_wb_day.txt`, `hru_wb_aa.txt`	HRU water balance, daily and average annual
`hru_nb_day.txt`, `hru_nb_aa.txt`, `hru_ncycle_day.txt`, `hru_ncycle_aa.txt`	HRU nutrient balance and nitrogen cycle
`hru_ls_aa.txt`	HRU sediment and nutrient losses, average annual
`hru_pw_day.txt`, `hru_pw_aa.txt`	HRU plant growth and stress
`crop_yld_yr.txt`, `crop_yld_aa.txt`, `basin_crop_yld_yr.txt`, `basin_crop_yld_aa.txt`	crop yields

Files	Content
`hru_carbvars.txt`, `hru_cbn_lyr.txt`, `hru_seq_lyr.txt`, `hru_cflux_stat.txt`, `hru_cpool_stat.txt`, `hru_n_p_pool_stat.txt`, `hru_org_*_vars.txt`, `hru_plc_stat.txt`, `basin_carbon_all.txt`, `hru_begsim_soil_prop.txt`, `hru_endsim_soil_prop.txt`	carbon and soil organic matter
`mgt_out.txt`, `lu_change_out.txt`	management operations performed, land use changes
`simulation.out`, `diagnostics.out`, `checker.out`, `erosion.out`, `files_out.out`, `co2.out`	run log, warnings, input summaries

A few other files appear as well: `paddy_test.csv` (a rice paddy test output written on every run), `wtps_wuses.wal` and an empty file named `null`.

Open `hru_wb_aa.txt` to see the average annual water balance of each HRU. For the first HRU in this run:

precip	surq_gen	latq	perc	et
952.6 mm	34.7 mm	0.1 mm	0.0 mm	932.5 mm

The columns are described in Water balance outputs.

Warnings in diagnostics.out

`diagnostics.out` lists anything the model could not match while reading the inputs. For `Ames_sub1` at 62.0.1 it contains:

```
agrr_lum      AGRR_rot      not found in management.sch
soyb_lum      SOYB_rot      not found in management.sch
oats_lum      OATS_rot      not found in management.sch
wwht_lum      WWHT_rot      not found in management.sch
ames.tem                    file not found (tgage)
```

The first four are land uses in `landuse.lum` that name management schedules missing from `management.sch`; no HRU uses those land uses. The last line means the station's temperature file, `ames.tem` in `weather-sta.cli`, is not in the list in `tmp.cli` (which names `ames.tmp`), so the model generates daily temperatures from the weather generator statistics in `weather-wgn.cli`. The Ames_sub1 walkthrough explains both. Check this file after every first run of a new project.

Troubleshooting

- `Cannot find object.cnt input file`: SWAT+ was started outside the project folder. Change into the folder first.
- `Permission denied` on Linux or macOS: run `chmod +x` on the executable.
- A command or the model stops with no message: check free disk space.
- The run stops early: read `simulation.out`, `diagnostics.out` and `checker.out` for the last message before the stop.

See FAQ and troubleshooting for other errors.

Next

- Project folder anatomy explains what each file in the folder does.
- The Ames_sub1 walkthrough goes through the dataset file by file.
- Your first watershed builds a project for a new area.

Sources

- Dataset: refdata/Ames_sub1 at 62.0.1 (`object.cnt`: 12 HRUs and no other objects; `time.sim`: 1975 to 2020, daily; `print.prt`: HRU rows only, with `hru_cb` and `hru_cb_vars` daily set to `1`).
- Run: `swatplus-62.0.1-gnu-lin_x86_64-Rel` from the 62.0.1 release, 20 October 2026. Output size and file list are from that run.
- Completion message and `success.fin`: src/main.f90.in. `paddy_test.csv` is opened unconditionally in `main.f90.in` as a temporary paddy output.

A SWAT+ project is one folder of plain-text files, usually called `TxtInOut`. The model reads every input from that folder and writes its outputs back into it. Weather data files can be kept in separate folders named in file.cio; everything else is in the one folder.

The master file

file.cio is the index of the project. Each line names a group of inputs and the files to read for it, in a fixed order. `null` means that the file is not used. Read this file first when you open a project. The full description is on file.cio.

file.cio of the `Ames_sub1` reference dataset at 62.0.1 (spacing shortened):

```
file.cio: AMES
simulation      time.sim  print.prt  null  object.cnt  null
basin           codes.bsn  parameters.bsn  carbon.bsn
climate         weather-sta.cli  weather-wgn.cli  null  pcp.cli  tmp.cli  null  null  null  null
connect         hru.con  null  null  null  null  null  null  null  null  null  null  null  null
channel         null  null  null  null  null  null  null  null
reservoir       null  null  null  null  null  null  null  null
routing_unit    null  null  null  null
hru             hru-data.hru  null
exco            null  null  null  null  null  null
recall          null
dr              null  null  null  null  null  null
aquifer         nbull  null
herd            null  null  null
water_rights    null  null  null
link            null  null
hydrology       hydrology.hyd  topography.hyd  null
structural      tiledrain.str  septic.str  filterstrip.str  grassedww.str  bmpuser.str
hru_parm_db     plants.plt  fertilizer.frt  tillage.til  pesticide.pes  null  null  null  urban.urb  septi
ops             harv.ops  graze.ops  irr.ops  chem_app.ops  fire.ops  sweep.ops
lum             landuse.lum  management.sch  cntable.lum  cons_practice.lum  ovn_table.lum
chg             null  null  null  null  null  null  null  null  null
init            plant.ini  soil_plant.ini  null  null  null  null  null  null  null  null  null  null
soils           soils.sol  nutrients.sol  null
decision_table  lum.dtl  null  null  null
regions         ls_unit.ele  null  null  null  null  null  null  null  aqu_catunit.ele  null  null  null
pcp_path        null
tmp_path        null
slr_path        null
hmd_path        null
wnd_path        null
```

The `basin` line names three files from revision 62 on (carbon.bsn was added). A file.cio written for an earlier revision has two names there and is misread by 62.0.1; rewrite it with SWAT+ Editor 4 or add the third name. The same applies to the `water_rights` line, which names the water allocation files. See file.cio for the details.

Input groups

Simulation control

File	Content
time.sim	start and end dates, time step
print.prt	which output files to write, and at which time steps
object.cnt	number of objects of each type (HRUs, channels, aquifers, …)
object.prt	extra output for individual objects (optional)

Basin

File	Content
codes.bsn	method switches (PET method, runoff method, routing, carbon model, gwflow, …)
parameters.bsn	basin-wide parameters
carbon.bsn, carbon_lyr.bsn	carbon model parameters; carbon_lyr.bsn is found from the name of carbon.bsn and is not listed in file.cio

Climate

File	Content
weather-sta.cli	weather stations and the data files each one uses
weather-wgn.cli	weather generator statistics, used to fill missing data or generate whole records
pcp.cli, tmp.cli, slr.cli, hmd.cli, wnd.cli	lists of the data files for each variable
*.pcp, *.tmp, *.slr, *.hmd, *.wnd	daily (or sub-daily) records

Objects and connections

File	Content
hru.con, rout_unit.con, aquifer.con, chandeg.con, reservoir.con, recall.con, …	position of each object and where its outflow goes
hru-data.hru	for each HRU, the names of its topography, hydrology, soil, land use, initial conditions, snow and field records
rout_unit.def, rout_unit.ele, ls_unit.def, ls_unit.ele, aqu_catunit.ele	which HRUs belong to each routing unit, landscape unit and aquifer catchment

Hydrology and topography

File	Content
hydrology.hyd	HRU hydrology parameters (lateral flow, evaporation, percolation, …)
topography.hyd	slope, slope length and related values
field.fld	field length, width and orientation

Soils

File	Content
soils.sol	soil layers and their physical properties
nutrients.sol	initial soil nutrient contents

Land use and management

File	Content
landuse.lum	land uses, each naming its plant community, management schedule, curve number set, conservation practice and structural practices
management.sch	management schedules: dated operations and decision tables
cntable.lum, cons_practice.lum, ovn_table.lum	curve numbers, USLE support practice factors, overland Manning's n
lum.dtl	decision tables for land use and management
harv.ops, graze.ops, irr.ops, chem_app.ops, fire.ops, sweep.ops	parameters of each type of operation

Databases

File	Content
plants.plt	plant growth parameters
fertilizer.frt, manure_db.frt, manure_om.frt	fertilizers and manures
tillage.til, pesticide.pes, urban.urb, septic.sep, snow.sno	tillage, pesticide, urban, septic system and snow parameters

Initial conditions

File	Content
plant.ini	plant communities at the start of the run
soil_plant.ini	which soil nutrient and constituent records each HRU starts with

Channels, aquifers, reservoirs and wetlands

File	Content
channel-lte.cha, hyd-sed-lte.cha, nutrients.cha, initial.cha	channels (the SWAT-DEG channel model, connected through chandeg.con)
aquifer.aqu, initial.aqu	aquifers
reservoir.res, hydrology.res, sediment.res, nutrients.res, initial.res, weir.res	reservoirs
wetland.wet, hydrology.wet	wetlands

Calibration

File	Content
cal_parms.cal, calibration.cal	parameter changes applied at start-up (Hard calibration)
codes.sft and the other *.sft files	Soft calibration

Optional groups (structural practices, water allocation, gwflow, salts and constituents, pesticides and pathogens, export coefficients, recall) are described in the Input Reference.

Outputs

Output files end in _day, _mon, _yr or _aa (average annual) and .txt, or .csv when CSV output is switched on in print.prt. The first part of the name gives the object (basin_, lsunit_, hru_, channel_sd_, aquifer_, reservoir_, …) and the next part the variables:

Code	Variables
`wb`	water balance
`nb`	nutrient balance
`ls`	sediment and nutrient losses
`pw`	plant growth and stress
`aqu`, `sd_cha`, `res`	aquifer, channel, reservoir

The Output Reference describes each file.

Log and check files

File	Content
`simulation.out`	run progress and the completion message
`diagnostics.out`	names that could not be matched, missing files, unexpected values
`checker.out`	summary of the inputs for each HRU, written before the simulation
`files_out.out`	list of the output files written
`area_calc.out`	area summary
`erosion.out`	header lines only in 62.0.1
`success.fin`	written only when the run completes

Check `simulation.out` and `diagnostics.out` after every first run of a new or changed project.

Sources

- File groups and default names: the `input_*` types in `src/input_file_module.f90`; `file.cio` is read by `src/readcio_read.f90`.
- Example: `refdata/Ames_sub1/file.cio`. The `aquifer` line contains `nbull`, a typing error in the shipped file; the dataset has no aquifers, so the name is never used.

The steps below build a SWAT+ project for an area of your choice, run it and look at the output, with what each step produces. They do not go through QSWAT+ screen by screen.

Requirements

Data:

- A digital elevation model (DEM) covering your area, in a projected coordinate system, with no holes.
- A soil map, either SSURGO (US), HWSD (global), or a custom raster paired with a soil property lookup table.
- A land use map, either NLCD (US), ESA WorldCover (global), or a custom raster paired with a SWAT+ land use lookup table.
- A daily climate record (precipitation and temperature at minimum, optionally solar radiation, wind, humidity) from one or more stations covering the simulation period.
- An outlet point (the gauge or stream location you want to drain to).

Software:

- QGIS with the QSWAT+ plugin, SWAT+ Editor and, for calibration, SWAT+ Toolbox. See Install.

Step 1: Delineate the watershed (QSWAT+)

In QGIS, create a new QSWAT+ project and load the DEM. QSWAT+ will:

1. Fill sinks in the DEM.
2. Compute flow direction and accumulation.

3. Build the stream network at a threshold drainage area you choose.
4. Subdivide the watershed into subbasins at stream junctions.
5. Snap your outlet point to the stream network.

Output: a polygon layer of subbasins and a polyline layer of channels (reaches).

Step 2: Create HRUs

Overlay the land use map, soil map, and a slope class layer on the subbasins. Each unique combination of (subbasin, land use, soil, slope class) is a candidate HRU. QSWAT+ writes the HRUs and the connectivity files.

You can keep all candidate HRUs (high spatial detail, slow run) or filter by area threshold (faster, less detail).

Output: HRU definitions and `*.con` connectivity files.

Step 3: Build the input files (SWAT+ Editor)

Open the project in SWAT+ Editor. The editor will:

1. Import climate station data and write the `.cli`, `.pcp`, `.tmp`, and `weather-wgn.cli` files. Stations without data get filled by the weather generator.
2. Populate soil layer parameters from the soil property table.
3. Populate plant, fertiliser, tillage, and pesticide databases.
4. Let you assign management schedules to each land use.
5. Write the final input folder.

Set the simulation period and choose the outputs. Start the simulation one to three years before the period you want to analyse, and set the number of warm-up years in `nyskip` of `print.prt` so they are left out of the outputs.

Output: a folder of plain-text SWAT+ inputs, ready to run.

Step 4: Run

Run the model from SWAT+ Editor, or from a terminal inside the project's `TxtInOut` folder:

```
cd path/to/your/project/Scenarios/Default/TxtInOut
path/to/swatplus-62.0.1-gnu-lin_x86_64-Rel
```

Watch for the `Execution successfully completed` message. Then read `diagnostics.out`: it lists every name that could not be matched (a management schedule, a weather station, a plant) and every missing file. Fix those before looking at results.

Step 5: Look at output

Start with the basin water balance:

`basin_wb_aa.txt`

The annual-average precipitation, evapotranspiration, surface runoff, lateral flow, percolation, and water yield should be in the range you expect for your climate. If precipitation is far from the expected value, the weather stations are probably assigned wrongly. If ET takes up nearly all of the precipitation, check land use and plant parameters.

Then look at the channel output (`basin_sd_cha_aa.txt` or the per-channel `channel_sd_aa.txt`) for streamflow.

If you have measured streamflow at the outlet, compare to `channel_sd_day.txt` for the gauge channel.

Step 6: Calibrate

The first run is rarely a good match to observed data. Move on to Calibration to adjust parameters and improve the fit.

Common first-run problems

- No precipitation anywhere: the climate station coordinates are outside the watershed extent, so QSWAT+ assigned every HRU to the weather generator instead of the station. Check `weather-sta.cli`.
- Crops never grow: the management schedule is not assigned, or the planting date is outside the simulation period.
- All flow is surface runoff: curve numbers are too high. Check `cntable.lum` and the soil hydrologic group.
- Streamflow is zero downstream: a channel object has no upstream connection in `chandeg.con` or `hru.con`. Inspect the connectivity.

Sources

- File names and groups: `src/input_file_module.f90`. Completion message: `src/main.f90.in`.

Overview

These pages describe how SWAT+ represents the processes that move water, sediment, nutrients and other constituents through a watershed. Each page covers one group of processes: the equations the model solves, the input files and columns that control them, and where SWAT+ 62.0.1 differs from the SWAT 2009 theoretical documentation.

The landscape processes run once per day for every hydrologic response unit (HRU): weather, snow, evapotranspiration, surface runoff, soil water, erosion, plant growth and the soil nutrient cycles. Their results leave the HRU as surface runoff, lateral flow, tile flow and percolation, and pass to the routing objects: channels, aquifers, reservoirs and wetlands, and recall points. Decision tables apply management and reservoir release rules across both groups.

Pages

Group	Page	Contents
Landscape water	Overview and water balance	HRU water balance, the order of daily calculations, where each term is reported
Landscape water	Climate and weather	Measured and generated weather, day length, sub-daily precipitation, elevation lapse, potential evapotranspiration
Landscape water	Hydrology	Canopy interception, surface runoff, lateral flow, percolation, tile flow, lag to the channel
Landscape water	Soil water and temperature	Soil layers, water retention, initial soil water, soil evaporation, soil temperature
Landscape water	Erosion and sediment	Peak runoff rate, MUSLE, sub-daily erosion, sediment in lateral flow, urban loads, filter strips, grassed waterways and user BMPs
Routing	Channels	Channel routing, sediment transport and in-stream water quality
Routing	Aquifers	Aquifer objects and the gwflow groundwater module
Routing	Reservoirs and wetlands	Reservoir water balance and release rules, wetlands
Routing	Recall and point sources	Recall time series and point source loads
Routing	Decision tables	Condition evaluation and actions
Plants and chemistry	Plant growth and land use management	Heat units, biomass, leaf area, stresses, harvest, scheduled management
Plants and chemistry	Nutrient cycling	Soil nitrogen and phosphorus pools, transformations, losses; algae, CBOD and oxygen in runoff; septic systems
Plants and chemistry	Carbon	Static and CENTURY-based soil carbon
Plants and chemistry	Pesticides	Application, partitioning, decay and transport

Group	Page	Contents
Plants and chemistry	Salts and constituents	Salt ion chemistry, other constituents, pathogens and heavy metals

SWAT 2009 processes not in SWAT+ 62.0.1

The SWAT 2009 theoretical documentation describes some processes that SWAT+ 62.0.1 does not simulate, or simulates in a different way:

- Elevation bands within a subbasin. SWAT+ corrects precipitation and temperature for the elevation of each object instead (see Climate and weather).
- Monthly climate change adjustments of precipitation, temperature, solar radiation and humidity. SWAT+ 62.0.1 has no input for them; adjust the weather files instead.
- Hourly air temperature and the distribution of solar radiation over the hours of the day.
- The adjustment of the curve number for slope.
- Transmission losses from tributary channels within a subbasin. SWAT+ has no tributary channels; losses are computed for each channel object.
- Bank storage in the main channel.
- The SWAT 2009 channel sediment routing (simplified Bagnold stream power equation, channel erodibility and cover factors, downcutting and widening). SWAT+ uses the channel sediment model described in Channels.
- Potholes and other depressional areas, replaced by wetlands and paddies in HRUs (see Reservoirs and wetlands).
- Reservoir release from measured monthly outflow or a target storage. SWAT+ uses release decision tables and measured releases from recall files.
- Chlorophyll-a and Secchi depth relations for lakes and reservoirs.
- The stream and dam reaeration formulas of QUAL2E; SWAT+ uses the reaeration rate given in `nutrients.cha`.
- The two-population bacteria model (persistent and less persistent bacteria), replaced by the pathogen routines described in Salts and constituents.
- Continuous fertilization and automatic fertilization by nitrogen stress. Decision tables can fertilize in response to conditions instead (see Decision tables).
- Consumptive water use and water transfer between objects, replaced by water allocation (see Water allocation).

Page layout

Every symbol is defined under its equation. Where a symbol is a user input, the input file and the column header the SWAT+ Editor writes are named with it, and the basin switches and parameters that change the result are listed together on each page.

The last section of every page, Sources, is for developers. It links to the Fortran routines at the 62.0.1 tag, maps symbols and columns to Fortran variable names, and records implementation details.

Sources

- Daily HRU sequence: `hru_control.f90`
- Object loop that calls HRUs, channels, aquifers and reservoirs: `command.f90`

The pages in this section were checked against SWAT+ 62.0.1 (commit `6f51536`). Where a page and the Fortran source disagree, the source describes what the model does.

Overview

The water balance drives everything else SWAT+ simulates: to predict the movement of sediment, nutrients, pesticides or salts, the model first has to reproduce the movement of water. SWAT+ splits the hydrology of a watershed into two parts. The land phase, covered here and in the pages that follow, controls how much water, sediment and chemical load each hydrologic response unit (HRU) delivers. The routing phase moves those loads through channels, aquifers, reservoirs and wetlands to the outlet.

Each HRU is computed once per day (or per sub-daily time step for runoff when Green-Ampt infiltration is used). The HRU keeps track of four water stores: the plant canopy, the snowpack, the soil profile and, where one is defined, a ponded surface such as a wetland or rice paddy. Water that leaves the soil profile at the bottom becomes recharge to an aquifer object or to the gwflow module.

Soil water balance

The land-phase balance for the soil profile of an HRU over a period of t days is

$$SW_t = SW_0 + \sum_{i=1}^{t} \left(R_{day} + I - Q_{surf} - E_a - w_{perc} - Q_{lat} - Q_{tile} \right)$$

Symbol	Meaning	Units
SW_t	Soil water content at the end of day t	mm H2O
SW_0	Soil water content at the start of the period	mm H2O
R_{day}	Rain and snowmelt reaching the soil surface, after canopy interception and snowfall	mm H2O
I	Irrigation and run-on from upslope objects	mm H2O
Q_{surf}	Surface runoff generated on the day	mm H2O
E_a	Evaporation from the soil and transpiration by plants	mm H2O
w_{perc}	Percolation from the bottom of the soil profile	mm H2O
Q_{lat}	Lateral subsurface flow leaving the soil layers	mm H2O
Q_{tile}	Tile drain flow	mm H2O

All terms are depths over the HRU area. Water in the canopy and in the snowpack is stored outside the soil profile: precipitation first fills canopy storage, then falls as snow on cold days, and only the remainder (plus snowmelt) is available for runoff and infiltration. Total evapotranspiration reported for the HRU is the sum of canopy evaporation, plant transpiration and soil evaporation, with snow sublimation and evaporation from ponded water counted in soil evaporation.

Surface runoff, lateral flow and tile flow do not all reach the channel on the day they are generated. Each passes through a lag store (see Hydrology), so the HRU water yield on a day is

$$Q_{yld} = Q_{surf,lag} + Q_{lat,lag} + Q_{tile,lag}$$

where the subscript *lag* marks the part of each store released that day.

Daily order of calculations

The order matters because each process sees the state left by the processes before it. For each HRU on each day SWAT+ 62.0.1 runs:

1. Elevation lapse corrections to precipitation and temperature, when `lapse` is on in `codes.bsn`.
2. Decision table operations and water allocation demands for the HRU.
3. Base-zero heat unit accumulation, albedo, salt and constituent soil chemistry.
4. Soil temperature for every layer.
5. Canopy interception.
6. Snowfall, snowpack temperature and snowmelt.
7. Run-on from upslope objects: surface runoff, lateral flow, tile flow and aquifer flow routed onto the HRU.
8. Soil crack volume, when `crack` is on in `codes.bsn`.
9. Potential evapotranspiration, canopy evaporation and soil evaporation.
10. Scheduled management operations from the HRU's management schedule.
11. Surface runoff, peak runoff rate and erosion. HRUs with a ponded surface (wetland or paddy) produce no surface runoff or erosion here; their water is handled in the next step.

12. Wetland and paddy water balance.
13. Infiltration and soil water routing: percolation, lateral flow, tile flow and saturation excess.
14. Grazing, residue decomposition, mineralisation, nitrification and volatilisation, and mineral phosphorus transformations.
15. Plant growth, including plant water and nutrient uptake (transpiration).
16. Pesticide wash-off, uptake, decay and leaching; nutrient and pesticide losses with runoff and sediment; atmospheric deposition; nitrate and soluble phosphorus movement.
17. Urban build-up and wash-off, sediment in lateral flow, and the surface lag store for sediment and chemicals.
18. Lag of lateral and tile flow, filter strips, grassed waterways and user-defined BMPs.
19. Lag of surface runoff.
20. Water yield and routing of the HRU's outflow to the objects it is connected to.

Switches and parameters

Input	File	Effect
`pet`	`codes.bsn`	Potential evapotranspiration method. See Climate and weather.
`gampt`	`codes.bsn`	Curve number (`0`) or Green-Ampt (`1`) infiltration. See Hydrology.
`lapse`	`codes.bsn`	Elevation lapse of precipitation and temperature.
`crack`	`codes.bsn`	Bypass flow through soil cracks.
`surq_lag`	`parameters.bsn`	Surface runoff lag coefficient.
`sw_init`	`parameters.bsn`	Initial soil water as a fraction of field capacity. See Soil water and temperature.
`can_max`, `esco`, `epco`, `lat_ttime`, `perco`, `cn3_swf`, `latq_co`, `pet_co`	`hydrology.hyd`	HRU-level hydrology parameters, described on the pages for each process.

Output

The terms of the HRU balance are written to the water balance files at HRU, landscape unit and basin level (`hru_wb_*`, `lsunit_wb_*`, `basin_wb_*`). `precip`, `snofall` and `snomlt` give the inputs; `surq_gen`, `latq`, `qtile`, `perc`, `et`, `eplant`, `esoil` and `ecanopy` the losses; `sw_init`, `sw_final` and `snopack` the stores; and `lagsurf` and `laglatq` the water still in the lag stores. See Water balance output.

Related pages

- Climate and weather for precipitation, temperature and potential evapotranspiration.
- Hydrology for interception, runoff, lateral flow, percolation and tile flow.
- Soil water and temperature for the soil profile and soil evaporation.
- Aquifers for the fate of percolation below the soil profile.
- Reservoirs and wetlands for ponded HRUs.

Sources

- Daily HRU sequence: `hru_control.f90`
- Evapotranspiration totals: `et_act.f90`
- Soil water routing: `swr_percmain.f90`
- Lateral and tile lag: `swr_substor.f90`; surface runoff lag: `sq_surfst.f90`
- Water balance output type: `type output_waterbal` in `output_landscape_module.f90`

Symbol or output	Fortran variable
SW	`soil(j)%sw`
R_{day} (before infiltration)	`precip_eff`
Infiltration into layer 1	`inflpcp`
Q_{surf} (generated, lagged)	`surfq(j)`, `qday`

Symbol or output	Fortran variable
Q_{lat}	`latq(j)`
Q_{tile}	`qtile`
w_{perc}	`sepbtm(j)`
Canopy evaporation, transpiration, soil evaporation	`canev`, `ep_day`, `es_day`
Total ET (`et` output)	`etday = ep_day + es_day + canev`
Water yield	`qdr(j) = qday + latq(j) + qtile`
Lag stores (`lagsurf`, `laglatq`, `lagsatex`)	`surf_bs(1,j)`, `bss(1,j)`, `bss_ex(1,j)`

Overview

Climate supplies the moisture and energy that drive the water balance and sets the relative size of its parts. SWAT+ needs daily precipitation, maximum and minimum air temperature, solar radiation, relative humidity and wind speed. Each weather station in `weather-sta.cli` takes each variable either from a measured data file or from the weather generator, and every HRU, channel and other spatial object uses the station assigned to it in its connection file. Potential evapotranspiration (PET) is computed in the model by one of three methods. A fourth option is meant to read PET from a file, but in 62.0.1 it returns the Hargreaves estimate.

Weather inputs

Measured and generated data

Each row of `weather-sta.cli` names a weather generator station from `weather-wgn.cli` and one data set for each variable: precipitation, temperature, solar radiation, relative humidity and wind speed, followed by a PET data set and an atmospheric deposition station. The value `sim` tells SWAT+ to generate that variable from the weather generator statistics.

Measured values come from the files listed in `pcp.cli`, `tmp.cli`, `slr.cli`, `hmd.cli`, `wnd.cli` and `pet.cli` (see `*.cli`, `*.pcp`, `*.tmp` and `*.tem`). The names in `weather-sta.cli` are matched to these lists with a search that assumes the lists are sorted by name; the `*.cli` page describes what happens when they are not. A value of -97 or less (for example -99) marks a missing record, and SWAT+ fills that day with a generated value. Days outside the period covered by a measured file are also generated.

Weather generator

The generator produces daily weather from the monthly statistics of a station in `weather-wgn.cli`. Precipitation is generated first, because temperature, radiation and humidity are adjusted for wet and dry days.

Whether a day is wet follows a first-order Markov chain. The probability of rain is `pr_wd` (a wet day following a dry day) or `pr_ww` (a wet day following a wet day) for the month. On a wet day the amount comes from a skewed distribution:

$$R_{day} = \mu_{mon} + 2\,\sigma_{mon}\left(\frac{\left[\left(SND_{day} - \frac{g_{mon}}{6}\right)\frac{g_{mon}}{6} + 1\right]^3 - 1}{g_{mon}}\right)$$

Symbol	Meaning	Input
R_{day}	Generated precipitation, at least 0.1 mm	mm H2O
μ_{mon}	Mean daily precipitation on wet days in the month	from `pcpmm` and `pcpd` in `weather-wgn.cli`
σ_{mon}	Standard deviation of daily precipitation	`pcpstd`, mm
g_{mon}	Skew coefficient of daily precipitation	`pcpskw`
SND_{day}	Standard normal deviate for the day	generated

The result is scaled by a monthly correction factor so that the long-term mean of generated wet-day amounts equals μ_{mon}.

Maximum and minimum temperature are drawn around the monthly means `tmpmx` and `tmpmn` with standard deviations `tmpstdmx` and `tmpstdmn`. The maximum is lowered on wet days and raised on dry days by half the monthly temperature range, weighted by the fraction of wet days. Relative humidity is drawn from a triangular distribution around the monthly mean, which comes from the dew point (`dewpt`) and mean temperature, and is raised on wet days. Wind speed follows a modified exponential distribution around the monthly mean `windav`.

The generator in 62.0.1 returns the monthly mean solar radiation `solarav` for every generated day. The wet and dry day adjustment is computed but then replaced by the monthly mean, so generated radiation does not vary from day to day within a month.

`igen` in `parameters.bsn` sets the random number seeds. With `igen = 0` every run uses the same seeds and the same generated weather. Any other value offsets the seeds, so different values give different weather sequences.

Maximum half-hour rainfall

The fraction of daily rain that falls in the wettest half hour, $\alpha_{0.5}$, is used by the peak runoff rate and erosivity calculations (see Erosion and sediment). It is generated every day from a triangular distribution between 0.02083 and $1-\exp(-125/(R_{day}+5))$, with its mode set from the monthly maximum half-hour rainfall `rainhmx` in `weather-wgn.cli`. The mode is multiplied by `adj_pkrt` in `parameters.bsn` and limited to between 0.1 and 0.95.

Day length and maximum radiation

Each day SWAT+ computes the day length and the maximum possible solar radiation for every weather generator station from its latitude. The solar declination δ and the relative distance between the earth and the sun d_r on day of year d are

$$\delta = \sin^{-1}\left[0.4\,\sin\left(\frac{d-82}{58.09}\right)\right], \qquad d_r = 1 + 0.033\,\cos\left(\frac{d}{58.09}\right)$$

and the hour angle of sunset h, the day length T_{DL} and the maximum solar radiation H_{max} are

$$h = \cos^{-1}(-\tan\delta\,\tan\phi), \qquad T_{DL} = 7.6394\,h, \qquad H_{max} = 30\,d_r\,(h\,\sin\phi\,\sin\delta + \cos\phi\,\cos\delta\,\sin h)$$

Symbol	Meaning	Units
ϕ	Latitude of the weather generator station, `lat` in `weather-wgn.cli`	radians
h	Hour angle of sunset; 0 during polar night and π during polar day	radians
T_{DL}	Day length	h
H_{max}	Maximum possible solar radiation reaching the ground on a cloudless day	MJ/m²/day

H_{max} is used by net radiation and Hargreaves PET. The day length controls plant dormancy (see Plant growth and land use management). SWAT+ also computes how the day's radiation is spread over the hours of the day, but no process in 62.0.1 uses that distribution.

Sub-daily precipitation

With more than one time step per day (`step` in `time.sim`), a day of generated precipitation is spread over the day's time steps with a double exponential intensity curve. The peak rate is

$$i_{pk} = 2\,R_{day}\,\alpha_{0.5}$$

where R_{day} is the day's precipitation (mm) and $\alpha_{0.5}$ the fraction falling in the wettest half hour, so i_{pk} is in mm/h. The time to the peak, as a fraction v of the storm duration, is drawn from a triangular distribution between 0.05 and 0.95 with a mode of 0.25. The storm duration is

$$T_{dur} = \frac{R_{day}}{i_{pk}\,(k_1 + k_2)}, \qquad k_1 = \frac{v}{4.605}, \qquad k_2 = \frac{1 - v}{4.605}$$

limited to 24 hours, with i_{pk} recomputed when the limit applies. The cumulative rainfall at time t (h) from the start of the storm is

$$R(t) = R_{pk} - i_{pk}\,T_{dur}\,k_1\left[1 - \exp\left(\frac{t - t_{pk}}{T_{dur}\,k_1}\right)\right] \qquad (t \le t_{pk})$$

$$R(t) = R_{pk} + i_{pk}\,T_{dur}\,k_2\left[1 - \exp\left(\frac{t_{pk} - t}{T_{dur}\,k_2}\right)\right] \qquad (t > t_{pk})$$

with $R_{pk} = v\,R_{day}$ and $t_{pk} = v\,T_{dur}$. The storm starts at the beginning of the day, the rainfall of each time step is the increase of $R(t)$ over the step, and any amount left at the end is added to the last step that received rain.

The same curve fills a day of a measured sub-daily record that contains a missing value. Daily precipitation records are not split into time steps: in a sub-daily run a station with daily records gives no rainfall in its time steps. Use sub-daily records, or generated precipitation, for sub-daily runs.

Elevation lapse

When `lapse` is 1 in `codes.bsn`, precipitation and temperature are corrected for the difference in elevation between each object and the gauge (or the weather generator station, for generated data) that supplies it:

$$R_{obj} = \max\left(0,\; R_{gage} + \frac{\text{plaps}\,(z_{obj} - z_{gage})}{1000}\right)$$

$$T_{obj} = T_{gage} + \frac{\text{tlaps}\,(z_{gage} - z_{obj})}{1000}$$

Symbol	Meaning	Units
plaps	Precipitation lapse rate, `plaps` in `parameters.bsn`	mm/km
tlaps	Temperature lapse rate, `tlaps` in `parameters.bsn`	°C/km
z_{obj}, z_{gage}	Elevation of the object and of the gauge or generator station	m

The temperature correction is applied to the maximum, minimum and mean temperature. A positive `tlaps` makes objects above the gauge cooler. The precipitation correction is a fixed depth per day, added on dry days as well as wet ones, and the result is never negative.

Carbon dioxide

The CO2 concentration used by plant growth and Penman-Monteith PET is set for each year of the run. If the file `co2_yr.dat` is present, yearly values are read from it; years before the record use the first value and years after it the last. Without the file every year uses `co2` from `parameters.bsn`. The values used are written to `co2.out`.

Snow

On days when the mean air temperature is at or below `fall_tmp`, precipitation that passes the canopy is added to the snowpack. Snow melts on days when the maximum temperature is above `melt_tmp`, at a rate that varies through the year between `melt_min` and `melt_max` and is reduced by the fraction of the HRU covered by snow. The equations and the parameters are on the `snow.sno` page. Snow sublimation is taken from the soil evaporation demand (see Soil water and temperature).

Potential evapotranspiration

Potential evapotranspiration is the rate of evapotranspiration from a large area covered completely by a short, actively growing crop with plenty of water. The method is chosen with `pet` in `codes.bsn`:

`pet`	Method
0	Priestley-Taylor
1	Penman-Monteith
2	Hargreaves
3	Station PET (see PET read from file)

The three computed methods share these terms:

Symbol	Meaning	Calculation
$\bar{T}$	Mean air temperature, °C	from the weather inputs
P	Atmospheric pressure, kPa	$101.3 - z\,(0.01152 - 0.544 \times 10^{-6}\,z)$ with HRU elevation z in m
λ	Latent heat of vaporization, MJ/kg	$2.501 - 2.361 \times 10^{-3}\,\bar{T}$
γ	Psychrometric constant, kPa/°C	$1.013 \times 10^{-3}\,P/(0.622\,\lambda)$
e^0, e	Saturation and actual vapour pressure, kPa	$e = e^0 \times$ relative humidity
Δ	Slope of the saturation vapour pressure curve, kPa/°C	$4098\,e^0/(\bar{T} + 237.3)^2$

Net radiation

Priestley-Taylor and Penman-Monteith use net radiation for a reference crop:

$$H_{net} = (1 - \alpha)\,H_{day} - (0.34 - 0.139\sqrt{e})\left(0.9\,\frac{H_{day}}{H_{max}} + 0.1\right)\sigma\,T_K^4$$

where H_{day} is the solar radiation reaching the ground (MJ/m²/day), H_{max} the maximum possible solar radiation for the day, $\sigma = 4.903 \times 10^{-9}$ MJ m^{-2} K^{-4} day^{-1}, T_K the mean air temperature in kelvin, and α the albedo: 0.23 for the reference crop, or 0.8 when the snowpack contains more than 0.5 mm of water.

Priestley-Taylor

$$PET = 1.28\,\frac{\Delta}{\Delta + \gamma}\,\frac{H_{net}}{\lambda}$$

Priestley and Taylor (1972) replaced the aerodynamic term of Penman's equation with the coefficient 1.28, which suits humid areas. The method tends to underestimate PET in arid and semi-arid areas, where advection is a large part of the energy balance.

Penman-Monteith

$$PET = \frac{\Delta\,H_{net} + \gamma\,\rho_K\,(e^0 - e)/r_a}{\lambda\,[\Delta + \gamma\,(1 + r_c/r_a)]}$$

For the reference crop SWAT+ uses the aerodynamic resistance $r_a = 114/u_{1.7}$ (s/m), where $u_{1.7} = u_{10}\,(0.17)^{0.2}$ is the wind speed at 1.7 m estimated from the input wind speed u_{10} (taken as measured at 10 m), and a canopy resistance that responds to CO2:

$$r_c = \frac{49}{1.4 - 0.4\,CO_2/330}$$

ρ_K combines air density, the latent heat of vaporization and unit conversions, and is computed as $1710 - 6.85\,\bar{T}$. The method needs measured or generated relative humidity and wind speed.

Penman-Monteith also computes a crop-specific maximum transpiration from the plant canopy height, leaf area and stomatal conductance (`stcon_max`, `vpd` and `frac_stcon` in `plants.plt`). In 62.0.1 that value is replaced in the actual evapotranspiration step by the leaf area based estimate used with the other methods (see Soil water and temperature), so the choice of PET method changes PET but not the way PET is split into transpiration and soil evaporation.

Hargreaves

$$PET = 0.0023\,\frac{H_0}{\lambda}\,(\bar{T} + 17.8)\sqrt{T_{max} - T_{min}}$$

where H_0 is extraterrestrial radiation (MJ/m²/day), taken as $H_{max} \times 37.59/30$. PET is zero on days when T_{max} is not above T_{min}. Hargreaves needs only temperature, which makes it a common choice where humidity, wind and radiation records are poor.

PET read from file

With `pet = 3` the HRU takes the daily PET of its weather station. SWAT+ 62.0.1 first loads the station's measured PET (filling missing days with Hargreaves), but later in the same day it recomputes the station PET with the Hargreaves equation for every station, for use in the climatic moisture index. The HRU therefore receives the Hargreaves value, and `pet = 3` gives the same PET as `pet = 2`.

The same station PET (Hargreaves in every case) is the evaporative demand for channel and reservoir evaporation and for aquifer revap, whatever the `pet` setting.

HRU adjustment

Whatever the method, PET is multiplied by `pet_co` from `hydrology.hyd`:

$$PET_{HRU} = \text{pet_co} \times PET$$

A blank or zero `pet_co` is taken as 1. A value below 0.01 is read as a Hargreaves coefficient and converted to a multiplier by dividing it by 0.0023.

Switches and parameters

Input	File	Effect
`pet`	`codes.bsn`	PET method
`lapse`	`codes.bsn`	Elevation lapse of precipitation and temperature
`plaps`, `tlaps`	`parameters.bsn`	Lapse rates
`adj_pkrt`	`parameters.bsn`	Multiplier on the generated maximum half-hour rainfall fraction
`co2`	`parameters.bsn`	CO2 concentration when `co2_yr.dat` is absent, ppm
`igen`	`parameters.bsn`	Random number seed selection for the generator
`pet_co`	`hydrology.hyd`	HRU multiplier on PET
`petco_pmpt`	`parameters.bsn`	Read and converted to a fraction, but not applied to PET in 62.0.1

Related pages

- `*.cli` files, `*.pcp`, `*.tmp` for the weather input formats.
- `snow.sno` for snowfall and snowmelt.

- Soil water and temperature for actual evapotranspiration.
- Overview and water balance for where climate enters the daily sequence.

Sources

- Daily weather for all stations, maximum half-hour rainfall: `climate_control.f90`, `cli_precip_control.f90`
- Station file reader: `cli_staread.f90`; generator statistics: `cli_wgnread.f90`, `cli_initwgn.f90`
- Generators: `cli_pgen.f90`, `cli_pgenhr.f90`, `cli_tgen.f90`, `cli_slrgen.f90`, `cli_rhgen.f90`, `cli_wndgen.f90`
- Day length, declination, maximum radiation and hourly radiation fractions: `cli_clgen.f90`, called from `climate_control.f90`. The hourly fractions `frad` are computed and not used.
- Sub-daily precipitation curve: `cli_pgenhr.f90`, called from `cli_precip_control.f90` for generated days and for sub-daily records with missing values; daily records leave `wst%weat%ts` unfilled.
- Lapse: `cli_lapse.f90`, applied in `hru_control.f90`
- CO2: `co2_read.f90`
- PET: `et_pot.f90`; `pet_co` setup in `topohyd_init.f90`
- Basin codes and parameters: `basin_module.f90`, `basin_prm_default.f90`

Symbol or column	Fortran variable
`pet` (codes.bsn)	`bsn_cc%pet`
`lapse` (codes.bsn)	`bsn_cc%lapse`
`plaps`, `tlaps`	`bsn_prm%plaps`, `bsn_prm%tlaps`; per object `ob(iob)%plaps`, `ob(iob)%tlaps`
`adj_pkrt`	`bsn_prm%adj_pkr`
`co2`, yearly CO2	`bsn_prm%co2`, `co2y(time%yrs)`
`petco_pmpt`	`bsn_prm%petco_pmpt`
`pet_co`	`hru(j)%hyd%pet_co`
$\alpha_{0.5}$	`wst(iwst)%weat%precip_half_hr`
H_{day}, H_{max}	`w%solrad`, `w%solradmx`
δ, d_r, h, T_{DL}	`sd`, `dd`, `h`, `wst(iwst)%weat%daylength`
i_{pk}, v, T_{dur}	`pkrr`, `vv`, `dur`
H_{net} (reference crop)	`rn_pet`
Δ, γ, λ	`dlt`, `gma`, `xl`
r_a, r_c	`rv`, `rc`
PET	`pet_day`

Overview

Precipitation reaching an HRU may be caught by the plant canopy or fall on the soil surface. Water on the surface infiltrates into the soil profile or runs off. Runoff moves quickly towards a channel and gives the short-term response of a stream. Infiltrated water is stored in the soil and later evaporated or transpired, or it moves slowly through the profile: laterally towards the channel, into tile drains, or down past the bottom of the profile to recharge groundwater.

Evaporation from the soil is described in Soil water and temperature. What happens to the water once it reaches a channel is on the Channels page.

Canopy interception

The plant canopy intercepts part of the rain, and the amount stored depends on how much canopy there is. The storage capacity on a day is

$$can_{day} = can_{mx} \frac{LAI}{LAI_{mx}}$$

Symbol	Meaning	Units
can_{day}	Maximum canopy storage on the day	mm H2O
can_{mx}	Maximum canopy storage when the canopy is fully developed, `can_max` in `hydrology.hyd`	mm H2O
LAI	Leaf area index of the plant community on the day	m^2/m^2
LAI_{mx}	Maximum leaf area index of the plant community	m^2/m^2

Precipitation first fills the empty part of the canopy store, and only the rest reaches the ground. In sub-daily runs the store is filled from the first time steps of the day. Water stored in the canopy is the first water evaporated (see Soil water and temperature). Interception applies with both the curve number and the Green-Ampt method. HRUs with no leaf area have no interception.

Surface runoff

Surface runoff, or overland flow, occurs when water reaches the ground faster than it can infiltrate. SWAT+ has two methods, chosen with `gampt` in `codes.bsn`: the SCS curve number method (`0`) and the Green and Ampt infiltration method (`1`). Runoff is computed only on days when more than 0.1 mm of water reaches the surface. Runoff from irrigation is added to it afterwards.

SCS curve number

The SCS runoff equation (SCS, 1972) is an empirical model that came into common use in the 1950s, built from more than 20 years of rainfall and runoff studies on small rural watersheds across the U.S.:

$$Q_{surf} = \frac{(R_{day} - I_a)^2}{R_{day} - I_a + S} \quad \text{for } R_{day} > I_a, \qquad Q_{surf} = 0 \text{ otherwise}$$

with the initial abstraction $I_a = 0.2\,S$, so that

$$Q_{surf} = \frac{(R_{day} - 0.2\,S)^2}{R_{day} + 0.8\,S}$$

Symbol	Meaning	Units
Q_{surf}	Accumulated runoff or rainfall excess	mm H2O
R_{day}	Rain and snowmelt reaching the surface on the day, after interception	mm H2O
I_a	Initial abstraction: surface storage, interception and infiltration before runoff	mm H2O
S	Retention parameter for the day	mm H2O

The retention parameter is linked to the curve number for the day, CN:

$$S = 25.4\left(\frac{1000}{CN} - 10\right)$$

Curve number for average conditions. Each HRU takes its moisture condition II curve number, CN_2, from the row of `cntable.lum` named in the `cn2` column of its `landuse.lum` row, using the column for the soil's hydrologic group (`cn_a` to `cn_d`, from `hyd_grp` in `soils.sol`). SWAT+ 62.0.1 does not adjust CN_2 for slope. The curve number for dry conditions (wilting point) and for wet conditions are

$$CN_1 = CN_2 - \frac{20\,(100 - CN_2)}{100 - CN_2 + \exp\left[2.533 - 0.0636\,(100 - CN_2)\right]}, \qquad CN_1 \geq 0.4\,CN_2$$

$$CN_3 = CN_2 \exp\left[0.006729\,(100 - CN_2)\right]$$

Daily curve number from soil water. The retention parameter varies with the water content of the soil profile:

$$S = S_{max}\left(1 - \frac{SW}{SW + \exp(w_1 - w_2\,SW)}\right), \qquad S \geq 3 \text{ mm}$$

Symbol	Meaning	Units
S_{max}	Retention parameter for CN_1	mm H2O
SW	Soil water content of the whole profile, excluding water below wilting point	mm H2O
w_1, w_2	Shape coefficients	-

The shape coefficients are fixed at the start of the run so that S equals S_{max} for a dry profile, equals the CN_3 retention parameter when the profile water content is SW_3, and equals 2.54 mm (a curve number of 99) when the profile is saturated:

$$SW_3 = FC + \text{cn3_swf}\,(SAT - FC)$$

where FC and SAT are the profile water contents at field capacity and saturation, and `cn3_swf` is the column in `hydrology.hyd`. With `cn3_swf = 0` the wet condition is reached at field capacity; values towards 1 move it towards saturation and reduce runoff from wet soils. For HRUs with a tile drain SWAT+ sets `cn3_swf` to 0.95.

SWAT 2009 offered a second method that ties the curve number to accumulated plant evapotranspiration. SWAT+ 62.0.1 has only the soil water method.

Frozen soil. When the second soil layer is at or below 0 °C, infiltration is reduced:

$$S_{frz} = S_{max}\left[1 - \exp(-\text{cn_froz}\ S)\right]$$

where `cn_froz` is the frozen soil parameter in `parameters.bsn`.

Urban areas. For an HRU with an urban land use, runoff from the directly connected impervious fraction (`frac_dc_imp` in `urban.urb`) is computed with a curve number of 98 and mixed with the runoff from the rest of the HRU by area. The `urb_cn` column of `urban.urb` is read but not used.

Green and Ampt infiltration

The Green and Ampt method (Green and Ampt, 1911) needs precipitation at a sub-daily time step, set in `time.sim` and supplied in the precipitation files. It assumes a sharp wetting front that separates saturated soil above from soil at its initial water content below. SWAT+ uses the Mein and Larson (1973) form with the modification of Kim et al., where the infiltration rate at the end of a time step is

$$f_{inf} = K_e\left(1 + \frac{\Psi_{wf}\,\Delta\theta_v}{F_{inf}}\right)$$

Symbol	Meaning	Units
f_{inf}	Infiltration rate	mm/hr
K_e	Effective hydraulic conductivity	mm/hr
Ψ_{wf}	Wetting front matric potential	mm
$\Delta\theta_v$	Change in volumetric water content across the wetting front	mm/mm
F_{inf}	Cumulative infiltration since the start of the day	mm H2O

Within each time step, while the rainfall intensity is below the infiltration rate all rain infiltrates. Otherwise the cumulative infiltration at the end of the step is solved by iteration from

$$F_{inf,t} = F_{inf,t-1} + K_e\,\Delta t + \Psi_{wf}\,\Delta\theta_v\,\ln\left(\frac{F_{inf,t} + \Psi_{wf}\,\Delta\theta_v}{F_{inf,t-1} + \Psi_{wf}\,\Delta\theta_v}\right)$$

and the rain that does not infiltrate becomes runoff for that step.

The effective conductivity is estimated from the saturated conductivity of the top soil layer, K_{sat}, and the curve number (Nearing et al., 1996):

$$K_e = \frac{56.82\,K_{sat}^{0.286}}{1 + 0.051\,\exp(0.062\,CN)} - 2, \qquad K_e \geq 0.001$$

The curve number used here is updated every time step from the soil water plus the water infiltrated so far, with the same relationship as the curve number method. The moisture deficit is

$$\Delta\theta_v = 0.95\,\phi_{soil}\left(1 - \frac{SW}{FC}\right)$$

where ϕ_{soil} is the porosity of the top layer and SW/FC is limited to 0.999. If more than 12 mm of rain fell on the previous day, the soil is assumed to be nearly saturated at the start of the day and the previous day's final infiltration rate is used as the starting rate. Ψ_{wf} is estimated once for each HRU from the porosity and the sand and clay content of the top layer (Rawls and Brakensiek, 1985).

On urban HRUs the directly connected impervious fraction produces runoff from the rain left after an initial abstraction store. The store is depleted by rain and recovers on dry time steps up to `urb_init_abst` in `parameters.bsn`.

Crack flow

When `crack` is 1 in `codes.bsn`, SWAT+ computes a crack volume for each layer from the soil's `perc_crk` value in `soils.sol` and its current water deficit. Cracks develop slowly as the soil dries and close as it wets. Runoff up to the total crack volume is moved into the cracks, where it fills layers below field capacity from the bottom of the profile upward, and any remainder percolates from the bottom of the profile.

Surface runoff lag

In large HRUs, or HRUs with a long time of concentration, only part of the runoff generated on a day reaches the channel that day. SWAT+ keeps a surface runoff store and releases a fixed fraction of it each day:

$$Q_{surf,lag} = \left(Q'_{surf} + Q_{stor,i-1}\right)\left[1 - \exp\left(\frac{-\text{surq_lag}}{t_{conc}}\right)\right]$$

Symbol	Meaning	Units
$Q_{surf,lag}$	Surface runoff released to the channel on the day	mm H2O
Q'_{surf}	Surface runoff generated on the day	mm H2O
$Q_{stor,i-1}$	Runoff stored or lagged from the previous day	mm H2O
surq_lag	Surface runoff lag coefficient, `surq_lag` in `parameters.bsn`	-
t_{conc}	Time of concentration of the HRU	hr

For a given time of concentration, a smaller `surq_lag` keeps more water in storage. In sub-daily runs the time of concentration is expressed in time steps, and the same fraction is released every step. Sediment, nutrients and other constituents in surface runoff are lagged in the same way.

The time of concentration is the sum of the overland flow time and a channel flow time within the HRU:

$$t_{ov} = \frac{0.0556\,(L_{slp}\,n)^{0.6}}{slp^{0.3}}, \qquad t_{ch} = \frac{0.31\,L\,n^{0.75}}{A^{0.125}\,(0.5\,slp + 0.001)^{0.375}}$$

Symbol	Meaning	Units
L_{slp}	Slope length, `slp_len` in `topography.hyd`	m
slp	Average slope, `slp` in `topography.hyd`	m/m
n	Manning's n for overland flow, from the `ov_mann` column of `landuse.lum`	-
L	Flow length, taken as $0.5\sqrt{\text{HRU area in ha}/2}$	km
A	HRU area	km²

Soil water routing

Water that infiltrates enters the top soil layer and is routed down the profile layer by layer. A layer can lose water by percolation to the layer below and by lateral flow, but only the water stored above field capacity can move.

Percolation

Percolation from a layer is computed with a storage routing method:

$$w_{perc} = SW_{ex}\left[1 - \exp\left(\frac{-24}{TT_{perc}}\right)\right], \qquad TT_{perc} = \frac{SAT - FC}{K_{sat}}$$

Symbol	Meaning	Units
w_{perc}	Water percolating to the next layer on the day	mm H2O
SW_{ex}	Water in the layer above field capacity	mm H2O
TT_{perc}	Travel time for percolation, at least 2 hours	hr
SAT, FC	Water content of the layer at saturation and at field capacity	mm H2O
K_{sat}	Saturated hydraulic conductivity of the layer, `soil_k` in `soils.sol`	mm/hr

No water percolates or flows laterally from a layer whose temperature is at or below 0 °C. Percolation from the bottom layer leaves the soil profile as recharge, and is multiplied by a factor derived from `perco` in `hydrology.hyd`:

$$f_{perco} = \min\left(1,\ \exp\left[-\left(1.0052\,\ln(-\ln \text{perco}) + 5.6862\right)\right]\right)$$

`perco` close to 1 gives a factor of 1, so the bottom layer drains freely. Smaller values keep water in the profile, and `perco = 0` stops deep percolation. For HRUs with a tile drain, SWAT+ sets `perco` to 0.1. Above the bottom layer, percolation is not restricted.

Lateral flow

Lateral subsurface flow is computed with a kinematic storage model (Sloan et al., 1983), applied to every layer below the top one:

$$Q_{lat} = \text{latq_co} \times 0.024\,\frac{2\,SW_{ex}\,K_{sat}\,slp}{\phi_d\,L_{hill}}$$

Symbol	Meaning	Units
Q_{lat}	Lateral flow from the layer	mm H2O/day
latq_co	Lateral flow coefficient, `latq_co` in `hydrology.hyd`	-
ϕ_d	Drainable porosity of the layer, $(SAT - FC)$ divided by layer thickness	mm/mm
slp	Average slope of the HRU	m/m
L_{hill}	Slope length for lateral flow, `lat_len` in `topography.hyd`; 50 m when blank or zero	m

If percolation and lateral flow together exceed the water above field capacity, both are reduced in proportion. The top soil layer, which is the 10 mm surface layer for most soils (see Soil water and temperature), produces no lateral flow in SWAT+.

Saturation excess

After percolation, any layer with more water than it can at saturation passes the excess to the layer above. Excess in the top layer is pushed down to the next layer with room, and what cannot be stored becomes surface runoff (or, on HRUs with a ponded surface, flows into the wetland or over its weir).

Tile drainage

Tile drains take water from the soil when a perched water table rises above the drain. Tile flow is computed for HRUs with a tile drain (`tile` in `landuse.lum`, pointing to a row of `tiledrain.str`) when the second soil layer is above freezing. The method is set by `tiledrain` in `codes.bsn`: `0` uses the drawdown equation below, `1` the DRAINMOD equations.

With `wtable = 0` in `codes.bsn`, the height of the water table above the bottom of the profile is estimated from the water stored above field capacity:

$$h_{wtbl} = \frac{SW - FC}{SAT_{90} - FC}\, depth_{prof}, \qquad SAT_{90} = \max(0.9\, SAT,\ 1.1\, FC)$$

where SW, FC and SAT are profile totals and $depth_{prof}$ is the profile depth. When the water table is above the drain, the drawdown method gives

$$Q_{tile} = \frac{h_{wtbl} - h_{drain}}{h_{wtbl}}\,(SW - FC)\left[1 - \exp\left(\frac{-24}{t_{drain}}\right)\right]$$

Symbol	Meaning	Units
Q_{tile}	Tile flow on the day, at most `drain` in `tiledrain.str`	mm H2O
h_{drain}	Height of the drain above the bottom of the profile, profile depth minus `dp`	mm
t_{drain}	Time to drain the soil to field capacity, `t_fc` in `tiledrain.str`; all excess drains in one day when it is under 1 hour	hr

Tile flow is taken from the layers with water above field capacity, starting at the top.

Lag of lateral and tile flow

In large HRUs with a mild slope, lateral and tile flow can take more than a day to reach the channel. Both pass through a store, and the part released on a day is

$$Q_{lat,lag} = (Q'_{lat} + Q_{latstor,i-1}) \left[1 - \exp\left(\frac{-1}{TT_{lag}}\right)\right]$$

TT_{lag} is `lat_ttime` in `hydrology.hyd`, in days. When `lat_ttime` is zero, SWAT+ estimates it from the slope length for lateral flow and the highest conductivity in the profile, $TT_{lag} = 10.4\, L_{hill}/K_{sat,mx}$ (at least one day). Tile flow uses $1 - \exp(-24/lag)$, with `lag` in hours from `tiledrain.str`. Nitrate in lateral and tile flow is lagged with the water.

Channel routing

Water yield from the HRUs enters channels, where it is routed downstream by the variable storage method or the Muskingum method, chosen with `rte_cha` in `codes.bsn`. See Channels.

Switches and parameters

Input	File	Effect
`gampt`	`codes.bsn`	`0` curve number, `1` Green-Ampt
`crack`	`codes.bsn`	`1` computes crack flow
`tiledrain`	`codes.bsn`	`0` drawdown tile equation, `1` DRAINMOD
`wtable`	`codes.bsn`	`0` water table from profile water, `1` DRAINMOD water table
`surq_lag`	`parameters.bsn`	Surface runoff lag coefficient
`cn_froz`	`parameters.bsn`	Frozen soil infiltration factor
`urb_init_abst`	`parameters.bsn`	Maximum initial abstraction on impervious urban area, Green-Ampt only
`can_max`, `cn3_swf`, `perco`, `latq_co`, `lat_ttime`	`hydrology.hyd`	Canopy storage, wet-condition curve number, deep percolation, lateral flow, lateral flow lag
`slp`, `slp_len`, `lat_len`	`topography.hyd`	Slope, slope length, slope length for lateral flow
`cn2`, `ov_mann`, `tile`	`landuse.lum`	Curve number table row, overland Manning's n, tile drain
`dp`, `t_fc`, `lag`, `drain`	`tiledrain.str`	Drain depth, drainage time, tile lag, drainage coefficient
`soil_k`, `awc`, `bd`, `perc_crk`	`soils.sol`	Layer conductivity and water retention, crack volume

Related pages

- Overview and water balance for the order of these calculations within the day.
- Soil water and temperature for soil layers and soil evaporation.
- Erosion and sediment for the peak runoff rate and soil loss.
- Aquifers for recharge below the soil profile.
- Channels for channel routing.
- Water balance output.

Sources

- Daily sequence: `hru_control.f90`, `surface.f90`
- Interception: `sq_canopyint.f90`
- Curve number: `sq_volq.f90`, `sq_daycn.f90`, `sq_dailycn.f90`, `curno.f90`, `cn2_init.f90`
- Green-Ampt: `sq_greenampt.f90`; wetting front suction in `soils_init.f90`
- Cracks: `sq_crackvol.f90`, `sq_crackflow.f90`, `swr_percmacro.f90`
- Runoff lag and time of concentration: `sq_surfst.f90`, `time_conc_init.f90`
- Soil water routing: `swr_percmain.f90`, `swr_percmicro.f90`, `swr_satexcess.f90`
- Tile drainage: `swr_origtile.f90`, `swr_drains.f90`
- Subsurface lag: `swr_substor.f90`; lag factors in `hydro_init.f90`
- HRU hydrology parameters: `topohyd_init.f90`

Symbol or column	Fortran variable
gampt, crack, tiledrain, wtable (codes.bsn)	bsn_cc%gampt, bsn_cc%crk, bsn_cc%tdrn, bsn_cc%wtdn
surq_lag, cn_froz, urb_init_abst	bsn_prm%surlag, bsn_prm%cn_froz, bsn_prm%urb_init_abst
can_max	hru(j)%hyd%canmx
cn3_swf, perco, latq_co, lat_ttime	hru(j)%hyd%cn3_swf, hru(j)%hyd%perco, hru(j)%hyd%latq_co, hru(j)%hyd%lat_ttime
f_{perco}	hru(j)%hyd%perco_lim
slp, slp_len, lat_len	hru(j)%topo%slope, hru(j)%topo%slope_len, hru(j)%topo%lat_len
ov_mann	hru(j)%luse%ovn
dp, t_fc, lag, drain	sdr%depth, sdr%time, sdr%lag, sdr%drain_co
CN, S_{max}, w_1, w_2	cnday(j), smx(j), wrt(1,j), wrt(2,j)
Ψ_{wf}	wfsh(j)
t_{conc}, lag fraction	tconc(j), brt(j)
SW_{ex}, w_{perc}, Q_{lat} (layer)	sw_excess, sepday, latlyr
TT_{perc}	soil(j)%phys(ly)%hk
h_{wtbl}	wt_shall
Lag stores	surf_bs(1,j), bss(1,j) (lateral), bss(3,j) (tile)

Overview

The soil profile is the main water store of an HRU. Water that infiltrates is stored in the soil layers, taken up by plants, evaporated from the soil surface, or moved down and sideways as described in Hydrology.

Soil layers

Each HRU uses one soil from soils.sol. The soil can have any number of layers, each defined by the depth to its bottom (dp) and its physical properties.

If the first layer of the soil is deeper than 19.5 mm, SWAT+ adds a 10 mm surface layer with the properties of the first layer, so most HRUs have one more layer than the soil in soils.sol. The thin surface layer is used for soil evaporation, surface nutrient processes and erosion. When a file named soil_lyr_depths.sol is present in the project folder, SWAT+ instead rebuilds the layers of every soil at the depths listed in it, with a 10 mm surface layer on top. HRUs with an active septic system get extra layer boundaries at the top and bottom of the septic biozone.

Missing or out-of-range layer properties are replaced when the soil is read:

Property	Replacement
Bulk density bd	1.3 Mg/m^3 when zero; at most 2.0 Mg/m^3
Available water capacity awc	0.005 mm/mm when zero; at most 0.8 mm/mm
Saturated conductivity soil_k	When zero: 50, 20, 5 or 2 mm/hr for hydrologic group (hyd_grp) A, B, C or D, 10 mm/hr otherwise
Rock fragments rock	at most 98%
Soil albedo alb	at least 0.1
Calcium carbonate caco3 and ph	2.8% and 6.5 when zero
Anion exclusion anion_excl	0.5 when zero; at most 0.99

Water retention

SWAT+ stores soil water as the depth of water above the wilting point. The volumetric water contents of each layer are

$$\theta_{wp} = 0.40\,\frac{m_c\,\rho_b}{100}, \qquad \theta_{fc} = \theta_{wp} + AWC, \qquad \phi = 1 - \frac{\rho_b}{2.65}$$

Symbol	Meaning	Units
θ_{wp}	Water content at wilting point (−1.5 MPa)	mm/mm
θ_{fc}	Water content at field capacity (−0.033 MPa)	mm/mm
ϕ	Porosity	mm/mm
m_c	Clay content, `clay` in `soils.sol`	%
ρ_b	Moist bulk density, `bd` in `soils.sol`	Mg/m³
AWC	Available water capacity, `awc` in `soils.sol`	mm/mm

If field capacity reaches the porosity, it is set to $\phi - 0.05$ and the wilting point is moved down with it. The water a layer can store above wilting point is then

$$FC_{ly} = (\theta_{fc} - \theta_{wp})\, depth_{ly}, \qquad SAT_{ly} = (\phi - \theta_{wp})\, depth_{ly}$$

where $depth_{ly}$ is the layer thickness in mm. FC_{ly} is the water that plants can use, and the difference $SAT_{ly} - FC_{ly}$ is the drainable water that can percolate or flow laterally.

Initial soil water

Each layer starts the run with the same fraction of its field capacity water. The fraction is `sw_init` in `parameters.bsn`. When `sw_init` is zero, SWAT+ estimates it from the average annual precipitation of the HRU's weather generator station:

$$f_{init} = \frac{P_{an}}{P_{an} + \exp(9.043 - 0.002135\, P_{an})}$$

with P_{an} in mm. Dry climates start near zero and wet climates near field capacity. The water content then adjusts during the warm-up years skipped in `print.prt`.

Actual evapotranspiration

Evapotranspiration covers every process that turns liquid or solid water at or near the land surface into water vapour: evaporation from the canopy, transpiration, sublimation and evaporation from the soil and from ponded water. SWAT+ starts from the HRU's potential evapotranspiration (PET, see Climate and weather) and removes water in this order.

Canopy evaporation

Water stored on the canopy (see Hydrology) evaporates first. If PET is smaller than the canopy store, the canopy loses PET and there is no transpiration or soil evaporation that day. Otherwise the canopy is emptied and the remaining demand is $E'_0 = PET - E_{can}$.

Transpiration and soil evaporation demand

The maximum transpiration on the day depends on the leaf area of the plant community:

$$E_t = \frac{E'_0\, LAI}{3} \quad \text{for } LAI \le 3, \qquad E_t = E'_0 \quad \text{for } LAI > 3$$

The soil evaporation demand is reduced by shading from plants and residue:

$$E_s = E'_0\, cov_{sol}, \qquad cov_{sol} = \exp(-5.0 \times 10^{-5}\, CV)$$

where CV is the above-ground biomass plus surface residue (kg/ha). When the snowpack contains 0.5 mm of water or more, cov_{sol} is 0.5. If $E_t + E_s$ is larger than the demand, E_s is reduced so that the two together do not exceed it. The transpiration demand is met by plant water uptake (see Plant growth and land use management).

The same leaf area rule sets the transpiration demand for all three PET methods. SWAT+ 62.0.1 does not use the crop transpiration computed by the Penman-Monteith method.

Sublimation and ponded water

On days with a mean air temperature above 0 °C, the soil evaporation demand is taken first from the snowpack as sublimation. On HRUs with ponded water (wetlands and paddies), the evaporation demand is

$$E_s = \text{evap}\left(1 - \frac{LAI}{4}\right) E_0' \quad \text{for } LAI \le 4, \qquad E_s = 0 \text{ otherwise}$$

where `evap` is the evaporation coefficient in `hydrology.wet`. It is taken from the ponded water, and the soil itself does not evaporate while ponded water remains.

Soil evaporation by layer

The remaining demand is distributed with depth. The demand down to depth z is

$$E_{soil,z} = E_s'' \frac{z}{z + \exp(2.374 - 0.00713\, z)}$$

where E_s'' is the remaining demand and z is in mm. Half of the demand comes from the top 10 mm and 95% from the top 100 mm. The demand on a layer with its top at z_u and bottom at z_l is

$$E_{soil,ly} = E_{soil,z_l} - E_{soil,z_u}\,(1 - \text{esco})$$

Symbol	Meaning	Input
esco	Soil evaporation compensation factor	`esco` in `hydrology.hyd`, 0 to 1

With `esco = 0` each layer meets only its share of the demand. As `esco` increases, deeper layers can make up for demand that the layers above could not meet, so more water can be evaporated from lower in the profile. This direction is the opposite of SWAT 2009, where lowering `esco` allowed more compensation.

When a layer is drier than field capacity its demand is reduced:

$$E'_{soil,ly} = E_{soil,ly} \exp\left(\frac{2.5\,(SW_{ly} - FC_{ly})}{FC_{ly}}\right)$$

and a layer never gives up more than 80% of its plant-available water on one day. Only layers whose top is shallower than 500 mm evaporate. As water evaporates from the second layer, a small amount of nitrate moves up with it into the surface layer.

Soil temperature

Soil temperature affects water movement and the rate of residue decomposition and nutrient transformations. SWAT+ computes the temperature at the centre of each layer every day from the soil surface temperature, the annual average air temperature and a damping depth.

Damping depth

The maximum damping depth depends on the average bulk density of the profile, ρ_b:

$$dd_{max} = 1000 + \frac{2500\,\rho_b}{\rho_b + 686\,\exp(-5.63\,\rho_b)}$$

The damping depth on a day is scaled by the soil water content:

$$dd = dd_{max}\,\exp\left[\ln\left(\frac{500}{dd_{max}}\right)\left(\frac{1-wc}{1+wc}\right)^2\right], \qquad wc = \frac{SW}{(0.356 - 0.144\,\rho_b)\,z_{tot}}$$

where SW is the profile water content (mm) and z_{tot} the profile depth (mm).

Surface temperature

The temperature of a bare soil surface is

$$T_{bare} = \bar{T} + \frac{T_{max} - T_{min}}{2} \cdot \frac{H_{day}(1-\alpha) - 14}{20}$$

where H_{day} is the solar radiation and α the albedo of the day. Biomass, residue and snow insulate the surface. The cover weighting factor is

$$bcv = \max\left(\frac{CV}{CV + \exp(7.563 - 1.297 \times 10^{-4}\,CV)},\ \frac{SNO}{SNO + \exp(6.055 - 0.3002\,SNO)}\right)$$

with CV the above-ground biomass and residue (kg/ha) and SNO the snow water content (mm); the snow term is 1 above 120 mm. The surface temperature is

$$T_{srf} = \frac{1}{2}\left[T_{bare} + bcv\,T_{ly2} + (1 - bcv)\,T_{bare}\right]$$

where T_{ly2} is the previous day's temperature of the second soil layer.

Layer temperature

$$T_{ly} = \ell\,T_{ly,prev} + (1-\ell)\left[df\,(\bar{T}_{AA} - T_{srf}) + T_{srf}\right]$$

$$df = \frac{zd}{zd + \exp(-0.8669 - 2.0775\,zd)}, \qquad zd = \frac{z_{mid}}{dd}$$

Symbol	Meaning	Units
ℓ	Lag coefficient, 0.8	-
$T_{ly,prev}$	Layer temperature on the previous day	°C
$\bar{T}_{AA}$	Average annual air temperature of the weather generator station	°C
z_{mid}	Depth to the centre of the layer	mm

Layers start the run at the average December maximum temperature of the weather generator station. In septic HRUs, layers at and below the biozone are kept from falling far below 10 °C.

Switches and parameters

Input	File	Effect
`sw_init`	parameters.bsn	Initial soil water as a fraction of field capacity; zero estimates it from precipitation
`esco`	hydrology.hyd	Soil evaporation compensation
`epco`	hydrology.hyd	Plant uptake compensation (see Plant growth)
`can_max`	hydrology.hyd	Maximum canopy storage
`evap`	hydrology.wet	Evaporation coefficient for ponded water
`dp`, `bd`, `awc`, `soil_k`, `clay`, `rock`, `alb`	soils.sol	Layer depth, bulk density, water retention, conductivity, texture, albedo
`lai_noevap`	parameters.bsn	Read but not used: the leaf area index of 3.0 in the transpiration rule is fixed

Related pages

- soils.sol for the soil input file.
- Hydrology for infiltration, percolation and lateral flow.
- Plant growth and land use management for transpiration and root water uptake.
- Nutrient cycling for the effect of soil water and temperature on nutrient transformations.
- Water balance output for `esoil`, `eplant`, `ecanopy` and soil water.

Sources

- Soil database and layering: soil_db_read.f90, soils_init.f90
- Physical properties: soil_phys_init.f90
- Initial soil water and temperature: hydro_init.f90
- Actual evapotranspiration: et_act.f90
- Soil temperature: stmp_solt.f90
- Wetland evaporation coefficient: `type wetland_hyd_data` in reservoir_data_module.f90

Symbol or column	Fortran variable
`sw_init`	`bsn_prm%ffcb`; per soil `soil(j)%ffc`
`lai_noevap`	`bsn_prm%evlai`
`esco`, `epco`	`hru(j)%hyd%esco`, `hru(j)%hyd%epco`
`evap` (hydrology.wet)	`wet_hyd(ihyd)%evrsv`
θ_{wp}, θ_{fc}, ϕ	`soil(j)%phys(ly)%wp`, `%up`, `%por`
FC_{ly}, SAT_{ly}, SW_{ly}	`soil(j)%phys(ly)%fc`, `%ul`, `%st`
Profile totals	`soil(j)%sumfc`, `soil(j)%sumul`, `soil(j)%sw`
E'_0, E_t, E_s	`pet`, `ep_max`, `es_max`
E_{can}, soil evaporation, sublimation	`canev`, `es_day`, `snoev`
dd_{max}, dd, ρ_b	`dp`, `dd`, `soil(j)%avbd`
T_{srf}, T_{ly}	`soil(j)%tmp_srf`, `soil(j)%phys(ly)%tmp`

Overview

Erosion is the detachment, transport and deposition of soil particles by raindrop impact and flowing water. Raindrops detach particles on unprotected ground between rills, and the particles move to rills, then to larger channels and rivers, and can be deposited anywhere along the way. Erosion matters for reservoirs, which lose storage to deposited sediment, and for the soil, which loses the organic matter and nutrients in its surface layer.

SWAT+ computes soil loss from each HRU with the Modified Universal Soil Loss Equation (MUSLE) (Williams, 1975) on daily runs, or with a splash and overland flow erosion model on sub-daily runs. MUSLE replaces the rainfall energy factor of the Universal Soil Loss Equation (USLE) (Wischmeier and Smith, 1965, 1978) with a runoff factor, so it applies to single storms, reflects antecedent moisture through runoff, and needs no delivery ratio. Sediment is then lagged on its way to the channel together with surface runoff. Erosion and deposition in channels are covered on the Channels page.

Erosion is computed only on days with surface runoff and a positive peak runoff rate. HRUs with a ponded surface (wetlands and paddies) produce no surface runoff and no erosion.

Peak runoff rate

The peak runoff rate is the largest flow rate during a runoff event. It sets the erosive power of the storm in MUSLE. The method is chosen with `sed_det` in `codes.bsn`.

NRCS peak rate equation (`sed_det = 0`):

$$q_{peak} = \frac{prf}{6578.6} \cdot \frac{A\,Q_{surf}}{t_{conc}} \cdot \frac{1}{35.3}$$

Symbol	Meaning	Units
q_{peak}	Peak runoff rate	m³/s
prf	Peak rate factor, `adj_pkrt_sed` in `parameters.bsn`	-
A	HRU area	ha
Q_{surf}	Surface runoff generated on the day	mm H2O
t_{conc}	Time of concentration of the HRU (see Hydrology)	hr

This is the NRCS triangular unit hydrograph peak, $q = 484\,A\,Q/t_p$ in U.S. units, with the time of concentration used as the time to peak. The NRCS value of the peak rate factor is 484, which is also the built-in value. A zero in `parameters.bsn` is replaced by 1.0, and the SWAT+ reference data sets use 1.0, which gives peak rates 484 times smaller than the NRCS equation. Users of this method should check the value against their calibration.

Rational formula with half-hour rainfall (`sed_det = 1`):

$$q_{peak} = \frac{\alpha_{tc}\,Q_{surf}\,A_{km}}{3.6\,t_{conc}}, \qquad \alpha_{tc} = 1 - \exp\left[2\,t_{conc}\,\ln(1-\alpha_{0.5})\right]$$

where α_{tc} is the fraction of daily rain that falls in the time of concentration, $\alpha_{0.5}$ the fraction that falls in the wettest half hour (generated daily, see Climate and weather) and A_{km} the HRU area in km².

MUSLE

SWAT+ computes daily sediment yield as

$$sed = 11.8\left(V_{surf}\,q_{peak}\right)^{0.56} K_{USLE}\,C_{USLE}\,P_{USLE}\,LS_{USLE}\,CFRG$$

Symbol	Meaning	Units
sed	Sediment yield on the day	metric tons
V_{surf}	Surface runoff volume, $10\,Q_{surf}\,A$	m³
K_{USLE}	Soil erodibility factor, `usle_k` of the first layer in `soils.sol`	0.013 t m² hr/(m³ t cm)
C_{USLE}	Cover and management factor, computed daily	-
P_{USLE}	Support practice factor, `usle_p` in `cons_practice.lum`	-
LS_{USLE}	Topographic factor	-
$CFRG$	Coarse fragment factor	-

The runoff factor is the original MUSLE form of Williams (1975), with the runoff volume in m³. The SWAT 2009 documentation writes the same factor with runoff in mm and area in ha, without the factor 10, so SWAT+ sediment yields for the same inputs are larger by $10^{0.56}$, about 3.6.

Soil erodibility

Some soils erode more easily than others even when all other factors are the same. K_{USLE} is the soil loss per unit of erosion index on a standard plot 22.1 m long on a 9% slope, kept in continuous fallow. Values are entered for each soil in soils.sol (see that page for estimation methods).

Cover and management

The cover and management factor is the ratio of soil loss from land under given conditions to soil loss from clean-tilled, continuous fallow. Plant cover changes through the year, so SWAT+ computes C_{USLE} every day from the surface residue and the plant canopy:

$$C_{USLE} = \exp\left[-\text{rsd_cover}\,\frac{rsd+1}{1000}\right]\left[1 - f_{gc}\,\exp(-0.328\,h_c)\right]$$

$$f_{gc} = \frac{B}{B + \exp(1.175 - 1.748\,B)}$$

Symbol	Meaning	Units
rsd_cover	Residue cover factor, **rsd_cover** in parameters.bsn (0.75 when zero)	-
rsd	Surface residue	kg/ha
B	Above-ground biomass of the plant community	t/ha
h_c	Canopy height of the plant community	m

The minimum C factor of each plant (**usle_c_min** in plants.plt) is not used by this daily calculation. It is used only by the sub-daily erosion model below. The daily C factor and its two parts are written to the erosion output.

Support practice

The support practice factor is the ratio of soil loss with a practice such as contouring, strip cropping or terracing to soil loss with up-and-down-slope culture. It comes from the row of cons_practice.lum named in the **cons_prac** column of landuse.lum.

Topographic factor

$$LS_{USLE} = \left(\frac{L_{hill}}{22.1}\right)^{m}\left(65.41\,\sin^2\alpha_{hill} + 4.56\,\sin\alpha_{hill} + 0.065\right), \qquad m = 0.6\left[1 - \exp(-35.835\,slp)\right]$$

where L_{hill} is the slope length (**slp_len** in topography.hyd, m), slp the slope (**slp**, m/m) and $\alpha_{hill} = \arctan(slp)$. cons_practice.lum also has a maximum slope length for each practice (**slp_len_max**); SWAT+ 62.0.1 reads it but the LS factor uses the full slope length.

Coarse fragments

$$CFRG = \exp(-0.053\,rock)$$

where $rock$ is the rock content of the first soil layer (**rock** in soils.sol, %).

Snow cover

The SWAT 2009 documentation reduces MUSLE sediment yield under a snowpack. In SWAT+ 62.0.1 that reduction is not reached for any snow depth, so snow cover does not change the sediment yield. Snowmelt runoff still produces erosion through Q_{surf} and q_{peak}.

USLE soil loss for output

Alongside MUSLE, SWAT+ computes the USLE soil loss from rainfall energy for output (`usle` in the HRU loss files):

$$A_{USLE} = 1.292\, EI_{USLE}\, K_{USLE}\, C_{USLE}\, P_{USLE}\, LS_{USLE}\, CFRG$$

$$EI_{USLE} = R_{day}\left[12.1 + 8.9\left(\log_{10} I - 0.434\right)\right]\frac{I_{30}}{1000}$$

where $I_{30} = 2\,\alpha_{0.5}\,R_{day}$ is the maximum 30-minute intensity (mm/hr) and $I = -2\ln(1-\alpha_{0.5})\,R_{day}$ the peak intensity. The USLE value is reported only. It does not add to the sediment yield.

Sub-daily erosion

When the simulation uses a sub-daily time step with Green-Ampt infiltration, sediment for each time step comes from raindrop splash and overland flow erosion instead of MUSLE.

Splash erosion. Rain is split into direct throughfall and leaf drip by the canopy cover, $\min(LAI, 1)$. Their kinetic energies are

$$KE_{direct} = 8.95 + 8.44\,\log_{10} I, \qquad KE_{leaf} = 15.8\sqrt{0.5\,h_c} - 5.87$$

in J/m²/mm, where I is the rainfall intensity of the step (mm/hr). Splash detachment is

$$sed_{spl} = k_d\, KE\, \exp\left(-\text{splash}\,\frac{h_{flow}}{1000}\right) A_{km}$$

with k_d a soil detachability (g/J) set from the texture of the top layer (1.2 for silt up to 3.0 for loamy sand), KE the total kinetic energy of the step, h_{flow} the runoff depth of the step (mm) and `splash` in `parameters.bsn`. Splash stops when the runoff depth is at least three times the median raindrop diameter, or is negligible.

Overland flow erosion.

$$sed_{ov} = 11.02\,\text{rill}\; K_{USLE}\; \text{cov_mgt}\; C\; \tau^{\,\text{surq_exp}}, \qquad \tau = 9807\,\frac{h_{flow}}{1000}\,slp$$

in kg/hr/m², converted to tons for the step and the HRU area. `rill`, `cov_mgt` and `surq_exp` are in `parameters.bsn`; C is the SWAT 2009 cover factor computed from `usle_c_min` and the soil cover. All three default to zero. The rate is proportional to `rill` and `cov_mgt`, so overland flow erosion is zero unless both are set; a zero `surq_exp` makes the shear stress term equal to 1.

On urban HRUs both terms apply only to the pervious fraction. The sediment of the day is the sum over the steps after the lag described below.

With a sub-daily time step and the curve number method, the splash and overland flow terms are not computed and MUSLE is not used either, so the HRU produces no lagged sediment from the surface.

Sediment in lateral and tile flow

Lateral and tile flow transport a small sediment load:

$$sed_{lat} = \frac{\text{lat_sed}\,(Q_{lat} + Q_{tile})\,A}{100000}$$

in metric tons, with `lat_sed` from `hydrology.hyd` treated as a concentration in mg/L, and Q_{lat}, Q_{tile} in mm. Sediment concentrations in subsurface flow are usually very low and matter only where return flow is large. Organic nitrogen and phosphorus in lateral flow are set by `lat_orgn` and `lat_orgp` in the same file.

Particle size

The sediment from the surface is split into five classes (sand, silt, clay, small aggregates and large aggregates) using fractions set from the sand, silt and clay content of the top soil layer (Foster et al., 1980):

Class	Fraction of detached sediment
Sand	$f_{sa}(1 - f_{cl})^{2.49}$
Silt	$0.13\, f_{si}$
Clay	$0.20\, f_{cl}$
Small aggregates	$2.0\, f_{cl}$ for $f_{cl} < 0.25$; $0.28\,(f_{cl} - 0.25) + 0.5$ up to $f_{cl} = 0.5$; 0.57 above
Large aggregates	the remainder

where f_{sa}, f_{si} and f_{cl} are the sand, silt and clay fractions. If the first four add up to more than 1, they are rescaled and large aggregates are set to zero.

Sediment lag

Sediment, the particle size classes and the nutrients attached to sediment pass through the same surface lag store as runoff, and the same fraction, $1 - \exp(-\text{surq_lag}/t_{conc})$, is released each day (see Hydrology).

Urban areas

For an HRU with an urban land use (the `urban` and `urb_ro` columns of `landuse.lum`, parameters in `urban.urb`), the sediment and nutrient loads of the impervious fraction f_{imp} (`frac_imp`) are computed separately and mixed with the loads of the pervious part:

$$L = L_{urb}\, f_{imp} + L_{perv}\,(1 - f_{imp})$$

where L_{perv} is the load from MUSLE and the soil nutrient routines. Urban loads are computed before the surface lag store, and sediment from impervious surfaces is all silt. `urb_ro` selects one of two methods; any other value gives no urban loads.

USGS regression equations (`usgs_reg`). On days with more than 0.1 mm of precipitation and of surface runoff, the storm load of each constituent (kg) is

$$Y = \frac{\beta_0}{2.205} \left(\frac{R_{day}}{25.4}\right)^{\beta_1} \left(\frac{A\, f_{imp}}{2.589}\right)^{\beta_2} (100\, f_{imp} + 1)^{\beta_3}\, \beta_4$$

with R_{day} the precipitation (mm) and A the HRU area (km²). The coefficients (Driver and Tasker, 1988) depend on the mean annual precipitation of the HRU's weather generator station:

Constituent	Below 508 mm	508 to 1016 mm	1016 mm and above
Suspended solids	1778, 0.867, 0.728, 0.157, 2.367	812, 1.236, 0.436, 0.202, 1.938	97.7, 1.002, 1.009, 0.837, 2.818
Total N	20.2, 0.825, 1.070, 0.479, 1.258	4.04, 0.936, 0.937, 0.692, 1.373	1.66, 0.703, 0.465, 0.521, 1.845
Total P	1.725, 0.884, 0.826, 0.467, 2.130	0.697, 1.008, 0.628, 0.469, 1.790	1.618, 0.954, 0.789, 0.289, 2.247

Each cell lists β_0 to β_4. Total N is split into 70 % organic N and 30 % nitrate, and total P into 75 % sediment-bound and 25 % soluble P.

Build-up and wash-off (`buildup_washoff`). Solids build up on impervious surfaces between storms:

$$SED = \frac{SED_{mx}\, t}{t_{1/2} + t}$$

where SED_{mx} is `dirt_max` (kg per curb km), $t_{1/2}$ is `t_halfmax` (days) and t the number of days since the surface was last washed. On a day with more than 0.1 mm of surface runoff the solids left after wash-off are

$$SED_{after} = SED\, \exp(-k_w\, q_{pk}\, t_{ro})$$

with k_w = `urb_wash` (1/mm), q_{pk} the peak runoff rate (mm/h) and t_{ro} the storm duration from the half-hour rainfall fraction plus the time of concentration (h, at most 24). The washed-off solids are $(SED - SED_{after})$ times `curb_den`, in kg/ha; their N, P and nitrate contents are `conc_totn`, `conc_totp` and `conc_no3n` (mg/kg). Organic N is total N minus nitrate, and total P is split 75 % sediment-bound and 25 % soluble. After the storm, t is reset to the time that would build up SED_{after}. Sub-daily runs apply the same method for each time step. Street sweeping has no effect in 62.0.1 (see `sweep.ops`).

Filter strips, grassed waterways and user BMPs

These practices are attached to a land use through the `vfs`, `grww` and `bmp` columns of `landuse.lum` (parameters in the structural practice files). They act on the loads released from the surface lag store each day. None of them changes the runoff volume, so the water balance is unchanged.

Filter strips

A vegetative filter strip at the field edge (White and Arnold, 2009) is split into two sections. Section 1, 90 % of the strip area, receives the fraction $1 - c$ of the field runoff; section 2, 10 % of the strip area, receives the fraction $c\,(1 - h)$. The rest, $c\,h$, is channelized and passes the strip unchanged. Here c is `con_vfs` and h is `cha_q`, and the strip area is the HRU area divided by `fld_vfs`.

For each section i, the runoff depth over the strip q_i (mm) and the sediment load per strip area s_i (kg/m²) give the percent reductions

$$RR_i = 75.8 - 10.8\, \ln q_i + 25.9\, \ln K_{sat}, \qquad SR_i = 79.0 - 1.04\, s_i + 0.213\, RR_i$$

with K_{sat} the saturated conductivity of the top soil layer (mm/h). The runoff reduction RR_i is used only to estimate nutrient removal. The other reductions follow from these:

Load	Reduction (%)
Sediment	SR_i
Organic N	$0.036\, SR_i^{1.69}$
Nitrate in surface runoff	$39.4 + 0.584\, RR_i$
Sediment-bound P	$0.903\, SR_i$
Soluble P	$29.3 + 0.51\, RR_i$
Pesticide in runoff and on sediment	RR and SR

Each reduction is limited to 0 to 100 % and weighted by the share of the HRU runoff each section receives. Pesticide removed by the strip is returned to the top soil layer. In 62.0.1 the soluble P reduction of section 2 uses the value computed for sediment-bound P, and pesticide loads are reduced a second time after the removed amount is returned.

Grassed waterways

A grassed waterway is a vegetated trapezoidal channel with 8:1 side slopes through the field. The peak flow is computed from the day's runoff with a triangular hydrograph, and the flow depth in the waterway from Manning's equation. Part of the runoff enters the waterway as sheet flow over its side slopes; 20 % of the wetted side slope area above the flow, at most 10 % of the HRU, acts as a filter, with reductions computed as for filter strips:

$$RR = 95.6 - 10.79 \ln q_{sf}, \qquad SR = 79.0 - 1.04\, s_{sf} + 0.213\, RR$$

both limited to 95 %. The sediment left after this filtering is then routed through the waterway: where its concentration exceeds the transport capacity $C_{ch} = \text{sed_co}\ v^{1.5}$ (with v the flow velocity), the excess is deposited. Sediment, organic N and sediment-bound P are reduced by the fraction of sediment that leaves the waterway, and nitrate and soluble P in surface runoff by RR.

User BMPs

A user BMP multiplies the HRU loads by $1 - e$ for each removal efficiency e in `bmpuser.str`: sediment, sediment-bound P, soluble P, organic N, and nitrate in surface runoff and lateral flow.

Riparian buffers and saturated buffers

A riparian buffer reduction of nitrate in lateral flow is coded but depends on a buffer width that no input sets, so it has no effect in 62.0.1. A saturated buffer diverts part of the tile flow of one HRU into the soil of another HRU under a decision table (see `satbuffer.str`).

Channel erosion and deposition

Sediment that reaches a channel can be deposited on the floodplain, deposited in the channel, or joined by sediment eroded from the channel banks and bed. These processes use the channel properties in `hyd-sed-lte.cha` and are described on the Channels page.

Switches and parameters

Input	File	Effect
`sed_det`	`codes.bsn`	`0` NRCS peak rate, `1` rational formula with half-hour rainfall
`adj_pkrt_sed`	`parameters.bsn`	Peak rate factor for `sed_det = 0`
`adj_pkrt`	`parameters.bsn`	Multiplier on the generated half-hour rainfall fraction
`rsd_cover`	`parameters.bsn`	Residue term of the daily C factor
`splash`, `rill`, `surq_exp`, `cov_mgt`	`parameters.bsn`	Sub-daily splash and overland flow erosion
`usle_k`, `rock`, `sand`, `silt`, `clay`	`soils.sol`	Erodibility, coarse fragments, particle size
`usle_p`	`cons_practice.lum`	Support practice factor
`slp`, `slp_len`	`topography.hyd`	Slope and slope length
`lat_sed`	`hydrology.hyd`	Sediment concentration in lateral and tile flow

Related pages

- Hydrology for surface runoff, time of concentration and the surface lag.
- Nutrient cycling for nitrogen and phosphorus attached to sediment.
- Channels for channel erosion and deposition.
- Channel, sediment and nutrient output.

Sources

- Daily call sequence: `surface.f90`, `hru_control.f90`

- Peak rate: ero_pkq.f90
- MUSLE: ero_ysed.f90; C factor: ero_cfactor.f90; erosivity: ero_eiusle.f90
- K, P and CFRG product: hydro_init.f90; LS and P: plant_init.f90
- Sub-daily erosion: ero_ovrsed.f90
- Lateral sediment: swr_latsed.f90
- Particle size fractions: soil_phys_init.f90
- Sediment lag: stor_surfstor.f90
- Urban loads: hru_urban.f90, hru_urbanhr.f90 (sub-daily), regression in regres.f90; precipitation region `wgn_pms%ireg` set in `cli_initwgn.f90`; days since wash-off `twash`. The regression for COD is computed but not used.
- Filter strips: smp_filter.f90 (section 2 soluble P assigns `remove21` and then uses `remove2`); riparian buffer: smp_buffer.f90 (`filterw` is allocated as zero and never set). Grassed waterways: smp_grass_wway.f90. User BMPs: smp_bmpfixed.f90. Called from `hru_control.f90` after `stor_surfstor`.
- References: Driver, N. E., and Tasker, G. D. (1988), Techniques for estimation of storm-runoff loads, volumes, and selected constituent concentrations in urban watersheds in the United States, USGS Open-File Report 88-191. White, M. J., and Arnold, J. G. (2009), Development of a simplistic vegetative filter strip model for sediment and nutrient retention at the field scale, Hydrological Processes 23, 1602–1616.
- Half-hour rainfall: climate_control.f90, cli_initwgn.f90
- Defaults for zero parameters: basin_prm_default.f90

Symbol or column	Fortran variable
`sed_det`	`bsn_cc%sed_det`
`adj_pkrt_sed` (prf), `adj_pkrt`	`bsn_prm%prf`, `bsn_prm%adj_pkr`
`rsd_cover`	`bsn_prm%rsd_covco`
`splash`, `rill`, `surq_exp`, `cov_mgt`	`bsn_prm%eros_spl`, `bsn_prm%rill_mult`, `bsn_prm%eros_expo`, `bsn_prm%c_factor`
`usle_c_min`	`pldb%usle_c`, community value `cvm_com(j)`
`lat_sed`	`hru(j)%hyd%lat_sed`
q_{peak}	`qp_cms`
sed	`sedyld(j)`
C_{USLE}	`usle_cfac(j)`
$KPLSCFRG \times 11.8$	`hru(j)%lumv%usle_mult`
$C\times$ the above	`cklsp(j)`
LS_{USLE}, P_{USLE}	`hru(j)%lumv%usle_ls`, `hru(j)%lumv%usle_p`
EI_{USLE}, A_{USLE}	`usle_ei`, `usle`
Sub-daily sediment	`hhsedy(j,k)`
Particle size fractions	`soil(j)%det_san`, `det_sil`, `det_cla`, `det_sag`, `det_lag`

Overview

Each HRU has a plant community: one or more plants listed in plant.ini and described in plants.plt. SWAT+ grows every living plant of the community each day with a simplified version of the EPIC plant growth model. Phenological development follows accumulated heat units. Potential growth comes from intercepted light and the radiation use efficiency of the plant, and is reduced by water, temperature, nitrogen, phosphorus, aeration and (when salts are simulated) salinity stress. Growth removes water and nutrients from the soil, and harvest, kill and senescence return plant material to the residue and soil pools.

Plants are planted, harvested and killed by the management schedule of the HRU and by decision tables (see Land use management).

Plant types

The `plnt_typ` column of plants.plt sets how a plant develops:

plnt_typ	Behaviour
warm_annual, warm_annual_tuber	Annual; leaf area declines towards the end of the season
cold_annual, cold_annual_tuber	Annual; goes dormant in winter if planted late enough
perennial	Grows over several years towards maturity (yrs_mat); goes dormant each year

The gro_trig column sets what restarts growth: temp_gro for plants driven by temperature and day length, moisture_gro for plants in climates with a wet and dry season, where growth follows the ratio of precipitation to PET.

Heat units

Plants develop at a rate set by temperature. Each day a plant above its base temperature accumulates heat units:

$$HU = \bar{T} - T_{base} \quad \text{when } \bar{T} > T_{base}$$

and the fraction of potential heat units reached is

$$fr_{PHU} = \sum_{i=1}^{d} \frac{HU_i}{PHU}$$

Symbol	Meaning	Input
$\bar{T}$	Mean air temperature of the day	°C
T_{base}	Minimum (base) temperature for growth	tmp_base in plants.plt, °C
PHU	Potential heat units needed to bring the plant to maturity	see below

fr_{PHU} starts at 0 at planting and reaches 1 at maturity. SWAT+ sets PHU for each plant at the start of the run from the long-term monthly temperatures of the weather generator station and days_mat in plants.plt:

- days_mat > 0: the heat units accumulated over days_mat days, starting from an estimated planting date. For annuals the planting date is taken where base-zero heat units reach 15% of their annual total.
- days_mat $= 0$: 95% of the heat units above T_{base} in a whole year, so the plant can stay active through the entire growing season.

For perennials a second counter runs over the years to maturity, with $PHU \times$ yrs_mat as its target. It sets canopy height and root depth.

Base-zero heat units ($T_{base} = 0$, divided by the annual total) are also accumulated each year from 1 January. They schedule operations that happen when no plant is growing.

Dormancy

Perennials and cold-season annuals go dormant when the day length falls below the minimum day length of the station plus a threshold. While dormant a plant does not grow. When a perennial goes dormant its seed mass is moved to residue and its heat unit counter is reset; it starts accumulating heat units again when the day length rises above the threshold. Cold-season annuals go dormant only if they have reached less than 75% of their heat units.

Light interception and biomass

Each growing plant intercepts photosynthetically active radiation (PAR), taken as half of incoming solar radiation, according to Beer's law:

$$H_{phosyn} = 0.5\, H_{day}\, [1 - \exp\left(-k_\ell\, (LAI + 0.05)\right)]$$

where H_{day} is the solar radiation (MJ/m²), k_ℓ the light extinction coefficient (`ext_co` in `plants.plt`) and LAI the leaf area index of the plant. Each plant intercepts light as though it were alone: SWAT+ 62.0.1 has a routine for competition for light between taller and shorter plants, but it is switched off.

Radiation use efficiency

The radiation use efficiency (RUE) is the biomass produced per unit of intercepted light. It responds to CO2 (Stockle et al., 1992). When `co2` in `parameters.bsn` is above 350 ppm,

$$RUE = \frac{100\, CO_2}{CO_2 + \exp(r_1 - r_2\, CO_2)}$$

where r_1 and r_2 are fitted so that RUE equals `bm_e` at 330 ppm and `bm_e_hi` at `co2_hi`. Otherwise RUE is `bm_e`. CO_2 is the concentration for the year (see Climate and weather).

When the vapour pressure deficit vpd is above 1 kPa, RUE declines:

$$RUE = RUE - \Delta ruedcl_{vpd}\, (vpd - 1), \qquad RUE \geq 0.27\, \text{bm_e}$$

where $\Delta ruedcl_{vpd}$ is `ru_vpd` in `plants.plt`.

Biomass growth

Potential daily biomass growth is $\Delta bio = RUE \times H_{phosyn}$ (kg/ha). The actual growth is

$$\Delta bio_{act} = \Delta bio\, \gamma_{reg}, \qquad \gamma_{reg} = \min\left(\gamma_w,\ \gamma_t,\ \gamma_n,\ \gamma_p,\ \gamma_a,\ \gamma_s\right)$$

where each γ is a growth factor between 0 (no growth) and 1 (no stress) for water, temperature, nitrogen, phosphorus, aeration and salinity. When salinity is simulated with the salt tolerance option, the salinity factor multiplies the minimum of the others instead. Plant carbon is taken as 42% of biomass. `nostress` in `codes.bsn` turns stresses off: `1` sets all factors to 1 and `2` sets only the nitrogen and phosphorus factors to 1.

The new biomass is split into roots, leaves, stems and seed. The root fraction falls from `rt_st_beg` at the start of the season to `rt_st_end` at maturity (for perennials, over the first 20% of the years to maturity).

Leaf area and canopy

Leaf area development

Leaf area follows an optimal curve defined by two points from `plants.plt`: at `frac_hu1` of the heat units the plant reaches `lai_max1` of its maximum LAI, and at `frac_hu2` it reaches `lai_max2`. The fraction of maximum LAI for a given fr_{PHU} is

$$fr_{LAImx} = \frac{fr_{PHU}}{fr_{PHU} + \exp(\ell_1 - \ell_2\, fr_{PHU})}$$

with the shape coefficients ℓ_1 and ℓ_2 fitted to the two points. Until fr_{PHU} reaches `hu_lai_decl`, the leaf area added on a day is

$$\Delta LAI = \left(fr_{LAImx,i} - fr_{LAImx,i-1}\right) LAI_{mx} \left[1 - \exp\left(5\left(LAI_{i-1} - LAI_{mx}\right)\right)\right] \sqrt{\gamma_w}$$

and is shared among plants in proportion to their LAI times canopy height. LAI_{mx} is `lai_pot`. For perennials it grows with age:

$$LAI_{mx} = \text{lai_pot} \left(\frac{yr_{cur}}{\text{yrs_mat}}\right)^{\text{bm_tree_acc}}$$

Leaf senescence

After `hu_lai_decl`, annuals lose leaf area until maturity:

$$LAI = LAI_{mx} \left(\frac{1 - fr_{PHU}}{1 - fr_{PHU,sen}}\right)^{\text{dlai_rate}}$$

where $fr_{PHU,sen}$ is the heat unit fraction at which decline started. Temperature-driven perennials drop to `lai_min` over 15 days once `hu_lai_decl` is passed. Moisture-driven perennials shed leaves continuously at a turnover rate between `leaf_tov_mn` and `leaf_tov_mx` months, faster when the 30-day precipitation to PET ratio is below 0.5. Leaves that fall are added to the surface residue.

Canopy height and roots

$$h_c = h_{c,mx} \sqrt{fr_{LAImx}}$$

with $h_{c,mx}$ = `can_ht_max`. Root depth grows with heat units:

$$z_{root} = 2.5\, fr_{PHU}\, z_{root,mx}, \qquad 25.4 \text{ mm} \le z_{root} \le \text{profile depth}$$

with $z_{root,mx}$ = `rt_dp_max` (converted to mm). Perennials use the multi-year heat unit fraction, so their roots keep deepening until maturity.

Water uptake

The potential transpiration of the HRU (see Soil water and temperature) is shared among growing plants in proportion to their LAI. Each plant takes up water from the layers within its root zone. The potential uptake down to depth z is

$$w_{up,z} = \frac{E_t}{1 - \exp(-\beta_w)} \left[1 - \exp\left(-\beta_w \frac{z}{z_{root}}\right)\right]$$

with the water use distribution parameter β_w fixed at 10, so about half of the demand comes from the top 7% of the root zone. The uptake from a layer is the potential uptake to its bottom minus that to its top, with the shortfall of the layers above added back in part:

$$w'_{up,ly} = w_{up,z_l} - w_{up,z_u}\left(1 - \text{epco}\right)$$

where `epco` is the plant uptake compensation factor in `hydrology.hyd`. With `epco` near 1 deeper layers can make up for dry layers above; near 0 they cannot. The second soil layer always compensates fully. Uptake is limited by the water in the layer, and SWAT+ does not reduce uptake as the soil dries below 25% of available water as SWAT 2009 did.

The water stress factor is the ratio of actual to potential transpiration:

$$\gamma_w = \frac{\sum w_{actualup}}{E_t}$$

Aeration stress

When the profile contains more water than field capacity, roots lack oxygen. With $sat_{co} = (SW - FC)/(SAT - FC)$ for the whole profile and $a =$ `aeration` in `plants.plt`,

$$\gamma_a = 1 - \frac{s}{s + \exp(2.9014 - 0.03867\, s)}, \qquad s = 100\, \frac{sat_{co} - a}{1.0001 - a}$$

for $s > 0$; otherwise $\gamma_a = 1$.

Temperature stress

In SWAT+ 62.0.1 temperature stress is

$$\gamma_t = \sin\left(\frac{\pi}{2} \cdot \frac{\bar{T} - T_{base}}{T_{opt} - T_{base}}\right), \qquad 0 \le \gamma_t \le 1$$

where T_{opt} is `tmp_opt`. The factor is 0 at or below the base temperature, 1 at the optimum, falls again above it, and reaches 0 at $2\,T_{opt} - T_{base}$. The SWAT 2009 exponential temperature stress and the cold-night rule (no growth when the minimum temperature is 15 °C below the annual mean) are computed first but replaced by this sine function.

Nitrogen and phosphorus

Demand

The optimal nitrogen fraction of the plant falls as it develops:

$$fr_N = (fr_{N,1} - fr_{N,3})\left[1 - \frac{x}{x + \exp(n_1 - n_2\, x)}\right] + fr_{N,3}$$

where $fr_{N,1}$, $fr_{N,2}$ and $fr_{N,3}$ are `frac_n_em`, `frac_n_50` and `frac_n_mat` (at emergence, at 50% of maturity and at maturity), n_1 and n_2 are fitted to them, and x is fr_{PHU} for annuals or the fraction of `yrs_mat` reached for perennials. The demand is the optimal nitrogen mass, $fr_N \times$ biomass, minus the nitrogen in the plant, and is limited to four times `frac_n_mat` times the day's potential growth. Phosphorus follows the same form with `frac_p_em`, `frac_p_50` and `frac_p_mat`.

Uptake

Nitrate is taken from the root zone with a depth distribution like water uptake, using `n_uptake` in `parameters.bsn` as the distribution parameter; labile phosphorus uses `p_uptake`. When several plants compete and the soil cannot meet the total demand, each plant gets a share in proportion to its demand.

Nitrogen fixation

Legumes (`nfix_co` > 0 in `plants.plt`) fix nitrogen when the soil cannot supply their demand. Fixation depends on growth stage, soil water and soil nitrate:

$$N_{fix} = \min\left(6,\ f_{gr}\ \min(1, f_{sw}, f_{no3})\,(N_{dem} - N_{up})\right)$$

Factor	Value
f_{gr}	0 before 15% of heat units; rises to 1 at 30%; 1 to 55%; falls to 0 at 75%
f_{sw}	$SW/(0.85\,FC)$ for the profile

f_{no3}	1 for profile nitrate up to 100 kg N/ha, falling linearly to 0 at 300 kg N/ha

The result is blended with the unmet demand by the legume coefficient, $N'_{fix} = \text{nfix_co}\, N_{fix} + (1-\text{nfix_co})(N_{dem} - N_{up})$, and limited to the unmet demand and to `n_fix_max` in `parameters.bsn` (kg N/ha/day).

Nutrient stress

For non-legumes the nitrogen stress factor is

$$\gamma_n = \frac{\varphi}{\varphi + \exp(3.535 - 0.02597\,\varphi)}, \qquad \varphi = 200\left(\frac{N_{plant}}{N_{opt}} - 0.5\right)$$

which is 0 when the plant contains half its optimal nitrogen and close to 1 at the optimum. It is raised to the ratio of the day's uptake to demand when that is larger. Legumes have no nitrogen stress. Phosphorus stress uses the same form.

Harvest index and yield

The harvest index is the fraction of above-ground biomass removed as yield. It grows through the season towards the optimal value `harv_idx` in `plants.plt`:

$$HI = \text{harv_idx}\,\frac{100\,fr_{PHU}}{100\,fr_{PHU} + \exp(11.1 - 10\,fr_{PHU})}$$

After half the heat units are reached, water deficit lowers it. With $wu = 100\sum E_t / \sum PET$ accumulated from that point,

$$HI_{act} = (HI - HI_{min})\frac{wu}{wu + \exp(6.13 - 0.0883\,wu)} + HI_{min}$$

with $HI_{min} = \text{HI}/2$. The seed (yield) pool of the plant is kept at this fraction of the above-ground biomass as the plant grows. A grain harvest raises the plant's harvest index target to at least `harv_idx_ws`, but only after the seed pool has been built, so `harv_idx_ws` does not change the yield of that harvest.

Harvest operations are defined in `harv.ops` by `harv_typ`, `harv_idx`, `harv_eff` and `harv_bm_min`. A harvest takes place only when the plant biomass is above `harv_bm_min` (kg/ha).

`harv_typ`	Yield removed
`grain`, `picker`	`harv_eff` × seed mass; the rest of the seed goes to the soil's active humus
`biomass`, `tree`, `stripper`	`harv_idx` × `harv_eff` × (seed + leaf + stem)
`tuber`, `peanuts`	Below-ground yield
`residue`	A fraction of surface residue, given in the operation

Yield is reduced by any pesticide stress on the plant, and pesticide in the plant is removed with the yield.

Land use management

landuse.lum

Each HRU points to a row of `landuse.lum`, which names the plant community (`plnt_com`), the management schedule (`mgt`), the curve number row (`cn2`), the conservation practice (`cons_prac`), the urban land type, the overland Manning's n row (`ov_mann`), and any tile drain, septic system, filter strip, grassed waterway or user BMP.

Management schedules

A schedule in management.sch lists operations by date or by heat units, followed by the names of any decision tables that apply to the HRU. Each operation has a type, a month and day, a heat unit value and up to three data columns:

Operation	Action
plnt	Plant one plant or the whole community
harv	Harvest (see above)
kill	Kill the plant; biomass goes to residue
hvkl	Harvest and kill
till	Tillage, mixing residue and nutrients into the soil (tillage.til)
irrm	Irrigation by date (irr.ops)
fert	Fertiliser (fertilizer.frt)
manu	Manure application
pest	Pesticide application (chem_app.ops)
graz	Start grazing (graze.ops)
cnup	Change the curve number
burn	Prescribed burn (fire.ops)
swep	Street sweeping
dwm	Set drain depth for drainage water management
weir	Set or change a paddy or wetland weir height
irrp, irpm, pudl	Rice paddy irrigation and puddling
mons	Start or end of a monsoon planting window
skip	Skip to the end of the year

What each operation does to plants and soil, with its equations, is described with its parameters: harvest types and harvest efficiency, grazing, irrigation, burning and street sweeping in *.ops; fertilizer and manure placement in fertilizer.frt; tillage and biological mixing in tillage.til; and automatic irrigation and fertilization by decision table in *.dtl.

An operation with a heat unit value runs when the heat unit fraction passes it: fr_{PHU} of the plant named in the operation if that plant is growing, or the base-zero heat units otherwise. An operation without a heat unit value runs on its month and day. When all remaining operations of a year share one date, the scheduler moves on after one full pass.

Decision tables

Decision tables in lum.dtl are conditional rule sets: each table has conditions, alternatives that combine them, and actions that fire when an alternative is true. The tables named in an HRU's schedule are evaluated every day before the other landscape processes, so they can plant, harvest, irrigate, fertilise or change land use in response to weather, soil water or plant growth. See Decision tables.

Switches and parameters

Input	File	Effect
nostress	codes.bsn	0 all stresses, 1 no stress, 2 no nutrient stress
co2	parameters.bsn	CO2 concentration without co2_yr.dat, ppm
n_uptake, p_uptake	parameters.bsn	Depth distribution of N and P uptake
n_fix_max	parameters.bsn	Maximum daily N fixation, kg N/ha
epco	hydrology.hyd	Plant uptake compensation
plnt_typ, gro_trig, days_mat	plants.plt	Plant type, growth trigger, days to maturity
bm_e, bm_e_hi, co2_hi, ru_vpd, ext_co	plants.plt	Light interception and RUE
lai_pot, frac_hu1, lai_max1, frac_hu2, lai_max2, hu_lai_decl, dlai_rate, lai_min	plants.plt	Leaf area curve and decline

Input	File	Effect
can_ht_max, rt_dp_max, rt_st_beg, rt_st_end	plants.plt	Canopy height, root depth, root fraction
tmp_base, tmp_opt	plants.plt	Temperature response
frac_n_*, frac_p_*, nfix_co	plants.plt	Nutrient demand and fixation
harv_idx, harv_idx_ws	plants.plt	Harvest index
aeration	plants.plt	Aeration stress threshold
yrs_mat, bm_tree_acc, bm_tree_max, leaf_tov_mn, leaf_tov_mx	plants.plt	Perennials
bm_dieoff	plants.plt	Read but not applied at dormancy in 62.0.1

Related pages

- plants.plt, landuse.lum, management.sch and operations, lum.dtl
- Soil water and temperature for the transpiration demand.
- Nutrient cycling for the soil nitrogen and phosphorus pools.
- Carbon for residue and soil carbon.
- Decision tables.

Sources

- Community, PAR and transpiration split: pl_community.f90
- Growth driver: pl_grow.f90; biomass: pl_biomass_gro.f90
- Heat units and nutrient demand: pl_nut_demand.f90, pl_nupd.f90, pl_pupd.f90; PHU setup in plant_init.f90
- Leaf area, roots, seed: pl_leaf_gro.f90, pl_leaf_senes.f90, pl_root_gro.f90, pl_seed_gro.f90
- Dormancy: pl_dormant.f90
- Stresses: pl_waterup.f90, pl_tstr.f90, nuts.f90
- Uptake and fixation: pl_nup.f90, pl_pup.f90, pl_nfix.f90
- Scheduling: mgt_operatn.f90, mgt_sched.f90, mgt_read_mgtops.f90; decision tables evaluated in hru_control.f90
- Harvest: mgt_harvgrain.f90, mgt_harvbiomass.f90, mgt_harvtuber.f90, mgt_harvresidue.f90; type harvest_operation in mgt_operations_module.f90
- Plant database type: type plant_db in plant_data_module.f90

plants.plt column	Fortran variable
plnt_typ, gro_trig	pldb%typ, pldb%trig
bm_e, bm_e_hi, co2_hi, ru_vpd	pldb%bio_e, pldb%bioehi, pldb%co2hi, pldb%wavp
harv_idx, harv_idx_ws	pldb%hvsti, pldb%wsyf
lai_pot, frac_hu1, lai_max1, frac_hu2, lai_max2	pldb%blai, pldb%frgrw1, pldb%laimx1, pldb%frgrw2, pldb%laimx2
hu_lai_decl, dlai_rate, lai_min	pldb%dlai, pldb%dlai_rate, pldb%alai_min
can_ht_max, rt_dp_max	pldb%chtmx, pldb%rdmx
tmp_opt, tmp_base	pldb%t_opt, pldb%t_base
frac_n_em, frac_n_50, frac_n_mat	pldb%pltnfr1, pldb%pltnfr2, pldb%pltnfr3
frac_p_em, frac_p_50, frac_p_mat	pldb%pltpfr1, pldb%pltpfr2, pldb%pltpfr3
stcon_max, vpd, frac_stcon	pldb%gsi, pldb%vpdfr, pldb%gmaxfr
usle_c_min	pldb%usle_c
plnt_decomp	pldb%rsdco_pl
bm_tree_acc, yrs_mat, bm_tree_max	pldb%laixco_tree, pldb%mat_yrs, pldb%bmx_peren
ext_co, leaf_tov_mn, leaf_tov_mx	pldb%ext_coef, pldb%leaf_tov_min, pldb%leaf_tov_max
rt_st_beg, rt_st_end	pldb%rsr1, pldb%rsr2
n_uptake, p_uptake, n_fix_max (parameters.bsn)	bsn_prm%n_updis, bsn_prm%p_updis, bsn_prm%nfixmx
harv_typ, harv_idx, harv_eff, harv_bm_min (harv.ops)	harvop_db%typ, %hi_ovr, %eff, %bm_min

Symbol	Fortran variable
fr_{PHU}, multi-year fraction	`pcom(j)%plcur(ipl)%phuacc`, `%phuacc_p`
PHU	`pcom(j)%plcur(ipl)%phumat`, `%phumat_p`
H_{phosyn}	`par(ipl)`
Δbio, γ_{reg}	`bioday`, `pcom(j)%plstr(ipl)%reg`
γ_w, γ_t, γ_n, γ_p, γ_a, γ_s	`pcom(j)%plstr(ipl)%strsw`, `%strst`, `%strsn`, `%strsp`, `%strsa`, `%strss`
E_t per plant	`epmax(ipl)`
β_w	`uptake%water_dis`
HI	`pcom(j)%plg(ipl)%hi_adj`

Overview

SWAT+ follows nitrogen and phosphorus through mineral, organic and plant pools in every soil layer. Fertiliser, manure, residue and atmospheric deposition add nutrients; plants take them up; and runoff, sediment, lateral flow, tile flow, percolation, denitrification and volatilisation remove them. The nutrients that leave the HRU are routed through channels, where in-stream processes are described on the Channels page.

The soil processes below are those of the static carbon option (`carbon = 0` in `codes.bsn`). With `carbon = 2` the residue and humus pools and their decomposition follow the CENTURY-based model on the Carbon page, while nitrification, volatilisation, mineral phosphorus, deposition and all the transport processes below stay the same. `carbon = 1` is reserved for C-FARM, which is not implemented; with it SWAT+ 62.0.1 skips residue decomposition and mineralisation altogether.

Plant uptake and nitrogen fixation are described in Plant growth and land use management.

Soil pools

Element	Pool	Contents
N	Nitrate	NO3-N in solution, mobile with water
N	Ammonium	NH4-N, adsorbed on the soil
N	Active humus N	The part of humus organic N that mineralises
N	Stable humus N	The rest of humus organic N
N	Residue N	Organic N in fresh residue, kept separately for each plant in each layer and on the surface
P	Labile (solution) P	Soluble mineral P available to plants and mobile with water
P	Active mineral P	Mineral P in rapid equilibrium with labile P
P	Stable mineral P	Mineral P in slow equilibrium with active P
P	Humus organic P	Organic P in humus
P	Residue P	Organic P in fresh residue

Manure applied to the soil has separate organic pools, which are also lost with sediment.

Initial values

The initial nutrient contents come from the row of `nutrients.sol` named for the HRU in `soil_plant.ini`, with built-in values when there is none:

Pool	Initial value
Nitrate	`nitrate` (7 mg/kg) at the surface, decreasing with depth as $\exp(-\text{exp_co}\, z)$, with z the depth to the bottom of the layer in mm
Labile P	`lab_p` (5 mg/kg) at the surface, decreasing with depth the same way
Active mineral P	Labile P $\times$ (1 − PSP)/PSP
Stable mineral P	4 $\times$ active P; with the Vadas and White P model, from the White et al. (2009) ratio of 1 to 7 times active plus labile P

Pool	Initial value
Humus C, N and P	Humus C from the organic carbon of the layer (`carbon` in `soils.sol`); N and P from the humus C:N and C:P ratios `hum_c_n` (10) and `hum_c_p` (80)
Active and stable humus	`fr_hum_act` (0.02) of humus in the active pool

Concentrations in mg/kg are converted to kg/ha with the bulk density and thickness of each layer. The other soil test columns of `nutrients.sol` (`inorgp`, `watersol_p`, `h3a_p`, `mehlich_p`, `bray_strong_p`) are not used.

Temperature and water factors

Mineralisation, decomposition and denitrification depend on soil temperature and water through two factors:

$$\gamma_{tmp} = \max\left(0.1,\ 0.9\,\frac{T_{soil}}{T_{soil} + \exp(9.93 - 0.312\,T_{soil})} + 0.1\right), \qquad \gamma_{sw} = \max\left(0.05,\ 0.1 + 0.9\sqrt{\frac{SW_{ly}}{FC_{ly}}}\right)$$

where T_{soil} is the layer temperature (°C), SW_{ly} the layer water and FC_{ly} its field capacity water. They combine as $\gamma = \sqrt{\gamma_{tmp}\,\gamma_{sw}}$. None of these processes runs in a layer at or below 0 °C. For the 10 mm surface layer, SWAT+ uses the temperature and water content of the second layer.

Humus mineralisation

Nitrogen moves between the active and stable humus pools towards a fixed active fraction:

$$N_{trns} = 10^{-5}\left[orgN_{act}\left(\frac{1}{fr_{actN}} - 1\right) - orgN_{sta}\right]$$

where fr_{actN} is `n_act_frac` in `carbon.bsn` (0.02). A positive value moves N from active to stable humus.

Active humus N mineralises to nitrate:

$$N_{mina} = \beta_{min}\,\gamma\,orgN_{act}$$

with β_{min} the rate coefficient for humus mineralisation, `orgn_min` in `parameters.bsn`. Phosphorus is mineralised from humus in proportion:

$$P_{mina} = 1.4\,N_{mina}\,\frac{orgP_{hum}}{orgN_{act} + orgN_{sta}}$$

and added to the labile pool.

Residue decomposition

Fresh residue decomposes at a rate limited by its C:N and C:P ratios. For each plant's residue in each layer,

$$\gamma_{ntr} = \min\left(\exp\left[-0.693\,\frac{\varepsilon_{C:N} - 25}{25}\right],\ \exp\left[-0.693\,\frac{\varepsilon_{C:P} - 200}{200}\right],\ 1\right)$$

with $\varepsilon_{C:N}$ capped at 500 and $\varepsilon_{C:P}$ at 5000. The fraction decomposed on the day is

$$\delta_{ntr} = \min\left[1,\ \max\left(\text{rsd_decay},\ \beta_{rsd}\,\gamma_{ntr}\,\gamma\right)\right]$$

where β_{rsd} is the residue decomposition coefficient of the plant (`plnt_decomp` in plants.plt) and `rsd_decay` in parameters.bsn is a minimum daily rate. Of the N and P released, 80% goes to nitrate and labile P, and 20% to humus (active humus N; stable humus P for soil residue, active humus P for surface residue).

Surface residue decomposes the same way, using the temperature and water of the top layer.

Nitrification and ammonia volatilisation

Ammonium is nitrified and volatilised together in each layer (Reddy et al., 1979; Godwin et al., 1984). The temperature factor is

$$\eta_{tmp} = 0.41\,\frac{T_{soil} - 5}{10}$$

and both processes need $\eta_{tmp} \geq 0.001$. The water factor for nitrification rises linearly from 0 at wilting point to 1 at wilting point plus 25% of the field capacity water. Volatilisation decreases with depth through

$$\eta_{midz} = 1 - \frac{z_{mid}}{z_{mid} + \exp(4.706 - 0.0305\,z_{mid})}$$

with z_{mid} the depth to the middle of the layer (mm), and uses a cation exchange factor of 0.15. With $\eta_{nit} = \eta_{tmp}\,\eta_{sw}$ and $\eta_{vol} = 0.15\,\eta_{tmp}\,\eta_{midz}$, the ammonium lost to both is

$$N_{nit|vol} = NH_4\,[1 - \exp(-\eta_{nit} - \eta_{vol})]$$

and is split between nitrification and volatilisation in proportion to $1 - \exp(-\eta_{nit})$ and $1 - \exp(-\eta_{vol})$.

Denitrification

When the water factor reaches a threshold, nitrate in the layer is denitrified:

$$N_{denit} = NO_3\left[1 - \exp\left(-\beta_{denit}\,\gamma_{tmp}\,\frac{orgC}{100}\right)\right] \quad \text{for } \gamma_{sw} \geq \gamma_{sw,thr}$$

Symbol	Meaning	Input
β_{denit}	Denitrification rate coefficient	`denit_exp` in parameters.bsn
$\gamma_{sw,thr}$	Threshold of the water factor	`denit_frac` in parameters.bsn
$orgC$	Organic carbon of the layer	`carbon` in soils.sol, %

The septic biozone layer of an active septic system is excluded.

Atmospheric deposition

Nitrate and ammonium in rain and dry deposition are added to the surface layer every day from the station named in the `atmo_dep` column of weather-sta.cli. Rain deposition is the concentration in rain (mg/L) times the day's precipitation; dry deposition is spread evenly over the days of the month or year, depending on whether atmodep.cli gives monthly, yearly or average annual values.

Nitrate movement

Nitrate is mobile and moves with water. In each layer the water that moves is the percolation, lateral flow, surface runoff (top layer only) and tile flow (tile layer only), w_{mobile}. The nitrate concentration in the mobile water is

$$conc_{NO_3} = \frac{NO_3 \left[1 - \exp\left(\frac{-w_{mobile}}{(1-\theta_e)\, SAT_{ly}}\right)\right]}{w_{mobile}}$$

where θ_e is the fraction of porosity from which anions are excluded (`anion_excl` in `soils.sol`) and SAT_{ly} the saturation water of the layer. Then:

Path	Nitrate removed
Surface runoff	$\beta_{NO_3}\, conc_{NO_3}\, Q_{surf}$, top layer, with β_{NO_3} = `n_perc` in `parameters.bsn`
Lateral flow	$conc_{NO_3}\, Q_{lat,ly}$ for each layer
Tile flow	`nperco_lchtile` × concentration in the tile flow × Q_{tile}, with the concentration taken over the tile layer and all layers below
Percolation	$conc_{NO_3}\, w_{perc,ly}$, added to the next layer; from the bottom layer it leaves the profile

`n_perc` close to 0 gives little nitrate in surface runoff; close to 1 gives runoff the same concentration as percolate. As water evaporates from the second layer, a small amount of nitrate moves up with it into the surface layer.

Nitrate in lateral and tile flow passes through the same lag stores as the water (see Hydrology). When the gwflow module passes groundwater into the soil profile, it also adds the nitrate and phosphorus that the groundwater contains.

Organic N and P with sediment

Organic nitrogen leaving with sediment is

$$orgN_{surf} = 1000\, \frac{orgN_{ly1}}{M_{ly1}}\, \varepsilon_N\, \frac{sed}{A}$$

Symbol	Meaning	Units
$orgN_{surf}$	Organic N removed in sediment	kg N/ha
$orgN_{ly1}$	Active plus stable humus N in the surface layer	kg N/ha
M_{ly1}	Mass of the surface layer, $10000\, \rho_b\, depth$	kg/ha
ε_N	Nitrogen enrichment ratio	-
sed	Sediment yield	metric tons
A	HRU area	ha

Sediment-attached phosphorus is computed the same way from the humus, manure and active and stable mineral P of the surface layer, using ε_P, and is reported as organic P and mineral P.

The enrichment ratio accounts for the finer, nutrient-rich particles that erosion removes. It is `orgn_enrich` (N) or `orgp_enrich` (P) in `hydrology.hyd` when that is above 0.001. Otherwise it is computed from the sediment concentration in runoff (Menzel, 1980):

$$\varepsilon = 0.78\, (conc_{sed})^{-0.2468}, \qquad conc_{sed} = \frac{sed}{10\, A\, Q_{surf}}, \qquad \varepsilon \leq 3$$

Soluble phosphorus

Labile phosphorus is strongly bound by the soil, so little of it moves with water. The amount removed in surface runoff is

$$P_{surf} = \frac{P_{lab,1}\, Q_{surf}}{\rho_b\, depth_1\, k_{d,surf} + 1}$$

where $P_{lab,1}$ is the labile P of the surface layer (kg/ha), ρ_b and $depth_1$ its bulk density and thickness (mm), and $k_{d,surf}$ the phosphorus soil partitioning coefficient, `p_soil` in `parameters.bsn` (m^3/Mg).

Leaching of labile P from each layer to the next is

$$P_{perc} = 0.001\, P_{lab} \left[1 - \exp\left(\frac{-w_{perc}}{0.01\, SW_{ly} + 0.1\, k_{d,perc}\, \rho_b}\right)\right]$$

with $k_{d,perc}$ the phosphorus percolation coefficient `p_perc` in `parameters.bsn`. The same expression with tile flow in place of percolation gives the P in tile flow from the tile layer. This form differs from the SWAT 2009 leaching equation.

Mineral phosphorus transformations

Original model

With `soil_p = 0` in `codes.bsn`, labile and active mineral P move towards the ratio set by the phosphorus availability index pai (`p_avail` in `parameters.bsn`):

$$P_{sol|act} = P_{lab} - P_{act}\, \frac{pai}{1 - pai}$$

Movement from labile to active P takes 10% of the imbalance per day; movement back takes 60%. Active and stable P move towards a stable pool four times the active pool:

$$P_{act|sta} = 0.01\, (4\, P_{act} - P_{sta})$$

with a tenth of that rate when stable P returns to the active pool.

Vadas and White model

With `soil_p = 1`, SWAT+ uses the phosphorus model of Vadas and White (2010). The availability index is computed for each layer from clay content, labile P and organic carbon, and averaged over 30 days. The rate of movement between labile and active P depends on how many days the imbalance has lasted. The active to stable equilibrium ratio follows the White et al. (2009) relationship, and the active to stable rate depends on the calcium carbonate content (`caco3` in `soils.sol`). Both rates are scaled by the water content of the layer relative to field capacity, with a lower limit of 0.25.

Lag of nutrients in surface runoff

Nitrate and labile P in surface runoff, organic N and P, and mineral P attached to sediment pass through the surface lag store with the runoff and sediment, and the same daily fraction is released (see Hydrology).

Algae, CBOD and dissolved oxygen in runoff

Each day SWAT+ also gives the water leaving an HRU a chlorophyll-a, carbonaceous BOD and dissolved oxygen load, which the channels then use as inflow concentrations. On days without water yield all three are zero. With water yield Q (surface runoff, lateral flow and tile flow, mm):

- Chlorophyll-a is 0.1 times the concentration of organic N in sediment plus nitrate in surface runoff, $100\,(N_{org,sed} + NO_{3,surq})/Q$.
- CBOD is 2.7 times the organic carbon in the day's sediment yield, divided by the water yield. The carbon is the soil carbon content of the top layer times the sediment yield and the enrichment ratio, or, with `carbon` = 2 in `codes.bsn`, the carbon the carbon model reports in the sediment.
- Dissolved oxygen is $O_{sat}\, e^{-0.1\,CBOD}$, limited to between 0 and O_{sat}, with O_{sat} the saturation concentration at a water temperature of $5 + 0.75\,\bar{T}_{air}$ (see Channels for the formula).

Septic systems

A land use with a septic system (the `sep` column of `landuse.lum`) adds a biozone layer to the soil, and the biozone routine runs on days when that layer is above freezing. In 62.0.1 the effluent volume is never computed, so septic systems add no water, nitrogen, phosphorus or BOD to the soil. See `septic.str` for the inputs.

In-stream processes

Nutrients entering channels are transformed by the QUAL2E water quality routines. With `i_fpwet = 1` in `codes.bsn` (the column read as the QUAL2E switch), a simplified exchange between dissolved and particulate N and P is added. See Channels.

Switches and parameters

Input	File	Effect
`carbon`	`codes.bsn`	`0` static pools, `2` CENTURY-based carbon, `1` reserved
`soil_p`	`codes.bsn`	`0` original mineral P model, `1` Vadas and White (2010)
`orgn_min`	`parameters.bsn`	Humus mineralisation rate coefficient
`rsd_decay`	`parameters.bsn`	Minimum daily residue decomposition
`denit_exp`, `denit_frac`	`parameters.bsn`	Denitrification rate and water threshold
`n_perc`	`parameters.bsn`	Nitrate in surface runoff relative to percolate
`nperco_lchtile`	`parameters.bsn`	Nitrate concentration coefficient for tile flow
`p_soil`, `p_perc`	`parameters.bsn`	Phosphorus partitioning in runoff and percolation
`p_avail`	`parameters.bsn`	Phosphorus availability index (original model)
`n_act_frac`	`carbon.bsn`	Active fraction of humus N
`orgn_enrich`, `orgp_enrich`	`hydrology.hyd`	Fixed enrichment ratios
`plnt_decomp`	`plants.plt`	Residue decomposition coefficient of each plant
`nitrate`, `lab_p`, `exp_co`, `fr_hum_act`, `hum_c_n`, `hum_c_p`	`nutrients.sol`	Initial nutrient contents
`carbon`, `anion_excl`, `caco3`	`soils.sol`	Organic carbon, anion exclusion, CaCO3

`orgn_min`, `n_perc`, `nperco_lchtile`, `p_soil`, `p_perc` and `p_avail` are copied to every HRU at the start of the run, so calibration can change them for single HRUs.

Related pages

- Plant growth and land use management for uptake and fixation.
- Carbon for the CENTURY-based pools.
- Erosion and sediment for the sediment yield that transports organic nutrients.

- fertilizer.frt and manure allocation for nutrient inputs.
- Channels for in-stream processes.

Sources

- Daily sequence and carbon switch: hru_control.f90
- Initial pools: soil_nutcarb_init.f90; `type soiltest_db` in soil_data_module.f90
- Mineralisation, soil residue decomposition, denitrification: nut_nminrl.f90; surface residue: rsd_decomp.f90
- Nitrification and volatilisation: nut_nitvol.f90
- Deposition: nut_nrain.f90
- Nitrate movement: nut_nlch.f90; upward nitrate with evaporation in et_act.f90
- Sediment-bound nutrients: nut_orgn.f90, nut_psed.f90; enrichment ratio: pest_enrsb.f90
- Soluble P: nut_solp.f90; mineral P: nut_pminrl.f90, nut_pminrl2.f90
- Chlorophyll-a, CBOD and dissolved oxygen in HRU runoff: swr_subwq.f90 (`chl_a`, `cbodu`, `doxq`; the code labels the organic N plus nitrate concentration `tp`), passed to the HRU outflow in hru_hyds.f90
- Septic biozone: sep_biozone.f90, called from `hru_control.f90`; set up in `soils_init.f90`
- Lag stores: swr_substor.f90, stor_surfstor.f90
- HRU copies of basin nutrient parameters: hrudb_init.f90
- `n_act_frac`: carbon_module.f90

Column or symbol	Fortran variable
`carbon`, `soil_p`, `i_fpwet` (codes.bsn)	`bsn_cc%cswat`, `bsn_cc%sol_p_model`, `bsn_cc%qual2e`
`orgn_min` (β_{min})	`bsn_prm%cmn`, per HRU `hru(j)%nut%cmn`
`rsd_decay`	`bsn_prm%decr_min`
`denit_exp`, `denit_frac`	`bsn_prm%cdn`, `bsn_prm%sdnco`
`n_perc`, `nperco_lchtile`	`hru(j)%nut%nperco`, `hru(j)%nut%nperco_lchtile`
`p_soil`, `p_perc`, `p_avail`	`hru(j)%nut%phoskd`, `hru(j)%nut%pperco`, `hru(j)%nut%psp`
`orgn_enrich`, `orgp_enrich`	`hru(j)%hyd%erorgn`, `hru(j)%hyd%erorgp`
`plnt_decomp`	`pldb%rsdco_pl`
Nitrate, ammonium	`soil1(j)%mn(ly)%no3`, `%nh4`
Active and stable humus	`soil1(j)%hact(ly)`, `soil1(j)%hsta(ly)`
Labile, active, stable mineral P	`soil1(j)%mp(ly)%lab`, `%act`, `%sta`
Residue in soil, on surface	`soil1(j)%pl(ipl)%rsd(ly)`, `pl_mass(j)%rsd(ipl)`
γ_{tmp}, γ_{sw}, γ	`cdg`, `sut`, `csf`
ε (computed)	`enratio`
Nitrate in runoff, lateral, tile, percolation	`surqno3(j)`, `latno3(j)`, `tileno3(j)`, `percn(j)`
Sediment-bound N and P	`sedorgn(j)`, `sedorgp(j)`, `sedminpa(j)`, `sedminps(j)`
Soluble P in runoff	`surqsolp(j)`

Overview

The `carbon` column of codes.bsn selects how SWAT+ handles soil organic matter:

`carbon`	Model
0	Static model. Residue decomposes and organic N and P mineralise as in SWAT; soil carbon is not tracked as a state. See Nutrient cycling.
1	Reserved for C-FARM, which is not implemented.
2	Carbon model based on CENTURY (Parton et al., 1993, 1994) as implemented in SWAT-C by Zhang (2018). Carbon, nitrogen and phosphorus move through litter and soil organic matter pools in every soil layer.

The carbon model is switched on by `carbon = 2`; earlier revisions used 1. With `carbon = 1` neither model runs, so residue does not decompose and organic N and P are not mineralised. Set the column to 0 or 2.

When `carbon = 2`, the files `carbon.bsn` (basin-wide settings) and `carbon_lyr.bsn` (rates and CO_2 fractions) are required, and the run stops if either is missing. The legacy `carb_coefs.cbn` file is no longer read.

Pools

Each soil layer has three litter pools and three soil organic matter pools. Every pool contains mass, carbon, nitrogen and phosphorus.

Pool	Content
Metabolic litter	Easily decomposed part of residue and manure
Structural litter	Resistant part of residue, of which a share is lignin
Lignin	Lignin within structural litter
Microbial biomass	Active microbes and their products
Slow humus	Organic matter with turnover times of years to decades
Passive humus	Stabilised organic matter with turnover times of centuries

Surface residue is kept separately for each plant until it decomposes into the litter pools of the top soil layer.

Carbon moves between pools as follows. Each transfer releases part of the carbon as CO_2.

- structural litter (non-lignin part) and metabolic litter to microbial biomass
- lignin to slow humus (70 %; the other 30 % is respired)
- microbial biomass to slow humus and passive humus
- slow humus to microbial biomass and passive humus
- passive humus to microbial biomass

Residue inputs

Surface residue

Each plant's surface residue decomposes when the top soil layer is above 0 °C:

$$\Delta R = \delta\, R, \qquad \delta = \min\left(1,\ \max\left(\delta_{min},\ k_{rsd}\ \gamma_{nut}\ \sqrt{\gamma_{tmp}\,\gamma_{sw}}\right)\right)$$

$$\gamma_{tmp} = \max\left(0.1,\ 0.9\,\frac{T}{T + e^{9.93 - 0.312\,T}} + 0.1\right), \qquad \gamma_{sw} = \max\left(0.05,\ 0.1 + 0.9\sqrt{SW/FC}\right)$$

$$\gamma_{nut} = \min\left(1,\ e^{-0.693\,(CN - CN_{ref})/CN_{ref}},\ e^{-0.693\,(CP - CP_{ref})/CP_{ref}}\right)$$

Symbol	Meaning	Input
R	surface residue of a plant (mass, C, N, P)	
k_{rsd}	residue decomposition coefficient of the plant	`plnt_decomp` in `plants.plt`
δ_{min}	minimum daily decomposition fraction	`rsd_decay` in `parameters.bsn`
T, SW, FC	top layer temperature (°C), soil water and field capacity (mm)	
CN, CP	residue C:N and C:P ratios, capped at `cnr_cap` and `cpr_cap`	`carbon.bsn`
CN_{ref}, CP_{ref}	ratios at which the factor is 1	`cnr_ref`, `cpr_ref` in `carbon.bsn`

The decomposed residue is added to the litter pools of the top soil layer. The plant's above-ground lignin fraction (`ab_lig_frac` in `plants.plt`) goes to lignin, the lignin fraction divided by 0.8 goes to structural litter (lignin included), and the rest goes to metabolic litter.

Before decomposition, a fraction `sfc_rsd_photodeg` (`carbon.bsn`) of each plant's surface residue is lost each day to photodegradation and counted as emitted carbon.

Roots and incorporated residue

Residue in the soil (dead roots and residue mixed in by tillage) is moved to the litter pools of its layer on each day the layer is above 0 °C, using the plant's below-ground lignin fraction (`bg_lig_frac` in `plants.plt`) in the same way.

Manure adds to the litter pools according to the manure partitioning settings in `carbon.bsn`.

Pool transformations

Environmental controls

The daily rate of every transformation in a layer is the optimal rate times a combined control factor:

$$c_s = \min\left(15,\ 0.9\sqrt{f_{tmp}\, f_{sw}}\ f_{ox}\ f_{till}\right)$$

Temperature f_{tmp} is chosen with `tmpf_eqn` in `carbon.bsn`:

`tmpf_eqn`	Function of layer temperature T (°C)
1	$T/(T + e^{5.058 - 0.2504T})$ up to 35 °C, then $1 - 0.04(T - 35)$ up to 60 °C, then 0
2	$\dfrac{(T - T_n)^q\,(T_x - T)}{(T_o - T_n)^q\,(T_x - T_o)}$ with $q = (T_n - T_o)/(T_o - T_x)$, and 0 outside $T_n < T < T_x$; T_n, T_o, T_x are `t_cbn_min`, `t_cbn_opt`, `t_cbn_max`
3	$0.9\,T/(T + e^{9.93 - 0.312T}) + 0.1$

Soil water f_{sw} is chosen with `watf_eqn`:

`watf_eqn`	Function
1	$0.1 + 0.9\sqrt{SW/FC}$, limited to 0.05 to 1
2	Product of a water factor (rising from 0 at no water to 1 at field capacity) and an aeration factor based on the air-filled pore space; reduces activity in both dry and waterlogged soil

Oxygen decreases with the depth z (mm) of the middle of the layer:

$$f_{ox} = 1 - \frac{0.8\,z}{z + e^{18.40961 - 0.023683632\,z}}$$

Tillage f_{till} depends on `idc_till` in `codes.bsn`:

`idc_till`	Effect
1	$f_{till} = 1.6$ in layers within the tillage depth for `till_eff_days` days after tillage (partly in the layer containing the tillage depth), otherwise 1
2, 4	No effect ($f_{till} = 1$)
3 (default)	f_{till} is 1 plus the layer tillage factor, which rises with each tillage operation and decays afterwards (Kemanian method)

Rates

Potential daily transformation of each pool is its carbon (and nitrogen) content times an optimal rate from `carbon_lyr.bsn`, times c_s:

Pool	Daily fraction transformed	Rate column
Structural litter	$k_{str}\, c_s\, e^{-3\,L}$, where L is the lignin share of structural litter (at most 0.8)	`str_rate`
Metabolic litter	$k_{meta}\, c_s$	`meta_rate`
Microbial biomass	$k_{mic}\, c_s\, x_{bm}$, with $x_{bm} = 1$ in the top layer and $0.25 + 0.0075\,\text{sand}$ below	`microb_top_rate` (top layer), `microb_rate`
Slow humus	$k_{hs}\, c_s$	`hs_rate`
Passive humus	$k_{hp}\, c_s$	`hp_rate`

`carbon_lyr.bsn` has two rows: row 1 applies to the top soil layer and row 2 to all other layers.

Allocation and CO_2

Transfer	Fraction respired as CO_2	Remainder goes to
Metabolic and non-lignin structural litter	`a1co2`	microbial biomass
Lignin	0.3	slow humus
Microbial biomass	`abco2` in the top layer; $0.17 + 0.0068\,\text{sand}$ below	passive humus ($0.003 + 0.00032\,\text{clay}$), rest to slow humus
Slow humus	`asco2`	passive humus (the larger of 0.001 and `hs_hp` minus $0.00009 \times$ clay), rest to microbial biomass
Passive humus	`apco2`	microbial biomass

Sand and clay are in percent.

Nitrogen

Each receiving pool has a target N:C ratio. In the top layer these depend on the N content of the residue; in deeper layers they depend on the mineral N concentration of the layer. When the N released by the decomposing pools is less than the N needed to build the receiving pools, the difference is immobilised from soil nitrate and ammonium. If mineral N cannot cover it, the transformations that need N are reduced in proportion. Surplus N is mineralised to ammonium.

Before the transformations, denitrification removes nitrate when the soil water factor exceeds `denit_frac` in `parameters.bsn`, at a rate set by `denit_exp`, the temperature factor and the layer organic carbon.

Phosphorus

Phosphorus is mineralised from passive humus at 1.4 times the fraction of humus N that was transformed on the day, and goes to the labile mineral pool. Phosphorus in decomposing litter is released in proportion to the litter carbon transformed: 80 % to labile mineral P and 20 % to passive humus.

Carbon in water and sediment

- Sediment: eroded soil removes organic carbon from the top layer in proportion to the share of the layer mass eroded (at most 90 %).
- Dissolved carbon from microbial biomass: in the top layer, microbial carbon is lost to water leaving the layer with $1 - \exp\left(-w/(SAT + 10^{-4}\, K_{oc}\, C)\right)$, where w is runoff plus lateral flow plus percolation, K_{oc} is `koc_c` and C the layer organic carbon. The concentration in surface runoff and lateral flow is `solc_ratio` times the concentration in percolation. In deeper layers, carbon moving with percolation and lateral flow is removed with $1 - \exp(-w/(\phi z - WP))$.
- Carbon leaving the bottom of the profile is reported as percolated carbon.

Initial pools

At the start of the run, the organic carbon of each layer (from soils.sol) is split into pools. A share `init_seq` is humus and microbial carbon, divided between microbial biomass (`init_microb`), slow humus (`init_slow`) and passive humus (`init_passive`). Humus pools start with C:N of 10 and C:P of 80; microbial biomass with C:N of 8 and C:P of 80. With `mathers_method = 1`, slow humus is initialised with the Mathers relation based on clay plus silt content.

Parameters

Parameter	File	Effect
`carbon`	codes.bsn	0 = static, 1 = reserved, 2 = carbon model
`idc_till`	codes.bsn	Tillage effect on decomposition (default 3)
`tmpf_eqn`, `watf_eqn`, `t_cbn_min`, `t_cbn_opt`, `t_cbn_max`	carbon.bsn	Temperature and soil water controls
`till_eff_days`, `bmix_*`, `tillmix_*`, `bio_consol`, `till_consol`	carbon.bsn	Duration and shape of tillage and biological mixing effects
`cnr_cap`, `cnr_ref`, `cpr_cap`, `cpr_ref`, `sfc_rsd_photodeg`	carbon.bsn	Surface residue decomposition
`koc_c`, `solc_ratio`	carbon.bsn	Dissolved carbon losses
`init_seq`, `init_microb`, `init_slow`, `init_passive`, `mathers_method`	carbon.bsn	Initial pool split
`hp_rate`, `hs_rate`, `microb_rate`, `microb_top_rate`, `meta_rate`, `str_rate`	carbon_lyr.bsn	Optimal transformation rates (1/day)
`hs_hp`, `a1co2`, `asco2`, `apco2`, `abco2`	carbon_lyr.bsn	Allocation and CO_2 fractions
`plnt_decomp`, `ab_lig_frac`, `bg_lig_frac`	plants.plt	Residue decomposition rate and lignin fractions

Carbon model parameters can be calibrated through the parameter change files; see Parameter change files.

Related pages

- carbon.bsn and carbon_lyr.bsn
- Carbon outputs
- Nutrient cycling for the static model
- Plant growth and land use management

Sources

Code at revision 62.0.1:

- Model switch: `cswat` in src/basin_module.f90; daily calls in src/hru_control.f90
- Inputs: src/carbon_bsn_read.f90 (reads `carbon.bsn` and `carbon_lyr.bsn`), types in src/carbon_module.f90
- Pool transformations: src/cbn_zhang2.f90; temperature option 2: src/fcgd.f90
- Residue: src/cbn_surfrsd_decomp.f90, src/cbn_rsd_transfer.f90; lignin partitioning in src/plant_parm_read.f90
- Carbon in runoff, sediment and percolation: src/nut_orgnc2.f90
- Initial pools: src/soil_nutcarb_init.f90
- Tillage: src/mgt_newtillmix_cswat1.f90, src/mgt_tillfactor.f90, src/mgt_biomix.f90

Symbols and variables:

Symbol or column	Variable
`carbon` in `codes.bsn`	`bsn_cc%cswat`
c_s, f_{tmp}, f_{sw}, f_{ox}, f_{till}	`org_con%cs`, `%cdg`, `%sut`, `%ox`, `%till_eff`
`tmpf_eqn`, `watf_eqn`, `t_cbn_min/opt/max`	`org_con%tmpf`, `%watf`, `%tn`, `%top`, `%tx`
`koc_c`, `solc_ratio`	`cb_wtr_coef%prmt_21`, `cb_wtr_coef%prmt_44`

Symbol or column	Variable
`init_seq`, `init_microb`, `init_slow`, `init_passive`	`org_frac%frac_seq`, `%frac_hum_microb`, `%frac_hum_slow`, `%frac_hum_passive`
rates in `carbon_lyr.bsn`	`carbdb(1:2)%hp_rate`, `%hs_rate`, `%microb_rate`, `%microb_top_rate`, `%meta_rate`, `%str_rate`, `%hs_hp`
CO_2 fractions	`org_allo(1:2)%a1co2`, `%asco2`, `%apco2`, `%abco2`
microbial to passive, slow to passive	`org_allo%abp`, `org_allo%asp`
pools	`soil1%meta`, `%str`, `%lig`, `%microb`, `%hs`, `%hp`
lignin fractions of residue	`cswat_1_part_fracs%lig_frac_abg`, `%lig_frac_blg` (from `pldb%res_part_fracs%str_frac` and `%lig_frac`)

Overview

SWAT+ simulates any number of pesticides, listed in the constituent setup and described in `pesticide.pes`. Each pesticide has a sorption coefficient, a solubility, half-lives on foliage, in soil, in water and in benthic sediment, and optional degradation products (metabolites). On land, a pesticide is applied by a management operation, split between plant foliage and the soil, washed from foliage by rain, decayed, taken up by plants, and moved by surface runoff, lateral flow, tile flow, percolation and eroded sediment. Pesticide that reaches channels, reservoirs and aquifers continues to react and move there.

The daily order in an HRU is washoff (on days with at least 2.54 mm of rain), plant uptake, decay, transport in water, then transport with sediment (on days with erosion).

Application

A pesticide application operation names the pesticide, the amount and an application method from `chem_app.ops`. The amount reaching the field is the applied amount multiplied by the application efficiency of the method:

$$P_{app} = P_{op}\,\varepsilon_{app}$$

The fraction intercepted by the canopy depends on the leaf area index of all plants in the HRU:

$$f_{gc} = \frac{1.99532 - \operatorname{erfc}(1.333\,LAI - 2)}{2.1}, \qquad f_{gc} \geq 0$$

Symbol	Meaning	Units	Input
P_{app}	pesticide reaching the field	kg/ha	
P_{op}	amount in the operation	kg/ha	application operation
ε_{app}	application efficiency	fraction	`app_eff` in `chem_app.ops`
f_{gc}	fraction intercepted by foliage	fraction	
LAI	sum of leaf area index of all plants	m^2/m^2	

The intercepted part, $f_{gc}P_{app}$, is divided among the plants in proportion to their leaf area. The rest reaches the soil: the surface fraction `surf_frac` of the application method goes to the top soil layer (10 mm) and the remainder to the second layer.

Washoff

On days with at least 2.54 mm of precipitation, a fixed fraction of the pesticide on each plant's foliage is washed to the top soil layer:

$$P_{wash} = f_{wash}\,P_{fol}$$

where f_{wash} is the washoff fraction `frac_wash` of the pesticide and P_{fol} the pesticide on the foliage (kg/ha).

Decay

Pesticide on foliage and in each soil layer decays by first-order kinetics. The daily survival factor is computed from the half-life:

$$P_i = P_{i-1} \exp\left(\frac{-0.693}{t_{1/2}}\right)$$

with $t_{1/2}$ the foliar half-life `hl_foliage` for foliage and the soil half-life `hl_soil` for soil (days). A half-life of 0 removes all of the pesticide in one day.

Temperature adjustment

When the temperature switch for pesticide decay in `parameters.bsn` is 1, the half-lives are treated as values at 25 °C and adjusted each day with a Q10 relation:

$$t_{1/2}(T) = t_{1/2,25}\, Q_{10}^{(25-T)/10}, \qquad Q_{10} = 2.58$$

Pool	Half-life used	Temperature T
Soil layer	`hl_soil`	temperature of that layer
Foliage	`hl_foliage`	temperature of the top soil layer
Channel water	`aq_hlife`	channel water temperature (at least 0.1 °C)
Channel benthic sediment	`ben_hlife`	channel water temperature
Aquifer	`aq_hlife`	0 °C; the aquifer temperature is not set in 62.0.1

Each 10 °C below 25 °C multiplies the half-life by 2.58, and each 10 °C above 25 °C divides it by 2.58. Reservoirs always use the unadjusted half-lives.

The temperature switch and the pesticide groundwater mixing factor are read as the 6th and 7th values of `parameters.bsn`. Files written by the SWAT+ Editor label these columns `lin_sed` and `exp_sed`, which older revisions did not use. A file with `exp_sed = 1` therefore turns temperature-adjusted decay on, and the `lin_sed` value becomes the groundwater mixing factor. Check both columns before running pesticides with an older `parameters.bsn`.

Metabolites

When a pesticide has degradation products, a share of the mass that decays is added to each daughter pesticide:

$$\Delta P_d = \Delta P_{parent}\, f_d\, \frac{M_d}{M_{parent}}$$

Symbol	Meaning	Units
ΔP_d	mass added to the daughter	kg/ha
ΔP_{parent}	parent mass decayed on the day	kg/ha
f_d	fraction of the parent that degrades to this daughter, set separately for foliage, soil, water and benthic sediment	fraction
M	molecular weight `mol_wt`	g/mol

Metabolites form on foliage, in soil, in channel water and benthic sediment, and in aquifers.

Plant uptake

Growing plants take up dissolved pesticide with the water they extract from each soil layer. Uptake from a layer is

$$P_{up} = f_{TSCF}\ \frac{w_{up}}{SW_{eff} + w_{up} + K_d\,\rho_b\,z}\ P_{ly}$$

$$SW_{eff} = SW + n\ \max(0,\ FC - SW)$$

Symbol	Meaning	Units	Input
P_{up}	pesticide taken up from the layer	kg/ha	
f_{TSCF}	transpiration stream concentration factor	-	`pl_uptake`
w_{up}	plant water uptake from the layer	mm	
SW, FC	layer soil water and field capacity	mm	
n	resistance blending factor, 0 uses actual soil water and 1 uses field capacity	0 to 1	`resistance_n` (default 1)
K_d	soil partition coefficient, $K_{oc} \cdot OC/100$	mL/g	`soil_ads`
ρ_b, z	bulk density and layer thickness	Mg/m³, mm	
P_{ly}	pesticide in the layer	kg/ha	

When `harvest_sink` is 1 (default), pesticide taken up is lost from the system. When it is 0, it is stored in the plant, removed with the harvested yield, and returned to the soil surface when the plant is killed.

Transport in water

In each soil layer, pesticide is shared between the soil water and sorbed phases with a linear partition coefficient based on the organic carbon content of the layer:

$$K_d = \frac{K_{oc}\ OC}{100}$$

where K_{oc} is `soil_ads` (mL/g) and OC the organic carbon content of the layer (%). The concentration of pesticide in water leaving the layer is

$$C = \min\left(\frac{P_{ly}\left[1 - \exp\left(\dfrac{-w}{SAT + K_d\,\rho_b\,z}\right)\right]}{w},\ \frac{S_w}{100}\right)$$

Symbol	Meaning	Units
C	pesticide concentration in mobile water	kg/ha per mm
w	water leaving the layer: percolation and lateral flow, plus surface runoff for the top layer and tile flow for the drained layer	mm
SAT	water content of the layer at saturation	mm
S_w	solubility `solub`	mg/L

Mass moved by each flow path is the concentration times the flow:

- surface runoff removes $\beta_{perc}\,C$ times the runoff depth, where β_{perc} is the pesticide percolation coefficient `pest_perc` in `parameters.bsn`;
- lateral flow from the top layer also uses $\beta_{perc}\,C$; lateral flow from deeper layers uses C;
- tile flow and percolation use C. Percolation moves pesticide to the next layer, and out of the bottom layer to the aquifer.

Transport with sediment

On days with erosion, pesticide sorbed to the top soil layer leaves with the sediment yield:

$$P_{sed} = 0.001\,\frac{C_{sorb}\;sed\;\varepsilon_{er}}{A}$$

Symbol	Meaning	Units
P_{sed}	pesticide removed with sediment	kg/ha
C_{sorb}	sorbed concentration in the top layer	g/t
sed	sediment yield	t
ε_{er}	enrichment ratio, `erorgn` in `hydrology.hyd` if above 0.001, otherwise computed from the sediment concentration	-
A	HRU area	ha

Channels

Pesticide entering a channel is split into dissolved and sorbed phases using the channel carbon content and the sediment concentration of the inflow. Over the travel time through the reach (as a fraction of a day, at most 1) the dissolved and sorbed mass in water decays with the aquatic half-life `aq_hlife`. Dissolved pesticide volatilises at rate `aq_volat` and sorbed pesticide settles to the bed at rate `aq_settle` (m/day, divided by flow depth). Bed pesticide is resuspended at rate `aq_resus`, exchanges with the water by diffusion (a mixing velocity derived from the molecular weight and bed porosity), is buried at rate `ben_bury` over the active bed depth `ben_act_dep`, and decays with the benthic half-life `ben_hlife`. Dissolved pesticide above the solubility limit is moved to the bed. The pesticide left in the water leaves the reach in proportion to outflow over outflow plus storage.

Reservoirs follow the same set of processes (decay, volatilisation, settling, resuspension, diffusion and burial) without temperature adjustment. Pesticide reactions in wetlands are not simulated in this revision.

Aquifers

Pesticide reaching an aquifer decays with the aquatic half-life and leaves in baseflow and deep seepage. Its concentration is the aquifer pesticide mass divided by a mixing volume made of baseflow, seepage, revap and a fraction of aquifer storage. That fraction is the pesticide groundwater mixing factor in `parameters.bsn` (default 0, so only the day's outflows dilute the pesticide). See Aquifers.

Parameters

Parameter	File	Effect
`soil_ads`	`pesticide.pes`	Sorption coefficient normalised for organic carbon, K_{oc} (mL/g)
`frac_wash`	`pesticide.pes`	Fraction washed off foliage by rain
`hl_foliage`, `hl_soil`	`pesticide.pes`	Half-lives on foliage and in soil (days)
`solub`	`pesticide.pes`	Water solubility (mg/L)
`aq_hlife`, `ben_hlife`	`pesticide.pes`	Half-lives in water and in benthic sediment (days)

Parameter	File	Effect
aq_volat, aq_resus, aq_settle, ben_bury	pesticide.pes	Channel volatilisation, resuspension, settling and burial velocities (m/day)
ben_act_dep	pesticide.pes	Depth of the active benthic layer (m)
mol_wt	pesticide.pes	Molecular weight (g/mol)
pl_uptake, harvest_sink, resistance_n	pesticide.pes	Plant uptake factor and its fate in the plant
app_eff, surf_frac	chem_app.ops	Application efficiency and share placed in the top 10 mm
pest_perc	parameters.bsn	Pesticide concentration in runoff relative to soil water
6th and 7th values	parameters.bsn	Groundwater mixing factor and temperature switch (columns labelled lin_sed and exp_sed by the Editor)

Related pages

- pesticide.pes and operation files
- parameters.bsn
- Plant growth and land use management
- Erosion and sediment
- Channels, Reservoirs and wetlands, Aquifers

Sources

Code at revision 62.0.1:

- Application: src/pest_apply.f90, amount and efficiency in src/mgt_sched.f90 and src/actions.f90
- Washoff: src/pest_washp.f90; call order and the 2.54 mm rain threshold: src/hru_control.f90
- Decay and metabolites: src/pest_decay.f90; Q10 function: src/pest_tempdecay_module.f90
- Plant uptake: src/pest_pl_up.f90; harvest and kill: src/mgt_harvtuber.f90, src/mgt_killop.f90
- Transport in water: src/pest_lch.f90; with sediment: src/pest_pesty.f90, src/pest_enrsb.f90
- Database and decay factors: src/pest_parm_read.f90, src/pesticide_data_module.f90
- Channels: src/ch_rtpest.f90, mixing velocity in src/sd_hydsed_init.f90; reservoirs: src/res_pest.f90; aquifers: src/aqu_1d_control.f90
- Basin parameters: src/basin_module.f90 (basin_parms), read list-directed in src/basin_read_prm.f90

Symbols and variables:

Symbol or column	Variable
f_{gc}	gc
app_eff, surf_frac	chemapp_db%app_eff, chemapp_db%surf_frac
soil_ads, frac_wash, hl_foliage, hl_soil	pestdb%koc, %washoff, %foliar_hlife, %soil_hlife
daily survival factors	pestcp%decay_f, %decay_s, %decay_a, %decay_b
f_d	pestcp%daughter%foliar_fr, %soil_fr, %aq_fr, %ben_fr
f_{TSCF}, n	pestdb%pl_uptake, pestdb%resistance_n
β_{perc}	bsn_prm%percop
temperature switch	bsn_prm%temp_decay
groundwater mixing factor	bsn_prm%pestgwfact
pesticide on foliage, in the plant, in soil	cs_pl%pl_on%pest, cs_pl%pl_in%pest, cs_soil%ly%pest

Overview

SWAT+ can simulate two optional groups of dissolved substances, both listed in constituents.cs:

- Salt ions: the eight major ions sulfate (SO_4), calcium (Ca), magnesium (Mg), sodium (Na), potassium (K), chloride (Cl), carbonate (CO_3) and bicarbonate (HCO_3). In soil and aquifers they take part in mineral precipitation and dissolution and in cation exchange, and soil salinity can reduce plant growth.

- Constituents: selenate (SeO_4), selenite (SeO_3) and boron. Selenium species react in the soil (reduction, release from shale), and all three sorb to soil.

Both groups move with water along the same paths as nitrate: surface runoff, lateral flow, tile flow and percolation in the soil profile, then through aquifers, channels, reservoirs and wetlands. Inputs come from initial conditions, rain and dry deposition, fertiliser, irrigation, urban runoff and, for salts, road salt.

Transport in soil water

Salt ions and constituents are transported in each layer with the nitrate leaching equations, including anion exclusion. For each ion or constituent m in a layer:

$$C_m = \frac{M_m}{w}\left[1 - \exp\left(\frac{-w}{(1-\theta_e)\,SAT}\right)\right]$$

Symbol	Meaning	Units	Input
C_m	concentration in water leaving the layer	kg/ha per mm	
M_m	mass in the layer	kg/ha	
w	percolation plus lateral flow, plus surface runoff for the top layer and tile flow for the drained layer	mm	
θ_e	fraction of porosity from which anions are excluded	fraction	`anion_excl` in `soils.sol`
SAT	water content at saturation	mm	

Percolation, tile flow and lateral flow below the top layer remove mass at concentration C_m. Surface runoff and lateral flow from the top layer remove mass at $\beta_N C_m$, where β_N is the nitrate percolation coefficient `n_perc` in `parameters.bsn` (adjusted per HRU by calibration). Percolation moves mass to the next layer and out of the bottom of the profile to the aquifer. When gwflow is active with solute transport, groundwater discharging into the soil profile also adds mass.

Salt chemistry

In each soil layer and in each aquifer, the dissolved ions are brought into equilibrium with five minerals and with the exchange sites of the soil:

- Precipitation and dissolution of calcite ($CaCO_3$), magnesite ($MgCO_3$), gypsum ($CaSO_4$), epsomite ($MgSO_4$) and halite (NaCl). A mineral dissolves when the ion activity product is below its solubility product and precipitates when above it.
- Activity coefficients from the ionic strength I of the solution: the extended Debye-Hückel equation for $I \leq 0.1$ mol/L and the Davies equation above. For $I \geq 5$ mol/L the Davies equation is evaluated at $I = 0.5$.
- Cation exchange among Ca, Mg, Na and K.

The calculation iterates until concentrations converge. The solubility products are fixed in the code:

Mineral	Solubility product
$CaCO_3$	3.0702×10^{-9}
$MgCO_3$	4.7937×10^{-6}
$CaSO_4$	7.888×10^{-5}
$MgSO_4$	7.244×10^{-3}
NaCl	37.3

Salinity stress on plants

When the stress option in `salt_plants` is on and salts are simulated, each rooted soil layer gets a reduction factor from the FAO crop salt tolerance relation:

$$f_{salt} = \begin{cases} 1 & EC_e \le a \\ \dfrac{100 - b\,(EC_e - a)}{100} & EC_e > a \end{cases}, \qquad 0 \le f_{salt} \le 1$$

$$EC_e = \frac{TDS}{k_{EC}}$$

Symbol	Meaning	Units	Input
EC_e	electrical conductivity of the soil water	dS/m	
TDS	sum of the salt ion concentrations in the soil water, scaled by water content	mg/L	
k_{EC}	TDS to EC conversion factor	(mg/L)/(dS/m)	`salt_plants`
a	salinity threshold of the plant	dS/m	`salt_plants`, per plant
b	yield loss per dS/m above the threshold	%/(dS/m)	`salt_plants`, per plant

For gypsiferous soils (soil type 1 in `salt_plants`) the threshold is raised by 2 dS/m. A plant with $a = 0$ has no salt stress. The smallest factor over the rooted layers is the plant's salt stress.

Salt stress reduces daily biomass growth. With the "after other stresses" option (`salt_effect = 1`), growth is multiplied by the salt stress after taking the smallest of the water, temperature, nitrogen, phosphorus and aeration stresses. With option 2, salt stress is included in that minimum.

Salt uptake by plants

If the file `salt_uptake` is present, each ion is removed from the soil in the amounts it specifies (kg/ha per day), distributed over the layers in proportion to root mass. The first plant of the HRU's plant community is used. Without the file, plants take up no salt.

Selenium reactions

Selenate, selenite and nitrate in each soil layer are solved together each day (fourth-order Runge-Kutta, one-day step). The processes are:

- reduction of selenate to selenite and of selenite to elemental selenium, both first order and slowed by nitrate through the inhibition term $I/(I + C_{NO3})$;
- oxidation of selenium-bearing shale by oxygen and by nitrate, which releases selenate and consumes nitrate, in proportion to the shale fraction of the soil.

Rate constants, the inhibition constant, oxygen content and shale fractions are given per soil group in `cs_reactions`. Boron does not react.

Sorption of constituents

Selenate, selenite and boron are split between soil water and the soil matrix with a linear partition coefficient:

$$M_{sol} = \frac{M_{tot}}{1 + \dfrac{K_d\,\rho_b\,V_{soil}}{V_w}}, \qquad M_{sorb} = M_{tot} - M_{sol}$$

Symbol	Meaning	Units	Input
M_{tot}, M_{sol}, M_{sorb}	total, dissolved and sorbed mass in the layer	kg	
K_d	partition coefficient for each constituent	L/kg	`cs_reactions`
ρ_b	bulk density	Mg/m^3	
V_{soil}, V_w	volumes of soil and of soil water in the layer	m^3	

Aquifers use the same reaction and sorption equations with the aquifer parameters in `cs_reactions`.

Inputs

Source	File
Initial soil and aquifer content	`salt_hru.ini`, `salt_aqu.ini`, `cs_hru.ini`, `cs_aqu.ini`; channels: `salt_channel.ini`, `cs_channel.ini`
Rain and dry deposition	`salt_atmo.cli`
Fertiliser	`salt_fertilizer.frt`, `fertilizer.frt_cs`
Irrigation water	`salt_irrigation`, `cs_irrigation`
Urban runoff	`salt_urban`, `cs_urban`
Road salt (monthly, daily or annual amounts added to the top layer)	`salt_road`
Plant tolerance and stress options	`salt_plants`
Plant uptake	`salt_uptake`, `cs_uptake`
Reaction and sorption parameters	`cs_reactions`
Point sources	recall files (see Recall and point sources)

Aquifers, channels and water bodies

In the 1D aquifer, recharge adds salt and constituent mass; the salt chemistry, constituent reactions and sorption are applied; and baseflow and seepage remove mass at the concentration of the aquifer water (mass divided by groundwater volume). Channels, reservoirs and wetlands mix incoming mass with the stored water and release it with outflow. With gwflow, salts and constituents are transported between grid cells.

Pathogens and heavy metals

`constituents.cs` can also list pathogens and heavy metals.

Pathogens (bacteria and other microorganisms) are followed on foliage and in the top soil layer. Each day their number changes by first-order die-off minus growth, corrected for temperature:

$$N_d = N_{d-1}\,\exp\!\left(-(k_{die} - k_{grow})\,\theta^{T-20}\right)$$

with the rates of `pathogens.pth` for foliage or soil, θ = `t_adj` and T the mean air temperature. Rain of at least 2.54 mm washes a fraction of the pathogens on foliage to the soil; pathogens then leave the top layer with percolation, surface runoff and sediment. In channels they die off over the travel time at `do_stream`, corrected to the water temperature. The pathogen routines of 62.0.1 do not form a complete model: the HRU routines use one `pathogens.pth` row for all pathogens, the runoff step removes all pathogens left in the top layer every day, and the channel die-off does not act on the routed water. `pathogens.pth` lists these points.

Heavy metals are not transformed. They enter the network only from export coefficient and delivery ratio objects and move with the water as conservative loads (see `metals` and Recall and point sources).

Related pages

- Salt inputs and constituent inputs
- Nutrient cycling for the nitrate transport equations
- Plant growth and land use management for plant stresses
- Aquifers, Channels, Reservoirs and wetlands
- Recall and point sources

Sources

Code at revision 62.0.1:

- Pathogens: src/path_ls_process.f90 (die-off, growth, wash-off), src/path_ls_runoff.f90, src/path_ls_swrouting.f90, channels src/ch_rtpath.f90; heavy metals: src/exco_read_hmet.f90, src/dr_read_hmet.f90
- Soil transport: src/salt_lch.f90, src/cs_lch.f90
- Salt chemistry: src/salt_chem_hru.f90, src/salt_chem_soil_single.f90, src/salt_chem_aqu.f90; solubility products in src/salt_data_module.f90
- Salinity stress: src/pl_waterup.f90, src/pl_biomass_gro.f90; options read in src/salt_plant_read.f90
- Salt uptake: src/salt_uptake.f90, src/salt_uptake_read.f90; road salt: src/salt_roadsalt.f90
- Selenium reactions: src/cs_rctn_hru.f90, src/se_reactions_soil.f90; sorption: src/cs_sorb_hru.f90; parameters: src/cs_reactions_read.f90
- Aquifers: src/aqu_1d_control.f90, src/cs_rctn_aqu.f90, src/cs_sorb_aqu.f90

Symbols and variables:

Symbol or setting	Variable
M_m, concentration	cs_soil%ly%salt, %saltc; cs_soil%ly%cs, %csc
θ_e	soil%anion_excl
β_N	hru%nut%nperco
solubility products (soil, aquifer)	Ksp11 to Ksp51, Ksp12 to Ksp52
k_{EC}, stress option, soil type, effect option	salt_tds_ec, salt_tol_sim, salt_soil_type, salt_effect
a, b	salt_stress_a, salt_stress_b
f_{salt}	pcom%plstr%strss
salt uptake switch	salt_uptake_on
selenium rates, inhibition, oxygen, shale	cs_rct_soil%kseo4, %kseo3, %se_ino3, %oxy_soil, %shale, %ko2a, %kno3a
K_d	cs_rct_soil%kd_seo4, %kd_seo3, %kd_born

Overview

Channels move water, sediment and dissolved and particulate constituents from upstream objects to the watershed outlet. A SWAT+ channel is defined by its geometry (width, depth, slope, length, side slope, Manning's n) in hyd-sed-lte.cha, from which a rating curve of depth, area, wetted perimeter, top width, velocity and travel time against flow rate is built at the start of the run.

Each day a channel:

1. adds rain falling on the channel surface to the inflow;
2. checks for overbank flow, traps sediment on the floodplain and fills floodplain wetlands;
3. computes bank and bed erosion;
4. routes the water through channel and floodplain storage;
5. removes transmission losses and evaporation;
6. runs in-stream water quality, pesticides, pathogens, salts and constituents;
7. computes water temperature.

When gwflow is active, the channel also exchanges water with the groundwater cells it crosses (see Aquifers).

Rating curve

The rating curve is computed for a trapezoidal main channel with a floodplain above bankfull. For a given flow rate it gives flow depth, cross-section area, wetted perimeter, hydraulic radius, velocity (Manning's equation), and travel time $TT = L/(3.6\,v)$ in hours, with L the channel length (km) and v the velocity (m/s).

Flow routing

Inflow is added to channel storage on each routing step. The routing method is set by `rte_cha` in `codes.bsn`.

Variable storage (`rte_cha = 0`, default)

$$q_{out} = \min\left(1,\ \frac{2\,\Delta t\,C_{st}}{2\,TT + \Delta t}\right) V_{stor}$$

Symbol	Meaning	Units	Input
q_{out}	outflow during the step	m³	
Δt	routing step	h	
TT	travel time at the previous step's outflow rate	h	
C_{st}	storage coefficient multiplier	-	`scoef` in `parameters.bsn`
V_{stor}	water in channel and floodplain storage	m³	

With a daily time step there is one routing step per day.

Muskingum (`rte_cha = 1`)

$$O_2 = C_1 I_2 + C_2 I_1 + C_3 O_1$$

$$C_1 = \frac{\Delta t - 2KX}{2K(1-X) + \Delta t}, \quad C_2 = \frac{\Delta t + 2KX}{2K(1-X) + \Delta t}, \quad C_3 = \frac{2K(1-X) - \Delta t}{2K(1-X) + \Delta t}$$

C_1 is not allowed to fall below 0, and the three coefficients are scaled to sum to 1. The storage time constant is a weighted mean of the travel times at bankfull depth and at 0.1 × bankfull depth (using the kinematic wave celerity, 5/3 of the velocity):

$$K = \frac{c_1\,K_{bf} + c_2\,K_{0.1bf}}{c_1 + c_2}$$

Symbol	Meaning	Input
I, O	inflow and outflow volumes at the start (1) and end (2) of the step	
K	storage time constant (h)	
X	weighting factor	`msk_x` in `parameters.bsn`
c_1, c_2	weights for bankfull and low flow	`msk_co1`, `msk_co2` in `parameters.bsn`

When Δt exceeds $2K(1-X)$, the step is divided into sub-steps so that the method stays stable.

Floodplain storage

After each routing step, the part of storage above the bankfull channel volume becomes floodplain storage and the rest stays as channel storage. Both drain through the routing equation.

Transmission losses and evaporation

Once a day, after routing, water is removed from channel storage:

$$TL = 24\,K_{ch}\,L\,W, \qquad E_{ch} = \eta\,E_o\,L\,W$$

Symbol	Meaning	Units	Input
TL	transmission loss	m^3	
K_{ch}	effective hydraulic conductivity of the channel bed	mm/h	`k` in `hyd-sed-lte.cha`
L, W	channel length and bankfull width	km, m	`len`, `wd`
E_{ch}	evaporation from the channel	m^3	
η	evaporation coefficient	-	`evap_adj` in `parameters.bsn`
E_o	potential evapotranspiration	mm	

Both are limited to the water in channel storage and use the full bankfull width, whatever the flow. Transmission losses are reported as channel seepage. In SWAT, transmission losses used the wetted perimeter and could recharge the shallow aquifer; in SWAT+ the loss leaves channel storage and is not routed anywhere unless gwflow is used.

Sediment

Overbank flow and floodplain deposition

The daily peak flow is estimated from the mean daily inflow with a peak rate factor that decreases with drainage area:

$$q_{pk} = r_{pk}\left[1 + 2.66\left(\frac{A}{100}\right)^{-0.3}\right]\frac{Q_{day}}{86400}$$

where r_{pk} is `pk_rto` in `sed_nut.cha` (default 1) and A the drainage area (ha). When the peak exceeds 1.5 times the bankfull flow rate, part of the sediment, organic N and particulate P is trapped on the floodplain:

$$\varepsilon_{fp} = \min\left(1,\ \frac{q_{pk}}{\bar{q}}\left[1 - \exp\left(-10^{-4}\,\frac{A_{fp}}{q_{pk}}\right)\right]\right)$$

with $\bar{q}$ the mean daily flow rate and A_{fp} a floodplain area of five channel widths times the channel length (m^2). Organic N and particulate P are trapped with the enrichment ratios `n_dep_enr` and `p_dep_enr`. Overbank water also fills the wetlands of HRUs that lie on the floodplain.

Bank erosion

Banks erode when the reach velocity, raised to account for flow around bends, exceeds a critical velocity that depends on bank cohesion (from clay content), vegetation cover and bulk density:

$$v_{cr} = c_{vcr}\,\log_{10}(2200\,D)\,\sqrt{0.0004\,(f_{bd} + 0.021\,\tau_{coh} + f_{veg})}$$

$$f_{veg} = 4500\,c_{cov}\,e^{-5D}, \qquad \tau_{coh} = \max\left(1000,\ -87.1 + 42.82\,cl - 0.261\,cl^2 + 0.029\,cl^3\right)$$

The bank retreat rate (m/day) follows a logistic function of v/v_{cr}, scaled by channel width and a duration factor that decreases with drainage area, and limited by an exhaustion rate of $0.0002\,W$. Eroded bank mass is retreat × depth × a quarter of the channel length × bulk density. Bank material adds organic N and P at the concentrations

`n_conc` and `p_conc`, with a share `p_bio` of the P as soluble P. A fraction `wash_bed_fr` of the eroded bank material is deposited in the channel; the rest moves downstream.

Symbol	Meaning	Input
D	bankfull depth (m)	`dp`
c_{cov}	bank cover factor	`cov_fact`
cl	bank clay content (%)	`clay`
f_{bd}	bulk density factor, $\max(0.001,\ 39.24\,\rho_b - 1000)$	`dry_bd`
c_{vcr}	critical velocity coefficient	12th value, labelled `eq_slp` in Editor files

Bed erosion

Bed erosion is computed when the velocity exceeds $0.293\sqrt{d_{50}}$ m/s, as $10^{-4}(v/v_{cr})$ m/day of bed lowering. It is reported in the channel morphology output but is not added to the sediment in the flow.

Water quality

In-stream nitrogen, phosphorus, algae, dissolved oxygen and carbonaceous BOD follow the QUAL2E kinetics (Brown and Barnwell, 1987), with rate constants from `nutrients.cha`. The calculation runs once a day for every channel with inflow, on the concentrations of the day's inflow.

Temperature and solution method

Every rate is given for 20 °C and corrected to the water temperature T_w:

$$k_T = k_{20}\,\theta^{\,T_w - 20}, \qquad T_w = 5 + 0.75\,\bar{T}_{air}$$

with T_w at least 0.1 °C and $\bar{T}_{air}$ the day's mean air temperature at the channel's weather station.

Process	θ
Algal growth and respiration; organic N hydrolysis; nitrite oxidation; organic P mineralisation; CBOD decay	1.047
Algal, organic N and organic P settling; CBOD settling; reaeration (also used for ammonium oxidation)	1.024
Benthic P and ammonium sources	1.074

The channel is treated as a mixed reach with a residence time τ, the travel time through the channel in days (at most 1). For each constituent with inflow concentration C_{in}, a constant source or sink M (mg/L/day) and a first-order rate K (1/day), the outflow concentration is

$$C_{out} = C_{in}\,e^{-a} + \frac{C_{in}/\tau + M}{1/\tau - K}\,(1 - e^{-a}), \qquad a = \left(\frac{1}{\tau} - K\right)\Delta t$$

with $\Delta t = 1$ day.

A first-order transformation from one constituent to another (for example organic N to ammonium) is converted into the equivalent constant term, so the mass removed from one constituent is added to the next.

Algae

Algal biomass A (mg/L) is computed from chlorophyll-a with `chla_alg`. The growth rate combines light and nutrient limitation:

$$\mu = \mu_{max}\, f_L\, f_{NP}, \qquad f_N = \frac{N}{N + K_N}, \qquad f_P = \frac{P_{sol}}{P_{sol} + K_P}$$

where f_{NP} is $f_N f_P$, $\min(f_N, f_P)$ or $2/(1/f_N + 1/f_P)$ for `q2e_alg` 1, 2 or 3. The light factor averages light over the water depth d:

$$f_L = 0.92\, \frac{T_L}{24}\, \frac{1}{\lambda\, d}\, \ln\!\left(\frac{K_L + I}{K_L + I\, e^{-\lambda d}}\right), \qquad I = \frac{f_{PAR}\, H_{day}}{T_L}$$

Symbol	Meaning	Input
μ_{max}	Maximum specific growth rate (1/day)	`alg_grow`
ρ	Respiration rate (1/day)	`alg_resp`
σ_1	Settling rate (m/day)	`alg_stl`
K_N, K_P	Half-saturation constants for N and P (mg/L)	`const_n`, `const_p`
K_L	Half-saturation constant for light (MJ/m²/h)	`lt_co` × 0.06
λ	Light extinction coefficient (1/m)	`lt_nonalg`
f_{PAR}	Fraction of solar radiation that is photosynthetically active	`slr_act`
H_{day}	Solar radiation of the day (MJ/m²)	weather
N	Ammonium plus nitrate in the channel storage (mg/L)	
T_L	Hours used as the day length (see the list below)	

The net growth $\mu - \rho$ is the source term for algae and settling at σ_1/d the first-order loss. Biomass is kept between 0 and 5000 mg/L. Algal growth takes up soluble P at `alg_p` per unit of biomass.

Nitrogen, phosphorus, CBOD and oxygen

Constituent	Sources and sinks	Inputs
CBOD	decay at κ_1; settling at κ_3/d	`cbn_bod_co`, `cbn_bod_stl`
Organic N	hydrolysis to ammonium at $2\beta_3$; settling at σ_4/d	`ptln_nh3n`, `ptln_stl`
Ammonium	gain from organic N hydrolysis; oxidation to nitrite at κ_2	`air_rt`
Nitrite	gain from ammonium oxidation; oxidation to nitrate at β_2	`no2n_no3n`
Nitrate	gain from nitrite oxidation	
Organic P	mineralisation to soluble P at β_4; settling at σ_5/d	`ptlp_solp`, `ptlp_stl`
Soluble P	gain $\beta_4 P_{org} + \sigma_2/d_{ch}$; loss to algal growth	`ben_disp`
Dissolved oxygen	reaeration $\kappa_2(O_{sat} - O)$; CBOD decay; α_5 per unit of ammonium and α_6 per unit of nitrite oxidised; a loss of σ_4/d	`air_rt`, `o2_nh3n`, `o2_no2n`

Soluble P is updated directly, $P_{out} = P_{in} + (\beta_4 P_{org} + \sigma_2/d_{ch} - \alpha_2\, \mu\, A)\, \tau$, with d_{ch} the bankfull depth of the channel and α_2 = `alg_p`. The oxygen saturation concentration (mg/L) depends on the water temperature T_K in kelvin:

$$\ln O_{sat} = -139.34410 + \frac{1.575701 \times 10^5}{T_K} - \frac{6.642308 \times 10^7}{T_K^2} + \frac{1.243800 \times 10^{10}}{T_K^3} - \frac{8.621949 \times 10^{11}}{T_K^4}$$

Reaeration uses `air_rt` directly; the stream reaeration formulas of QUAL2E and reaeration over dams are not computed. The inflow oxygen concentration is limited to between 0.001 and 30 mg/L before the calculation.

Behaviour in 62.0.1

Several parts of the in-stream calculation differ from QUAL2E:

- T_L is the day-length threshold for plant dormancy (`dorm_hr` in `parameters.bsn`, or a value of at most 1 hour derived from latitude), not the day length. At latitudes below 20° with `dorm_hr` = 0 it is zero or negative, the light intensity is set to almost zero, and algae do not grow.
- The extinction coefficient never exceeds `lt_nonalg`, so algal self-shading (`alg_shd_l`, `alg_shd_nl`) has no effect.
- Ammonium is oxidised at the reaeration rate `air_rt`; `nh3n_no2n` has no effect. Organic N hydrolysis uses twice `ptln_nh3n`.
- Algal uptake of ammonium and nitrate and algal oxygen production and respiration are computed but not applied.
- Sediment oxygen demand (`ben_bod`) is not taken from dissolved oxygen; the organic N settling term σ_4/d is subtracted instead. The benthic ammonium source (`ben_nh3n`) only enters the inflow concentration used to compute ammonium oxidation.
- The benthic P source is added as `ben_disp` (mg/m²/day) divided by the depth in m, which gives mg/m³ per day, but is added to a concentration in mg/L.

When the 24th value of `codes.bsn` is 1 (the column Editor files label `i_fpwet`; the `wq_cha` column is not used), a simplified exchange replaces the QUAL2E results for nitrate, organic N, soluble P and particulate P. Over the travel time TT (h), soluble N and P are converted to particulate forms:

$$\Delta N = \min\left(NO_3,\ \left[1 - e^{-k_N TT}\right] N_{org}\right), \qquad \Delta P = \min\left(P_{sol},\ \left[1 - e^{-k_P TT}\right] P_{sed}\right)$$

with k_N and k_P from `n_sol_part` and `p_sol_part` in `sed_nut.cha` (default 0.01).

Water temperature

When the channel's weather station has measured temperatures, water temperature is built from:

- the flow-weighted temperature of upstream channels;
- local inflows from the routing units draining to the channel (snowmelt, surface runoff and lateral flow) and groundwater, each given the mean air temperature over its lag period (`sno_lag`, `surf_lag`, `lat_lag`, `gw_lag` days) times a coefficient;
- heat exchange with the air, using dew point, wind speed and optional shading factors, over the travel time.

Parameters are in `temperature.cha`. Without measured air temperature, water temperature is $5 + 0.75\,T_{air}$.

Parameters

Parameter	File	Effect
`rte_cha`	`codes.bsn`	0 = variable storage; 1 = Muskingum
24th value (Editor column `i_fpwet`)	`codes.bsn`	1 = simplified soluble to particulate N and P exchange
`scoef`	`parameters.bsn`	Variable storage coefficient multiplier
`msk_co1`, `msk_co2`, `msk_x`	`parameters.bsn`	Muskingum weights and weighting factor
`evap_adj`	`parameters.bsn`	Channel evaporation coefficient
`wd`, `dp`, `slp`, `len`, `mann`, `side_slp`	`hyd-sed-lte.cha`	Geometry and roughness
`k`	`hyd-sed-lte.cha`	Bed hydraulic conductivity for transmission losses (mm/h)
`cov_fact`, `clay`, `dry_bd`	`hyd-sed-lte.cha`	Bank erosion resistance
`n_conc`, `p_conc`, `p_bio`	`hyd-sed-lte.cha`	N and P content of eroded bank material
`pk_rto`, `n_dep_enr`, `p_dep_enr`, `wash_bed_fr`, `n_sol_part`, `p_sol_part`	`sed_nut.cha` (optional)	Peak rate, floodplain enrichment, channel deposition and simplified nutrient exchange

Related pages

- Channel inputs and legacy channel inputs
- Channel, sediment and nutrient outputs
- Hydrology, Aquifers, Reservoirs and wetlands
- Pesticides, Salts and constituents

Sources

Code at revision 62.0.1:

- Daily channel driver: src/sd_channel_control3.f90
- Routing, transmission loss and evaporation: src/ch_rtmusk.f90
- Rating curve and Muskingum coefficients: src/sd_hydsed_init.f90, src/sd_rating_curve.f90, src/rcurv_interp_flo.f90
- Sediment and floodplain: src/sd_channel_sediment3.f90
- Inputs: src/sd_hydsed_read.f90 (`hyd-sed-lte.cha`, `sed_nut.cha`), src/sd_channel_read.f90 (`channel-lte.cha`), types in src/sd_channel_module.f90
- Water quality: src/ch_watqual4.f90, with the solution in src/wq_semianalyt.f90 and src/wq_k2m.f90 and the temperature correction in src/theta.f90. QUAL2E reference: Brown, L. C., and Barnwell, T. O. (1987), The enhanced stream water quality models QUAL2E and QUAL2E-UNCAS, EPA/600/3-87/007.
- Water quality symbols: μ_{max}, ρ = `ch_nut%mumax`, `%rhoq`; σ_1 to σ_5 = `%rs1` to `%rs5`; κ_1 to κ_4 = `%rk1` to `%rk4`; β_1 to β_4 = `%bc1` to `%bc4`; α_0 to α_6 = `%ai0` to `%ai6`; K_L, K_N, K_P = `%k_l`, `%k_n`, `%k_p`; λ = `%lambda0`; f_{PAR} = `%tfact`; T_L = `wgn_pms(iwgn)%daylth`; τ = `rcurv%ttime / 24`; d = `rcurv%dep`; d_{ch} = `sd_chd(jrch)%chd`.
- Temperature: src/ch_temp.f90, src/ch_read_temp.f90

Columns and variables (`hyd-sed-lte.cha` is read list-directed into `swatdeg_hydsed_data`, so position decides the variable):

Position	Editor column	Variable
3 to 8	`wd`, `dp`, `slp`, `len`, `mann`, `k`	`chw`, `chd`, `chs`, `chl`, `chn`, `chk`
9	`erod_fact`	`bank_exp`
10	`cov_fact`	`cov`
11	`wd_rto`	`sinu` (sinuosity, used for the bend radius)
12	`eq_slp`	`vcr_coef` (critical velocity coefficient)
13 to 16	`d50`, `clay`, `carbon`, `dry_bd`	`d50`, `ch_clay`, `carbon`, `ch_bd`
17	`side_slp`	`chss`
18	`bed_load`	`bankfull_flo`
19 to 23	`fps`, `fpn`, `n_conc`, `p_conc`, `p_bio`	same names

Other symbols: C_{st} = `bsn_prm%scoef`; X, c_1, c_2 = `bsn_prm%msk_x`, `%msk_co1`, `%msk_co2`; η = `bsn_prm%evrch`; r_{pk} = `sd_ch%pk_rto`; `rte_cha` = `bsn_cc%rte`. `codes.bsn` is read list-directed into `basin_control_codes`, so the 24th value (Editor column `i_fpwet`) sets `bsn_cc%qual2e`, and `wq_cha` sets `bsn_cc%wq`, which is not used.

Overview

Water that percolates below the soil profile recharges groundwater. Groundwater returns to the stream network as baseflow, moves up toward the root zone in dry periods ("revap"), and leaks downward out of the shallow system as deep seepage. SWAT+ offers two ways to simulate this:

- 1D aquifer objects (the default): each aquifer is a lumped storage defined in aquifer.aqu and connected to channels, reservoirs or other aquifers through aquifer.con. Recharge, baseflow, revap and seepage are computed once a day from a few parameters.
- gwflow (`gwflow = 1` in codes.bsn): the aquifer is replaced by a grid of cells. Each cell has a groundwater head, and water moves between cells by Darcy's law and exchanges with channels, reservoirs, wetlands, tile drains, canals and pumping wells.

In SWAT, every subbasin had a shallow and a deep aquifer. In SWAT+ an aquifer is an independent spatial object. A watershed can have one aquifer or many, and deep seepage from a shallow aquifer can be routed to a second (deep) aquifer object or allowed to leave the system.

1D aquifer

Recharge

Recharge is the water an aquifer receives from the objects routed to it: percolation from HRUs, routing units or other aquifers, and any channel or reservoir seepage directed to it in the connection files. The daily recharge depth is

$$w_{rchg} = \frac{V_{in}}{10\,A}$$

Symbol	Meaning	Units
w_{rchg}	recharge to the aquifer	mm
V_{in}	volume of water entering the aquifer on the day	m^3
A	aquifer area	ha

SWAT delayed recharge with an exponential lag (`GW_DELAY`). The SWAT+ 1D aquifer has no recharge delay: water leaving the soil profile is added to aquifer storage on the day it arrives. Recharge is added to storage first, and all outflows on the day are then computed from the new storage.

Water table depth

Storage is converted to a water table depth using the specific yield:

$$d_{wt} = d_{bot} - \frac{S}{1000\,S_y}, \qquad d_{wt} \geq 0$$

Symbol	Meaning	Units	Input
d_{wt}	depth from the land surface to the water table	m	initial value `dep_wt`
d_{bot}	depth from the land surface to the bottom of the aquifer	m	`dep_bot`
S	water stored in the aquifer	mm	
S_y	specific yield	m^3/m^3	`spec_yld`

At the start of the simulation storage is set from the initial water table depth, $S = 1000\,(d_{bot} - d_{wt})\,S_y$.

Baseflow

Baseflow (return flow) occurs only while the water table is at or above the threshold depth `flo_min`. It follows the recession equation used in SWAT:

$$Q_{gw,i} = \begin{cases} Q_{gw,i-1}\,e^{-\alpha} + w_{rchg}\,(1 - e^{-\alpha}) & d_{wt} \leq d_{flo} \\ 0 & d_{wt} > d_{flo} \end{cases}$$

Symbol	Meaning	Units	Input
$Q_{gw,i}$	baseflow on day i	mm	initial value `gw_flo`

Symbol	Meaning	Units	Input
α	baseflow recession constant	1/day	`alpha_bf`
d_{flo}	water table depth at or above which baseflow occurs	m	`flo_min`

Baseflow is limited to the water in storage. Because the threshold is a depth, a larger `flo_min` allows baseflow from a deeper water table. In SWAT the equivalent threshold `GWQMN` was a storage in mm.

The baseflow hydrograph goes to every object listed in `aquifer.con` with hydrograph type `tot`, split by the fractions given there. When channels are linked to the aquifer through the aquifer-channel link file, baseflow is spread over the linked channels from the outlet upward. The contributing channel length grows with the ratio of baseflow to `bf_max`, so `bf_max` is the baseflow at which every linked channel receives water.

Deep seepage

A fixed fraction of the day's recharge leaves the aquifer through its bottom:

$$w_{seep} = \beta_{deep}\, w_{rchg}$$

where β_{deep} is `rchg_dp` (fraction). Seepage is limited to the water left in storage after baseflow. It leaves the aquifer as its second hydrograph; routing it in `aquifer.con` with hydrograph type `rhg` sends it to another object, usually a deep aquifer.

Revap

When the water table is shallower than `revap_min`, water is removed from the aquifer in response to evaporative demand:

$$w_{revap} = \begin{cases} \beta_{rev}\, E_o & d_{wt} < d_{rev} \\ 0 & d_{wt} \geq d_{rev} \end{cases}$$

Symbol	Meaning	Units	Input
w_{revap}	revap	mm	
β_{rev}	revap coefficient	-	`revap`
E_o	potential evapotranspiration at the aquifer's weather station	mm	
d_{rev}	water table depth above which revap occurs	m	`revap_min`

Revap is limited to the water left in storage. In SWAT the revap threshold `REVAPMN` was a storage in mm and revap tapered off as storage approached it. In SWAT+ the threshold is a depth and revap is either on or off. Revap is removed from the aquifer and is not added back to the soil profile.

The daily order is recharge, water table depth, baseflow, seepage, then revap. When the aquifer is a point of diversion in the water allocation inputs, withdrawals are taken after recharge and before baseflow.

Nitrate and phosphorus

Nitrate in recharge water is added to the aquifer nitrate pool. Baseflow and seepage remove nitrate at the concentration of the aquifer water, $C_{NO3} = N_{aq}/S$. The pool also decays with half-life `hl_no3n`:

$$N_{aq,i} = N_{aq,i-1}\, \exp\left(\frac{-0.693}{t_{1/2} + 0.1}\right)$$

where $t_{1/2}$ is `hl_no3n` (days). A half-life of 0 removes almost all aquifer nitrate each day.

Soluble phosphorus in baseflow is the volume of water entering the aquifer on the day multiplied by the concentration `sol_p` (mg/L). It is a fixed concentration and does not draw on a phosphorus pool.

Pesticides, salts and constituents

Pesticides entering with recharge decay with the aquatic half-life of the pesticide. The concentration available for transport is the pesticide mass divided by a mixing volume made of baseflow, seepage, revap and a fraction of storage set by the pesticide groundwater mixing factor in `parameters.bsn`. Baseflow and seepage remove pesticide at that concentration. All aquifer pesticide is soluble.

Salt ions and other constituents are mixed through the aquifer water. Their concentration is mass divided by the groundwater volume, and baseflow and seepage remove mass at that concentration. See Salts and constituents.

gwflow grid model

Activation

Setting `gwflow = 1` in `codes.bsn` turns on gwflow. SWAT+ then reads the gwflow input files (`codes.gw`, `zones.gw`, `cells.gw`, `cellcon.gw`, `chancell.gw` and the optional files for tiles, reservoirs, floodplains, canals, pumping and phreatophytes), and the 1D aquifer objects are switched off. If `chancell.gw` is missing, gwflow is switched off and the 1D aquifers are used.

Cell water balance

Each active cell has a head h, a bottom elevation, a land surface elevation, a hydraulic conductivity K, a specific yield S_y, an ET extinction depth and an area A_c. The day is divided into flow time steps of length Δt (`time_step` in `codes.gw`). On each step the change in cell storage is the sum of lateral flows from neighbouring cells and the sources and sinks of the day:

$$\Delta V = \left(\sum_k Q_k + Q_{ss} \right) \Delta t, \qquad h^{new} = h + \frac{\Delta V}{S_y \, A_c}$$

The solution is explicit: all heads are updated from the heads at the start of the step.

Lateral flow between cells

Flow between a cell and each connected neighbour follows Darcy's law:

$$Q_k = K_f \, \frac{h_k - h}{L} \, b_f \, w$$

Symbol	Meaning	Units
Q_k	flow from neighbour k into the cell	m³/day
K_f	face conductivity, the harmonic mean of the two cells' K	m/day
L	distance between the cell centroids	m
b_f	mean saturated thickness of the two cells	m
w	face width, the square root of the smaller cell's area	m

Inactive cells and no-flow boundary cells pass no water. Constant-head boundary cells exchange water with their neighbours but keep their initial head.

Sources and sinks

- Recharge: percolation from each HRU or landscape unit is delayed and passed to the cells it overlaps: $R_i = (1-\delta)\, w_{perc,i} + \delta\, R_{i-1}$, with $\delta = e^{-1/\tau}$ and τ the recharge delay (days) of the cell's zone.
- Groundwater ET: ET demand not met by the soil profile is taken from the cell: all of it when the head is at or above the land surface, none when the head is below the extinction depth, and a linearly varying share in between.
- Channel exchange: each channel cell exchanges water with the stream through the streambed, $Q = K_{bed}\,(\Delta h / b_{bed})\, W\, L_{ch}$, where W is channel width, L_{ch} the channel length in the cell and b_{bed} the bed thickness. When the head is above the channel stage, groundwater discharges to the channel and Δh is head minus stage. When the head is between the bed and the stage, the channel leaks to the aquifer and Δh is stage minus head. When the head is below the bed, leakage uses the flow depth as Δh.
- Other exchanges: reservoirs, ponds, wetlands, floodplains, tile drains, canals, pumping (from water allocation or an external schedule) and phreatophyte transpiration are added when their input files are present.
- Saturation excess: when the head rises above the land surface, the excess water goes to the nearest channel.

gwflow can also transport heat and solutes (nitrate, phosphorus, salt ions and constituents) between cells and can track groundwater travel time. Inputs are described in gwflow inputs and outputs in gwflow outputs.

Parameters

Parameter	File	Effect
`gwflow`	`codes.bsn`	0 = 1D aquifer objects; 1 = gwflow grid model
`gw_flo`	`aquifer.aqu`	Initial baseflow (mm)
`dep_bot`	`aquifer.aqu`	Depth to the aquifer bottom (m)
`dep_wt`	`aquifer.aqu`	Initial water table depth (m)
`no3_n`	`aquifer.aqu`	Initial nitrate-N concentration (mg/L)
`sol_p`	`aquifer.aqu`	Soluble P concentration of baseflow (mg/L)
`bf_max`	`aquifer.aqu`	Baseflow at which all linked channels receive flow (mm)
`alpha_bf`	`aquifer.aqu`	Baseflow recession constant (1/day)
`revap`	`aquifer.aqu`	Revap coefficient
`rchg_dp`	`aquifer.aqu`	Fraction of recharge lost to deep seepage
`spec_yld`	`aquifer.aqu`	Specific yield (m^3/m^3)
`hl_no3n`	`aquifer.aqu`	Half-life of nitrate in the aquifer (days)
`flo_min`	`aquifer.aqu`	Water table depth at or above which baseflow occurs (m)
`revap_min`	`aquifer.aqu`	Water table depth above which revap occurs (m)

Related pages

- `aquifer.aqu` and connection files
- gwflow inputs and gwflow outputs
- Hydrology for percolation out of the soil profile
- Channels for baseflow in the stream network
- Aquifer, reservoir and plant output
- Water allocation for aquifer withdrawals

Sources

Code at revision 62.0.1:

- 1D aquifer daily routine: `src/aqu_1d_control.f90`
- Initialisation of storage, recession and nitrate decay factors: `src/aqu_initial.f90`
- Reader for `aquifer.aqu`: `src/aqu_read.f90`
- Types `aquifer_database`, `aquifer_data_parameters`, `aquifer_dynamic`: `src/aquifer_module.f90`
- Aquifer-channel links: `src/aqu2d_init.f90`, `src/aqu2d_read.f90`

- Hydrograph types `tot` and `rhg`: src/hyd_connect.f90
- gwflow activation: src/basin_read_objs.f90; inputs: src/gwflow_read.f90
- gwflow daily driver and solution: src/gwflow_simulate.f90, src/gwflow_lateral.f90
- gwflow sources and sinks: src/gwflow_rech.f90, src/gwflow_gwet.f90, src/gwflow_channel_exch.f90, src/gwflow_satexcess.f90, src/gwflow_tile.f90, src/gwflow_reservoir.f90, src/gwflow_canal.f90

Symbols and variables:

Symbol or column	Variable
w_{rchg}	`aqu_d%rchrg`
S	`aqu_d%stor`
d_{wt}	`aqu_d%dep_wt`
Q_{gw}	`aqu_d%flo`
w_{seep}	`aqu_d%seep`
w_{revap}	`aqu_d%revap`
N_{aq}	`aqu_d%no3_st`
`gw_flo`, `no3_n`, `sol_p`, `carbon`	`aqudb%flo`, `%no3`, `%minp`, `%cbn`
`alpha_bf`, `revap`, `rchg_dp`, `spec_yld`, `hl_no3n`	`aqudb%alpha`, `%revap_co`, `%seep`, `%spyld`, `%hlife_n`
$e^{-\alpha}$ and the nitrate decay factor	`aqu_prm%alpha_e`, `aqu_prm%nloss`
pesticide groundwater mixing factor	`bsn_prm%pestgwfact`
gwflow h, K, S_y, extinction depth	`gw_state%head`, `%hydc`, `%spyd`, `%exdp`
gwflow Δt, δ	`gw_time_step`, `gw_delay`

Overview

SWAT+ has two kinds of water body:

- Reservoirs are spatial objects in the routing network. They receive water from channels, HRUs, routing units, aquifers or recall objects, store it, and release it by rules written in a decision table. They are defined in reservoir.res, which points to records in hydrology.res, sediment.res, nutrients.res, initial.res and a release table in res_rel.dtl.
- Wetlands belong to an HRU. They store water on the HRU surface (including rice paddies and other impoundments), receive rain, irrigation and the HRU's runoff, and release water by the same kind of decision table. They are defined in wetland.wet and hydrology.wet and assigned to HRUs through the `surf_stor` column of hru-data.hru.

Both use the same release, sediment and nutrient calculations.

Reservoir water balance

A reservoir starts to operate in the year and month given by `yr_op` and `mon_op` in hydrology.res. Before that date, inflow passes through unchanged.

Each day:

1. Inflow is added to storage. If the reservoir is a point of diversion in the water allocation inputs, withdrawals are taken.
2. The release table is evaluated and the release computed (see below). The release is smoothed with lag factors.
3. Rain on the water surface is added, then release, evaporation and seepage are removed, each limited to the water available.
4. The surface area is updated from the new storage.

$$V_i = V_{i-1} + V_{in} + V_{pcp} - V_{out} - V_{evap} - V_{seep}$$

$$V_{pcp} = 10\,R\,A, \qquad V_{evap} = 10\,\eta\,E_o\,A, \qquad V_{seep} = 240\,K_{sat}\,A$$

Symbol	Meaning	Units	Input
V	water stored	m^3	
R	precipitation	mm	
E_o	potential evapotranspiration	mm	
η	evaporation coefficient	-	`evap_co`
K_{sat}	hydraulic conductivity of the reservoir bottom	mm/h	`k`
A	water surface area on the previous day	ha	

When gwflow is active, seepage is computed by gwflow as exchange with the groundwater cells under the reservoir.

Surface area

Surface area is a power function of storage, $A = \beta_1 V^{\beta_2}$. The coefficients are computed from the areas and volumes at the principal and emergency spillways (`area_ps`, `vol_ps`, `area_es`, `vol_es`):

$$\beta_2 = \frac{\log_{10} A_{es} - \log_{10} A_{ps}}{\log_{10} V_{es} - \log_{10} V_{ps}} \ (\leq 0.9), \qquad \beta_1 = \left(\frac{A_{es}}{V_{es}}\right)^{\beta_2}$$

If β_2 would exceed 0.9, it is set to 0.9 and $\beta_1 = (A_{ps}/V_{ps})^{0.9}$.

Release smoothing

The release from the decision table is blended with the previous day's release so that outflow does not jump:

$$Q_i = \alpha\, Q^* + (1 - \alpha)\, Q_{i-1}, \qquad \alpha = e^{-\lambda}$$

where Q^* is the release from the table and λ is the lag for rising releases or the lag for falling releases. The two lags are read from the last two columns of `hydrology.res`, which the SWAT+ Editor labels `shp_co1` and `shp_co2` (formerly area-volume shape coefficients). A lag of 0 gives no smoothing.

Release rules

Each reservoir names a release table. Names that begin with `ctbl_` refer to a conditions table; all others refer to a decision table in `res_rel.dtl`. The decision table conditions are evaluated as described in Decision tables, and every action that fires adds a release. Release options (the `option` of a `release` action):

Option	Release for the day
`rate`	constant × 86400 (constant in m^3/s)
`rate_pct`	constant % of the principal volume
`inflo_rate`	larger of (inflow + second constant × 86400) and constant × 86400
`inflo_frac`	constant × inflow
`ab_emer`	all storage above the emergency volume
`days`	(storage − lower bound) / constant days, where the lower bound is 0, or a fraction (second constant) of the principal or emergency volume
`dyrt`	(storage − lower bound) / constant days + second constant % of the principal volume, with the lower bound 0, the principal or the emergency volume
`dyrt1`	as `dyrt`, with the lower bound taken from a condition of the table
`inflo_targ`	inflow + (storage − target), with the target a fraction of the principal volume taken from a condition
`weir`	discharge over a weir from `weir.res`
`meas`	measured release from a recall file

Option	Release for the day
`natlake`	natural lake outflow, $k_r\, S_{eff}\, (S_{eff}/(S_{max} - S_0))^{1.5}$ with S_0 a fraction of the principal volume and S_{max} the principal volume
`nonirr-h06`, `irr-h06`	Hanasaki et al. (2006) release for non-irrigation and irrigation reservoirs, from the long-term mean monthly inflow and demand
`hydrop`	hydropower release: mean inflow scaled by a storage factor and a seasonal factor

Reservoir sediment

When the suspended sediment concentration is above the equilibrium concentration `nsed`, the excess settles:

$$C_{sed,i} = (C_{sed} - C_{eq})\, e^{-0.184\, d_{50}} + C_{eq}$$

where d_{50} is the median particle size in `sediment.res` (µm). Sediment leaves with the release at the settled concentration. When the concentration is at or below `nsed`, no sediment is released.

Reservoir nutrients

Organic N, particulate P, soluble N (nitrate, ammonium, nitrite) and soluble P are removed by settling. The fraction removed each day is

$$f = \min\left(1,\ (C - C_{min})\, r\, \theta^{T-20}\right)$$

Symbol	Meaning	Input
C	concentration of the pool	mg/L
C_{min}	minimum concentration for settling	`n_min_stl`, `p_min_stl`
r	settling rate; for organic N and particulate P it differs between the mid-year period (`mid_start` to `mid_end` months) and the rest of the year; soluble N and P have one rate each	`mid_n_stl`, `n_stl`, `mid_p_stl`, `p_stl`; positions 8 and 9 for soluble N and P
θ	temperature factor	`theta_n`, `theta_p`
T	air temperature	°C

For soluble N and P the fraction is multiplied by 0.2. After settling, the release takes nutrients in proportion to its share of the water.

Pesticides, salts and constituents in reservoirs are described on the Pesticides and Salts and constituents pages.

Wetlands and paddies

A wetland covers part of its HRU and stores water on the surface. Its principal and emergency volumes are the HRU area times the depths `dp_ps` and `dp_es` in `hydrology.wet`. When a weir is assigned in `weir.res`, the principal volume is the HRU area times the weir height, and the emergency volume is at least 1.2 times that.

Each day:

1. Rain and irrigation are added to the wetland.
2. The wetted area is updated from storage with the volume-depth coefficients of `hydrology.wet`. Rice paddies are treated as flat and cover the whole HRU.
3. Seepage into the soil is $24\, K_{sat}\, A$, with K_{sat} the `k` of `hydrology.wet` (or the conductivity set by a puddling operation). Seepage that the soil cannot take because it is saturated stays in the wetland. Nutrients move into the top soil layer with the seepage.

4. Release follows the decision table, or the weir equation for paddies, and becomes surface runoff of the HRU.
5. Sediment, nutrients, salts and constituents are updated as for reservoirs.

Evaporation from standing water is taken from the wetland first, as part of the HRU soil evaporation, before any water is taken from the soil.

Parameters

Parameter	File	Effect
`yr_op`, `mon_op`	`hydrology.res`	Year and month the reservoir starts operating
`area_ps`, `vol_ps`, `area_es`, `vol_es`	`hydrology.res`	Surface areas (ha) and volumes (ha-m) at the principal and emergency spillways
`k`	`hydrology.res`, `hydrology.wet`	Bottom hydraulic conductivity (mm/h)
`evap_co`	`hydrology.res`	Evaporation coefficient
`shp_co1`, `shp_co2`	`hydrology.res`	Release lags for rising and falling outflow
`nsed`, `d50`	`sediment.res`	Equilibrium sediment concentration (mg/L) and median particle size (µm)
settling rates, θ, minimum concentrations	`nutrients.res`	Nutrient settling
`dp_ps`, `dp_es`, `hru_ps`, `hru_es`	`hydrology.wet`	Wetland depths (mm) and HRU fractions at the principal and emergency levels
release table	`res_rel.dtl`	Release rules

Related pages

- Reservoir inputs and wetland inputs
- Decision tables and the `*.dtl` files
- Channels, Aquifers
- Aquifer, reservoir and plant output

Sources

Code at revision 62.0.1:

- Reservoir daily driver: `src/res_control.f90`
- Release options: `src/res_hydro.f90`; conditions tables: `src/res_rel_conds.f90`; weir release for paddies: `src/res_weir_release.f90`
- Initialisation, area coefficients and lags: `src/res_initial.f90`; reader: `src/res_read.f90`
- Sediment and nutrients: `src/res_sediment.f90`, `src/res_nutrient.f90`, `src/theta.f90`
- Types: `src/reservoir_module.f90`, `src/reservoir_data_module.f90`
- Wetlands: `src/wetland_control.f90`, `src/wet_initial.f90`; evaporation from standing water: `src/et_act.f90`

Columns and variables:

Column	Variable
`yr_op`, `mon_op`	`res_hyd%iyres`, `%mores`
`area_ps`, `vol_ps`, `area_es`, `vol_es`	`res_hyd%psa`, `%pvol`, `%esa`, `%evol`
`evap_co`, `k`	`res_hyd%evrsv`, `%k`
`shp_co1`, `shp_co2`	`res_hyd%br1`, `%br2`, copied to `res_ob%lag_up`, `%lag_down`
β_1, β_2	`res_ob%br1`, `res_ob%br2`
`nsed`, `d50`	`res_sed%nsed`, `%d50`; settling factor `res_prm%sed_stlr_co`
`mid_start`, `mid_end`, `mid_n_stl`, `n_stl`, `mid_p_stl`, `p_stl`	`res_nut%ires1`, `%ires2`, `%nsetlr1`, `%nsetlr2`, `%psetlr1`, `%psetlr2`
`chla_co`, `secchi_co` (positions 8 and 9)	`res_nut%nsolr`, `%psolr` (soluble N and P settling rates)
`theta_n`, `theta_p`, `n_min_stl`, `p_min_stl`	`res_nut%theta_n`, `%theta_p`, `%conc_nmin`, `%conc_pmin`
wetland `dp_ps`, `dp_es`, `k`	`wet_hyd%pdep`, `%edep`, `%k` (copied to `hru%wet_hc`)

Overview

Three object types put water and loads into the routing network without simulating the land surface that produced them, or change how much of a load reaches the next object:

- Recall objects release a time series of flow and loads read from a file: measured inflow at a gauge, a wastewater treatment plant, an industrial discharge, or the output of another model. A negative flow represents a diversion.
- Export coefficient objects release the same daily flow and loads every day of the run.
- Delivery ratios scale the loads leaving the elements of a routing unit, or the loads passing through a delivery ratio object.

Recall, export coefficient and delivery ratio objects are connected to the network through `recall.con`, `exco.con` and `delratio.con`, like any other spatial object.

Recall objects

Input structure

`recall.rec` (the file named in the `recall` line of `file.cio`) lists the recall objects. Each row has an id, a name, a time step code and the name of the data file:

Code	Data file content	Value used each day
0	sub-daily values	the values of the day's time steps
1	daily values	the value of the day
2	monthly values	the value of the month, on every day of that month
3	annual values	the value of the year, on every day of that year
4	none; the file name is the name of an export coefficient record	the export coefficient record

Each data file has a title line, a line with the number of years, a column header, and one row per day, month or year: day of year, month, day of month, year, object type, object name, and then the hydrograph values

```
flo sed orgn sedp no3 solp chla nh3 no2 cbod dox sand silt clay sag lag gravel temp
```

in m^3 for flow, metric tons for sediment and its classes, kg for nutrients, chlorophyll-a, CBOD and oxygen, and °C for temperature. The values are daily amounts, also in monthly and annual files: a monthly value is released on every day of the month, so it must be the daily amount for that month.

Rows before the simulation start year are skipped, and reading stops after the simulation end year.

Daily release

On each day inside the period covered by its file, a recall object sends the stored values for that day, month or year to the objects listed in `recall.con`. Outside that period a daily or monthly recall object releases nothing.

When a daily value has negative flow, the object takes water out of the network (a diversion). Nutrients are then removed in proportion to the diverted volume at the concentration of the source, as are salts and constituents when they are simulated.

Salts, constituents and pesticides

Salt and constituent loads for recall objects are read from separate salt and constituent recall files (see Salts and constituents). Pesticide loads for recall objects are read from `pest.com` when that file is present.

Recall database

recall_db.rec is a separate index that names, for each source, the organic-mineral, pesticide, pathogen, heavy metal, salt and constituent data. Water allocation uses it to set the quality of water from outside sources, treatment plants and water uses, and reservoir release tables use it to find measured releases.

Export coefficient objects

exco.exc lists the export coefficient objects. Each row has a name followed by the names of its records in the organic-mineral (exco_om.exc), pesticide, pathogen, heavy metal and salt files. Each row of exco_om.exc has a name followed by the same hydrograph values as a recall file.

The hydrograph of an export coefficient object is set once at the start of the run and released unchanged every day, so the values in exco_om.exc are daily amounts. An export coefficient can also be an element of a routing unit, in which case the routing unit's delivery ratio is applied to it.

Delivery ratios

Routing unit elements

Each element of a routing unit (an HRU, an HRU-lte, a channel or another routing unit) has a delivery ratio named in the `dlr` column of rout_unit.ele. The ratio multiplies the flow and loads the element contributes to the routing unit outflow.

`dlr` value	Ratio
`calc` or `0`	Flow is delivered in full. Sediment and nutrients are multiplied by $\min\left(1, \sqrt{t_{c,elem}/t_{c,ru}}\right)$, the square root of the ratio of the element's time of concentration to the routing unit's, for HRU and HRU-lte elements. For `sdc` channel and routing unit elements the ratio is 1. Other element types, including legacy channels, take the ratio last calculated for an earlier element (0 if there is none).
`full` or `1`	Everything is delivered in full.
any other value	0 for every variable, so the element delivers only its recharge. This includes the name of a delratio.del record: the names are looked up before delratio.del is read, so none is found.

The details are on Routing and landscape units.

Pesticide loads from routing unit elements are not scaled by the delivery ratio.

Delivery ratio objects

A delivery ratio object placed in the routing network multiplies all of its incoming hydrograph by the ratios of its delratio.del record and passes the result on.

Related pages

- Recall inputs and connection files
- Channels and Reservoirs and wetlands for receiving objects
- Water allocation
- Salts and constituents

Sources

Code at revision 62.0.1:

- Recall objects: src/recall_read.f90 (reads `recall.rec`, the data files and `pest.com`); daily release in the `recall` case of src/command.f90; diversions: src/recall_nut.f90, src/recall_salt.f90, src/recall_cs.f90

- Recall database: src/recalldb_read.f90, type in src/recall_module.f90
- Export coefficients: src/exco_db_read.f90, src/exco_read_om.f90, type in src/exco_module.f90
- Delivery ratios: src/dr_ru.f90, src/ru_read_elements.f90, src/ru_control.f90, src/dr_db_read.f90
- Hydrograph fields: `hyd_output` in src/hydrograph_module.f90

Variables:

Item	Variable
recall time step code, file, first and last year	`recall%typ`, `%filename`, `%start_yr`, `%end_yr`
stored values	`recall%hd(day or month or 1, year)`, sub-daily flow in `recall%hyd_flo`
`exco.exc` row	`exco_db` (`name`, `om_file`, `pest_file`, `path_file`, `hmet_file`, `salts_file`)
`exco_om.exc` row	`exco_om_name`, `exco` (type `hyd_output`)
element delivery ratio	`ru_elem%dr_name`, `ru_elem%dr`

Overview

A decision table is a set of rules of the form "if these conditions are met, do these actions". SWAT+ uses them wherever an operation depends on the state of the model instead of a fixed date: irrigation when plants are water stressed, planting when the soil is warm enough, fertilising only a share of the fields in a land use, releasing reservoir water by storage and season, or changing land use partway through a run.

There are four decision table files. They have the same format and are evaluated the same way.

File	Used for	Evaluated
lum.dtl	Conditional management of HRUs	daily, for each HRU whose management schedule lists the table as an automatic operation
res_rel.dtl	Reservoir and wetland releases	daily, for each reservoir or wetland that names the table as its release rule
scen_lu.dtl	Land use and management change scenarios	daily, for every HRU, for each table listed in `scen_dtl.upd`
flo_con.dtl	Flow control (water allocation, saturated buffers)	when the controlled object is processed

Structure

Each table has conditions, alternatives and actions.

- Conditions (rows) name a state variable, the object it is taken from, and a limit.
- Alternatives (columns) are rules. For each condition, an alternative gives a comparison operator (`<`, `>`, `<=`, `>=`, `=`, `/=`) or `-` when that condition does not matter for the rule.
- Actions (rows) say what to do. For each alternative, an action has an outcome `y` (do it if this rule is true) or `n`.

An example from `refdata/Ames_sub1/lum.dtl` irrigates with a sprinkler when the plant is water stressed, has accumulated more than 20 % of its heat units and has a leaf area index above 0.5, as long as less than 500 mm has been applied in the year:

```
name                  conds      alts      acts
irr_year_irr            4         1         1
var                    obj    obj_num          lim_var           lim_op  lim_const      alt1
irr_year               hru          0             null                -   500.00000         <
w_stress               hru          0             null                -   0.80000           <
phu_plant              hru          0          phu_mat                -     0.20000         >
leaf_area              hru          0          null                   -     0.50000         >
act_typ                obj    obj_num             name           option    const        const2
irrigate               hru          0         sprinkler     sprinkler_ilm    25.00000   20.00000  unli
```

An object number of 0 means the object the table is being evaluated for (the current HRU, reservoir or wetland).

Evaluation

Each time a table is evaluated:

1. Every alternative starts as true.
2. For each condition, the current value of the variable is compared with the limit. An alternative whose operator for that condition is not satisfied becomes false. A `-` leaves the alternative unchanged.
3. Each action is taken if at least one alternative that is still true has `y` for that action.

Any number of alternatives can be true at once, and an action shared by several alternatives is taken once.

Limits

The limit a variable is compared with is built from three columns: `lim_var`, `lim_op` and `lim_const`. For variables measured against a property of the object, `lim_var` names the property and `lim_op` combines it with the constant. For example, for `soil_water` the property can be `wp` (taken as 0), `fc` (profile water at field capacity) or `ul` (profile water at saturation), and `fc * 0.8` is written as `lim_var = fc`, `lim_op = *`, `lim_const = 0.8`. For reservoir volume, the property can be `pvol` or `evol`. For variables compared with a plain number, `lim_var` is `null` and only `lim_const` is used.

Condition variables

Group	Variables
Plant stress and growth	`w_stress`, `n_stress`, `p_stress`, `phu_plant`, `phu_base0`, `plant_gro`, `plant_name_gro`, `biomass`, `leaf_area`, `ground_cov`, `cur_yrs_mat`, `plant_com`
Time	`jday`, `month`, `year_rot`, `year_gro`, `year_cal`, `year_seq`, `year_start`, `days_plant`, `days_harv`, `days_irr`, `days_act`, `day_start`
Weather	`precip_cur`, `precip_next`, `p_pet`
Soil and land	`soil_water`, `soil_temp2`, `soil_carbon`, `p_lab_150`, `slope`, `hyd_soil_group`, `p_factor`, `k_factor`, `tile_drained`, `tillage`, `land_use`, `cal_group`
Water	`irr_year`, `irr_demand`, `irr_pou_dmd`, `tile_flo`, `channel_flo`, `aqu_dep`, `ch_order`
Water bodies	`vol`, `res_inflo`, `wet_depth`, `vol_wet`, `wtow_vol`
Probability	`prob`, `prob_unif`, `prob_unif1`, `prob_unif_lu`

Probabilistic conditions

Probability conditions make actions happen on a random day or on a random share of HRUs:

- `prob` compares a uniform random number with `lim_const`, so the action happens on that fraction of evaluations.
- `prob_unif` picks one random day in the window from day `obj_num` to day `lim_const` of the year, so the action happens once in the window.
- `prob_unif1` spreads the probability evenly over a window of `lim_const` days that starts on 1 January of the calendar year given in `obj_num`.
- `prob_unif_lu` spreads an operation over the HRUs of a land use (named in a `land_use` condition) during the window from day `obj_num` to day `lim_const`, so that the share of HRUs treated grows evenly through the window. This is how an operation such as “fertilise 30 % of corn fields in May” is set up.

Actions

Group	Action types
Crops and soil	`plant`, `harvest`, `kill`, `harvest_kill`, `till`, `fertilize`, `manure`, `fert_future`, `pest_apply`, `graze`, `burn`, `grow_init`, `grow_end`, `puddle`, `herd`
Irrigation and demand	`irrigate`, `irr_demand`, `manure_demand`, `wallo_dmd`, `set_dmd`, `irr_dmd`, `res_irr_dmd`, `fill_res`, `fill_wtow`, `fill_can`, `duty_reset`, `duty_fr`
Counters	`rot_reset`, `harv_reset`, `kill_reset`, `plant_reset`, `irr_reset`, `pheno_reset`
Land use and practices	`lu_change`, `hru_fr_update`, `cn_update`, `p_factor`, `contour`, `stripcrop`, `terrace`, `tile_install`, `septic_install`, `fstrip_install`, `grassww_install`, `grassww_uninstall`, `user_def_bmp`, `snow_change`, `chan_change`
Water control	`tileflo_control`, `tiledep_control`, `impound_on`, `impound_off`, `weir_height`
Releases	`release` with the options described in Reservoirs and wetlands

For management actions, `name` points to a record in the matching database (a plant, fertiliser, tillage implement, harvest operation, irrigation operation and so on), and `option`, `const` and `const2` give the amount or method. The `fp` column names an extra file record or object, depending on the action.

Related pages

- `*.dtl` input files
- Management for management schedules and automatic operations
- Reservoirs and wetlands for release options
- Plant growth and land use management
- Water allocation

Sources

Code at revision 62.0.1:

- Types `conditions_var`, `actions_var`, `decision_table` and the arrays `dtbl_lum`, `dtbl_res`, `dtbl_scen`, `dtbl_flo`: `src/conditional_module.f90`
- Readers: `src/dtbl_lum_read.f90`, `src/dtbl_res_read.f90`, `src/dtbl_scen_read.f90`, `src/dtbl_flocon_read.f90`; scenario activation: `src/cal_cond_read.f90` (`scen_dtl.upd`)
- Condition evaluation: `src/conditions.f90`, `src/cond_real.f90`, `src/cond_integer.f90`
- Actions: `src/actions.f90`; release actions: `src/res_hydro.f90`
- Call sites: automatic operations in `src/hru_control.f90`, scenarios in `src/time_control.f90`, reservoirs in `src/res_control.f90`, wetlands in `src/wetland_control.f90`

Columns and variables:

Column	Variable
`var`, `obj`, `obj_num`, `lim_var`, `lim_op`, `lim_const`	`d_tbl%cond%var`, `%ob`, `%ob_num`, `%lim_var`, `%lim_op`, `%lim_const`
`alt1` …	`d_tbl%alt(cond, alt)`
`act_typ`, `obj`, `obj_num`, `name`, `option`, `const`, `const2`, `fp`	`d_tbl%act%typ`, `%ob`, `%ob_num`, `%name`, `%option`, `%const`, `%const2`, `%file_pointer`
outcomes	`d_tbl%act_outcomes(act, alt)`
alternative still true	`d_tbl%act_hit(alt)`

This section describes the SWAT+ input files for revision 62.0.1. Each page gives the purpose of a file, its columns with units and defaults, how the model uses the values, an example from the reference datasets, and the files it is linked to. Where the code behaves in a way a user would not expect (a column that is read but has no effect, a default that differs from the one SWAT+ Editor writes), the page says so.

Every page was checked against the SWAT+ source at the commit recorded as `verified_at_commit` in its front matter. Where the source and older documentation disagree, the pages follow the source. The last section of each page, Sources, links to the Fortran routines and lists the variable names for developers.

file.cio names every other input file, so it is the place to start.

Older projects

A project set up for an earlier revision needs these changes before it runs correctly with 62.0.1:

- The `basin` line of file.cio now has three file names, and the `water_rights` line has two. An old `basin` line shifts every later line of file.cio by one without a message. See older file.cio files.
- The dynamic carbon model is now `carbon = 2` in codes.bsn. With `carbon = 1` no organic matter routine runs. See carbon model.
- carb_coefs.cbn is no longer read. Its settings moved to carbon.bsn and carbon_lyr.bsn, and its output switch to print.prt.

Simulation control

- file.cio: master index. Names the input file for every category, the weather data folders and the output folder.
- time.sim: simulation period and number of time steps per day.
- print.prt: warm-up years, daily print window and the output files written at each interval.
- object.cnt: basin name, areas and the number of spatial objects of each type.

Basin parameters

- codes.bsn: basin-wide method codes (PET method, channel routing, carbon model, runoff method and others).
- parameters.bsn: basin-wide coefficients for runoff, peak rate, nutrients, pesticides, routing, lapse rates, erosion and CO_2.
- carbon.bsn: parameters of the dynamic soil carbon model.
- carbon_lyr.bsn: decomposition rates and CO_2 fractions of the dynamic soil carbon model, for the surface layer and the deeper layers.

Spatial objects and connectivity

- Connect files: hru.con, rout_unit.con, chandeg.con, aquifer.con, reservoir.con, recall.con and the other connect files, which place each object, assign its weather station and route its outflow.
- hru-data.hru: the records each HRU uses for topography, hydrology, soil, land use, initial conditions, surface storage, snow and field geometry.
- Hydrology and topography: hydrology.hyd, topography.hyd, field.fld and `shade_factor.shf`.
- Recall: point sources and inlets given as time series.
- hru-lte.hru: the simplified HRU-lte objects.
- Routing and landscape units: rout_unit.def, rout_unit.ele, rout_unit.rtu, ls_unit.def, ls_unit.ele and aqu_catunit.ele.
- Link files: chan-surf.lin for overbank flow to floodplain HRUs and aqu_cha.lin for aquifer return flow to several channels.

Climate

- Climate files: weather stations (weather-sta.cli), weather generator statistics (weather-wgn.cli), the lists of observed data files and atmospheric deposition.
- *.pcp: measured daily or sub-daily precipitation for one gauge.
- *.tmp: measured daily maximum and minimum air temperature for one gauge.
- *.tem: temperature files with the `.tem` extension, read like `.tmp` files.
- snow.sno: snowfall, snowmelt and snow cover parameters.

Soils and land use

- soils.sol: soil profiles and layer properties.
- landuse.lum: land use and management records used by HRUs.
- Land use tables: curve number, conservation practice and overland flow Manning's n tables named in landuse.lum.

Channels

- Channel group: the channel line of file.cio and which of its files are used, initial channel contents, stream temperature and the legacy channel files.
- Chandeg channel files: channel-lte.cha, hyd-sed-lte.cha, sed_nut.cha and nutrients.cha, with the geometry, hydraulic, sediment and water quality parameters of the channels SWAT+ routes.

Aquifers, reservoirs and wetlands

- Aquifer files: aquifer.aqu and its initial conditions.
- Reservoir files: reservoir.res and its initial condition, hydrology, sediment, nutrient and weir databases.
- Wetland files: wetland.wet and hydrology.wet, for HRUs with ponded water including rice paddies.

Management

- Operation databases: harvest, grazing, irrigation, chemical application, fire and street sweeping.
- Decision tables: lum.dtl, res_rel.dtl, scen_lu.dtl and flo_con.dtl.
- plants.plt: plant parameter database.
- fertilizer.frt: fertilizer and manure databases.
- tillage.til: tillage implements and biological mixing.
- carb_coefs.cbn: no longer read in 62.0.1.
- treatment.trt: placeholder file, not read.
- Manure allocation: manure sources and the HRUs that draw from them.
- Management schedules and other inputs: management.sch, puddle.ops, transplant.ops, co2_yr.dat and object.prt.

Initial conditions

- Initialization files: plant.ini, soil_plant.ini, nutrients.sol and om_water.ini. Pesticide, pathogen, salt and constituent initial conditions are on the pages for those modules.

Structural practices

- Structural practice files: tile drains, septic systems, filter strips, grassed waterways, user-defined practices, the septic and urban databases, and saturated buffers.

Pollutants

- Pesticides: pesticide.pes, initial pesticide amounts and the metabolite chain.
- Pathogens: pathogens.pth and initial pathogen amounts.
- Heavy metals: heavy metal inputs.

Groundwater

- gwflow: the cell-based groundwater module, used in place of the lumped aquifers when gwflow = 1 in codes.bsn.

Salts and constituents

- Salt module: inputs for the eight major salt ions.
- Constituent module: `constituents.cs` and the inputs for selenate, selenite and boron.

Water allocation

- Water allocation: places of use, points of diversion and return, and the treatment, use, tower, canal, pipe and outside-basin files.

  Sources

  - File names and their order in `file.cio`: `readcio_read.f90` and `input_file_module.f90`.
  - Each page lists the readers and the routines that use its inputs in its Sources section.

Purpose

`file.cio` is the master index of a SWAT+ project. It is the first file SWAT+ opens, and it must be in the run folder under that name. Each line names the files for one input category, so every other input file is reached through `file.cio`. The last six lines give optional folders for the weather data files and for the output files.

Format

Line 1 is a title and is skipped. Lines 2 to 26 each start with a category label and continue with file names in a fixed order. Lines 27 to 31 each give one weather data folder, and line 32 gives the output folder.

The reader takes the lines by position. The category label is read and discarded, so its text is not checked, and the file names are matched to slots by their order on the line. Values are separated by spaces.

- Write `null` for a file the project does not use. Most readers treat a name that does not exist in the run folder the same way as `null` and skip the file. The exceptions are listed under Missing and null files.
- File names can be up to 25 characters long. Longer names are cut to 25 characters.
- Each line must give at least as many names as its category has slots. If a line is short, the reader continues onto the next line to fill the missing slots, and every later line is then read one line out of step. Extra names at the end of a line are ignored.

Categories and default file names

The labels below are the ones the SWAT+ Editor writes. The default names are used only for slots that are never read, which happens when `file.cio` ends early. A project normally gives every name explicitly.

Line	Label	Files, in slot order (default names)	Documented in
2	`simulation`	`time.sim`, `print.prt`, `object.prt`, `object.cnt`, `constituents.cs`	time.sim, print.prt, object.prt, object.cnt, constituents.cs
3	`basin`	`codes.bsn`, `parameters.bsn`, `carbon.bsn`	codes.bsn, parameters.bsn, carbon.bsn
4	`climate`	`weather-sta.cli`, `weather-wgn.cli`, `pet.cli`, `pcp.cli`, `tmp.cli`, `slr.cli`, `hmd.cli`, `wnd.cli`, `atmodep.cli`	climate index files
5	`connect`	`hru.con`, `hru-lte.con`, `rout_unit.con`, `gwflow.con`, `aquifer.con`, `aquifer2d.con`, `channel.con`, `reservoir.con`, `recall.con`, `exco.con`, `delratio.con`, `outlet.con`, `chandeg.con`	connectivity files
6	`channel`	`initial.cha`, `channel.cha`, `hydrology.cha`, `sediment.cha`, `nutrients.cha`, `channel-lte.cha`, `hyd-sed-lte.cha`, `temperature.cha`	legacy channel files, channel tables
7	`reservoir`	`initial.res`, `reservoir.res`, `hydrology.res`, `sediment.res`, `nutrients.res`, `weir.res`, `wetland.wet`, `hydrology.wet`	reservoir files, wetland files
8	`routing_unit`	`rout_unit.def`, `rout_unit.ele`, `rout_unit.rtu`, `rout_unit.dr`	connectivity files
9	`hru`	`hru-data.hru`, `hru-lte.hru`	hru-data.hru

Line	Label	Files, in slot order (default names)	Documented in
10	exco	exco.exc, exco_om.exc, exco_pest.exc, exco_path.exc, exco_hmet.exc, exco_salt.exc	connectivity files
11	recall	recall.rec	recall
12	dr	delratio.del, dr_om.del, dr_pest.del, dr_path.del, dr_hmet.del, dr_salt.del	connectivity files
13	aquifer	initial.aqu, aquifer.aqu	aquifer files
14	herd	animal.hrd, herd.hrd, ranch.hrd	not read
15	water_rights	place_of_use.wro, point_of_diver.wro	water allocation
16	link	chan-surf.lin, aqu_cha.lin	Link files
17	hydrology	hydrology.hyd, topography.hyd, field.fld	hydrology and topography
18	structural	tiledrain.str, septic.str, filterstrip.str, grassedww.str, bmpuser.str	structural BMPs
19	hru_parm_db	plants.plt, fertilizer.frt, tillage.til, pesticide.pes, pathogens.pth, metals.mtl, salt.slt, urban.urb, septic.sep, snow.sno	plants.plt, fertilizer.frt, tillage.til, pesticide.pes, pathogens.pth, metals.mtl, salt inputs, urban.urb and septic.sep, snow.sno
20	ops	harv.ops, graze.ops, irr.ops, chem_app.ops, fire.ops, sweep.ops	operation databases
21	lum	landuse.lum, management.sch, cntable.lum, cons_practice.lum, ovn_table.lum	landuse.lum, management.sch, lookup tables
22	chg	cal_parms.cal, calibration.cal, codes.sft, wb_parms.sft, water_balance.sft, ch_sed_budget.sft, ch_sed_parms.sft, plant_parms.sft, plant_gro.sft	parameter change files, soft calibration
23	init	plant.ini, soil_plant.ini, om_water.ini, pest_hru.ini, pest_water.ini, path_hru.ini, path_water.ini, hmet_hru.ini, hmet_water.ini, salt_hru.ini, salt_water.ini	initial conditions
24	soils	soils.sol, nutrients.sol, soils_lte.sol	soils.sol
25	decision_table	lum.dtl, res_rel.dtl, scen_lu.dtl, flo_con.dtl	decision tables
26	regions	ls_unit.ele, ls_unit.def, ls_reg.ele, ls_reg.def, ls_cal.reg, ch_catunit.ele, ch_catunit.def, ch_reg.def, aqu_catunit.ele, aqu_catunit.def, aqu_reg.def, res_catunit.ele, res_catunit.def, res_reg.def, rec_catunit.ele, rec_catunit.def, rec_reg.def	aquifer files, recall, soft calibration
27	pcp_path	folder for the precipitation data files	precipitation data
28	tmp_path	folder for the temperature data files	temperature data
29	slr_path	folder for the solar radiation data files	climate index files
30	hmd_path	folder for the relative humidity data files	climate index files
31	wnd_path	folder for the wind speed data files	climate index files
32	any word	output folder	Output folder

Unused slots

These names are read from `file.cio`, but SWAT+ 62.0.1 never opens the file they name. Any name or `null` can be given.

- `connect`: aquifer2d.con (slot 6).
- `routing_unit`: `rout_unit.dr` (slot 4).
- `herd`: all three slots.
- `hru_parm_db`: metals.mtl and `salt.slt` (slots 6 and 7).
- `init`: `hmet_water.ini`, salt_hru.ini and `salt_water.ini` (slots 9 to 11).
- `regions`: `ls_cal.reg` and `ch_catunit.ele` (slots 5 and 6).

Two of these files are read under a fixed name instead. When salts are simulated, the HRU salt initial conditions come from a file named salt_hru.ini in the run folder, whatever slot 10 of the `init` line says. Channel catchment unit elements come from a file named `element.ccu`.

The `climate` line has no wind direction file. It has nine slots, and slot 3 is pet.cli.

Missing and null files

For most slots, a name of `null` or a name that does not exist in the run folder means the file is skipped and the related feature is off. Some files behave differently:

- `object.cnt`: if it is missing or `null`, SWAT+ prints `Cannot find object.cnt input file` and stops.
- `carbon.bsn`: read only when `carbon = 2` in `codes.bsn`. It is then required, and SWAT+ stops with an error, written to the screen and to `diagnostics.out`, if the file named in slot 3 of the `basin` line does not exist or cannot be opened. The per-layer carbon file has no slot on the line: its name is built from the carbon file name, so `carbon.bsn` gives `carbon_lyr.bsn`.
- `time.sim` and `print.prt`: there is no check for a missing file. If the name is not `null` and the file does not exist, SWAT+ creates an empty file of that name and continues with built-in values. For `time.sim` this leaves the run without a valid simulation period, so the file is required in practice.
- `place_of_use.wro` and `point_of_diver.wro`: if either is missing or `null`, SWAT+ creates an empty file of that name (a file called `null` in the second case), reads nothing from it, and water allocation is off. See water allocation.
- `chan-surf.lin`: if the name is not `null` and the file does not exist, SWAT+ creates an empty file of that name. See link files.

Weather data folders

Lines 27 to 31 let the precipitation, temperature, solar radiation, humidity and wind data files be kept outside the run folder. Each line has a label and one value of up to 80 characters. A value of `null` means the data files are in the run folder.

SWAT+ joins the folder and the file name from the climate index file (for example `pcp.cli`) without adding a separator, so the folder must end with `/` or `\`. The folder is read in the same way as a file name, with two consequences:

- A folder name that contains spaces must be in quotes.
- An unquoted `/` ends the value. `data/pcp/` is read as `data`, and an unquoted value that starts with `/` is read as empty, so the files are looked for in the run folder. Put any folder that uses forward slashes in quotes, for example `"C:/swat/weather/pcp/"`, or use backslashes on Windows.

There is no folder line for potential evapotranspiration data. Files listed in `pet.cli` are always read from the run folder.

Output folder

Line 32 is optional. It has a label (any word) and a folder name. SWAT+ reads the whole line, drops the first word and takes the rest of the line as the folder, so the folder name can contain spaces and forward slashes without quotes. Up to 256 characters are kept. If the line is missing or empty, or the folder is `null` or `NULL`, the output files are written to the run folder.

When a folder is given, SWAT+ converts the separators to the ones the operating system uses, creates the folder and any missing parent folders, and prints `Output path set to:` with the folder name. A relative folder is taken from the run folder. On Linux and macOS a folder that contains `:` is treated as a Windows path, and SWAT+ stops with an error. On Windows a folder that starts with `/` is placed on drive `C:`. SWAT+ also stops if it cannot create or open the folder.

The standard output files go to this folder, including `files_out.out`, `diagnostics.out`, `checker.out` and the time series outputs switched on in `print.prt`. In 62.0.1 the gwflow output files and the soft calibration output files (`hru-out.cal`, `hru-new.cal` and others) are still written to the run folder.

Older file.cio files

Two lines changed length in recent revisions, and a `file.cio` written for an earlier revision needs both updated before it is run with 62.0.1:

- The `basin` line now has three slots. Add `carbon.bsn` or `null` after `parameters.bsn`. With only two names on the `basin` line, SWAT+ takes the first word of line 4 (`climate`) as the carbon file name and discards the rest of that line. Every later category is then read from the line below the one it belongs to: the climate files are taken from the `connect` line, the connection files from the `channel` line, and so on. No message reports the shift. If `carbon = 2` in `codes.bsn`, SWAT+ stops with `ERROR: climate is required when carbon is enabled`; otherwise the run continues with wrong file names in every later slot.
- The `water_rights` line (line 15) now has two slots, the place-of-use file and the point-of-diversion file. An older line with `water_allocation.wro element.wro water_rights.wro` makes SWAT+ read `water_allocation.wro` as the place-of-use file and `element.wro` as the point-of-diversion file, and the third name is ignored. See water allocation for those files.

Example

`refdata/Osu_1hru/file.cio` in the SWAT+ repository (long lines cut at the right):

```
file.cio: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
simulation        time.sim          print.prt         null              object.cnt        null
basin             codes.bsn         parameters.bsn    null
climate           weather-sta.cli   weather-wgn.cli   null              pcp.cli           tmp.cli
connect           hru.con           null              rout_unit.con     null              aquifer.con
channel           initial.cha       null              null              null              nutrients.cha
reservoir         initial.res       null              null              sediment.res      nutrients.res
routing_unit      rout_unit.def     rout_unit.ele     rout_unit.rtu     null
hru               hru-data.hru      null
exco              null              null              null              null              null
recall            recall.rec
dr                null              null              null              null              null
aquifer           initial.aqu       aquifer.aqu
herd              null              null              null
water_rights      null              null              null
link              null              null
hydrology         hydrology.hyd     topography.hyd    field.fld
structural        tiledrain.str     septic.str        filterstrip.str   grassedww.str     bmpuser.str
hru_parm_db       plants.plt        fertilizer.frt    tillage.til       pesticide.pes     null
ops               harv.ops          graze.ops         irr.ops           chem_app.ops      fire.ops
lum               landuse.lum       management.sch    cntable.lum       cons_practice.lum  ovn_table.lum
chg               cal_parms.cal     calibration.cal   null              null              null
init              plant.ini         soil_plant.ini    om_water.ini      null              null
soils             soils.sol         nutrients.sol     null
decision_table    lum.dtl           res_rel.dtl       null              null
regions           ls_unit.ele       ls_unit.def       null              null              null
pcp_path          null
tmp_path          null
slr_path          null
hmd_path          null
wnd_path          null
```

The `basin` line has three slots, with `null` for the carbon file because the project does not use the carbon model. The `climate` line uses measured precipitation, temperature, solar radiation and wind; the PET, humidity and atmospheric deposition files are not used. The `water_rights` line still has the older three slots, all `null`; the first two switch water allocation off and the third is ignored. All weather data files are in the run folder, and there is no output folder line, so the outputs are written to the run folder.

In `refdata/Ames_sub1/file.cio` the `aquifer` line reads `aquifer nbull null`. `nbull` is taken as a file name. No file of that name exists, so the initial aquifer file is skipped in the same way as for `null`.

Related files

- time.sim, print.prt and object.cnt: the simulation control files named on line 2.
- codes.bsn: its `carbon` code decides whether the carbon file on line 3 is read.
- Input reference: every input file reached from `file.cio`.

Sources

- Reader: readcio_read.f90, called first from proc_bsn.f90
- Slot names and defaults: input_file_module.f90
- Output folder: output_path_module.f90
- Weather folders: cli_pmeas.f90, cli_tmeas.f90, cli_smeas.f90, cli_hmeas.f90, cli_wmeas.f90, cli_petmeas.f90
- Carbon file check: carbon_bsn_read.f90
- Water allocation files: water_allocation_read.f90
- Fixed-name readers: ch_read_elements.f90 (`element.ccu`), salt_hru_read.f90 (`salt_hru.ini`)

Line	Fortran variable
2 `simulation`	`in_sim` (`time`, `prt`, `object_prt`, `object_cnt`, `cs_db`)
3 `basin`	`in_basin` (`codes_bas`, `parms_bas`, `carbon_bsn`)
4 `climate`	`in_cli`
5 `connect`	`in_con`
6 `channel`	`in_cha`
7 `reservoir`	`in_res`
8 `routing_unit`	`in_ru`
9 `hru`	`in_hru`
10 `exco`	`in_exco`
11 `recall`	`in_rec`
12 `dr`	`in_delr`
13 `aquifer`	`in_aqu`
14 `herd`	`in_herd`
15 `water_rights`	`in_wallo` (`pou`, `pod`)
16 `link`	`in_link`
17 `hydrology`	`in_hyd`
18 `structural`	`in_str`
19 `hru_parm_db`	`in_parmdb`
20 `ops`	`in_ops`
21 `lum`	`in_lum`
22 `chg`	`in_chg`
23 `init`	`in_init`
24 `soils`	`in_sol`
25 `decision_table`	`in_cond`
26 `regions`	`in_regs`
27 to 31	`in_path_pcp%pcp`, `in_path_tmp%tmp`, `in_path_slr%slr`, `in_path_hmd%hmd`, `in_path_wnd%wnd`
32	`out_path` in `output_path_module`

Purpose

`time.sim` sets the first and last day of the simulation and the number of time steps per day. Weather input, routing and every output file follow this period. The file is named in slot 1 of the `simulation` line of file.cio (default `time.sim`).

Format

Free format, whitespace-separated. Line 1 is a title and line 2 is the column header; both are skipped. Line 3 contains five integers in the order below. The reader does not match columns by header name.

#	Column	Units	Default	Description
1	`day_start`	day of year	1	Day of the year on which the simulation starts, 1 to 366. A value of 0 or less is set to 1 (1 January).
2	`yrc_start`	year	-	Calendar year in which the simulation starts.
3	`day_end`	day of year	0	Day of the year on which the simulation ends in `yrc_end`. A value of 0 or less means the last day of that year (31 December).
4	`yrc_end`	year	-	Calendar year in which the simulation ends.
5	`step`	time steps per day	1	Number of time steps per day for precipitation, surface runoff and routing. A value of 0 or less is set to 1 (daily).

Simulation period

The simulation runs over the calendar years `yrc_start` to `yrc_end`, which is

$$N_{yr} = \text{yrc_end} - \text{yrc_start} + 1$$

years. The first year starts on `day_start`, and every later year starts on 1 January. Every year but the last runs to 31 December. The last year ends on `day_end`, or on 31 December if `day_end` is 0. A `day_end` of 366 in a year that is not a leap year is treated as day 365. Leap years follow the Gregorian rule: years divisible by 4 are leap years, except century years that are not divisible by 400.

`day_start` and `day_end` make it possible to simulate water years or other periods that do not follow the calendar year. For example, `day_start = 274`, `yrc_start = 1999`, `day_end = 273` and `yrc_end = 2019` runs from 1 October 1999 to 30 September 2019. Neither year is a leap year, so day 274 is 1 October and day 273 is 30 September; in a leap year the same dates are days 275 and 274.

The warm-up years and the print window for daily output are set in `print.prt`. A zero `yrc_start` or `yrc_end` in `print.prt` is filled from this file.

Time step

`step` is the number of time steps in a day:

`step`	Time step
0 or 1	1 day
2	12 hours
24	1 hour
96	15 minutes
1440	1 minute

Any positive whole number works. The length of one time step in minutes is

$$\Delta t = \frac{1440}{\text{step}}$$

With `step` greater than 1, the day is split into sub-daily steps for precipitation, infiltration and surface runoff, hydrograph routing through channels and reservoirs, and sub-daily channel output. The Green and Ampt infiltration method (`gampt = 1` in `codes.bsn`) uses the sub-daily rainfall. Other processes, such as plant growth, soil water and nutrient cycling, still run once a day.

Sub-daily precipitation records (*.pcp files with a time step above 0) must have the same number of values per day as `step`. SWAT+ stores `step` values per day and does not check the record against it.

When `step` is greater than 1 and precipitation is generated, the weather generator also distributes each day's rainfall over the time steps. For a station with measured daily records, 62.0.1 does not fill the sub-daily values: the daily total is read, but the rainfall in each time step stays at zero. A sub-daily run therefore needs sub-daily precipitation records or generated precipitation.

Weather outside the observed record

When the simulation starts before, or ends after, the measured daily weather records listed in the climate index files, SWAT+ generates the weather for the days outside the record with the weather generator (weather-wgn.cli). The same applies to days that are flagged as missing in a record.

Example

`refdata/Ames_sub1/time.sim` in the SWAT+ repository:

```
time.sim:
day_start  yrc_start   day_end   yrc_end      step
        0       1975         0      2020         0
```

The simulation runs from 1 January 1975 to 31 December 2020, 46 years. `day_start` and `day_end` are 0, so the first and last days of the year are used. `step` is 0, which SWAT+ sets to 1, so the simulation is daily.

Related files

- file.cio: names this file.
- print.prt: warm-up years and print window for the outputs inside this period.
- pcp.cli and the precipitation data files: their time step must match `step` for sub-daily records.
- codes.bsn: selects Green and Ampt infiltration, which uses sub-daily rainfall.

Sources

- Reader: time_read.f90
- Type definition: `type time_current` in time_module.f90
- Time step length: proc_bsn.f90
- Year and day loop, leap years, end day: time_control.f90
- Measured and generated precipitation: cli_pmeas.f90, cli_precip_control.f90
- File name slot: `in_sim%time` in input_file_module.f90

Column	Fortran variable
day_start	time%day_start
yrc_start	time%yrc_start
day_end	time%day_end
yrc_end	time%yrc_end
step	time%step

Purpose

`print.prt` controls the output of a SWAT+ run. It sets the number of warm-up years left out of the outputs, the period and spacing of daily output, the extra output formats, a few special output files, and one row per output group with switches for daily, monthly, yearly and average annual files. The file is named in slot 2 of the `simulation` line of file.cio (default `print.prt`).

If the name in file.cio is null, SWAT+ uses the built-in values: no warm-up, every output group off, and crop yields written as average annual values only. A name that does not exist in the run folder has the same effect, and SWAT+ creates an empty file of that name. The files listed in Output naming convention are written in every case.

Format

Free format, whitespace-separated. The file has four short blocks, each a header line followed by a line of values, then a header line and one row per output group. Line 1 is a title. All header lines are skipped, and values are read by position within each line.

Line	Content
1	title
2	header
3	nyskip, day_start, yrc_start, day_end, yrc_end, interval
4	header
5	aa_int_cnt, followed by aa_int_cnt end years
6	header
7	csvout, use_obj_lbls, cdfout
8	header
9	crop_yld, mgtout, hydcon, fdcout
10	header (objects daily monthly yearly avann)
11 and on	output group rows

Print period (line 3)

#	Column	Units	Default	Description
1	nyskip	years	0	Number of years at the start of the simulation left out of all outputs (warm-up period).
2	day_start	day of year	1	Day of the year on which daily output starts. 0 means 1.
3	yrc_start	year	first simulated year	Year in which daily output starts. 0 means the first year in time.sim.
4	day_end	day of year	366	Day of the year on which daily output ends. 0 means 366, the end of the year.
5	yrc_end	year	after the last simulated year	Year in which daily output ends. 0 means daily output runs to the end of the simulation.
6	interval	days	1	Days between daily output rows. 0 or less means 1. With 2, every other day is written.

A warm-up period lets soil water, nutrient pools and plant cover settle from their initial values before results are used. It matters more for short simulations; for a simulation of five years or less, a warm-up period is recommended.

Average annual intervals (line 5)

#	Column	Units	Default	Description
1	aa_int_cnt	count	0	Number of average annual print intervals. When it is above 0, the end year of each interval follows on the same line, in order.

For example, `3 1995 2005 2015` would end intervals in 1995, 2005 and 2015. In 62.0.1 the end years are read but have no effect on the outputs: every average annual file covers the whole printed period. See Average annual values.

Output formats (line 7)

#	Column	Values	Default	Description
1	csvout	y, n	n	y also writes a comma-separated .csv copy of most output files.
2	use_obj_lbls	y, n	n	How the output group rows are read: y by name, n by position. See Output group rows.
3	cdfout	y, n	n	netCDF output. Read and not used.

print.prt files written by older versions of the SWAT+ Editor have the header csvout dbout cdfout on line 6. SWAT+ reads the second value as use_obj_lbls whatever the header says, so an old file with n there is read by position.

Special outputs (line 9)

#	Column	Values	Default	Description
1	crop_yld	a, y, b, n	a	Crop yield files: a average annual (crop_yld_aa), y yearly (crop_yld_yr and the basin crop yield files), b both, n none.
2	mgtout	y, n	n	y writes mgt_out.txt, the log of management operations, decision table actions and water allocation deliveries.
3	hydcon	y, n	n	y writes hydcon.out, the list of hydrograph connections between objects. It is mostly useful for finding connection errors.
4	fdcout	y, n	n	y computes flow duration curves and a flashiness index for SWAT+ channels and writes them to flow_duration_curve.out.

Older Editor versions write the header soilout in place of crop_yld; the value is read as crop_yld.

Output group rows

Each row from line 11 on has a group name and four switches, for the daily, monthly, yearly and average annual files of that group. A switch of y opens the file for that interval and any other value leaves it off. The legacy carbon rows hru_cb and hru_cb_vars also accept l. The files each row opens, and the pages that describe them, are listed in Output naming convention.

How the rows are read depends on use_obj_lbls on line 7:

- use_obj_lbls = y: each row is matched by its name, so rows can be in any order and rows can be left out. A row that is left out has all four switches off. A name SWAT+ does not know stops the run with Error: The output object <name> in the input file print.prt is not a valid object, and a name that appears twice stops it with print object is duplicated. Names are case-sensitive.
- use_obj_lbls = n: the rows are read in the fixed order of the table below, and the names in the first column are not checked. A missing row or an extra row shifts every later row, so its switches are applied to the wrong output group, with no message. Rows after the end of the file are off. The carbon and gwflow rows cannot be set in this mode.

Order	Row	Output
1	basin_wb	basin water balance
2	basin_nb	basin nutrient balance
3	basin_ls	basin losses from the land

Order	Row	Output
4	`basin_pw`	basin plant and weather
5	`basin_aqu`	basin aquifers
6	`basin_res`	basin reservoirs
7	`basin_cha`	basin legacy channels
8	`basin_sd_cha`	basin SWAT+ channels, channel morphology and channel budget
9	`basin_psc`	basin point sources (recall objects summed)
10 to 17	`region_wb`, `region_nb`, `region_ls`, `region_pw`, `region_aqu`, `region_res`, `region_sd_cha`, `region_psc`	read, but no region files are written in 62.0.1
18	`water_allo`	water allocation, water use and treatment
19 to 22	`lsunit_wb`, `lsunit_nb`, `lsunit_ls`, `lsunit_pw`	landscape unit water balance, nutrient balance, losses, plant and weather
23 to 26	`hru_wb`, `hru_nb`, `hru_ls`, `hru_pw`	HRU water balance, nutrient balance (with the nitrogen cycle file), losses, plant and weather
27 to 30	`hru-lte_wb`, `hru-lte_nb`, `hru-lte_ls`, `hru-lte_pw`	HRU-lte water balance, losses, plant and weather; `hru-lte_nb` is read but writes nothing
31	`channel`	legacy channels
32	`channel_sd`	SWAT+ channels, channel morphology, channel budget, and the sediment budget files per channel, per stream order and for the basin
33	`aquifer`	aquifers
34	`reservoir`	reservoirs and wetlands
35	`recall`	recall objects (point sources and inlets)
36	`hyd`	inflow and outflow hydrographs of every object, and deposition
37	`ru`	routing unit hydrographs
38	`pest`	pesticides in HRUs, channels, reservoirs, aquifers and the basin
39 to 45	`basin_salt`, `hru_salt`, `ru_salt`, `aqu_salt`, `channel_salt`, `res_salt`, `wetland_salt`	salts
46 to 52	`basin_cs`, `hru_cs`, `ru_cs`, `aqu_cs`, `channel_cs`, `res_cs`, `wetland_cs`	constituents; `res_cs` also opens the wetland constituent files, and `wetland_cs` writes nothing

These rows are recognised only when `use_obj_lbls = y`:

Row	Output
`hru_cb_gl`, `hru_cb_trf`, `hru_cb_lyr`, `hru_cb_cpool`, `hru_cb_npool`, `hru_cb_plt`, `hru_cb_flux`, `hru_cb_drv`, `hru_cb_dyn`, `hru_cb_snap`	HRU carbon output, one row per file family. See Carbon outputs.
`lsu_cb_gl`, `lsu_cb_trf`, `lsu_cb_plt`	landscape unit carbon output, area-weighted from the HRUs. See Carbon outputs.
`hru_cb`, `hru_cb_vars`	legacy CSU carbon files, switched with `y` or `1`. See Legacy CSU carbon outputs.
`gwflow_wb`, `gwflow_flux`, `gwflow_heat`, `gwflow_solute`, `gwflow_obs`, `gwflow_pump`	gwflow output. A row that is present replaces the setting from the gwflow input files; a row that is absent leaves it. See gwflow output switches.

Some rows write data only when the project has the matching objects. The `channel_sd` and `basin_sd_cha` files need SWAT+ channels, and the `channel` and `basin_cha` files need legacy channels. The basin landscape files (`basin_wb`, `basin_nb`, `basin_ls`, `basin_pw`) are summed from the landscape unit element file `ls_unit.ele`, and the landscape unit files need `ls_unit.def`; both are named on the `regions` line of `file.cio`.

Print timing

Every output file leaves out the first `nyskip` years of the simulation. After those years:

- Daily rows are written from `day_start` of `yrc_start` until `day_end` of `yrc_end`, every `interval` days, starting with the first day of the window. Output starts on the first simulated day whose day of the year is at least `day_start` in a year at or after `yrc_start`. If the simulation starts in a later year than `yrc_start`, daily output therefore starts on day `day_start` of the first simulated year. Days before it in that year are not written.
- Monthly rows are written at the end of each month, yearly rows at the end of each year, and average annual rows once, at the end of the simulation.

The print window applies to daily rows only. Monthly, yearly and average annual files always cover all years after the warm-up.

Average annual values

The average annual files give the sum over the printed period divided by the number of printed years Y. SWAT+ counts the printed days D (all days after the warm-up) and divides by 365:

$$Y = \frac{D}{365}$$

Leap days are counted in D, so Y is slightly larger than the number of calendar years. When `nyskip` is above 0, 62.0.1 starts the count at the number of simulated years N_{yr} instead of 0:

$$Y = \frac{D + N_{yr}}{365}$$

For a 10-year simulation with 2 warm-up years, Y is about 8.03 instead of 8.00, and the average annual values are about 0.3% lower than the true averages over the printed years. If `nyskip` is equal to or larger than the number of simulated years, no output is written.

Example

`refdata/Ames_sub1/print.prt` in the SWAT+ repository:

```
print.prt:
nyskip      day_start  yrc_start  day_end   yrc_end   interval
0           0               1975         0      2020          1
aa_int_cnt
0
csvout        use_obj_lbls  cdfout
n             y             n
crop_yld      mgtout        hydcon        fdcout
b             y             n             n
objects                  daily       monthly        yearly         avann
hru_wb                     y             n             n             y
hru_nb                     y             n             n             y
hru_ls                     n             n             n             y
hru_pw                     y             n             n             y
hru_cb                     l             y             y             y
hru_cb_vars                l             y             y             y
```

There is no warm-up period. Daily output covers the whole simulation (1975 to 2020) at an interval of one day. There is no `.csv` output. Crop yields are written both yearly and as average annual values, and the management log `mgt_out.txt` is written.

`use_obj_lbls = y`, so the rows are read by name and every row not listed is off. The HRU water balance, nutrient balance, and plant and weather files are written daily and as average annual values; the HRU losses only as average annual values. The `hru_cb` and `hru_cb_vars` rows switch on the legacy carbon files, with `l` in the daily column for

the full set of layer diagnostics. The `hru_cb_vars` files are written only when the carbon model is on (`carbon = 2` in `codes.bsn`).

`refdata/Osu_1hru/print.prt`, written by an older SWAT+ Editor, has `dbout` in place of `use_obj_lbls` with the value `n`, so its rows are read by position. Its list has an `hru_cb` row after `hru_pw`, which is not part of the fixed order: the reader applies that row to `hru-lte_wb` and every later row moves down one group. All rows in that file have the same switches (`n n n y`), so the shift has no visible effect there.

Related files

- `file.cio`: names this file, the output folder, and the landscape unit files the basin and landscape unit outputs need.
- `time.sim`: the simulation period that contains the print period.
- Output naming convention: file names and the files each row opens.
- Output reference: contents of each output file.

Sources

- Reader: `basin_print_codes_read.f90`, called from `proc_bsn.f90` after `time_read`
- Type definitions: `type basin_print_codes` and `type print_interval`, and `print_prt_error`, in `basin_module.f90`
- Print window, warm-up and average annual years: `time_control.f90`
- Output calls and the `nyskip` test: `command.f90`
- File opening: `output_landscape_init.f90`, `header_write.f90`, `header_sd_channel.f90`, `header_yield.f90`, `header_mgt.f90`, `header_hyd.f90`
- gwflow overrides: `gwflow_read.f90`
- Legacy carbon flags: `carbon_legacy_module.f90`
- Region output (not called): `lsreg_output.f90`

Column or row	Fortran variable
`nyskip`, `day_start`, `yrc_start`, `day_end`, `yrc_end`, `interval`	`pco%nyskip`, `pco%day_start`, `pco%yrc_start`, `pco%day_end`, `pco%yrc_end`, `pco%int_day`
`aa_int_cnt` and end years	`pco%aa_numint`, `pco%aa_yrs`
`csvout`, `use_obj_lbls`, `cdfout`	`pco%csvout`, `pco%use_obj_labels`, `pco%cdfout`
`crop_yld`, `mgtout`, `hydcon`, `fdcout`	`pco%crop_yld`, `pco%mgtout`, `pco%hydcon`, `pco%fdcout`
`basin_wb` … `basin_psc`	`pco%wb_bsn`, `nb_bsn`, `ls_bsn`, `pw_bsn`, `aqu_bsn`, `res_bsn`, `chan_bsn`, `sd_chan_bsn`, `recall_bsn`
`region_wb` … `region_psc`, `water_allo`	`pco%wb_reg`, `nb_reg`, `ls_reg`, `pw_reg`, `aqu_reg`, `res_reg`, `sd_chan_reg`, `recall_reg`, `water_allo`
`lsunit_*`, `hru_*`, `hru-lte_*`	`pco%{wb,nb,ls,pw}_lsu`, `pco%{wb,nb,ls,pw}_hru`, `pco%{wb,nb,ls,pw}_sd`
`channel`, `channel_sd`, `aquifer`, `reservoir`, `recall`, `hyd`, `ru`, `pest`	`pco%chan`, `sd_chan`, `aqu`, `res`, `recall`, `hyd`, `ru`, `pest`
salt rows	`pco%salt_basin`, `salt_hru`, `salt_ru`, `salt_aqu`, `salt_chn`, `salt_res`, `salt_wet`
constituent rows	`pco%cs_basin`, `cs_hru`, `cs_ru`, `cs_aqu`, `cs_chn`, `cs_res`, `cs_wet`
`hru_cb`, `hru_cb_vars`, `hru_cb_<family>`, `lsu_cb_<family>`	`pco%cb_hru`, `pco%cb_vars_hru`, `pco%cb_<family>_hru`, `pco%cb_<family>_lsu`
`gwflow_<part>`	`pco%gwflow_<part>`

Purpose

`object.cnt` gives the basin name, the basin areas and the number of spatial objects of each type: HRUs, routing units, aquifers, channels, reservoirs, point sources and the others listed below. SWAT+ uses the counts to set up its list of objects before any connection file is read. The file is named in slot 4 of the `simulation` line of `file.cio` (default `object.cnt`). It is required: if it is missing or `null`, SWAT+ prints `Cannot find object.cnt input file` and stops.

Format

Free format, whitespace-separated. Line 1 is a title and line 2 is the column header; both are skipped. Line 3 contains 21 values in the order below. The reader does not match columns by header name.

#	Column	Units	Default	Description
1	`name`	-	blank	Basin name, up to 25 characters. It appears in the name column of the basin output files.
2	`ls_area`	ha	0	Land area of the basin.
3	`tot_area`	ha	0	Total area of the basin, including reservoirs and other water bodies.
4	`obj`	count	0	Total number of spatial objects. Must equal the sum of columns 5 to 18.
5	`hru`	count	0	Number of HRUs (`hru.con`).
6	`lhru`	count	0	Number of HRU-lte objects (`hru-lte.con`).
7	`rtu`	count	0	Number of routing units (`rout_unit.con`).
8	`mfl`	count	0	Number of gwflow objects (`gwflow.con`). See gwflow.
9	`aqu`	count	0	Number of aquifers (`aquifer.con`).
10	`cha`	count	0	Number of legacy channels (`channel.con`).
11	`res`	count	0	Number of reservoirs (`reservoir.con`).
12	`rec`	count	0	Number of recall objects: point sources and inlets (`recall.con`).
13	`exco`	count	0	Number of export coefficient objects (`exco.con`).
14	`dlr`	count	0	Number of delivery ratio objects (`delratio.con`).
15	`can`	count	0	Number of canals. No connection file is read for canals in 62.0.1.
16	`pmp`	count	0	Number of pumps. No connection file is read for pumps in 62.0.1.
17	`out`	count	0	Number of outlets (`outlet.con`).
18	`lcha`	count	0	Number of channels routed with the SWAT+ channel method (`chandeg.con`).
19	`aqu2d`	count	0	Number of 2D aquifers. Read and not used.
20	`hrd`	count	0	Number of herds. Read and not used.
21	`wro`	count	0	Number of water rights objects. Read and not used.

The column headers are the ones the SWAT+ Editor writes. The old GitBook documentation calls column 8 `gwfl`. Only the order matters to SWAT+.

Object counts

Each count must match the number of objects in the corresponding connection file named on the `connect` line of `file.cio`. SWAT+ numbers all objects in one list, in this order: HRUs, HRU-lte objects, routing units, gwflow objects, aquifers, legacy channels, reservoirs, recall objects, export coefficient objects, delivery ratio objects, canals, pumps, outlets and SWAT+ channels. The object list has `obj` entries, so `obj` must equal the sum of these counts. Counts for canals and pumps reserve places in the list, but no objects are read into them.

If line 3 ends early, the reader reaches the end of the file and the missing counts stay at zero. The `refdata/Osu_1hru` example uses this and leaves out `hrd` and `wro`. If the file has more lines after line 3, SWAT+ keeps reading them in groups of title, header and values, and the last complete group is the one used. A file with a single data line avoids this.

Basin areas

`ls_area` and `tot_area` are not used to compute HRU or object areas; those come from the connection files. They are used in these places:

- `ls_area`: the HRU area when a decision table changes the fraction of the basin an HRU covers, and the HRU areas of the regions in soft calibration (`water_balance.sft`, see soft calibration).
- `tot_area`: the area weights for the basin aquifer output (`basin_aqu_*` files), and the conversion of gwflow basin water balance volumes to depths.

gwflow

When `gwflow = 1` in `codes.bsn`, SWAT+ looks for `chancell.gw` in the run folder after reading `object.cnt`:

- If `chancell.gw` exists and `mfl` is 0, SWAT+ counts the channel cell rows in `chancell.gw` and uses that count as the number of gwflow objects. It adds that count to `obj`, removes the aquifers from `obj`, sets the aquifer count to 0 and reads the gwflow connections from `gwflow.con` in place of `aquifer.con`.
- If `chancell.gw` exists and `mfl` is already above 0, SWAT+ keeps `mfl` and `obj` as given and sets the aquifer count to 0. In that case `obj` should already leave out the aquifers.
- If `chancell.gw` is missing or has no data after its header, SWAT+ turns gwflow off and the run uses the lumped aquifers.

In all three cases SWAT+ opens the `gwflow_record` log file. See gwflow for the gwflow input files.

Example

`refdata/Ames_sub1/object.cnt` in the SWAT+ repository:

```
object.cnt:
name                    ls_area      tot_area       obj       hru      lhru       rtu       mfl       aqu
demo                       1.           1            12         12         0         0         0         0
```

The basin is named `demo` and both areas are 1 ha. The project has 12 objects, all of them HRUs, so `obj` equals `hru`. There are no routing units, aquifers or channels.

`refdata/Osu_1hru/object.cnt` has 4 objects: one HRU, one routing unit, one aquifer and one SWAT+ channel (`lcha`). Its data line stops after `aqu2d`, and the `hrd` and `wro` counts are taken as zero.

Related files

- `file.cio`: names this file and the connection files.
- Connection files: one row per object; the number of rows must match the counts here.
- `codes.bsn`: its `gwflow` code starts the gwflow check above.
- gwflow: `chancell.gw` and the other gwflow inputs.

Sources

- Reader: `basin_read_objs.f90`, called from `proc_bsn.f90` after `codes.bsn`
- Type definitions: `type basin_inputs` in `basin_module.f90`, `type spatial_objects` in `hydrograph_module.f90`
- Object numbering and connection files: `hyd_connect.f90`
- Uses of the areas: `hru_fr_change.f90`, `lcu_read_softcal.f90`, `basin_aquifer_output.f90`, `gwflow_output.f90`
- File name slot: `in_sim%object_cnt` in `input_file_module.f90`

Column	Fortran variable
`name`	`bsn%name`
`ls_area`	`bsn%area_ls_ha`
`tot_area`	`bsn%area_tot_ha`

Column	Fortran variable
obj	sp_ob%objs
hru	sp_ob%hru
lhru	sp_ob%hru_lte
rtu	sp_ob%ru
mfl	sp_ob%gwflow
aqu	sp_ob%aqu
cha	sp_ob%chan
res	sp_ob%res
rec	sp_ob%recall
exco	sp_ob%exco
dlr	sp_ob%dr
can	sp_ob%canal
pmp	sp_ob%pump
out	sp_ob%outlet
lcha	sp_ob%chandeg
aqu2d	sp_ob%aqu2d
hrd	sp_ob%herd
wro	sp_ob%wro

Purpose

codes.bsn contains the method codes that apply to the whole watershed: the potential evapotranspiration (PET) method, the channel routing method, the soil carbon model, the surface runoff method, the tile drainage equations, plant stress switches and a few output options. It is named in the first position of the basin line of file.cio (default codes.bsn). SWAT+ reads it at the start of the run, before the inputs that depend on these codes.

If the file named in file.cio does not exist, every code keeps the built-in value listed below.

Format

Free format, whitespace-separated. Line 1 is a title and line 2 is the column header; both are skipped. Line 3 contains 26 values in the order below. The reader matches values by position only, so all 26 must be present and in this order. The table gives the column headers the SWAT+ Editor writes.

#	Column	Type	Built-in value	Description
1	pet_file	text	pet.cli	Not used. Measured PET is set up through file.cio and weather-sta.cli (see PET method).
2	wq_file	text	blank	Not used.
3	pet	integer	0	PET method: 0 Priestley-Taylor, 1 Penman-Monteith, 2 Hargreaves, 3 PET of the weather station. See PET method.
4	event	integer	0	Extra column switch for two other inputs. 0: normal format. Any other value: plants.plt is read with an extra plant class column and soil_plant.ini with an extra constituent column. See Extra input columns.
5	crack	integer	0	Crack flow: 0 off, 1 on. See Crack flow.
6	swift_out	integer	0	SWIFT input files: 0 not written, 1 written to a SWIFT folder at the end of the run. Older Editor versions label this column rtu_wq.

#	Column	Type	Built-in value	Description
7	sed_det	integer	0	Peak runoff rate method for erosion: 0 NRCS method with the peak rate factor, 1 half-hour rainfall intensity method. See Peak runoff rate.
8	rte_cha	integer	0	Channel water routing: 0 variable storage, 1 Muskingum. See Channel routing.
9	deg_cha	integer	0	Not used.
10	wq_cha	integer	0	Not used.
11	nostress	integer	0	Plant stress: 0 all stresses applied, 1 all stresses turned off (water, temperature, nitrogen, phosphorus, aeration and salt), 2 nitrogen and phosphorus stress turned off.
12	cn	integer	0	Not used.
13	c_fact	integer	0	Read, then overridden: the model always uses the residue and biomass cover method for the USLE C factor. See USLE C factor.
14	carbon	integer	0	Soil carbon model: 0 static carbon (original SWAT humus and residue routines), 1 reserved for C-FARM (not implemented), 2 dynamic carbon model with CENTURY pools. See Carbon model.
15	lapse	integer	0	Elevation adjustment of HRU precipitation and temperature: 0 off, 1 on. See Lapse rates.
16	uhyd	integer	1	Unit hydrograph shape for sub-daily runs: 0 triangular, 1 gamma function. See Unit hydrograph.
17	sed_cha	integer	0	Not used.
18	tiledrain	integer	0	Tile flow equation: 0 drawdown-days equation, 1 DRAINMOD equations.
19	wtable	integer	0	Shallow water table depth: 0 original routine (water above field capacity fills the profile from the bottom), 1 DRAINMOD water table routine.
20	soil_p	integer	0	Soil phosphorus model: 0 original SWAT model with a fixed P availability index, 1 dynamic P availability of Vadas and White (2010).
21	gampt	integer	0	Infiltration and surface runoff: 0 curve number, 1 Green-Ampt. See Runoff method.
22	atmo_dep	single character	a	Not used.
23	stor_max	integer	0	Not used.
24	i_fpwet	integer	0	In-stream nutrient method: 0 QUAL2E, 1 QUAL2E followed by simplified first-order transformations. Called qual2e in earlier documentation. See In-stream nutrients.
25	gw_flow	integer	0	Gridded groundwater module: 0 off, 1 on. Called gwflow in earlier documentation, where it was described as flood routing. See gwflow module.

#	Column	Type	Built-in value	Description
26	idc_till	integer	3	Tillage effect on decomposition in the dynamic carbon model (carbon = 2): 1 DSSAT, 2 EPIC (placeholder), 3 Kemanian, 4 DNDC (placeholder). See Tillage effect on decomposition.

The 26 values are read in one free-format read. A line with fewer values continues onto the next line of the file, and the codes that are not reached keep their built-in values. A value that cannot be read (text in an integer column, for example) ends the read without a message, so the codes after it also keep their built-in values.

PET method

pet selects how daily potential evapotranspiration is calculated for each HRU:

pet	Method
0	Priestley-Taylor, with $\alpha = 1.28$.
1	Penman-Monteith, with a 40 cm alfalfa reference crop and a canopy resistance that depends on the year's CO2 concentration. This method also gives the maximum transpiration of the growing plants.
2	Hargreaves.
3	The daily PET value of the HRU's weather station.

In every case the result is multiplied by the HRU's pet_co from hydrology.hyd.

With pet = 3, the HRU uses the daily PET stored for its weather station. A station gets measured PET from the PET gage named in weather-sta.cli, and the gage files are listed in pet.cli, which is named on the climate line of file.cio (see climate files). In 62.0.1, however, the station PET is recalculated with the Hargreaves equation later in the same daily climate step, for every station, before the HRUs use it (the recalculated value feeds the 30-day precipitation to PET ratio). Measured PET is therefore replaced by Hargreaves PET with a minimum of 0.01 mm, and pet = 3 gives Hargreaves PET times pet_co. Channel and pond evaporation and aquifer revap use the same station value under every pet code.

With pet = 3 the model also opens a file called pet.cli in the run folder and reads its first three lines. The file is not used after that.

Extra input columns

event is not an event switch. When it is not 0:

- each row of plants.plt is read with one more value at the end, the plant class (for example row crop, tree, grass). Calibration conditions can select plants by this class.
- each row of soil_plant.ini is read with one more name at the end, a constituent initial-condition record (see initial conditions).

Files written by the SWAT+ Editor do not have these columns, so event should be 0 for them.

Crack flow

With crack = 1, SWAT+ calculates a daily crack volume for each HRU from the soil's crack volume potential (crk in soils.sol) and the layer water contents. Surface runoff is reduced by the crack volume (in sub-daily runs, the runoff of each time step in turn until the volume is filled), and water entering the soil flows through the cracks before the layer-by-layer percolation.

Peak runoff rate

The peak runoff rate is used in the MUSLE sediment yield of each HRU.

With `sed_det = 0`, the peak rate comes from the NRCS method:

$$q_{peak} = \frac{\text{prf}}{6578.6} \cdot \frac{A\,Q_{surf}}{t_{conc}} \cdot \frac{1}{35.3}$$

where q_{peak} is in m³/s, A is the HRU area (ha), Q_{surf} is the surface runoff (mm), t_{conc} is the time of concentration (h) and `prf` is the peak rate factor in `parameters.bsn`. The factor 6578.6 converts ha · mm to mi² · in, and 35.3 converts ft³/s to m³/s. Wetlands always use this equation, whatever the value of `sed_det`.

With `sed_det = 1`, the half-hour rainfall intensity method is used:

$$\alpha_{tc} = 1 - \exp[2\,t_{conc}\,\ln(1-\alpha_{0.5})]\,, \qquad q_{peak} = \frac{\alpha_{tc}\,Q_{surf}\,A_{km}}{3.6\,t_{conc}}$$

where $\alpha_{0.5}$ is the day's fraction of rain falling in the maximum half hour (from the weather generator, scaled by `adj_pkrt` in `parameters.bsn`) and A_{km} is the HRU area in km².

Channel routing

`rte_cha` selects how water is routed through channels.

With `rte_cha = 0` (variable storage), the fraction of channel storage that leaves the reach in each routing step is

$$C_{sto} = \min\left(1,\ \text{scoef} \cdot \frac{2\,\Delta t}{2\,TT + \Delta t}\right)$$

where Δt is the routing step (h), TT is the travel time at the outflow of the previous step (h) and `scoef` is the channel storage coefficient in `parameters.bsn`. Daily runs use one routing step per day.

With `rte_cha = 1` (Muskingum), the outflow of a step is $O_2 = C_1 I_2 + C_2 I_1 + C_3 O_1$. The storage time constant is a weighted mean of the values at bankfull and at 0.1 bankfull depth, with the weights `msk_co1` and `msk_co2` (scaled to sum to 1), and `msk_x` is the weighting factor; all three are in `parameters.bsn`. The day is divided into enough substeps to keep the scheme stable.

USLE C factor

`c_fact` is replaced by 1 each time the C factor is calculated, so the model always uses the method based on residue and growing biomass cover (from APEX). The residue part of that method uses `rsd_cover` in `parameters.bsn`. The older method based on the plants' minimum C factor is not used in 62.0.1.

Carbon model

`carbon` selects the soil organic matter model:

- `0`: static carbon. Soil organic matter is kept in active and stable humus pools and in residue, with the original SWAT residue decomposition, mineralization and annual biological mixing.
- `2`: dynamic carbon model with CENTURY litter, microbial and humus pools. The model requires `carbon.bsn` and `carbon_lyr.bsn` and stops with an error if either is missing. Tillage, fertilizer and manure applications, grazing and the daily soil processes use the carbon pool versions of their routines, and the carbon outputs selected in `print.prt` become available (see carbon outputs).
- `1`: reserved for C-FARM, which is not implemented. No organic matter routine runs under this code: the soil humus pools are not initialised, and residue decomposition and N and P mineralization are skipped for the whole run.

The numbering changed in revision 62. The dynamic carbon model used to be selected with `carbon = 1`. A `codes.bsn` from an older dataset that still has `carbon = 1` now runs with no organic matter cycling, so it must be changed to `2` (dynamic model) or `0` (static model). See soil carbon for the model description.

Lapse rates

With `lapse = 1`, each HRU's daily precipitation and temperatures are adjusted for the difference between the HRU's elevation and the elevation of the gage or weather generator station that supplies the data:

$$P' = \max\left(0,\ P + \text{plaps} \cdot \frac{z_{HRU} - z_{gage}}{1000}\right), \qquad T' = T + \text{tlaps} \cdot \frac{z_{gage} - z_{HRU}}{1000}$$

with elevations in m, `plaps` in mm/km and `tlaps` in °C/km from `parameters.bsn`. The precipitation adjustment is a fixed depth added on every day, wet or dry, so with a positive `plaps` an HRU above its gage also receives rain on dry days. The temperature adjustment applies to the maximum, minimum and mean temperature. Reservoirs and channels use the station weather without adjustment.

Unit hydrograph

In sub-daily runs (more than one time step per day in `time.sim`) with the curve number method (`gampt = 0`), daily surface runoff from HRUs and routing units is spread over the time steps with a unit hydrograph. The base time and time to peak are

$$t_b = \min(48,\ 0.5 + 0.6\,t_{conc} + \text{adj_uhyd}), \qquad t_p = 0.375\,t_b$$

in hours, where `adj_uhyd` is in `parameters.bsn`. With `uhyd = 0` the hydrograph is triangular, rising to the peak at t_p and falling to zero at t_b. With `uhyd = 1` it follows the gamma function

$$q(t) = \left(\frac{t}{t_p}\right)^{\alpha} \exp\left[\alpha\left(1 - \frac{t}{t_p}\right)\right]$$

with α = `uhyd_alpha` from `parameters.bsn`. The ordinates are scaled to a total of 1 and cover at most two days. `uhyd` has no effect in daily runs or with Green-Ampt.

Runoff method

With `gampt = 0`, daily surface runoff is calculated with the curve number method. With `gampt = 1`, infiltration and runoff are calculated with the Green-Ampt method from the rainfall of each time step, which is intended for sub-daily runs. In sub-daily runs with Green-Ampt the runoff hydrograph of a routing unit is shifted in time without attenuation, and `uhyd` is not used. The shift is a fixed 2 hours for every routing unit; the computed time of concentration is not used for it. For urban HRUs, Green-Ampt also uses `urb_init_abst` in `parameters.bsn`.

Any `gampt` value above 0 also switches on the sub-daily erosion calculation from raindrop splash and overland flow, which uses `splash`, `rill`, `surq_exp` and `cov_mgt` in `parameters.bsn`.

In-stream nutrients

Channel water quality is always calculated with the QUAL2E routine. With `i_fpwet = 1`, simplified first-order transformations of organic N and sediment-bound P follow, using each channel's N and P transformation rates and the travel time, and their results replace the QUAL2E results for those constituents.

gwflow module

With `gw_flow = 1`, SWAT+ looks for `chancell.gw` in the run folder. If it is present and has data, the gridded groundwater module replaces the aquifer objects: the aquifer connection file is set to `null` and the groundwater cells connected to channels are added as objects. If `chancell.gw` is missing or empty, `gw_flow` is set back to 0 and the aquifer objects are used. See gwflow and `object.cnt`.

Tillage effect on decomposition

`idc_till` is used only with `carbon = 2`. It sets the tillage factor that multiplies the decomposition rates of the carbon pools in each soil layer:

idc_till	Tillage factor
1	DSSAT method: 1.6 in the tilled layers for `till_eff_days` days after tillage (from `carbon.bsn`), with a linear transition in a partly tilled layer; 1.0 otherwise.
2	Placeholder for the EPIC method. No tillage effect (factor 1.0).
3	Kemanian method: 1 plus the layer's tillage factor from tillage and biological mixing, calculated with the mixing curves in `carbon.bsn`.
4	Placeholder for the DNDC method. No tillage effect (factor 1.0).

Example

`refdata/Ames_sub1/codes.bsn` in the SWAT+ repository:

`codes.bsn`:

pet_file	wq_file	pet	event	crack	swift_out	sed_det	rte_cha	deg_cha
null	null	2	0	0	0	0	0	0

This file selects Hargreaves PET (`pet = 2`), the NRCS peak rate method, variable storage channel routing, curve number runoff, the dynamic carbon model (`carbon = 2`) and the Kemanian tillage method (`idc_till = 3`). `cn = 2` and `atmo_dep = 0` are in unused columns and have no effect. `uhyd = 1` would select a gamma unit hydrograph if the run were sub-daily.

Related files

- `file.cio`: names `codes.bsn` on its `basin` line.
- `parameters.bsn`: coefficients used by the methods selected here.
- `carbon.bsn` and `carbon_lyr.bsn`: required when `carbon = 2`.
- `hydrology.hyd`: `pet_co` multiplies the PET of every method.
- Climate files: `weather-sta.cli` and `pet.cli` for station PET.
- gwflow: inputs of the gridded groundwater module.
- Soil carbon, nutrient cycling and channels: process descriptions.

Sources

- Reader: `basin_read_cc.f90`, called from `proc_bsn.f90`
- Type definition: `type basin_control_codes` in `basin_module.f90`
- File name slot: `in_basin%codes_bas` in `input_file_module.f90`, read in `readcio_read.f90`
- PET: `et_pot.f90`; station PET in `climate_control.f90`
- `event`: `plant_parm_read.f90`, `soil_plant_init.f90`
- Crack flow: `sq_crackvol.f90`, `sq_crackflow.f90`, `swr_percmacro.f90`, `swr_percmain.f90`
- SWIFT output: annual sums in `command.f90`, files written by `swift_output.f90`, called at the end of `main.f90.in`
- Peak rate and erosion: `ero_pkq.f90`, `ero_cfactor.f90`, `ero_ovrsed.f90`, `surface.f90`
- Channel routing: `ch_rtmusk.f90`, `sd_hydsed_init.f90`; in-stream nutrients in `sd_channel_control3.f90`
- Plant stress: `pl_biomass_gro.f90`

- Carbon model branches: hru_control.f90, soil_nutcarb_init.f90, carbon_bsn_read.f90, cbn_zhang2.f90
- Lapse rates: cli_lapse.f90, applied in hru_control.f90
- Unit hydrograph: unit_hyd.f90, unit_hyd_ru_hru.f90, hru_hyds.f90
- Runoff method: sq_volq.f90
- Tile drainage and water table: swr_percmain.f90
- gwflow activation: basin_read_objs.f90

Column	Fortran variable
pet_file	bsn_cc%petfile
wq_file	bsn_cc%wwqfile
pet	bsn_cc%pet
event	bsn_cc%nam1
crack	bsn_cc%crk
swift_out	bsn_cc%swift_out
sed_det	bsn_cc%sed_det
rte_cha	bsn_cc%rte
deg_cha	bsn_cc%deg
wq_cha	bsn_cc%wq
nostress	bsn_cc%nostress
cn	bsn_cc%cn
c_fact	bsn_cc%cfac
carbon	bsn_cc%cswat
lapse	bsn_cc%lapse
uhyd	bsn_cc%uhyd
sed_cha	bsn_cc%sed_ch
tiledrain	bsn_cc%tdrn
wtable	bsn_cc%wtdn
soil_p	bsn_cc%sol_p_model
gampt	bsn_cc%gampt
atmo_dep	bsn_cc%atmo
stor_max	bsn_cc%smax
i_fpwet	bsn_cc%qual2e
gw_flow	bsn_cc%gwflow
idc_till	bsn_cc%idc_till

Purpose

parameters.bsn contains the coefficients that apply to the whole watershed: initial soil water, surface runoff lag, peak rate factors, nutrient and pesticide transport coefficients, denitrification, channel routing coefficients, lapse rates, erosion coefficients for sub-daily runs and the starting CO2 concentration. It is named in the second position of the basin line of file.cio (default parameters.bsn). Several of the parameters only matter for the methods selected in codes.bsn.

If the file named in file.cio does not exist, every parameter keeps its built-in value, after the zero-value replacements described in Zero and missing values.

Format

Free format, whitespace-separated. Line 1 is a title and line 2 is the column header; both are skipped. Line 3 contains 44 values in the order below: 42 real numbers followed by two integers. The reader matches values by position only, so all 44 must be present and in this order. The table gives the column headers the SWAT+ Editor writes; several no longer describe what the model does with the value.

#	Column	Units	Built-in value	Typical range	Description
1	`lai_noevap`	-	3.0	-	Not used. Read and can be changed by calibration, but no routine uses it.
2	`sw_init`	fraction	0.0	0 to 1	Initial soil water as a fraction of field capacity, the same for all soils. 0 means it is calculated from average annual precipitation. See Initial soil water.
3	`surq_lag`	-	4.0	1 to 24	Surface runoff lag coefficient. See Surface runoff lag.
4	`adj_pkrt`	-	1.0	0.5 to 2	Multiplier on the monthly mean fraction of daily rain that falls in the maximum half hour, calculated from the weather generator statistics (limited to 0.1 to 0.95). The daily fraction drawn from it is used by the half-hour intensity peak rate method (`sed_det = 1`), the rainfall erosivity of the MUSLE, urban washoff and the generation of sub-daily rainfall.
5	`adj_pkrt_sed`	-	484.0	-	Peak rate factor of the NRCS peak runoff equation, used for HRU erosion when `sed_det = 0` in `codes.bsn` and for wetlands. See Peak rate factor.
6	`lin_sed`	-	0.0	0 and up	Pesticide groundwater mixing factor. See Pesticides in aquifers.
7	`exp_sed`	-	0.0	0 or 1	Temperature-dependent pesticide decay: `1` on, any other value off. See Pesticide decay and temperature.
8	`orgn_min`	1/day	0.003	0.0003 to 0.003	Rate factor for mineralization of the active humus N and P pools (static carbon model, `carbon = 0`).
9	`n_uptake`	-	20.0	0 to 100	Nitrogen uptake distribution parameter. See Uptake distribution.
10	`p_uptake`	-	20.0	0 to 100	Phosphorus uptake distribution parameter.
11	`n_perc`	-	0.10	0 to 1	Nitrate percolation coefficient: ratio of the nitrate concentration in surface runoff to the concentration in water leaving the top layer.
12	`p_perc`	m^3/Mg	10.0	10 to 17.5	Phosphorus percolation coefficient, used in the leaching of soluble P from each layer.
13	`p_soil`	m^3/Mg	175.0	100 to 200	Phosphorus soil partitioning coefficient: ratio of soluble P in the top 10 mm of soil to soluble P in surface runoff.

#	Column	Units	Built-in value	Typical range	Description
14	p_avail	fraction	0.40	0.01 to 0.7	Phosphorus availability index: fraction of fertilizer P that is in solution after the fast reaction period. Sets the balance between the labile and active mineral P pools in the original P model (soil_p = 0).
15	rsd_decomp	fraction/day	0.05	0.02 to 0.1	Residue decomposition coefficient: daily fraction of residue that decomposes under ideal conditions. Used for every plant whose plnt_decomp in plants.plt is 0.
16	pest_perc	-	0.5	0 to 1	Pesticide percolation coefficient: ratio of the pesticide concentration in surface runoff and lateral flow to the concentration in percolate.
17	msk_co1	-	0.75	0 to 10	Weight of the storage time constant at bankfull depth in the Muskingum storage constant.
18	msk_co2	-	0.25	0 to 10	Weight of the storage time constant at 0.1 bankfull depth in the Muskingum storage constant.
19	msk_x	-	0.20	0 to 0.3	Muskingum weighting factor for inflow and outflow in channel storage.
20	nperco_lchtile	-	0.5	0 to 1	Ratio of the nitrate concentration in tile flow to the concentration in the mobile water of the tile layer.
21	evap_adj	-	0.60	0.5 to 1	Channel evaporation adjustment factor: multiplier on station PET for evaporation from the channel water surface.
22	scoef	-	1.0	0 to 1	Channel storage coefficient of the variable storage routing method (rte_cha = 0 in codes.bsn).
23	denit_exp	-	1.40	0 to 3	Denitrification exponential rate coefficient. See Denitrification.
24	denit_frac	-	1.30	see text	Denitrification threshold on the soil water factor. See Denitrification.
25	man_bact	fraction	0.15	-	Not used.
26	adj_uhyd	h	0.0	0 and up	Added to the base time of the sub-daily unit hydrograph. See unit hydrograph.
27	cn_froz	1/mm	0.000862	-	Frozen soil coefficient of the curve number retention parameter. See Frozen soil.

#	Column	Units	Built-in value	Typical range	Description
28	`dorm_hr`	h	-1.0	0 to 24	Day length threshold for dormancy. 0 or less means it is calculated from latitude. See Dormancy threshold.
29	`plaps`	mm/km	0.0	-	Precipitation lapse rate. Positive values increase precipitation with elevation. Used when `lapse = 1` in `codes.bsn`.
30	`tlaps`	°C/km	6.5	-	Temperature lapse rate. Positive values decrease temperature with elevation. Used when `lapse = 1` in `codes.bsn`.
31	`n_fix_max`	kg N/ha/day	20.0	1 to 20	Maximum daily nitrogen fixation of a plant.
32	`rsd_decay`	fraction/day	0.01	0 to 0.05	Minimum daily residue decay fraction. The daily decay rate of residue is never below this value.
33	`rsd_cover`	ha/t	0.75	-	Residue cover factor in the USLE C factor. See USLE C factor.
34	`urb_init_abst`	mm	1.0	0 to 10	Maximum initial abstraction on impervious urban areas. Used only with Green-Ampt (`gampt = 1`).
35	`petco_pmpt`	%	100.0	-	Not used. The PET of every method is adjusted with `pet_co` in `hydrology.hyd` instead.
36	`uhyd_alpha`	-	1.0	0.5 to 10	Shape coefficient of the gamma unit hydrograph (`uhyd = 1` in `codes.bsn`).
37	`splash`	-	0.0	0.9 to 3.1	Splash erosion coefficient. See Sub-daily erosion.
38	`rill`	-	0.0	0.5 to 2	Rill erosion coefficient, a multiplier on the USLE K factor of the top layer.
39	`surq_exp`	-	0.0	1 to 3	Exponent on bed shear stress in overland flow erosion.
40	`cov_mgt`	-	0.0	0.001 to 0.45	Scaling factor on the cover and management factor in overland flow erosion.
41	`cha_d50`	mm	0.0	-	Not used. The median particle size of each channel is read from the channel input.
42	`co2`	ppm	400.0	-	Atmospheric CO2 concentration, used for every year when there is no `co2_yr.dat`. See CO2.
43	`day_lag_max`	days	0	-	Not used as read. The model always sets it to 2.
44	`igen`	-	5	0 and up	Random number seed code. See Random numbers.

Values are read in one free-format read. A line with fewer values continues onto the next line, and the parameters that are not reached keep their built-in values. A value that cannot be read ends the read without a message.

Zero and missing values

After reading, SWAT+ replaces values that are zero or negative (strictly, below 10^{-6}) for some parameters. For most of them the replacement is the built-in value, so leaving a column at 0 has the same effect as entering the built-in value. These are: `lai_noevap`, `surq_lag`, `adj_pkrt`, `n_uptake`, `p_uptake`, `n_perc`, `p_perc`, `p_soil`, `p_avail`, `rsd_decomp`, `pest_perc`, `msk_co1`, `msk_co2`, `msk_x`, `nperco_lchtile`, `evap_adj`, `denit_exp`, `denit_frac`, `man_bact`, `cn_froz`, `n_fix_max`, `rsd_decay`, `rsd_cover` and `uhyd_alpha`.

The exceptions are:

Column	Value read	Value used
`adj_pkrt_sed`	0 or less	1.0 (the built-in value is 484.0)
`orgn_min`	0 or less	0.0003 (the built-in value is 0.003)
`urb_init_abst`	0 or less	0.0 (the built-in value is 1.0)
`co2`	below 100	400.0
`day_lag_max`	any	2

`sw_init`, `adj_uhyd`, `dorm_hr`, `splash`, `rill`, `surq_exp`, `cov_mgt` and `cha_d50` stay at 0 when 0 is read, and 0 has a meaning for some of them (see the sections below). `lin_sed`, `exp_sed`, `scoef`, `plaps`, `tlaps` and `igen` are used as read.

Initial soil water

All soils start at the same fraction of field capacity. If `sw_init` is 0 or less, the fraction is calculated from the average annual precipitation P (mm) of the HRU's weather generator station:

$$f_{fc} = \frac{P}{P + \exp(9.043 - 0.002135\,P)}$$

A warm-up period of at least one year (`nyskip` in `print.prt`) makes results largely independent of `sw_init`.

Surface runoff lag

In HRUs with a long time of concentration, only part of the surface runoff reaches the channel on the day it is generated. The fraction released each day is

$$f_{rel} = 1 - \exp\left(-\frac{\text{surq_lag}}{t_{conc}}\right)$$

with t_{conc} in hours. In sub-daily runs t_{conc} is expressed in time steps. The rest is kept in surface storage and added to the next day's runoff. Smaller values of `surq_lag` keep more water in storage and smooth the hydrograph.

Peak rate factor

The column labelled `adj_pkrt_sed` by the SWAT+ Editor is the peak rate factor of the NRCS peak runoff equation (see peak runoff rate). The built-in value, 484, is the standard factor for the NRCS dimensionless unit hydrograph. The Editor writes 1.0 in this column, and a value of 0 is also replaced by 1.0. A factor of 1 gives peak rates 484 times lower than the standard factor, which lowers MUSLE sediment yields from HRUs when `sed_det = 0`. The value only affects the peak rate used for erosion and in wetlands; it does not change runoff volumes.

Pesticides in aquifers

The columns labelled `lin_sed` and `exp_sed` by the Editor are pesticide parameters in 62.0.1. In each aquifer, the pesticide concentration in return flow and seepage is the pesticide mass divided by a mixing volume:

$$C = \frac{M_{pest}}{f_{mix}\, S_{aqu} + Q_{rtn} + Q_{revap} + Q_{seep}}$$

where f_{mix} is `lin_sed`, S_{aqu} is the aquifer storage and the Q terms are the day's return flow, revap and seepage (mm). With `lin_sed = 0` the concentration is based on the day's outflows only. Larger values dilute the pesticide in more of the stored water and slow its release.

Pesticide decay and temperature

With `exp_sed = 1` (the value 1.0; other values such as 2 do not count), pesticide decay in soil, on foliage, in aquifers, in channel water and in channel bed sediment uses half-lives adjusted for temperature with a Q10 model:

$$t_{1/2}(T) = t_{1/2,25}\, Q_{10}^{(25-T)/10}, \qquad Q_{10} = 2.58$$

where $t_{1/2,25}$ is the half-life from the pesticide database, taken as the value at 25 °C, and the daily fraction remaining is $\exp(-0.693/t_{1/2}(T))$. Soil decay uses the layer temperature, foliar decay the top soil layer temperature, and channel decay the water temperature. In aquifers the temperature is fixed at 0 °C, which makes aquifer half-lives about 10.7 times longer than the database value. Any other value of `exp_sed` uses the database half-lives without a temperature adjustment.

The Ames example file (see Example) has `exp_sed = 1.0`, so temperature-dependent decay is on in that dataset.

Uptake distribution

The maximum nitrogen uptake from the soil above depth z is

$$U(z) = U_{pot}\, \frac{1 - \exp(-\beta_n\, z / z_{root})}{1 - \exp(-\beta_n)}$$

where β_n is `n_uptake` and z_{root} is the root depth. Larger values concentrate uptake near the surface. Because the top 10 mm interacts with surface runoff, `n_uptake` affects the nitrate available for runoff; lower layers can make up a shortfall in the upper layers, so plant N stress changes little with this parameter. `p_uptake` works the same way for phosphorus.

Denitrification

Denitrification in a soil layer is

$$N_{denit} = NO_3 \left[1 - \exp\left(-\text{denit_exp}\ \gamma_{tmp}\ \frac{OC}{100}\right)\right]$$

where γ_{tmp} is the temperature factor and OC the organic carbon content (%). It only occurs on days when the soil water factor γ_{sw} is at or above `denit_frac`.

With the static carbon model (`carbon = 0`), $\gamma_{sw} = 0.1 + 0.9\sqrt{SW/FC}$, with no upper limit. With the default `denit_frac` of 1.3, denitrification needs $SW/FC \geq 1.78$, that is, a layer well above field capacity.

With the dynamic carbon model (`carbon = 2`), γ_{sw} comes from the water factor selected in `carbon.bsn` and is never above 1. Any `denit_frac` above 1, including the default 1.3, then switches denitrification off for the whole run. Datasets using `carbon = 2` need a `denit_frac` of 1 or less to simulate denitrification.

Frozen soil

When the second soil layer is at or below 0 °C, the curve number retention parameter S is reduced:

$$S_{frz} = S_{max}\,[1 - \exp(-\text{cn_froz} \cdot S)]$$

which increases runoff from frozen soil. The same adjustment is applied with Green-Ampt.

Dormancy threshold

A plant goes dormant when the day length falls below the station's minimum day length plus a threshold t_{dorm} (hours). If `dorm_hr` is above 0, t_{dorm} = `dorm_hr` everywhere. Otherwise it depends on the latitude ϕ of the weather generator station: 1 h above 40°, $(|\phi| - 20)/20$ h between 20° and 40°, and -1 h below 20°.

USLE C factor

The C factor is the product of a residue factor and a biomass factor:

$$C = \exp\!\left(-\text{rsd_cover} \cdot \frac{R+1}{1000}\right)\,[1 - f_{cov}\,\exp(-0.328\,h_c)]$$

where R is the residue on the surface (kg/ha), h_c is the canopy height (m), and f_{cov} is the ground cover fraction of the above-ground biomass. This is the only C factor method in 62.0.1 (see USLE C factor).

Sub-daily erosion

With Green-Ampt (`gampt` above 0 in `codes.bsn`), sediment from each time step is the sum of splash and overland flow erosion. Splash erosion decreases with runoff depth h (mm) as $\exp(-\text{splash} \cdot h/1000)$ and depends on rainfall kinetic energy and a soil texture erodibility. Overland flow erosion is

$$E_{ov} = 11.02\ \text{rill}\ K\ \text{cov_mgt}\ C\ \tau^{\,\text{surq_exp}}$$

in kg/h/m², where K is the USLE K factor of the top layer, C is a cover factor from biomass and residue, and τ is the bed shear stress (N/m²). With the built-in values of 0 for `rill` or `cov_mgt`, overland flow erosion is zero.

CO2

Without a `co2_yr.dat` file in the run folder, every year uses `co2`. With the file, the yearly values come from it (see co2_yr.dat). The radiation use efficiency of plants is adjusted for CO2 only when `co2` in `parameters.bsn` is above 350 ppm, even when the yearly values come from `co2_yr.dat`. Penman-Monteith PET uses the yearly values in all cases.

Random numbers

With `igen = 0`, the weather generator and other random processes start from fixed seeds, so every run gives the same random sequence. With any other value, each seed is advanced by a number of steps that depends on `igen`, so different `igen` values give different sequences. The same `igen` always gives the same sequence, so a new value is needed for each different realisation.

Calibration

These parameters can be changed during calibration through calibration.cal with object type bsn, using their model names: surlag (surq_lag), adj_pkr (adj_pkrt), prf (adj_pkrt_sed), evlai (lai_noevap), ffcb (sw_init), cmn (orgn_min), nperco (n_perc), pperco (p_perc), phoskd (p_soil), psp (p_avail), rsdco (rsd_decomp), percop (pest_perc), msk_co1, msk_co2, msk_x, nperco_lchtile, evrch (evap_adj), cdn (denit_exp), sdnco (denit_frac), tb_adj (adj_uhyd), n_updis (n_uptake), rsd_covco (rsd_cover), dorm_hr, plaps and tlaps. See parameter change files.

The changes in calibration.cal are applied once, after the HRUs, channels and weather stations have been set up. Several of these parameters are only used during that setup, so changing them in calibration.cal has no effect on the run:

- cmn, nperco, pperco, phoskd, psp and nperco_lchtile have already been copied to each HRU. Use the HRU-level names cmn_h, nperco_h, pperco_h, phoskd_h, psp_h and nperco_lchtile_h instead.
- ffcb: the initial soil water has already been set.
- adj_pkr and dorm_hr: the half-hour rainfall fractions and the dormancy thresholds of the weather stations have already been calculated.
- msk_co1, msk_co2 and msk_x: the Muskingum coefficients of the channels have already been calculated.
- rsdco: it has already been copied to the plants with plnt_decomp = 0.

surlag is recalculated into the runoff release fraction when it is changed, and the others are used directly during the simulation, so their changes take effect.

Example

refdata/Ames_sub1/parameters.bsn in the SWAT+ repository:

```
parameters.bsn: written by SWAT+ editor v2.3.3 on 2023-10-25 08:58 for SWAT+ rev.60.5.7
  lai_noevap       sw_init      surq_lag      adj_pkrt  adj_pkrt_sed       lin_sed       exp_sed      orgn
     3.00000       0.50000       4.00000       1.00000       1.00000       0.00000       1.00000       0.0
```

How the model reads some of these values:

- sw_init = 0.5: soils start at half of field capacity.
- adj_pkrt_sed = 1.0: the NRCS peak rate factor is 1. The standard factor is 484.
- exp_sed = 1.0: temperature-dependent pesticide decay is on.
- orgn_min = 0.0: replaced by 0.0003.
- cn_froz = 0.0: replaced by 0.000862.
- dorm_hr = 0.0: the dormancy threshold is calculated from latitude.
- plaps = 0.0 and tlaps = 0.0: no lapse rate adjustment, even if lapse = 1 in codes.bsn.
- denit_frac = 1.3: this dataset uses carbon = 2 in its codes.bsn, so no denitrification occurs (see Denitrification).
- petco_pmpt, cha_d50 and man_bact have no effect; day_lag_max is set to 2.
- igen = 0: fixed random seeds.

Related files

- file.cio: names parameters.bsn on its basin line.
- codes.bsn: method codes that decide which of these parameters are used.
- hydrology.hyd: HRU-level hydrology parameters, including pet_co.
- plants.plt: plnt_decomp overrides rsd_decomp for a plant.
- Parameter change files: calibration names for these parameters.
- Nutrient cycling, pesticides, erosion and sediment, channels and hydrology: process descriptions.

Sources

- Reader: basin_read_prm.f90, followed by the zero-value replacements in basin_prm_default.f90; both called from proc_bsn.f90

- Type definition: `type basin_parms` in basin_module.f90
- File name slot: `in_basin%parms_bas` in input_file_module.f90
- Copy to HRUs: hrudb_init.f90
- Hydrology: hydro_init.f90, time_conc_init.f90, sq_dailycn.f90, sq_greenampt.f90, unit_hyd.f90, cli_lapse.f90
- Weather generator: cli_initwgn.f90, gcycl.f90, co2_read.f90
- Peak rate and erosion: ero_pkq.f90, wetland_control.f90, ero_cfactor.f90, ero_ovrsed.f90
- Pesticides: aqu_1d_control.f90, pest_decay.f90, pest_tempdecay_module.f90, ch_rtpest.f90, pest_lch.f90
- Nutrients: nut_nminrl.f90, cbn_zhang2.f90, nut_nlch.f90, nut_solp.f90, nut_pminrl.f90, rsd_decomp.f90, cbn_surfrsd_decomp.f90, plantparm_init.f90
- Plants: pl_nup.f90, pl_pup.f90, pl_nfix.f90, pl_biomass_gro.f90
- Channels: sd_hydsed_init.f90, ch_rtmusk.f90, ch_rthr.f90, ch_rtday.f90
- Calibration: cal_parm_select.f90

Column	Fortran variable
lai_noevap	bsn_prm%evlai
sw_init	bsn_prm%ffcb
surq_lag	bsn_prm%surlag
adj_pkrt	bsn_prm%adj_pkr
adj_pkrt_sed	bsn_prm%prf
lin_sed	bsn_prm%pestgwfact
exp_sed	bsn_prm%temp_decay
orgn_min	bsn_prm%cmn
n_uptake	bsn_prm%n_updis
p_uptake	bsn_prm%p_updis
n_perc	bsn_prm%nperco
p_perc	bsn_prm%pperco
p_soil	bsn_prm%phoskd
p_avail	bsn_prm%psp
rsd_decomp	bsn_prm%rsdco
pest_perc	bsn_prm%percop
msk_co1, msk_co2, msk_x	bsn_prm%msk_co1, %msk_co2, %msk_x
nperco_lchtile	bsn_prm%nperco_lchtile
evap_adj	bsn_prm%evrch
scoef	bsn_prm%scoef
denit_exp	bsn_prm%cdn
denit_frac	bsn_prm%sdnco
man_bact	bsn_prm%bact_swf
adj_uhyd	bsn_prm%tb_adj
cn_froz	bsn_prm%cn_froz
dorm_hr	bsn_prm%dorm_hr
plaps, tlaps	bsn_prm%plaps, %tlaps
n_fix_max	bsn_prm%nfixmx
rsd_decay	bsn_prm%decr_min
rsd_cover	bsn_prm%rsd_covco
urb_init_abst	bsn_prm%urb_init_abst
petco_pmpt	bsn_prm%petco_pmpt
uhyd_alpha	bsn_prm%uhalpha
splash	bsn_prm%eros_spl
rill	bsn_prm%rill_mult
surq_exp	bsn_prm%eros_expo
cov_mgt	bsn_prm%c_factor
cha_d50	bsn_prm%ch_d50
co2	bsn_prm%co2
day_lag_max	bsn_prm%day_lag_mx
igen	bsn_prm%igen

Purpose

carbon.bsn contains the basin-wide parameters of the dynamic soil carbon model (carbon = 2 in codes.bsn): how soil organic carbon is split into pools at the start of the run, losses of soluble carbon in water, the split of organic N and P in fertilizer and manure, the duration and decay of tillage effects, the curves for tillage and biological mixing, the temperature and water factors on decomposition, surface residue photodegradation, and the C:N and C:P controls on residue decomposition.

It works together with carbon_lyr.bsn, which contains the decomposition rates and CO2 fractions for the surface layer and for the deeper layers. In revision 62 these two files replaced carb_coefs.cbn (see carb_coefs.cbn).

Activation and file names

The file is read only when carbon = 2 in codes.bsn. With any other carbon code it is ignored and need not exist.

With carbon = 2 the file is required. SWAT+ writes an error to the screen and to diagnostics.out and stops if:

- the file does not exist or cannot be opened;
- its data line is missing or one of the 28 values cannot be read;
- the companion file carbon_lyr.bsn is missing or has too few valid rows (see carbon_lyr.bsn).

There are no built-in values to fall back on.

The file is named in the third position of the basin line of file.cio, after codes.bsn and parameters.bsn. The default name is carbon.bsn. The name of the layer file is built from it by inserting _lyr before the extension: carbon.bsn gives carbon_lyr.bsn, mybasin.bsn gives mybasin_lyr.bsn, and a name without an extension gets _lyr.bsn added. The layer file is not listed in file.cio.

The basin line of file.cio must have three names. The line is read as three names in one free-format read, so a basin line from an older dataset with only two names takes the first word of the next line (normally climate) as the carbon file name, and the rest of file.cio is then read one line out of step. Datasets that do not use the carbon model can put null in the third position.

Format

Free format, whitespace-separated. Line 1 is a title and line 2 is the column header; both are skipped and may be blank. Line 3 contains the 28 values in the order below. The reader matches values by position only. Columns 7, 11, 12 and 28 are integers and must be written without a decimal point, for example 2; a value such as 2.0 in these columns stops the run with a read error.

#	Column	Units	Example value	Description
1	init_seq	fraction	0.95	Fraction of each layer's soil organic carbon put into the humus and microbial pools at the start. The rest, 1− init_seq, goes to metabolic (85%) and structural (15%) litter.
2	init_microb	fraction	0.02	Fraction of the init_seq share that goes to the microbial biomass pool.
3	init_slow	fraction	0.44	Fraction of the init_seq share that goes to the slow humus pool (when mathers_method is 0).
4	init_passive	fraction	0.54	Fraction of the init_seq share that goes to the passive humus pool.
5	koc_c	-	1000	Partition coefficient of microbial carbon between soil and water, controlling its loss in runoff, lateral flow and percolation from the top layer. Typical range 500 to 1500.

#	Column	Units	Example value	Description
6	`solc_ratio`	-	0.5	Ratio of the soluble carbon concentration in surface runoff and lateral flow to the concentration in percolate. Typical range 0.1 to 1.
7	`till_eff_days`	days (integer)	100	Number of days a tillage operation affects decomposition.
8	`manure_c_frac`	fraction	0.5	Split of the organic N and P in applied fertilizer and manure between fresh residue and the slow humus pool. See Organic fertilizer and manure.
9	`bio_consol`	fraction/day	0.15	Daily recovery rate of biological mixing after tillage.
10	`till_consol`	fraction/day	0.10	Daily decline of the tillage factor as the soil settles.
11	`tmpf_eqn`	code (integer)	2	Temperature factor equation: `1`, `2` or `3`. See Temperature factor.
12	`watf_eqn`	code (integer)	1	Water factor equation: `1` or `2`. See Water factor.
13	`t_cbn_min`	°C	-0.5	Minimum temperature of temperature equation 2.
14	`t_cbn_opt`	°C	30.0	Optimum temperature of temperature equation 2.
15	`t_cbn_max`	°C	50.0	Maximum temperature of temperature equation 2.
16	`bmix_a`	-	3.0	Intercept a of the texture curve for biological mixing.
17	`bmix_b`	-	5.0	Coefficient b of the texture curve for biological mixing.
18	`bmix_c`	-	-5.5	Exponent coefficient c of the texture curve for biological mixing.
19	`tillmix_a`	-	3.0	Intercept a of the texture curve for tillage.
20	`tillmix_b`	-	15.0	Coefficient b of the texture curve for tillage.
21	`tillmix_c`	-	-3.5	Exponent coefficient c of the texture curve for tillage.
22	`sfc_rsd_photodeg`	fraction/day	0.001	Fraction of surface residue lost each day to photodegradation. The lost carbon is counted as a residue emission.
23	`n_act_frac`	fraction	0.02	Fraction of organic N in the active humus pool. Not used with `carbon = 2`. See Notes on individual columns.
24	`cnr_cap`	-	500	Upper limit on the residue C:N ratio in the C:N decomposition factor.
25	`cnr_ref`	-	25	Residue C:N ratio at which the C:N decomposition factor equals 1.
26	`cpr_cap`	-	5000	Upper limit on the residue C:P ratio in the C:P decomposition factor.
27	`cpr_ref`	-	200	Residue C:P ratio at which the C:P decomposition factor equals 1.
28	`mathers_method`	code (integer)	1	Slow humus initialisation: `1` Mathers method, any other value the `init_slow` fraction. See Initial carbon pools.

The example values are those of `refdata/Ames_sub1/carbon.bsn` (see Example). Since the file is required, there are no defaults in the usual sense.

Initial carbon pools

At the start of the run, the organic carbon C of each soil layer (from the organic carbon content in `soils.sol`) is divided into pools:

Pool	Carbon	C:N	C:P
passive humus	$C \cdot \text{init_seq} \cdot \text{init_passive}$	10	80
slow humus	$C \cdot \text{init_seq} \cdot \text{init_slow}$	10	80
microbial biomass	$C \cdot \text{init_seq} \cdot \text{init_microb}$	8	80
metabolic litter	$C \cdot (1 - \text{init_seq}) \cdot 0.85$	10	100
structural litter	$C \cdot (1 - \text{init_seq}) \cdot 0.15$	150	1500

The reader does not check the fractions. `init_microb`, `init_slow` and `init_passive` should add up to 1, or the pools will not add up to the soil's organic carbon.

With `mathers_method = 1`, the slow humus pool is set from the clay and silt content instead of `init_slow`, following "Updating carbon pool initialization with DSSAT-CENTURY". In 62.0.1 the test and equation are written for clay plus silt as a fraction, while the soil values are in percent. As a result, for every layer with more than 0.35% clay plus silt the slow humus share is

$$f_{slow} = 0.0041 + 0.0053\,(\text{clay} + \text{silt})$$

with clay and silt in percent (0.32 for a layer with 60% clay plus silt), and the slow humus carbon is $C \cdot \text{init_seq} \cdot f_{slow}$. The passive and microbial pools keep their `init_passive` and `init_microb` shares, so with the Mathers method the pools generally do not add up to the soil's organic carbon.

Soluble carbon losses

Each day, microbial carbon in the top layer is lost with the water leaving it:

$$C_{loss} = C_{mic}\left[1 - \exp\left(-\frac{Q_{surf} + Q_{perc} + Q_{lat}}{UL + 0.0001\,\text{koc_c} \cdot C_{tot}}\right)\right]$$

where UL is the water content of the layer at saturation (mm) and C_{tot} is the layer's total organic carbon. The loss is shared between percolation and surface runoff plus lateral flow, with the concentration in surface runoff and lateral flow equal to `solc_ratio` times the concentration in percolate. Microbial carbon attached to eroded sediment is lost in proportion to the sediment yield.

Organic fertilizer and manure

When fertilizer with organic N or P, or manure, is applied, the organic matter goes to the slow humus pool and to metabolic and structural litter. In the code, the slow humus pool receives the fraction 1− `manure_c_frac` of the organic matter in each layer, and the metabolic and structural litter together also receive 1− `manure_c_frac`, split by a metabolic fraction from the C:N ratio of the material. The code comments say litter should receive the fraction `manure_c_frac`. With the value 0.5 used in the reference datasets the two readings give the same result; with other values the amounts added differ from the comments.

Tillage and biological mixing

The tillage factor of a soil layer is calculated from a texture curve and the cumulative soil disturbance x of tillage operations:

$$Z = a + b\,\exp\left(c\,\frac{\text{clay}}{100}\right), \qquad F = Z\,\frac{x}{x + \exp(1 - 2x)}$$

with clay in percent. Tillage operations use `tillmix_a`, `tillmix_b` and `tillmix_c`; biological mixing uses `bmix_a`, `bmix_b` and `bmix_c`. The layer's tillage factor is the sum of the two parts. The original equations were $Z = 3 + 12\,\exp(-5.5\,\text{clay}/100)$ for tillage and $Z = 3 + 5\,\exp(-5.5\,\text{clay}/100)$ for biological mixing.

After tillage, the tillage part declines each day by the fraction `till_consol` (scaled down by SW/FC when the layer is drier than field capacity), and biological mixing efficiency recovers each day by `bio_consol` times its initial value until it is back to that value. `till_eff_days` days after a tillage operation the tillage effect ends and the tillage part is reset to 0.

How the tillage factor changes decomposition depends on `idc_till` in `codes.bsn`. With `idc_till = 3` the decomposition rates are multiplied by $1 + F$. With `idc_till = 1` they are multiplied by 1.6 in tilled layers for `till_eff_days` days.

Temperature factor

`tmpf_eqn` selects the temperature factor γ_{tmp} on decomposition, using the layer temperature T (°C). Decomposition only occurs when $T > 0$.

`tmpf_eqn`	Equation
1	$\gamma_{tmp} = T/[T + \exp(5.058 - 0.2504\,T)]$ for $T \le 35$; $1 - 0.04\,(T - 35)$ for $35 < T \le 60$; 0 above 60.
2	$\gamma_{tmp} = \dfrac{(T - T_n)^q\,(T_x - T)}{(T_o - T_n)^q\,(T_x - T_o)}$ with $q = \dfrac{T_n - T_o}{T_o - T_x}$, and 0 outside $T_n < T < T_x$. T_n, T_o and T_x are `t_cbn_min`, `t_cbn_opt` and `t_cbn_max`.
3	$\gamma_{tmp} = 0.9\,T/[T + \exp(9.93 - 0.312\,T)] + 0.1$

Biological mixing is also reduced with temperature: by equation 3 when `tmpf_eqn = 3`, and by equation 2 otherwise. So `t_cbn_min`, `t_cbn_opt` and `t_cbn_max` affect biological mixing even when `tmpf_eqn = 1`.

Water factor

`watf_eqn` selects the water factor γ_{sw}:

- `1`: $\gamma_{sw} = 0.1 + 0.9\sqrt{SW/FC}$, limited to 0.05 to 1, where SW and FC are the layer's water content and field capacity above wilting point.
- `2`: the product of a water term, rising from 0 at zero water content to 0.4 at wilting point and 1 at field capacity, and an aeration term that falls as the air-filled porosity drops below 0.1. Both terms are at most 1.

Decomposition rates are multiplied by the combined factor $\min\left(15,\ 0.9\,\sqrt{\gamma_{tmp}\,\gamma_{sw}}\,\gamma_{ox}\,f_{till}\right)$, where γ_{ox} is an oxygen factor that decreases with depth and f_{till} is the tillage factor.

With either equation γ_{sw} never exceeds 1. Denitrification in the dynamic carbon model occurs only when γ_{sw} reaches `denit_frac` in `parameters.bsn`, so a `denit_frac` above 1 (the default is 1.3) switches denitrification off.

Residue decomposition

The daily decay rate of surface residue for each plant is

$$k = \max\left(\text{rsd_decay},\ k_{pl}\ \min(f_{CN}, f_{CP}, 1)\ \sqrt{\gamma_{tmp}\,\gamma_{sw}}\right), \qquad f_{CN} = \exp\left(-0.693\,\frac{CN - \text{cnr_ref}}{\text{cnr_ref}}\right)$$

limited to 1, where k_{pl} is the plant's residue decomposition coefficient (`plnt_decomp` in `plants.plt`, or `rsd_decomp` in `parameters.bsn`), CN is the residue C:N ratio limited to `cnr_cap`, and f_{CP} is the same expression with the C:P ratio, `cpr_ref` and `cpr_cap`. A residue C:N ratio equal to `cnr_ref` gives $f_{CN} = 1$; each further `cnr_ref` halves it. Before decomposition, the fraction `sfc_rsd_photodeg` of the surface residue is removed by photodegradation.

Notes on individual columns

- `n_act_frac` is used only by the static carbon routines (`carbon = 0`), which do not read `carbon.bsn`. With `carbon = 2` the value read has no effect, and with `carbon = 0` the model uses 0.02.
- `till_eff_days` is the only tillage duration in the model. Under `idc_till = 3` it also limits how long the tillage part of the tillage factor lasts.
- `tmpf_eqn` and `watf_eqn` values other than those listed leave the factor at 0, which stops decomposition of the soil carbon pools and denitrification in every layer.

Calibration

These parameters can be changed during calibration through calibration.cal with object type `bsn`: `init_seq`, `init_microb`, `init_slow`, `init_passive`, `koc_c`, `solc_ratio`, `manure_c_frac`, `bio_consol`, `till_consol`, `sfc_rsd_photodeg`, `n_act_frac`, `cnr_cap`, `cnr_ref`, `cpr_cap`, `cpr_ref`, `t_cbn_min`, `t_cbn_opt`, `t_cbn_max`, `bmix_a`, `bmix_b`, `bmix_c`, `tillmix_a`, `tillmix_b` and `tillmix_c`. `till_eff_days`, `tmpf_eqn`, `watf_eqn` and `mathers_method` cannot be calibrated. A change to `n_act_frac` has no effect with `carbon = 2`. See parameter change files.

Example

`refdata/Ames_sub1/carbon.bsn` in the SWAT+ repository:

```
carbon.bsn: derived from carb_coefs.cbn (basin carbon scalars)
          init_seq       init_microb         init_slow      init_passive             koc_c        solc_rat
              0.95              0.02              0.44              0.54            1000.0              0
```

95% of soil organic carbon starts in the humus and microbial pools. Because `mathers_method = 1`, the slow humus pool is set from clay and silt, and `init_slow` is not used. Temperature equation 2 runs from -0.5 °C through an optimum of 30 °C to 50 °C, and the water factor is equation 1. Tillage affects decomposition for 100 days.

Related files

- carbon_lyr.bsn: decomposition rates and CO2 fractions for the surface and deeper layers.
- codes.bsn: `carbon = 2` turns the carbon model on; `idc_till` selects the tillage method.
- file.cio: names `carbon.bsn` on its `basin` line.
- parameters.bsn: `rsd_decomp`, `rsd_decay`, `denit_exp` and `denit_frac` are also used by the carbon model.
- soils.sol: organic carbon, clay and silt of each layer.
- print.prt and carbon outputs: carbon output files, and the optional `carbon_layers.prt` that sets how many soil layers the per-layer carbon outputs contain.
- Soil carbon: description of the carbon model.

Sources

- Reader: carbon_bsn_read.f90, called from proc_bsn.f90
- Type definitions and module variables: `organic_fractions`, `carbon_water_coef`, `manure_coef`, `organic_controls` and the residue tunables in carbon_module.f90; tillage variables in tillage_data_module.f90; `photo_degrade_factor` in plant_data_module.f90
- File name slot: `in_basin%carbon_bsn` in input_file_module.f90, read in readcio_read.f90
- Pool initialisation: soil_nutcarb_init.f90, called from main.f90.in
- Soluble carbon losses: nut_orgnc2.f90
- Organic amendments: pl_fert.f90, pl_manure.f90
- Tillage and mixing: mgt_tillfactor.f90, mgt_biomix.f90, mgt_newtillmix_cswat1.f90, tillage day counter in hru_control.f90
- Decomposition factors: cbn_zhang2.f90, fcgd.f90
- Residue decomposition: cbn_surfrsd_decomp.f90
- `n_act_frac`: nut_nminrl.f90
- Calibration: cal_parm_select.f90

Column	Fortran variable
init_seq	org_frac%frac_seq
init_microb	org_frac%frac_hum_microb
init_slow	org_frac%frac_hum_slow
init_passive	org_frac%frac_hum_passive
koc_c	cb_wtr_coef%prmt_21
solc_ratio	cb_wtr_coef%prmt_44
till_eff_days	till_eff_days (integer)
manure_c_frac	man_coef%rtof
bio_consol	bio_consf
till_consol	till_consf
tmpf_eqn	org_con%tmpf (integer)
watf_eqn	org_con%watf (integer)
t_cbn_min, t_cbn_opt, t_cbn_max	org_con%tn, org_con%top, org_con%tx
bmix_a, bmix_b, bmix_c	bmix_a, bmix_b, bmix_c
tillmix_a, tillmix_b, tillmix_c	tillmix_a, tillmix_b, tillmix_c
sfc_rsd_photodeg	photo_degrade_factor
n_act_frac, cnr_cap, cnr_ref, cpr_cap, cpr_ref	module variables of the same names
mathers_method	mathers_int, stored as org_frac%mathers_method = (mathers_int == 1)

Purpose

carbon_lyr.bsn contains the decomposition rates of the litter, microbial and humus pools of the dynamic soil carbon model (carbon = 2 in codes.bsn), and the fractions of decomposed carbon released as CO2. There is one row for each of two layer groups:

- layer group 1: the first soil layer of each HRU. SWAT+ adds a 10 mm surface layer when the first layer in soils.sol is deeper than 19.5 mm, so this is normally the 10 mm surface layer.
- layer group 2: all other soil layers.

The basin-wide parameters of the carbon model are in carbon.bsn.

Activation and file name

The file is read only when carbon = 2, right after carbon.bsn, and it is then required. It is not listed in file.cio. Its name is built from the carbon.bsn name on the basin line of file.cio by inserting _lyr before the extension, so the default carbon.bsn gives carbon_lyr.bsn, and mybasin.bsn gives mybasin_lyr.bsn.

SWAT+ writes an error to the screen and to diagnostics.out and stops if the file does not exist, cannot be opened, or has fewer than two valid data rows.

Format

Free format, whitespace-separated. Line 1 is a title and line 2 is the column header; both are skipped and may be blank. Each later line is one layer group: an integer layer group number followed by 11 values, in the order below. The reader matches values by position only.

#	Column	Units	Group 1 (example)	Group 2 (example)	Description
1	`layer`	-	1	2	Layer group: `1` first soil layer, `2` all deeper layers.
2	`hp_rate`	1/day	1.2e-05	1.2e-05	Decomposition rate of passive humus under optimal conditions.
3	`hs_rate`	1/day	2.92e-04	1.81e-04	Decomposition rate of slow humus under optimal conditions.
4	`microb_rate`	1/day	0.0164	0.02	Decomposition rate of microbial biomass under optimal conditions. Used only in group 2.
5	`meta_rate`	1/day	0.0405	0.0507	Decomposition rate of metabolic litter under optimal conditions.
6	`str_rate`	1/day	0.0107	0.0134	Decomposition rate of structural litter under optimal conditions, before the lignin reduction.
7	`microb_top_rate`	1/day	0.0164	0.02	Decomposition rate of microbial biomass in the first soil layer. Used only in group 1.
8	`hs_hp`	fraction	0.05	0.05	Fraction of decomposed slow humus that goes to passive humus, before the clay adjustment.
9	`a1co2`	fraction	0.60	0.55	Fraction of decomposed metabolic litter and non-lignin structural litter released as CO2.
10	`asco2`	fraction	0.55	0.55	Fraction of decomposed slow humus released as CO2.
11	`apco2`	fraction	0.55	0.55	Fraction of decomposed passive humus released as CO2.
12	`abco2`	fraction	0.55	0.0	Fraction of decomposed microbial biomass released as CO2. Used only in group 1.

The example values are those of `refdata/Ames_sub1/carbon_lyr.bsn` (see Example).

Rows are read until the end of the file. A row whose layer group number is below 1 or above 2 is skipped with a message in `diagnostics.out`. A row that cannot be read (a missing value, or text in a number column) ends the reading at that row without a message, and the rows after it are ignored. The rows may be in any order. The reader counts accepted rows without checking that both groups are present, so a file with two rows for group 1 and none for group 2 passes the check and leaves the group 2 values at 0, which stops decomposition in all layers below the first.

Use in the carbon model

Each day, for each soil layer, the potential transformation of each pool is its carbon times its rate from the layer's group times the combined temperature, water, oxygen and tillage factor (see `carbon.bsn`):

$$\Delta C_{pool} = k_{pool}\, f_{env}\, C_{pool}$$

with these additions:

- metabolic litter: `a1co2` is released as CO2 and the rest goes to microbial biomass.
- structural litter: the rate is reduced by the lignin fraction L of the litter (at most 0.8), $k = \text{str_rate} \cdot \exp(-3L)$. Of the decomposed lignin, 30% is released as CO2 and 70% goes to slow humus. Of the non-lignin part, `a1co2` is released as CO2 and the rest goes to microbial biomass.
- microbial biomass: the first soil layer uses `microb_top_rate`. Deeper layers use `microb_rate` times a texture factor $0.25 + 0.0075\,\text{sand}$ (sand in %).
- slow humus: the fraction going to passive humus is $\max(0.001,\ \text{hs_hp} - 0.00009\,\text{clay})$, with clay in %. `asco2` is released as CO2 and the rest goes to microbial biomass.
- passive humus: `apco2` is released as CO2 and the rest goes to microbial biomass.
- microbial biomass: in the first soil layer `abco2` is released as CO2. In deeper layers the CO2 fraction is calculated from sand content, $0.17 + 0.0068\,\text{sand}$, and the `abco2` of group 2 is not used. The fraction going to passive humus is $0.003 + 0.00032\,\text{clay}$ in all layers.

The actual transformations are reduced when there is not enough mineral N to meet the N demand of the receiving pools. See soil carbon for the full pool structure.

Calibration

The parameter names `hp_rate`, `hs_rate`, `microb_rate`, `meta_rate`, `str_rate`, `microb_top_rate`, `hs_hp`, `a1co2`, `asco2`, `apco2` and `abco2` are recognised in `calibration.cal` with object type `bsn`, and the layer group is meant to be given as the layer of the change. In 62.0.1, basin parameters are passed to the change routine without a layer number, so these changes are skipped and have no effect (see parameter change files). Calibrating these rates means editing `carbon_lyr.bsn`.

Example

`refdata/Ames_sub1/carbon_lyr.bsn` in the SWAT+ repository:

```
carbon_lyr.bsn: derived from carb_coefs.cbn (per-layer carbon coefficients)
           layer         hp_rate         hs_rate     microb_rate       meta_rate        str_rate microb_to
               1         1.2e-05        2.92e-04          0.0164          0.0405          0.0107
               2         1.2e-05        1.81e-04            0.02          0.0507          0.0134
```

The first soil layer decomposes metabolic and structural litter more slowly than the deeper layers (0.0405 and 0.0107 per day, against 0.0507 and 0.0134) and slow humus faster (2.92e-04 against 1.81e-04 per day). The 0.0 for `abco2` in group 2 has no effect, because deeper layers calculate that fraction from their sand content. `microb_rate` in group 1 and `microb_top_rate` in group 2 are not used.

Related files

- carbon.bsn: basin-wide carbon parameters; its name sets the name of this file.
- codes.bsn: `carbon = 2` turns the carbon model on.
- soils.sol: layer depths, clay and sand contents.
- Carbon outputs: per-layer pools, fluxes and CO2.
- Soil carbon: description of the carbon model.

Sources

- Reader: second part of carbon_bsn_read.f90
- Type definitions: `type carbon_inputs` (array `carbdb(2)`) and `type organic_allocations` (array `org_allo(2)`) in carbon_module.f90
- Use: cbn_zhang2.f90
- Surface layer: soils_init.f90
- Calibration: cal_parm_select.f90, cal_conditions.f90

Column	Fortran variable
`layer`	`layer_id`, index into `carbdb` and `org_allo`
`hp_rate`, `hs_rate`, `microb_rate`, `meta_rate`, `str_rate`, `microb_top_rate`, `hs_hp`	`carbdb(layer)%...` of the same names
`a1co2`, `asco2`, `apco2`, `abco2`	`org_allo(layer)%...` of the same names

Purpose

The connect files describe how water and the material it moves travel from one spatial object to the next. Every spatial object (HRU, routing unit, aquifer, channel, reservoir, recall, export coefficient, delivery ratio, outlet, gwflow cell) has one row in the connect file for its object type. The row gives the object its number and name, its area, location and elevation, the record it uses in its property file, its weather station, and the list of objects that receive its outflow. SWAT+ builds the routing network and the order in which objects are run from these rows.

All connect files have the same format. The file names are given on the `connect` line of file.cio, in this order:

Position on the `connect` line	Default name	Object	Count column in `object.cnt`	Property column header	Property file
1	`hru.con`	HRU	`hru`	`hru`	`hru-data.hru`
2	`hru-lte.con`	HRU-lte	`lhru`	`hlt`	`hru-lte.hru`
3	`rout_unit.con`	Routing unit	`rtu`	`rtu`	`rout_unit.rtu`
4	`gwflow.con`	gwflow cell	`mfl`		see gwflow
5	`aquifer.con`	Aquifer	`aqu`	`aqu`	`aquifer.aqu`
6	`aquifer2d.con`	2-D aquifer	`aqu2d`		not read in 62.0.1
7	`channel.con`	Channel (legacy)	`cha`		`channel.cha`
8	`reservoir.con`	Reservoir	`res`	`res`	`reservoir.res`
9	`recall.con`	Recall (point source or inlet)	`rec`	`rec`	`recall.rec`
10	`exco.con`	Export coefficient	`exco`	`exc`	`exco.exc`
11	`delratio.con`	Delivery ratio	`dlr`	`dlr`	`delratio.del`
12	`outlet.con`	Outlet	`out`	`out`	none
13	`chandeg.con`	SWAT-DEG channel	`lcha`	`lcha`	`channel-lte.cha`

A file is read only when the matching count in object.cnt is above zero. When a count is above zero the file has to exist, because nothing checks for it: a missing file leaves those objects without connection data. The file named in position 6 is never opened in 62.0.1.

Format

Free format, whitespace-separated. Line 1 is a title and line 2 is the column header; both are skipped. Each later line is one object. SWAT+ reads as many rows as the count for that object type in object.cnt. It does not match columns by header name, so every row must give the values below in this order.

Each row has 13 leading values followed by out_tot groups of four values that describe the outflow connections, all on the same line.

#	Column	Units	Description
1	id	-	Object number within its type. Number the rows 1, 2, 3 and so on in file order: the obj_id column of other connect files refers to an object by its row position, while SWAT+ uses id as the object number during the run.
2	name	-	Object name, up to 16 characters. It appears in the output files and is not used in any calculation.
3	gis_id	-	Object number in the GIS interface. Not used in calculations. The SWAT+ Editor renumbers objects consecutively when it writes the files, so gis_id and id often differ; use id when matching a channel to a stream gauge.
4	area	ha	Area of the object. See Area, location and elevation.
5	lat	decimal degrees	Latitude.
6	lon	decimal degrees	Longitude. Not used in calculations.
7	elev	m	Elevation.
8	property column (hru, hlt, rtu, aqu, lcha, res, rec, exc, dlr, out)	-	Row number in the property file listed in the table above. Called obj in earlier documentation. In recall.con this column is not used: a recall object uses the recall.rec record whose id equals the recall object's id.
9	wst	-	Weather station name from weather-sta.cli. See Weather station lookup.
10	cst	-	Constituent pointer. Read but not used.
11	ovfl	-	Overbank flooding pointer. Read but has no effect in 62.0.1.
12	rule	-	Name of a flow-control decision table in flo_con.dtl, or null. See Flow-control rule.
13	out_tot	-	Number of outflow connections that follow on the line. May be 0.

For each of the out_tot connections, four values follow:

Offset	Column	Units	Description
+1	obj_typ	-	Type code of the receiving object. See Receiving object codes.
+2	obj_id	-	Row number of the receiving object in its connect file.
+3	hyd_typ	-	Which outflow hydrograph of this object is sent. See Hydrograph codes.
+4	frac	fraction	Fraction of that hydrograph sent to the receiving object.

When out_tot is 0 the row ends after column 13 and the object sends no outflow through this file. HRUs that belong to a routing unit are usually written this way, because the routing unit collects their outflow (see rout_unit.def and rout_unit.ele).

Area, location and elevation

The `area` value is used in calculations for some object types only:

Object	Use of `area`
HRU	Area of the HRU. Converts depths (mm) to volumes (m^3) and loads per hectare to totals.
Routing unit	Area of the routing unit. Converts inflow volumes to depths when water is routed across it.
Aquifer	Area of the aquifer. Converts recharge, storage and seepage between mm and m^3.
HRU-lte	Area of the HRU-lte. Converts runoff and sediment to totals.
SWAT-DEG channel	Used with `aqu_cha.lin` to order the channels linked to an aquifer by drainage area, and to express aquifer inflow as a depth in the channel output. See link files.
Other objects	Not used in calculations.

At the start of the run SWAT+ also adds up the drainage area of every object from the areas of the HRUs, routing units and recall objects upstream of it, weighted by `frac`, and writes the computed and the input areas to `area_calc.out`. Comparing the two columns in that file is a quick check of the routing.

`elev` is used for every object to adjust precipitation and temperature for the difference in elevation between the object and its weather data. With the lapse rates `plaps` (mm per km) and `tlaps` (°C per km) from `parameters.bsn`,

$$\Delta P = \text{plaps}\,\frac{z_{obj} - z_{pcp}}{1000}, \qquad \Delta T = \text{tlaps}\,\frac{z_{tmp} - z_{obj}}{1000}$$

where z_{obj} is `elev` and z_{pcp} and z_{tmp} are the elevations of the precipitation and temperature gauges, or of the weather generator station when the variable is simulated. For HRUs `elev` is also the elevation used to compute atmospheric pressure for potential evapotranspiration, and it can be changed in calibration with the parameter `elev`.

`lat` matters only for HRUs: an HRU with a negative latitude is treated as being in the Southern Hemisphere, and its yearly reset of base heat units and of `skip` operations happens on day 181 instead of 31 December.

Weather station lookup

The `wst` names are looked up in the list of stations read from `weather-sta.cli` with a binary search. The search assumes the stations are listed in ascending order of name (character-code order). If the stations are not in that order, or the name is not in the file, the object uses the first station in `weather-sta.cli` and no message is written.

Receiving object codes

`obj_typ` accepts these codes:

Code	Receiving object	Notes
`hru`	HRU	Inflow is routed across the HRU. See Inflow to HRUs and routing units.
`hlt`	HRU-lte	
`ru`	Routing unit	The routing unit and each HRU in it are registered as receivers.
`aqu`	Aquifer	
`cha`	Channel in `channel.con`	62.0.1 does not run these legacy channels, so flow sent to them does not continue downstream. Use `sdc` for SWAT-DEG channels.
`res`	Reservoir	

Code	Receiving object	Notes
`exc`	Export coefficient	Export coefficient objects are not run as part of the routing, so inflow to them has no effect.
`dr`	Delivery ratio	The object multiplies its inflow by the ratios in `delratio.del`.
`out`	Outlet	
`sdc`	SWAT-DEG channel in `chandeg.con`	

A recall object cannot receive flow. The routing code also has a branch for gwflow cells as receivers, but its code is longer than the three characters `obj_typ` can store, so no value in a connect file can select it. Any other code is not recognised and leaves the connection without a receiving object, which is likely to stop the run.

Hydrograph codes

`hyd_typ` selects one of the outflow hydrographs of the sending object:

Code	HRU, HRU-lte, routing unit	Aquifer	Channel, reservoir, recall, outlet, export coefficient, delivery ratio
`tot`	Surface runoff + lateral flow + tile flow	Return flow (groundwater discharge)	Total outflow
`rhg`	Percolation out of the bottom of the soil profile (recharge)	Deep seepage	Not filled, sends nothing
`sur`	Surface runoff	not used	not used
`lat`	Lateral soil flow	not used	not used
`til`	Tile flow	not used	not used

Any other code is not recognised. The usual connections are `tot` from an HRU or routing unit to a channel together with `rhg` to its aquifer, and `tot` from an aquifer to a channel.

`frac` is not checked. The fractions of one hydrograph type from one object should add up to 1, otherwise water is created or lost. Fractions of different types are independent, which is why a routing unit can send 100% of `tot` to a channel and 100% of `rhg` to an aquifer. Sending both `tot` and one of its parts (`sur`, `lat`, `til`) counts that part twice.

Inflow to HRUs and routing units

An HRU or routing unit that receives flow keeps surface, lateral and tile inflow separate, so it can route each through the right part of the landscape:

- From another HRU, HRU-lte or routing unit with `tot`, the sender's surface runoff is added to surface runon, its lateral flow to lateral inflow and its tile flow to tile inflow.
- From another HRU, HRU-lte or routing unit with `sur`, `lat` or `til`, that hydrograph goes to surface runon, lateral inflow or tile inflow. An `rhg` connection to an HRU or routing unit is accepted, but the water is not added anywhere.
- From any other object (channel, aquifer, reservoir, recall and so on) the total outflow is added to surface runon, whatever `hyd_typ` says.

For all other receiving objects the selected hydrograph, multiplied by `frac`, is added to the object's inflow.

Flow-control rule

A `rule` name other than `null` is matched against the decision tables in `flo_con.dtl` when the files are read. In 62.0.1 the match is stored but no routine uses it, so `rule` does not change the outflow fractions during the run.

Export coefficient objects

In `exco.con` the `exc` column is the row number in `exco.exc`. The `om` name in that row selects a row of `exco_om.exc`, and that row becomes the object's outflow hydrograph. It is set once when the files are read and stays constant for the whole run. Send it on with `hyd_typ tot`.

Groundwater flow (gwflow) models

When the gwflow module is active (`gwflow` = 1 in `codes.bsn`) and `object.cnt` lists gwflow cells, any row that has at least one connection with `obj_typ aqu` has its connection count reduced by one after reading, because gwflow replaces the aquifer objects. The connection that is dropped is the last one on the line, which is the aquifer connection only when that connection is written last. Put aquifer connections at the end of the line in gwflow models.

Routing loops

SWAT+ orders the objects so that every object runs after all objects that send it flow. If the connections contain a loop, the ordering cannot finish: after 5000 passes SWAT+ writes the objects that could not be ordered, with their inflow objects, to `looping.con`, prints an error and stops.

Example

`refdata/Osu_1hru/aquifer.con` in the SWAT+ repository:

```
aquifer.con: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
      id  name                gis_id          area           lat           lon          elev       aqu
       1  aqu001                   1      10.00000      35.64206     127.38513     322.26395         1
```

One aquifer of 10 ha that uses row 1 of `aquifer.aqu` and sends all of its return flow (`tot`) to SWAT-DEG channel 1.

`refdata/Osu_1hru/rout_unit.con`:

```
rout_unit.con: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
      id  name                gis_id          area           lat           lon          elev       rtu
       1  rtu001                   1       10.0000      35.52014     127.32787     113.51276         1
```

The routing unit sends its surface, lateral and tile flow (`tot`) to SWAT-DEG channel 1 and its recharge (`rhg`) to aquifer 1, each with fraction 1.0. The `rule` value `0` does not name a table in `flo_con.dtl`, so it has the same effect as `null`.

`refdata/Osu_1hru/hru.con`:

```
hru.con: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
      id  name                gis_id          area           lat           lon          elev       hru
       1  hru0001                  1           10       35.65311     127.36763     350.96629         1
```

The HRU has no connections in `hru.con` (`out_tot` = 0). Its outflow reaches the channel and the aquifer through the routing unit it belongs to.

Related files

- `object.cnt`: number of objects of each type, and so the number of rows read from each connect file.
- `file.cio`: names the connect files on its `connect` line.
- `hru-data.hru`, `aquifer.aqu`, `reservoir.res`, `recall.rec` and the other property files named in the first table.
- `aqu_cha.lin`: read right after `aquifer.con` to link an aquifer to more than one channel.
- `weather-sta.cli`: station names used in `wst`.

- Decision tables: `flo_con.dtl`, named in `rule`.
- `parameters.bsn`: lapse rates applied with `elev`.
- Recall, export coefficient and delivery ratio objects in the model theory.

Sources

- Reader: `hyd_read_connect.f90`, called once per object type from `hyd_connect.f90`, which also resolves receiving objects, hydrograph codes, drainage areas and the run order
- Type definitions: `type object_connectivity`, `type spatial_objects`, `type object_total_hydrographs` in `hydrograph_module.f90`
- File names: `type input_con` in `input_file_module.f90`, read in `readcio_read.f90`
- Inflow summing and hydrograph use: `command.f90`, `hru_hyds.f90`, `aqu_1d_control.f90`
- Area, elevation and latitude: `hrudb_init.f90`, `topohyd_init.f90`, `cli_lapse.f90`, `et_pot.f90`, `time_control.f90`
- Weather station lookup: `search.f90`, station list from `cli_staread.f90`
- Rule lookup: `dtbl_flocon_read.f90`
- Export coefficient hydrographs: `exco_read_om.f90`
- `aqu_cha.lin` reader: `aqu2d_read.f90`
- `area_calc.out` unit opened in `proc_bsn.f90`

Column	Fortran variable
`id`	`ob(i)%num`
`name`	`ob(i)%name`
`gis_id`	`ob(i)%gis_id`
`area`	`ob(i)%area_ha`
`lat`	`ob(i)%lat`
`lon`	`ob(i)%long`
`elev`	`ob(i)%elev`
property column	`ob(i)%props`
`wst`	`ob(i)%wst_c` (resolved to `ob(i)%wst`)
`cst`	`ob(i)%constit`
`ovfl`	`ob(i)%props2`
`rule`	`ob(i)%ruleset` (resolved to `ob(i)%flo_dtbl`)
`out_tot`	`ob(i)%src_tot`
`obj_typ`	`ob(i)%obtyp_out(k)`
`obj_id`	`ob(i)%obtypno_out(k)` (resolved to `ob(i)%obj_out(k)`)
`hyd_typ`	`ob(i)%htyp_out(k)` (resolved to `ob(i)%ihtyp_out(k)`)
`frac`	`ob(i)%frac_out(k)`

Purpose

`hru-data.hru` lists the HRU property sets of the project. Each row names one record in each of the parameter files an HRU needs: topography, hydrology, soil, land use and management, soil and plant initialization, surface storage (wetland), snow and field. An HRU picks its row through the `hru` column of `hru.con`, which gives the `id` of a row in this file. The file is named on the `hru` line of `file.cio` (default `hru-data.hru`).

The parameter values themselves are in the files the names point to. This file only links them together, so several HRUs can share one hydrology or topography record while having different soils or land uses.

Format

Free format, whitespace-separated. Line 1 is a title and line 2 is the column header; both are skipped. Each later line is one property set with ten values in the order below. The reader does not match columns by header name, so all ten values must be present and in this order.

#	Column	Looked up in	Description
1	id	-	Number of the property set. The hru column of hru.con refers to this value. The numbers do not have to be consecutive; the largest id sets the size of the table.
2	name	-	Name of the property set, up to 40 characters. Not used in calculations.
3	topo	topography.hyd	Topography record: slope, slope lengths and deposition coefficient.
4	hydro	hydrology.hyd	Hydrology record: lateral flow, canopy storage, evaporation and percolation parameters.
5	soil	soils.sol	Soil profile.
6	lu_mgt	landuse.lum	Land use and management record, which sets the plant community, management schedule, curve numbers, conservation practices and structures such as tile drains.
7	soil_plant_init	soil_plant.ini (see initial conditions)	Initial soil water, soil nutrients and constituents. The names of the soil test, pesticide, pathogen, heavy metal, salt and constituent records given in that row are matched at the same time.
8	surf_stor	wetland.wet	Surface storage (wetland or paddy) record, or null for an HRU without surface storage.
9	snow	snow.sno	Snow parameter set.
10	field	field.fld	Field dimensions, or null to use the built-in values.

A name must match the record name in the target file character for character, including case. Names in topography.hyd, hydrology.hyd and field.fld are stored with at most 16 characters and soil names with at most 20, so a longer name in this file can never match a record there.

Name lookup

Each name is looked up once, when the files are read. The table gives what happens when a name is not found:

Column	null or a name that is not found
topo, hydro, soil, lu_mgt, soil_plant_init, snow	The HRU uses the first record of the target file. No message is written.
surf_stor	null means no surface storage. Any other name that is not in wetland.wet is reported in diagnostics.out as <name> not found (wetland.wet) and the HRU has no surface storage.
field	The HRU uses the built-in field values (length 500 m, width 100 m, angle 30°). A name other than null that is not found is reported in diagnostics.out as <name> not found (field.fld).

The reader has checks that would write not found messages for topo, hydro, soil, lu_mgt, soil_plant_init and snow, but they test for a pointer of zero while the pointers start at one, so in 62.0.1 these messages are never written. Check these names by hand, or in the SWAT+ Editor, before a run.

For snow this means that null selects the first row of snow.sno, the same as an unmatched name. A project that has no snow.sno file at all has no first row to fall back on.

The surf_stor name is matched when the wetlands are initialized, which happens only when wetland.wet has at least one record.

Tile drainage

The lu_mgt record decides whether an HRU has tile drains. For a tile-drained HRU, SWAT+ replaces two values from its hydro record at the start of the run: cn3_swf is set to 0.95 and perco to 0.1. Calibration changes to

`cn3_swf` and `perco` are also skipped for these HRUs. See hydrology.hyd.

Example

`refdata/Ames_sub1/hru-data.hru` in the SWAT+ repository (first four rows):

`hru-data.hru:`

id	name	topo	hydro	soil	lu_mgt	soil_
1	hru0001	topohru0001	hyd0001	soil_01-h1	cosy_lum	
2	hru0002	topohru0002	hyd0002	soil_02	mntill_corn_lum	
3	hru0003	topohru0003	hyd0003	soil_03-h3	cosy_lum	
4	hru0004	topohru0004	hyd0004	soil_04	cosy_lum	

Each HRU has a separate topography record, hydrology record and soil. HRU 2 uses the land use `mntill_corn_lum`, the others `cosy_lum`. No HRU has surface storage, all use the snow set `snow001`, and `field` is `null`, so every HRU uses the built-in field dimensions.

Related files

- hru.con: one row per HRU; its `hru` column selects a row of this file by `id`. The HRU's area, elevation, latitude and weather station come from hru.con.
- object.cnt: number of HRUs.
- topography.hyd, [hydrology.hyd](#input-reference-hyd-hydrologyhyd) and field.fld, soils.sol, landuse.lum, initial conditions, wetland.wet, snow.sno: the files the names point to.
- file.cio: names this file on its `hru` line.
- Water allocation: an HRU can be named as a point of diversion in point_of_diver.wro. That link is made in the water allocation files; there is no column for it here.

Sources

- Reader: hru_read.f90, called from proc_hru.f90
- Type definitions: `type hru_databases`, `type hru_databases_char`, `type hydrologic_response_unit_db` in hru_module.f90
- Copy to each HRU through `hru.con`: hrudb_init.f90
- Use of the topography, hydrology, snow and field pointers, and the tile-drain override: topohyd_init.f90
- Surface storage lookup: wet_initial.f90, called from wet_all_initial.f90
- Name lengths of the target records: hydrology_data_module.f90, topography_data_module.f90, soil_data_module.f90, landuse_data_module.f90
- Points of diversion: water_allocation_read.f90
- File name: `in_hru%hru_data` in input_file_module.f90

Column	Fortran variable	Resolved pointer
`id`	array index `i` of `hru_db`	
`name`	`hru_db(i)%dbsc%name`	
`topo`	`hru_db(i)%dbsc%topo`	`hru_db(i)%dbs%topo` into `topo_db`
`hydro`	`hru_db(i)%dbsc%hyd`	`hru_db(i)%dbs%hyd` into `hyd_db`
`soil`	`hru_db(i)%dbsc%soil`	`hru_db(i)%dbs%soil` into `soildb`
`lu_mgt`	`hru_db(i)%dbsc%land_use_mgt`	`hru_db(i)%dbs%land_use_mgt` into `lum`
`soil_plant_init`	`hru_db(i)%dbsc%soil_plant_init`	`hru_db(i)%dbs%soil_plant_init` into `sol_plt_ini`
`surf_stor`	`hru_db(i)%dbsc%surf_stor`	`hru(j)%dbs%surf_stor` into `wet_dat` (set in `wet_initial`)
`snow`	`hru_db(i)%dbsc%snow`	`hru_db(i)%dbs%snow` into `snodb`
`field`	`hru_db(i)%dbsc%field`	`hru_db(i)%dbs%field` into `field_db`

Purpose

`hru-lte.hru` describes the HRU-lte objects, a simplified kind of HRU. An HRU-lte has one soil layer described by a texture class, a single plant whose growing season is started and ended by decision tables, a curve number for surface runoff, separate shallow and deep groundwater stores and a fixed USLE factor for erosion. It computes no nutrients, pesticides or bacteria. HRU-lte objects can be used alone or together with ordinary HRUs, and they send water and sediment to channels, routing units and other objects like any other spatial object.

The file is named in the second position of the `hru` line of file.cio (default `hru-lte.hru`). The objects themselves are defined in `hru-lte.con`, the second file on the `connect` line, and their number is the `lhru` count in object.cnt. The `hlt` column of `hru-lte.con` gives the `id` of the `hru-lte.hru` row each object uses. The area, latitude, elevation and weather station of the object come from `hru-lte.con`; see the connection file format.

`hru-lte.hru` is read only when the file exists and is not named `null`. When `lhru` is above zero, the file must be present: without it the HRU-lte objects have no properties and the run fails. A project with HRU-lte objects also needs `soils_lte.sol` (texture classes), `plants.plt` (plant parameters) and `lum.dtl` (the growing-season decision tables).

hru-lte.hru

Free format, whitespace-separated. Line 1 is a title and line 2 is the column header; both are skipped. Each later line is one HRU-lte with 35 values in the order below. The reader does not match columns by header name, so all 35 values must be present and in this order. Text values must not contain spaces.

#	Column	Units	Typical range	Description
1	`id`	-	1 and up	Row number. The `hlt` column of `hru-lte.con` refers to the row by this number. Ids must be unique; the rows may be in any order.
2	`name`	-	-	Name of the HRU-lte, up to 16 characters. Not used in calculations; the names in the output files come from `hru-lte.con`.
3	`area`	km² (in the source)	-	Drainage area. Used only to estimate the time of concentration when `t_conc` is zero, which does not work in 62.0.1 (see Time of concentration). The SWAT+ Editor writes the HRU area in hectares here.
4	`cn2`	-	35 to 98	SCS curve number for moisture condition II. See Surface runoff.
5	`cn3_swf`	fraction	0 to 0.95	Soil water factor for the curve number: the soil water content at which the retention parameter reaches its condition III value, as a fraction of the way from field capacity (0) to saturation (1). A value of 1 makes the shape coefficients of the retention curve infinite; use a value below 1. The SWAT+ Editor writes 0.
6	`t_conc`	min	10 to 1000	Time of concentration. Sets the peak runoff rate for erosion, and the delivery ratio when the HRU-lte is an element of a routing unit with a calculated delivery ratio. Give a value above zero. The SWAT+ Editor writes 26.
7	`soil_dp`	mm	300 to 3000	Depth of the soil profile. Multiplies the available water capacity and porosity of the soil texture.

#	Column	Units	Typical range	Description
8	`perc_co`	fraction	0 to 1	Percolation threshold: percolation from the soil profile starts when soil water exceeds `perc_co` times the available water capacity. Called `perco_co` in earlier documentation, which gave a range of 0 to 6000. See Percolation.
9	`slp`	m/m	0 to 0.6	Average land surface slope. Used for lateral flow and the USLE slope factor.
10	`slp_len`	m	10 to 150	Slope length. Used for lateral flow and the USLE slope length factor.
11	`et_co`	-	0.8 to 1.2	Evapotranspiration coefficient. Multiplies potential evapotranspiration, and again actual evapotranspiration. Values below 0.4 make evapotranspiration outside the growing season negative. See Evapotranspiration.
12	`aqu_sp_yld`	-	0.01 to 0.3	Specific yield of the shallow aquifer. Read but not used.
13	`alpha_bf`	1/day	0 to 1	Baseflow recession constant of the groundwater store. See Groundwater.
14	`revap`	fraction	0 to 0.2	Revap coefficient: water removed from the shallow groundwater store each day as a fraction of the day's actual evapotranspiration.
15	`rchg_dp`	fraction	0 to 1	Fraction of the day's percolation that moves from the shallow store to the deep store.
16	`sw_init`	fraction	0 to 1	Initial soil water as a fraction of the available water capacity.
17	`aqu_init`	mm	-	Initial shallow aquifer storage. Read but not used: the store starts at zero.
18	`aqu_sh_flo`	mm	-	Initial groundwater flow. Read but not used: the flow starts at zero.
19	`aqu_dp_flo`	mm	-	Initial deep aquifer storage. Read but not used: the store starts at zero.
20	`snow_h2o`	mm H_2O	-	Initial snow water content. Read but not used: the snowpack starts at zero.
21	`lat`	degrees	-90 to 90	Latitude. Read, and used only in a day length calculation whose result is not used, so it has no effect.
22	`soil_text`	-	-	Soil texture class, matched against the `texture` column of `soils_lte.sol`. See Name matching.
23	`trop_flag`	-	`non_trop`, `trop`	Tropical flag. Read but not used.
24	`grow_start`	-	-	Name of the `lum.dtl` decision table that starts the growing season, usually `pl_grow_sum` (or `pl_grow_win` for winter crops).
25	`grow_end`	-	-	Name of the `lum.dtl` decision table that ends the growing season and computes the yield, usually `pl_end_sum` (or `pl_end_win`).
26	`plnt_typ`	-	-	Plant name, matched against the `name` column of `plants.plt`.

#	Column	Units	Typical range	Description
27	`stress`	fraction	0 to 1	Plant stress factor for stresses the HRU-lte does not simulate (pests, root restriction, soil quality, nutrients). It multiplies daily biomass growth; 1 means no such stress. With 0 no biomass grows and the yield is zero. The SWAT+ Editor writes 0.
28	`pet_flag`	-	`harg`, `p_t`	Potential evapotranspiration method: `harg` for Hargreaves; any other value selects Priestley-Taylor.
29	`irr_flag`	-	-	Irrigation switch. See Irrigation for the values that switch irrigation on in 62.0.1.
30	`irr_src`	-	`outside_bsn`, `shal_aqu`, `deep_aqu`	Source of irrigation water: the HRU-lte's shallow or deep groundwater store, or an unlimited source outside the basin (any value other than `shal_aqu` and `deep_aqu`).
31	`t_drain`	hr	0, or 24 to 72	Time to drain the soil to field capacity through tile drains. 0 means no tile drains.
32	`usle_k`	0.013 t m² h/(m³ t cm)	0 to 0.65	USLE soil erodibility factor K.
33	`usle_c`	-	0.001 to 0.5	USLE cover and management factor C. It is constant through the year. The SWAT+ Editor writes the `usle_c_min` of the plant.
34	`usle_p`	-	0 to 1	USLE support practice factor P.
35	`usle_ls`	-	-	USLE slope length and steepness factor LS. Read but not used: SWAT+ computes LS from `slp` and `slp_len`.

Name matching

The text columns are compared character by character with the names in the other files. The comparison is case-sensitive and trailing blanks are ignored.

- `soil_text` is looked up among the twelve texture classes of `soils_lte.sol` (see the soils_lte.sol format). The matching class gives the available water capacity and porosity per mm of soil depth and the saturated hydraulic conductivity. If no class matches, the HRU-lte has no soil water capacity, and the run fails or produces invalid numbers when it divides by it. The SWAT+ Editor datasets contain the classes `sand`, `loamy_sand`, `sandy_loam`, `loam`, `silt_loam`, `silt`, `silty_clay_loam`, `clay_loam`, `sandy_clay_loam`, `sandy_clay`, `silty_clay` and `clay`, and the Editor picks one for each HRU-lte from the sand, silt and clay content of the first soil layer.
- `plnt_typ` is looked up in the `name` column of `plants.plt`. An unknown name writes `<name> not found (plants.plt)` to `diagnostics.out`, and the HRU-lte grows with the built-in default plant record, whose leaf area curve is not set up.
- `grow_start` and `grow_end` are looked up by table name in `lum.dtl`. An unknown name writes `<name> entry in (hru-lte.hru) not found in (lum.dtl)` to `diagnostics.out`; the run then refers to a decision table that does not exist, so both names must be present in `lum.dtl`.
- `pet_flag` is compared with `harg` only.
- `irr_flag` and `irr_src` are compared as described under Irrigation.

Daily water balance

Each HRU-lte runs once a day, in the order of the object list. The steps below follow that order. SW is the soil water content (mm), AWC and POR are the available water capacity and the porosity of the profile (mm), and

K_{sat} is the saturated hydraulic conductivity of the texture class (mm/h):

$$AWC = awc \cdot \text{soil_dp}, \qquad POR = por \cdot \text{soil_dp}, \qquad SW_0 = \text{sw_init} \cdot AWC$$

where *awc* and *por* are the per-mm values of the texture class in `soils_lte.sol`.

Snow

When the mean air temperature $\bar{T}$ is below 0 °C, the whole day's precipitation is added to the snowpack and there is no runoff or infiltration that day. Otherwise snowmelt is

$$SNO_{mlt} = 4.57\,\bar{T}$$

limited to the snowpack, and rain and melt together form the water input P for the day. The `snow.sno` parameters are not used by HRU-lte objects.

Surface runoff

Surface runoff uses the SCS curve number method with a retention parameter that varies with soil water (see SCS curve number). At the start of the run the condition I and III curve numbers are derived from `cn2`:

$$CN_1 = CN_2 - \frac{20\,(100 - CN_2)}{100 - CN_2 + \exp\,(2.533 - 0.063\,(100 - CN_2))}, \qquad CN_1 \geq 0.4\,CN_2$$

$$CN_3 = CN_2 \exp\,(0.00673\,(100 - CN_2))$$

$$S_{max} = 254\left(\frac{100}{CN_1} - 1\right), \qquad S_3 = 254\left(\frac{100}{CN_3} - 1\right)$$

Each day the retention parameter is

$$S = S_{max}\left(1 - \frac{SW}{SW + \exp(w_1 - w_2\,SW)}\right), \qquad S \geq 3$$

The shape coefficients w_1 and w_2 are fixed at the start so that $S = S_3$ when $SW = AWC + \text{cn3_swf}\,(POR - AWC)$, and $S = 2.54$ when $SW = POR$. Runoff is

$$Q_{surf} = \frac{(P - 0.2\,S)^2}{P + 0.8\,S} \quad \text{for } P > 0.2\,S$$

and zero otherwise. The rest of P is added to the soil water. SWAT+ also stores `cn2` limited to 35 to 98, but the runoff curve is built from the value as given in the file, so values outside that range are used as they are.

Evapotranspiration

Potential evapotranspiration is computed with the method chosen in `pet_flag` and multiplied by `et_co`. With Hargreaves,

$$PET = \text{et_co} \cdot 0.0032\,\frac{H_0}{2.5 - 0.0022\,\bar{T}}\,(\bar{T} + 17.8)\,(T_{max} - T_{min})^{0.6}$$

where H_0 is the day's maximum possible solar radiation (MJ/m²). With Priestley-Taylor,

$$PET = \text{et_co} \cdot 1.28 \, \frac{\Delta}{\Delta + 0.68} \cdot \frac{23.9\, H\,(1 - 0.23)}{58.3}, \qquad \Delta = \frac{5304}{T_K^2} \exp\left(21.255 - \frac{5304}{T_K}\right)$$

where H is the day's solar radiation (MJ/m²) and T_K the mean air temperature in kelvin. Both are simpler forms than those used for ordinary HRUs (see Potential evapotranspiration).

Actual evapotranspiration is proportional to the soil water content:

$$E_a = b \cdot PET \, \frac{SW}{AWC}, \qquad b = \begin{cases} \text{et_co} & \text{during the growing season} \\ \text{et_co} - 0.4 & \text{outside it} \end{cases}$$

It is taken from the soil after lateral flow, tile flow and percolation, and is limited to the soil water left at that point. Because `et_co` enters both PET and b, a change in `et_co` changes actual evapotranspiration roughly with its square during the growing season.

Lateral flow

When soil water is above the available water capacity,

$$Q_{lat} = 0.024 \, \frac{SW - AWC}{POR - AWC} \, K_{sat} \, \frac{\text{slp}}{\text{slp_len}}$$

limited to the soil water.

Tile flow

When `t_drain` is above zero and soil water is above the available water capacity,

$$Q_{tile} = (SW - AWC)\left(1 - \exp\left(\frac{-24}{\text{t_drain}}\right)\right), \qquad Q_{tile} \le 10 \text{ mm}$$

Percolation

Water above `perc_co` times the available water capacity drains from the bottom of the profile:

$$w_{perc} = (SW - \text{perc_co} \cdot AWC)\left(1 - \exp\left(\frac{-24}{TT}\right)\right), \qquad TT = \max\left(2, \; \frac{POR - AWC}{K_{sat}}\right)$$

where TT is the travel time in hours. With `perc_co` at 1 only water above field capacity percolates; with `perc_co` at 0 (the value the SWAT+ Editor writes) the profile can drain well below field capacity.

Groundwater

The shallow and deep groundwater stores of an HRU-lte are separate from the aquifer objects. Percolation is added to the shallow store. Groundwater flow follows a recession with $\alpha =$ `alpha_bf`:

$$Q_{gw,d} = Q_{gw,d-1} \, e^{-\alpha} + w_{perc} \, (1 - e^{-\alpha})$$

limited to the shallow storage. Then revap ($\text{revap} \cdot E_a$) and deep percolation ($\text{rchg_dp} \cdot w_{perc}$) are taken from what is left in the shallow store, each limited to the storage. Deep percolation is added to the deep store, which only irrigation from `deep_aqu` draws on. Revap leaves the HRU-lte; it is not added to the soil or to the reported evapotranspiration. Both stores and the groundwater flow start at zero.

Water yield

The water the HRU-lte sends downstream each day is

$$WYLD = Q_{surf} + Q_{lat} + Q_{tile} + Q_{gw}$$

In the `hru-lte_wb` output files, `latq` is the sum of lateral flow and groundwater flow.

Plant growth

The plant named in `plnt_typ` grows between the two decision tables. On each day outside the growing season the `grow_start` table is checked; when its `grow_init` action runs, growth starts with zero biomass and leaf area. During the season the `grow_end` table is checked; its `grow_end` action computes the yield and ends the season. The `pl_grow_sum` and `pl_end_sum` tables of the SWAT+ reference data start growth when the fraction of the year's base-zero heat units at the weather station is between 0.15 and 0.2, and end it when the fraction is above 0.5 or on day 330. See decision table actions.

Heat units to maturity

At the start of the run SWAT+ estimates the heat units to maturity, PHU, from the long-term monthly mean temperatures in the weather generator of the HRU-lte's station. Daily values of $\bar{T} - T_{base}$ above zero are summed over the days between a start and an end day, and

$$PHU = \max(500,\ 0.9 \sum(\bar{T} - T_{base}))$$

limited to 2000 for annual plant types (`warm_annual`, `cold_annual` and their tuber forms). In 62.0.1 the start and end days of the sum are the positions of the `grow_start` and `grow_end` tables in `lum.dtl` (first table 1, second table 2, and so on). The days of the year of the growing season are not used. For two tables next to each other in `lum.dtl` the sum covers no days and PHU takes its minimum of 500.

Biomass and leaf area

During the growing season the fraction of heat units g increases each day by $(\bar{T} - T_{base})/PHU$. Biomass grows with intercepted radiation:

$$\Delta B = \gamma \cdot \text{stress} \cdot bm_e \cdot 0.5\, H \left(1 - e^{-0.65\,(LAI+0.05)}\right), \qquad \gamma = \min(ws,\ ts,\ as)$$

where bm_e is `bm_e` of `plants.plt`, $ws = E_a/PET$ is the water stress factor (before the adjustment b), ts the temperature stress factor and as the aeration stress factor:

$$ts = \exp\left(-0.1054 \left(\frac{T_{opt} - \bar{T}}{2T_{opt} - T_{base} - \bar{T}}\right)^2\right) \text{ for } \bar{T} > T_{base}, \qquad ts = 0 \text{ otherwise}$$

Aeration stress starts when SW exceeds AWC by more than 85% of the drainable range $POR - AWC$. Leaf area follows the leaf area curve of the plant (`frac_hu1`, `lai_max1`, `frac_hu2`, `lai_max2` and `lai_pot` in `plants.plt`), with the daily increase scaled by $\sqrt{\gamma}$. See plant growth for the background of these relations.

Yield

When the season ends, the yield is

$$Y = 0.8\, B\, HI', \qquad HI' = (HI - HI_{min}) \frac{wur}{wur + \exp(6.13 - 0.0883\, wur)} + HI_{min}$$

where HI is `harv_idx`, HI_{min} is `harv_idx_ws` of `plants.plt`, and wur is 100 times the ratio of actual to potential evapotranspiration summed over the season (100 when the potential sum is below 10 mm). Biomass and leaf area are then set to zero.

Plant growth has no effect on the water balance or on erosion: evapotranspiration does not depend on leaf area, and `usle_c` is constant. The plant affects the water balance only through the growing season itself (the factor b) and through irrigation.

Irrigation

During the growing season, on days when the water stress factor ws is below 0.8, the soil is filled to the available water capacity with at most 76.2 mm. With `irr_src` set to `shal_aqu` or `deep_aqu` the water is taken from the HRU-lte's shallow or deep store and limited to what it contains; any other value takes it from an unlimited source outside the basin.

In 62.0.1 irrigation is switched on when `irr_flag` sorts alphabetically after `no_irr`. The value `irr`, listed as the "irrigation" value in earlier documentation, sorts before `no_irr` and so leaves irrigation off. Values such as `yes` or `sprinkler` switch it on.

Erosion

Sediment yield is computed with MUSLE (see MUSLE). The slope length and steepness factor is computed at the start of the run from `slp` and `slp_len`; the `usle_ls` column is not used:

$$LS = \left(\frac{\text{slp_len}}{22.128}\right)^{m} \left(65.41\,\sin^2\theta + 4.56\,\sin\theta + 0.065\right), \qquad m = 0.6\left(1 - e^{-35.835\,\text{slp}}\right)$$

with $\theta = \arctan(\text{slp})$. Sediment from surface runoff is

$$sed = 11.8\,(Q_{surf}\; q_{peak} \cdot 1000\,A)^{0.56}\; K\,C\,P\,LS$$

with K, C and P from `usle_k`, `usle_c` and `usle_p`, and A the `area` of `hru-lte.con`. A concentration of 500 mg/L in lateral and groundwater flow is added to this. There is no snow cover or residue effect.

Time of concentration

The peak runoff rate uses the time of concentration t_c in seconds (`t_conc` times 60):

$$q_{peak} = \frac{2\,Q_{surf} \cdot 1000\,A}{1.5\,t_c} + \frac{(Q_{lat} + Q_{gw}) \cdot 1000\,A}{86400}$$

When `t_conc` is zero, SWAT+ tries to estimate it with the Kirpich formula, using `area` as the channel length. The slope in that formula is always zero in 62.0.1, so the estimate is infinite: depending on how the model was compiled the run either stops with a floating-point error or continues with no surface runoff in the peak rate. Give `t_conc` a value above zero.

Area units

The HRU-lte calculations convert a depth D in mm to a volume with $1000\,D\,A$ and report sediment yield per $100\,A$, which are the conversions for an area A in km². A is the `area` column of `hru-lte.con`, which is in hectares for every other object type and which the SWAT+ Editor writes in hectares. With hectares, the flow volumes and sediment loads an HRU-lte sends downstream are 100 times the depth over its area, and the peak rate and sediment yield in the output are affected in the same way. Only the total hydrograph is filled: an HRU-lte connection in `hru-lte.con` that asks for the surface, lateral, tile or recharge hydrograph sends nothing.

Calibration

Parameters with the suffix `_lte` in `calibration.cal` change `hru-lte.hru` values (see simplified HRU parameters). The changes are applied after the file has been read and the HRU-lte objects have been set up, so in 62.0.1 they have an effect only for values the daily calculation reads directly from the file:

- effective: `awc_lte` (available water capacity of the object), `slope_lte` and `slopelen_lte` (lateral flow only), `revapc_lte`, `percc_lte`, `abf_lte` and `tc_lte`
- no effect: `cn2_lte`, `etco_lte`, `soildep_lte`, `sw_lte`, `sy_lte`, `gw_lte`, `gwflow_lte`, `gwdeep_lte`, `snow_lte`, `uslek_lte`, `uslec_lte`, `uslep_lte` and `uslels_lte`

`abf_lte` changes the daily recession factor $e^{-\alpha}$ in place of `alpha_bf`, and `tc_lte` changes the time of concentration in seconds. Except for `awc_lte`, a change to object number n is applied to the `hru-lte.hru` row with `id` n; the two agree when row numbers follow the objects, as the SWAT+ Editor writes them.

Example

No `hru-lte.hru` is included in the SWAT+ reference data. The file below is illustrative, with the column headers the SWAT+ Editor writes.

```
hru-lte.hru: illustrative example
id  name    area   cn2  cn3_swf  t_conc  soil_dp  perc_co   slp  slp_len  et_co  aqu_sp_yld  alpha_bf  rev
1   hlt001  52.3  77.0     0.00    26.0   1500.0     1.00  0.04     60.0   1.00        0.05      0.05   0.
```

One HRU-lte with a 1.5 m loam profile that starts half full. Percolation starts above field capacity (`perc_co` 1). Corn grows between the `pl_grow_sum` and `pl_end_sum` tables with no extra stress (`stress` 1), there is no irrigation and no tile drainage, and potential evapotranspiration follows Hargreaves. The values of `area`, `aqu_sp_yld`, `aqu_init`, `aqu_sh_flo`, `aqu_dp_flo`, `snow_h2o`, `lat`, `trop_flag` and `usle_ls` have no effect in 62.0.1.

Related files

- Connection files: `hru-lte.con` defines the HRU-lte objects, their area, weather station and outflow connections; its `hlt` column selects a row of this file by `id`.
- Object counts: the `lhru` column of `object.cnt` gives the number of HRU-lte objects.
- Master file: `file.cio` names this file on its `hru` line, and `soils_lte.sol` on its `soils` line.
- Soil texture classes: `soils_lte.sol` supplies available water capacity, porosity and conductivity for `soil_text`.
- Plant parameters: `plants.plt` supplies the plant parameters for `plnt_typ`.
- Decision tables: the `grow_start` and `grow_end` tables in `lum.dtl`.
- Weather files: the weather station named in `hru-lte.con` supplies daily weather and, through its weather generator, the temperatures for the heat units to maturity.
- Water balance output: `hru-lte_wb` files; plant growth and stress go to the `hru-lte_pw` files.

Sources

- Reader: `hru_lte_read.f90`, called from `main.f90.in` after `dtbl_lum_read`
- Type definitions: `type swatdeg_hru_data` and `type swatdeg_hru_dynamic` in `hru_lte_module.f90`
- Daily water balance, plant growth, irrigation and erosion: `hru_lte_control.f90`, called from `command.f90`
- Retention curve shape: `ascrv.f90`
- Growing season actions `grow_init` and `grow_end`: `actions.f90`; conditions with `obj hlt`: `conditions.f90`
- Texture classes: `soil_lte_db_read.f90`, `type soil_lte_database` in `soil_data_module.f90`
- Plant parameters: `plant_data_module.f90`, `plant_parm_read.f90`
- Connection and object numbering: `hyd_connect.f90`, `hyd_read_connect.f90`
- Delivery ratio from `t_conc`: `dr_ru.f90`
- Calibration: `cal_parm_select.f90`, `cal_parmchg_read.f90`, `proc_cal.f90`
- Output: `hru_lte_output.f90`, `output_landscape_init.f90`
- File name slot: `in_hru%hru_ez` in `input_file_module.f90`, read in `readcio_read.f90`

Column	Fortran variable
id	row index of hlt_db
name	hlt_db%name
area	hlt_db%dakm2
cn2	hlt_db%cn2
cn3_swf	hlt_db%cn3_swf
t_conc	hlt_db%tc
soil_dp	hlt_db%soildep
perc_co	hlt_db%perco
slp	hlt_db%slope
slp_len	hlt_db%slopelen
et_co	hlt_db%etco
aqu_sp_yld	hlt_db%sy
alpha_bf	hlt_db%abf
revap	hlt_db%revapc
rchg_dp	hlt_db%percc
sw_init	hlt_db%sw
aqu_init	hlt_db%gw
aqu_sh_flo	hlt_db%gwflow
aqu_dp_flo	hlt_db%gwdeep
snow_h2o	hlt_db%snow
lat	hlt_db%xlat
soil_text	hlt_db%text
trop_flag	hlt_db%tropical
grow_start	hlt_db%igrow1
grow_end	hlt_db%igrow2
plnt_typ	hlt_db%plant
stress	hlt_db%stress
pet_flag	hlt_db%ipet
irr_flag	hlt_db%irr
irr_src	hlt_db%irrsrc
t_drain	hlt_db%tdrain
usle_k	hlt_db%uslek
usle_c	hlt_db%uslec
usle_p	hlt_db%uslep
usle_ls	hlt_db%uslels

Purpose

A routing unit collects the outflow of a group of objects, usually the HRUs of one landscape unit, and routes the sum as a single object. Its row in rout_unit.con sends the result on, typically the total flow to a channel and the recharge to an aquifer. Three files describe routing units: rout_unit.def lists the elements of each unit, rout_unit.ele describes each element, and rout_unit.rtu gives each unit its topography and field records.

A landscape unit is a group of HRUs used only for output. ls_unit.def lists the elements of each landscape unit and ls_unit.ele gives each element its area fractions. The same element file also weights the HRUs in the basin output files. aqu_catunit.ele lists aquifer elements; in 62.0.1 its only effect is to switch on the data rows of the basin aquifer output.

The files are named in file.cio:

File	file.cio line	Slot	Read
rout_unit.def	routing_unit (line 8)	1	when the rtu count in object.cnt is above 0
rout_unit.ele	routing_unit (line 8)	2	when the rtu count is above 0
rout_unit.rtu	routing_unit (line 8)	3	when the rtu count is above 0
ls_unit.ele	regions (line 26)	1	every run

File	file.cio line	Slot	Read
ls_unit.def	regions (line 26)	2	every run
aqu_catunit.ele	regions (line 26)	9	every run, with the calibration inputs

Slot 4 of the routing_unit line (rout_unit.dr) is read from file.cio but never opened.

All six files are free format and whitespace-separated. The readers take the values in column order and do not check the header names.

For rout_unit.rtu, a name of null or a name that does not exist in the run folder means the file is not read. The other five files are skipped when the name is null. When the name is not null and the file does not exist, SWAT+ creates an empty file of that name in the run folder, reads nothing from it, and continues as if the name were null. What a missing file means for the run is given under each file.

rout_unit.def

Format

Line 1 is a title and line 2 is the column header; both are skipped. Each later line defines one routing unit. The line ends with a list of element numbers, and only the first number of that list has a header.

#	Column	Units	Description
1	id	-	Number of the routing unit. Not used: the rows are taken in order, so row n defines routing unit n.
2	name	-	Name of the routing unit. Not used.
3	elem_tot	-	Number of values in the element list that follows. When the list uses ranges, this is smaller than the number of elements.
4 and on	elements	-	Element list: id values of rows in rout_unit.ele, written as described under Element lists.

Element lists

rout_unit.def and ls_unit.def use the same list format. Each value is an element number, and a negative value closes a range that starts at the value before it. The list 1 -4 7 therefore means elements 1, 2, 3, 4 and 7, and elem_tot is 3 because the list has three values. A list with a single value gives one element. An elem_tot of 0 gives a unit with no elements.

A negative value must follow a positive one. Two negative values in a row, or a negative first value in a list of more than one value, put negative element numbers into the list.

Behaviour

- Routing unit n is the routing unit whose id in rout_unit.con is n. It uses row n of rout_unit.def, counted from the first data line, whatever the id column says. SWAT+ reads as many rows as the rtu count in object.cnt. Extra rows are ignored, and units without a row have no elements.
- The define column of rout_unit.rtu is not used to find the row. The routing unit ids in rout_unit.con, the rows of rout_unit.def and the ids in rout_unit.rtu therefore have to run 1, 2, 3 in the same order. The SWAT+ Editor writes them that way.
- An object can be an element of one routing unit only. If two element rows of rout_unit.ele point to the same object and both are listed in rout_unit.def (in one unit or in two), SWAT+ stops at start-up with an allocation error.
- A routing unit with no elements produces no outflow, and no message is written.
- If rout_unit.def is missing or null in a project with routing units, every routing unit has no elements. The HRUs then reach the rest of the model only through their rows in hru.con, and an HRU without outflow connections there loses its water.

Example

`refdata/0su_1hru/rout_unit.def` in the SWAT+ repository:

```
rout_unit.def: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
      id              name  elem_tot  elements
       1             rtu001         1         1
```

Routing unit 1 has one element, element 1 of `rout_unit.ele`.

rout_unit.ele

Format

Line 1 is a title and line 2 is the column header; both are skipped. Each later line is one element.

#	Column	Units	Description
1	`id`	-	Element number, used in the element lists of `rout_unit.def`. The rows can be in any order and the numbers do not have to be consecutive; the largest `id` sets the size of the table.
2	`name`	-	Name of the element. Not used.
3	`obj_typ`	-	Object type of the element, one of the codes below.
4	`obj_id`	-	Number of the object within its type: its `id` in the connection file of that type.
5	`frac`	fraction	Fraction of the element in the routing unit, used to scale the element's outflow. See Expansion factor.
6	`dlr`	-	Delivery ratio: `0` or `calc`, `1` or `full`. See Delivery ratio.

The object type codes SWAT+ recognises are:

`obj_typ`	Object	Connection file
`hru`	HRU	`hru.con`
`hlt`	HRU-lte	`hru-lte.con`
`ru`	routing unit	`rout_unit.con`
`cha`	legacy channel	`channel.con`
`sdc`	SWAT+ channel	`chandeg.con`
`res`	reservoir	`reservoir.con`
`exc`	export coefficient	`exco.con`
`dr`	delivery ratio	`delratio.con`
`out`	outlet	`outlet.con`

Any other code, including `aqu`, `rec` and `rtu`, is not recognised. No message is written, and the element is linked to object 1, the first object in the model (HRU 1 in a model with HRUs). Earlier documentation listed aquifers, recall objects and "another rtu" as possible elements; in 62.0.1 a routing unit inside a routing unit has the code `ru`.

Expansion factor

Each day a routing unit adds up the hydrographs of its elements. Each element's hydrographs are multiplied by an expansion factor e before they are added:

- HRU and HRU-lte elements with `frac` below 0.99999:

$$e = \text{frac}\ \frac{A_{ru}}{A_{obj}}$$

  where A_{ru} is the routing unit's `area` in `rout_unit.con` and A_{obj} the HRU's `area` in its connection file. When `frac` is the HRU's share of the routing unit area, e is 1 and the whole HRU outflow is added.

- HRU and HRU-lte elements with `frac` of 0.99999 or more: $e = 1$, so the whole HRU outflow is added whatever the areas.
- Export coefficient elements: e as for HRUs when the export coefficient object has an area above 0.01 ha, otherwise 1.
- All other elements: $e =$ frac, the fraction of the object's outflow that enters the routing unit.

The `frac` values are not checked. If they do not add up as intended, the routing unit outflow is scaled accordingly.

The factor and the delivery ratio apply to the total, surface, lateral and tile hydrographs. The recharge hydrograph is multiplied by the factor only. Pesticide, salt and constituent loads are added in full: 62.0.1 applies neither the expansion factor nor the delivery ratio to them.

When another object sends flow to a routing unit through its connection file, the inflow is shared among the unit's elements. Each element receives the sender's fraction multiplied by its `frac`. How an HRU uses that inflow is described under Inflow to HRUs and routing units.

Delivery ratio

The `dlr` column sets a ratio for each hydrograph variable of the element:

`dlr`	Ratio
`0` or `calc`	Flow 1. All other variables (sediment, nutrients, chlorophyll-a, CBOD, dissolved oxygen, sediment size classes) are multiplied by a calculated ratio, given below.
`1` or `full`	1 for every variable.
any other value	0 for every variable.

For HRU and HRU-lte elements the calculated ratio is

$$DR = \min\left(1,\ \sqrt{\frac{t_{c,\mathrm{elem}}}{t_{c,ru}}}\right)$$

where $t_{c,\mathrm{elem}}$ is the HRU's time of concentration (see Surface runoff lag), or for an HRU-lte the time of concentration of its `hru-lte.hru` record, and $t_{c,ru}$ is the routing unit's time of concentration (see Time of concentration). For `sdc` and `ru` elements the calculated ratio is 1. For the other element types no ratio is calculated, and the element takes the ratio last calculated for an earlier element (0 if there is none).

A `dlr` value that names a record in `delratio.del` does not work in 62.0.1. The names are looked up when the routing units are set up, and `delratio.del` is read only after that, so no name is ever found. The ratio then stays at 0 for every variable, and the element contributes only its recharge hydrograph. The same happens for any other value, such as `null`. Use `0`, `calc`, `1` or `full`. The SWAT+ Editor writes `0`.

The decision table action `hru_fr_update` (see decision tables) reads a replacement `rout_unit.ele` during the run. By then `delratio.del` has been read, so a `dlr` that names a `delratio.del` record does take that record's ratios from `dr_om.del`. Elements whose `dlr` is `0`, `calc`, `1`, `full` or an unknown name keep the ratios they had before.

Behaviour

- `rout_unit.ele` is required in a project with routing units. If it is missing or `null`, the routing unit tables are never set up and the run fails during start-up.
- In sub-daily runs (time step below one day) with Green and Ampt infiltration (`gampt` = 1 in `codes.bsn`), the routing unit's sub-daily flow is set from each HRU element in turn, so only the last HRU element in the list is kept, and the result is shifted by a fixed 2 hours instead of the routing unit's time of concentration.
- The `ru` row of `print.prt` writes the sum of the element hydrographs of each routing unit to the `ru_*` files (see ru files).

Example

`refdata/Osu_1hru/rout_unit.ele`:

```
rout_unit.ele: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
      id  name                    obj_typ    obj_id          frac               dlr
       1  hru0001                     hru         1             1                 0
```

Element 1 is HRU 1. With `frac` = 1 the whole HRU outflow enters the routing unit. With `dlr` = 0 the water is delivered in full, and sediment and nutrients are multiplied by the calculated delivery ratio.

rout_unit.rtu

Format

Line 1 is a title and line 2 is the column header; both are skipped. Each later line is one routing unit.

#	Column	Units	Description
1	`id`	-	Number of the routing unit. Routing unit n (the `id` in `rout_unit.con`) uses the row whose `id` is n. The ids must lie between 1 and the `rtu` count in `object.cnt`. An `id` of 0 ends the file.
2	`name`	-	Name of the routing unit. Not used.
3	`define`	-	Name of the `rout_unit.def` row. Not used: routing unit n always uses row n of `rout_unit.def`.
4	`dlr`	-	Delivery ratio of the routing unit. Not used; delivery ratios are set per element in `rout_unit.ele`.
5	`topo`	-	Name of a record in `topography.hyd`. Its slope and slope length are used for the routing unit's time of concentration.
6	`field`	-	Name of a record in `field.fld`. Its field length is used for the routing unit's time of concentration.

Time of concentration

The routing unit's time of concentration is the sum of an overland flow time and a channel flow time:

$$t_{ov} = \frac{0.0556\,(L_{slp}\,n)^{0.6}}{(slp + 0.001)^{0.3}}, \qquad t_{ch} = \frac{0.62\,L\,n^{0.75}}{A^{0.125}\;(0.5\,(slp + 0.001))^{0.375}}, \qquad t_{c,ru} = t_{ov} + t_{ch}$$

Symbol	Meaning	Units
t_{ov}, t_{ch}, $t_{c,ru}$	overland, channel and total time of concentration	hr
L_{slp}	slope length, `slp_len` of the `topo` record	m
slp	average slope, `slp` of the `topo` record	m/m
L	channel length, `len` of the `field` record divided by 1000	km
A	routing unit area, `area` in `rout_unit.con`	km²
n	Manning's n for overland flow, averaged over the HRU elements	-

The Manning's n is the area-weighted mean of the `ov_mann` values of the HRU elements' land uses (see `landuse.lum`):

$$n = \frac{\sum_i n_i\,A_i}{A}$$

where A_i is the full area of HRU element i; `frac` is not used. When the routing unit has an element that is not an HRU, the running sum is set to 0.1 at that element and the HRU elements after it are added to 0.1. The result then depends on the order of the elements.

The time of concentration is used for the calculated delivery ratio of the elements and, in sub-daily runs, for the unit hydrograph of the routing unit.

Behaviour

- A `topo` name that is `null` or not found in topography.hyd gives the first record of that file. A `field` name that is `null` or not found gives the built-in field length of 500 m. No message is written in either case.
- `rout_unit.rtu` is required in a project with routing units. If it is missing or `null`, no routing unit records are set up and the run fails during start-up.

Example

`refdata/Osu_1hru/rout_unit.rtu`:

```
rout_unit.rtu: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
      id              name            define               dlr              topo             field
       1            rtu01             rtu001              null       toportu011            fld011
```

`fld011` is in field.fld with a length of 500 m. `toportu011` is not in this dataset's topography.hyd, so the routing unit uses its first record, `topohru0001` (slope 0.01078, slope length 10 m), without a message. `define` and `dlr` have no effect.

ls_unit.def

Format

Line 1 is a title, line 2 gives the number of landscape units, and line 3 is the column header. Each later line defines one landscape unit, and the line ends with an element list as in `rout_unit.def`.

#	Column	Units	Description
1	`id`	-	Number of the landscape unit. Not used: row n defines landscape unit n.
2	`name`	-	Name of the landscape unit, written in the `name` column of the landscape unit output files.
3	`area`	ha	Area of the landscape unit. Not used.
4	`elem_tot`	-	Number of values in the element list. 0 gives a landscape unit with no elements, whose output is all zeros.
5 and on	`elements`	-	Element list: `id` values of rows in `ls_unit.ele`, written as described under Element lists.

SWAT+ reads as many rows as line 2 gives.

Behaviour

- Landscape units are used for output only. Each one gets a row in the `lsunit_wb`, `lsunit_nb`, `lsunit_ls` and `lsunit_pw` files and in the landscape unit carbon files (`lsu_cb_gl`, `lsu_cb_trf`, `lsu_cb_plt`), switched on in print.prt. Each value is the sum over the unit's elements of the HRU (or HRU-lte) value multiplied by the element's `sub_frac` in `ls_unit.ele`. See Aggregation.
- The element number is used as the HRU number: element n adds the output of HRU n (or HRU-lte n), and its `obj_typ_no` is not consulted. The element ids in `ls_unit.ele` therefore have to equal the HRU numbers.
- In the output files, `unit` is the landscape unit number and `gis_id` is taken from the routing unit with the same number in rout_unit.con.
- With gwflow and landscape unit connection (`conn_type` 2, see lsucell.gw), the element lists also decide which HRUs pass recharge and remaining evapotranspiration to the cells of each landscape unit. Here too element n is HRU n.
- `ls_unit.def` needs `ls_unit.ele`. If `ls_unit.def` is missing or `null`, the project has no landscape units and the landscape unit output files are not opened.

Example

`refdata/Osu_1hru/ls_unit.def`:

```
ls_unit.def: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
1
      id              name          area  elem_tot  elements
       1            rtu001       10.0000         1         1
```

One landscape unit of 10 ha, named `rtu001` after the routing unit, with element 1 of `ls_unit.ele`.

ls_unit.ele

Format

Line 1 is a title and line 2 is the column header; both are skipped. Each later line is one element. Six values are read from each line.

#	Column	Units	Description
1	`id`	-	Element number, used in the element lists of `ls_unit.def`. The largest `id` sets the size of the table. Must equal the HRU number (see below).
2	`name`	-	Name of the element. Not used.
3	`obj_typ`	-	`hru` or `hlt`. Elements of any other type are skipped in the sums.
4	`obj_typ_no`	-	HRU or HRU-lte number.
5	`bsn_frac`	fraction	Fraction of the basin land area (`ls_area` in `object.cnt`) covered by the HRU. Weight of the HRU in the basin output files.
6	`sub_frac`	fraction	Fraction of the landscape unit area covered by the HRU. Weight of the HRU in the landscape unit output files. Called `lsu_frac` in earlier documentation; the `Ames_sub1` file labels it `RU_FRAC`.

Behaviour

- Basin output: the `basin_wb`, `basin_nb`, `basin_ls` and `basin_pw` files, including the starting soil water and snow in `basin_wb`, are the sums of the HRU values multiplied by `bsn_frac`. The fractions are used as given; if they do not add up to 1, the basin values are scaled by their sum.
- For each HRU number n, the basin sum reads row n, takes its `obj_typ_no` m, and then uses the `obj_typ` and `bsn_frac` of row m with the output of HRU m. The sum is right when row n describes HRU n, which is how the SWAT+ Editor writes the file. HRU-lte objects are summed the same way over the first rows of the file.
- Landscape unit output uses `sub_frac` (see `ls_unit.def`).
- The soft calibration of the whole basin in `water_balance.sft` takes each HRU's area as `ls_area` times its `bsn_frac` (see soft calibration).
- The decision table action `hru_fr_update` reads a replacement `ls_unit.ele` during the run and sets the area of each `hru` element to `bsn_frac` times `ls_area`.
- If `ls_unit.ele` is missing or `null`, the basin landscape files are opened and get their headers, and no data rows are written. The routine that stores the basin's starting soil water at the end of the warm-up years still reads the element table, so a model with HRUs and no `ls_unit.ele` can stop with a memory error at that point. Treat the file as required in any model with HRUs.

Example

`refdata/Ames_sub1/ls_unit.ele`:

```
lsu_unit.ele
          ID    NAME      OBJ_TYP  OBJ_TYP_NO    BSN_FRAC       RU_FRAC
           1    hru0001       hru           1     0.20           0.0
           2    hru0002       hru           2     0.20           0.0
```

```
 3    hru0003      hru          3     0.20          0.0
 4    hru0004      hru          4     0.20          0.0
 5    hru0005      hru          5     0.10          0.0
 6    hru0006      hru          6     0.10          0.0
 7    hru0007      hru          7     0.20          0.0
 8    hru0008      hru          8     0.20          0.0
 9    hru0009      hru          9     0.20          0.0
10    hru0010      hru         10     0.20          0.0
11    hru0011      hru         11     0.10          0.0
12    hru0012      hru         12     0.10          0.0
```

This file is not in the SWAT+ Editor layout: it has six columns and upper-case headers. Each row describes the HRU of the same number, as the basin sums require. The `BSN_FRAC` values add up to 2.0, so every basin output of this dataset is twice the area-weighted HRU average. `Ames_sub1` has no `ls_unit.def`, so the `RU_FRAC` values of 0 are not used.

aqu_catunit.ele

Format

Line 1 is a title and line 2 is the column header; both are skipped. Each later line is one element, and seven values are read from it. A line with fewer values takes the missing ones from the next line.

#	Column	Units	Description
1	`id`	-	Element number. The largest `id` sets the size of the table.
2	`name`	-	Name of the element.
3	`obj_typ`	-	Object type, `aqu` in files written by the SWAT+ Editor.
4	`obj_typ_no`	-	Aquifer number.
5	`bsn_frac`	fraction	Fraction of the basin covered by the aquifer.
6	`sub_frac`	fraction	Fraction of the aquifer region covered by the element.
7	`reg_frac`	fraction	Fraction of the calibration region covered by the element.

Behaviour

- In 62.0.1 the file works as a switch. When it has at least one element, the basin aquifer files (`basin_aqu` row in `print.prt`) get one data row per interval. Without it they get headers only.
- The basin aquifer values are weighted by each aquifer's area divided by `tot_area` in `object.cnt` (see aquifer output). The columns of `aqu_catunit.ele` are not used for this.
- The columns are meant for aquifer regions defined in `aqu_catunit.def` and `aqu_reg.def`. SWAT+ 62.0.1 reads those files, but no calculation or output uses the regions.

Example

The first rows of `refdata/Osu_1hru/aqu_catunit.ele`:

```
aqu_catunit.ele: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
      id  name                    obj_typ  obj_typ_no      bsn_frac      sub_frac      reg_frac
       1  aqu011                      aqu           1       0.00241       0.00000       0.00000
       2  aqu012                      aqu           2       0.10460       0.00000       0.00000
       3  aqu021                      aqu           3       0.01290       0.00000       0.00000
```

The file has 21 elements, while the dataset has one aquifer. The rows have no effect beyond switching on the basin aquifer output. In `refdata/Ames_sub1` the file is empty, so that dataset writes no basin aquifer data rows.

Columns in earlier documentation

- `ls_unit.ele`: the SWAT+ Editor writes a seventh column, `reg_frac`. SWAT+ 62.0.1 reads six values per line and ignores it.
- `ls_unit.def`: earlier documentation did not list the line with the number of landscape units (line 2) or the `area` column. Both are read, and a file without them is read out of step.

Related files

- `rout_unit.con`: the routing unit objects, their areas and their outflow connections.
- `hru.con` and `hru-data.hru`: the HRUs that are elements of routing and landscape units.
- `object.cnt`: the `rtu` count, and `ls_area` and `tot_area` used with `bsn_frac` and the basin aquifer output.
- `file.cio`: names the files on the `routing_unit` and `regions` lines.
- `topography.hyd` and `field.fld`: the `topo` and `field` records of `rout_unit.rtu`.
- `aquifer.aqu`: the aquifers listed in `aqu_catunit.ele`.
- `print.prt`: switches for the `ru`, `lsunit_*`, `basin_*` and `basin_aqu` outputs.
- Decision tables: the `hru_fr_update` action.
- Outputs: water balance, nutrient balance and losses, hydrographs, aquifer output.
- Delivery ratios in the model theory.

Sources

- File name slots: `type input_ru` and `type input_regions` in `input_file_module.f90`, read in `readcio_read.f90`; read order in `main.f90.in` and `hyd_connect.f90`
- Routing unit readers: `ru_read.f90` (`rout_unit.rtu`), `ru_read_elements.f90` (`rout_unit.ele`, `rout_unit.def`); element lists: `define_unit_elements.f90`
- Routing unit types: `ru_parameters` in `ru_module.f90`, `routing_unit_data` and `routing_unit_elements` in `hydrograph_module.f90`
- Routing unit linkage and inflow sharing: `hyd_connect.f90`, `hyd_read_connect.f90`
- Daily routing unit sum: `ru_control.f90`, called from `command.f90`
- Delivery ratios: `dr_ru.f90`, `dr_db_read.f90`
- Time of concentration and unit hydrographs: `time_conc_init.f90`, `unit_hyd_ru_hru.f90`
- Landscape unit reader: `lsu_read_elements.f90`; types `landscape_units` and `landscape_elements` in `calibration_data_module.f90`
- Landscape unit and basin output: `lsu_output.f90`, `lsu_carbon_output.f90`, `basin_output.f90`, `basin_sw_init.f90` (called from `time_control.f90`), file opening in `output_landscape_init.f90`
- Other uses of `ls_unit.ele` and `ls_unit.def`: `lcu_read_softcal.f90`, `hru_fr_change.f90` (action `hru_fr_update` in `actions.f90`), `gwflow_rech.f90`, `gwflow_gwet.f90`
- Aquifer elements: `aqu_read_elements.f90`, called from `proc_cal.f90`; basin aquifer output in `basin_aquifer_output.f90`, files opened in `header_write.f90`

File	Column	Fortran variable
`rout_unit.def`	`id`	not stored
`rout_unit.def`	`name`	`ru_def%name`
`rout_unit.def`	`elem_tot`	`nspu`
`rout_unit.def`	`elements`	`elem_cnt`, expanded into `ru_def%num` and `ru_def%num_tot`
`rout_unit.ele`	`id`	index of `ru_elem`
`rout_unit.ele`	`name`	`ru_elem%name`
`rout_unit.ele`	`obj_typ`	`ru_elem%obtyp`
`rout_unit.ele`	`obj_id`	`ru_elem%obtypno` (object number in `ru_elem%obj`)
`rout_unit.ele`	`frac`	`ru_elem%frac`
`rout_unit.ele`	`dlr`	`ru_elem%dr_name` (ratios in `ru_elem%dr`)
`rout_unit.rtu`	`id`	index of `ru`
`rout_unit.rtu`	`name`	`ru%name`
`rout_unit.rtu`	`define`	`ru%dbsc%elem_def`
`rout_unit.rtu`	`dlr`	`ru%dbsc%elem_dr`
`rout_unit.rtu`	`topo`	`ru%dbsc%toposub_db` (index `ru%dbs%toposub_db`)
`rout_unit.rtu`	`field`	`ru%dbsc%field_db` (index `ru%dbs%field_db`, values in `ru%field`)

File	Column	Fortran variable
ls_unit.def	line 2	mlsu, stored as db_mx%lsu_out
ls_unit.def	name	lsu_out%name
ls_unit.def	area	lsu_out%area_ha
ls_unit.def	elements	lsu_out%num, lsu_out%num_tot
ls_unit.ele	obj_typ	lsu_elem%obtyp
ls_unit.ele	obj_typ_no	lsu_elem%obtypno
ls_unit.ele	bsn_frac	lsu_elem%bsn_frac
ls_unit.ele	sub_frac	lsu_elem%ru_frac
aqu_catunit.ele	obj_typ, obj_typ_no	acu_elem%obtyp, acu_elem%obtypno
aqu_catunit.ele	bsn_frac, sub_frac, reg_frac	acu_elem%bsn_frac, acu_elem%ru_frac, acu_elem%reg_frac
aqu_catunit.ele	largest id	db_mx%aqu_elem

Purpose

The **link** line of file.cio names two optional files that connect objects outside the routing set up by the connectivity files:

- **chan-surf.lin** lists, for a SWAT+ channel, the HRUs or routing units on its floodplain. On days with overbank flow, the wetlands of those HRUs are filled from the channel.
- **aqu_cha.lin** lists, for an aquifer, the channels that receive its return flow. The flow is shared among the channels by channel length and drainage area.

The line gives the two names in this order. The defaults, **chan-surf.lin** and **aqu_cha.lin**, apply only when file.cio ends before the line (see the master file). Projects that do not use the files give **null** for both, as the reference datasets in the SWAT+ repository do. Earlier documentation called the first file **chan_surf.lin**.

Neither file has column names that SWAT+ checks. Both readers skip the header line and take the values by position, so the column names on this page are descriptive.

chan-surf.lin

Free format, whitespace-separated. Line 1 is a title and is skipped. Line 2 is a single whole number, the number of channel rows; SWAT+ also counts the rows itself, and the count only sets a minimum size for the table. Line 3 is a column header and is skipped. Each later line links one channel to its floodplain objects:

#	Column	Units	Description
1	id	-	Number of the SWAT+ channel, as numbered in chandeg.con.
2	name	-	Name of the floodplain. Not used in calculations.
3	num_obj	count	Number of floodplain objects that follow. A row with 0 gives the channel no floodplain links.
4	obj_typ	-	Type of the first floodplain object: **hru** or **ru** (routing unit).
5	obj_id	-	Number of the first floodplain object, as numbered in hru.con or rout_unit.con.
6, 7, ...	obj_typ, obj_id	-	One further pair for each remaining object, **num_obj** pairs in all.

Object types other than **hru** and **ru** are read and then ignored. For an **ru** object, SWAT+ takes the element numbers that rout_unit.def lists for that routing unit and uses each one as an HRU number. This selects the routing unit's HRUs only when each element number in rout_unit.ele equals the number of the HRU it refers to.

The links apply to channels routed with the SWAT+ channel method (chandeg.con). Legacy channels (channel.con) make no use of them.

Overbank flow to floodplain wetlands

Overbank flow happens on days when the channel's peak flow rate is above the bankfull threshold described under floodplain deposition. The overbank volume V_{ob} is the floodplain volume of the channel's rating curve at the peak flow rate. After floodplain deposition, SWAT+ goes through the floodplain HRUs in the order listed (a routing unit contributes its HRUs in the order of its elements). For each HRU it takes the room left in the HRU's wetland,

$$V_{fill} = V_{emr} - V_{wet}$$

where V_{emr} is the wetland's emergency volume and V_{wet} its current storage (m^3), and subtracts it from the overbank volume still available. When the overbank volume runs out, V_{fill} is cut to what was left. The wetland then receives the fraction

$$r = \min\left(1,\ \frac{V_{fill}}{Q_{in}}\right)$$

of the channel's inflow Q_{in} for the day (m^3), with the same fraction of its sediment, nutrients and other constituents, and the channel inflow is reduced by that fraction.

Only HRUs with surface storage, a `surf_stor` entry in `hru-data.hru` that names a record in `wetland.wet`, receive water. Other HRUs in the list take nothing from the channel. The water each wetland receives is reported in the `overbank` column (mm) of the HRU water balance output. See also overbank flow and floodplain deposition in the model theory, and the wetland inputs for the emergency volume.

Missing file and repeated read

SWAT+ reads `chan-surf.lin` while it sets up the SWAT+ channels, whether or not the model has any.

- If the slot is `null`, the file is skipped and no channel has floodplain links.
- If the slot gives a name and no file of that name exists, SWAT+ opens it anyway. This creates an empty file of that name in the run folder, nothing is read from it, and no channel has floodplain links.
- When `object.cnt` counts legacy channels (`cha` above 0), SWAT+ reads the file a second time, earlier in the setup, before the SWAT+ channels exist. If the file exists, the second read tries to set up the same table again and the run stops with an error.

aqu_cha.lin

Free format, whitespace-separated. Line 1 is a title and line 2 a column header; both are skipped. Each later line links one aquifer to a set of channels:

#	Column	Units	Description
1	`id`	-	Number of the aquifer, as numbered in `aquifer.con`.
2	`name`	-	Name of the link. Not used in calculations.
3	`num_elem`	count	Number of values in the channel list that follows. A row with 0 links no channels.
4, 5, …	`elements`	-	Channel numbers, as numbered in `chandeg.con`. A negative value marks a range: `1 -4` means channels 1 to 4.

The aquifer numbers must lie between 1 and the number of aquifers. SWAT+ reads as many rows as the largest aquifer number in the first column, so each aquifer should appear on one row only. Earlier documentation described the file as a replacement for `aquifer.con`. In 62.0.1 `aquifer.con` is still required, see overlap with aquifer outflows.

Return flow shares

For each linked aquifer, SWAT+ sorts the linked channels by drainage area, smallest first. The drainage area is the `area` column of `chandeg.con` and the channel length L_i is the `len` column of `hyd-sed-lte.cha` (km). With L_{tot} the total length of the linked channels, the length left when channel i and all smaller channels have stopped receiving flow is

$$L_{left,i} = L_{tot} - \sum_{j \le i} L_j$$

Each day the contributing channel length grows with the aquifer's return flow Q (mm):

$$L_c = L_{tot} \frac{Q}{\text{bf_max}}$$

where `bf_max` is given in `aquifer.aqu` (mm; see aquifer files). The first contributing channel k is the smallest channel with $L_c \ge L_{left,k}$, and each channel from k upward receives the share

$$f_i = \frac{L_i}{L_{left,k} + L_k}, \qquad i \ge k$$

while the smaller channels receive nothing. The shares of the contributing channels add up to 1, so the whole return flow of the aquifer is passed on. When Q is at or above `bf_max`, every linked channel receives flow in proportion to its length. As Q falls, the channels with the smallest drainage areas stop receiving flow first. See baseflow in the model theory.

Each linked channel adds its share of the aquifer's return flow hydrograph, with its nitrate, phosphorus, salts and constituents, to its inflow. This happens when the channel is routed: on the same day if the aquifer comes before the channel in the routing order, otherwise on the next day. The water appears in the `geo_bf` column (m^3/s) of the channel output.

Channel order, lengths and bf_max

- The shares are computed in drainage-area order. Each channel looks up its share by its position in the `aqu_cha.lin` list. A channel receives its intended share only when the list for each aquifer is already in order of increasing drainage area. In any other order, channels receive each other's shares; the total for the aquifer does not change.
- The channel length is taken from the row of `hyd-sed-lte.cha` whose position equals the channel's record number in `channel-lte.cha`. This gives the right length when both files list the channels in the same order.
- Once the file is read, the calculation divides by `bf_max` for every aquifer, including aquifers the file does not list. With `bf_max` = 0 the division fails, and builds that stop on floating-point errors (the Intel builds, for example) stop the run. Give every aquifer a `bf_max` above zero when `aqu_cha.lin` is used.

Overlap with aquifer outflows

`aqu_cha.lin` is read only when `object.cnt` counts aquifers, right after `aquifer.con`. The aquifer still routes its return flow through the outflows listed in `aquifer.con`, and SWAT+ also passes the shares above to the linked channels. A model that lists the same channels as outflows in `aquifer.con` and in `aqu_cha.lin` sends the return flow to them twice.

When gwflow is active, SWAT+ sets the aquifer count to 0 and `aqu_cha.lin` is not read.

Missing or short file

- If the slot is `null` or the file does not exist, the file is skipped and no aquifer is linked to channels.
- If the file exists but has fewer than two lines, the run stops with an end-of-file error.

Example

No dataset in the SWAT+ repository uses either file. The two files below are illustrative.

```
chan-surf.lin: illustrative example
2
id  name    num_obj  obj_typ  obj_id  obj_typ  obj_id
1   fp001   2        hru      4       hru      5
3   fp003   1        ru       2
```

Channel 1 is linked to HRUs 4 and 5, and channel 3 to the HRUs of routing unit 2. On a day with overbank flow in channel 1, the wetland of HRU 4 is filled first, then that of HRU 5, as long as overbank volume remains. An HRU without surface storage receives nothing.

```
aqu_cha.lin: illustrative example
id  name    num_elem  elements
1   aqu01   2         1  -4
2   aqu02   3         5  7  9
```

Aquifer 1 shares its return flow among channels 1 to 4, and aquifer 2 among channels 5, 7 and 9. For the shares to reach the intended channels, channels 1 to 4 must have increasing drainage areas in that order, and so must channels 5, 7 and 9. Neither aquifer should also route its return flow to these channels in aquifer.con.

Related files

- file.cio: names both files on the link line (master file).
- chandeg.con, aquifer.con, hru.con, rout_unit.con, rout_unit.def and rout_unit.ele: number the objects the files refer to (connectivity files).
- channel-lte.cha and hyd-sed-lte.cha: channel length and bankfull flow (channel tables).
- aquifer.aqu: bf_max and the return flow (aquifer files).
- hru-data.hru, wetland.wet and hydrology.wet: the surface storage that receives overbank water (HRU data, wetland inputs).
- object.cnt: the aquifer and legacy channel counts (object counts).

Sources

- File name slots: type input_link in input_file_module.f90, read in readcio_read.f90
- Readers: overbank_read.f90 (chan-surf.lin), aqu2d_read.f90 (aqu_cha.lin), with element ranges expanded in define_unit_elements.f90
- Call sites: hyd_connect.f90 (both readers), proc_cha.f90 (overbank_read again, sd_channel_surf_link, aqu2d_init); aquifer count reset for gwflow in basin_read_objs.f90
- Type definitions: floodplain_parameters in sd_channel_module.f90; channel_aquifer_elements and geomorphic_baseflow_channel_data in hydrograph_module.f90
- Floodplain HRU lists: sd_channel_surf_link.f90; routing unit elements from ru_read_elements.f90
- Overbank filling of wetlands: sd_channel_sediment3.f90; floodplain wetland storage in ch_rtmusk.f90; overbank output in hru_control.f90
- Channel sorting and lengths: aqu2d_init.f90
- Daily shares: aqu_1d_control.f90; added to channel inflow in sd_channel_control3.f90

File	Column	Fortran variable
chan-surf.lin	line 2 count	mcha_sp
chan-surf.lin	id	index i of sd_ch
chan-surf.lin	name	sd_ch%fp%name
chan-surf.lin	num_obj	nspu, sd_ch%fp%obj_tot
chan-surf.lin	obj_typ	sd_ch%fp%obtyp
chan-surf.lin	obj_id	sd_ch%fp%obtypno
aqu_cha.lin	id	index iaq of aq_ch
aqu_cha.lin	name	aq_ch%name
aqu_cha.lin	num_elem	nspu

File	Column	Fortran variable
aqu_cha.lin	elements	elem_cnt, expanded to aq_ch%num
aquifer.aqu	bf_max	aqudb%bf_max
hyd-sed-lte.cha	len	sd_chd%chl
chandeg.con	area	ob%area_ha

Purpose

This page covers three HRU parameter files and one channel file:

- hydrology.hyd: lateral flow, canopy storage, evaporation, percolation and enrichment parameters.
- topography.hyd: slope, slope lengths and the deposition coefficient. Also used by routing units.
- field.fld: field length and width for routing water and sediment across the landscape. Also used by routing units.
- shade_factor.shf: daily shade factors for the channel water temperature routine.

An HRU picks one record from each of the first three files by name, in the hydro, topo and field columns of hru-data.hru. hydrology.hyd, topography.hyd and field.fld are named on the hydrology line of file.cio. shade_factor.shf is not listed in file.cio; SWAT+ looks for a file with that name in the project folder.

All four files are free format and whitespace-separated. Line 1 is a title and line 2 is the column header; both are skipped. Each later line is one record. The readers do not match columns by header name, so every value must be present and in the order given. Record names are stored with at most 16 characters.

An unmatched or misspelt hydro or topo name in hru-data.hru makes the HRU use the first record of the file without a message (see name lookup). hydrology.hyd and topography.hyd must therefore exist and must contain every name used.

hydrology.hyd

#	Column	Units	Typical range	Description
1	name	-	-	Name of the hydrology record, matched against the hydro column of hru-data.hru.
2	lat_ttime	days	0 to 180	Lateral flow travel time, called lat_time in earlier documentation. 0 lets SWAT+ compute it from the soil. See Lateral flow.
3	lat_sed	mg/L	0 to 5000	Sediment concentration in lateral and tile flow.
4	can_max	mm H_2O	0 to 100	Maximum canopy storage when the canopy is fully developed.
5	esco	-	0.01 to 1	Soil evaporation compensation factor. See Soil evaporation.
6	epco	-	0.01 to 1	Plant uptake compensation factor.
7	orgn_enrich	-	0 to 5	Organic N enrichment ratio for loading with sediment. 0 lets SWAT+ compute it for each runoff event.
8	orgp_enrich	-	0 to 5	Organic and sediment-attached P enrichment ratio for loading with sediment. 0 lets SWAT+ compute it for each runoff event.
9	cn3_swf	-	0 to 1	Soil water factor for the moisture condition III curve number: 0 puts it at field capacity, 1 at saturation.
10	bio_mix	-	0 to 1	Biological mixing efficiency.
11	perco	-	0 to 1	Percolation coefficient for the bottom soil layer. See Percolation from the soil profile.

#	Column	Units	Typical range	Description
12	`lat_orgn`	mg/L	0 to 200	Organic N concentration in lateral flow.
13	`lat_orgp`	mg/L	0 to 200	Organic P concentration in lateral flow.
14	`pet_co`	-	0.8 to 1.2	Multiplier on potential evapotranspiration. Older files call this column `harg_pet`. See Potential evapotranspiration.
15	`latq_co`	-	0 to 1	Lateral flow coefficient. See Lateral flow.

For an HRU with tile drains (set in its `landuse.lum` record), SWAT+ replaces `cn3_swf` with 0.95 and `perco` with 0.1 at the start of the run, whatever the record gives, and calibration changes to these two parameters are skipped for that HRU.

Lateral flow

Lateral flow is computed for each soil layer below the first with a hillslope storage method:

$$q_{lat,ly} = 0.024 \text{ latq_co } \frac{2\,SW_{ex,ly}}{\phi_{d,ly}/h_{ly}}\; K_{sat,ly}\; \frac{slp}{L_{lat}}$$

where $SW_{ex,ly}$ is the water above field capacity in the layer (mm), $\phi_{d,ly}$ the drainable porosity (saturation minus field capacity, mm), h_{ly} the layer thickness (mm), $K_{sat,ly}$ the saturated hydraulic conductivity (mm/h), slp the `slp` value from `topography.hyd` and L_{lat} its `lat_len` (m). The result is limited to the water above field capacity. The top layer produces no lateral flow. `latq_co` scales lateral flow linearly; 0 turns it off.

Lateral flow does not leave the HRU on the day it is generated. A fraction of the stored lateral flow is released each day:

$$f_{lat} = 1 - \exp\left(-\frac{1}{TT_{lat}}\right)$$

where TT_{lat} is `lat_ttime` in days. When `lat_ttime` is 0, SWAT+ computes it from the soil and the lateral slope length:

$$TT_{lat} = \max\left(1,\; 10.4\,\frac{L_{lat}}{K_{sat,mx}}\right)$$

with $K_{sat,mx}$ the highest saturated hydraulic conductivity in the profile (mm/h). Set `lat_ttime` to a specific value only when the base flow behaviour of the watershed is well known.

Soil evaporation

The evaporative demand on the soil is distributed with depth. The cumulative demand down to depth z (mm) is

$$E_z = E_s\,\frac{z}{z + \exp(2.374 - 0.00713\,z)}$$

where E_s is the soil evaporation demand for the day. A layer whose top is at depth z_u and bottom at z_l is asked for

$$E_{ly} = E_{z_l} - (1 - \text{esco})\,E_{z_u}$$

so with `esco` = 0 each layer meets only its share of the depth distribution, and as `esco` increases more of the demand can be met from lower layers. Layers below the soil evaporation depth do not take part. The demand on a layer is further reduced when the layer is drier than field capacity.

Plant uptake compensation

`epco` is copied to every plant of the HRU's plant community when the plants are initialized. As `epco` approaches 1, more of the water uptake demand can be met by lower layers when upper layers are dry; as it approaches 0, uptake stays closer to the original depth distribution.

Enrichment ratios

When `orgn_enrich` or `orgp_enrich` is above 0.001, it is used as a fixed enrichment ratio for organic N, or for organic and sediment-attached P, for every runoff event. Otherwise SWAT+ computes the enrichment ratio for each event from the sediment concentration in surface runoff. The enrichment ratio is the ratio of the concentration of the constituent in the transported sediment to its concentration in the soil surface layer; it is above 1 because the sediment in runoff contains a larger share of the fine particles that the nutrients are attached to.

Curve number soil water factor

`cn3_swf` sets the soil water content at which the curve number for moisture condition III applies:

$$SW_{CN3} = FC + \text{cn3_swf}\,(SAT - FC)$$

where FC and SAT are the profile water contents at field capacity and saturation. Larger values delay the onset of surface runoff after dry periods. See hydrology in the model theory.

Biological mixing

Biological mixing redistributes residue and nutrients in the soil through the activity of earthworms and other soil biota, using the same method as a tillage operation with `bio_mix` as the mixing efficiency. With `carbon` = 0 in `codes.bsn`, it is applied once at the end of every calendar year, down to 50 mm or the bottom of the profile if that is shallower. With the other `carbon` codes this column is not used; `carbon` = 2 applies a daily biological mixing with an efficiency taken from the tillage data.

Percolation from the soil profile

`perco` limits percolation out of the bottom soil layer, for example to represent an impermeable layer or a high water table. Percolation between the upper layers is not affected. The potential percolation from the bottom layer is multiplied by

$$f_{perc} = \min[1,\ \exp(-\,(1.0052\ \ln(-\ln \text{perco}) + 5.6862))]$$

The relation is strongly non-linear:

`perco`	0	0.1	0.5	0.8	0.9	0.95	0.99	0.999 and above
fraction of potential percolation	0	0.002	0.005	0.015	0.033	0.067	0.35	1

So 0 stops percolation from the profile, values up to about 0.9 allow only a few percent of it, and the restriction is lifted only as `perco` approaches 1.

Potential evapotranspiration

`pet_co` multiplies the potential evapotranspiration of the HRU, whichever method is selected in `codes.bsn` (Priestley-Taylor, Penman-Monteith, Hargreaves or values read from file). A value of 0 is treated as 1. Values below 0.01 are taken as the old Hargreaves coefficient written to `harg_pet` by earlier editors and divided by 0.0023, so 0.0023 becomes 1.0.

Canopy storage

The canopy storage available on a day is

$$CAN_{day} = \text{can_max}\,\frac{LAI}{LAI_{mx}}$$

where LAI is the current leaf area index of the plant community and LAI_{mx} its maximum. Precipitation fills the canopy before it reaches the ground.

Sediment and organic nutrients in lateral flow

`lat_sed` adds sediment to the HRU sediment yield in proportion to lateral and tile flow, and `lat_orgn` and `lat_orgp` add organic N and P in proportion to lateral flow. In 62.0.1 the sediment from `lat_sed` is added twice: once as a mass computed from the HRU area and once again as a value that does not use the area. Sediment yields from HRUs with a non-zero `lat_sed` therefore include an extra term. The conversion used for `lat_orgn` and `lat_orgp` is 100 times smaller than mg/L of lateral flow would give, so the loads are 1% of what the stated units suggest.

topography.hyd

#	Column	Units	Built-in value	Description
1	`name`	-	`default`	Name of the topography record, matched against the `topo` column of `hru-data.hru` and of `rout_unit.rtu`.
2	`slp`	m/m	0.02	Average slope steepness, usually computed from the DEM. Used for lateral flow, the USLE slope-length factor, time of concentration and sediment transport across the HRU.
3	`slp_len`	m	50	Average slope length for erosion: the distance over which sheet flow is the main surface runoff process, measured to the point where flow begins to concentrate. Used in the USLE slope-length factor. The full value is used: 62.0.1 does not apply the maximum slope length of the HRU's conservation practice.
4	`lat_len`	m	50	Slope length for lateral subsurface flow, usually longer than `slp_len`. A value of 0 is replaced by 50 m.
5	`dist_cha`	m	100	Average distance to the stream. Not used.

#	Column	Units	Built-in value	Description
6	`depos`	-	1	Deposition coefficient for sediment transport capacity when surface runoff from another object is routed across the HRU. A larger value gives a higher transport capacity, so more sediment leaves the HRU.

The built-in values apply only to a record that is never read; an HRU always uses a record from the file.

When runoff from upslope objects is routed across an HRU, its sediment transport capacity is proportional to

$$\text{depos}\ C\ slp^{1.4}\ w^{1.4}$$

and to the overland flow, where C is the USLE cover factor of the HRU and w the field width `wd` from `field.fld`.

field.fld

#	Column	Units	Built-in value	Description
1	`name`	-	`default`	Name of the field record, matched against the `field` column of `hru-data.hru` and of `rout_unit.rtu`.
2	`len`	m	500	Field length. Used for the time of concentration of routing units; not used for HRUs.
3	`wd`	m	100	Field width. Used in the sediment transport capacity when surface runoff is routed across an HRU.
4	`ang`	degrees	30	Field angle. Not used.

An HRU whose `field` entry is `null`, or a name that is not found, uses the built-in values.

shade_factor.shf

#	Column	Units	Description
1	`jday`	day of year	Day the value applies to (1 to 365).
2	`cha`	-	Channel number (row in `chandeg.con`). In revisions before 62.0.1 this column was a landscape unit number.
3	`value`	fraction	Shade factor, the fraction of incoming solar radiation blocked before it reaches the water surface.

The file has no name column and is read as a list of (day, channel, value) rows. The channel water temperature routine reduces short-wave radiation at the water surface by the factor (1 − value). On day 366 of leap years the factor is 0.5.

The file is meant to be used when `sf_on` = 1 in `temperature.cha`. In 62.0.1 the temperature routine takes all its settings from the built-in record instead of from the rows of `temperature.cha`. The built-in record has `sf_on` = 0 and `ssff` = 0.5, so the shade factor is 0.5 for every channel and day, and `shade_factor.shf` is read but not used. No example dataset in `refdata/` has a `shade_factor.shf` file.

Calibration

These parameters can be changed during calibration through `calibration.cal` (see parameter change files), using these names: `lat_ttime`, `lat_sed`, `canmx` (`can_max`), `esco`, `epco`, `erorgn` (`orgn_enrich`), `erorgp` (`orgp_enrich`), `cn3_swf`, `biomix` (`bio_mix`), `perco`, `lat_orgn`, `lat_orgp`, `petco` (`pet_co`), `latq_co`, `slope` (`slp`), `slope_len` (`slp_len`), `lat_len`, `dis_stream` (`dist_cha`), `field_len` (`len`), `field_wid` (`wd`) and `field_ang` (`ang`).

A change to `lat_ttime` is applied to the daily release fraction f_{lat}, which SWAT+ has already computed from the input, and the result is then converted again as if it were a travel time in days. The effect of a `lat_ttime` change is therefore not the effect of changing the travel time in the file by the same amount. Changes to `dis_stream` and `field_ang` have no effect, because the values are not used.

Examples

`refdata/Ames_sub1/hydrology.hyd` in the SWAT+ repository (first two rows):

```
hydrology.hyd:
name                lat_ttime       lat_sed       can_max          esco          epco   orgn_enrich   org
hyd0001               0.00000       0.00000       0.00000       0.95000       1.00000       0.00000
hyd0002               0.00000       0.00000       0.00000       0.95000       1.00000       0.00000
```

Lateral flow travel time and enrichment ratios are computed by SWAT+, there is no canopy storage, and `cn3_swf` = 0 puts the moisture condition III curve number at field capacity. `perco` = 0.5 lets only about 0.5% of the potential percolation leave the bottom of the profile (unless the HRU is tile drained, which resets `perco` to 0.1), and `latq_co` = 0.01 keeps lateral flow small.

`refdata/Osu_1hru/hydrology.hyd` uses the older column header `harg_pet` with the value 0, which SWAT+ treats as a `pet_co` of 1.

`refdata/Ames_sub1/topography.hyd` (first two rows):

```
topography.hyd:
name                        slp       slp_len       lat_len      dist_cha         depos
topohru0001             0.02500      91.46300      91.46300     100.00000       1.00000
topohru0002             0.02500      91.46300      91.46300     100.00000       1.00000
```

`refdata/Osu_1hru/field.fld` (first two rows):

```
field.fld: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                        len            wd           ang
fld011                500.00000     100.00000      30.00000
fld012                500.00000     100.00000      30.00000
```

Related files

- `hru-data.hru`: selects the records by name in its `hydro`, `topo` and `field` columns.
- `file.cio`: names the three HRU files on its `hydrology` line.
- `landuse.lum`: sets tile drainage, which overrides `cn3_swf` and `perco`.
- `codes.bsn`: PET method and `carbon` code that `pet_co` and `bio_mix` depend on.
- `temperature.cha`: channel temperature settings, including `sf_on` for `shade_factor.shf`.
- Hydrology and soil water in the model theory.

Sources

- Readers: `hydrol_read.f90`, `topo_read.f90`, `field_read.f90`, `shade_factor_read.f90` (called from `proc_read.f90`)
- Type definitions: `type hydrology_db` in `hydrology_data_module.f90`; `type topography_db` and `type fields_db` in `topography_data_module.f90`; `type shade_factor_data` in `hydrograph_module.f90`
- File names: `type input_hydrology` and `type shade_factor` in `input_file_module.f90`, `in_hyd` read in `readcio_read.f90`

- Copy to each HRU, tile override, `pet_co` and `perco` conversion: topohyd_init.f90; routing units in ru_read.f90
- Lateral flow: hydro_init.f90, swr_percmicro.f90, swr_substor.f90
- Evaporation, PET and uptake: et_act.f90, et_pot.f90, plant_init.f90
- Canopy: sq_canopyint.f90; curve number: curno.f90
- Enrichment ratios: nut_orgn.f90, nut_orgnc2.f90, nut_psed.f90
- Lateral sediment and organic nutrients: swr_latsed.f90
- Biological mixing: time_control.f90, mgt_newtillmix_cswat0.f90
- Landscape routing and time of concentration: rls_routesurf.f90, time_conc_init.f90
- Shade factor use: ch_temp.f90, ch_read_temp.f90, `type channel_temperature_data` in channel_data_module.f90
- Calibration: cal_parm_select.f90

File	Column	Fortran variable
hydrology.hyd	`lat_ttime`	`hyd_db%lat_ttime`
hydrology.hyd	`lat_sed`	`hyd_db%lat_sed`
hydrology.hyd	`can_max`	`hyd_db%canmx`
hydrology.hyd	`esco`	`hyd_db%esco`
hydrology.hyd	`epco`	`hyd_db%epco`
hydrology.hyd	`orgn_enrich`	`hyd_db%erorgn`
hydrology.hyd	`orgp_enrich`	`hyd_db%erorgp`
hydrology.hyd	`cn3_swf`	`hyd_db%cn3_swf`
hydrology.hyd	`bio_mix`	`hyd_db%biomix`
hydrology.hyd	`perco`	`hyd_db%perco` (converted to `hru%hyd%perco_lim`)
hydrology.hyd	`lat_orgn`	`hyd_db%lat_orgn`
hydrology.hyd	`lat_orgp`	`hyd_db%lat_orgp`
hydrology.hyd	`pet_co`	`hyd_db%pet_co`
hydrology.hyd	`latq_co`	`hyd_db%latq_co`
topography.hyd	`slp`	`topo_db%slope`
topography.hyd	`slp_len`	`topo_db%slope_len`
topography.hyd	`lat_len`	`topo_db%lat_len`
topography.hyd	`dist_cha`	`topo_db%dis_stream`
topography.hyd	`depos`	`topo_db%dep_co`
field.fld	`len`	`field_db%length`
field.fld	`wd`	`field_db%wid`
field.fld	`ang`	`field_db%ang`
shade_factor.shf	`jday`, `cha`, `value`	`shf_db%jday`, `shf_db%cha`, `shf_db%value`

Purpose

A recall object is a spatial object whose outflow is read from a file instead of simulated. It is used for point sources such as wastewater treatment plants, for inflow from outside the modelled area (an upstream gauge or another model), and for diversions. Each day the recall object sends the flow and loads for that day to the objects named in its row of recall.con, in the same way an HRU or channel sends its outflow downstream.

A recall object needs three pieces of input:

1. A row in recall.con, which gives its location and the receiving object (usually a channel, `sdc`).
2. A record in `recall.rec` with the same `id`, which gives the time step and the name of the data file.
3. The data file with the flows and loads.

The number of recall objects is the `rec` column of object.cnt. `recall.rec` is named on the `recall` line of file.cio (default `recall.rec`; `null` turns recall input off).

recall.rec

Free format, whitespace-separated. Line 1 is a title and line 2 is the column header; both are skipped. Each later line is one recall record with four values. The header names are not checked; the example in `refdata/Osu_1hru`

uses `NUM NAME TYPE FNAME`. Anything after the fourth value on a line is ignored.

#	Column	Units	Description
1	`id`	-	Record number. A recall object uses the record whose `id` equals its `id` in `recall.con`; the `rec` column of `recall.con` is not used. The largest `id` sets the size of the table.
2	`name`	-	Name of the record, up to 25 characters.
3	`typ`	-	Time step of the data: 0 = sub-daily, 1 = daily, 2 = monthly, 3 = yearly, 4 = constant taken from an export coefficient record.
4	`filename`	-	Name of the data file (up to 25 characters), or for `typ` 4 the name of a record in `exco.exc`.

In revision 8a1edd4, recall objects were read from `recall_db.rec`, with six data file names (organic and mineral, pesticide, pathogen, heavy metal, salt and constituent) each followed by its units and a time step word (`sub`, `day`, `mo`, `yr`), and `recall.rec` was not read. In 62.0.1 recall objects are read from `recall.rec` with one data file and an integer time step per record. `recall_db.rec` is still read, with plain file names, but only for the uses listed under recall_db.rec.

Recall data file

One data file per recall record, named in `recall.rec`.

- Line 1: title, skipped.
- Line 2: number of years in the file. Read but not used; SWAT+ works out the years from the rows.
- Line 3: column header, skipped.
- Each later line: one time step.

Each data line has six leading values followed by 18 values in this order:

#	Column	Units	Description
1	`jday`	-	Day of year (1 to 366). Selects the day for daily data.
2	`mo`	-	Month (1 to 12). Selects the month for monthly data.
3	`day_mo`	-	Day of month. Not used.
4	`yr`	-	Year.
5	`ob_typ`	-	Object type label. Not used.
6	`ob_name`	-	Object name label. Not used.
7	`flo`	m^3	Water volume for one day.
8	`sed`	t	Sediment.
9	`orgn`	kg N	Organic N.
10	`sedp`	kg P	Organic P.
11	`no3`	kg N	Nitrate N.
12	`solp`	kg P	Soluble (mineral) P.
13	`chla`	kg	Chlorophyll-a.
14	`nh3`	kg N	Ammonium N.
15	`no2`	kg N	Nitrite N.
16	`cbod`	kg	Carbonaceous biological oxygen demand.
17	`dox`	kg	Dissolved oxygen.
18	`san`	t	Detached sand.
19	`sil`	t	Detached silt.
20	`cla`	t	Detached clay.
21	`sag`	t	Detached small aggregates.
22	`lag`	t	Detached large aggregates.
23	`grv`	t	Gravel.
24	`temp`	°C	Water temperature.

The values are used as given, as the recall object's outflow for one day. For daily data this is the day's volume and loads. Monthly and yearly values are also applied on every day of their month or year, so give them as daily

amounts: the mean daily volume and load for the month or year. A monthly or yearly total would be applied once for every day of the period. Pesticides, pathogens, heavy metals, salts and other constituents cannot be given in this file; their recall loads are zero.

Time steps

`typ`	Rows	Value used on a simulation day
1, daily	One row per day	The row with that day of year (`jday`) in that year.
2, monthly	One row per month	The row with that month (`mo`) in that year.
3, yearly	One row per year	The row for that year.
0, sub-daily		Does not work, see below.
4, constant	No data file	Does not work, see below.

Years

SWAT+ reads the first data row to find the first year in the file.

- If the file starts in or before the first simulation year, the rows before that year are skipped. The file must then contain the first simulation year: if it ends before that year, the reader keeps searching and the run does not get past reading the inputs.
- If the file starts after the first simulation year, its first year is placed at the matching simulation year and earlier years get no recall input.

From there each change of year in the file moves to the next simulation year, and reading stops at the last simulation year. A year missing from the middle of the file is not skipped over: the following years shift one year earlier. Give every year from the start of the data to the end without gaps.

For daily and monthly data, the recall object sends nothing on days outside the years covered by the file. For yearly data the test of the year range always passes, but years without data have zero values, so the effect is the same.

Diversions

A negative `flo` in daily data removes water. When the recall object's first connection in `recall.con` is a channel with more than 10 m³ of water, SWAT+ also removes nitrate, soluble P, ammonium, nitrite, dissolved oxygen and organic N from that channel at the channel's concentration, limited to what is in the channel. Salt and constituent mass are handled in the same way when those modules are active.

Sub-daily and constant records

Sub-daily records (`typ` 0) do not work in 62.0.1. The reader has no step that reads a sub-daily data line, so it keeps returning to the same line and never reaches the end of the file; a run with a sub-daily recall record does not get past reading the inputs.

Constant records (`typ` 4) are meant to take a fixed daily hydrograph from the `exco_om.exc` row of the `exco.exc` record named in `filename`. In 62.0.1 `recall.rec` is read before the export coefficient files, so the name is looked up in an empty list and the constant hydrograph stays zero. The daily recall step also has no branch for `typ` 4. A recall object with a `typ` 4 record therefore sends nothing. Use an export coefficient object in `exco.con` for a constant source instead.

recall_db.rec

`recall_db.rec` is a second recall list, read when a file with this name is in the project folder. It is not named in `file.cio`. It is used by the water allocation files and by reservoir release decision tables, not by recall objects in the routing.

Line 1 is a title and line 2 the header; both are skipped. Each later line has nine values:

#	Column	Description
1	id	Record number.
2	name	Record name, up to 13 characters.
3	org_min	Organic and mineral data file name.
4	pest	Pesticide data file name.
5	path	Pathogen data file name.
6	hmet	Heavy metal data file name.
7	salt	Salt data file name.
8	constit	Other constituent data file name.
9	descrip	Description, one word or quoted.

All nine values are required; a missing value makes the reader continue on the next line. The file names are stored as text only. None of these files is opened through `recall_db.rec`.

The two uses in 62.0.1:

- A reservoir release decision table action of type `meas` names a `recall_db.rec` record. The reservoir then releases the daily, monthly or yearly flow of the `recall.rec` record with the same number, without the year-range test used for recall objects. Keep the numbering of the two files the same.
- Water use, water treatment and outside-source entries in the water allocation files can take their water quality from a recall record by matching their name against `org_min`. The link from a `recall_db.rec` record to the recall data is never set in 62.0.1, so this option does not deliver the intended data.

pest.com

If a file named `pest.com` is in the project folder, SWAT+ reads it as a list of pesticide recall files. The values it reads are stored in the flow and nutrient records of the recall objects with the same numbers, replacing the data read through `recall.rec`, and no pesticide load is set. Do not put a file with this name in the project folder.

Salt and constituent recall files

`salt_recall.rec` and `cs_recall.rec` give salt-ion and constituent loads for recall objects when the salt or constituent modules are active. Their readers are in the source, but 62.0.1 does not call them, so these files are not read. The daily recall step still looks up salt and constituent recall data whenever salts or constituents are simulated. A model that simulates salts or constituents and has recall objects is therefore likely to stop with a memory error. See salts and constituents.

For reference, the format the readers expect: a title, a header, then one row per recall object with `id`, `name`, `typ` (1 = daily, 2 = monthly, 3 = yearly, 4 = export coefficient) and `filename`. Each data file has a title (the word `Incoming` marks a source from outside the watershed), the number of years, a header, and rows of `jday`, `mo`, `day_mo`, `yr`, `ob_typ`, `ob_name` followed by one value per salt ion or constituent in the order of `constituents.cs`.

Region files

`rec_catunit.def`, `rec_reg.def` and `rec_catunit.ele` group recall objects into regions. They are named on the `regions` line of `file.cio` and read when the calibration inputs are set up.

- `rec_catunit.def` and `rec_reg.def`: a title, the number of regions, a header, then per region `id`, `name`, `area` (ha), the number of elements, and the element numbers. With 0 elements a region contains every recall object.
- `rec_catunit.ele`: a title, a header, then per element `id`, `name`, `obj_typ`, `obj_typ_no`, `bsn_frac`, `sub_frac` and `reg_frac`.

In 62.0.1 these regions are not used for any output or calibration step, and the tables they fill are not set up before reading, so a definition file with one or more regions is likely to stop the run. Set these entries to `null` in `file.cio`.

Output

Recall objects are printed to `recall_day.txt`, `recall_mon.txt`, `recall_yr.txt` and `recall_aa.txt` (and the matching `.csv` files), selected by the `recall` row of `print.prt`. The sum over all recall objects is printed to the `basin_psc` files. The columns are described in recall outputs.

Columns in earlier documentation

Earlier documentation listed `recall.rec` with the columns `id`, `name`, `om_name`, `om_unit`, `om_tstep`, `pest_name`, `pest_unit`, `pest_tstep`, `path_name`, `path_unit`, `path_tstep`, `hmet_name`, `hmet_unit`, `hmet_tstep`, `salt_name`, `salt_unit`, `salt_tstep`, `cst_name`, `cst_unit` and `cst_tstep`. That is the layout recall objects used before 62.0.1. SWAT+ 62.0.1 no longer reads these columns: `recall.rec` has the four columns described above, and `recall_db.rec` has plain file names without units or time steps.

Example

`refdata/Osu_1hru/recall.rec` in the SWAT+ repository:

```
recall.rec
NUM      NAME     TYPE      FNAME     OBJ_TYP      OBJ_NAME        flo      sed     orgn     sedp      no3     solp
   1     PSTYR     3       RECYR.         hru          hru1    12120.5        0        0        0        0        0
   1       1           1     2011         hru          hru2     9000.7        0        0        0        0        0
```

SWAT+ reads the four values `1 PSTYR 3 RECYR.`: record 1, named `PSTYR`, yearly data in a file named `RECYR.`. The rest of the line, and the second data line, are ignored, because the largest `id` is 1 and only one record is read. This dataset has `rec` = 0 in `object.cnt`, so the file is not used in the run, and `RECYR.` is not included.

A daily data file for one recall record looks like this (illustrative values, columns 8 to 24 shortened):

```
daily point source, wastewater plant 1
2
jday  mo  day_mo    yr  ob_typ  ob_name      flo     sed    orgn    sedp     no3    solp  ...  temp
   1   1       1  2010  sdc     cha05     4320.0   0.050    1.20    0.30    8.50    1.10  ...  12.0
   2   1       2  2010  sdc     cha05     4410.0   0.050    1.25    0.31    8.60    1.12  ...  12.0
```

Each row gives the volume (m^3) and loads (t, kg) discharged on that day, here about 0.05 m^3/s.

Related files

- `recall.con`: location of each recall object and the objects that receive its outflow.
- `object.cnt`: number of recall objects (`rec`).
- `file.cio`: names `recall.rec` on the `recall` line and the region files on the `regions` line.
- `print.prt`: recall output switches.
- Recall outputs: columns of the recall output files.
- Decision tables: reservoir release action `meas`.
- Water allocation: water quality from recall records.
- Recall, export coefficient and delivery ratio objects in the model theory.

Sources

- Reader for `recall.rec`, the data files and `pest.com`: `recall_read.f90`; reader for `recall_db.rec`: `recalldb_read.f90`. Both are called from `main.f90.in`, after `hyd_connect.f90` and before `exco_db_read.f90`
- Type definitions: `type recall_hydrograph_inputs` and `type hyd_output` in `hydrograph_module.f90`; `type recall_databases` in `recall_module.f90`
- Daily use: `case ("recall")` in `command.f90`; diversions in `recall_nut.f90`, `recall_salt.f90`, `recall_cs.f90`
- Salt and constituent readers (not called): `recall_read_salt.f90`, `recall_read_cs.f90`

- Region files: rec_read_elements.f90, called from proc_cal.f90; arrays in calibration_data_module.f90
- Uses of recall_db.rec: dtbl_res_read.f90, res_hydro.f90, water_use_read.f90, water_treatment_read.f90, water_osrc_read.f90, wallo_use.f90, wallo_treatment.f90, wallo_start_day.f90
- Export coefficient lookup: exco_read_om.f90
- File names: in_rec%recall_rec and in_regs%def_psc, def_psc_reg, ele_psc in input_file_module.f90; output files opened in header_write.f90

File	Column	Fortran variable
recall.rec	id	array index i of recall
recall.rec	name	recall(i)%name
recall.rec	typ	recall(i)%typ
recall.rec	filename	recall(i)%filename
data file	flo to temp	recall(i)%hd(istep, iyrs) (type hyd_output)
recall_db.rec	name, org_min, pest, path, hmet, salt, constit, descrip	recall_db(i)%name, %org_min, %pest, %path, %hmet, %salt, %constit, %descrip

Purpose

The climate files tell SWAT+ where each spatial object gets its weather. Every object names a weather station in the wst column of its connect file (for example hru.con). The weather station, a row of weather-sta.cli, names a weather generator station and, for each weather variable, either an observed data file or sim to generate the variable.

The files below are named on the climate line of file.cio, in this order:

Position	Default name	Contents
1	weather-sta.cli	Weather stations.
2	weather-wgn.cli	Weather generator statistics.
3	pet.cli	List of observed potential evapotranspiration files.
4	pcp.cli	List of precipitation files (*.pcp).
5	tmp.cli	List of temperature files (*.tmp).
6	slr.cli	List of solar radiation files.
7	hmd.cli	List of relative humidity files.
8	wnd.cli	List of wind speed files.
9	atmodep.cli	Atmospheric deposition of nitrogen.

Two more deposition files, salt_atmo.cli and cs_atmo.cli, have fixed names and are not listed in file.cio.

A position set to null, or naming a file that does not exist, is skipped. A model needs weather-sta.cli and, for any generated variable or missing value, weather-wgn.cli. SWAT+ 62.0.1 reads climate input only from these text files; it has no netCDF climate input.

Common rules

All climate files are free format, with values separated by spaces or tabs. The first line is a title and is skipped; most files also skip a column header line, as described for each file.

Names (stations, files) can be up to 50 characters and are matched with upper and lower case distinguished. A name that contains a slash, a comma or a space must be put in quotes.

SWAT+ looks names up with a binary search, so every list that is searched must be in ascending order by character code (digits before upper-case letters, upper case before lower case). This applies to the station names in weather-sta.cli and weather-wgn.cli, the file names in each observed data list, and the station names in atmodep.cli. A list out of order can make SWAT+ miss names that are present.

Path prefixes

The last lines of file.cio give an optional folder for each type of observed data file: pcp_path, tmp_path, slr_path, hmd_path and wnd_path. When the value is null, the data files are opened from the project folder. Otherwise the value is put in front of each file name with nothing added in between, so it must end with a path separator (for example C:\weather\pcp\), and it can be up to 80 characters. A path containing a forward slash must be put in quotes, because an unquoted slash ends a free-format value. There is no path line for PET files; they are always opened from the project folder.

weather-sta.cli

One line per weather station. Line 1 is a title and line 2 is the column header; both are skipped. Each later line has nine values in the order below.

#	Column	Description
1	name	Weather station name, referenced by the wst column of the connect files.
2	wgn	Name of a weather generator station in weather-wgn.cli.
3	pcp	Name of a precipitation file listed in pcp.cli, or sim to generate precipitation.
4	tmp	Name of a temperature file listed in tmp.cli, or sim to generate maximum and minimum temperature.
5	slr	Name of a solar radiation file listed in slr.cli, or sim.
6	hmd	Name of a relative humidity file listed in hmd.cli, or sim.
7	wnd	Name of a wind speed file listed in wnd.cli, or sim.
8	wnd_dir	Read as the name of a PET file listed in pet.cli, or null for none. SWAT+ Editor labels this column wnd_dir; SWAT+ 62.0.1 has no wind direction input.
9	atmo_dep	Name of a deposition station in atmodep.cli, or null.

A tenth value on a line is ignored, so a PET file name placed after atmo_dep is not read.

Unmatched names

Column	Name not found	Message in diagnostics.out
pcp, tmp, slr, hmd, wnd	The variable is generated for the whole run.	<name> file not found (pgage), (tgage), (sgage), (hgage) or (wgage), unless the value is sim
wnd_dir (PET)	No observed PET for the station.	<name> file not found (petgage), unless the value is null; sim gives the message
atmo_dep	See deposition stations.	<name> file not found (atmodep), unless the value is null
wgn	The station uses the first station in weather-wgn.cli.	none

An object whose wst name is not in weather-sta.cli uses the first weather station, also without a message.

The six-day air temperature record used for stream temperature starts each station with the same value, the mean of the January tmp_min_ave and tmp_max_sd of the first station in weather-wgn.cli. It is replaced by the daily mean temperatures of the run within the first six days.

Example

refdata/Ames_sub1/weather-sta.cli in the SWAT+ repository:

```
weather-sta.cli:
name                          wgn              pcp              tmp              slr              hm
wgn_sta1                  wgn_001         ames.pcp         ames.tem              sim              si
```

One station, wgn_sta1, with measured precipitation from ames.pcp and generated solar radiation, humidity and wind. Two names do not match. The generator station in weather-wgn.cli is called wgn001, so wgn_001 is not found and the first (and only) generator station is used without a message. tmp.cli lists ames.tmp, so ames.tem is not found and temperature is generated for the whole run (see *.tem).

weather-wgn.cli

Monthly weather statistics for the weather generator. Each weather station uses its generator station to generate the variables set to sim, to fill missing days in the observed files, and for quantities that depend on location. Line 1 is a title. Each generator station then takes 14 lines:

1. a station line with five values (no header line above it),
2. a column header line, which is skipped,
3. twelve monthly lines, January to December, with 14 values each.

The number of stations is set by the length of the file.

Station line

#	Column	Units	Description
1	name	-	Generator station name, referenced by the wgn column of weather-sta.cli.
2	lat	decimal degrees	Latitude. Sets day length, maximum possible solar radiation and the day length at which plants go dormant for every weather station that uses this generator station, also when all variables are observed. When SWAT+ estimates plant start dates, a latitude of 0 or less is treated as the Southern Hemisphere.
3	lon	decimal degrees	Longitude. Not used by SWAT+.
4	elev	m	Elevation. Used for the elevation adjustment of generated precipitation and temperature.
5	rain_yrs	years	Number of years of record used to find pcp_hhr. A value below 1 is replaced by 10. Called yrs_pcp in earlier documentation.

Monthly lines

The column headers below are those SWAT+ Editor writes; older files use the upper-case names in the second column. SWAT+ reads by position only.

#	Column	Older header	Units	Typical range	Description
1	`tmp_max_ave`	`TMPMX`	°C	-30 to 50	Mean daily maximum air temperature for the month.
2	`tmp_min_ave`	`TMPMN`	°C	-40 to 40	Mean daily minimum air temperature for the month.
3	`tmp_max_sd`	`TMPSTDMX`	°C	0.1 to 100	Standard deviation of daily maximum air temperature in the month.
4	`tmp_min_sd`	`TMPSTDMN`	°C	0.1 to 30	Standard deviation of daily minimum air temperature in the month.
5	`pcp_ave`	`PCPMM`	mm	0 to 600	Mean total precipitation for the month.
6	`pcp_sd`	`PCPSTD`	mm/day	0.1 to 50	Standard deviation of daily precipitation in the month, with dry days included.
7	`pcp_skew`	`PCPSKW`	-	0.2 to 20	Skew coefficient of daily precipitation in the month, with dry days included. Values below 0.2 are replaced by 0.2.
8	`wet_dry`	`PR_WD`	fraction	0 to 0.95	Probability of a wet day following a dry day in the month.
9	`wet_wet`	`PR_WW`	fraction	0 to 0.95	Probability of a wet day following a wet day in the month.
10	`pcp_days`	`PCPD`	days	0 to 31	Mean number of days with precipitation in the month.
11	`pcp_hhr`	`RAINHMX`	mm	0 to 125	Largest 0.5-hour rainfall in the month over the `rain_yrs` years of record.
12	`slr_ave`	`SOLARAV`	MJ/m²/day	0 to 40	Mean daily solar radiation for the month.
13	`dew_ave`	`DEWPT`	°C or fraction	-50 to 25	Mean daily dew point temperature for the month, or mean relative humidity as a fraction.
14	`wnd_ave`	`WINDAV`	m/s	0 to 100	Mean daily wind speed for the month.

The statistics count a day as wet when it has more than 0 mm of precipitation. During the run, the generator counts a day as wet, for the next day's wet-day probability, when it has at least 0.1 mm.

Checks and derived values

SWAT+ checks and completes the monthly values for each generator station when it reads the file.

Wet-day probabilities and wet days. If `wet_wet` is not greater than `wet_dry`, or `wet_dry` is 0 or less, the two probabilities are estimated from `pcp_days` (at least 0.1 day):

$$P(W/D) = 0.75\,\frac{d_{wet}}{N}, \qquad P(W/W) = 0.25 + P(W/D)$$

where d_{wet} is `pcp_days` and N the number of days in the month. Otherwise `pcp_days` is not used as given and is recalculated from the probabilities:

$$d_{wet} = \frac{N\,P(W/D)}{1 - P(W/W) + P(W/D)}$$

Mean precipitation per wet day. The generator draws daily amounts around $\mu = \text{pcp_ave}/d_{wet}$. `pcp_ave` must therefore be the mean monthly total. A mean daily value in this column makes generated precipitation far too small.

Dew point or humidity. If all twelve `dew_ave` values are between 0 and 1, they are taken as relative humidity (fraction). If any value is outside that range, all twelve are taken as dew point temperature in °C.

Half-hour rainfall. `pcp_hhr` is smoothed over three months (the month and its two neighbours, December and January wrapping round) and turned into the mean fraction of daily rain that falls in the wettest half hour:

$$\alpha_{0.5} = \text{adj_pkr}\left[1 - \exp\left(\frac{\bar{R}_{0.5}}{\mu\,\ln\left(\dfrac{0.5}{\text{rain_yrs}\;d_{wet}}\right)}\right)\right]$$

limited to 0.1 to 0.95, where $\bar{R}_{0.5}$ is the smoothed `pcp_hhr` and `adj_pkr` is the peak rate adjustment factor (`adj_pkrt` in the `parameters.bsn` header). Each day's fraction is drawn at random around this monthly value and is used for peak runoff rates, erosion and the generated distribution of rain within the day.

The generator equations themselves are described in Climate and weather.

Example

Start of `refdata/Ames_sub1/weather-wgn.cli` in the SWAT+ repository:

```
weather-wgn.cli: Ames
wgn001       37.48     -100.83    887.00     10
   TMPMX   TMPMN  TMPSTDMX TMPTDMN  PCPMM  PCPSTD  PCPSKW   PR_WD   PR_WW   PCPD  RAINHMX SOLARAV  DEWPT
    7.61   -8.13     8.06    5.99     7.4     3.3    0.19    0.07    0.25    2.65      5.1   10.39  -8.14
   11.07   -5.53     8.06    5.34    10.2     5.1    2.67    0.08    0.27    2.86      1.8   13.41  -5.58
   15.22   -1.87     7.99    5.22    28.1     8.9    1.64    0.11    0.34    4.43      7.6   17.85  -4.71
...
```

One generator station, `wgn001`, at 37.48° N and 887 m, with 10 years of half-hour rainfall record; ten more monthly lines follow. The January skew of 0.19 is raised to 0.2. The `DEWPT` values go below 0, so the column is read as dew point temperature.

Observed data lists

pcp.cli, tmp.cli, slr.cli, hmd.cli, wnd.cli and pet.cli all have the same layout: a title line, a header line (SWAT+ Editor writes filename), and one data file name per line. Each name is both the file to open and the name that the matching column of weather-sta.cli refers to. Every file listed is read, whether or not a station uses it.

List	weather-sta.cli column	Records in each data file	Units
pcp.cli	pcp	year, day, precipitation (or six values for sub-daily data)	mm (*.pcp)
tmp.cli	tmp	year, day, maximum, minimum temperature	°C (*.tmp)
slr.cli	slr	year, day, solar radiation	MJ/m²/day
hmd.cli	hmd	year, day, relative humidity	fraction, 0 to 1
wnd.cli	wnd	year, day, wind speed	m/s, at 10 m height
pet.cli	wnd_dir	year, day, potential evapotranspiration	mm

Example, refdata/Osu_1hru/pcp.cli:

```
pcp.cli: Precipitation file names - file written by SWAT+ editor 2023-03-22 04:22:29.020522
filename
Imsilpcp.pcp
```

Solar radiation, humidity, wind and PET files

These data files have the layout of a daily precipitation file: a title line, a header line (nbyr tstep lat lon elev), a gauge line with those five values, and one record per day with year, day of year (called jday in earlier documentation) and the value. Only nbyr is used from the gauge line. The records follow the same rules for order, storage, values of -97 or less, and days outside the record as daily precipitation. A missing solar radiation, humidity or wind value is generated from the weather generator station.

Relative humidity is limited to 0.01 to 0.99 every day after it is read or generated, so a file in percent gives 0.99 on every day. Solar radiation below 0 is set to 0.

Observed PET

A PET file is read when it is listed in pet.cli, and a station uses it when its wnd_dir column names it. With pet = 3 in codes.bsn, HRUs take their potential evapotranspiration from the station. In 62.0.1 the station value is replaced by a Hargreaves estimate (at least 0.01 mm) later in the same daily step, for every station, so the values in the PET files do not reach the HRUs and pet = 3 gives Hargreaves PET. A missing PET value is also replaced by a Hargreaves estimate. Channel and reservoir evaporation and aquifer revap use the same station value under every pet setting.

atmodep.cli

Atmospheric deposition of ammonium and nitrate, as average annual, monthly or yearly values for one or more stations. Earlier documentation calls this file atmo.cli. The deposition values are in this file; it does not point to data files. The atmo_dep column of codes.bsn is not used; the time step comes from this file.

Line 1 is a title and line 2 a column header; both are skipped. Line 3 is the control line:

#	Column	Description
1	num_sta	Number of deposition stations in the file.
2	timestep	aa average annual, mo monthly, yr yearly.
3	mo_init	First month of the series (1 to 12). Used with mo.
4	yr_init	First year of the series. Used with mo and yr.
5	num_aa	Number of months (mo) or years (yr) in each series. Not used with aa.

#	Column	Description

Each station then takes five lines:

Line	Contents	Units
1	Station name, referenced by the `atmo_dep` column of `weather-sta.cli`.	-
2	Ammonium concentration in rain.	mg N/L
3	Nitrate concentration in rain.	mg N/L
4	Ammonium dry deposition.	kg N/ha per year (`aa`, `yr`) or per month (`mo`)
5	Nitrate dry deposition.	kg N/ha per year (`aa`, `yr`) or per month (`mo`)

With `aa` each of lines 2 to 5 has one value. With `mo` or `yr` each has `num_aa` values, one per month or year from `mo_init`/`yr_init` on.

Daily deposition

Every day, each HRU receives in its top soil layer, for ammonium and nitrate separately,

$$N_{wet} = 0.01\, C\, P, \qquad N_{dry} = \frac{D}{365}\ \ (\texttt{aa}, \texttt{yr}), \qquad N_{dry} = \frac{D}{N_{mo}}\ \ (\texttt{mo})$$

where N_{wet} and N_{dry} are in kg N/ha, C is the concentration in rain (mg/L), P the HRU precipitation for the day (mm), D the dry deposition value and N_{mo} the number of days in the month.

Monthly and yearly series

SWAT+ starts the series at the entry for the first month or year of the simulation. If the series starts later than the simulation, deposition begins when its first month or year is reached. Before the series starts and after its last entry, HRUs receive no deposition. The entry used moves on at the last day of each month (`mo`) or year (`yr`), so the last day of a month already uses the next month's values.

When the series starts before the simulation and `mo_init` is not 1, the first entry used is `mo_init` - 1 months later than the simulation start month, and the whole series is shifted by that amount.

Deposition stations

A weather station with `atmo_dep` set to `null`, or to a name not in the file, uses a built-in station. With `aa` this station gives 1.0 mg/L ammonium and 0.2 mg/L nitrate in rain and no dry deposition. With `mo` or `yr` the built-in station has no series, and its values are undefined; give every weather station a deposition station in that case.

Example

Average annual deposition for one station:

```
atmodep.cli
num_sta  timestep  mo_init  yr_init  num_aa
      1        aa        0        0       0
station_001
1.0
0.2
0.0
0.0
```

Salt and constituent deposition

Wet and dry deposition of salt ions and of other constituents, used with the salt and constituent inputs. Both files have fixed names and are read from the project folder only when `atmodep.cli` is present and the corresponding ions or constituents are defined in `constituents.cs`. They have no control line: the number of stations, the time step and the series length come from `atmodep.cli`. Stations are matched by position, so they must be in the same order as in `atmodep.cli`.

salt_atmo.cli

The first six lines are skipped. For each station:

- a line with the station name,
- one line per salt ion, in the order of `constituents.cs`, with the ion label followed by the concentration in rain (mg/L), one value for `aa` or `num_aa` values for `mo` and `yr`,
- one line per salt ion with the ion label followed by the dry deposition, in the same units as nitrogen dry deposition.

cs_atmo.cli

The first three lines are skipped. For each station:

- a station line, which is read as a number, so it must contain a number such as the station's position (a text name stops the run with a read error),
- one line per constituent with the concentration in rain (mg/L), without a label,
- one line per constituent with the dry deposition, without a label.

For both files, the daily amounts are calculated as for nitrogen and added to the top soil layer.

Related files

- `file.cio`: names the climate files and the data path prefixes.
- `hru.con` and the other connect files: assign a weather station to each object in the `wst` column.
- `*.pcp`, `*.tmp`, `*.tem`: observed precipitation and temperature.
- `time.sim`: simulation period and number of steps per day.
- `codes.bsn`: PET method and `lapse` switch.
- `parameters.bsn`: lapse rates and `adj_pkrt`.
- Salt inputs and constituent inputs: ion and constituent lists for the deposition files.
- Climate and weather: weather generator and PET methods.

Sources

- File name slots: `type input_cli` and the `in_path_*` types in `input_file_module.f90`, read in `readcio_read.f90`
- Type definitions: `weather_station`, `weather_codes_station_char`, `weather_generator_db`, `climate_measured_data`, `atmospheric_deposition`, `atmospheric_deposition_control` in `climate_module.f90`
- `weather-sta.cli`: `cli_staread.f90`; name lookup `search.f90`; station lookup from connect files `hyd_read_connect.f90`
- `weather-wgn.cli`: `cli_wgnread.f90`, `cli_initwgn.f90`, `cli_clgen.f90`, `cli_tgen.f90`
- Observed data lists: `cli_pmeas.f90`, `cli_tmeas.f90`, `cli_smeas.f90`, `cli_hmeas.f90`, `cli_wmeas.f90`, `cli_petmeas.f90`
- Daily climate step, missing values, PET recalculation and humidity limits: `climate_control.f90`; HRU PET `et_pot.f90`
- `atmodep.cli`: `cli_read_atmodep.f90`, `cli_atmodep_time_control.f90`, `nut_nrain.f90`
- `salt_atmo.cli` and `cs_atmo.cli`: `cli_read_atmodep_salt.f90`, `cli_read_atmodep_cs.f90`, `salt_rain.f90`, `cs_rain.f90`
- `atmo_dep` code in `codes.bsn`: `bsn_cc%atmo` in `basin_module.f90`

File	Column	Fortran variable
weather-sta.cli	name	wst(i)%name
weather-sta.cli	wgn	wst(i)%wco_c%wgn
weather-sta.cli	pcp	wst(i)%wco_c%pgage
weather-sta.cli	tmp	wst(i)%wco_c%tgage
weather-sta.cli	slr	wst(i)%wco_c%sgage
weather-sta.cli	hmd	wst(i)%wco_c%hgage
weather-sta.cli	wnd	wst(i)%wco_c%wgage
weather-sta.cli	wnd_dir	wst(i)%wco_c%petgage
weather-sta.cli	atmo_dep	wst(i)%wco_c%atmodep
weather-wgn.cli	name	wgn_n(i)
weather-wgn.cli	rain_yrs	wgn(i)%rain_yrs
weather-wgn.cli	tmp_max_ave, tmp_min_ave	wgn(i)%tmpmx, wgn(i)%tmpmn
weather-wgn.cli	tmp_max_sd, tmp_min_sd	wgn(i)%tmpstdmx, wgn(i)%tmpstdmn
weather-wgn.cli	pcp_ave, pcp_sd, pcp_skew	wgn(i)%pcpmm, wgn(i)%pcpstd, wgn(i)%pcpskw
weather-wgn.cli	wet_dry, wet_wet, pcp_days	wgn(i)%pr_wd, wgn(i)%pr_ww, wgn(i)%pcpd
weather-wgn.cli	pcp_hhr	wgn(i)%rainhmx
weather-wgn.cli	slr_ave, dew_ave, wnd_ave	wgn(i)%solarav, wgn(i)%dewpt, wgn(i)%windav
observed lists	file name	pcp_n, tmp_n, slr_n, hmd_n, wnd_n, petm_n and %filename
atmodep.cli	control line	atmodep_cont%num_sta, %timestep, %mo_init, %yr_init, %num
atmodep.cli	aa lines 2 to 5	atmodep(i)%nh4_rf, %no3_rf, %nh4_dry, %no3_dry
atmodep.cli	mo lines 2 to 5	atmodep(i)%nh4_rfmo, %no3_rfmo, %nh4_drymo, %no3_drymo
atmodep.cli	yr lines 2 to 5	atmodep(i)%nh4_rfyr, %no3_rfyr, %nh4_dryyr, %no3_dryyr

Purpose

A precipitation data file contains the measured precipitation of one gauge, as daily totals or as values for each sub-daily time step. Each file is named on one line of pcp.cli, and a weather station uses it when the pcp column of weather-sta.cli gives the same name. Days without a usable value are filled by the weather generator station named in the wgn column of the same weather station.

SWAT+ does not check the file extension. .pcp is the usual one. If pcp_path in file.cio is not null, its value is put in front of every file name in pcp.cli (see path prefixes).

Format

Free format, whitespace-separated. Line 1 is a title and line 2 is a column header; both are skipped. Line 3 describes the gauge and every later line is one record.

Gauge line

SWAT+ Editor writes the header nbyr tstep lat lon elev above this line.

#	Column	Units	Description
1	nbyr	years	Number of years of record. SWAT+ uses it to size the store for the data, so it must be at least the number of calendar years the record spans.
2	tstep	values per day	0 for daily data. For sub-daily data, the number of values per day, which must equal step in time.sim (24 for hourly data).
3	lat	decimal degrees	Latitude of the gauge. Not used by SWAT+.
4	lon	decimal degrees	Longitude of the gauge. Not used by SWAT+.
5	elev	m	Elevation of the gauge. Used only for the elevation adjustment.

Daily records

With tstep = 0, each record has three values:

#	Column	Units	Description
1	year	-	Calendar year.
2	day	-	Day of the year, 1 to 366. Called `jday` in earlier documentation.
3	pcp	mm	Precipitation for the day. A value of -97 or less marks the day as missing.

Sub-daily records

With `tstep` greater than 0, each record has six values:

#	Column	Units	Description
1	year	-	Calendar year.
2	day	-	Day of the year, 1 to 366. Called `jday` in earlier documentation.
3	month	-	Month, called `mon` in earlier documentation. Not used by SWAT+.
4	day of month	-	Not used by SWAT+.
5	step	-	Time step within the day, called `hr` in earlier documentation. Not used by SWAT+.
6	pcp	mm	Precipitation for the time step.

SWAT+ places the values of each day in the order they appear, so every day needs `step` records in time order. The month, day of month and step columns do not position the value.

Record order

Records must be continuous and in date order. SWAT+ counts years by watching for day 365 or 366 followed by a record from a new year, so a gap at the end of a year shifts every later year of the record. A file that starts or ends part way through a year is fine.

Reading and storage

Records before the start of the simulation are skipped when the file is read. Storage starts with the first record whose year is at or after `yrc_start` and whose day is at or after `day_start` in `time.sim`.

When the record starts in a year after the simulation start year and `day_start` is later than the first day in the file, the records of that first year before `day_start` are also skipped. Those days then read as 0 mm instead of being filled by the weather generator.

Missing days

In a daily run, SWAT+ fills a day from the weather generator when

- the value is -97 or less (SWAT+ Editor and most tools write -99), or
- the day is before the first record or after the last record of the file.

Generated precipitation uses the monthly statistics of the station's `weather-wgn.cli` entry and the wet or dry state of the previous day. The method is described in Climate and weather.

Sub-daily runs

A sub-daily file is read when `tstep` is greater than 0. `step` in `time.sim` sets the number of steps per day of the run, and SWAT+ stores that many values per day. If the two numbers differ, the values are assigned to the wrong steps: with `step` = 0 or 1 and an hourly file, only the last hour of each day is kept.

Missing and out-of-range values are handled differently from daily data in 62.0.1:

- When a step has a value of -97 or less, SWAT+ calls the generator, but the day's total is still the sum of the file values for that day, missing-value flags included.
- Days outside the record are not generated. Inside the stored years they read as 0 mm.
- Values are looked up by simulation year counted from the simulation start, so the record must start on or before the simulation start date.
- From the second simulation year on, the values used for 1 January come from 1 January of the previous simulation year.

Sub-daily files should therefore be complete, without missing values, and cover the whole simulation period.

Elevation adjustment

When `lapse` = 1 in `codes.bsn`, the precipitation of each HRU is changed for the difference between the HRU elevation and the gauge elevation:

$$P_{hru} = \max\left(0,\ P + \text{plaps}\,\frac{z_{hru} - z_{gauge}}{1000}\right)$$

where P is the station precipitation (mm), `plaps` is the precipitation lapse rate in `parameters.bsn` (mm per km, default 0), z_{hru} is the `elev` column of `hru.con` and z_{gauge} is `elev` from this file (m). When the station's `pcp` column is `sim`, the elevation of the weather generator station is used for z_{gauge}.

The adjustment is added every day, dry days included, so a positive `plaps` gives HRUs above the gauge a small amount of precipitation on every day. The adjustment values are calculated during channel and reservoir routing, so in a model without channels or reservoirs they remain zero.

Calibration

The calibration parameter `precip` (object type `cli`) changes the stored daily values of listed precipitation records over a window of days and years. See Climate changes.

Example

First lines of `refdata/Ames_sub1/ames.pcp` in the SWAT+ repository, and part of 2012:

```
AME.pcp
 NBYR    TSTEP      LAT     LONG     ELEV
   121        0    42.04   -93.89   316
1900 1 -99
1900 2 -99
1900 3 -99
...
2012 55 12.4
2012 56 0
2012 57 -99
2012 58 0
```

A daily record (`tstep` = 0) of 121 years, 1900 to 2020, at 316 m. The Ames run starts in 1975, so the records from 1900 to 1974 are skipped when the file is read. Of the days from 1975 on, 31 are marked missing (`-99`), such as day 57 of 2012, and the weather generator fills them during the run.

Related files

- `pcp.cli`: lists the precipitation files.
- `weather-sta.cli`: assigns a precipitation file to each weather station in its `pcp` column.
- `weather-wgn.cli`: weather generator statistics that fill missing days.

- time.sim: simulation period and number of steps per day.
- file.cio: names pcp.cli and the pcp_path prefix.
- codes.bsn and parameters.bsn: lapse switch and plaps rate.

Sources

- Reader: cli_pmeas.f90
- Type definition: type climate_measured_data in climate_module.f90
- Gauge name lookup from weather-sta.cli: cli_staread.f90, search.f90
- Daily use, missing values and range check: cli_precip_control.f90, cli_bounds_check.f90, called from time_control.f90
- Weather generator: cli_pgen.f90, cli_pgenhr.f90
- Elevation adjustment: cli_lapse.f90, called from sd_channel_control3.f90 and res_control.f90, applied in hru_control.f90
- Time step: time_read.f90
- Path prefix: in_path_pcp in input_file_module.f90, read in readcio_read.f90
- Calibration change precip: cal_conditions.f90

Column	Fortran variable
nbyr	pcp(i)%nbyr
tstep	pcp(i)%tstep
lat	pcp(i)%lat
lon	pcp(i)%long
elev	pcp(i)%elev
daily pcp	pcp(i)%ts(day, iyr)
sub-daily pcp	pcp(i)%tss(step, day, iyr)

Purpose

A temperature data file contains the measured daily maximum and minimum air temperature of one gauge. Each file is named on one line of tmp.cli, and a weather station uses it when the tmp column of weather-sta.cli gives the same name. Days without usable values are filled by the weather generator station named in the wgn column of the same weather station.

SWAT+ does not check the file extension. SWAT+ Editor writes .tmp; some older data sets use .tem. If tmp_path in file.cio is not null, its value is put in front of every file name in tmp.cli (see path prefixes).

Temperature is always read as daily data, also in sub-daily runs.

Format

Free format, whitespace-separated. Line 1 is a title and line 2 is a column header; both are skipped. Line 3 describes the gauge and every later line is one daily record.

Gauge line

SWAT+ Editor writes the header nbyr tstep lat lon elev above this line.

#	Column	Units	Description
1	nbyr	years	Number of years of record. SWAT+ uses it to size the store for the data, so it must be at least the number of calendar years the record spans.
2	tstep	-	Time step of the data. Not used by SWAT+; give 0.
3	lat	decimal degrees	Latitude of the gauge. Not used by SWAT+.
4	lon	decimal degrees	Longitude of the gauge. Not used by SWAT+.
5	elev	m	Elevation of the gauge. Used only for the elevation adjustment.

Daily records

#	Column	Units	Description
1	year	-	Calendar year.
2	day	-	Day of the year, 1 to 366. Called `jday` in earlier documentation.
3	tmax	°C	Maximum air temperature for the day, called `tmpmax` in earlier documentation. A value of -97 or less marks it as missing.
4	tmin	°C	Minimum air temperature for the day, called `tmpmin` in earlier documentation. A value of -97 or less marks it as missing.

Records must be continuous and in date order, as for precipitation: SWAT+ counts years by watching for day 365 or 366 followed by a record from a new year. Records before the simulation start are skipped when the file is read, by the rule described for precipitation files.

Missing days

SWAT+ generates both temperatures for a day when

- `tmax` or `tmin` is -97 or less (SWAT+ Editor and most tools write -99), or
- the day is before the first record or after the last record of the file.

When only one of the two values is missing, the measured value of the other is replaced by a generated one as well, so the pair always comes from one source. The generated maximum is lower on wet days than on dry days, using the day's precipitation from the same weather station. The method is described in Climate and weather.

The daily mean air temperature used by snow, plant growth and other processes is

$$T_{av} = \frac{T_{max} + T_{min}}{2}$$

Elevation adjustment

When `lapse` = 1 in `codes.bsn`, the temperatures of each HRU are changed for the difference between the gauge elevation and the HRU elevation:

$$T_{hru} = T + \text{tlaps}\,\frac{z_{gauge} - z_{hru}}{1000}$$

applied to the maximum, minimum and mean temperature. `tlaps` is the temperature lapse rate in `parameters.bsn` (°C per km, default 6.5), z_{hru} is the `elev` column of `hru.con` and z_{gauge} is `elev` from this file (m). When the station's `tmp` column is `sim`, the elevation of the weather generator station is used for z_{gauge}. As for precipitation, the adjustment values are calculated during channel and reservoir routing, so in a model without channels or reservoirs they remain zero.

Calibration

The calibration parameter `temp` (object type `cli`) is meant to change the stored values of listed temperature records, but in 62.0.1 it does not change the listed records. See Climate changes.

Example

First lines of `refdata/Osu_1hru/Imsiltmp.tmp` in the SWAT+ repository:

```
Imsiltmp.tmp: Temperature data - file written by SWAT+ editor 2023-03-22 04:22:29.141050
nbyr     tstep       lat       lon      elev
  42         0    35.610   127.290   247.900
1981    1    2.70000   -17.50000
1981    2   -3.00000    -7.50000
...
2023  364        2.3        -4.3
2023  365        0.8        -8.4
```

A daily record from 1 January 1981 to 31 December 2023 at 247.9 m. The record spans 43 calendar years while `nbyr` is 42. The Osu run starts in 2010, so only the 14 years from 2010 are stored and the smaller `nbyr` does no harm here; a run starting in 1981 would need `nbyr` of at least 43.

Related files

- `tmp.cli`: lists the temperature files.
- `weather-sta.cli`: assigns a temperature file to each weather station in its `tmp` column.
- `weather-wgn.cli`: weather generator statistics that fill missing days.
- `*.tem`: the same format under another extension.
- `file.cio`: names `tmp.cli` and the `tmp_path` prefix.
- `codes.bsn` and `parameters.bsn`: `lapse` switch and `tlaps` rate.
- `snow.sno`: snowfall and melt use the daily mean and maximum temperature.

Sources

- Reader: `cli_tmeas.f90`
- Type definition: `type climate_measured_data` in `climate_module.f90`
- Gauge name lookup from `weather-sta.cli`: `cli_staread.f90`, `search.f90`
- Daily use, missing values and range check: `climate_control.f90`, `cli_bounds_check.f90`
- Weather generator: `cli_weatgn.f90`, `cli_tgen.f90`
- Elevation adjustment: `cli_lapse.f90`, applied in `hru_control.f90`
- Path prefix: `in_path_tmp` in `input_file_module.f90`, read in `readcio_read.f90`
- Calibration change `temp`: `cal_conditions.f90`

Column	Fortran variable
`nbyr`	`tmp(i)%nbyr`
`tstep`	`tmp(i)%tstep`
`lat`	`tmp(i)%lat`
`lon`	`tmp(i)%long`
`elev`	`tmp(i)%elev`
tmax	`tmp(i)%ts(day, iyr)`
tmin	`tmp(i)%ts2(day, iyr)`

Purpose

`.tem` is an older extension for temperature data files, used by some data sets and preprocessing tools. SWAT+ opens a temperature file by the name given in `tmp.cli` and does not look at the extension, so a `.tem` file is read in the same way as a `.tmp` file. SWAT+ Editor writes `.tmp`.

The name has to be the same, extension included, in two places:

- a line of `tmp.cli`, which tells SWAT+ to read the file, and
- the `tmp` column of `weather-sta.cli`, which assigns it to a weather station.

If the `tmp` column names a file that is not listed in `tmp.cli`, SWAT+ writes `<name> file not found (tgage)` to `diagnostics.out` and generates the temperatures of that station for the whole run.

Format

The format is that of a .tmp file: a title line, a header line, a gauge line with **nbyr**, **tstep**, **lat**, **lon** and **elev**, and one record per day with year, day of year, maximum and minimum temperature in °C. Values of -97 or less mark missing data.

Example

refdata/Ames_sub1/ames.tem in the SWAT+ repository:

```
AME.Tmp
 NBYR    TSTEP       LAT      LONG      ELEV
   121        0     42.04    -93.89    316
1900    1   -99 -99
1900    2   -99 -99
...
1975    1   4.4 -7.2
1975    2   5   -7.8
...
2020    365 -1.7    -7.8
2020    366 -2.8    -8.9
```

A daily record of 121 years, 1900 to 2020, separated by tabs, which free-format reading accepts. In this data set the **tmp** column of weather-sta.cli names **ames.tem**, while tmp.cli lists **ames.tmp**, which is an empty file. SWAT+ therefore does not find **ames.tem**, never reads it, and generates the temperatures of the Ames run. Listing **ames.tem** in tmp.cli instead of **ames.tmp** makes the run use the measured record.

Related files

- *.tmp: the same format; missing values, elevation adjustment and storage are described there.
- tmp.cli: lists the temperature files to read.
- weather-sta.cli: assigns a temperature file to each weather station.

Sources

- Reader: cli_tmeas.f90
- Type definition: **type climate_measured_data** in climate_module.f90
- Name lookup and the **file not found (tgage)** message: cli_staread.f90, search.f90
- Daily use: climate_control.f90

Purpose

snow.sno is the snow parameter database. Each row is a named set of parameters for snowfall, snowpack temperature, snowmelt and snow cover. An HRU picks a row by name in the **snow** column of hru-data.hru. The file is named on the **hru_parm_db** line of file.cio (default **snow.sno**).

If hru-data.hru gives **null** for an HRU's snow entry, or a name that is not in **snow.sno**, the HRU uses the first row of **snow.sno**. No message is written in either case. A model therefore needs a **snow.sno** file with at least one row: if the file is missing or named **null** in file.cio, every HRU refers to a first row that does not exist.

Format

Free format, whitespace-separated. Line 1 is a title and line 2 is the column header; both are skipped. Each later line is one parameter set with nine values in the order below. The reader does not match columns by header name, so all nine values must be present and in this order.

#	Column	Units	Typical range	Description
1	`name`	-	-	Name of the snow parameter set, matched against the `snow` column of `hru-data.hru`.
2	`fall_tmp`	°C	-5 to 5	Snowfall temperature. On a day when the mean air temperature is at or below `fall_tmp`, all precipitation falls as snow and is added to the snowpack.
3	`melt_tmp`	°C	-5 to 5	Snowmelt base temperature. Melt occurs only on days when the maximum air temperature is above `melt_tmp`.
4	`melt_max`	mm H_2O/°C/day	0 to 10	Melt factor on 21 June (day 172).
5	`melt_min`	mm H_2O/°C/day	0 to 10	Melt factor on 21 December (day 355).
6	`tmp_lag`	-	0.01 to 1	Snowpack temperature lag factor. See Snowpack temperature for how 62.0.1 uses it.
7	`snow_h2o`	mm H_2O	0 to 500	Snow water content at and above which the HRU is fully covered by snow.
8	`cov50`	fraction	0.05 to 0.95	Fraction of `snow_h2o` at which 50% of the HRU is covered by snow.
9	`snow_init`	mm H_2O	0 and up	Snow water content at the start of the simulation.

Snow processes

All snow calculations run daily for each HRU, after canopy interception.

Snowfall

If the mean air temperature $\bar{T}$ is at or below `fall_tmp`, the day's precipitation (after canopy interception) is added to the snowpack and no rain reaches the ground. In sub-daily runs the rainfall for every time step of that day is set to zero.

Snowpack temperature

The intended snowpack temperature keeps part of the previous day's value:

$$T_{snow,d} = T_{snow,d-1}\,(1-\ell) + \bar{T}_d\,\ell$$

where ℓ is `tmp_lag`. Values near 1 give the current day's air temperature most of the weight. Values near 0 make the snowpack respond slowly.

In 62.0.1 the previous day's snowpack temperature is reset to zero at the start of each day, so the calculation reduces to

$$T_{snow,d} = \ell\,\bar{T}_d$$

and `tmp_lag` acts as a scaling factor on the day's mean air temperature. It has no lag effect.

Melt factor

The melt factor b_{mlt} varies through the year between `melt_min` and `melt_max`, which accounts for the effect of snowpack density on melt:

$$b_{mlt} = \frac{\text{melt_max} + \text{melt_min}}{2} + \frac{\text{melt_max} - \text{melt_min}}{2} \sin\left(\frac{2\pi}{365}(d - 81)\right)$$

where d is the day of the year. The factor equals `melt_max` on day 172 and `melt_min` on day 355 for every HRU. SWAT+ 62.0.1 does not swap the two for the Southern Hemisphere, so a model there should give the winter (June) factor in `melt_max` and the summer (December) factor in `melt_min`.

In rural areas the melt factor ranges from 1.4 to 6.9 mm H_2O/°C/day (Huber and Dickinson, 1988). Urban values tend to the high end because traffic compresses the snowpack: urban studies in Sweden reported 3.0 to 8.0 mm H_2O/°C/day (Bengston, 1981; Westerstrom, 1981), and snow on asphalt gave 1.7 to 6.5 mm H_2O/°C/day (Westerstrom, 1984).

Snow cover

Drifting, shading and topography mean that snow rarely covers an HRU evenly. SWAT+ describes the covered fraction with an areal depletion curve. With $r = SNO/\text{snow_h2o}$, the ratio of the current snow water content to `snow_h2o`,

$$sno_{cov} = \frac{r}{r + \exp(c_1 - c_2\, r)} \quad \text{for } r < 1, \qquad sno_{cov} = 1 \quad \text{for } r \geq 1$$

The shape coefficients c_1 and c_2 are set once at the start of the run so that the curve passes through 50% cover at $r = \text{cov50}$ and 95% cover at $r = 0.95$. `cov50` must therefore lie between 0 and 0.95.

The depletion curve affects melt only while the snow water content is between 0 and `snow_h2o`. A small `snow_h2o` makes its effect on melt minor; a large one makes it more important.

Snowmelt

When the maximum air temperature T_{max} is above `melt_tmp` and there is snow on the ground, melt is

$$SNO_{mlt} = b_{mlt}\, sno_{cov} \left(\frac{T_{snow} + T_{max}}{2} - \text{melt_tmp}\right)$$

limited to between zero and the snow water content. Melt water is added to the day's water input at the surface, together with any rain. In sub-daily runs it is spread evenly over the day's time steps.

Calibration and decision tables

Five of the parameters can be changed during calibration through `calibration.cal`, using these parameter names: `snofall_tmp` (`fall_tmp`), `snomelt_tmp` (`melt_tmp`), `snomelt_max` (`melt_max`), `snomelt_min` (`melt_min`) and `snomelt_lag` (`tmp_lag`). `snow_h2o`, `cov50` and `snow_init` have no calibration parameter. See parameter change files.

A decision table action `snow_change` switches an HRU to another `snow.sno` row during the run and logs the change to `lu_change_out.txt`. The snow cover curve is not recalculated after the switch, so it keeps the shape from the HRU's original `cov50`.

Example

`refdata/Ames_sub1/snow.sno` in the SWAT+ repository:

```
snow.sno: written by SWAT+ editor v2.3.3 on 2023-10-25 08:58 for SWAT+ rev.60.5.7
name                  fall_tmp      melt_tmp      melt_max      melt_min       tmp_lag      snow_h2o
snow001                1.00000       0.50000       4.50000       4.50000       1.00000       1.00000
```

One parameter set named `snow001`. `melt_max` and `melt_min` are both 4.5, so the melt factor is constant through the year. With `tmp_lag` at 1.0 the snowpack temperature equals the day's mean air temperature, and with `snow_h2o` at 1 mm the HRU is fully covered by any snowpack of 1 mm or more.

Related files

- `hru-data.hru`: selects a `snow.sno` row by name in its `snow` column; `null` or an unknown name selects the first row.
- `file.cio`: names the snow file.
- `tmp.cli` and `pcp.cli`: supply the air temperatures and precipitation the snow routine uses.

Sources

- Reader: `snowdb_read.f90`
- Type definition: `type snow_parameters` in `hru_module.f90`
- Name lookup from `hru-data.hru`: `hru_read.f90`
- HRU setup and depletion curve: `topohyd_init.f90`, `ascrv.f90`
- Snowfall and melt: `sq_snom.f90`, called from `hru_control.f90`
- Decision table action `snow_change`: `actions.f90`
- Calibration parameters: `cal_parm_select.f90`
- File name slot: `in_parmdb%snow` in `input_file_module.f90`

Column	Fortran variable
`name`	`snodb%name`
`fall_tmp`	`snodb%falltmp`
`melt_tmp`	`snodb%melttmp`
`melt_max`	`snodb%meltmx`
`melt_min`	`snodb%meltmn`
`tmp_lag`	`snodb%timp`
`snow_h2o`	`snodb%covmx`
`cov50`	`snodb%cov50`
`snow_init`	`snodb%init_mm`

Purpose

`soils.sol` defines every soil used in the watershed. Each soil is a block of lines: one profile line with the soil name, number of layers, hydrologic soil group and other whole-profile values, followed by one line per layer with depth, bulk density, available water capacity, saturated hydraulic conductivity, organic carbon, texture and other layer properties. An HRU picks a soil by name in the `soil` column of `hru-data.hru`. The file is named first on the `soils` line of `file.cio` (default `soils.sol`).

Every HRU should name a soil that is in the file. An unknown name is not reported: the HRU silently gets the first soil in `soils.sol`. Check the `sname` column of `checker.out`, which lists the soil each HRU uses.

The initial nutrient and carbon contents of the soil come from `nutrients.sol`, which an HRU selects through `soil_plant.ini`. The same `soils` line of `file.cio` names `nutrients.sol` second and `soils_lte.sol` third (see soils_lte.sol below).

Format

Free format, whitespace-separated. Line 1 is a title and line 2 is the column header; both are skipped. Each soil then takes `nly + 1` lines: the profile line and `nly` layer lines. The reader counts soils by reading each profile line and skipping the next `nly` lines, so `nly` must equal the number of layer lines that follow. Columns are read by position; the header names are not checked. Spacing is not significant, so the wide indentation the SWAT+ Editor gives the layer lines has no meaning.

Profile line

#	Column	Units	Used if zero or out of range	Description
1	`name`	-	-	Soil name, up to 20 characters, matched against the `soil` column of `hru-data.hru`.
2	`nly`	-	-	Number of layer lines that follow. There is no upper limit in the model.
3	`hyd_grp`	-	-	Hydrologic soil group: `A`, `B`, `C` or `D`. It selects the curve number from `cntable.lum` and the conductivity used when `soil_k` is zero. Dual groups such as `A/D` and lower-case letters are not recognized.
4	`dp_tot`	mm	-	Maximum rooting depth. See Rooting depth for how 62.0.1 uses it.
5	`anion_excl`	fraction	0.5 if 0; 0.99 if 1 or more	Fraction of porosity from which anions are excluded.
6	`perc_crk`	fraction	-	Potential (maximum) crack volume of the soil profile as a fraction of total soil volume. Used only when crack flow is switched on (`crack` = 1 in `codes.bsn`).
7	`texture`	-	-	Texture label, up to 16 characters. Only the first whitespace-delimited word is read, so `L-L-L-L` is read whole but `L- L- L- L` is read as `L-`. The label is not used in any process; it can serve as a condition in `calibration.cal` (condition `texture`).

Layer lines

Layers are listed from the surface down. Each line has fourteen values.

#	Column	Units	Used if zero or out of range	Description
1	`dp`	mm	-	Depth from the soil surface to the bottom of the layer.
2	`bd`	Mg/m^3	1.3 if 0; capped at 2.0	Moist bulk density: oven-dry mass divided by the soil volume at or near field capacity.
3	`awc`	mm H_2O/mm soil	0.005 if 0; capped at 0.8	Available water capacity: water held between field capacity (-0.033 MPa) and permanent wilting point (-1.5 MPa).
4	`soil_k`	mm/h	by `hyd_grp` if 0 or less: A 50, B 20, C 5, D 2, other 10	Saturated hydraulic conductivity.
5	`carbon`	% of soil mass	0.001 if below 0.001	Organic carbon content of the fine earth (material passing a 2 mm sieve).
6	`clay`	%	-	Clay, particles below 0.002 mm.

#	Column	Units	Used if zero or out of range	Description
7	`silt`	%	-	Silt, particles between 0.002 and 0.05 mm.
8	`sand`	%	-	Sand, particles between 0.05 and 2.0 mm.
9	`rock`	% of total mass	capped at 98	Rock fragments, particles above 2 mm.
10	`alb`	fraction	-	Moist soil albedo. Only the top layer value is used.
11	`usle_k`	0.013 t m^2 h/(m^3 t cm)	-	USLE soil erodibility factor. Only the top layer value is used.
12	`ec`	dS/m	-	Electrical conductivity. Read and reported, but not used by any process in 62.0.1.
13	`caco3`	%	2.8 if 0	Calcium carbonate content.
14	`ph`	-	6.5 if 0	Soil pH. Read and reported, but not used by any process in 62.0.1.

`clay`, `silt` and `sand` are percentages of the mineral fraction and should add up to 100. The model does not check this, except that in sub-daily runs the sand content of the top layer is reset to 100 − clay − silt.

Layering

Thin first layer

After reading a soil, the reader checks the first layer. If it is shallower than 20 mm and the soil has only one layer, or the second layer reaches deeper than 20 mm, the bottom of the first layer is set to 20 mm.

Added 10 mm surface layer

Without a `soil_lyr_depths.sol` file in the run folder (the usual case), SWAT+ adds a 10 mm surface layer to every soil whose first layer is deeper than 19.5 mm. The new layer takes all its properties from the first input layer, so a soil with four input layers is simulated with five. The first input layer then runs from 10 mm to its listed depth. Outputs by layer count this added layer.

If the first layer is 19.5 mm or shallower (possible only when the second layer is also within 20 mm), no layer is added and the first layer is given a depth of 10 mm.

soil_lyr_depths.sol

If a file named `soil_lyr_depths.sol` is in the run folder, every soil is re-layered to the depths it lists. The file is not named in `file.cio`; SWAT+ looks for this name.

- Line 1 is a title, line 2 a header and line 3 a units label; all three are skipped.
- Each later line gives one depth in mm, from the surface down, as a whole number.
- Depths of 10 mm or less are skipped. The first layer is always 0 to 10 mm.
- A depth beyond the bottom of a soil is replaced by its total depth, and the list ends there for that soil. If the deepest listed depth is shallower than a soil, the profile is cut off at that depth.

Each new layer takes the depth-weighted average of the input layers it overlaps, computed millimetre by millimetre, for every layer property. `refdata/Ames_sub1/soil_lyr_depths.sol` lists 10, 25, 50, 100, 150, 200, 300, 450, 600, 750, 900, 1050, 1200, 1500, 1800 and 2100 mm, so a 2000 mm soil gets sixteen layers with the last from 1800 to 2000 mm.

Septic biozone layer

In an HRU with a septic system (the `sep` column of `landuse.lum`), the profile is split further so that the biozone set in `septic.str` is a separate layer.

Derived layer properties

At the start of the run SWAT+ derives the water contents of each layer from `bd`, `awc` and `clay`. As volume fractions:

$$\theta_{wp} = 0.40\,\frac{c\,\rho_b}{100}, \qquad \theta_{fc} = \theta_{wp} + AWC, \qquad \phi = 1 - \frac{\rho_b}{2.65}$$

where c is `clay` (%), ρ_b is `bd` and AWC is `awc`. If θ_{wp} is zero (no clay) it is set to 0.005. If θ_{fc} reaches the porosity ϕ, field capacity is set to $\phi - 0.05$ and the wilting point to $\theta_{fc} - AWC$; if that is not positive, field capacity becomes $0.75\,\phi$ and the wilting point $0.25\,\phi$.

For a layer of thickness h (mm), the plant-available water at field capacity is $FC = h\,(\theta_{fc} - \theta_{wp})$, the water between wilting point and saturation is $UL = h\,(\phi - \theta_{wp})$, and the percolation travel time (h) is

$$TT_{perc} = \frac{UL - FC}{K_{sat}}$$

with a minimum of 1 hour, where K_{sat} is `soil_k`. The profile sums of FC and UL are reported as `sumfc` and `sumul` in `checker.out`.

The crack volume of each layer is

$$V_{crk} = \text{perc_crk} \times 0.916\,e^{-0.0012\,z}\,h$$

where z is the depth to the bottom of the layer (mm).

The top layer's sand, silt and clay set the particle-size make-up of eroded sediment (sand, silt, clay, small and large aggregates). For the Green and Ampt infiltration method, the wetting front suction head is estimated from the porosity, clay and sand of the top layer.

Use of the soil properties

Initial soil water

Each layer starts at a fraction of its field capacity water. The fraction is `sw_init` in `parameters.bsn`. If `sw_init` is zero, it is estimated from the average annual precipitation P (mm) of the HRU's weather generator:

$$f = \frac{P}{P + \exp(9.043 - 0.002135\,P)}$$

The `sw_frac` column of `soil_plant.ini` is read but not used for this in 62.0.1.

Rooting depth

`dp_tot` limits root depth only while plants that start the run already growing are initialized. After that the model sets the HRU's maximum rooting depth to the bottom of the soil profile. Each planting operation then lowers it to the smaller of its current value and the plant's maximum root depth in `plants.plt`. The value is never raised again during the run, so after a shallow-rooted crop is planted, later crops in the same HRU are limited to that depth as well. The `zmx` column of `checker.out` shows this profile depth. It does not show `dp_tot`.

Runoff, percolation and solute leaching

`hyd_grp` picks one of the four curve numbers in `cntable.lum` for the HRU's land use. `anion_excl` reduces the pore volume available to nitrate, salts and other constituents moving down with percolating water. The nitrate that moves with a water volume w (mm) leaving a layer is

$$NO3_{mobile} = NO3_{ly}\left[1 - \exp\left(\frac{-w}{(1 - \text{anion_excl})\,UL}\right)\right]$$

See hydrology, soil water and temperature and nutrient cycling.

Erosion

The MUSLE soil factor of an HRU uses the top layer only:

$$K \cdot CFRG = \text{usle_k}_1 \; e^{-0.053\,\text{rock}_1}$$

which is multiplied by the support practice factor from `cons_practice.lum` and the slope length and steepness factor. See erosion and sediment.

Albedo

On days with less than 0.5 mm of snow, the albedo of a bare HRU is the top layer `alb`. With plant cover it is $0.23\,(1 - e) + \text{alb}_1\, e$, where $e = \exp(-5 \times 10^{-5}(CV + 0.1))$ and CV is above-ground biomass plus residue (kg/ha). There is no lower limit on `alb`, so a zero value gives zero albedo for bare soil.

Carbon and nutrients

`carbon`, `bd` and the layer thickness set the starting organic matter of each layer: the soil mass is $10\,000\,h\,\rho_b$ kg/ha, organic carbon is `carbon` percent of that, and organic matter is carbon divided by 0.58. How this is split into pools depends on the carbon model (the `carbon` column of `codes.bsn`) and on `nutrients.sol`; see nutrients.sol. In 62.0.1 the 0.001% floor on `carbon` applies to every layer; earlier revisions applied it to the top layer only.

With the Vadas and White phosphorus model (`soil_p` = 1 in `codes.bsn`), `clay` and `carbon` set the initial phosphorus sorption coefficient and `caco3` sets the rate of transfer between the active and stable mineral P pools:

$$k_{as} = 0.0023\,\text{caco3} + 0.005, \qquad 0.002 \le k_{as} \le 0.05$$

`caco3` is not used with the original phosphorus model.

Calibration and conditions

Soil columns can be changed in `calibration.cal` with these parameter names, for a range of layers: `anion_excl`, `crk` (`perc_crk`), `z` (`dp`), `bd`, `awc`, `k` (`soil_k`), `cbn` (`carbon`), `clay`, `silt`, `sand`, `rock`, `alb`, `usle_k`, `ec`, `cal` (`caco3`) and `ph`. Changes to `z`, `bd`, `awc` and `clay` recompute the layer water contents and the curve number coefficients. See parameter change files.

Calibration changes are applied before the soil organic pools are set up, and that setup works from the `carbon` column. A `cbn` change acts on the layer's organic carbon pool, so it is replaced when the pools are initialized and has no lasting effect in 62.0.1.

The `calibration.cal` conditions `hsg` and `texture` select HRUs by `hyd_grp` and `texture`. Decision tables can test `hyd_soil_group` and `k_factor` (the top layer `usle_k`).

soils_lte.sol

`soils_lte.sol` is used only by HRU-LTE objects (`hru-lte.hru`), which select a soil texture by name instead of a layered profile. Line 1 is a title and line 2 a header. The next twelve lines are read, one per texture class, each with four values:

#	Column	Units	Description
1	`texture`	-	Texture class name, matched against the soil texture column of `hru-lte.hru`.
2	`awc`	mm/mm	Available water capacity, multiplied by the soil depth given in `hru-lte.hru`.
3	`por`	mm/mm	Porosity, multiplied by the same soil depth.
4	`scon`	mm/h	Saturated hydraulic conductivity.

Twelve lines are always read. A project with HRU-LTE objects needs this file, since there is no built-in texture table.

Example

`refdata/Osu_1hru/soils.sol` in the SWAT+ repository, first soil, with the spacing reduced:

```
soils.sol: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name     nly hyd_grp     dp_tot anion_excl perc_crk texture  dp      bd     awc   soil_k  carbon   clay   s
SONGSAN    5       A 1200.00000    0.50000  0.50000 LS
                                                              180.0 1.39000 0.10871 16.10093 1.41760 17.300 25
                                                              250.0 1.64000 0.09959 19.31345 0.58820 15.700 25
                                                              460.0 1.48000 0.10069 18.05822 0.42350 16.100 20
                                                              820.0 2.00000 0.07661 37.49110 0.20000 11.300 18
                                                             1200.0 2.00000 0.06645 59.38071 0.26470  7.800 15
```

A loamy sand in hydrologic group A with five layers to 1200 mm. Because the first layer reaches 180 mm, SWAT+ adds a 10 mm surface layer and simulates six layers. `caco3` and `ph` are zero, so the model uses 2.8% and 6.5. The albedo (0.20) and K factor (0.24977) of the first layer are the only values of those two columns that affect the run.

Related files

- `hru-data.hru`: selects a soil by name in its `soil` column.
- `soil_plant.ini` and `nutrients.sol`: initial soil water, nutrients and other constituents of the soil.
- `cntable.lum`: curve numbers chosen by `hyd_grp`.
- `plants.plt`: plant maximum root depth, which limits rooting together with the soil depth.
- `codes.bsn` and `parameters.bsn`: crack flow switch, phosphorus model, carbon model and `sw_init`.
- `septic.str`: septic biozone layer.
- `checker.out`: profile summary by HRU.

Sources

- Readers: `soil_db_read.f90`, `soil_lte_db_read.f90`, called from `proc_read.f90`
- Type definitions: `type soil_profile_db`, `type soilayer_db` and `type soil_lte_database` in `soil_data_module.f90`; simulated soil in `soil_module.f90`
- Name lookup from `hru-data.hru`: `hru_read.f90`
- Layering, `soil_lyr_depths.sol` and septic split: `soils_init.f90`, `layersplit.f90`
- Value checks and derived properties: `soil_phys_init.f90`, `soil_awc_init.f90`, `soil_text_init.f90`
- Initial soil water, rooting depth and MUSLE factor: `hydro_init.f90`, `mgt_plantop.f90`, `pl_root_gro.f90`
- Curve number: `cn2_init.f90`, `curno.f90`
- Albedo, leaching and P: `albedo.f90`, `nut_nlch.f90`, `nut_pminrl2.f90`
- Carbon and nutrient start values: `soil_nutcarb_init.f90`, called from `main.f90.in`

- HRU-LTE texture lookup: hru_lte_read.f90
- Calibration and conditions: cal_parm_select.f90, cal_conditions.f90, conditions.f90
- File name slots: in_sol%soils_sol, in_sol%nut_sol, in_sol%lte_sol in input_file_module.f90

Column	Fortran variable
name	soildb%s%snam
nly	soildb%s%nly
hyd_grp	soildb%s%hydgrp
dp_tot	soildb%s%zmx
anion_excl	soildb%s%anion_excl
perc_crk	soildb%s%crk
texture	soildb%s%texture
dp	soildb%ly%z, then sol%phys%d
bd	soildb%ly%bd, then sol%phys%bd
awc	soildb%ly%awc, then sol%phys%awc
soil_k	soildb%ly%k, then sol%phys%k
carbon	soildb%ly%cbn, then sol%phys%cbn and soil1%cbn
clay, silt, sand, rock	soildb%ly%clay, %silt, %sand, %rock, then sol%phys%...
alb, usle_k, ec	soildb%ly%alb, %usle_k, %ec, then sol%ly%...
caco3	soildb%ly%cal, then sol%ly%cal
ph	soildb%ly%ph, then sol%ly%ph
soils_lte.sol columns	soil_lte%texture, %awc, %por, %scon

Purpose

landuse.lum lists the land use and management records of the project. Each row has a name and a set of names that point to records in other files: the plant community in plant.ini, the management schedule in management.sch, the curve numbers in cntable.lum, the support practice in cons_practice.lum, the urban land type in urban.urb, the Manning's n in ovn_table.lum and the structural practices in the .str files. An HRU picks a row by name in the lu_mgt column of hru-data.hru. The file is named first on the lum line of file.cio (default landuse.lum).

If an HRU names a land use that is not in the file, it silently gets the first row of landuse.lum; nothing is reported.

Format

Free format, whitespace-separated. Line 1 is a title and line 2 is the column header; both are skipped. Each later line is one land use with fourteen values, read by position. Every column must be filled; write null for an unused link. Names are up to 40 characters.

#	Column	Points to	Description
1	name	-	Land use and management name, matched against lu_mgt in hru-data.hru and named by the decision table action lu_change. Names must be unique.
2	cal_group	-	Calibration group label, called cal_grp in earlier documentation. See Calibration groups.
3	plnt_com	plant.ini	Plant community at the start of the run. null starts the HRU with no plants (bare soil or an urban surface).
4	mgt	management.sch	Management schedule. null means no scheduled operations: no planting, harvest or kill, so only plants that start the run growing (lc_status y in plant.ini) grow.
5	cn2	cntable.lum	Curve number record. The HRU's soil group picks one of its four values.
6	cons_prac	cons_practice.lum	Conservation practice, which sets the USLE P factor.
7	urban	urban.urb	Urban land type. null for land that is not urban.

#	Column	Points to	Description
8	urb_ro	-	Urban build-up and wash-off method: usgs_reg (USGS regression equations) or buildup_washoff (build-up and wash-off, as in SWMM). Any other value, including null, gives no urban sediment and nutrient loads even when urban is set.
9	ov_mann	ovn_table.lum	Overland flow Manning's n class.
10	tile	tiledrain.str	Tile drainage.
11	sep	septic.str	Septic system. See Septic systems.
12	vfs	filterstrip.str	Vegetative filter strip.
13	grww	grassedww.str	Grassed waterway.
14	bmp	bmpuser.str	User-defined BMP removal efficiencies.

Name matching

All names are matched case-sensitively when the files are read, before the HRUs are set up. For each name other than null that is not found, SWAT+ writes a line to diagnostics.out with the land use name and the missing name:

Column	Message	Result when not found
plnt_com	not found in plant.ini	No plants
mgt	not found in management.sch	No schedule
cn2	not found in cntable.lum	Built-in curve numbers 30, 55, 70, 77 (soil groups A to D)
cons_prac	not found (landuse-consprac)	P factor 1.0
tile	not found (landuse-tiledr)	No tile drain
sep	not found (landuse-septic)	No septic system
vfs	not found (landuse-fstrip)	No filter strip
grww	not found (landuse-grassww)	No grassed waterway
bmp	not found (landuse-bmpuser)	No user BMP

urban and ov_mann are matched later, when each HRU's plant community is set up, and an unknown name is not reported. An unknown urban name leaves the HRU non-urban. An unknown ov_mann name leaves the HRU's Manning's n at its built-in value of 0.05.

Use at the start of the run

For each HRU, SWAT+ takes from its land use:

- the plant community, the management schedule and the starting operation in the schedule (from rot_yr_ini in plant.ini);
- the curve number for moisture condition II from cntable.lum, for the soil's hydrologic group, from which the condition I and III values follow (see hydrology);
- the USLE P factor from cons_practice.lum;
- the urban land type and urban runoff method;
- the overland Manning's n (the ovn_mean column of ovn_table.lum), used for the time of concentration;
- the structural practice parameters from the .str files. The septic record is assigned before the soil layers are set up, so that a biozone layer can be added to the profile.

Land use change

The decision table action `lu_change` (see decision tables) switches an HRU to another `landuse.lum` row during the run and logs the change to `lu_change_out.txt`. The new plant community, schedule, curve number, P factor (and the USLE multiplier), urban type and Manning's n replace the old ones. Some values set at the start are not recalculated:

- The time of concentration keeps the old Manning's n.
- The structural practice parameters (tile, filter strip, grassed waterway, user BMP, septic) are not set again. The HRU keeps those of its first land use.
- The `cal_group` used by `calibration.cal` and decision table conditions stays that of the first land use, while soft calibration uses the group of the current land use.

Calibration groups

`cal_group` labels land uses for calibration:

- Soft calibration matches the `name` column of the land use regions in `water_balance.sft` against `cal_group`; the name `basin` matches every HRU.
- In `calibration.cal`, the condition `cal_group` limits a parameter change to HRUs with that group, and the condition `landuse` limits it to HRUs with that `landuse.lum` name.
- Decision tables can test `cal_group` as a condition.

A value of `null` is an ordinary label: all land uses with `null` form one group.

Septic systems

When `sep` names a record in `septic.str`, that record is assigned to every HRU with this land use. If its operation flag is not zero, SWAT+ adds a biozone layer to the soil profile at the depth and thickness given in `septic.str`, and from the year the system becomes operational it runs the biozone routine on each day the biozone layer is above freezing.

In 62.0.1 the daily volume of septic tank effluent is never calculated and stays at zero. No effluent water is added to the biozone layer, and the effluent concentrations in `septic.sep`, which are multiplied by that volume, add no nitrogen, phosphorus or BOD to the soil. The biozone routine still runs on the water and nutrients already in the layer. Septic systems set up through this column therefore have no effluent load on the HRU water or nutrient balance.

Example

`refdata/Ames_sub1/landuse.lum` in the SWAT+ repository, four of its rows, with the spacing reduced:

```
landuse.lum: written by SWAT+ editor v2.3.3 on 2023-10-25 08:58 for SWAT+ rev.60.5.7
name      cal_group plnt_com  mgt      cn2        cons_prac     urban urb_ro          ov_mann        tile
cosy_lum  null      cosy_comm mgt_01   rc_strow_g up_down_slope null  null            convtill_nores null
agrr_lum  null      agrr_comm AGRR_rot rc_strow_g cross_slope   null  null            convtill_res   null
frst_lum  null      frst_comm null     wood_f     up_down_slope null  null            forest_med     null
urld_lum  null      null      null     urban      up_down_slope URLD  buildup_washoff urban_asphalt  null
```

- `cosy_lum` grows the corn and soybean community `cosy_comm` under schedule `mgt_01`, with row crop curve numbers (straight row, good condition) and no support practice (P = 1 for up-and-down-slope tillage).
- `frst_lum` has no schedule, so the forest in `frst_comm` has no operations; it grows only if `frst_comm` starts it growing.
- `urld_lum` has no plants. It links to the urban type `URLD` in `urban.urb`, uses build-up and wash-off for urban loads, the `urban` curve numbers (98 for all soil groups) and the Manning's n of asphalt.
- In this project `AGRR_rot` is not in `management.sch`, so `diagnostics.out` reports `agrr_lum AGRR_rot not found in management.sch` and HRUs with `agrr_lum` run without a schedule.

Related files

- hru-data.hru: selects a land use by name in its `lu_mgt` column.
- plant.ini: plant communities.
- management.sch: management schedules.
- cntable.lum, [cons_practice.lum](#input-reference-lum-tables-cons-practicelum) and ovn_table.lum: the lookup tables.
- Structural practices and urban.urb: tiledrain.str, septic.str, filterstrip.str, grassedww.str, bmpuser.str, septic.sep and urban.urb.
- lum.dtl: decision tables, including `lu_change`.
- Plant growth and land use management.

Sources

- Reader: landuse_read.f90, called from proc_db.f90 after the plant community, schedule, lookup table and structural files
- Type definitions: `type land_use_management` and `type land_use_structures` in landuse_data_module.f90
- Name lookup from `hru-data.hru`: hru_read.f90; `cal_group` copied to the HRU in hrudb_init.f90
- HRU setup: hru_lum_init.f90, plant_init.f90, cn2_init.f90, time_conc_init.f90
- Structural practices: structure_init.f90, structure_set_parms.f90; septic setup in proc_hru.f90 and soils_init.f90; daily septic code in hru_control.f90, swr_percmain.f90, sep_biozone.f90
- Urban loads: hru_urban.f90
- Land use change: `lu_change` in actions.f90
- Calibration groups: cal_conditions.f90, conditions.f90, calsoft_hyd.f90
- File name slot: `in_lum%landuse_lum` in input_file_module.f90

Column	Fortran variable
name	lum%name
cal_group	lum%cal_group, copied to hru%cal_group and hru%lum_group_c
plnt_com	lum%plant_cov, index lum_str%plant_cov
mgt	lum%mgt_ops, index lum_str%mgt_ops
cn2	lum%cn_lu, index lum_str%cn_lu
cons_prac	lum%cons_prac, index lum_str%cons_prac
urban	lum%urb_lu, index hru%luse%urb_lu
urb_ro	lum%urb_ro, copied to hru%luse%urb_ro
ov_mann	lum%ovn, value hru%luse%ovn
tile	lum%tiledrain, index lum_str%tiledrain
sep	lum%septic, index lum_str%septic, then iseptic
vfs	lum%fstrip, index lum_str%fstrip
grww	lum%grasww, index lum_str%grasww
bmp	lum%bmpuser, index lum_str%bmpuser

Purpose

Three lookup tables supply values that landuse.lum names by record: `cntable.lum` (SCS curve numbers), `cons_practice.lum` (USLE support practice factor) and `ovn_table.lum` (Manning's n for overland flow). They are named third, fourth and fifth on the `lum` line of file.cio, after landuse.lum and management.sch. If a file is missing or named `null`, every land use gets the built-in values given below.

All three files have the same layout. Free format, whitespace-separated. Line 1 is a title and line 2 is the column header; both are skipped. Each later line is one record, read by position. Values after the last numeric column (the descriptions the SWAT+ Editor writes) are not read. Record names are up to 40 characters and are matched case-sensitively.

cntable.lum

Curve numbers for moisture condition II, one value per hydrologic soil group, for each combination of land cover, treatment and hydrologic condition. The `cn2` column of `landuse.lum` names a record.

#	Column	Units	Built-in value	Range	Description
1	`name`	-	-	-	Curve number record name.
2	`cn_a`	-	30	30 to 100	Curve number for hydrologic soil group A.
3	`cn_b`	-	55	30 to 100	Curve number for hydrologic soil group B.
4	`cn_c`	-	70	30 to 100	Curve number for hydrologic soil group C.
5	`cn_d`	-	77	30 to 100	Curve number for hydrologic soil group D.

The SWAT+ Editor also writes `description`, `treat` and `cond_cov` columns, which are not read. The built-in values apply to land uses whose `cn2` is `null` or not found.

Curve number in the model

At the start of the run each HRU takes the value for the hydrologic soil group (`hyd_grp`) of its soil in `soils.sol`. The table values are those of SCS Engineering Division (1986), which apply to a 5% slope; SWAT+ 62.0.1 does not adjust them for the HRU slope. From CN_2 SWAT+ derives the dry and wet condition values

$$CN_1 = CN_2 - \frac{20\,(100 - CN_2)}{(100 - CN_2) + \exp[2.533 - 0.0636\,(100 - CN_2)]}, \qquad CN_1 \geq 0.4\,CN_2$$

$$CN_3 = CN_2\,\exp[0.006729\,(100 - CN_2)]$$

and the retention parameter curve used for daily runoff, as described in hydrology.

CN_2 can change during the run through the management operation `cnup` in `management.sch`, the decision table action `cn_update` (both limited to 35 to 95), a land use change (`lu_change`, which picks the new land use's record) and the calibration parameter `cn2` in `calibration.cal`.

Example

`refdata/Ames_sub1/cntable.lum` in the SWAT+ repository, first records, with the spacing reduced:

```
cntable.lum: written by SWAT+ editor v2.3.3 on 2023-10-25 08:58 for SWAT+ rev.60.5.7
name        cn_a      cn_b      cn_c      cn_d  description  treat              cond_cov
fal_bare    77.00000  86.00000  91.00000  94.00000  Fallow   Bare_soil          ----
fal_res_p   76.00000  85.00000  90.00000  93.00000  Fallow   Crop_residue_cover  Poor
fal_res_g   74.00000  83.00000  88.00000  90.00000  Fallow   Crop_residue_cover  Good
rc_strow_p  72.00000  81.00000  88.00000  91.00000  Row_crops  Straight_row     Poor
```

The names combine land cover, treatment and condition: `rc_strow_p` is row crops, straight row, poor condition. The SWAT+ Editor's table has 52 records, from fallow and row crops to forest, farmsteads and roads.

cons_practice.lum

USLE support practice factors for conservation practices. The `cons_prac` column of `landuse.lum` names a record.

#	Column	Units	Built-in value	Range	Description
1	`name`	-	-	-	Conservation practice name.
2	`usle_p`	-	1.0	0 to 1	USLE support practice factor P.
3	`slp_len_max`	m	1.0	-	Maximum slope length for the practice. Read but not used (see below).

The SWAT+ Editor also writes a `description` column, which is not read. The built-in values apply to land uses whose `cons_prac` is `null` or not found, so a P factor of 1 means no support practice.

P factor in the model

P enters the MUSLE sediment yield through a multiplier fixed at the start of the run:

$$K \cdot C_{frg} \cdot P \cdot LS \cdot 11.8$$

where K and C_{frg} come from the top soil layer (see soils.sol), and the slope length and steepness factor is

$$LS = \left(\frac{L}{22.128}\right)^{m} \left(65.41 \, \sin^2 \alpha + 4.56 \, \sin \alpha + 0.065\right), \qquad m = 0.6 \left[1 - \exp(-35.835 \, S)\right]$$

with L the slope length (`slp_len` in `topography.hyd`, m), S the slope (m/m) and $\alpha = \arctan S$. The model works out the smaller of L and `slp_len_max`, but then computes LS from L, so `slp_len_max` has no effect on LS in 62.0.1. See erosion and sediment.

P can also be set during the run by the decision table actions `p_factor`, `contour`, `stripcrop` and `terrace`, which recompute the multiplier. The calibration parameter `usle_p` in `calibration.cal` changes the HRU's P factor after the multiplier has been computed and does not recompute it, so by itself it does not change sediment yield.

Example

`refdata/Ames_sub1/cons_practice.lum`, first records:

```
cons_practice.lum: written by SWAT+ editor v2.3.3 on 2023-10-25 08:58 for SWAT+ rev.60.5.7
name                 usle_p   slp_len_max  description
up_down_slope       1.00000     121.00000  Up_and_down_slope
cross_slope         0.75000     121.00000  Cross_slope_tillage
contour_farming     0.50000     121.00000  Contour_tillage
strip_cros_slope    0.37000     121.00000  Strip_cropping_cross_slope
```

The SWAT+ Editor's table has 38 records: up-and-down and cross-slope tillage, contouring and strip cropping by slope class (1-2% to 21-25%, with pasture or row crops), and terraces by slope class with sod or underflow outlets.

ovn_table.lum

Manning's roughness coefficient for overland flow by tillage, residue and land cover. The `ov_mann` column of `landuse.lum` names a record.

#	Column	Units	Built-in value	Description
1	`name`	-	-	Manning's n class name.
2	`ovn_mean`	s/m1/3	0.5	Mean Manning's n for overland flow. This is the value the model uses.
3	`ovn_min`	s/m1/3	0.5	Minimum Manning's n. Read but not used; a guide for calibration ranges.
4	`ovn_max`	s/m1/3	0.5	Maximum Manning's n. Read but not used; a guide for calibration ranges.

The SWAT+ Editor also writes a `description` column, which is not read. A land use whose `ov_mann` is `null` or not found does not take a value from this file; its HRUs keep the model's starting value of 0.05.

Manning's n in the model

The HRU's Manning's n sets the overland part of its time of concentration (hours), which controls the peak runoff rate:

$$t_{ov} = \frac{0.0556\,(L\,n)^{0.6}}{(S + 0.0001)^{0.3}}$$

with L the slope length (m) and S the slope (m/m). The same n is used as the roughness of the HRU's concentrated flow channel in the channel part of the time of concentration, and in the area-weighted roughness of routing units.

The time of concentration is computed once, before calibration is applied. The calibration parameter `ovn` in `calibration.cal` changes the HRU's Manning's n after that, and a land use change (`lu_change`) also leaves the time of concentration unchanged, so neither alters the peak runoff rate in 62.0.1.

Example

`refdata/Ames_sub1/ovn_table.lum`, first records:

```
ovn_table.lum: written by SWAT+ editor v2.3.3 on 2023-10-25 08:58 for SWAT+ rev.60.5.7
name                  ovn_mean       ovn_min       ovn_max  description
fallow_nores           0.01000       0.00800       0.01200  Fallow_no_residue
convtill_nores         0.09000       0.06000       0.12000  Conventional_tillage_no_residue
convtill_res           0.19000       0.16000       0.22000  Conventional_tillage_residue
chisplow_nores         0.09000       0.06000       0.12000  Chisel_plow_no_residue
```

The SWAT+ Editor's table has 20 records, from fallow (0.01) and tilled cropland through range, grasses and forest (up to 0.80 for heavy forest) to urban surfaces (0.011 for asphalt).

Related files

- `landuse.lum`: names records in these tables in its `cn2`, `cons_prac` and `ov_mann` columns.
- `soils.sol`: the hydrologic soil group that picks the curve number, and the K factor and rock content of the MUSLE multiplier.
- `topography.hyd`: slope and slope length.
- `management.sch` and `lum.dtl`: the `cnup` operation and the `cn_update`, `p_factor`, `contour`, `stripcrop` and `terrace` actions.
- Parameter change files: calibration parameters `cn2`, `usle_p` and `ovn`.

Sources

- Readers: `cntbl_read.f90`, `cons_prac_read.f90`, `overland_n_read.f90`, called from `proc_db.f90`
- Type definitions: `type curvenumber_table`, `type conservation_practice_table` and `type overlandflow_n_table` in `landuse_data_module.f90`
- Name lookup: `landuse_read.f90` (`cn2`, `cons_prac`); `plant_init.f90` (`ov_mann`)
- Curve number: `cn2_init.f90`, `curno.f90`; changes in `mgt_sched.f90` (`cnup`) and `actions.f90` (`cn_update`)
- P factor and LS: `plant_init.f90`, `hydro_init.f90`, `ero_ysed.f90`, `actions.f90`
- Manning's n: `time_conc_init.f90`
- Calibration: `cal_parm_select.f90`; call order in `main.f90.in`
- File name slots: `in_lum%cntable_lum`, `in_lum%cons_prac_lum`, `in_lum%ovn_lum` in `input_file_module.f90`

Column	Fortran variable
`cntable.lum name`	`cn%name`
`cn_a`, `cn_b`, `cn_c`, `cn_d`	`cn%cn(1)` to `cn%cn(4)`, copied to `cn2(j)`
`cons_practice.lum name`	`cons_prac%name`
`usle_p`	`cons_prac%pfac`, copied to `hru%lumv%usle_p`
`slp_len_max`	`cons_prac%sl_len_mx`

Column	Fortran variable
ovn_table.lum name	overland_n%name
ovn_mean	overland_n%ovn, copied to hru%luse%ovn
ovn_min, ovn_max	overland_n%ovn_min, overland_n%ovn_max

Purpose

The channel line of file.cio names eight channel input files. The table below gives the use of each in 62.0.1. initial.cha (initial channel contents), temperature.cha (stream temperature) and the legacy files channel.cha, hydrology.cha and sediment.cha are described below; the other files are on the chandeg channel page.

SWAT+ has two channel objects. The channels listed in chandeg.con (SWAT-DEG or "lte" channels) are the ones 62.0.1 routes; their parameter files are on the chandeg channel page. The legacy channel object listed in channel.con is set up at the start of a run, but its daily routing is switched off in 62.0.1, so water sent to it does not leave it. Build channel networks with chandeg.con.

Files on the channel line

Each slot can be null. A missing file is treated like null.

Slot	Default name	Used in 62.0.1 for	Documented on
1	initial.cha	Initial water, sediment, nutrients and pesticides in chandeg channels.	initial.cha
2	channel.cha	Legacy channel object only. No effect on results.	legacy files
3	hydrology.cha	Legacy channel object only. No effect on results.	legacy files
4	sediment.cha	Legacy channel object only. No effect on results.	legacy files
5	nutrients.cha	In-stream water quality of chandeg channels.	chandeg channels
6	channel-lte.cha	One row per chandeg channel, pointing to the other files.	chandeg channels
7	hyd-sed-lte.cha	Geometry, hydraulics and sediment properties of chandeg channels.	chandeg channels
8	temperature.cha	Read, but the stream temperature calculation uses only built-in values.	temperature.cha

Three more channel files have fixed names and are read when present in the run folder: initial.cha_cs (initial pathogens, salts and constituents, below), sed_nut.cha (chandeg channels) and shade_factor.shf (below).

initial.cha

Named sets of initial conditions for chandeg channels. Each row of channel-lte.cha selects a set by name in its cha_ini column. Free format: two header lines are skipped, then one set per line. There is no id column, so the first value on each line is the name. The Editor adds a trailing description value, which the reader ignores.

#	Column	Description
1	name	Name of the set, matched against cha_ini in channel-lte.cha.
2	org_min	Name of a row in om_water.ini: initial water, sediment and nutrient concentrations.
3	pest	Name of a row in pest_water.ini: initial pesticide in the water and the bed.
4	path	Read but not used. Initial pathogens come from initial.cha_cs.
5	hmet	Read but not used.
6	salt	Read but not used. Initial salts come from initial.cha_cs.

A `cha_ini` name not found in `initial.cha`, or an `org_min` or `pest` name not found in its file, falls back to row 1 of that file without a message. Keep `initial.cha` and an `om_water.ini` in any model with chandeg channels.

Initial water and constituents

The `flo` value of the `om_water.ini` row is the initial water depth as a fraction of the bankfull depth `dp`. The channel starts with the volume the rating curve gives at that depth, and the concentrations of the row are converted to masses with that volume. With `flo` of 1 or less all of it is channel storage. Above 1 the volume is split between channel and floodplain in proportion to their volumes on the rating curve. Channels 0.001 km (1 m) long or shorter start empty.

Initial pesticides in the water are the `pest_water.ini` concentration (mg/kg) times the stored water; pesticides in the bed are the benthic concentration times the volume of the active bed layer (width times length times the pesticide's active bed depth).

initial.cha_cs

An optional file with a fixed name, read when present, with initial pathogens, salts and other constituents for chandeg channels. Two header lines are skipped, then one row per line:

#	Column	Description
1	`name`	Name of the row. Not checked.
2	`pest`	Read but not used. Pesticides come from `initial.cha`.
3	`path`	Name of a row in `path_water.ini`.
4	`hmet`	Read but not used.
5	`salt`	Name of a row in `salt_channel.ini`: initial salt ion concentrations (g/m^3).
6	`cs`	Name of a row in `cs_channel.ini`: initial constituent concentrations (g/m^3).

Rows are paired with `initial.cha` by position: row 3 of `initial.cha_cs` applies to channels that use row 3 of `initial.cha`, whatever its name. Keep the two files in the same order with the same number of rows. Salt and constituent masses are the concentrations times the initial water volume.

In 62.0.1 the pathogen pointer is resolved, but the initial pathogen values are then read from the `path_water.ini` row with the position of the channel's `pest_water.ini` row.

temperature.cha

Parameters of the stream temperature calculation for chandeg channels. Free format: two header lines are skipped, then one set per line with 15 values. The name is limited to 13 characters.

#	Column	Units	Built-in value	Typical range	Description
1	`name`	-	-	-	Name of the set (up to 13 characters).
2	`sno_mlt`	-	1.0	-	Weight of the snowmelt contribution.
3	`gw`	-	1.0	-	Weight of the groundwater contribution.
4	`sur_lat`	-	1.0	-	Weight of the surface runoff and lateral flow contributions.
5	`sno_lag`	days	2	1 to 3	Air temperature averaging period for snowmelt.
6	`gw_lag`	days	250	200 to 365	Air temperature averaging period for groundwater.
7	`surf_lag`	days	5	2 to 5	Air temperature averaging period for surface runoff.
8	`lat_lag`	days	10	5 to 10	Air temperature averaging period for lateral flow.
9	`lat_lag_coef`	-	0.75	-	Multiplier on the lagged air temperature for lateral flow.
10	`surf_lag_coef`	-	0.75	-	Multiplier on the lagged air temperature for surface runoff and snowmelt.
11	`gw_lag_coef`	-	0.75	-	Multiplier on the lagged air temperature for groundwater.
12	`hex_coef1`	-	0.75	-	Multiplier on the dew point temperature in the heat exchange.
13	`hex_coef2`	-	1.5	-	Divisor on the channel depth when the flow depth for heat exchange is calculated.
14	`sf_on`	-	0	0 or 1	Shade factor source: `1` daily values from `shade_factor.shf`, `0` the single value `ssff`. Integer.
15	`ssff`	fraction	0.5	0 to 1	Shade factor: the fraction of solar radiation kept off the water surface.

In 62.0.1 the rows of `temperature.cha` have no effect. The reader stores them, but the temperature calculation always uses the built-in values in the table, whether or not the file exists, for every channel. With the built-in `sf_on` of 0, `shade_factor.shf` is not used either. The stream temperature calculation itself runs every day for every chandeg channel.

Stream temperature

The temperature of the water leaving a channel is calculated in three steps. Values in this section are the built-in values above.

Local inflow. For the routing units that drain directly into the channel, the day's HRU snowmelt Q_{sno}, surface runoff Q_{sur}, lateral flow Q_{lat} and water yield Q_{wyld} are summed (mm), and the groundwater inflow Q_{gw} (m³/s, at most 10) is taken from the channel's hydrograph separation. Each component gets an air temperature averaged over its lag period: T_x is the mean of the daily $(T_{max} + T_{min})/2$ over the last `surf_lag`, `lat_lag`, `gw_lag` or `sno_lag` days, at least 0.01 °C, and for snowmelt at most 2.5 °C. In the first simulation year the lag is limited to the day of the year. Then

$$T_{loc} = \frac{a_{sno} k_{sur} T_{sno} Q_{sno} + a_{gw} k_{gw} T_{gw} Q_{gw} + a_{sl} k_{lat} T_{lat} Q_{lat} + a_{sl} k_{sur} T_{sur} Q_{sur}}{Q_{wyld}}$$

with a_{sno} = `sno_mlt`, a_{gw} = `gw`, a_{sl} = `sur_lat`, and k_{sur}, k_{lat} and k_{gw} the lag coefficients `surf_lag_coef`, `lat_lag_coef` and `gw_lag_coef`. With gwflow active the groundwater term is left out. If T_{loc} differs from the previous day's outflow temperature by more than 5 °C, or the weather station has no temperature gauge, $T_{loc} = 5 + 0.75\,\bar{T}$ with $\bar{T}$ the day's mean air temperature.

Mixing with upstream water. Water from upstream channels arrives at their flow-weighted outflow temperature T_{up} (or $5 + 0.75\,\bar{T}$ for a channel without upstream channels). It is weighted by q, the outflow of the channel itself for the day (m³/s):

$$T_0 = \frac{T_{up}\, q + T_{loc}\, Q_{wyld}}{q + Q_{wyld}}$$

or $T_0 = T_{loc}$ when the outflow is zero.

Heat exchange. Following Edinger et al. (1974), with dew point $T_d = \min(h_1\, T_{dew},\ \bar{T})$, h_1 = `hex_coef1`, $T_m = (T_0 + T_d)/2$ and wind speed U (m/s),

$$\beta = 0.35 + 0.015\, T_m + 0.0012\, T_m^2, \qquad K_e = 4.48 + 0.05\, T_0 + (\beta + 0.47)(9.2 + 0.46\, U^2)$$

The short-wave input is $H_s = 0.97 \times 11.57\, R_s\, (1 - SF)$ W/m², with R_s the solar radiation (MJ/m²/day) and SF the shade factor `ssff`. The long-wave input is $H_a = 0.96\, \varepsilon\, \sigma\, (\bar{T} + 273.15)^4$ with $\varepsilon = 0.74 + 0.0065\, e_a\, (1 + 0.17\, C^2)$, e_a the vapour pressure (hPa) and $C = 0.9\, R_s / R_{s,max} + 0.1$ the cloud factor. The equilibrium temperature is

$$T_{eq} = T_d + \frac{H_s + H_a - 305.5 - 4.48\, T_d}{K_e}$$

and the outflow temperature is

$$T = T_0 + \frac{K_e\, (T_{eq} - T_0)\, t}{\rho\, c_p\, d_f / 86400}, \qquad T \geq 0.001$$

where t is the travel time through the channel (days) at the day's peak flow, $\rho c_p = 4.186 \times 10^6$ J/m³/°C and d_f the flow depth (m). The flow depth comes from a triangular section with depth-to-width ratio $(D/h_2)/W$, h_2 = `hex_coef2`, that passes the day's outflow at the channel velocity; it is at least 0.1 m and at most $2D$.

The result is the outflow temperature passed downstream and reported in the outflow block of the `channel_sd` output (see channel outputs). The in-stream water quality and the `water_temp` column of `channel_sd` use $5 + 0.75\,\bar{T}$ instead.

Shade factor file

`shade_factor.shf` has a fixed name. Two header lines are skipped, then one record per line with three values: day of year, channel number (the chandeg channel's sequence number) and shade factor (fraction). It would be used with `sf_on` = 1 on day 1 to 365 (day 366 uses 0.5); in 62.0.1 `sf_on` is always the built-in 0.

Legacy channel files

`channel.cha`, `hydrology.cha` and `sediment.cha` describe the legacy channel object of `channel.con`. SWAT+ 62.0.1 reads them and sets up those channels at the start of the run, but does not route water through them, so the values have no effect on results. The formats are given for existing models. Two header lines are skipped in each file and the reader does not match columns by header name; the column names below are the names the reader uses.

channel.cha

#	Column	Description
1	`id`	Channel number.
2	`name`	Channel name.
3	`init`	Name of a row in `initial.cha`.
4	`hyd`	Name of a row in `hydrology.cha`.
5	`sed`	Name of a row in `sediment.cha`.
6	`nut`	Name of a row in `nutrients.cha`.

Names that are not found are written to `diagnostics.out` as `<name> not found (hydrology.cha)` and similar.

hydrology.cha

#	Column	Units	Built-in value	Description
1	`name`	-	-	Name of the set.
2	`w`	m	2.0	Average width of the main channel.
3	`d`	m	0.5	Average depth of the main channel.
4	`s`	m/m	0.01	Average slope. Values of 0 or less become 0.0001.
5	`l`	km	0.1	Length. Values of 0 or less become 0.001.
6	`n`	-	0.05	Manning's n, limited to 0.01 to 0.70.
7	`k`	mm/h	0.01	Effective hydraulic conductivity of the channel alluvium.
8	`wdr`	m/m	6.0	Width-to-depth ratio. Values of 0 or less become 3.5.
9	`alpha_bnk`	days	0.03	Bank storage recession factor, stored as $e^{-\alpha}$.
10	`side`	-	0.0	Side slope (horizontal per unit rise). Values of 10^{-6} or less become 2.0.

sediment.cha

#	Column	Units	Built-in value	Description
1	`name`	-	-	Name of the set.
2	`eqn`	-	0	Sediment routing method: `0` original SWAT, `1` Bagnold, `2` Kodatie, `3` Molinas and Wu, `4` Yang.

#	Column	Units	Built-in value	Description
3	cov1	-	0.1	Channel erodibility factor. Limited to 0 to 1 for eqn 0; for the other methods, values of 0 or less become 1 and values above 25 become 25.
4	cov2	-	0.1	Channel cover factor, with the same limits as cov1.
5	bnk_bd	g/cm³	0	Bank bulk density. Values near 0 become 1.40.
6	bed_bd	g/cm³	0	Bed bulk density. Values near 0 become 1.50.
7	bnk_kd	cm³/N/s	0	Bank erodibility from a jet test. Values near 0 become $0.2/\sqrt{\tau_c}$, or 0.2 without a critical shear stress.
8	bed_kd	cm³/N/s	0	Bed erodibility, as bnk_kd.
9	bnk_d50	µm	0	Bank median particle size. Values near 0 become 50; at most 10000.
10	bed_d50	µm	0	Bed median particle size. Values near 0 become 500; at most 10000.
11	tc_bnk	N/m²	0	Critical shear stress of the bank.
12	tc_bed	N/m²	0	Critical shear stress of the bed.
13 to 24	erod(1) to erod(12)	-	0	Monthly erodibility, 0 non-erosive to 1 no resistance. If all twelve are zero, all are set to cov1.

Example

refdata/Osu_1hru in the SWAT+ repository, file.cio channel line and initial.cha:

```
channel            initial.cha       null              null              null              nutrients.cha

initial.cha: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                        org_min             pest             path             hmet              salt
initcha1                    no_init             null             null             null              null
```

The model uses only chandeg channels: slots 2 to 4 and 8 are null. All channels use the initial set initcha1, whose org_min points to the row no_init of om_water.ini; that row has all values zero, so the channels start empty. The pest, path, hmet and salt pointers are null. No reference dataset includes temperature.cha, initial.cha_cs, channel.cha, hydrology.cha or sediment.cha.

Related files

- Chandeg channel files: channel-lte.cha, hyd-sed-lte.cha, sed_nut.cha and nutrients.cha.
- chandeg.con and channel.con: the two channel object types.
- file.cio: the channel line.
- om_water.ini, pest_water.ini, path_water.ini, salt_channel.ini, cs_channel.ini: initial contents referenced from initial.cha and initial.cha_cs.
- tmp.cli: the daily maximum and minimum temperatures used for the lagged air temperatures.
- Channel outputs and channels in the model theory.

Sources

- File name slots: type input_cha in input_file_module.f90, read in readcio_read.f90; in_shf in the same module
- Read order: proc_cha.f90; temperature.cha and shade_factor.shf from proc_read.f90
- Readers: ch_read_init.f90, ch_read_init_cs.f90, ch_read_temp.f90, shade_factor_read.f90, ch_read.f90, ch_read_hyd.f90, ch_read_sed.f90
- Type definitions: channel_data_module.f90 (channel_init_datafiles, channel_init_datafiles_cs, water_temperature_data, channel_data_char_input, channel_hyd_data, channel_sed_data); swatdeg_init_datafiles in sd_channel_module.f90; shade_factor_data in hydrograph_module.f90

- Initial pointers and contents: sd_channel_read.f90, sd_hydsed_init.f90, om_water_init.f90
- Stream temperature: ch_temp.f90, called from sd_channel_control3.f90; travel time and velocity from sd_channel_sediment3.f90
- Legacy object: set-up in ch_initial.f90; the "chan" case is commented out in command.f90

File and column	Fortran variable
initial.cha row	ch_init(i) (name, org_min, pest, path, hmet, salt)
initial.cha_cs row	ch_init_cs(i) (name, pest, path, hmet, salt, cs)
resolved pointers	sd_init(i)%org_min, %pest, %path, %salt, %cs
temperature.cha row	w_temp(i), components as the column names
shade_factor.shf row	shf_db(i)%jday, %cha, %value
channel.cha row	ch_dat_c(i) -> ch_dat(i)
hydrology.cha row	ch_hyd(i)
sediment.cha row	ch_sed(i)

Purpose

SWAT+ 62.0.1 routes water, sediment, nutrients and other constituents through the channels listed in chandeg.con. These are often called SWAT-DEG or "lte" channels, and their outputs are the channel_sd files. They use these parameter files:

- channel-lte.cha: one row per channel, pointing by name to the other files.
- hyd-sed-lte.cha: channel geometry, hydraulics, bank and bed properties.
- sed_nut.cha: optional extra sediment and nutrient coefficients. The SWAT+ Editor does not write it.
- nutrients.cha: in-stream water quality (QUAL2E) rate constants.

Initial channel contents come from initial.cha, described on the channel group page together with temperature.cha and the legacy files channel.cha, hydrology.cha and sediment.cha. The legacy channel object of channel.con is not routed in 62.0.1, so those legacy files have no effect on the results.

Links between the files

The lcha column of chandeg.con gives the id of a row in channel-lte.cha. That row names one row in each of the other files:

channel-lte.cha column	Matched against	If the name is not found
cha_ini	name in initial.cha	row 1 of initial.cha
cha_hyd	name in hyd-sed-lte.cha, and name in sed_nut.cha	row 1 of each file (or the built-in values of sed_nut.cha when that file is absent)
cha_sed	not used	
cha_nut	name in nutrients.cha	row 1 of nutrients.cha

An unmatched name writes no message to diagnostics.out; the channel takes row 1 without warning. Check the names before a run.

The file names of channel-lte.cha, hyd-sed-lte.cha, nutrients.cha and initial.cha are set on the channel line of file.cio (slots 6, 7, 5 and 1). sed_nut.cha has a fixed name and is read when a file of that name is in the run folder.

channel-lte.cha

Free format, whitespace-separated. Line 1 is a title and line 2 is the column header; both are skipped. Each later line is one channel with six values in this order.

#	Column	Units	Description
1	id	-	Channel number. chandeg.con refers to the row by this number in its lcha column.
2	name	-	Channel name.
3	cha_ini	-	Name of a row in initial.cha.
4	cha_hyd	-	Name of a row in hyd-sed-lte.cha, and of a row in sed_nut.cha when that file is used.
5	cha_sed	-	Read but not used. The Editor writes null.
6	cha_nut	-	Name of a row in nutrients.cha.

Number the rows 1, 2, 3 and so on in file order. The row is stored under its id, but the name lookups are applied to the row's position in the file, so rows whose id differs from their position get the wrong parameter sets.

hyd-sed-lte.cha

Free format. Two header lines are skipped, then one parameter set per line with 23 values in this order. The reader does not match columns by header name. The Editor adds a description header with no values under it, which the reader ignores.

Several column headers written by the Editor do not describe what SWAT+ 62.0.1 does with the value. The table gives the Editor header and the use in 62.0.1.

#	Column	Units	Typical range	Description
1	name	-	-	Name of the parameter set, matched against cha_hyd in channel-lte.cha.
2	order	-	1 to 12	Stream order (integer). Groups channels in the chanbud_order output and is tested by the decision table condition ch_order. Use 1 to 12: values outside that range fall outside the order budget table.
3	wd	m	from QSWAT+	Bankfull top width of the channel, calculated by QSWAT+ from the drainage area.
4	dp	m	from QSWAT+	Bankfull depth, from the top of the bank to the channel bottom, calculated by QSWAT+ from the drainage area.
5	slp	m/m	from QSWAT+	Average channel slope along its length, from the DEM. Values below 10^{-9} are raised to 10^{-6}.
6	len	km	from QSWAT+	Channel length. A channel of 0.001 km (1 m) or less passes its inflow through unchanged.
7	mann	-	0.025 to 0.15	Manning's n of the channel. Values below 0.01 are raised to 0.01. Chow (1959) gives about 0.03 for clean straight channels, 0.04 to 0.05 for winding channels with pools, weeds or stones, and 0.07 to 0.15 for sluggish or very weedy reaches.

#	Column	Units	Typical range	Description
8	`k`	mm/h	0 to 130	Effective hydraulic conductivity of the channel bed, used for transmission losses. Lane (1983) gives more than 127 mm/h for very clean gravel and large sand, 51 to 127 for clean sand and gravel, 25 to 76 for sand and gravel with little silt and clay, 6 to 25 for sand and gravel with much silt and clay, and 0.025 to 2.5 for consolidated beds with high silt and clay. Use 0 for perennial streams fed by groundwater.
9	`erod_fact`	-	-	Bank erosion exponent. Read but not used.
10	`cov_fact`	-	0 to 1	Channel cover factor. It adds a vegetation term to the critical velocity for bank erosion, so a larger value protects the banks more. See Bank erosion.
11	`wd_rto`	-	1.05 and up	Read as the channel sinuosity (channel length over straight-line length), although the header and the values the Editor writes are a width-to-depth ratio. Values below 1.05 are raised to 1.05. Used for the bend velocity in bank erosion.
12	`eq_slp`	-	about 1	Critical velocity coefficient: a multiplier on the calculated critical velocity for bank erosion. The header name suggests an equilibrium slope; 62.0.1 has no such parameter.
13	`d50`	mm	-	Median bed sediment size. Read, but replaced by 1.0 mm at the start of every day's sediment calculation, so the value in the file has no effect.
14	`clay`	%	0 to 100	Clay content of bank and bed. Sets the bank cohesion.
15	`carbon`	%	0 to 100	Organic carbon content of the bed. Sets the pesticide sorption coefficient in the channel.
16	`dry_bd`	t/m^3	1.1 to 1.9	Dry bulk density of bank and bed. Converts bank and bed erosion from metres to tonnes, and sets the bed porosity for pesticide mixing.
17	`side_slp`	-	-	Channel side slope. See Channel geometry and rating curve.
18	`bed_load`	-	-	Read as the bankfull flow multiplier, the ratio of the flow at which overbank flooding starts to the bankfull flow. Replaced by 1.5 at the start of every day's sediment calculation, so the value in the file has no effect.
19	`fps`	m/m	0.00001 to `slp`	Lateral slope of the floodplain away from the channel (rise over run). Values above `slp` are lowered to `slp`; values of 10^{-6} or less are set to 10^{-5}.
20	`fpn`	-	0.025 to 0.16	Manning's n of the floodplain. Chow (1959) gives 0.025 to 0.05 for pasture and cultivated land, 0.035 to 0.16 for brush and 0.08 to 0.2 for trees.
21	`n_conc`	mg/kg	-	Nitrogen content of eroded bank sediment, added to the flow as organic N.

#	Column	Units	Typical range	Description
22	`p_conc`	mg/kg	-	Phosphorus content of eroded bank sediment.
23	`p_bio`	fraction	0 to 1	Fraction of the bank phosphorus that enters the flow as soluble P. The rest enters as sediment-bound P.

There are no built-in values for this file. A channel whose `cha_hyd` name is not found uses row 1. Without `hyd-sed-lte.cha` the chandeg channels have no parameters, so the file is required whenever `chandeg.con` lists channels.

sed_nut.cha

An optional table with extra coefficients for the chandeg sediment and nutrient calculations. It is read only when a file named `sed_nut.cha` is present. Rows are matched by the same `cha_hyd` name as `hyd-sed-lte.cha`. Two header lines are skipped, then 13 values per line:

#	Column	Units	Built-in value	Description
1	`name`	-	-	Name, matched against `cha_hyd` in `channel-lte.cha`.
2	`order`	-	-	Text field, read and not used.
3	`pk_rto`	ratio	1.0	Multiplier on the peak-to-mean daily flow ratio. See Peak flow.
4	`fp_inun_days`	days	5.0	Days the floodplain stays inundated after a flood. Replaced by 1.0 at the start of every day's sediment calculation, so neither the file value nor the built-in value has an effect.
5	`n_setl`	ratio	0.5	Ratio of N settling to sediment settling. Not used.
6	`p_setl`	ratio	0.5	Ratio of P settling to sediment settling. Not used.
7	`n_sol_part`	1/h	0.01	Rate of nitrate conversion to organic N, used only with the simplified transformations. See Nutrients.
8	`p_sol_part`	1/h	0.01	Rate of soluble P conversion to sediment-bound P, used only with the simplified transformations.
9	`n_dep_enr`	ratio	0.5	Enrichment ratio for organic N deposited with sediment on the floodplain and in the channel.
10	`p_dep_enr`	ratio	0.5	Enrichment ratio for sediment-bound P deposited with sediment.
11	`arc_len_fr`	fraction	1.2	Fraction of the meander arc length where banks erode. Read but not used in the erosion mass in 62.0.1.
12	`bed_exp`	-	1.5	Exponent of the bed erosion equation. Replaced by 1.0 at the start of every day's sediment calculation.
13	`wash_bed_fr`	fraction	0.1	Fraction of bank erosion deposited in the channel bed. See Channel deposition.

Without `sed_nut.cha`, every channel uses the built-in values. With the file present, a channel whose name is not found uses row 1.

nutrients.cha

Free format. Two header lines are skipped, then one parameter set per line with 39 values in this order. The reader does not match columns by header name. Every value of zero or less in the columns that have a value in the "Replacement" column is replaced by that value.

All rates are for 20 °C and are adjusted to the water temperature in the calculation. The water temperature used here is $5 + 0.75\,\bar{T}$, where $\bar{T}$ is the day's mean air temperature at the channel's weather station, with a minimum of 0.1 °C. The stream temperature from the heat balance (see `temperature.cha`) sets only the outflow temperature.

#	Column	Units	Replacement	Typical range	Description
1	`name`	-	-	-	Name, matched against `cha_nut` in `channel-lte.cha`.
2	`plt_n`	ppm	-	0 to 100	Organic N concentration. Not used.
3	`ptl_p`	ppm	-	0 to 100	Organic P concentration, called `plt_p` in earlier documentation. Not used.
4	`alg_stl`	m/day	1.0	0.15 to 1.82	Algal settling rate.
5	`ben_disp`	mg P/m²/day	0.05	0.001 to 0.1	Benthic source rate for dissolved P.
6	`ben_nh3n`	mg N/m²/day	0.5	0 to 1	Benthic source rate for ammonium N.
7	`ptln_stl`	1/day	0.05	0.001 to 0.1	Organic N settling rate.
8	`ptlp_stl`	1/day	0.05	0.001 to 0.1	Organic P settling rate.
9	`cst_stl`	1/day	2.5	0.01 to 10	Settling rate of an arbitrary non-conservative constituent. Not used.
10	`ben_cst`	mg/m²/day	2.5	0.01 to 10	Benthic source rate of an arbitrary non-conservative constituent. Not used.
11	`cbn_bod_co`	1/day	1.71	0.02 to 3.4	CBOD deoxygenation rate.
12	`air_rt`	1/day	1.0	0 to 100	Reaeration rate (Fickian diffusion).
13	`cbn_bod_stl`	1/day	none	-0.36 to 0.36	Rate of CBOD loss by settling. A value of zero or less is kept.
14	`ben_bod`	mg O_2/m²/day	2.0	0 to 100	Sediment oxygen demand.
15	`bact_die`	1/day	2.0	0.05 to 4	Coliform die-off rate. Not used; pathogen die-off comes from the pathogen database.
16	`cst_decay`	1/day	1.71	0 to 10	Decay rate of an arbitrary non-conservative constituent. Not used.
17	`nh3n_no2n`	1/day	0.55	0.1 to 1	Rate of biological oxidation of NH_3 to NO_2.
18	`no2n_no3n`	1/day	1.1	0.2 to 2	Rate of biological oxidation of NO_2 to NO_3.
19	`ptln_nh3n`	1/day	0.21	0.2 to 0.4	Rate of hydrolysis of organic N to ammonium.
20	`ptlp_solp`	1/day	0.35	0.01 to 0.7	Rate of mineralisation of organic P to dissolved P.
21	`q2e_lt`	-	2	1 to 4	QUAL2E light averaging option. Not used.
22	`q2e_alg`	-	2	1 to 3	Algal growth limitation: `1` multiplicative ($\mu_{max} f_L f_N f_P$), `2` limiting nutrient ($\mu_{max} f_L \min(f_N, f_P)$), `3` harmonic mean of f_N and f_P. Integer.
23	`chla_alg`	µg chl-a/mg algae	50	10 to 100	Ratio of chlorophyll-a to algal biomass.
24	`alg_n`	mg N/mg algae	0.08	0.07 to 0.09	Fraction of algal biomass that is N.
25	`alg_p`	mg P/mg algae	0.015	0.01 to 0.02	Fraction of algal biomass that is P.

#	Column	Units	Replacement	Typical range	Description
26	`alg_o2_prod`	mg O_2/mg algae	1.6	1.4 to 1.8	O_2 production per unit of algal photosynthesis.
27	`alg_o2_resp`	mg O_2/mg algae	2.0	1.6 to 2.3	O_2 uptake per unit of algal respiration.
28	`o2_nh3n`	mg O_2/mg N	3.5	3 to 4	O_2 uptake per unit of NH_3 oxidised.
29	`o2_no2n`	mg O_2/mg N	1.07	1 to 1.4	O_2 uptake per unit of NO_2 oxidised.
30	`alg_grow`	1/day	2.0	1 to 3	Maximum specific algal growth rate.
31	`alg_resp`	1/day	2.5	0.05 to 5	Algal respiration rate.
32	`slr_act`	fraction	0.3	0 to 1	Fraction of solar radiation that is photosynthetically active.
33	`lt_co`	kJ/m²/min	0.75	-	Half-saturation coefficient for light. Multiplied by 0.06 on reading, which converts kJ/m²/min to MJ/m²/h.
34	`const_n`	mg N/L	0.02	0.01 to 0.3	Michaelis-Menten half-saturation constant for N.
35	`const_p`	mg P/L	0.025	0.001 to 0.05	Michaelis-Menten half-saturation constant for P.
36	`lt_nonalg`	1/m	1.0	0 to 10	Non-algal part of the light extinction coefficient.
37	`alg_shd_l`	1/(m · µg chl-a/L)	0.03	0.006 to 0.065	Linear algal self-shading coefficient.
38	`alg_shd_nl`	(1/m)(µg chl-a/L)^-2/3	0.054	0 to 1	Non-linear algal self-shading coefficient.
39	`nh3_pref`	-	0.5	0 to 1	Algal preference for ammonium over nitrate.

Units of the half-saturation coefficient for light differ between sources: the old GitBook pages give `lt_co` in MJ/m²/h with a range of 0.223 to 1.135, while the 62.0.1 reader treats the value as kJ/m²/min and multiplies it by 0.06. A file value of 0.75 is used as 0.045 MJ/m²/h.

In sub-daily runs the reader divides the rates in columns 4 to 8, 11 to 14, 17 to 20, 30 and 31 by the number of time steps per day, while the water quality calculation still runs once per day. Sub-daily runs therefore use smaller effective rates than daily runs with the same file.

When `nutrients.cha` is missing, the chandeg channels have no water quality parameters, so keep the file in any model with channels.

Channel geometry and rating curve

At the start of the run each channel gets a trapezoidal cross-section and a rating curve at four depths: 0.1 and 1.0 times the bankfull depth `dp`, and 1.2 and 2.0 times `dp` with water on the floodplain. Flow, velocity, area, volume and travel time at any other flow are interpolated between these points.

With top width W (`wd`), bankfull depth D (`dp`), side slope z (`side_slp`), length L (`len`, km), slope S (`slp`) and roughness n (`mann`), the bottom width is

$$b = W - 2\,D\,z$$

If this is zero or negative, b is set to $W/2$ and z is reset to $W/(4D)$. For a flow depth d up to bankfull,

$$P = b + 2\,d\,\sqrt{1 + 1/z^2}, \qquad A = b\,d + d/z, \qquad R = A/P$$

$$v = \frac{R^{2/3}\,S^{1/2}}{n + 0.001}, \qquad Q = A\,v, \qquad TT = \frac{L}{3.6\,v}$$

where P is the wetted perimeter (m), A the flow area (m²), v the velocity (m/s), Q the flow (m³/s) and TT the travel time (h). The channel volume is $1000\,A\,L$ (m³).

The bottom width treats `side_slp` as horizontal distance per unit rise, while the perimeter and area treat it as rise per unit horizontal distance. The side term of the area, d/z, is the area of two side triangles with slope z (rise over run) only at $d = 1$ m. These are the expressions SWAT+ 62.0.1 uses.

Above bankfull, with h the depth above the bank (0.2 D and 1.0 D) and s_{fp} the floodplain slope `fps`,

$$P = P_{bf} + 2\,h\,\sqrt{1 + 1/s_{fp}^2}, \qquad A = A_{bf} + b\,h + h^2/s_{fp}, \qquad W_{fp} = W + 2\,h/s_{fp}$$

The floodplain volume is $1000\,W_{fp}\,h\,L$, and flow and velocity use `fpn` in place of `mann`, with the channel slope `slp`.

A smaller `fps` gives a wider, flatter floodplain that stores more water at a given depth.

Flow routing

Each day the inflow from upstream objects is first increased by any water allocation transfer into the channel, aquifer flow from `aqu_cha.lin`, gwflow exchange when gwflow is active, and, on days with inflow, precipitation on the channel surface ($P\,L\,W$ m³, with P in mm, L in km and W in m). Water is then routed with one of two methods, chosen by `rte_cha` in `codes.bsn`.

Variable storage method

With `rte_cha` = 0, the outflow in each time step is a fraction of the stored water:

$$O = c\,V_{stor}, \qquad c = \min\left(1,\ \frac{2\,s_c\,\Delta t}{2\,TT + \Delta t}\right)$$

where s_c is `scoef` in `parameters.bsn`, Δt the time step (h) and TT the travel time (h) at the previous step's outflow rate.

Muskingum method

With `rte_cha` = 1, outflow follows the Muskingum equation

$$O_2 = C_1 I_2 + C_2 I_1 + C_3 O_1$$

The storage constant is a weighted mean of the travel times at bankfull and 0.1 bankfull depth, each computed with the kinematic wave celerity $5v/3$:

$$K = \frac{\text{msk_co1}\,K_{bf} + \text{msk_co2}\,K_{0.1}}{\text{msk_co1} + \text{msk_co2}}$$

With X = `msk_x` (all three in `parameters.bsn`), the coefficients are

$$C_1 = \frac{\Delta t - 2KX}{2K(1-X) + \Delta t}, \quad C_2 = \frac{\Delta t + 2KX}{2K(1-X) + \Delta t}, \quad C_3 = \frac{2K(1-X) - \Delta t}{2K(1-X) + \Delta t}$$

C_1 is kept at zero or above, and the three are scaled to sum to 1. When $\Delta t > 2K(1-X)$ the coefficients are computed for a shorter sub-step that meets this limit. Sub-steps are used only in sub-daily runs; a daily run routes once per day with the sub-step coefficients.

Stored water above the bankfull channel volume is counted as floodplain storage.

Water losses

After routing, transmission losses and evaporation are taken from the water left in the channel:

$$TL = 24\,k\,L\,W \quad (\text{m}^3/\text{day}), \qquad E = \text{evrch}\ PET\ L\,W$$

where k is `k` (mm/h), L is in km, W in m, PET is the potential evapotranspiration of the weather station (mm), and `evrch` is the reach evaporation factor (`evap_adj` in `parameters.bsn`). Both use the bankfull top width over the full channel length, and neither can exceed the stored volume. With gwflow active, gwflow also calculates an exchange with the channel; the transmission loss above is still taken from the channel, but the channel output reports it as zero seepage.

Sediment

Sediment calculations run once per day for channels with inflow. In 62.0.1 four inputs are replaced at the start of each day's calculation: the bankfull flow multiplier by 1.5, the floodplain inundation days by 1.0, the bed `d50` by 1.0 mm and the bed erosion exponent by 1.0. The file values of `bed_load`, `d50`, `fp_inun_days` and `bed_exp`, and any calibration changes to them, therefore have no effect.

Peak flow

The day's peak flow is

$$q_{pk} = \text{pk_rto}\,(1 + 2.66\,A_{km^2}^{-0.3})\,\frac{Q_{day}}{86400}$$

(Fuller's equation), where A_{km^2} is the `area` of the channel in `chandeg.con` converted to km² and Q_{day} is the day's inflow (m³). The peak sets the velocity v from the rating curve, which is used for erosion and the travel time.

Floodplain deposition

When q_{pk} exceeds 1.5 times the bankfull flow, the floodplain traps a fraction of the incoming sediment (Dynamic SedNet, 2017):

$$TE = \min\!\left(1,\ N_{inun}\,\frac{q_{fp}}{\bar{q}}\,\left(1 - e^{-0.0001\,A_{fp}/q_{fp}}\right)\right)$$

with $N_{inun} = 1$ day, q_{fp} the rating curve flow at the peak, $\bar{q}$ the mean daily flow (m³/s) and $A_{fp} = 5\,W\,L$ the floodplain area (m², L in m). Organic N and sediment-bound P are deposited with TE times `n_dep_enr` and `p_dep_enr`. Overbank water then fills wetlands of HRUs linked to the channel's floodplain up to their emergency volume.

Bank erosion

Banks erode when the reach velocity exceeds a critical velocity. The cohesion from the clay content C (`clay`, %) is

$$c = \max(1000,\ -87.1 + 42.82\,C - 0.261\,C^2 + 0.029\,C^3)$$

and the critical velocity is

$$v_{cr} = \text{eq_slp}\ \log_{10}(2200\,D)\,\sqrt{0.0004\,(f_{bd} + 0.021\,c + 4500\,\text{cov}\,e^{-5D})}$$

where $f_{bd} = \max(0.001,\ 39.24\,\rho_b - 1000)$ with ρ_b the bulk density `dry_bd` (t/m³). For bulk densities below 25.5 t/m³, which covers all real sediments, f_{bd} stays at 0.001, so `dry_bd` does not change v_{cr}.

The velocity at the bends uses the radius of curvature from the sinuosity σ (column `wd_rto`):

$$r_c = \frac{12\,W\,\sigma^{1.5}}{13\,\sqrt{\sigma - 1}}, \qquad v_{rch} = 0.33\,v\,(1 + 1/r_c) + 0.66\,v$$

If $v_{rch} > v_{cr}$, the bank retreat for the day (m) is

$$e_{bank} = \left(\frac{1}{v_c} + \frac{1}{0.0002\,W}\right)^{-1}, \qquad v_c = \frac{0.0001\,A_{km^2}^{-0.0858}\,W}{1 + e^{-4\,(v_{rch}/v_{cr} - 1)}}$$

and the eroded mass is $1000\,e_{bank}\,D\,(0.25\,L)\,\rho_b$ tonnes, with L in km. The bank sediment brings organic N at `n_conc` and P at `p_conc`; the fraction `p_bio` of that P is soluble.

Channel deposition

A fraction `wash_bed_fr` of the bank erosion is deposited in the channel bed and removed from the flow, with organic N and sediment-bound P scaled by `n_dep_enr` and `p_dep_enr`.

Bed erosion

For channels steeper than 10^{-6}, bed erosion occurs when $v > 0.293\sqrt{d_{50}}$, which is 0.293 m/s with $d_{50} = 1$ mm:

$$e_{bed} = 0.0001\left(\frac{v}{0.293}\right)\ \text{m}, \qquad M_{bed} = 1000\, e_{bed}\, W\, L\, \rho_b\ \text{t}$$

In 62.0.1, e_{bed} and M_{bed} appear in the channel morphology and sediment budget outputs, but the eroded bed sediment is not added to the flow.

Nutrients

For channels with inflow, the day's water quality follows the QUAL2E kinetics with the rates of `nutrients.cha`, applied over the travel time (at most one day). This covers algae and chlorophyll-a, CBOD, dissolved oxygen, the nitrogen chain from organic N through ammonium and nitrite to nitrate, and organic and soluble P. See Channels in the model theory.

When column 24 of `codes.bsn` (headed `i_fpwet` by the Editor) is 1, a simplified transformation then replaces the organic N, nitrate, sediment-bound P and soluble P of the outflow:

$$\Delta N_{org} = \min\left(NO_3,\ \left(1 - e^{-k_N\, TT}\right) N_{org}\right)$$

with $k_N =$ `n_sol_part` and TT the travel time (h). ΔN_{org} is added to organic N and taken from nitrate. Phosphorus follows the same form with `p_sol_part`, moving soluble P to sediment-bound P.

Pesticides

In the channel, pesticide sorption uses $K_d = K_{oc}\,\text{carbon}/100$, with K_{oc} from `pesticide.pes` and `carbon` from `hyd-sed-lte.cha`. The mixing velocity between water and bed uses the bed porosity $1 - \rho_b/2.65$. Pesticides and pathogens are routed only on days with inflow.

Calibration and decision tables

Parameters can be changed during calibration through `calibration.cal` (see parameter change files), using these names:

- `hyd-sed-lte.cha`: `chw` (`wd`), `chd` (`dp`), `chs` (`slp`), `chl` (`len`), `chn` (`mann`), `chk` (`k`), `bank_exp` (`erod_fact`), `cov` (`cov_fact`), `vcr_coef` (`eq_slp`), `d50`, `ch_clay` (`clay`), `carbon`, `ch_bd` (`dry_bd`), `chss` (`side_slp`), `bankfull_flo` (`bed_load`), `fps`, `fpn`, `ch_n_conc`, `ch_p_conc`, `ch_p_bio`.
- `sed_nut.cha`: `pk_rto`, `fp_inun_days`, `n_setl`, `p_setl`, `n_sol_part`, `p_sol_part`, `n_dep_enr`, `p_dep_enr`, `arc_len_fr`, `bed_exp`, `wash_bed_fr`.
- `nutrients.cha`: `rs1` to `rs7`, `rk1` to `rk4`, `rk6`, `bc1` to `bc4`, `mumax`. The parameter name `rk5` changes `ben_disp` (the same column as `rs2`); no calibration name changes `bact_die`.

Calibration changes are applied after the rating curve and Muskingum coefficients are set, and these are not recalculated. Changes to `chw`, `chd`, `chs`, `chn`, `chss`, `fps` and `fpn` therefore do not change the rating curve or the routing coefficients; they change only the other calculations that use the values directly (transmission losses, evaporation, erosion). Changes to `bankfull_flo`, `fp_inun_days`, `d50` and `bed_exp` are overwritten each day (see Sediment). The `nutrients.cha` changes are applied to the row whose position equals the channel number.

The decision table action `chan_change` sets a channel's cover factor to the action's first constant and its stream order to the second. The condition `ch_order` tests a channel's stream order. See decision tables.

Example

`refdata/Osu_1hru` in the SWAT+ repository, first rows of each file:

```
channel-lte.cha: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
      id  name                        cha_ini           cha_hyd           cha_sed           cha_nut
       1  cha01                      initcha1          hydcha01              null           nutcha1
       2  cha03                      initcha1          hydcha03              null           nutcha1
```

```
hyd-sed-lte.cha: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name        order       wd       dp      slp      len     mann        k  erod_fact  cov_fact   wd_rto   eq
hydcha01        4 27.17726  0.99164  0.00146  3.42760  0.05000  5.00000    0.01000   0.00500 27.40638  0.0
```

(columns narrowed for display)

Channel `cha01` is a fourth-order channel 27.2 m wide, 0.99 m deep and 3.43 km long, with Manning's n of 0.05 and bed conductivity of 5 mm/h. Some values in this file are read differently from what the Editor header suggests:

- `wd_rto` = 27.4 is the width-to-depth ratio, but SWAT+ reads it as a sinuosity of 27.4. The bends then have a large radius of curvature, and the bend velocity is close to the mean velocity.
- `eq_slp` = 0.001 multiplies the critical velocity for bank erosion by 0.001, so bank erosion occurs on almost every day with flow.
- `d50` = 12 and `bed_load` = 0.5 are replaced by 1.0 mm and 1.5 during the run.
- `fps` = 0.00001 gives a very flat, wide floodplain.

`nutrients.cha` in the same dataset has one set, `nutcha1`, with `air_rt` = 50 and `cbn_bod_stl` = 0.36. The other values are the replacement values in the table above, or zero for the two unused concentrations.

Related files

- `chandeg.con`: lists the channels and points to `channel-lte.cha` rows in its `lcha` column.
- `initial.cha`: initial water, sediment and constituents in each channel, through `om_water.ini`, `pest_water.ini`, `salt_channel.ini` and `cs_channel.ini`.
- `temperature.cha`: stream temperature parameters.
- `codes.bsn` (`rte_cha`, column 24) and `parameters.bsn` (`msk_co1`, `msk_co2`, `msk_x`, `evap_adj`, `scoef`): routing switches and coefficients.
- `file.cio`: the `channel` line names the files.
- Outputs: channel, sediment and nutrient files (`channel_sd`, `channel_sdmorph`, `sd_chanbud`, `ch_sedbud`, `chanbud_order`).
- Channels and erosion and sediment in the model theory.

Sources

- File name slots: `type input_cha` in `input_file_module.f90`, read in `readcio_read.f90`; read order in `proc_cha.f90`
- Readers: `sd_channel_read.f90` (`channel-lte.cha`), `sd_hydsed_read.f90` (`hyd-sed-lte.cha`, `sed_nut.cha`), `ch_read_nut.f90` (`nutrients.cha`)
- Type definitions: `swatdeg_datafiles`, `swatdeg_hydsed_data`, `swatdeg_sednut_data` and `swatdeg_channel_dynamic` in `sd_channel_module.f90`; `channel_nut_data` in `channel_data_module.f90`
- Initialisation, limits, rating curve and Muskingum coefficients: `sd_hydsed_init.f90`, `sd_rating_curve.f90`, `qman.f90`
- Daily channel calculation: `sd_channel_control3.f90`, called from `command.f90`; sediment in `sd_channel_sediment3.f90`; routing and losses in `ch_rtmusk.f90`; water quality in `ch_watqual4.f90`; pesticides in `ch_rtpest.f90`
- `chandeg.con` pointer column: `hyd_read_connect.f90`
- Calibration: `cal_parm_select.f90`; decision tables: `actions.f90`, `conditions.f90`

`channel-lte.cha`:

Column	Fortran variable
`id`	index of `sd_dat`
`name`	`sd_dat%name`
`cha_ini`	`sd_dat%initc` -> `sd_dat%init`
`cha_hyd`	`sd_dat%hydc` -> `sd_dat%hyd`, `sd_dat%sednut`

Column	Fortran variable
cha_sed	sd_dat%sedc (unused)
cha_nut	sd_dat%nutc -> sd_dat%nut

hyd-sed-lte.cha (copied to sd_ch in sd_hydsed_init):

Column	Fortran variable
order	sd_chd%order
wd	sd_chd%chw
dp	sd_chd%chd
slp	sd_chd%chs
len	sd_chd%chl
mann	sd_chd%chn
k	sd_chd%chk
erod_fact	sd_chd%bank_exp
cov_fact	sd_chd%cov
wd_rto	sd_chd%sinu
eq_slp	sd_chd%vcr_coef
d50	sd_chd%d50
clay	sd_chd%ch_clay
carbon	sd_chd%carbon
dry_bd	sd_chd%ch_bd
side_slp	sd_chd%chss
bed_load	sd_chd%bankfull_flo
fps, fpn, n_conc, p_conc, p_bio	sd_chd% same names

sed_nut.cha columns map to sd_chd1% components of the same names.

nutrients.cha:

Columns	Fortran variables (ch_nut%)
plt_n, ptl_p	onco, opco
alg_stl to ben_cst	rs1 to rs7
cbn_bod_co, air_rt, cbn_bod_stl, ben_bod, bact_die, cst_decay	rk1 to rk6
nh3n_no2n to ptlp_solp	bc1 to bc4
q2e_lt, q2e_alg	lao, igropt
chla_alg to o2_no2n	ai0 to ai6
alg_grow, alg_resp, slr_act	mumax, rhoq, tfact
lt_co, const_n, const_p	k_l, k_n, k_p
lt_nonalg, alg_shd_l, alg_shd_nl, nh3_pref	lambda0, lambda1, lambda2, p_n

Purpose

aquifer.aqu contains the properties of the one-dimensional aquifers: initial water table, specific yield, baseflow recession, deep seepage, revap and groundwater nitrate and phosphorus. Each aquifer object in aquifer.con selects a row by number in its aqu column. The aquifer line of file.cio names two files, initial.aqu and aquifer.aqu (in that order); either can be null.

A model has one aquifer object per row of aquifer.con. Usually there is a shallow aquifer for each land area and one or more deep aquifers that receive the seepage of the shallow ones. Models that simulate groundwater with the gwflow grid model use gwflow.con instead of these aquifers.

initial.aqu lists named sets of initial-condition files. In 62.0.1 its rows are read but not used; initial pesticides, pathogens, salts and constituents of aquifers come from initial.aqu_cs, described below.

aquifer.aqu

Free format, whitespace-separated. Line 1 is a title and line 2 the column header; both are skipped. Each later line is one aquifer property set with 18 values in this order. The reader does not match columns by header name.

#	Column	Units	Built-in value	Typical range	Description
1	id	-	-	-	Number of the property set. The aqu column of aquifer.con refers to the row by this number.
2	name	-	-	-	Name of the aquifer. Not used by the model.
3	init	-	-	-	Name of an initial-condition set, matched against initial.aqu_cs. See initial.aqu_cs.
4	gw_flo	mm	0.05	0 to 2	Return flow on the day before the simulation starts. It starts the recession, and sets the water volume for the initial nitrate mass.
5	dep_bot	m	0	0 to 10	Depth from the mid-slope surface to the bottom of the aquifer. Must be larger than dep_wt.
6	dep_wt	m	0	0 to 10	Initial depth from the mid-slope surface to the water table.
7	no3_n	mg N/L	0	0 to 1000	Initial nitrate-N concentration.
8	sol_p	mg P/L	0	0 to 1000	Mineral P concentration of the water entering the stream, constant through the run.
9	carbon	%	0.5	0 to 15	Organic carbon content. Reported in the aquifer output; not used in any calculation in 62.0.1.
10	flo_dist	m	50	0 to 1000	Average flow distance to the stream. Not used.
11	bf_max	mm	0	0 to 2	Return flow at which all channels linked to the aquifer receive groundwater. Used only when aquifer flow is distributed to channels with aqu_cha.lin.
12	alpha_bf	1/day	0	0 to 1	Baseflow recession constant.
13	revap	-	0	0 to 1	Revap coefficient: revap as a fraction of potential evapotranspiration.
14	rchg_dp	fraction	0	0 to 1	Fraction of the day's recharge that leaves the aquifer as deep seepage.
15	spec_yld	m^3/m^3	0	0 to 0.4	Specific yield: the volume of water drained by gravity per volume of aquifer. Must be above 0.
16	hl_no3n	days	30	0 to 200	Half-life of nitrate in the aquifer.
17	flo_min	m	0	0 to 10	Water table depth for return flow: the aquifer sends return flow only while the water table is at this depth below the surface or shallower.

#	Column	Units	Built-in value	Typical range	Description
18	`revap_min`	m	0	0 to 10	Water table depth for revap: revap occurs only while the water table is shallower than this depth.

The built-in values apply when aquifer.con points to a number between 1 and the largest `id` that has no row in the file. They are not filled in for missing columns.

Give every row an `id`, and keep the ids unique. The ids do not have to be consecutive.

Aquifer water balance

Each day, for each aquifer, in this order. Depths of water are in mm over the aquifer area (the `area` column of aquifer.con).

Storage and water table

At the start of the run the stored water is

$$S_0 = 1000\,(\text{dep_bot} - \text{dep_wt})\ \text{spec_yld}$$

Each day the recharge R, all water routed into the aquifer from HRUs, routing units and other aquifers, is added to storage, and withdrawals for water allocation are then taken. Irrigation drawn from the aquifer by decision tables is removed from storage when it is applied. The water table depth follows from the storage:

$$h = \max\left(0,\ \text{dep_bot} - \frac{S}{1000\ \text{spec_yld}}\right)$$

The aquifer calculation in 62.0.1 has no recharge delay: recharge is added on the day it arrives.

Return flow

When $h \le$ `flo_min`, return flow follows the baseflow recession

$$Q_d = Q_{d-1}\,e^{-\alpha} + R_d\,(1 - e^{-\alpha})$$

limited to between zero and the storage, where α is `alpha_bf`. When $h >$ `flo_min`, $Q_d = 0$, and the next day's recession starts from zero.

The recession constant is a direct index of how fast groundwater flow responds to recharge (Smedema and Rycroft, 1983). Values are about 0.1 to 0.3 for land with a slow response and 0.9 to 1.0 for land with a rapid response. It is best estimated from measured streamflow during periods without recharge. When the number of baseflow days BFD (the days for baseflow to decline through one log cycle) is known,

$$\alpha = \frac{1}{BFD}\ln 10 = \frac{2.3}{BFD}$$

Return flow leaves through the outflows of type `tot` in aquifer.con.

Deep seepage

$$\text{seep}_d = \text{rchg_dp}\ R_d$$

limited to the storage. Seepage leaves through the outflows of type `rhg` in aquifer.con, usually to a deep aquifer. Without such an outflow it leaves the model.

Revap

When $h <$ `revap_min`,

$$\text{revap}_d = \text{revap}\ PET_d$$

limited to the storage, where PET_d is the potential evapotranspiration at the aquifer's weather station (mm). Revap represents water that moves from the aquifer into the unsaturated zone above it when that zone is dry, and uptake by deep-rooted plants. It is removed from the aquifer and not added to the soil. As `revap` approaches 0, the transfer is restricted; as it approaches 1, revap approaches the potential evapotranspiration.

Nitrate and phosphorus

Nitrate recharge is added to the stored nitrate N (kg/ha). The concentration is $c = N/S$, and the nitrate leaving with return flow and seepage is $c\,Q$ and $c\,\text{seep}$. Each day the remaining nitrate decays by

$$N \leftarrow N\ \exp\left(\frac{-0.693}{\text{hl_no3n} + 0.1}\right)$$

The decay represents uptake by bacteria and chemical conversion where the aquifer is depleted of oxygen; it is the net loss from all processes.

The initial nitrate mass is $10\,\text{gw_flo}\,A\,\text{no3_n}/1000$, the concentration times the volume of one day's initial return flow (A in ha). This is a mass in kg for the whole aquifer, but the daily calculation treats the store as kg/ha, so the initial store scales with the aquifer area. It does not use the stored water S_0.

Soluble P in the return flow is sol_p times the day's recharge volume (m^3), divided by 1000, in kg. The load therefore follows the recharge. It appears on days with recharge even when there is no return flow, and is zero on days without recharge. The aquifer stores no phosphorus between days.

Pesticides, salts and constituents

Pesticides entering with recharge are added to the aquifer store and decay with the aquifer half-life of `pesticide.pes` (adjusted to temperature when the temperature decay option is on). The concentration is the stored mass divided by $f\,S + Q + \text{revap} + \text{seep}$, where f is the pesticide groundwater mixing factor of `parameters.bsn`. Return flow and seepage take pesticide out at that concentration, all of it dissolved. Revap takes none.

Salts and other constituents (see `salt` and `cs`) are mixed in the stored water and leave with return flow and seepage at the stored concentration.

Aquifer flow to channels with aqu_cha.lin

When `aqu_cha.lin` links aquifers to channels (see `file.cio`), the return flow is spread over the linked channels as each channel is routed: on the same day when the aquifer comes before the channel in the routing order, otherwise on the next day. The length of channel that receives flow is the total linked channel length times Q/bf_max; when $Q \geq$ `bf_max` all linked channels receive flow, and when it is lower only part of the channel length does, each channel in proportion to its length. Set `bf_max` to a value above zero whenever `aqu_cha.lin` is used.

Calibration and decision tables

These parameters can be changed during calibration through `calibration.cal` (see parameter change files): `flo_init_mm` (`gw_flo`), `dep_bot`, `dep_wt_init` (`dep_wt`), `no3_init` (`no3_n`), `minp_init` (`sol_p`), `cbn_init` (`carbon`), `flo_dist`, `bf_max`, `alpha` (`alpha_bf`), `revap_co` (`revap`), `deep_seep` (`rchg_dp`), `sp_yld` (`spec_yld`), `hlife_n` (`hl_no3n`), `flo_min` and `revap_min`.

Calibration changes are applied after the aquifers are initialised. A change to `sp_yld` recalculates the storage from the current water table, and a change to `no3_init` recalculates the nitrate mass. Changes to `flo_init_mm` and `dep_wt_init` come too late to change the initial state and have no effect. A change to `dep_bot` does not change the stored water; it moves the water table that is calculated from it.

The decision table condition `aqu_dep` tests the water table depth of an aquifer (m). See decision tables.

initial.aqu

Named sets of initial-condition files, in the same layout as `initial.cha`. Two header lines are skipped, then one set per line. The Editor adds a trailing `description` value, which the reader ignores.

#	Column	Description
1	`name`	Name of the set.
2	`org_min`	Name of an organic-mineral initial record.
3	`pest`	Name of a pesticide initial record.
4	`path`	Name of a pathogen initial record.
5	`hmet`	Name of a heavy metal initial record.
6	`salt`	Name of a salt initial record.

In 62.0.1 the rows are read and nothing is done with them: the `init` column of `aquifer.aqu` is not matched against this file, and the aquifer's initial water and nitrate come from `aquifer.aqu`.

initial.aqu_cs

An optional file with a fixed name, read when present. Two header lines are skipped, then one row per line with six names: `name`, `pest`, `path`, `hmet`, `salt` and `cs`.

- `pest` names a row in `pest_water.ini`. The initial pesticide in the aquifer (kg/ha) is the water concentration (mg/kg) times an assumed bulk density of 2.0 t/m³ times `dep_bot` times 10.
- `path` names a row in `path_water.ini`.
- `hmet` is read but not used.
- `salt` names a row in `salt_aqu.ini`: initial salt ion concentrations (g/m³) and salt mineral fractions.
- `cs` names a row in `cs_aqu.ini`: initial constituent concentrations (g/m³) and sorbed concentrations (mg/kg).

Pesticide and pathogen rows are selected by matching `name` with the `init` column of `aquifer.aqu`. Salt and constituent rows are selected by position instead: aquifer 3 (the third row of `aquifer.con`) uses row 3 of `initial.aqu_cs`, whatever its name. Salt and constituent masses are the concentrations times the initial stored water.

Example

`refdata/Osu_1hru` in the SWAT+ repository:

```
aquifer.aqu: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
      id  name                          init        gw_flo       dep_bot        dep_wt         no3_n
       1  aqu011                    initaqu1       0.05000      10.00000       6.00000       0.00000
       2  aqu012                    initaqu1       0.05000      10.00000       6.00000       0.00000
      21  aqu_deep010               initaqu1       0.00000      10.00000      20.00000       0.00000
```

(three of the 21 rows)

The shallow aquifers start with the water table 6 m below a 10 m deep aquifer, so they contain $1000 \times (10-6) \times 0.05 = 200$ mm of water. Return flow and revap both need the water table within 5 m of the surface, so neither occurs until recharge raises it by 1 m (50 mm of storage). The recession constant of 0.95 1/day gives a fast response. One percent of recharge goes to deep seepage. `hl_no3n` is 0, which gives a daily nitrate decay factor of $e^{-6.93}$, so nearly all nitrate entering the aquifer is lost on the day it arrives.

The deep aquifer `aqu_deep010` has `dep_wt` = 20 m, below its `dep_bot` of 10 m, so its initial storage is negative ($1000 \times (10 - 20) \times 0.03 = -300$ mm) and the water table starts 20 m down. It is recharged by the seepage the shallow aquifers send to it. With `flo_min` = 0 it sends return flow only when the water table reaches the surface, and with `revap_min` = 0 it has no revap.

```
initial.aqu: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                      org_min             pest             path             hmet             salt
initaqu1                  no_init             null             null             null             null
```

Related files

- aquifer.con: the aquifer objects, their areas, the `aqu` pointer to `aquifer.aqu` and the `tot` and `rhg` outflows.
- file.cio: the aquifer line, and aqu_cha.lin on the link line.
- pest_water.ini, path_water.ini, salt_aqu.ini, cs_aqu.ini: initial constituents referenced from `initial.aqu_cs`.
- gwflow: the grid groundwater model used instead of these aquifers.
- Outputs: aquifer output files (`aquifer_day` and others).
- Aquifers in the model theory.

Sources

- File name slots: `type input_aqu` in input_file_module.f90, read in readcio_read.f90; read order in proc_aqu.f90
- Readers: aqu_read.f90 (`aquifer.aqu`), aqu_read_init.f90 (`initial.aqu`), aqu_read_init_cs.f90 (`initial.aqu_cs`)
- Type definitions: `aquifer_database`, `aquifer_data_parameters`, `aquifer_dynamic`, `aquifer_init_data_char` and `aquifer_init_data_char_cs` in aquifer_module.f90
- Initialisation: aqu_initial.f90
- Daily calculation: aqu_1d_control.f90, called from command.f90
- `aquifer.con` pointer and outflow types: hyd_read_connect.f90, hyd_connect.f90
- Calibration: cal_parm_select.f90; decision tables: conditions.f90, actions.f90

Column	Fortran variable
id	index of aqudb
name	aqudb%aqunm
init	aqudb%aqu_ini
gw_flo	aqudb%flo
dep_bot	aqudb%dep_bot
dep_wt	aqudb%dep_wt
no3_n	aqudb%no3
sol_p	aqudb%minp
carbon	aqudb%cbn
flo_dist	aqudb%flo_dist
bf_max	aqudb%bf_max
alpha_bf	aqudb%alpha
revap	aqudb%revap_co
rchg_dp	aqudb%seep
spec_yld	aqudb%spyld
hl_no3n	aqudb%hlife_n
flo_min	aqudb%flo_min
revap_min	aqudb%revap_min

`aqudb` is copied to `aqu_dat(iaq)` for each aquifer object in `aqu_initial`; `initial.aqu` rows go to `aqu_init_dat_c`, `initial.aqu_cs` rows to `aqu_init_dat_c_cs`.

Purpose

Reservoirs are impoundments on the channel network. They receive water, sediment and nutrients from upstream objects and release to downstream objects through a release rule. Ponds that are routed as separate objects use the same files. The reservoir group has six files, named on the **reservoir** line of file.cio in this order:

Slot	Default name	Content
1	initial.res	Named initial-condition sets for reservoirs and wetlands
2	reservoir.res	One row per reservoir, naming the records below
3	hydrology.res	Reservoir hydrology: spillway areas and volumes, seepage, evaporation, outflow smoothing
4	sediment.res	Sediment settling parameters, shared with wetlands
5	nutrients.res	Nutrient settling parameters, shared with wetlands
6	weir.res	Weir discharge parameters, used by **weir** release rules and by wetlands

Slots 7 and 8 of the same line name the wetland files wetland.wet and hydrology.wet. A slot set to **null**, or a file that does not exist, leaves that table empty.

reservoir.res refers to the other files by name. The release rule named in **reservoir.res** is a decision table in res_rel.dtl (see decision tables) or, for names starting with **ctbl_**, a conditions table in **res_conds.dat**.

The five databases are read in every run. **reservoir.res** is read, and the reservoirs are set up, only when object.cnt gives one or more reservoirs. **weir.res** is read before the decision tables, so that **weir** actions in res_rel.dtl can find their weir record.

All files are free format and whitespace-separated. Line 1 is a title and line 2 is the column header; both are skipped. The readers do not match columns by header name, so every row needs all of its values in the order given below. Columns after the last one listed (for example the **description** column the SWAT+ Editor writes) are ignored. Names must match, including case.

reservoir.res

One row per reservoir.

#	Column	Units	Description
1	id	-	Reservoir number. The row with **id** n is used for reservoir n, the nth reservoir in reservoir.con. Numbers must run from 1 to the number of rows.
2	name	-	Reservoir name. Not used by the model.
3	init	-	Name of a row in **initial.res**.
4	hyd	-	Name of a row in **hydrology.res**.
5	rel	-	Release rule. A decision table name in res_rel.dtl, or a name starting with **ctbl_** for a conditions table in **res_conds.dat**.
6	sed	-	Name of a row in **sediment.res**.
7	nut	-	Name of a row in **nutrients.res**.

A name that is not found is reported in **diagnostics.out** as **<name> not found (res-hyd)**, **(res-release)**, **(res-sed)** or **(res-nut)**, and the reservoir keeps zero or built-in values for that part. An unmatched **rel** gives an empty release table, so the reservoir releases no water through its rule. An unmatched **init** is not reported; the reservoir then starts from the first row of om_water.ini.

`hyd`, `sed` and `nut` are taken from the row whose `id` equals the reservoir number. `init` and `rel` are taken from the row named by the reservoir's parameter pointer in column 8 of `reservoir.con`. The SWAT+ Editor writes the same number in both places, so the two agree in Editor-built projects.

initial.res

Named initial-condition sets, shared by reservoirs (`init` in `reservoir.res`) and wetlands (`init` in `wetland.wet`).

#	Column	Units	Description
1	`name`	-	Name of the initial-condition set.
2	`org_min`	-	Name of a row in `om_water.ini`: initial water volume and concentrations.
3	`pest`	-	Name of a row in `pest_water.ini`: initial pesticides in water and benthic sediment.
4	`path`	-	Name of a pathogen initialization row.
5	`hmet`	-	Heavy-metal initialization. Read; not used.
6	`salt`	-	Salt initialization. Read; not used. Reservoir and wetland salts come from the salt input files.

`org_min`, `pest` and `path` names that are not found fall back to the first row of the referenced file, without a message.

The initial water volume is the `flo` value of the `om_water.ini` row, a fraction of the principal spillway volume. The concentrations in that row are converted to masses with this volume. A reservoir that is not yet operational at the start of the run (see Operation start) starts empty instead.

hydrology.res

One row per hydrology record.

#	Column	Units	Default	Description
1	`name`	-	-	Name of the hydrology record, matched against `hyd` in `reservoir.res`.
2	`yr_op`	year	-	Calendar year in which the reservoir becomes operational. Whole number.
3	`mon_op`	month	-	Month of `yr_op` in which the reservoir becomes operational. Whole number.
4	`area_ps`	ha	0.08 × `vol_ps`	Surface area at the principal spillway.
5	`vol_ps`	ha-m	0.9 × `vol_es`, or 60000	Volume of water needed to fill the reservoir to the principal spillway.
6	`area_es`	ha	1.5 × `area_ps`	Surface area at the emergency spillway.
7	`vol_es`	ha-m	1.11 × `vol_ps`	Volume of water needed to fill the reservoir to the emergency spillway.
8	`k`	mm/hr	-	Hydraulic conductivity of the reservoir bottom. Use 0 for no seepage. Typical range 0 to 1.
9	`evap_co`	-	0.6	Evaporation coefficient, a multiplier on potential evapotranspiration. Typical range 0 to 1.
10	`shp_co1`	-	-	Smoothing factor for rising outflow. See Outflow smoothing.
11	`shp_co2`	-	-	Smoothing factor for falling outflow. See Outflow smoothing.

The Default column gives the value the reader puts in when the column is zero or negative. The rules are applied in this order: if `vol_ps` is not positive, it becomes 0.9 × `vol_es` when `vol_es` is positive and 60000 ha-m otherwise; then `vol_es`, `area_ps`, `area_es` and `evap_co` are filled in as listed. The `area_ps` default corresponds to a mean depth of 12.5 m at the principal spillway.

Principal spillway values describe the normal storage of the reservoir, and emergency spillway values its maximum storage. The volumes are converted from ha-m to m³ (× 10000) when the reservoirs are set up.

A reservoir whose `hyd` name is not found has zero spillway areas and volumes.

Operation start

A reservoir is operational from month `mon_op` of calendar year `yr_op`. Before then it passes its inflow straight through to its downstream objects and keeps no storage. `yr_op` is compared with the calendar year, so a value of 0, or any year before the start of the simulation, makes the reservoir operational from the first day. Only reservoirs that are operational on the first day get the initial storage from `initial.res`; one that starts later begins empty.

Surface area

Each day the surface area A (ha) is calculated from the storage V (m³):

$$A = \beta_1 \, V^{\beta_2}$$

The coefficients are set once at the start of the run from the spillway values, with volumes in m³. When `area_es` is larger than `area_ps` and `vol_es` is larger than `vol_ps`,

$$\beta_2 = \frac{\log_{10} \text{area_es} - \log_{10} \text{area_ps}}{L}, \qquad \beta_1 = \left(\frac{\text{area_es}}{\text{vol_es}} \right)^{\beta_2}$$

where L is the whole-number part of $\log_{10} \text{vol_es} - \log_{10} \text{vol_ps}$. When L is 0 the divisor 0.001 is used instead. β_2 is limited to 0.9, and when the limit applies $\beta_1 = (\text{area_ps}/\text{vol_ps})^{0.9}$. In all other cases $\beta_2 = 0.9$ and $\beta_1 = (\text{area_ps}/\text{vol_ps})^{0.9}$.

In 62.0.1 L is 0 unless the emergency volume is at least ten times the principal volume. For almost every reservoir, then, $\beta_2 = 0.9$ and the relation reduces to

$$A = \left(\text{area_ps} \, \frac{V}{\text{vol_ps}} \right)^{0.9}$$

At the principal spillway this curve gives area_ps$^{0.9}$: 63 ha for an `area_ps` of 100 ha. The surface area sets precipitation on the reservoir, evaporation and seepage, so `area_ps` and `area_es` act on the water balance only through this relation.

Daily water balance

On each day of operation the reservoir:

1. adds the inflow from upstream objects to storage;
2. calculates the release from its release rule, using the storage after inflow, and smooths it (decision tables only);
3. settles sediment (decision tables only);
4. adds precipitation $P = 10 \, R \, A$ (m³, with R the day's precipitation in mm);
5. subtracts the release, limited to the water in storage;
6. subtracts evaporation $E = 10$ evap_co PET A (m³, with PET the day's potential evapotranspiration in mm), limited to storage;
7. subtracts seepage $S = 240 \, k \, A$ (m³, k in mm/hr), limited to storage;
8. updates the surface area from the new storage and calculates nutrient settling and the nutrient load in the release.

Precipitation and potential evapotranspiration come from the weather station assigned to the reservoir in `reservoir.con`. When gwflow is active, seepage is exchanged with the groundwater model and `k` is not used (see gwflow).

Outflow smoothing

For release by decision table, the release Q is smoothed against the previous day's release Q_{d-1} to soften jumps when the rule switches between alternatives:

$$Q_d = \alpha\, Q + (1-\alpha)\, Q_{d-1}, \qquad \alpha = \begin{cases} e^{-\text{shp_co1}} & Q > Q_{d-1} \\ e^{-\text{shp_co2}} & \text{otherwise} \end{cases}$$

With both factors at 0 there is no smoothing. Larger values give more weight to the previous day. The columns keep the names `shp_co1` and `shp_co2` from earlier versions, where they were surface-area shape coefficients. In 62.0.1 the shape coefficients are always calculated by the model as described above and cannot be entered.

Release rules

The `rel` column of `reservoir.res` selects how the reservoir releases water.

Decision tables

A decision table in `res_rel.dtl` evaluates its conditions each day (storage relative to the principal or emergency volume, month, inflow and others; see decision tables) and applies the `release` actions of the alternatives that are met. The `option` field of each action gives the method. Releases from several actions that apply on the same day are added.

`option`	Release per day
`rate`	`const` m³/s.
`rate_pct`	`const` percent of the principal volume.
`inflo_rate`	The larger of inflow plus `const2` m³/s and `const` m³/s.
`inflo_frac`	Inflow multiplied by `const`.
`ab_emer`	All storage above the emergency volume.
`days`	(storage minus a base volume) / `const`, where `const` is the number of drawdown days. The base volume is 0 when `fp` is `null`, `const2` × principal volume when `fp` is `pvol`, and `const2` × emergency volume when `fp` is `evol`.
`dyrt`	(storage minus base volume) / `const` plus `const2` percent of the principal volume. The base volume is 0, the principal volume or the emergency volume for `fp` = `null`, `pvol` or `evol`.
`dyrt1`	As `dyrt`, with the base volume taken from condition 5, 4 or 3 of the table for `fp` = `con1`, `con2` or `con3`.
`inflo_targ`	Inflow plus storage above a target, where the target is the principal volume multiplied by the limit constant of the condition numbered `const`.
`weir`	Weir equation with the `weir.res` row named in `fp`. See weir.res.
`meas`	Measured outflow from the recall record named in `fp` (daily, monthly or annual).
`natlake`	Natural lake outflow after Döll et al. (2003): $Q = k_r\, S_e\, \left(S_e/(V_p - S_0)\right)^{1.5}$ with dead storage S_0 = `const` × principal volume, S_e the storage above S_0, V_p the principal volume and k_r = `const2` (1/day).
`nonirr-h06`, `irr-h06`	Hanazaki et al. (2006) operating rules for non-irrigation and irrigation reservoirs.
`hydrop`	Hydropower rule after Arheimer et al. (2019).

For `days`, the base volume is `const2` times the spillway volume, so an action with `fp` set to `pvol` and `const2` set to 0 draws the reservoir down toward empty. Set `const2` to 1 to draw down to the principal volume.

The `nonirr-h06`, `irr-h06` and `hydrop` options base the release on a five-year record of monthly mean inflow (and, for `irr-h06`, irrigation demand) that each reservoir keeps during the run. In 62.0.1 this record is never filled and stays at zero, so `nonirr-h06` and `hydrop` release no water and `irr-h06` gives an undefined release (zero divided by zero).

In sub-daily runs the actions are applied once for each time step of the day. `days`, `dyrt` (its percentage term) and `meas` divide their daily amount by the number of steps; the other options add their full daily amount at every step. The `weir` option replaces the release with one time step's weir flow at each step.

For a reservoir, the `weir` option takes the weir crest height from the wetland data of the HRU with the same number as the reservoir. When that HRU has no wetland the crest height is zero, so the whole water depth (storage divided by surface area) acts as head. The `height` column of `weir.res` and the `weir_height` decision table action do not change the crest height that the reservoir weir release uses.

Conditions tables

A `rel` name starting with `ctbl_` is looked up in `res_conds.dat`, a file with a fixed name that is read when it exists in the run directory. The release is the value given by the matching condition of the table. Outflow smoothing and sediment settling are not applied to these reservoirs, and their release contains no sediment.

sediment.res

One row per sediment record. Reservoirs refer to it through `sed` in `reservoir.res`, wetlands through `sed` in wetland.wet.

#	Column	Units	Description
1	`name`	-	Name of the sediment record.
2	`sed_amt`	mg/L	Equilibrium (normal) suspended sediment concentration. Settling occurs only above this concentration. Typical range 1 to 5000.
3	`d50`	µm	Median particle size of suspended and benthic sediment. Sets the settling rate.
4	`carbon`	%	Organic carbon in suspended and benthic sediment. Used in pesticide partitioning.
5	`bd`	t/m³	Bulk density of benthic sediment. Used in pesticide partitioning between water and benthic sediment.
6	`sed_stl`	-	Sediment settling rate. Read but not used.
7	`stl_vel`	m/day	Sediment settling velocity. Read but not used.

Sediment settling

Each day, when the sediment concentration C (mg/L) in the reservoir is above the equilibrium concentration C_{eq} (`sed_amt`), the excess settles at a rate set by the median particle size:

$$C' = (C - C_{eq})\, e^{-0.184\, d_{50}} + C_{eq}$$

With `d50` at 10 µm, 16% of the excess concentration remains after one day; at 2 µm, 69% remains. The release takes sediment at the concentration C', and all silt, clay, sand and aggregate fractions in the reservoir settle.

When the concentration is at or below `sed_amt`, nothing settles and the release takes no sediment that day.

Median particle sizes of sediment classes: clay 1.95 µm, fine silt 4.9 µm, medium silt 11.7 µm, coarse silt 47 µm, fine sand 94 µm, medium sand 375 µm, coarse sand 1500 µm. A reservoir receives sediment from its whole upstream area, so a representative value for that area suits it better than the texture of one HRU.

nutrients.res

One row per nutrient record. Reservoirs refer to it through `nut` in `reservoir.res`, wetlands through `nut` in wetland.wet.

#	Column	Units	Description
1	`name`	-	Name of the nutrient record.
2	`mid_start`	month	First month of the mid-year settling period. Whole number, 1 to 12.
3	`mid_end`	month	Last month of the mid-year settling period. Whole number, 1 to 12.

#	Column	Units	Description
4	mid_n_stl	m/year	Organic nitrogen settling rate in the mid-year period.
5	n_stl	m/year	Organic nitrogen settling rate in the rest of the year.
6	mid_p_stl	m/year	Particulate phosphorus settling rate in the mid-year period.
7	p_stl	m/year	Particulate phosphorus settling rate in the rest of the year.
8	chla_co	m/year	Loss rate for soluble nitrogen (nitrate, ammonium and nitrite).
9	secchi_co	m/year	Loss rate for soluble phosphorus.
10	theta_n	-	Temperature adjustment factor for nitrogen losses.
11	theta_p	-	Temperature adjustment factor for phosphorus losses.
12	n_min_stl	mg/L	Nitrogen concentration below which no nitrogen is lost.
13	p_min_stl	mg/L	Phosphorus concentration below which no phosphorus is lost.

The Editor headers `chla_co` and `secchi_co` come from older versions, where these columns were a chlorophyll-a coefficient and a water clarity coefficient. In 62.0.1 they are the soluble nitrogen and phosphorus loss rates. Chlorophyll-a and Secchi depth are not calculated from them.

The six rates are entered in m/year and divided by 365 when the file is read.

Nutrient losses

The mid-year rates apply in the months from `mid_start` to `mid_end`, and the other rates in the remaining months. The test is `mid_start` ≤ month ≤ `mid_end`, so a period that runs across the end of the year (for example November to March) never applies and the other rates are used all year. To use one rate all year, give both columns the same value.

Each day the fraction of organic nitrogen lost is

$$k_N = \frac{\text{mid_n_stl}}{365}\,\theta_N^{T-20}\,(C_N - C_{N,min})$$

limited to between 0 and 1, where C_N is the organic nitrogen concentration (mg/L), $C_{N,min}$ is `n_min_stl`, θ_N is `theta_n` and T is the day's mean air temperature (°C) at the weather station. Particulate phosphorus is treated the same way with `mid_p_stl` or `p_stl`, `theta_p` and `p_min_stl`. The organic nitrogen and particulate phosphorus masses are multiplied by $1 - k$.

Soluble nitrogen uses `chla_co` and the total soluble nitrogen concentration, and soluble phosphorus uses `secchi_co`. For these, the loss fraction is multiplied by a fixed 0.2 before it is applied to nitrate, ammonium, nitrite and soluble phosphorus.

The rate is multiplied by a concentration difference and the result is used as a fraction, so the units of the product do not match: a value of 1 m/year acts as a daily fraction of 1/365 per mg/L of excess concentration. The fraction cannot go below zero, so negative rates give no loss and the sediment never becomes a source of nutrients.

The release takes each nutrient in proportion to its share of the water: release / (storage + release).

Measured apparent settling velocities for phosphorus in natural lakes most often lie between 5 and 20 m/year, with reported values from below 1 to over 200 m/year (Chapra, 1997). Reservoirs show a wider range: Higgins and Kim (1981) reported −90 to 269 m/year for 18 reservoirs in Tennessee (median 42.2 m/year), and Walker and Kiihner (1978) reported −1 to 125 m/year for 27 Midwestern reservoirs (mean 12.7 m/year). Panuska and Robertson (1999) recommend these values:

Water body nutrient dynamics	Settling velocity (m/year)
Shallow, high net internal phosphorus flux	0 or less
Moderate net internal phosphorus flux	1 to 5
Minimal net internal phosphorus flux	5 to 16

Water body nutrient dynamics	Settling velocity (m/year)
High net internal phosphorus removal	more than 16

In 62.0.1 values of 0 or less give no phosphorus loss.

weir.res

One row per weir record. A reservoir uses a weir record through a `weir` action in its release decision table (the record named in the action's `fp` field). Wetlands use weir records as described under storage volumes and weir height on the wetland page.

#	Column	Units	Built-in value	Description
1	`name`	-	-	Name of the weir record.
2	`linear_c`	-	1.84	Weir discharge coefficient c.
3	`exp_k`	-	2.6	Weir discharge exponent k.
4	`width`	m	2.5	Weir width w.
5	`height`	m	0	Height of the weir crest above the bottom of the impoundment. Used for wetlands only.

The built-in values apply when a `weir` action names a record that is not in the file.

The weir discharge is

$$Q = 0.45\, A\, c\, w\, H^{k}$$

in m^3/s, where A is the surface area of the water body (ha) and H is the water depth above the weir crest (m). In daily runs the discharge is applied hour by hour for up to 24 hours, each hour limited to the water above the crest, and the depth is updated after each hour.

Calibration

These reservoir parameters can be changed through `calibration.cal`: `psa`, `pvol`, `esa` and `evol` (spillway areas in ha and volumes in m^3), `k_res` (`k`), `evrsv` (`evap_co`), `nsed` (`sed_amt`), `res_d50` (`d50`), `sed_stlr` (`sed_stl`), `velsetlr` (`stl_vel`), `nsetlr1`, `nsetlr2`, `psetlr1`, `psetlr2` (the four settling rates), `nsolr` (`chla_co`), `psolr` (`secchi_co`), `soln_stl_fr` and `solp_stl_fr` (the fixed 0.2 factors), the storage and constituent masses `vol`, `sed`, `orgn`, `orgp`, `no3`, `nh3`, `no2` and `solp`, and the release-table constants `drawdown_days` (`const`) and `withdraw_rate` (`const2`).

Calibration acts on the values after unit conversion, so settling rates are changed in m/day. Changing `psa`, `pvol`, `esa` or `evol` does not recalculate the surface area coefficients.

Example

`refdata/Osu_1hru` in the SWAT+ repository has no reservoirs. Its sediment, nutrient, weir and initial-condition files are used by its wetland, and show the formats.

`initial.res`:

```
initial.res: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                      org_min             pest             path             hmet             salt
initwet1                  no_init             null             null             null             null
```

`sediment.res`:

```
sediment.res: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                    sed_amt           d50        carbon            bd       sed_stl       stl_vel
sedwet1                   10.00           5.00       0.04            0.8          0.15       0.0002
```

Sediment settles above 10 mg/L. With `d50` at 5 µm, 40% of the excess remains after one day.

`nutrients.res`:

```
nutrients.res: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name               mid_start   mid_end     mid_n_stl         n_stl     mid_p_stl         p_stl       chla_c
nutwet1                    5        10       5.50000       5.50000      10.00000      10.00000       1.00000
```

The mid-year period is May to October, but the two rates for each nutrient are equal, so one rate applies all year. Both `theta` values are 1, so the losses do not depend on temperature.

`weir.res`:

```
weir.res: Reservoir weir inputs - asdf;lj
NAME   Linear_C    Exp_K     Width(m)   Depth(m)
weir1   1.83       1.50      5.00       0.0
```

The header line here differs from the Editor's. It is skipped, so only the order of the values matters.

A `hydrology.res` and `reservoir.res` pair for one reservoir, written for illustration:

```
hydrology.res: example
name        yr_op  mon_op   area_ps   vol_ps   area_es   vol_es      k   evap_co  shp_co1  shp_co2
hydres1         0       0     150.0   1500.0     200.0   2200.0   0.05      0.60     0.00     0.00
```

```
reservoir.res: example
id   name      init      hyd       rel             sed      nut
1    res001    initres1  hydres1   drawdown_days   sedres1  nutres1
```

The reservoir is operational from the start. Its principal volume is 1500 ha-m (15 million m³). Because `vol_es` is less than ten times `vol_ps`, the surface area follows $A = (150\,V/1.5 \times 10^7)^{0.9}$, which gives 91 ha at the principal spillway. Outflow is not smoothed. Seepage is $240 \times 0.05 \times A$ m³ per day.

The `drawdown_days` table in `refdata/Osu_1hru/res_rel.dtl`:

```
name                    conds      alts      acts
drawdown_days               2         2         2
var                       obj   obj_num           lim_var            lim_op     lim_const      alt1
vol                       res         0              pvol                 *       1.00000         >
vol                       res         0              evol                 *       1.00000         <
act_typ                   obj   obj_num              name            option         const        const2
release                   res         0         over_prin              days      15.00000       0.00000
release                   res         0    over_emergency              days       5.00000       0.00000
```

Between the principal and emergency volumes the reservoir releases storage / 15 per day; above the emergency volume, storage / 5 per day. Below the principal volume no alternative applies and nothing is released. Because `const2` is 0, the base volume is zero in both actions, and each releases a share of the whole storage.

Related files

- `reservoir.con`: places each reservoir in the routing network and assigns its weather station.
- `object.cnt`: number of reservoirs.
- `file.cio`: names the reservoir files.
- `res_rel.dtl`: release decision tables.
- `om_water.ini`: initial water and concentrations referenced from `initial.res`.
- `recall.rec`: measured outflows for `meas` releases.
- `wetland.wet` and `hydrology.wet`: wetlands, which share `initial.res`, `sediment.res`, `nutrients.res`, `weir.res` and `res_rel.dtl`.
- Reservoir output and reservoir and wetland theory.

Sources

- File names: `type input_res` in input_file_module.f90, read in readcio_read.f90
- Call order: main.f90.in, proc_res.f90
- Readers: res_read.f90, res_read_init.f90, res_read_hyd.f90, res_read_sed.f90, res_read_nut.f90, res_read_weir.f90, res_read_conds.f90, dtbl_res_read.f90
- Type definitions: reservoir_data_module.f90, reservoir_module.f90
- Setup: res_objects.f90, res_allo.f90, res_initial.f90, `res_convert_mass` in hydrograph_module.f90
- Daily routines: res_control.f90, res_hydro.f90, res_rel_conds.f90, res_sediment.f90, res_nutrient.f90, theta.f90, res_pest.f90
- Decision table conditions and actions: conditions.f90, actions.f90
- Calibration: cal_parm_select.f90

File	Column	Fortran variable
reservoir.res	`id`, `name`, `init`, `hyd`, `rel`, `sed`, `nut`	index into `res_dat_c`, then `res_dat_c%name`, `%init`, `%hyd`, `%release`, `%sed`, `%nut`
initial.res	`name`, `org_min`, `pest`, `path`, `hmet`, `salt`	`res_init_dat_c%init`, `%org_min`, `%pest`, `%path`, `%hmet`, `%salt`
hydrology.res	`yr_op`, `mon_op`	`res_hyddb%iyres`, `%mores`
hydrology.res	`area_ps`, `vol_ps`, `area_es`, `vol_es`	`res_hyddb%psa`, `%pvol`, `%esa`, `%evol`
hydrology.res	`k`, `evap_co`	`res_hyddb%k`, `%evrsv`
hydrology.res	`shp_co1`, `shp_co2`	`res_hyddb%br1`, `%br2`, copied to `res_ob%lag_up`, `%lag_down`
sediment.res	`sed_amt`, `d50`, `carbon`, `bd`, `sed_stl`, `stl_vel`	`res_sed%nsed`, `%d50`, `%carbon`, `%bd`, `%sed_stlr`, `%velsetlr`
nutrients.res	`mid_start`, `mid_end`	`res_nut%ires1`, `%ires2`
nutrients.res	`mid_n_stl`, `n_stl`, `mid_p_stl`, `p_stl`	`res_nut%nsetlr1`, `%nsetlr2`, `%psetlr1`, `%psetlr2`
nutrients.res	`chla_co`, `secchi_co`	`res_nut%nsolr`, `%psolr`
nutrients.res	`theta_n`, `theta_p`, `n_min_stl`, `p_min_stl`	`res_nut%theta_n`, `%theta_p`, `%conc_nmin`, `%conc_pmin`
weir.res	`linear_c`, `exp_k`, `width`, `height`	`res_weir%c`, `%k`, `%w`, `%h`

Purpose

A wetland in SWAT+ is an HRU that can store ponded water on its surface. The same mechanism simulates rice paddies. Two files are specific to wetlands, named in slots 7 and 8 of the `reservoir` line of file.cio:

- `wetland.wet`: one row per wetland, naming its initial conditions, hydrology, release rule, sediment and nutrient records.
- `hydrology.wet`: wetland hydrology records. Depths and areas are relative to the HRU.

Wetlands also use four reservoir files, described on the reservoir page: initial.res, sediment.res, nutrients.res and weir.res. Release rules are decision tables in res_rel.dtl (see decision tables).

An HRU becomes a wetland when the `surf_stor` column of hru-data.hru names a row of `wetland.wet`. With `null` there, the HRU has no ponded storage.

Both files are free format and whitespace-separated. Line 1 is a title and line 2 is the column header; both are skipped. The readers do not match columns by header name, so every row needs all of its values in the order given below. Columns after the last one listed are ignored. Names must match, including case. A slot in file.cio set to `null`, or a file that does not exist, leaves the table empty.

wetland.wet

One row per wetland.

#	Column	Units	Description
1	`id`	-	Wetland number. Read; not used. Rows are used in the order they appear.
2	`name`	-	Wetland name, matched against `surf_stor` in `hru-data.hru`.
3	`init`	-	Name of a row in `initial.res`.
4	`hyd`	-	Name of a row in `hydrology.wet`. The name `paddy` switches on rice paddy behaviour; see Outflow.
5	`rel`	-	Name of a release decision table in `res_rel.dtl`. Not used for paddies.
6	`sed`	-	Name of a row in `sediment.res`.
7	`nut`	-	Name of a row in `nutrients.res`.

Several HRUs can use the same row. A name that is not found is reported in `diagnostics.out` as `<name> not found (wet-init)`, `(wet-hyd)`, `(wet-release)`, `(wet-sed)` or `(wet-nut)`, and a `surf_stor` name that is not in `wetland.wet` as `<name>not found (wetland.wet)`. A wetland whose `hyd` is not found uses the built-in hydrology values (zero depths and areas). A non-paddy wetland whose `rel` is not found releases no water through its rule.

Some files, such as the example below, have an eighth column naming a weir record. 62.0.1 does not read it; how a wetland gets its weir record is described under Storage volumes and weir height.

hydrology.wet

One row per wetland hydrology record.

#	Column	Units	Built-in value	Description
1	`name`	-	-	Name of the hydrology record, matched against `hyd` in `wetland.wet`.
2	`hru_ps`	fraction	0	Fraction of the HRU area covered at the principal spillway (when flow over the surface inlet riser starts). Used only in the surface area set up before the first day, which the daily calculation replaces.
3	`dp_ps`	mm	0	Average water depth at the principal spillway. Sets the principal volume and, without weir records, the weir height.
4	`hru_es`	fraction	0	Fraction of the HRU area covered at the emergency spillway (when water starts to spill into the ditch). Not used in the daily simulation.
5	`dp_es`	mm	0	Average water depth at the emergency spillway. Sets the emergency volume.
6	`k`	mm/hr	0.01	Hydraulic conductivity of the wetland bottom. Sets the seepage rate.
7	`evap`	-	0.7	Evaporation coefficient, a multiplier on potential evapotranspiration for ponded water.
8	`vol_area_co`	-	1	Coefficient a of the volume–area relation.
9	`vol_dp_a`	-	1	Coefficient b of the volume–depth relation.
10	`vol_dp_b`	-	1	Coefficient c of the volume–depth relation.

#	Column	Units	Built-in value	Description
11	`hru_frac`	fraction	0.5	Fraction of the HRU that drains into the wetland. Read but not used.

The built-in values apply to wetlands whose `hyd` name is not found. In rows of the file, the reader replaces zero or negative values in four columns:

- `evap` becomes 0.6.
- `k` becomes the saturated hydraulic conductivity of the top soil layer of the HRU whose number equals the row number of the record in `hydrology.wet` (HRU 1 for the first row, and so on), which is not in general an HRU that uses the record.
- `hru_ps` and `hru_es` are set to 0.08 × and 1.5 × values that are still zero when the file is read, so they stay at 0.

Wetland behaviour

Storage volumes and weir height

At the start of the run, and again when a decision table switches the wetland on, the storage limits are set from the HRU area A_{hru} (ha) on the assumption that the whole HRU is ponded:

$$V_p = 10\,A_{hru}\,\mathrm{dp_ps}, \qquad V_e = 10\,A_{hru}\,\mathrm{dp_es}$$

in m³. `hru_ps` and `hru_es` do not enter the volumes.

If `weir.res` has at least one record, every wetland starts with the first record of `weir.res` as its weir: the weir height is that record's `height` (m), the principal volume becomes $V_p = 10^4\,A_{hru}$ height, and V_e is raised to at least $1.2\,V_p$. Without weir records the weir height is `dp_ps` / 1000 m. A `weir` operation in the management schedule can later change the height and select another `weir.res` record by name (see Management and decision tables).

The initial water volume is the `flo` fraction of the `om_water.ini` row named in `initial.res`, multiplied by V_p, with concentrations from the same row.

Daily water balance

For an HRU with a wetland, the daily HRU routines change as follows:

- Surface run-on from upstream objects (with tile flow) is added to the wetland storage.
- The HRU generates no surface runoff. The day's precipitation, over the whole HRU area ($10\,R\,A_{hru}$ m³), and any irrigation water are added to the wetland storage.
- Evaporation from the ponded water is taken before soil evaporation; while water is ponded there is no soil evaporation (see Evaporation).
- Seepage from the ponded water is the infiltration into the soil.
- Water above saturation in the soil profile is returned to the wetland storage. If the weir height is below 1 mm it becomes surface runoff instead.
- The wetland outflow is the surface runoff of the HRU.

Water depth is storage divided by the HRU area: $d = V/(10\,A_{hru})$ mm.

Surface area

For wetlands that are not paddies, the water surface area A_w follows the storage V relative to the principal volume V_p:

$$x = b^2 + 4c\left(1 - \frac{V}{V_p}\right), \qquad h = \frac{-b - \sqrt{x}}{2c} + b, \qquad A_w = A_{hru}\,\min(1, \max(0.01,\ 1 + a\,h))$$

with a = `vol_area_co`, b = `vol_dp_a` and c = `vol_dp_b`. When x is below 10^{-6}, h is 0. With the built-in values ($a = b = c = 1$) the whole HRU is covered at $V = V_p$, and 63% at $V = 0.5\,V_p$. The area set up before the first day also multiplies the fraction by `hru_ps`.

For paddies the surface area is always the HRU area.

The surface area sets seepage and the weir flow. Precipitation, evaporation and the water depth are always calculated over the whole HRU.

Evaporation

When the HRU has ponded water and its leaf area index is 4 or less, the maximum evaporation is

$$E_{max} = \text{evap}\,\left(1 - \frac{LAI}{4}\right) PET$$

With a leaf area index above 4 there is no evaporation from the ponded water. Evaporation is taken from the ponded water first, and from the soil only when no ponded water remains.

In 62.0.1 the `evap` value is looked up through the record's row number in `hydrology.wet` used as an HRU number: the coefficient applied is that of the wetland on the HRU with that number, or 0.7 if that HRU has no wetland. In a model where HRU 1 uses the first `hydrology.wet` record, HRU 2 the second and so on, this gives the intended value.

Seepage

Daily seepage is

$$S = \min\left(V,\ 240\,k\,A_w\right)$$

in m³, with k in mm/hr. When the seepage is more than 0.1 mm over the HRU, it is reduced by the amount the soil profile cannot take before it becomes saturated. Nitrate, ammonium, soluble phosphorus, organic nitrogen and organic phosphorus move with the seepage into the top soil layer in proportion to the water. A puddling operation can replace `k` during the run. When gwflow is active, the exchange with groundwater is calculated by the groundwater model (see gwflow).

Outflow

A wetland whose `hyd` name in `wetland.wet` is `paddy` is a rice paddy. Its outflow is weir flow from the `weir.res` record selected for it, and `rel` is not used:

- If the storage exceeds V_e, the water above V_e is released.
- While the water depth is above the weir height, flow over the weir is $Q = 0.45\,A\,c\,w\,H^k$ (m³/s), with the weir.res coefficients, the HRU area A (ha) and the depth above the weir H (m). It is applied hour by hour for up to 24 hours in a daily run, each hour limited to the water above the weir.
- With the weir height at zero the whole water depth acts as head, so the ponded water drains over the weir.

Other wetlands release through the decision table named in `rel`. The conditions and release options are those of reservoirs (release rules), evaluated against the wetland's storage, principal and emergency volumes, water depth and weir height. The outflow is not smoothed.

Paddy HRUs also write a daily line to `paddy_test.csv` with precipitation, irrigation, seepage, evapotranspiration terms, weir height, water depth, weir outflow, soil water, sediment and nitrate.

Sediment and nutrients

Sediment settling and nutrient losses in the ponded water follow the reservoir equations with the wetland's `sediment.res` and `nutrients.res` records (see sediment settling and nutrient losses), using the HRU's weather. When the wetland was empty at the start of the day and releases water, the sediment yield of the outflow is calculated with a MUSLE-type equation from the HRU's cover and erosion factors, as for a flushed field.

Management and decision tables

These operations and decision-table entries act on wetlands and paddies:

- Decision table conditions `vol_wet` (storage against the principal or emergency volume), `wet_depth` (water depth, mm) and `weirh` (weir height, mm). See decision tables.
- Decision table action `weir_height` with option `wet`: sets the weir height to `const` mm, sets V_p to the HRU area times that height, and raises V_e to at least $1.2\,V_p$. It does not change the weir record.
- Management operation `weir`: sets the weir height to its third data value (mm), sets V_p the same way with V_e raised to at least $1.1\,V_p$, and selects the `weir.res` record named in its first data field.
- Decision table actions `impound_off` and `impound_on`: `impound_off` removes the wetland and its stored water from the HRU. `impound_on` sets the wetland up again from the `surf_stor` row of `hru-data.hru`; the name given in the action's file pointer is not matched in 62.0.1.
- Puddling (`puddle` action, with `puddle.ops`): replaces the wetland bottom conductivity and raises the sediment concentration of the ponded water.
- Fertilizer applied to an HRU with ponded water goes into the water when `carbon` in `codes.bsn` is 2. With other `carbon` values it is counted in the fertilizer output but added to neither the water nor the soil.
- Tillage when the water depth is above 1 mm mixes the soil layers with the ponded-water routine, including pesticides in the soil.

See management schedules and operations for the operation formats.

When gwflow is active and `gwflow.wetland` exists, the wetland bed thickness (m) for each wetland HRU is read from it: two header lines, then one line per wetland with a name and a thickness, where the digits in the name give the HRU number (for example `wet019` for HRU 19).

Example

`refdata/Osu_1hru` in the SWAT+ repository is a single rice paddy HRU. `hru-data.hru` names `paddy0001` in `surf_stor`.

`wetland.wet`:

```
wetland.       wet: written by SWAT+ edi#############for SWAT+ rev.60.5.4
     id                name        init         hyd         rel         sed         nut        WEIR
      1          paddy0001    initwet1       paddy        weir     sedwet1     nutwet1       weir1
```

The title line is damaged but is skipped. The hydrology record is `paddy`, so the HRU is a paddy and the `weir` decision table named in `rel` is not used. The `WEIR` column is not read.

`hydrology.wet`:

```
hydrology.wet: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                    hru_ps         dp_ps        hru_es         dp_es             k          evap    vol
paddy                     1.00        150.00          1.00        150.00           0.5          0.75
```

The emergency volume is 150 mm of water over the HRU. Seepage is at most $0.5 \times 24 = 12$ mm per day. Ponded water evaporates at $0.75 \times$ PET while the rice canopy is sparse, falling to zero as the leaf area index reaches 4.

`weir.res` has one record, `weir1`, with a height of 0 m, so the paddy starts with a weir height of zero and a principal volume of zero, and any ponded water drains over the weir. During the season `weir_height` actions in `lum.dtl` raise the weir to 60 mm and lower it to 0 again. While water stands above 60 mm it flows over the 5 m weir at $Q = 0.45\,A \times 1.83 \times 5 \times H^{1.5}$ m³/s.

Related files

- `hru-data.hru`: assigns a `wetland.wet` row to an HRU through `surf_stor`.
- `file.cio`: names the wetland files.
- `initial.res`, [`sediment.res`](#input-reference-res-sedimentres), [`nutrients.res`](#input-reference-res-nutrientsres) and `weir.res`: shared with reservoirs.

- res_rel.dtl and lum.dtl: wetland release rules and wetland actions.
- management.sch and operations: weir, irrigation, puddling, fertilizer and tillage operations on paddies.
- Salt and constituent inputs for wetlands.
- Water balance output and reservoir and wetland theory.

Sources

- File names: `type input_res` in input_file_module.f90; call order in main.f90.in
- Readers: wet_read.f90, wet_read_hyd.f90
- Type definitions: `type reservoir_data_char_input` and `type wetland_hyd_data` in reservoir_data_module.f90; `type wetland` in reservoir_module.f90
- Setup: hru_allo.f90, wet_all_initial.f90, wet_initial.f90
- Daily routines: hru_control.f90, et_act.f90, wetland_control.f90, res_weir_release.f90, res_hydro.f90, res_sediment.f90, res_nutrient.f90, swr_satexcess.f90
- Conditions, actions and operations: conditions.f90, actions.f90, mgt_sched.f90, pl_fert_wet.f90, mgt_newtillmix_wet.f90

File	Column	Fortran variable
wetland.wet	`id`	read into a dummy `k`
wetland.wet	`name`, `init`, `hyd`, `rel`, `sed`, `nut`	`wet_dat_c%name`, `%init`, `%hyd`, `%release`, `%sed`, `%nut`
hydrology.wet	`hru_ps`, `dp_ps`, `hru_es`, `dp_es`	`wet_hyddb%psa`, `%pdep`, `%esa`, `%edep`
hydrology.wet	`k`, `evap`	`wet_hyddb%k`, `%evrsv`
hydrology.wet	`vol_area_co`, `vol_dp_a`, `vol_dp_b`	`wet_hyddb%acoef`, `%bcoef`, `%ccoef`
hydrology.wet	`hru_frac`	`wet_hyddb%frac`

Purpose

The six operation databases contain named parameter sets for management operations. A scheduled operation in management.sch or an action in a lum.dtl decision table names an entry, and SWAT+ uses that entry's values when the operation runs.

File	Contents	Used by
`harv.ops`	harvest operations	`harv` and `hvkl` in management.sch; `harvest` and `harvest_kill` in lum.dtl
`graze.ops`	grazing operations	`graz` in management.sch; `graze` in lum.dtl
`irr.ops`	irrigation operations	`irrm`, `irrp` and `irpm` in management.sch; `irrigate` and `irr_demand` in lum.dtl
`chem_app.ops`	application methods for fertilizer, manure and pesticide	`fert`, `manu` and `pest` in management.sch; `fertilize`, `fert_future`, `manure`, `pest_apply` and `manure_demand` in lum.dtl
`fire.ops`	burning operations	`burn` in management.sch and lum.dtl
`sweep.ops`	street sweeping operations	`swep` in management.sch

The six file names are given on the `ops` line of file.cio, in the order harvest, graze, irrigation, chemical application, fire, sweep. The defaults are the names in the table. If a file is missing, or named `null` in file.cio, the database is empty, and any operation that names an entry from it uses an entry with all values zero (irrigation depth 25.4 mm).

Format

All six files share one layout. Free format, whitespace-separated. Line 1 is a title and line 2 is the column header; both are skipped. Each later line is one operation. The columns are read by position, so every column in the tables below must be present and in this order. A trailing `description` column, which the SWAT+ Editor writes, is ignored.

Names are matched exactly against the names used in management.sch and lum.dtl. A name that is not found leaves the operation pointing at an empty entry with zero values, and no message is written.

harv.ops

#	Column	Units	Typical range	Description
1	`name`	-	-	Harvest operation name.
2	`harv_typ`	-	see below	Harvest type.
3	`harv_idx`	fraction	0 to 1	Harvest index override: the fraction of above-ground biomass cut. Used by biomass harvests and, as a multiplier, by `residue`. Ignored by grain and root harvests.
4	`harv_eff`	fraction	0 to 1	Harvest efficiency: the fraction of the harvested material removed from the HRU.
5	`harv_bm_min`	kg/ha	0 and up	Minimum plant biomass for the harvest to take place. For `residue` harvests, the minimum surface residue left behind.

The harvest types and what each one does:

`harv_typ`	Harvest only (`harv`, `harvest`)	Harvest and kill (`hvkl`, `harvest_kill`)
`grain`	grain harvest	grain harvest
`picker`	grain harvest	grain harvest
`stripper`	biomass harvest	grain harvest
`biomass`	biomass harvest	biomass harvest
`tree`	biomass harvest	no harvest; the plant is only killed
`tuber`	root harvest	root harvest
`peanuts`	root harvest	root harvest
`residue`	residue harvest	residue harvest

A harvest of a growing plant runs only if the plant's total biomass is above `harv_bm_min`. A `residue` harvest scheduled after the plant has died runs without this check.

Grain harvest. The yield is $Y = \text{harv_eff} \cdot M_{seed}$, where M_{seed} is the seed (yield) pool built up during growth from the harvest index in `plants.plt`. The seed mass that is not removed goes to the top soil layer as organic matter. `harv_idx` has no effect.

Biomass harvest. A fraction $h = \text{harv_idx} \cdot \text{harv_eff}$ of the seed, leaf and stem mass is cut. The cut takes place only if the above-ground biomass left afterwards is above `harv_bm_min`. The cut mass is reported as yield, and a further $(1 - \text{harv_eff})$ times the cut mass is added to the surface residue. Leaf area and accumulated heat units are multiplied by $1 - 0.5\,h$.

Root harvest (`tuber`, `peanuts`). The yield is `harv_eff` times the root mass. The remaining roots stay in the soil as residue.

Residue harvest. Each plant's surface residue is reduced by the fraction $e \cdot \text{harv_idx}$, but not below `harv_bm_min`. The efficiency e is the `op_data3` value of the `management.sch` row (or `const` of the decision table action) when it is greater than zero, and `harv_eff` otherwise.

For every type, the reported yield is reduced by the plant's pest stress.

graze.ops

#	Column	Units	Typical range	Description
1	`name`	-	-	Grazing operation name.

#	Column	Units	Typical range	Description
2	fert	-	-	Name of the manure in fertilizer.frt that the animals deposit. Called fertname in earlier documentation.
3	bm_eat	kg/ha/day	0 to 500	Dry biomass removed by grazing each day.
4	bm_tramp	kg/ha/day	0 to 500	Dry biomass removed by trampling each day. Read but not applied.
5	man_amt	kg/ha/day	0 to 500	Dry manure deposited each day.
6	grz_bm_min	kg/ha	0 to 5000	Minimum above-ground biomass for grazing to take place on a given day.

A graz row in management.sch starts a grazing period, and its op_data3 value is the length of the period in days. On each day of the period, if the HRU's above-ground biomass is at least grz_bm_min, the model removes bm_eat from the plants (spread over the plants in proportion to their above-ground biomass, and never more than is there) and adds man_amt of the named manure to the top soil layer. While a plant's biomass is below 2500 kg/ha, grazing also lowers its leaf area index by bm_eat/1000 and multiplies its accumulated heat units by 1 − bm_eat/1000. A graze action in a decision table applies one day of grazing each time it fires.

The parameter values in the SWAT+ Editor's default graze.ops are rough estimates. If grazing matters in a watershed, derive values from the stocking rate and type of animal. Trampling is often set equal to bm_eat (the animals trample as much as they eat), but in 62.0.1 bm_tramp has no effect.

irr.ops

#	Column	Units	Typical range	Description
1	name	-	-	Irrigation operation name.
2	amt_mm	mm	0 and up	Irrigation depth per application. Called irr_amt in earlier documentation.
3	eff_frac	fraction	0 to 1	In-field efficiency: the fraction of applied water available to the crop. Called irr_eff in earlier documentation.
4	sumq_frac	fraction	0 to 1	Fraction of the applied water that leaves as surface runoff. Called surq_rto in earlier documentation.
5	dep_sub	mm	0 and up	Application depth for subsurface irrigation. Read but not used. Called irr_dep in earlier documentation.
6	salt_ppm	ppm	0 and up	Salt concentration in the irrigation water. Used by the paddy irrigation operation irrp. Called irr_salt in earlier documentation.
7	no3_ppm	ppm	0 and up	Nitrate concentration in the irrigation water. Used by irrp. Called irr_no3n in earlier documentation.
8	po4_ppm	ppm	0 and up	Phosphate concentration in the irrigation water. Read but not used. Called irr_po4 in earlier documentation.

A scheduled irrigation (irrm or irpm in management.sch) applies

$$I_{applied} = \text{amt_mm} \cdot \text{eff_frac} \cdot (1 - \text{sumq_frac}), \qquad I_{runoff} = \text{amt_mm} \cdot \text{sumq_frac}$$

The `irrigate` action in `lum.dtl` uses the action's `const` (mm) in place of `amt_mm` and takes `eff_frac` and `sumq_frac` from the entry named in its `option` column. The `irr_demand` action with file pointer `unlim` uses `amt_mm`, but computes runoff as amt_mm · eff_frac · sumq_frac. See `*.dtl` for both actions.

chem_app.ops

#	Column	Units	Typical range	Description
1	`name`	-	-	Application method name, for example `broadcast`, `band`, `foliar`, `inject`, `aerial_liquid`, `aerial_solid`, `drill`, `side_dress`, `fertigate`, `basal`, `rope_wick`, `tree_inject`.
2	`chem_form`	-	`solid`, `liquid`	Form of the chemical. Read but not used.
3	`app_typ`	-	`spread`, `spray`, `inject`, `direct`	Application type. Read but not used.
4	`app_eff`	fraction	0 to 1	Application efficiency. Multiplies the amount of a `pest` operation or `pest_apply` action. Not applied to fertilizer or manure.
5	`foliar_eff`	fraction	0 to 1	Foliar efficiency. Read but not used.
6	`inject_dp`	mm	0 and up	Injection depth. Read but not used.
7	`surf_frac`	fraction	0 to 1	Fraction of the applied fertilizer, manure or pesticide placed in the top 10 mm soil layer. The rest goes to the second soil layer.
8	`drift_pot`	fraction	0 to 1	Drift potential. Read but not used.
9	`aerial_unif`	fraction	0 to 1	Aerial uniformity. Read but not used.

A decision table `fertilize` action on an HRU with standing water applies the fertilizer to the water instead of the soil when the method's `surf_frac` is 1.

fire.ops

#	Column	Units	Typical range	Description
1	`name`	-	-	Fire operation name, for example `grass`, `tree_intense`, `tree_low`.
2	`chg_cn2`	-	0 to 20	Amount added to the HRU's curve number for moisture condition II. The result is capped at 98.
3	`frac_burn`	fraction	0 to 1	Fraction of above-ground plant biomass, surface residue and top-layer soil organic matter burned.

A burn removes `frac_burn` of the above-ground biomass of every plant, of the surface residue, and of the slow and passive humus in the top soil layer. The phosphorus in the burned material is returned to the passive humus pool, and the burned plant carbon is reported as emitted carbon.

sweep.ops

#	Column	Units	Typical range	Description
1	name	-	-	Street sweeping operation name.
2	swp_eff	fraction	0 to 1	Removal efficiency: the fraction of built-up solids removed from the swept curb length.
3	frac_curb	fraction	0 to 1	Fraction of the curb length that can be swept. Parked cars and other obstacles often make part of the curb unavailable.

Street sweeping is meant for urban HRUs that use the build-up and wash-off method. The removal efficiency depends on the type of sweeper, whether flushing is part of the cleaning, the amount of solids and the frequency of rain, and is usually higher for particulate constituents than for nitrogen and phosphorus (Pitt, 1979). SWAT+ assumes fixed nutrient concentrations in the solids, so one efficiency applies to every constituent.

In 62.0.1 a `swep` operation reads the entry and logs it, but the sweeping routine uses a curb fraction of zero and is never called during the run, so `sweep.ops` has no effect on results.

Example

`refdata/Osu_1hru` in the SWAT+ repository, first rows of each file:

```
harv.ops: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                      harv_typ      harv_idx      harv_eff   harv_bm_min  description
forest_cut                    tree       0.95000       0.99000       0.00000
grain                        grain       0.00000       0.95000       0.00000
grass_bag                  biomass       0.50000       1.00000    2000.00000
grass_mulch                biomass       0.50000       0.00000    2000.00000
```

`grass_bag` and `grass_mulch` both use a cut fraction of 0.5. With `harv_eff` at 1, `grass_bag` removes everything it cuts; with `harv_eff` at 0, `grass_mulch` cuts nothing ($h = 0.5 \times 0$). Both run only on plants above 2000 kg/ha. `grain` removes 95% of the seed pool.

```
graze.ops: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                           fert        bm_eat      bm_tramp       man_amt    grz_bm_min  description
dairy_high                 dairy_fr      25.00000      10.00000       0.95000    3000.00000  High_Prod_Dair
```

```
irr.ops: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                       amt_mm      eff_frac     sumq_frac       dep_sub      salt_ppm       no3_ppm
sprinkler_med          25.00000       0.85000       0.00000       0.00000     100.00000      20.00000
drip                   25.00000       0.90000       0.00000     100.00000     100.00000      20.00000
```

A scheduled `sprinkler_med` irrigation puts $25 \times 0.85 = 21.25$ mm of water into the soil.

```
chem_app.ops: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                       chem_form           app_typ       app_eff    foliar_eff     inject_dp     surf_fr
broadcast                      solid            spread       0.90000       0.00000       0.00000       1.000
inject                        liquid            inject       0.95000       0.00000     150.00000       0.200
```

```
fire.ops: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                     chg_cn2     frac_burn  description
grass                    8.00000       1.00000
tree_low                 6.00000       0.70000
```

Related files

- `management.sch`: scheduled operations that name entries in these files.
- `*.dtl`: decision table actions that name entries in these files.
- `fertilizer.frt`: the manure named in `graze.ops`, and the fertilizers applied with a `chem_app.ops` method.

- plants.plt: the plant harvest index that builds the seed pool used by grain harvests.
- file.cio: names the six files on its ops line.

Sources

- Readers: mgt_read_harvops.f90, mgt_read_grazeops.f90, mgt_read_irrops.f90, mgt_read_chemapp.f90, mgt_read_fireops.f90, mgt_read_sweepops.f90, called from proc_db.f90
- Type definitions: mgt_operations_module.f90
- File name slots: type input_ops in input_file_module.f90
- Scheduled operations: mgt_sched.f90; decision table actions: actions.f90
- Harvest: mgt_harvgrain.f90, mgt_harvbiomass.f90, mgt_harvtuber.f90, mgt_harvresidue.f90
- Grazing: pl_graze.f90, daily loop in hru_control.f90
- Burning: pl_burnop.f90
- Street sweeping: hru_sweep.f90, hru_urbanhr.f90
- Application methods: pl_fert.f90, pest_apply.f90

File	Column	Fortran variable
harv.ops	harv_typ	harvop_db%typ
harv.ops	harv_idx	harvop_db%hi_ovr
harv.ops	harv_eff	harvop_db%eff
harv.ops	harv_bm_min	harvop_db%bm_min
graze.ops	fert	grazeop_db%fertnm (index stored in grazeop_db%manure_id)
graze.ops	bm_eat	grazeop_db%eat
graze.ops	bm_tramp	grazeop_db%tramp
graze.ops	man_amt	grazeop_db%manure
graze.ops	grz_bm_min	grazeop_db%biomin
irr.ops	eff_frac	irrop_db%eff
irr.ops	sumq_frac	irrop_db%surq
irr.ops	dep_sub	irrop_db%dep_mm
irr.ops	salt_ppm, no3_ppm, po4_ppm	irrop_db%salt, %no3, %po4
chem_app.ops	chem_form	chemapp_db%form
chem_app.ops	app_typ	chemapp_db%op_typ
chem_app.ops	inject_dp	chemapp_db%inject_dep
fire.ops	chg_cn2	fire_db%cn2_upd
fire.ops	frac_burn	fire_db%fr_burn
sweep.ops	swp_eff	sweepop_db%eff
sweep.ops	frac_curb	sweepop_db%fr_curb

Purpose

A decision table is a set of rules. Each day SWAT+ checks the table's conditions for an object (an HRU, a reservoir, a water allocation object), finds which of the table's alternatives are satisfied, and performs the actions that those alternatives switch on. Decision tables drive automatic management (planting, harvest, fertilizer, irrigation), reservoir and paddy releases, land use change scenarios, and water allocation demands. See decision tables for the concept.

There are four decision table files. Their names are given on the decision_table line of file.cio, in this order:

File	Default name	Used by
land use and management	lum.dtl	HRUs, through the automatic operations listed in management.sch; HRU-lte growing seasons; manure demands in manure_allo.mnu; irrigation demands, outside-source concentrations and treatment in water allocation
reservoir release	res_rel.dtl	reservoirs and HRU wetlands or paddies that name a release table (see *.res and *.wet)

File	Default name	Used by
land use scenarios	`scen_lu.dtl`	tables listed in `scen_dtl.upd`, checked for every HRU every day
flow control	`flo_con.dtl`	saturated buffers, and water allocation demand and source or receiver fractions

A file that is missing, or named `null` in `file.cio`, gives an empty set of tables.

Format

All four files have the same layout. Free format, whitespace-separated.

- Line 1: title (skipped).
- Line 2: the number of decision tables in the file.
- Line 3: skipped (normally blank).
- Then each table in turn:
  - a header line (skipped);
  - the table line: `name`, `conds`, `alts`, `acts` (table name, number of conditions, number of alternatives, number of actions);
  - a header line (skipped);
  - `conds` condition rows;
  - a header line (skipped);
  - `acts` action rows;
  - in `res_rel.dtl`, `scen_lu.dtl` and `flo_con.dtl` only: one more line, which is skipped. Leave a blank line here. Without it the next table's header is skipped in its place and the following lines are read out of step.

The counts on the table line must match the rows that follow. Extra or missing rows shift every later table. Names are matched exactly, and a name that is not found is not reported.

Condition rows

#	Column	Description
1	`var`	Condition variable (see Condition variables).
2	`obj`	Object type the variable belongs to, for example `hru`, `res`, `null`. For `prob_unif`, this column contains the application probability instead (see below).
3	`obj_num`	Object number. 0 means the object the table is being checked for.
4	`lim_var`	Limit variable. A name (land use, plant, soil group) or a reference volume, depending on the condition. `null` when not used.
5	`lim_op`	Limit operator: `*`, `+`, `-` or `/`. Combines `lim_var` with `lim_const` for `soil_water`, `vol` and `vol_wet`. `-` when not used.
6	`lim_const`	Limit constant.
7 and on	`alt1`, `alt2`, …	One alternative operator per alternative.

For most conditions the current value of the variable is compared with `lim_const`. For `soil_water`, `vol` and `vol_wet` it is compared with a target built from `lim_var`, `lim_op` and `lim_const`, for example "emergency volume × 0.8".

Alternatives

Each alternative is a column of operators, one per condition. An alternative is satisfied when every one of its conditions is true. For numeric conditions the operators are:

Operator	Condition is true when
`<`	value < limit
`>`	value > limit
`<=`	value limit
`>=`	value limit
`=`	value = limit
`/=`	value limit
`-`	always (the condition does not apply to this alternative)

Any other text in a numeric condition's alternative column is treated like `-`. In particular `/` does not mean "not equal" for numeric conditions.

The yes/no conditions (`plant_gro`, `plant_name_gro`, `hyd_soil_group`, `land_use`, `cal_group`, `tillage`, `prob_unif`, `prob_unif1`, `prob_unif_lu`, `ch_order`) use only `=`, meaning the condition must be true. `tile_drained` and `plant_com` also accept `/`, meaning the condition must be false. Every other operator is treated like `-`.

Action rows

#	Column	Description
1	`act_typ`	Action type (see Land use and management actions and Reservoir release actions).
2	`obj`	Object type. For `irrigate`, the type of the water source.
3	`obj_num`	Object number. 0 means the object the table is being checked for.
4	`name`	Action name. Used to link `days_act` conditions to this action, and by `irrigate`, `irr_demand` and `puddle` as described below. Otherwise free text.
5	`option`	Action option, specific to the action type.
6	`const`	First action constant.
7	`const2`	Second action constant. In `lum.dtl`, values below 1 are raised to 1.
8	`fp`	File pointer: a name in another input file, specific to the action type.
9 and on	`outcome1`, `outcome2`, …	`y` or `n` for each alternative.

An action runs when at least one alternative is satisfied and the action's outcome for that alternative is `y`. Actions run in the order of the rows.

In `lum.dtl`, many actions use `const2` as the maximum number of times the action may run in a calendar year for each HRU. The count is reset on 31 December. These actions are marked "per year: `const2`" in the table below.

Condition variables

Units are those of the compared value. "Plants" means the plants of the HRU's plant community.

`var`	Value compared	Notes
`w_stress`	Average water stress factor of the growing plants (1 = no stress)	1 when no plant is growing.
`n_stress`	Average nitrogen stress factor of the growing plants	0 when no plant is growing.
`p_stress`	Average phosphorus stress factor of the growing plants	0 when no plant is growing.
`phu_plant`	Fraction of potential heat units accumulated by the first plant that has started accumulating (plant 1 if none)	
`phu_base0`	Fraction of the annual base-zero heat units accumulated at the HRU's weather station	`obj` must be `hru` or `hlt`.
`precip_cur`	Precipitation today, mm	`obj` must be `hru` or `hlt`.
`precip_next`	Precipitation tomorrow, mm	`obj` must be `hru` or `hlt`.
`plant_gro`	Plant number `lim_const` (1 if 0) is growing	Yes/no.
`plant_name_gro`	The plant named in `lim_var` is in the community and growing	Yes/no.
`days_plant`	Days since the last planting	
`days_harv`	Days since the last harvest	
`days_irr`	Days since the last irrigation	

var	Value compared	Notes
days_act	Days since the action named in lim_var (an action name in the same table) last ran	Compared with lim_const + 2.
day_start	Day of year on which the simulation started	
irr_year	Irrigation applied to the HRU, mm	In 62.0.1 the total is not reset each year, so it is the total since the start of the run.
slope	HRU slope, m/m	
soil_water	Soil water content above wilting point, mm	Target: lim_var (wp = 0, fc = water at field capacity, ul = water at saturation), combined with lim_const by lim_op.
jday	Day of year	
month	Month	
year_rot	Year of the crop rotation	Starts at the plant community's initial rotation year and increases by 1 each 31 December.
year_gro, cur_yrs_mat	Years of growth of plant 1	Same value for both names.
year_cal	Calendar year	
year_seq	Year of the simulation (1 for the first year)	
year_start	First calendar year of the simulation	
biomass	Above-ground plant biomass, kg/ha	
leaf_area	Leaf area index of all plants	
ground_cov	Above-ground plant biomass, kg/ha	Residue is not included in 62.0.1, so it gives the same value as biomass.
p_factor	USLE P factor	
k_factor	USLE K factor of the top soil layer	
hyd_soil_group	Soil hydrologic group equals lim_var (A, B, C or D)	Yes/no.
p_pet	Ratio of average annual precipitation to PET from the weather generator	
p_lab_150	Labile phosphorus in the upper soil, ppm	Uses the first two soil layers.
soil_temp2	Temperature of soil layer 2, °C	
soil_carbon	Organic carbon of soil layer 1, %	
tile_drained	The HRU has tile drains	= true, / false.
prob	A uniform random number between 0 and 1	Compared with lim_const; use < to act with probability lim_const.
prob_unif	Today is the drawn application day	See Probability conditions.
prob_unif1	Random draw within a window that starts on 1 January	See Probability conditions.
prob_unif_lu	Random draw that spreads an operation over the HRUs of a land use	See Probability conditions.
channel_flo	Flow in the current hydrograph, m^3/s	
tile_flo	Tile flow from the HRU today, mm	
irr_demand	Irrigation demand of the HRU, m^3	
irr_pou_dmd	Demand of water allocation place of use obj_num, m^3	
aqu_dep	Depth to the water table of aquifer obj_num, m	
land_use	HRU land use equals lim_var (a name in landuse.lum)	Yes/no.
cal_group	HRU calibration group in landuse.lum equals lim_var	Yes/no.
tillage	The HRU's management schedule lists the decision table named in lim_var	Yes/no.
plant_com	The plant named in lim_var is in the plant community	= true, / false.
ch_order	Stream order of channel obj_num equals lim_const	Yes/no.
wtow_vol	Water in water tower obj_num, m^3	Target: lim_const × tower capacity.
vol	Reservoir volume, m^3	Target: lim_var pvol (principal volume) or evol (emergency volume) combined with lim_const by lim_op. With + and -, lim_const is in ha · m. With lim_var e-pv, the target is $V_p + \text{lim_const}\,(V_e - V_p)$.

var	Value compared	Notes
res_inflo	Inflow to reservoir obj_num, m^3/s	
wet_depth	Water depth of the HRU wetland or paddy	lim_const in mm.
weirh	Weir height of the HRU wetland or paddy	lim_const in mm.
vol_wet	Water volume of the HRU wetland or paddy, m^3	Target: lim_var pvol or evol combined with lim_const by lim_op.

Probability conditions

These three conditions let an operation happen on a random day within a window instead of on a fixed date.

prob_unif uses the columns differently from other conditions: obj is the probability f that the operation happens at all this year, obj_num is the first day of the window and lim_const is the last day (day of year). On the first day of the window SWAT+ decides, with probability f, whether the operation will happen this year, and if so draws one day in the window at random. The condition is true only on that day. A window that ends before it starts (for example 330 to 30) runs over the turn of the year.

prob_unif1 uses obj_num as a calendar year and lim_const as a number of days. The window starts on 1 January of that year. On each day of the window the condition is true with probability $1/n$, where n is the number of days left in the window.

prob_unif_lu uses obj_num as the first day and lim_const as the last day of the window. It is meant to be combined with a land_use condition in the same table: it spreads the operation over the HRUs of that land use so that the share of HRUs treated by day d follows a uniform distribution over the window.

Land use and management actions

Actions in lum.dtl. Unless stated otherwise, obj is hru and obj_num 0 means the HRU being checked.

act_typ	option	const	const2	fp	Effect
plant	plant name in the community	-	-	name in transplant.ops	Starts growth of the plant if it is not already growing. With a transplant entry, the plant starts from the transplant state (see management.sch).
harvest	plant name, or all	residue harvest efficiency	per year	name in harv.ops	Harvests the plant.
harvest_kill	plant name, or all	residue harvest efficiency	per year	name in harv.ops	Harvests and kills the plant.
kill	plant name, or all	-	per year	-	Kills the plant; biomass becomes residue.
till, tillage	name in tillage.til	-	per year	-	Tillage.
fertilize	name in fertilizer.frt	amount, kg/ha	per year	name in chem_app.ops	Applies fertilizer. On an HRU with standing water and a method with surf_frac 1, the fertilizer goes into the water.
fert_future	name in fertilizer.frt	amount, kg/ha	day of year	name in chem_app.ops	Schedules a fertilizer application on the given day. obj_num is the sequence number of the fert_future action in the table.
manure	-	amount, kg/ha	per year	-	In 62.0.1 the manure and method names are not looked up, so this action applies an empty fertilizer entry. Use fertilize with a manure from fertilizer.frt, or manure_demand.
manure_demand	name in chem_app.ops	amount, t/ha	per year	-	Sets a manure request for manure allocation. obj_num is the manure source number.
pest_apply	pesticide name listed in constituents.cs	amount, kg/ha	per year	name in chem_app.ops	Applies the amount times the method's app_eff.
graze	name in graze.ops	-	-	-	One day of grazing.
burn	name in fire.ops	-	per year	-	Burns plants and residue.

act_typ	option	const	const2	fp	Effect
irrigate	name in irr.ops	depth, mm	per year	-	Irrigates with const · eff_frac · (1 − sumq_frac). obj and obj_num name the source: aqu, cha or sdc, or res takes the water from that object and irrigates nothing if it has too little; any other obj (for example hru) is an unlimited source. With name ponding, the HRU is a paddy: const is the target ponding depth (mm), const2 the depth below which water is added, and the depth applied is the shortfall from the target.
irr_demand	name in irr.ops	depth, mm	per year	unlim or other	Sets the HRU's irrigation demand for water allocation (const× area). With fp unlim, the amt_mm of the irr.ops entry is applied directly. With name ponding, const and const2 are the target and threshold ponding depths (mm).
res_irr_dmd	-	depth, mm	-	-	Records an irrigation demand on reservoir obj_num.
rot_reset	-	rotation year	-	-	Sets the rotation year (minimum 1).
plant_reset, harv_reset, kill_reset, irr_reset	-	days	-	-	Sets the days since the last planting, harvest, kill or irrigation (minimum 1).
pheno_reset	-	-	per year	-	Restarts the phenology of plants with a moisture_gro trigger in plants.plt.
cn_update	absval, abschg, pctchg or relchg	value	per year	-	Changes the curve number for moisture condition II: set to the value, add the value, change by the value in percent, or multiply by (1 + value). The result is kept between 35 and 95.

act_typ	option	const	const2	fp	Effect
tiledep_control	-	drain depth, mm	per year	-	Sets the tile drain depth for drainage water management.
impound_on	-	-	-	name in wetland.wet	Turns on impounded water (paddy or wetland) on the HRU.
impound_off	-	-	-	-	Turns off impounded water on the HRU.
weir_height	wet, or other	height, mm	-	-	With wet, sets the HRU paddy weir height and the principal and emergency volumes from it. Otherwise sets the weir height of reservoir obj_num.
puddle	name in puddle.ops	-	per year	-	Puddles a paddy (see management.sch). The action name must be the tillage operation in tillage.til used for the soil mixing.
lu_change	-	-	-	name in landuse.lum	Changes the HRU's land use and management and reinitialises plants and curve number. Logged in lu_change_out.txt.
snow_change	-	-	-	name in snow.sno	Switches the HRU to another snow parameter set. Logged in lu_change_out.txt.
hru_fr_update	new ls_unit.ele file	-	-	new rout_unit.ele file	Updates HRU area fractions. Logged in lu_change_out.txt.
p_factor	-	new USLE P	-	-	Sets the USLE P factor.
contour	-	-	-	-	Sets $P = 2.7\,S + 0.26$, with S the slope (m/m).
stripcrop	-	-	-	pasture or other	Sets $P = 2.32\,S + 0.36$, halved when fp is pasture.
terrace	-	-	-	sod_outlet or other	Sets $P = 1.4\,S + 0.13$, doubled when fp is sod_outlet.
tile_install	-	-	-	name in tiledrain.str	Installs tile drains (see *.str).
septic_install	-	-	-	name in septic.str	Installs a septic system.
fstrip_install	-	-	-	name in filterstrip.str	Installs a filter strip.

act_typ	option	const	const2	fp	Effect
grassww_install, grassww_uninstall	-	-	-	name in grassedww.str	Installs or removes a grassed waterway.
user_def_bmp	-	-	-	name in bmpuser.str	Applies user-defined BMP removal fractions.
grow_init, grow_end	-	-	-	-	Start and end of the growing season of an HRU-lte (**obj hlt**). **grow_end** computes the yield.
chan_change	-	cover	stream order	-	Sets the cover and stream order of the channel being checked.
herd	-	-	-	-	No effect in 62.0.1.

The `plant`, `harvest` and `harvest_kill` actions of the generic tables `pl_hv_summer1`, `pl_hv_winter1` and `pl_hv_summer2` take the plant name from `management.sch` instead of `option` (see `management.sch`).

Flow control and water allocation actions, used in `flo_con.dtl`:

act_typ	option	const	const2	Effect
`tileflo_control`	`flo_mm`, `max_mm` (at most `const` mm), `min_mm` (at least `const` mm), `all_flo`, `zero_flo`, `frac` (`const` × tile flow)	mm or fraction	-	Sets the tile flow sent from a saturated buffer source HRU to its receiving HRU.
`wallo_dmd`	`set_dmd`	demand, m^3	-	Sets the demand of a water allocation place of use.
`wallo_dmd`	`irr_dmd`	-	-	Adds the irrigation demand of place of use `obj_num`.
`wallo_dmd`	`fill_res`, `fill_wtow`, `fill_can`	target fraction of capacity	fraction of the shortfall per day	Demand to fill reservoir, water tower or canal `obj_num` to `const` times its capacity (for `fill_res`, `fp` `pvol` or `evol`).
`duty_fr`	`fr1` to `fr6`	fraction	-	Fraction of the demand taken from point of diversion 1 to 6 (or sent to return 1 to 6).
`duty_reset`	-	point of diversion number	new maximum withdrawal	Resets the withdrawal limit of a point of diversion of place of use `obj_num`.

See water allocation for how these demands and fractions are used.

Reservoir release actions

In `res_rel.dtl` the action type is `release` and `option` selects how the release is computed. The releases of all actions that run on a day are added together. A table with zero alternatives runs all its actions every day. V is the current volume, V_p the principal and V_e the emergency volume.

option	const	const2	fp	Release
`rate`	rate, m^3/s	-	-	constant rate
`rate_pct`	percent	-	-	`const` % of V_p per day
`inflo_rate`	maximum rate, m^3/s	change to inflow, m^3/s	-	the larger of inflow + `const2` and `const`
`inflo_frac`	fraction	-	-	`const` × inflow
`ab_emer`	-	-	-	all volume above V_e
`days`	days	multiplier	`null`, `pvol` or `evol`	$(V - V_t)$/const, with target $V_t = 0$, `const2` × V_p or `const2` × V_e
`dyrt`	days	percent	`null`, `pvol` or `evol`	$(V - V_t)$/const plus `const2` % of V_p, with $V_t = 0$, V_p or V_e
`dyrt1`	days	percent	`con1`, `con2` or `con3`	as `dyrt`, with the target taken from condition 5, 4 or 3 of the table
`inflo_targ`	condition number	-	-	inflow plus all volume above V_p × that condition's `lim_const`

option	const	const2	fp	Release
weir	-	-	name in weir.res	weir equation
meas	-	-	name of a recall object	measured outflow
natlake	inactive storage, fraction of V_p	release coefficient, 1/day	-	natural lake outflow (Döll et al., 2003)
nonirr-h06	α	-	-	Hanazaki et al. (2006) scheme for non-irrigation reservoirs
irr-h06	α	β	-	Hanazaki et al. (2006) scheme for irrigation reservoirs
hydrop	amplitude	phase	-	HYPE hydropower scheme (Arheimer et al., 2019)

See *.res for the reservoir side of these options.

Land use scenarios

Tables in scen_lu.dtl run only if they are listed in scen_dtl.upd. That file has a title line, the number of entries, a header line, and one row per entry with three columns: max_hits, typ and the name of a table in scen_lu.dtl. Each listed table is checked for every HRU every day, before the HRUs are simulated. The usual actions are lu_change and snow_change.

Example

refdata/Ames_sub1/lum.dtl in the SWAT+ repository contains two tables:

```
lum.dtl: yearly irrigation limit example for Ames dataset
2

name                      conds      alts      acts       !limit yearly irrigation to 500 mm using irrigate
irr_year_irr                 4         1         1
var                        obj   obj_num           lim_var            lim_op  lim_const      alt1
irr_year                   hru         0              null                 -   500.00000         <
w_stress                   hru         0              null                 -   0.80000           <
phu_plant                  hru         0            phu_mat                -     0.20000         >
leaf_area                  hru         0              null                 -     0.50000         >
act_typ                    obj   obj_num              name            option    const        const2
irrigate                   hru         0          sprinkler     sprinkler_ilm    25.00000   20.00000  unli
```

Table irr_year_irr has four conditions, one alternative and one action. The alternative is satisfied on a day when the irrigation applied so far is below 500 mm, the average water stress factor is below 0.8, the crop has accumulated more than 20% of its heat units, and the leaf area index is above 0.5. On such a day the HRU is irrigated with 25 mm using the sprinkler_ilm entry of irr.ops, at most 20 times a year. obj is hru, so the water comes from an unlimited source. The table is used by naming irr_year_irr as an automatic operation in management.sch. In 62.0.1 the irrigation total behind irr_year is not reset each year, so once 500 mm has been applied over the whole run, the table stops irrigating for the rest of the simulation.

A reservoir table from refdata/Osu_1hru/res_rel.dtl, followed by the blank line that res_rel.dtl needs:

```
name                      conds      alts             acts
paddy_auto_rel               2         2                2
var                        obj   obj_num           lim_var            lim_op     lim_const      alt1    alt
wet_depth                  hru         0          weir_hgt                 -       0.00000         >
wet_depth                  hru         0              null                 -       2.00000         >
```

act_typ	obj	obj_num	name	option	const	const2
release	hru	0	above_emer	days	1.00000	0.00000
release	hru	0	below_emer	weir	10.00000	0.00000

Both alternatives need a water depth above 2 mm, and the first also above 0 mm. When the first alternative is satisfied, the paddy releases its whole volume within one day (`days` with `const` 1 and target 0) and also releases over the weir `weir1`.

Related files

- `management.sch`: lists the `lum.dtl` tables each HRU uses.
- `*.ops`, `tillage.til`, `fertilizer.frt`, `landuse.lum`, `snow.sno`, `*.str`: databases named by actions.
- `*.res` and `*.wet`: name the release tables in `res_rel.dtl`.
- Water allocation and manure allocation: name tables in `lum.dtl` and `flo_con.dtl`.
- `file.cio`: names the four files on its `decision_table` line.

Sources

- Readers: `dtbl_lum_read.f90`, `dtbl_res_read.f90`, `dtbl_scen_read.f90`, `dtbl_flocon_read.f90`, `cal_cond_read.f90` (`scen_dtl.upd`)
- Type definitions: `conditional_module.f90`
- Conditions: `conditions.f90`, `cond_real.f90`, `cond_integer.f90`
- Actions: `actions.f90`; reservoir releases: `res_hydro.f90`
- Linking tables to HRUs: `proc_cond.f90`, `hru_dtbl_actions_init.f90`
- Callers: `hru_control.f90`, `time_control.f90`, `res_control.f90`, `wallo_start_day.f90`, `mallo_control.f90`
- File name slots: `type input_condition` in `input_file_module.f90`

Column	Fortran variable
condition `var`, `obj`, `obj_num`, `lim_var`, `lim_op`, `lim_const`	`d_tbl%cond(ic)%var`, `%ob`, `%ob_num`, `%lim_var`, `%lim_op`, `%lim_const`
`alt1` …	`d_tbl%alt(ic, ialt)`
`prob_unif` probability (`obj` column)	`d_tbl%frac_app`
action `act_typ`	`d_tbl%act(iac)%typ`
action `obj`, `obj_num`, `name`, `option`, `const`, `const2`	`d_tbl%act(iac)%ob`, `%ob_num`, `%name`, `%option`, `%const`, `%const2`
action `fp`	`d_tbl%act(iac)%file_pointer`
`outcome1` …	`d_tbl%act_outcomes(iac, ialt)`

Purpose

`plants.plt` is the plant growth database. Each row is one plant species, crop variety or land cover, with the parameters SWAT+ needs to grow it: plant type, heat units to maturity, leaf area development, radiation use efficiency, canopy height, rooting depth, temperature limits, nutrient content, stomatal conductance and residue properties.

Plant communities in `plant.ini`, plant operations in `management.sch` and decision tables, and land uses in `landuse.lum` all refer to rows of this file by name. The names are also used by the GIS interfaces to link land cover maps to SWAT+ land uses.

The file is named on the `hru_parm_db` line of `file.cio` (default `plants.plt`). If the file is missing, the database is empty.

The database distributed with SWAT+ has parameters for most common crops, grasses, trees and land covers. If a land use or plant is not in it, start from a row for a plant with similar growth habits.

Format

Free format, whitespace-separated. Line 1 is a title and line 2 is the column header; both are skipped. Each later line is one plant with 56 values in the order below. The reader does not match columns by header name, so every value must be present and in this order. It ignores anything after the 56th value, so the `description` column written by the SWAT+ Editor has no effect.

If the `event` column of `codes.bsn` is not 0, one more word is read after the 56th value: a plant class name of up to 25 characters (for example `row_crop`). In that case the plant class takes the place of the description.

#	Column	Units	Default	Description
1	`name`	-	-	Name of the plant or land cover, up to 40 characters. Must be unique.
2	`plnt_typ`	-	-	Plant type: `warm_annual`, `cold_annual`, `warm_annual_tuber`, `cold_annual_tuber` or `perennial`. See Plant types.
3	`gro_trig`	-	-	Growth trigger: `temp_gro` (growth driven by temperature and day length) or `moisture_gro` (tropical plants whose growth starts with the monsoon).
4	`nfix_co`	-	0.0	Nitrogen fixation coefficient: 0.5 for legumes, 0 for non-legumes. Any value above 0 lets the plant fix nitrogen when soil nitrate cannot meet its demand.
5	`days_mat`	days	110	Days to maturity, an integer. Used to compute the heat units to maturity. 0 means the plant uses the heat units of the whole year. See Heat units to maturity.
6	`bm_e`	(kg/ha)/(MJ/m²)	15.0	Biomass-energy ratio (radiation use efficiency): dry biomass produced, including roots, per unit of intercepted photosynthetically active radiation, with no stress, at ambient CO2 and a vapor pressure deficit of 1 kPa.
7	`harv_idx`	(kg/ha)/(kg/ha)	0.76	Harvest index for optimal growing conditions: fraction of above-ground biomass removed as yield. Values above 0.7 are replaced by 0.02 (see Values set by the model).
8	`lai_pot`	m²/m²	5.0	Maximum potential leaf area index. Values are limited to 0 to 10.
9	`frac_hu1`	fraction	0.05	Fraction of heat units to maturity at the 1st point of the optimal leaf area development curve.
10	`lai_max1`	fraction	0.05	Fraction of `lai_pot` at the 1st point of the optimal leaf area development curve.
11	`frac_hu2`	fraction	0.40	Fraction of heat units to maturity at the 2nd point of the optimal leaf area development curve.
12	`lai_max2`	fraction	0.95	Fraction of `lai_pot` at the 2nd point of the optimal leaf area development curve.
13	`hu_lai_decl`	fraction	0.99	Fraction of heat units to maturity at which leaf area starts to decline.
14	`dlai_rate`	-	1.0	Exponent of the leaf area decline of annuals after `hu_lai_decl`. 1 gives a linear decline.
15	`can_ht_max`	m	6.0	Maximum canopy height.
16	`rt_dp_max`	m	3.5	Maximum rooting depth. The soil profile depth also limits roots.
17	`tmp_opt`	°C	30.0	Optimal temperature for plant growth.
18	`tmp_base`	°C	10.0	Base (minimum) temperature for plant growth. Heat units accumulate only above it.
19	`frac_n_yld`	kg N/kg yield	0.0015	Fraction of nitrogen in the yield, dry weight.

#	Column	Units	Default	Description
20	`frac_p_yld`	kg P/kg yield	0.0003	Fraction of phosphorus in the yield, dry weight.
21	`frac_n_em`	kg N/kg biomass	0.006	Normal fraction of nitrogen in plant biomass at emergence.
22	`frac_n_50`	kg N/kg biomass	0.002	Normal fraction of nitrogen in plant biomass at 50% maturity.
23	`frac_n_mat`	kg N/kg biomass	0.0015	Normal fraction of nitrogen in plant biomass at maturity.
24	`frac_p_em`	kg P/kg biomass	0.0007	Normal fraction of phosphorus in plant biomass at emergence.
25	`frac_p_50`	kg P/kg biomass	0.0004	Normal fraction of phosphorus in plant biomass at 50% maturity.
26	`frac_p_mat`	kg P/kg biomass	0.0003	Normal fraction of phosphorus in plant biomass at maturity.
27	`harv_idx_ws`	(kg/ha)/(kg/ha)	0.01	Harvest index under severe water stress: the lowest harvest index expected, between 0 and `harv_idx`.
28	`usle_c_min`	-	0.001	Minimum USLE C factor for water erosion. Values are limited to 0.001 to 1.
29	`stcon_max`	m/s	0.002	Maximum stomatal conductance at high solar radiation and low vapor pressure deficit. Used with the Penman-Monteith ET method.
30	`vpd`	kPa	4.0	Vapor pressure deficit at the 2nd point of the stomatal conductance curve.
31	`frac_stcon`	fraction	0.75	Fraction of `stcon_max` at the 2nd point of the stomatal conductance curve.
32	`ru_vpd`	(kg/ha)/(MJ/m²)/kPa	8.0	Rate of decline in radiation use efficiency per unit increase in vapor pressure deficit above 1 kPa. 0 turns the adjustment off.
33	`co2_hi`	µL CO2/L air	660.0	Elevated CO2 concentration at the 2nd point of the radiation use efficiency curve.
34	`bm_e_hi`	(kg/ha)/(MJ/m²)	16.0	Biomass-energy ratio at the CO2 concentration `co2_hi`.
35	`plnt_decomp`	-	0.05	Plant residue decomposition coefficient: fraction of residue that decomposes in a day under optimal moisture, temperature, C:N and C:P. 0 takes `rsd_decomp` from `parameters.bsn`.
36	`lai_min`	m²/m²	0.75	Minimum leaf area index of perennials during the dormant period.
37	`bm_tree_acc`	-	0.3	Exponent that limits the maximum leaf area index of young perennials. See Perennials and trees.
38	`yrs_mat`	years	10	Years for a perennial to reach full development, an integer. Values below 1 are set to 1.
39	`bm_tree_max`	t/ha	1000.0	Maximum above-ground biomass. Checked once a year.
40	`ext_co`	-	0.65	Light extinction coefficient for intercepted photosynthetically active radiation. 0 is replaced by 0.65.
41	`leaf_tov_mn`	months	12.0	Leaf turnover time of `moisture_gro` perennials with minimum stress. Called `leaf_tov_min` in earlier documentation.
42	`leaf_tov_mx`	months	3.0	Leaf turnover time of `moisture_gro` perennials with maximum stress. Called `leaf_tov_max` in earlier documentation.
43	`bm_dieoff`	fraction	0.0	Factor on the excess biomass removed when above-ground biomass exceeds `bm_tree_max`. 0 is replaced by 1.
44	`rt_st_beg`	fraction	0.0	Root to shoot ratio at the start of the growing season. 0 is replaced by 0.4.

#	Column	Units	Default	Description
45	rt_st_end	fraction	0.0	Root to shoot ratio at the end of the growing season. 0 is replaced by 0.2. Tuber types always use 0.7.
46	plnt_pop1	plants/m²	0.0	Plant population at the 1st point of the population and leaf area curve.
47	frac_lai1	fraction	0.0	Fraction of lai_pot at the 1st point of the population and leaf area curve.
48	plnt_pop2	plants/m²	0.0	Plant population at the 2nd point of the population and leaf area curve.
49	frac_lai2	fraction	0.0	Fraction of lai_pot at the 2nd point of the population and leaf area curve.
50	frac_sw_gro	-	0.5	Intended as the 30-day precipitation to PET ratio that starts growth of moisture_gro plants. Read but not used, which uses a fixed 0.5.
51	aeration	fraction	0.2	Aeration stress threshold: the fraction of pore space above field capacity that can fill with water before the plant is stressed by lack of air. 0 is replaced by 0.2.
52	rsd_pctcov	-	0.0	Residue factor for the percent cover equation. 0 is replaced by 0.4.
53	rsd_covfac	-	0.0	Residue factor for the surface cover (C factor) equation. 0 is replaced by 0.04.
54	avg_lig_frac	fraction	0.85	Average lignin fraction of residue. Read but not used.
55	ab_lig_frac	fraction	0.15	Lignin fraction of above-ground residue. Used by the dynamic carbon model only.
56	bg_lig_frac	fraction	0.12	Lignin fraction of below-ground residue (roots). Used by the dynamic carbon model only.

The defaults are the values SWAT+ uses for a component that is not set. They are not filled in for missing columns, since every row must have all 56 values.

Plant types

plnt_typ decides how the plant goes through the year.

- warm_annual and cold_annual are annual crops. A cold_annual is planted in the fall and goes dormant over winter when day length falls below the dormancy threshold. Leaf area declines after hu_lai_decl, following dlai_rate.
- warm_annual_tuber and cold_annual_tuber are annuals whose yield is below ground. Their root to shoot ratio at the end of the season is fixed at 0.7.
- perennial covers grasses, shrubs and trees. Perennials go dormant when day length falls below the threshold, keep at least lai_min of leaf area, and develop over yrs_mat years.

gro_trig = moisture_gro is for tropical plants. If the plant was planted with the monsoon option, it starts growing when the 30-day ratio of precipitation to potential ET reaches 0.5. Leaf turnover of moisture_gro perennials varies between leaf_tov_mx and leaf_tov_mn months with moisture stress.

Heat units to maturity

SWAT+ computes the heat units a plant needs to reach maturity at the start of the run, from the monthly mean temperatures of the HRU's weather generator station.

- days_mat = 0: the plant grows through the whole year. Heat units to maturity are 95% of the year's sum of mean daily temperature above tmp_base.
- days_mat > 0 for an annual: the planting date is taken as the day when the sum of mean daily temperatures above 0 °C reaches 15% of the annual sum. Heat units above tmp_base are then summed for days_mat days from that date.

The calculation works in both hemispheres. Giving the days of a variety (for example 90, 100, 110 or 120 days for corn) lets one database support several varieties. If `days_mat` is longer than the growing season at a site, the heat units of the whole season are used.

Leaf area development

The optimal leaf area index follows a curve through two points, (`frac_hu1`, `lai_max1`) and (`frac_hu2`, `lai_max2`), where the first value is the fraction of heat units to maturity and the second the fraction of `lai_pot`:

$$f_{LAI} = \frac{f_{PHU}}{f_{PHU} + \exp(l_1 - l_2\, f_{PHU})}$$

l_1 and l_2 are set at the start of the run so that the curve passes through both points. The values of `lai_pot` in the standard database are for typical plant densities in rainfed agriculture. Drought-prone regions with low planting densities, or irrigated land with high densities, may need different values. Canopy height grows with the square root of f_{LAI} up to `can_ht_max`.

When `plnt_pop1` and `plnt_pop2` are given, a second curve through (`plnt_pop1`, `frac_lai1`) and (`plnt_pop2`, `frac_lai2`) limits `lai_pot` by plant population.

Radiation use efficiency

When the `co2` column of `parameters.bsn` is above 350 µL/L, radiation use efficiency is adjusted for CO2 along a curve through (330 µL/L, `bm_e`) and (`co2_hi`, `bm_e_hi`). Otherwise `bm_e` is used as given. Values for `co2_hi` and `bm_e_hi` are needed even when climate change is not simulated, and `bm_e_hi` should be larger than `bm_e`. A `co2_hi` of 330 is replaced by 660.

When the vapor pressure deficit is above 1 kPa, radiation use efficiency drops by `ru_vpd` for each kPa above 1, down to no less than 27% of `bm_e`. A value of 6 to 8 is a reasonable approximation for most plants, and 0 turns the adjustment off.

Perennials and trees

The maximum leaf area index of a perennial grows with its age:

$$LAI_{max} = \text{lai_pot} \left(\frac{y}{\text{yrs_mat}} \right)^{\text{bm_tree_acc}}$$

where y is the number of years since the plant was established (at least 1), and LAI_{max} is at most `lai_pot`. With `bm_tree_acc` = 0 the plant reaches `lai_pot` in its first year.

At the end of each year, if above-ground biomass B exceeds `bm_tree_max` (converted to kg/ha), the amount

$$D = (1 + \text{bm_dieoff})\,(B - 1000\,\text{bm_tree_max})$$

dies and goes to the residue pools, with roots in the same proportion. Nothing dies if D is not smaller than B. A mature forest stand typically has 30 to 50 t/ha of biomass. `bm_dieoff` is not used at the start of dormancy in 62.0.1.

Nutrient uptake

The six fractions `frac_n_em` to `frac_p_mat` describe the nitrogen and phosphorus content of the whole plant, roots included, on a dry weight basis, at emergence, at 50% maturity and at maturity. They set the plant's nutrient demand through the season. Each fraction must be smaller than the one before it: if `frac_n_50` is not at least 0.0001 below `frac_n_em`, it is set to `frac_n_em` − 0.0001, and if `frac_n_mat` is not at least 0.0001 below `frac_n_50`, it is multiplied by 0.75. The phosphorus fractions are treated the same way.

Residue lignin

With `carbon` = 2 in codes.bsn, residue entering the soil is split into metabolic, structural and lignin pools. Below ground, the lignin fraction is `bg_lig_frac`, the structural fraction is `bg_lig_frac` / 0.8, and the rest is metabolic. Above ground the same split uses `ab_lig_frac`. With `carbon` = 0 these columns are not used.

Values set by the model

After the file is read, SWAT+ changes some values:

Column	Change
`harv_idx`	Values above 0.7 are set to 0.02.
`lai_pot`	Limited to 0 to 10.
`usle_c_min`	Limited to 0.001 to 1.
`plnt_decomp`	0 is replaced by `rsd_decomp` of parameters.bsn.
`yrs_mat`	Values below 1 are set to 1.
`ext_co`	0 is replaced by 0.65.
`bm_dieoff`	0 is replaced by 1.
`rt_st_beg`, `rt_st_end`	0 is replaced by 0.4 and 0.2. Tuber types get `rt_st_end` = 0.7.
`aeration`, `rsd_pctcov`, `rsd_covfac`	0 is replaced by 0.2, 0.4 and 0.04.
`co2_hi`	330 is replaced by 660.
`frac_n_50`, `frac_n_mat`, `frac_p_50`, `frac_p_mat`	Adjusted to decrease, as described above.

The `harv_idx` change affects crops whose harvest index is above 0.7, such as alfalfa (0.9 in the standard database) and tuber crops with a harvest index above 1. Their harvest index becomes 0.02.

Calibration

These parameters can be changed during calibration through calibration.cal: `bio_e` (`bm_e`), `hvsti` (`harv_idx`), `rdmx` (`rt_dp_max`), `t_opt` (`tmp_opt`), `t_base` (`tmp_base`) and `usle_c` (`usle_c_min`).

Example

The first rows of `refdata/Ames_sub1/plants.plt` in the SWAT+ repository:

```
plants.plt
name  plnt_typ  gro_trig  nfix_co  days_mat  bm_e  harv_idx  lai_pot  frac_hu1  lai_max1  frac_hu2  lai_ma
alfa  perennial  temp_gro  0.5  0  20  0.9  4  0.15  0.01  0.5  0.95  0.9  0.5  0.9  3  20  4  0.02  0.003
cana  warm_annual  temp_gro  0  100  34  0.3  4.5  0.15  0.02  0.45  0.95  0.5  0.3  1.3  1.4  21  5  0.03
```

- `alfa` (alfalfa) is a perennial legume (`nfix_co` = 0.5) with `days_mat` = 0, so it uses the heat units of the whole year. Its `harv_idx` of 0.9 is replaced by 0.02.
- `alfa` has `frac_n_50` = `frac_n_mat` = 0.02, so `frac_n_mat` is multiplied by 0.75 to 0.015.
- `bm_tree_max` is 0 for both rows. With `bm_dieoff` = 0.1, $D = 1.1\,B$ at the end of the year, which is not smaller than B, so no biomass dies off.
- Ames_sub1 has `event` = 0 and `carbon` = 2 in codes.bsn. No plant class is read, so the `description` column is ignored, and the lignin columns are used: for `alfa`, 9% of decomposing surface residue and 20% of root residue entering the soil goes to the lignin pool. `avg_lig_frac` is read but has no effect.

Related files

- plant.ini: plant communities for HRUs, by plant name.
- management.sch and decision tables: plant, harvest and kill operations name plants from this file.
- landuse.lum: land uses and their plant communities.
- codes.bsn: `event` switches the plant class column on; `carbon` selects whether the lignin fractions are used.

- parameters.bsn: rsd_decomp replaces a plnt_decomp of 0.
- Weather generator: monthly temperatures used for heat units to maturity.

Sources

- Reader: plant_parm_read.f90
- Type definition: type plant_db and type residue_partition_fracs in plant_data_module.f90
- Derived values and overrides: plantparm_init.f90
- Heat units to maturity: plant_init.f90
- Growth: pl_leaf_gro.f90, pl_leaf_senes.f90, pl_root_gro.f90, pl_biomass_gro.f90, pl_dormant.f90, pl_mortality.f90, pl_nfix.f90, pl_waterup.f90
- Monsoon trigger: hru_control.f90
- Residue lignin: cbn_rsd_transfer.f90, cbn_surfrsd_decomp.f90
- Calibration parameters: cal_parm_select.f90

Column	Fortran variable
name	pldb%plantnm
plnt_typ	pldb%typ
gro_trig	pldb%trig
days_mat	pldb%days_mat
bm_e	pldb%bio_e
harv_idx	pldb%hvsti
lai_pot	pldb%blai
frac_hu1, lai_max1	pldb%frgrw1, pldb%laimx1
frac_hu2, lai_max2	pldb%frgrw2, pldb%laimx2
hu_lai_decl	pldb%dlai
can_ht_max	pldb%chtmx
rt_dp_max	pldb%rdmx
tmp_opt, tmp_base	pldb%t_opt, pldb%t_base
frac_n_yld, frac_p_yld	pldb%cnyld, pldb%cpyld
frac_n_em, frac_n_50, frac_n_mat	pldb%pltnfr1, pltnfr2, pltnfr3
frac_p_em, frac_p_50, frac_p_mat	pldb%pltpfr1, pltpfr2, pltpfr3
harv_idx_ws	pldb%wsyf
usle_c_min	pldb%usle_c
stcon_max	pldb%gsi
vpd	pldb%vpdfr
frac_stcon	pldb%gmaxfr
ru_vpd	pldb%wavp
co2_hi, bm_e_hi	pldb%co2hi, pldb%bioehi
plnt_decomp	pldb%rsdco_pl
lai_min	pldb%alai_min
bm_tree_acc	pldb%laixco_tree
yrs_mat	pldb%mat_yrs
bm_tree_max	pldb%bmx_peren
ext_co	pldb%ext_coef
leaf_tov_mn, leaf_tov_mx	pldb%leaf_tov_min, pldb%leaf_tov_max
rt_st_beg, rt_st_end	pldb%rsr1, pldb%rsr2
plnt_pop1, frac_lai1	pldb%pop1, pldb%frlai1
plnt_pop2, frac_lai2	pldb%pop2, pldb%frlai2
frac_sw_gro	pldb%frsw_gro
avg_lig_frac	pldb%res_part_fracs%meta_frac
ab_lig_frac	pldb%res_part_fracs%str_frac
bg_lig_frac	pldb%res_part_fracs%lig_frac
plant class (event /= 0)	pl_class
event in codes.bsn	bsn_cc%nam1

Purpose

Fertilizer and manure applications use three databases.

- `fertilizer.frt` lists the fertilizer products: the fractions of mineral and organic nitrogen and phosphorus in each, and how much of the mineral nitrogen is ammonium. `fert` operations in `management.sch` and `fertilize` actions in decision tables name a row of this file.
- `manure_db.frt` lists the manure types. Each one points to a row of `manure_om.frt` and names optional pesticide, pathogen, metal, salt and constituent profiles. `manu` operations in `management.sch`, `manure` actions in decision tables and the manure allocation file name a row of this file.
- `manure_om.frt` gives the water, carbon and nutrient make-up of each manure.

`fertilizer.frt` is named on the `hru_parm_db` line of `file.cio` (default `fertilizer.frt`). The two manure files are not listed in `file.cio`. SWAT+ looks for them under the fixed names `manure_db.frt` and `manure_om.frt`, and leaves the manure databases empty when they are missing.

fertilizer.frt

Free format, whitespace-separated. Line 1 is a title and line 2 is the column header; both are skipped. Each later line is one product with the values below in this order. The reader does not match columns by header name. It reads the first six values and ignores the rest of the line, so the `pathogens` and `description` columns written by the SWAT+ Editor have no effect.

#	Column	Units	Default	Description
1	`name`	-	-	Name of the fertilizer, up to 16 characters. Must be unique.
2	`min_n`	kg mineral N/kg fertilizer	0.0	Fraction of the fertilizer that is mineral nitrogen (nitrate plus ammonium).
3	`min_p`	kg mineral P/kg fertilizer	0.0	Fraction of the fertilizer that is mineral phosphorus.
4	`org_n`	kg organic N/kg fertilizer	0.0	Fraction of the fertilizer that is organic nitrogen.
5	`org_p`	kg organic P/kg fertilizer	0.0	Fraction of the fertilizer that is organic phosphorus.
6	`nh3_n`	kg NH3-N/kg mineral N	0.0	Fraction of the mineral nitrogen that is ammonium. The rest is nitrate.

The fractions are by weight of the product. `min_n` + `min_p` + `org_n` + `org_p` is the nutrient content and should not exceed 1; the remainder is carrier.

For a commercial fertilizer the grade gives the nitrogen and phosphate content. The phosphate value in a grade is P2O5, so it has to be converted to P (multiply by 0.44) before it goes into `min_p`.

Fertilizer application

A fertilizer operation applies an amount in kg/ha of product. The application method row in `chem_app.ops` gives the fraction placed in the top 10 mm of soil (`surf_frac`); the rest goes into the second soil layer. In each of the two layers:

- `min_n` × (1 − `nh3_n`) of the amount is added as nitrate and `min_n` × `nh3_n` as ammonium.
- `min_p` of the amount is added as labile (soluble) phosphorus.
- With `carbon` = 0 in `codes.bsn`, half of the organic N and P goes to the fresh residue pool and half to the active humus pool.
- With `carbon` = 2, organic N and P are added to the soil organic matter pools with a carbon content of 10 times the organic N. The share that goes to the metabolic and structural litter pools depends on the C:N ratio. The split between the fresh and humus parts is `manure_c_frac` in `carbon.bsn`.

fertilizer.frt example

The first rows of `refdata/Ames_sub1/fertilizer.frt` in the SWAT+ repository:

```
fertilizer.frt: written by SWAT+ editor v2.2.2 on 2023-10-17 14:43 for SWAT+ rev.60.5.4
name                      min_n         min_p         org_n         org_p         nh3_n         pathogens  
```

```
elem_n          1.00000     0.00000     0.00000     0.00000     0.00000          null  l
elem_p          0.00000     1.00000     0.00000     0.00000     0.00000          null  l
anh_nh3         0.82000     0.00000     0.00000     0.00000     1.00000          null  A
urea            0.46000     0.00000     0.00000     0.00000     1.00000          null  U
18_46_00        0.18000     0.20200     0.00000     0.00000     0.00000          null
dairy_fr        0.00700     0.00500     0.03100     0.00300     0.99000          null  l
```

- `urea` is 46% mineral N, all of it applied as ammonium.
- `18_46_00` is an 18-46-0 grade: 18% N and 46% P2O5, which is 20.2% P.
- `dairy_fr` is fresh dairy manure, mostly organic N.
- `elem_n` has `nh3_n` = 0, so all its nitrogen is applied as nitrate.

manure_db.frt

Free format, whitespace-separated. Line 1 is a title and line 2 is the column header; both are skipped. Each later line is one manure type with eight values.

#	Column	Description
1	`name`	Name of the manure type, up to 25 characters. Manure operations and the manure allocation file refer to it by this name.
2	`org_min`	Name of a row in `manure_om.frt` that gives the water, carbon and nutrient make-up.
3	`pests`	Name of a pesticide profile, or `null`.
4	`paths`	Name of a pathogen profile, or `null`.
5	`hmets`	Name of a heavy metal profile, or `null`.
6	`salts`	Name of a salt profile, or `null`.
7	`constit`	Name of a profile for other constituents, or `null`.
8	`description`	Free text, one word.

In 62.0.1 the `pests`, `paths`, `hmets`, `salts` and `constit` names are read but not looked up. Salt and constituent loads in manure come from the fertilizer files of the salt and constituent modules, taken from the row in the same position as the manure type. They are added only when the manure type's `org_min` names the second or a later row of `manure_om.frt`.

A manure application takes its make-up from the row of `manure_om.frt` in the same position as the manure type in `manure_db.frt` (first with first, second with second). The `org_min` name is matched against `manure_om.frt` when the file is read, but the application does not use the match. Keep the two files in the same order, with `org_min` naming the row in the same position.

manure_db.frt example

`refdata/Osu_1hru/manure_db.frt`:

```
manure_db.frt: manure database
name          org_min            pests  paths  hmets  salts  constit  description
beef_salt     mw_beef_liq        null   null   null   null   null     midwest_beef_liquid_salt
```

One manure type, `beef_salt`, using the `mw_beef_liq` make-up. `mw_beef_liq` is also the first row of `manure_om.frt` below, so the position used by the application and the `org_min` name agree. Because `org_min` names the first row, no salt or constituent loads are added when `beef_salt` is applied, whatever the salt and constituent fertilizer files contain.

manure_om.frt

Free format, whitespace-separated. Line 1 is a title and line 2 is the column header; both are skipped. Each later line is one manure make-up with nine values.

#	Column	Units	Description
1	`name`	-	Name of the make-up, up to 64 characters.
2	`frac_water`	kg water/kg manure as applied	Water fraction of the manure as applied. Must be below 1.
3	`fcbn`	kg C/kg dry manure	Carbon fraction.
4	`fminn`	kg mineral N/kg dry manure	Mineral nitrogen fraction (nitrate plus ammonium).
5	`fminp`	kg mineral P/kg dry manure	Mineral phosphorus fraction.
6	`forgn`	kg organic N/kg dry manure	Organic nitrogen fraction.
7	`forgp`	kg organic P/kg dry manure	Organic phosphorus fraction.
8	`fnh3n`	kg NH3-N/kg mineral N	Fraction of the mineral nitrogen that is ammonium.
9	`description`	-	Free text, one word.

Manure application

The amount in a manure operation is the dry mass of manure in kg/ha. The nutrient fractions apply to that dry mass. The water that comes with it, amount / (1 − `frac_water`) − amount, is added to the soil water of the two top layers. As for fertilizer, `surf_frac` of the application method in `chem_app.ops` goes to the top 10 mm and the rest to the second layer.

Mineral N and P are added the same way as for fertilizer. With `carbon` = 0, the organic N and P are split between the fresh and humus pools with a fixed fraction of 0.5. With `carbon` = 2, carbon from `fcbn` and organic N and P go to the manure and litter pools, and the split uses `manure_c_frac` in `carbon.bsn`.

manure_om.frt example

`refdata/Osu_1hru/manure_om.frt`:

```
manure_om.frt: manure organic matter database
name            frac_water   fcbn          fminn         fminp         forgn         forgp         fnh3n  c
mw_beef_liq     0.988        0.000780576   0.000311751   0.00081295    0.002110312   0.000365707   0.99   r
mw_beef_slur    0.926        0.016786571   0.002697842   0.00081295    0.002110312   0.000365707   0.99   r
```

Two make-ups of Midwest beef manure. The liquid form is 98.8% water and the slurry 92.6%. Applying 1000 kg/ha of `mw_beef_liq` adds about 82,300 kg/ha of water, or 8.2 mm.

Related files

- `management.sch`: `fert` and `manu` operations.
- Decision tables: `fertilize` and `manure` actions.
- `chem_app.ops`: application methods and the surface fraction.
- `manure_allo.mnu`: moves manure from sources to the land.
- `codes.bsn`: the `carbon` column selects how organic nutrients enter the soil.
- `carbon.bsn`: `manure_c_frac` for the dynamic carbon model.

Sources

- Readers: `fert_parm_read.f90`, `manure_db_read.f90`, `manure_orgmin_read.f90`, called from `proc_db.f90`
- Type definitions: `type fertilizer_db`, `type manure_database` and `type manure_attributes` in `fertilizer_data_module.f90`
- Name lookup: `read_mgtops.f90`
- Operations: `mgt_sched.f90`, `actions.f90`
- Application: `pl_fert.f90`, `pl_manure.f90`, `cs_fert.f90`, `salt_fert.f90`
- Fresh and humus split: `man_coef%rtof` in `carbon_module.f90`, read from `carbon.bsn` in `carbon_bsn_read.f90`

File and column	Fortran variable
fertilizer.frt `name`	`fertdb%fertnm`

File and column	Fortran variable
fertilizer.frt `min_n`, `min_p`	`fertdb%fminn`, `fertdb%fminp`
fertilizer.frt `org_n`, `org_p`	`fertdb%forgn`, `fertdb%forgp`
fertilizer.frt `nh3_n`	`fertdb%fnh3n`
manure_db.frt `org_min`	`manure_db%org_min`, index in `manure_db%iorg_min`
manure_db.frt `description`	`manure_db%descrip`
manure_om.frt columns	`manure_om%` with the same names

Purpose

`tillage.til` is the tillage database. Each row describes one tillage implement or operation: how much of the soil it mixes and how deep. A `till` operation in `management.sch`, a `till` or `tillage` action in a decision table, and a puddling operation all name a row of this file.

One row has a special role. A row named `biomix` sets the biological mixing efficiency and depth used by the dynamic carbon model (see Biological mixing).

The file is named on the `hru_parm_db` line of `file.cio` (default `tillage.til`). If the file is missing or named `null`, the database is empty and tillage operations find no row.

Format

Free format, whitespace-separated. Line 1 is a title and line 2 is the column header; both are skipped. Each later line is one operation with the values below in this order. The reader does not match columns by header name. It reads the first six values and ignores the rest of the line, so the `description` column written by the SWAT+ Editor has no effect.

#	Column	Units	Default	Typical range	Description
1	`name`	-	-	-	Name of the tillage operation, up to 16 characters. Management schedules and decision tables refer to the operation by this name, so names must be unique. The name `biomix` is reserved for biological mixing.
2	`mix_eff`	fraction	0.0	0.0 to 1.0	Mixing efficiency. The fraction of the soil, residue, nutrients and other constituents in each layer within the mixing depth that is mixed uniformly over that depth. The rest stays in its original layer.
3	`mix_dp`	mm	0.0	0 to 750	Depth of mixing caused by the operation.
4	`rough`	mm	0.0	-	Random roughness. Read but not used.
5	`ridge_ht`	mm	0.0	-	Ridge height. Read but not used.
6	`ridge_sp`	mm	0.0	-	Ridge interval or row spacing. Read but not used.

Tillage mixing

On the day of a tillage operation SWAT+ takes `mix_eff` and `mix_dp` from the named row. For each soil layer whose bottom is within `mix_dp`, the fraction `mix_eff` of the layer's mass is mixed. A layer that straddles `mix_dp` contributes in proportion to the part of it above the mixing depth. The mixed material is then spread over the mixing depth in proportion to layer thickness. Soil water, texture, mineral and organic nutrient pools, residue and constituents are all mixed this way.

A mixing depth greater than 0 and less than 10 mm is raised to 11 mm. A row with `mix_dp` of 0 mixes nothing.

The `carbon` column of `codes.bsn` decides what else happens. With `carbon` = 2 (dynamic carbon), the operation also raises the tillage factor of each mixed layer, which speeds the decomposition of soil organic matter for `till_eff_days` days. The tillage factor curve uses `tillmix_a`, `tillmix_b` and `tillmix_c` from `carbon.bsn`. With `carbon` = 0, only the mixing is done.

A puddling operation (`pudl` in `management.sch`) names a row of `puddle.ops` and a row of this file. When the HRU has more than 1 mm of ponded water, the soil is mixed with a routine for flooded soils. Otherwise the mixing is the same as for an ordinary tillage operation.

Biological mixing

Biological mixing is the slow mixing of the soil by roots, worms and other organisms. How it is set depends on the carbon model.

With `carbon` = 2 in `codes.bsn`, the `biomix` row of `tillage.til` sets it. Its `mix_eff` is the biological mixing efficiency and its `mix_dp` the maximum biological mixing depth. Biological mixing runs every day in layers above 0 °C. After a tillage operation the efficiency drops to zero and builds back up over the following days at a rate set by `bio_consol` in `carbon.bsn`. If the file has no `biomix` row, the efficiency is 0.2 and the depth 50 mm. A `biomix` row with `mix_eff` of 0 turns daily biological mixing off.

With `carbon` = 0, the `biomix` row is not used for mixing. Biological mixing then happens once a year, at the end of the year, with the efficiency in the `bio_mix` column of `hydrology.hyd` and a fixed depth of 50 mm (or the soil depth, if smaller).

Surface roughness

`rough`, `ridge_ht` and `ridge_sp` are not used. The random roughness used for surface depression storage is 20 mm for every HRU in 62.0.1, whatever tillage is done.

Example

`refdata/Osu_1hru/tillage.til` in the SWAT+ repository:

```
tillage.til: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                    mix_eff        mix_dp         rough      ridge_ht      ridge_sp  description
puddle                  0.980       150.000        00.000         0.000         0.000    rice_paddy_puddlin
```

One operation named `puddle`, used for rice paddy puddling. It mixes 98% of the soil in the top 150 mm. The file has no `biomix` row; Osu_1hru runs with `carbon` = 0 in `codes.bsn`, so biological mixing comes from `hydrology.hyd` and the missing row has no effect.

The first lines of `refdata/Ames_sub1/tillage.til`, as shipped:

name	mix_eff	mix_dp	rough	ridge_ht	ridge_sp	description
bedderd	0.55	150	35	0	0	bedder(disk)
bedders	0.55	150	30	0	0	beddershaper
beddhipr	0.65	150	35	0	0	bedderdisk-hipper
beddkrow	0.85	100	40	0	0	bedderdisk-row
bedrollr	0.25	50	5	0	0	bedroller

beetcult	0.25	25	15	0	0	beetcultivator
biomix	0.01	50	0	0	0	biological_mixing

This file has no title line. SWAT+ skips the first two lines whatever they contain, so it takes the header as the title and the **bedderd** row as the header, and **bedderd** is not in the database. A schedule that names **bedderd** would not find it. The other 98 rows are read normally; for example **mldboard** (moldboard plow) mixes 95% of the top 250 mm. Ames_sub1 runs with **carbon** = 2, so its **biomix** row sets a daily biological mixing efficiency of 0.01 over the top 50 mm.

Related files

- management.sch: **till** and **pudl** operations name a row of this file.
- Decision tables: **till** and **tillage** actions name a row of this file.
- puddle.ops: puddling parameters that go with the tillage row.
- codes.bsn: the **carbon** column selects the mixing routine and how biological mixing is set.
- carbon.bsn: tillage factor coefficients, **till_eff_days** and **bio_consol** for the dynamic carbon model.
- hydrology.hyd: the **bio_mix** column sets biological mixing when **carbon** = 0.

Sources

- Reader: till_parm_read.f90
- Type definition and biological mixing defaults: tillage_data_module.f90
- Name lookup for schedules and decision tables: read_mgtops.f90, dtbl_lum_read.f90
- Operations: mgt_sched.f90, actions.f90
- Mixing: mgt_newtillmix_cswat0.f90, mgt_newtillmix_cswat1.f90, mgt_newtillmix_wet.f90, mgt_tillfactor.f90
- Biological mixing: mgt_biomix.f90, soil_nutcarb_init.f90, hru_control.f90, time_control.f90
- Random roughness: zero0.f90, swr_depstor.f90

Column	Fortran variable
name	tilldb%tillnm
mix_eff	tilldb%effmix
mix_dp	tilldb%deptil
rough	tilldb%ranrns
ridge_ht	tilldb%ridge_ht
ridge_sp	tilldb%ridge_sp

carb_coefs.cbn is no longer read.

Removed

SWAT+ 62.0.1 ignores a copy of **carb_coefs.cbn** left in the run directory. Its settings now come from carbon.bsn and carbon_lyr.bsn.

Replacement files

The dynamic carbon model is switched on with **carbon** = 2 in codes.bsn. In earlier revisions the value was 1; in 62.0.1 the value 1 is reserved for the C-FARM model, which is not implemented. With **carbon** = 0 soil carbon is static and neither replacement file is read.

With **carbon** = 2 SWAT+ needs two files, both with a title line, a header line and data rows like other SWAT+ inputs:

- carbon.bsn: one row of 28 basin-wide values. It is named on the basin line of file.cio (default carbon.bsn).
- carbon_lyr.bsn: decomposition rates and CO2 fractions, one row for the top soil layer (**layer** 1) and one for the layers below (**layer** 2). Its name is built from the carbon.bsn name by adding **_lyr** before the extension, so it is not listed in file.cio.

If either file is missing, or carbon.bsn does not have 28 values, SWAT+ stops with an error. There is no fallback to built-in values.

An optional file carbon_layers.prt sets how many soil layers the per-layer carbon outputs include. Without it, the outputs cover the largest number of layers of any HRU.

Keyword migration

Keyword in carb_coefs.cbn	Replacement
cbn_diagnostics	Removed. Carbon outputs are switched on per file in print.prt with the hru_cb_* and lsu_cb_* rows.
hp_rate, hs_rate, microb_rate, meta_rate, str_rate, microb_top_rate, hs_hp	Columns of the same names in carbon_lyr.bsn.
a1co2, asco2, apco2, abco2	Columns of the same names in carbon_lyr.bsn.
org_frac (4 values)	init_seq, init_microb, init_slow, init_passive in carbon.bsn.
prmt_21	koc_c in carbon.bsn.
prmt_44	solc_ratio in carbon.bsn.
till_eff_days	till_eff_days in carbon.bsn.
rtof	manure_c_frac in carbon.bsn.
cbn_consolidation_factors (2 values)	bio_consol, till_consol in carbon.bsn.
cbn_factor_approaches (2 values)	tmpf_eqn, watf_eqn in carbon.bsn.
tn, top, tx	t_cbn_min, t_cbn_opt, t_cbn_max in carbon.bsn.
zz_bmix_coefs (3 values)	bmix_a, bmix_b, bmix_c in carbon.bsn.
zz_emix_coefs (3 values)	tillmix_a, tillmix_b, tillmix_c in carbon.bsn.
photo_degrade_factor	sfc_rsd_photodeg in carbon.bsn.
nmbr_soil_test_layers, soil_test	Removed, no replacement.

Six values that were fixed in the code are now columns of carbon.bsn: n_act_frac (fraction of humus nitrogen kept in the active pool), cnr_cap and cnr_ref (cap and reference C:N ratio of surface residue, formerly 500 and 25), cpr_cap and cpr_ref (cap and reference C:P ratio, formerly 5000 and 200), and mathers_method (method for the initial slow humus pool).

Example

refdata/Ames_sub1 in the SWAT+ repository has both replacement files, converted from its former carb_coefs.cbn:

```
carbon.bsn: derived from carb_coefs.cbn (basin carbon scalars)
          init_seq       init_microb         init_slow      init_passive             koc_c        solc_rat
              0.95              0.02              0.44              0.54            1000.0               0

carbon_lyr.bsn: derived from carb_coefs.cbn (per-layer carbon coefficients)
           layer         hp_rate         hs_rate     microb_rate       meta_rate        str_rate microb_to
               1         1.2e-05        2.92e-04          0.0164          0.0405          0.0107
               2         1.2e-05        1.81e-04            0.02          0.0507          0.0134
```

Related files

- carbon.bsn and carbon_lyr.bsn: the replacement inputs.
- codes.bsn: the carbon switch.
- print.prt: carbon output switches.
- Carbon outputs: the files written by the dynamic carbon model.

Sources

- Readers: carbon_bsn_read.f90 (reads carbon.bsn and carbon_lyr.bsn), carbon_layers_read.f90, both called from proc_bsn.f90

- Variables: carbon_module.f90
- Carbon switch: bsn_cc%cswat in basin_module.f90
- Output switches: basin_print_codes_read.f90
- File name slot: in_basin%carbon_bsn in input_file_module.f90

Purpose

treatment.trt is a file name kept from the SWAT2012 treatment table. Project setup tools can still write it into a project folder, but SWAT+ 62.0.1 does not open it, and file.cio has no slot for it. Its contents have no effect on a run, and the file can be left empty or deleted.

Water treatment in SWAT+ is part of water allocation. Treatment plants are defined in water_treat.wal, and the composition of treated water in om_treat.wal.

Format

None. The file is not read.

Example

Both reference datasets in the SWAT+ repository, refdata/Ames_sub1 and refdata/Osu_1hru, contain an empty treatment.trt.

Related files

- Water allocation: water_treat.wal and om_treat.wal, the treatment inputs SWAT+ reads.
- file.cio: lists the input files SWAT+ reads.

  Sources

  - File name slots read from file.cio: input_file_module.f90, readcio_read.f90
  - Treatment inputs: water_treatment_read.f90 (water_treat.wal), om_treat_read.f90 (om_treat.wal)

Purpose

manure_allo.mnu sets up manure allocation: manure sources with a monthly production, and HRUs that receive manure from them when a decision table in lum.dtl asks for it. The file name is fixed. If the file is missing, manure allocation is off.

SWAT+ 62.0.1 reads this file and checks the decision tables every day, but applies no manure to any HRU. To apply manure, use a fertilize action in lum.dtl or a manu operation in management.sch.

Format

Free format, whitespace-separated.

- Line 1: title (skipped).
- Line 2: number of allocation objects.
- Then, for each allocation object:
  1. header line (skipped);
  2. object line: name, rule_typ, src_obs, trn_obs;
  3. header line (skipped);
  4. src_obs source lines;
  5. header line (skipped);
  6. trn_obs demand lines.

Object line

#	Column	Description
1	`name`	Allocation object name.
2	`rule_typ`	Allocation rule. Read but not used.
3	`src_obs`	Number of source lines.
4	`trn_obs`	Number of demand lines.

Source lines

#	Column	Units	Description
1	`num`	-	Source number, 1 to `src_obs`. The line is stored at this position.
2	`mois_typ`	-	Moisture class, `wet` or `dry`.
3	`manure_typ`	-	Manure name in `fertilizer.frt`.
4	`lat`	degrees	Latitude of the source.
5	`long`	degrees	Longitude of the source.
6	`stor_init`	t	Manure in storage at the start of the simulation.
7	`stor_max`	t	Storage capacity.
8 to 19	`prod_jan` … `prod_dec`	t/month	Manure produced in each month, January to December.

Demand lines

#	Column	Description
1	`num`	Demand number, 1 to `trn_obs`. The line is stored at this position.
2	`ob_typ`	Receiving object type: `hru`, `muni` (treatment) or `divert` (diversion out of the basin). Only `hru` demands are linked to a decision table.
3	`ob_num`	Number of the receiving object.
4	`dtbl`	Name of the decision table in `lum.dtl` that requests manure for this HRU, or `null`.
5	`right`	Manure right, `sr` (senior) or `jr` (junior).

The decision table requests manure with a `manure_demand` action: `obj_num` is the source number, `option` the application method in `chem_app.ops`, `const` the amount in t/ha and `const2` the maximum number of requests per year (see `*.dtl`).

Example

One allocation object with one source and one HRU demand:

```
manure_allo.mnu: example
1
name            rule_typ    src_obs  trn_obs
farm_a          fixed             1        1
num  mois_typ  manure_typ  lat       long      stor_init  stor_max   prod_jan  feb  mar  apr  may  jun  ju
  1  wet       beef_fr     41.99   -93.62          0.0       500.0    5.0  5.0  6.0  6.0  7.0  7.0  7.0  7
num  ob_typ    ob_num  dtbl                right
  1  hru            1  apply_manure         sr
```

With a table `apply_manure` in `lum.dtl` whose first action is `manure_demand` with source 1, method `broadcast` and 2 t/ha, the intent is to apply 2 t/ha of `beef_fr` from the source whenever the table's conditions are met. In 62.0.1 the request is recorded but nothing is applied.

Related files

- fertilizer.frt: the manure named in manure_typ.
- *.dtl: the decision tables named on demand lines and the manure_demand action.
- chem_app.ops: application methods.

Sources

- Reader: manure_allocation_read.f90, called from main.f90.in after dtbl_lum_read
- Types: manure_allocation_module.f90 (manure_allocation, manure_source_objects, manure_demand_objects)
- Daily calculation: mallo_control.f90, called from time_control.f90
- manure_demand action: actions.f90, cross-walk in dtbl_lum_read.f90
- HRU decision table slot: hru_dtbl_actions_init.f90
- Output: manure_source_output.f90, manure_demand_output.f90, called from command.f90

Column	Fortran variable
object line	mallo%name, %rule_typ, %src_obs, %trn_obs
source manure_typ	mallo%src%manure_typ (index in mallo%src%fertdb)
source prod_jan … prod_dec	mallo%src%prod_mon(1:12)
demand dtbl	mallo%trn%dtbl (index in mallo%trn%dtbl_num and hru%man_trn_dtbl)
manure_demand obj_num, const, option	manure_amt%src_obj, %app_t_ha, %app_method

The management schedules are in management.sch. Four small inputs referred to from other pages follow it: puddle.ops, transplant.ops, co2_yr.dat and object.prt.

management.sch

management.sch contains the management schedules. Each schedule has a list of operations on fixed dates or heat-unit fractions, a list of automatic operations (decision tables in lum.dtl), or both. An HRU uses the schedule named in the mgt column of its land use in landuse.lum. The file name is the second entry on the lum line of file.cio (default management.sch).

Format

Free format, whitespace-separated. Line 1 is a title and line 2 is the column header; both are skipped. Then, for each schedule:

1. The schedule line: name, numb_ops, numb_auto.
2. numb_auto lines, each naming one decision table in lum.dtl.
3. numb_ops operation lines.

Column	Description
name	Schedule name, referenced by mgt in landuse.lum.
numb_ops	Number of scheduled operation lines.
numb_auto	Number of automatic operation lines.

Automatic operations

Each automatic operation line names a decision table in lum.dtl. Every day, SWAT+ checks each listed table for every HRU that uses the schedule. A name that is not in lum.dtl is not reported.

Three generic tables can be shared by several schedules with different crops. For these, the crop name follows the table name on the same line, and the plant, harvest and harvest_kill actions of the table use it instead of their option column:

Table name	Line content
pl_hv_summer1	table name and one crop
pl_hv_winter1	table name and one crop
pl_hv_summer2	table name and two crops; the crop used is the one for the current rotation year

Operation lines

#	Column	Units	Description
1	op_typ	-	Operation code (see below).
2	mon	-	Month of the operation, or 0.
3	day	-	Day of the month, or 0.
4	hu_sch	fraction	Heat-unit fraction that triggers the operation, or 0.
5	op_data1	-	First name (see below).
6	op_data2	-	Second name (see below).
7	op_data3	varies	Number: amount, days or override (see below).

op_typ	Operation	op_data1	op_data2	op_data3
plnt	plant	plant name in the HRU's plant community	name in transplant.ops	> 0 to start from the transplant entry
harv	harvest	plant name, or all	name in harv.ops	residue harvest efficiency (0 uses harv_eff)
hvkl	harvest and kill	plant name, or all	name in harv.ops	residue harvest efficiency
kill	kill	plant name, or all	-	-
till	tillage	name in tillage.til	-	-
fert	fertilizer	name in fertilizer.frt	name in chem_app.ops	amount, kg/ha
manu	manure	name in manure_db.frt	name in chem_app.ops	amount, kg/ha
pest	pesticide	pesticide name listed in constituents.cs	name in chem_app.ops	amount, kg/ha (multiplied by app_eff)
graz	grazing	name in graze.ops	-	number of grazing days
burn	burning	name in fire.ops	-	-
swep	street sweeping	name in sweep.ops	-	-
irrm	irrigation	name in irr.ops	-	-
irpm	irrigation of a rice paddy	name in irr.ops	-	-
irrp	start, adjust or stop paddy irrigation	name in irr.ops	water source: cha, res, aqu or unlim	target ponding depth, mm (see below)
pudl	puddling	name in tillage.til	name in puddle.ops	-
weir	set paddy weir height	weir name in weir.res	-	weir height, mm
dwm	drainage water management	-	-	tile drain depth, mm
cnup	change curve number	absval, abschg, pctchg or relchg	-	value
mons	monsoon season	-	-	0 or 1
skip	end of a rotation year	-	-	-

Notes on individual operations:

- plnt, harv, hvkl and kill act on the plant in the HRU's plant community (plant.ini) whose name equals op_data1. A plant that is already growing is not planted again; the event is logged as PLANT_ALREADY_GROWING.
- fert on an HRU with standing water puts the fertilizer into the water.
- pest looks the name up in the pesticide list of constituents.cs when the operation runs, so the pesticide must be simulated as a constituent.
- irrm and irpm apply the depth in irr.ops (see irr.ops).
- irrp turns on automatic irrigation of a paddy: water is added whenever the ponding depth falls below 90% of the target. A positive op_data3 is the target depth; a negative value uses amt_mm of the irr.ops entry; zero

turns paddy irrigation off. The irr.ops entry also gives the efficiency, runoff fraction and salt and nitrate concentrations of the water.

- cnup changes the curve number for moisture condition II: set it to the value (absval), add the value (abschg), change it by the value in percent (pctchg), or multiply it by 1 + value (relchg). The result is kept between 35 and 95.
- mons applies to plants with a moisture_gro trigger in plants.plt. With op_data3 1 it marks the start of the monsoon season. With 0 it restarts the phenology of such plants that are not growing.
- Codes not in this table (for example pcom or prtp) are read but do nothing.

Timing

Operations run in the order of the lines, one rotation year after another. A skip line ends a rotation year. After the last line the schedule starts again at the first line, so a schedule with n skip lines is an n-year rotation.

An operation runs on the day its mon and day match the calendar date, or, when hu_sch is greater than 0, on the first day the heat-unit fraction exceeds hu_sch. The heat-unit fraction is that of the plant named in op_data1 while it grows, and the base-zero fraction of the year otherwise. Give each operation either a date or a heat-unit fraction and set the other to 0. Within a rotation year, lines must be in chronological order: SWAT+ waits for the date of the current line before it moves on.

When the schedule reaches a skip line it waits there until the end of the year: 31 December for HRUs in the northern hemisphere, day 181 for HRUs in the southern hemisphere. At the start of the run, SWAT+ begins at the rotation year given by the plant community in plant.ini and skips the operations dated before the first simulated day.

The operations performed are written to mgt_out.txt when mgtout is y in print.prt.

Example

refdata/Ames_sub1/management.sch in the SWAT+ repository, first lines:

```
management.sch file AMES
                          NAME NUMB_OPS NUMB_AUTO OP_TYP        MON        DAY HU_SCH   OP_DATA1    OP_DAT
mgt_01                             305         1
                                                irr_year_irr
                                                    till          4          1    0.0    chisplow        n
                                                    plnt          4         28    0.0       corn         n
                                                    irrm          5         15    0.0       sprinkler_ilm
                                                    fert          6          1    0.0       null       ele
                                                    hvkl         10          7    0.0       corn        gr
                                                    skip          0          0      0       null         n
```

Schedule mgt_01 has 305 operation lines and one automatic operation, the irr_year_irr table of lum.dtl (see the *.dtl example). In the first rotation year it tills with chisplow on 1 April, plants corn on 28 April, irrigates once on 15 May, and harvests and kills the corn with the grain harvest operation on 7 October.

For a fert line, 62.0.1 reads the fertilizer name from op_data1 and the application method from op_data2. The fert line above has null in op_data1 and the fertilizer name elem_n in op_data2. No fertilizer is named null, so the line matches no entry in fertilizer.frt and applies nothing; no warning is written. Every fert line in this file has the same layout, so as written the Ames schedules apply no fertilizer in 62.0.1. A line that applies 181.4 kg/ha of elem_n in 62.0.1 reads fert 6 1 0.0 elem_n broadcast 181.4, with broadcast an entry in chem_app.ops.

Two schedules from refdata/Osu_1hru/management.sch, one using a scheduled manu operation together with decision tables and one using only decision tables:

```
rice140_rot                      1          7
     plow
     weir60r
     weir0
     pad_irr_r
     puddle
```

```
     pl_hv_rice
     fert_paddy
manu      5          1      0     beef_salt   broadcast  12.9927
agrl_rot                        0          2
    pl_hv_agro
    fert_stress
```

As shipped, the `manu` line finds `beef_salt` in manure_db.frt and `broadcast` in chem_app.ops, so 12.99 kg/ha of manure is applied on 1 May. The schedule has one operation line and no `skip`, so the line runs every year.

Related files

- landuse.lum: selects a schedule by name in its `mgt` column.
- *.ops, tillage.til, fertilizer.frt: databases the operation lines name.
- *.dtl: the decision tables listed as automatic operations.
- plant.ini: plant community and initial rotation year.

puddle.ops

Puddling parameters for rice paddies. A puddling operation (`pudl` in `management.sch`, or the `puddle` action in lum.dtl) mixes the soil with a tillage operation, lowers the hydraulic conductivity of the paddy's upper soil, and sets the sediment concentration of the ponded water. The file name is fixed.

Free format. Line 1 is a title and line 2 is the column header; both are skipped. Each later line is one entry, read by position.

#	Column	Units	Description
1	`name`	-	Puddling operation name.
2	`hydcon_mm/h`	mm/h	Hydraulic conductivity of the upper soil after puddling. 0 leaves it unchanged.
3	`sed_ppm`	ppm	Sediment concentration of the ponded water after puddling.
4	`orgn_ppm`	ppm	Organic N concentration after puddling. Read but not used.
5	`sedp_ppm`	ppm	Organic P concentration after puddling. Read but not used.
6	`no3_ppm`	ppm	NO3-N concentration after puddling. Read but not used.
7	`solp_ppm`	ppm	Soluble P concentration after puddling. Read but not used.
8	`nh3_ppm`	ppm	NH3 concentration after puddling. Read but not used.
9	`no2_ppm`	ppm	NO2 concentration after puddling. Read but not used.

The tillage operation is named in `op_data1` of the `pudl` line (or the `name` column of the `puddle` action). If the paddy has more than 1 mm of water, the mixing is done in the ponded water; otherwise it is ordinary tillage.

`refdata/Osu_1hru/puddle.ops` as shipped:

```
name                hydcon_mm/h   sed_ppm      orgn_ppm      sedp_ppm     no3_ppm       solp_ppm       nh
high_eff            0.00000         10000       0.00000     800.00000     10.00000       20.00000        1
med_eff             0.00000         10000       0.00000     500.00000     10.00000       20.00000        1
low_eff             0.0           10000       0.00000     100.00000     10.00000       20.00000        1.000
```

The file has no title line. SWAT+ skips the column header as the title and the `high_eff` line as the header, so 62.0.1 reads only `med_eff` and `low_eff`. An operation that names `high_eff` finds no entry and uses an entry with all values zero: conductivity unchanged and no sediment added. The Osu dataset's `puddle` decision table names `med_eff`, so its puddling runs with a sediment concentration of 10000 ppm and unchanged conductivity (`hydcon_mm/h` is 0).

transplant.ops

Transplant database. A planting can start a plant from a transplanted state instead of from seed: a `plnt` line in `management.sch` with a `transplant.ops` name in `op_data2` and `op_data3` greater than 0, or a `plant` action in `lum.dtl` with the name in its `fp` column. The file name is fixed. Up to revision 61 it was `transplant.plt`; 62.0.1 reads only `transplant.ops`, so a project with a `transplant.plt` file needs it renamed.

Free format. Line 1 is a title and line 2 is the column header; both are skipped. Each later line is one entry, read by position.

#	Column	Units	Description
1	`name`	-	Transplant name.
2	`lai_ini`	m^2/m^2	Leaf area index at transplanting. Read but not used.
3	`bm_ini`	kg/ha	Plant biomass at transplanting.
4	`phu_acc_ini`	fraction	Fraction of potential heat units already accumulated.
5	`fr_yrmat`	fraction	Fraction of the years to maturity already reached (perennials).
6	`pop`	plants/m^2	Plant population. Sets the maximum leaf area index through the population curve in `plants.plt`. 0 uses the maximum LAI in `plants.plt`.

`refdata/Osu_1hru` contains the file under its old name:

```
transplant.plt
      NAME       LAI_INI     BM_INI    PHU_ACC_INI    FR_YRMAT      POP
   tr_rice140         0.1        0.2      0.2             1            100
```

62.0.1 does not open `transplant.plt`, so in this dataset the transplant database is empty. The `plant` actions in its `lum.dtl` that name `tr_rice140` find no entry, no message is written, and the rice is planted from seed with the initial values of an ordinary planting.

co2_yr.dat and co2.out

Atmospheric CO2 concentration by year. `co2_yr.dat` is optional; its name is fixed. SWAT+ builds one CO2 value per simulation year and writes the series to `co2.out`.

`co2_yr.dat` layout:

Line	Content
1	title (skipped)
2	number of year records n
3	header (skipped)
4 to $n + 3$	calendar year and CO2 concentration (ppm), one year per line, consecutive years in increasing order

How the series is built:

- If `co2_yr.dat` is missing or empty, every year uses `co2` from `parameters.bsn` (default 400 ppm).
- If the last record's year is at or before the first simulation year, every year uses the last record's value.
- Otherwise the records are aligned to the simulation by the first record's year. Years before the first record use the first value, and years after the last record repeat the last value.

The alignment counts records from the first one, so the years must be consecutive.

`co2.out` has the header `YR  CO2(ppm)` and one line per simulation year with the calendar year and the CO2 value used.

object.prt

Hydrograph and state output for individual objects. Each line asks for one variable of one object, written daily to a separate file. The file name is the third entry on the `simulation` line of `file.cio` (default `object.prt`; `null` for none).

Free format. Line 1 is a title and line 2 is the column header; both are skipped. Each later line is one request.

#	Column	Description
1	`id`	Sequence number, 1, 2, 3, … in order.
2	`obj_typ`	Object type: `hru`, `hlt` (HRU-lte), `ru` (routing unit), `res`, `cha` (channel), `sdc` (SWAT-DEG channel), `exc` (export coefficient), `dr` (delivery ratio) or `out` (outlet).
3	`obj_typ_no`	Object number within its type. For `hru` with `sol_water` or `solnut_ly`, 0 writes every HRU.
4	`hyd_typ`	Variable (see below).
5	`filename`	Output file name, up to 26 characters.

`hyd_typ`	Output
`tot`	total outflow hydrograph
`rhg`	recharge (percolation) hydrograph
`sur`	surface runoff hydrograph
`lat`	lateral flow hydrograph
`til`	tile flow hydrograph
`sol_water`	soil water of each layer
`solnut_ly`	soil N and P pools of each layer (no header line)
`solnut_pr`	soil N and P pools of the profile (no header line)
`plant`	plant status
`cha_fp`	channel and floodplain water balance

Each requested file gets a header line (except `solnut_ly` and `solnut_pr`) and one line per day with day of year, month, day, year, object type and name, followed by the values.

Sources

- `management.sch` readers: `mgt_read_mgtops.f90` (schedule and automatic operation lines), `read_mgtops.f90` (operation lines and name lookups)
- Schedule types: `mgt_operations_module.f90`
- Running scheduled operations: `mgt_operatn.f90`, `mgt_sched.f90`, called from `hru_control.f90`
- Starting operation and year-end `skip` handling: `plant_init.f90`, `time_control.f90`
- Automatic operation lookup: `proc_cond.f90`; generic crop tables: `actions.f90`
- Curve number change: `chg_par.f90`
- `puddle.ops`: `mgt_read_puddle.f90`
- `transplant.ops`: `plant_transplant_read.f90`, `mgt_transplant.f90`, type `plant_transplant_db` in `plant_data_module.f90`
- CO2: `co2_read.f90`, `bsn_prm%co2` in `basin_module.f90`
- `object.prt`: `object_read_output.f90`, `obj_output.f90`, type `object_output` in `hydrograph_module.f90`
- File name slots: `in_lum%management_sch` and `in_sim%object_prt` in `input_file_module.f90`

File	Column	Fortran variable
`management.sch`	`numb_ops`, `numb_auto`	`sched%num_ops`, `sched%num_autos`
`management.sch`	automatic operation line	`sched%auto_name`, `sched%auto_crop`
`management.sch`	`op_typ`, `mon`, `day`	`mgt_ops%op`, `%mon`, `%day`
`management.sch`	`hu_sch`	`mgt_ops%husc`
`management.sch`	`op_data1`, `op_data2`, `op_data3`	`mgt_ops%op_char`, `%op_plant`, `%op3`
`puddle.ops`	`hydcon_mm/h`	`pudl_db%wet_hc`
`transplant.ops`	`lai_ini`, `bm_ini`, `phu_acc_ini`	`transpl%lai`, `%bioms`, `%phuacc`

File	Column	Fortran variable
co2_yr.dat	year, CO2	co2_inc%co2_yr%iyr, %co2
object.prt	obj_typ, obj_typ_no, hyd_typ	ob_out%obtyp, %obtypno, %hydtyp

Purpose

The initialization files set the state of plants, soils and water bodies on the first day of the run. The general ones are:

- plant.ini: plant communities, with the starting state of each plant. A land use in landuse.lum names a community in its plnt_com column.
- soil_plant.ini: for each HRU, links to the records that initialize soil nutrients, pesticides, pathogens, salts, heavy metals and constituents. An HRU names a record in the soil_plant_init column of hru-data.hru.
- nutrients.sol: initial soil nitrate, labile phosphorus and humus properties, named in the nutrients column of soil_plant.ini.
- om_water.ini: initial water, sediment and nutrient concentrations of channels, reservoirs and wetlands, named in the org_min column of their initial files.

Initialization files

The init line of file.cio has eleven file names. nutrients.sol is the second name on the soils line. Several salt and constituent files are opened under fixed names, whatever file.cio says.

File	How the name is set	Status in 62.0.1	Documented in
plant.ini	init line, name 1	Read	plant.ini
soil_plant.ini	init line, name 2	Read	soil_plant.ini
om_water.ini	init line, name 3	Read	om_water.ini
pest_hru.ini	init line, name 4	Read	pesticides
pest_water.ini	init line, name 5	Read	pesticides
path_hru.ini	init line, name 6	Read	pathogens
path_water.ini	init line, name 7	Read	pathogens
hmet_hru.ini	init line, name 8	Read, values not used	heavy metals
hmet_water.ini	init line, name 9	Not read	heavy metals
salt_hru.ini	init line, name 10	The file.cio name is not used. SWAT+ opens a file called salt_hru.ini.	salts
salt_water.ini	init line, name 11	Not read. Water bodies take their initial salts from salt_aqu.ini and salt_channel.ini (fixed names) and from the reservoir salt file.	salts
salt_aqu.ini, salt_channel.ini	Fixed names, not in file.cio	Read	salts
cs_hru.ini, cs_aqu.ini, cs_channel.ini	Fixed names, not in file.cio	Read	constituents
nutrients.sol	soils line, name 2	Read	nutrients.sol

All of these files are free format and whitespace-separated, with a title on line 1 and a column header on line 2, both skipped. Columns are read by position. Names are matched case-sensitively.

plant.ini

Each plant community is a block: one community line, then one line for each plant in the community. A community can be plants growing together (a mixed stand) or the crops of a rotation, of which only some are growing at the start.

If `plant.ini` is missing or named `null`, no community exists, and every land use whose `plnt_com` is not `null` is reported in `diagnostics.out` as `<landuse> <community> not found in plant.ini`.

Community line

#	Column	Units	Description
1	`pcom_name`	-	Community name, up to 40 characters, matched against `plnt_com` in `landuse.lum`. Names must be unique.
2	`plt_cnt`	-	Number of plant lines that follow, called `plnt_cnt` in earlier documentation. The reader counts communities with this number, so it must equal the number of plant lines.
3	`rot_yr_ini`	-	Year of the rotation in which the run starts. SWAT+ steps through the management schedule, counting a new year at each `skip` operation or each drop in day of year, and starts at the first operation of this rotation year on or after the start day of the run. Decision tables see it as the current rotation year.

Plant lines

#	Column	Units	Range	Description
1	`plt_name`	-	-	Plant name in `plants.plt`, called `plnt_name` in earlier documentation.
2	`lc_status`	-	`y`, `n`	Land cover status at the start: `y` growing, `n` not growing (it starts when a planting operation names it).
3	`lai_init`	m^2/m^2	0 to 8	Initial leaf area index. Used only when `lc_status` is `y`.
4	`bm_init`	kg/ha	0 and up	Initial total plant biomass. It is set whatever `lc_status` is; its N and P contents follow from the plant's nutrient fractions at the starting stage of growth.
5	`phu_init`	fraction	0 to 1	Fraction of the heat units to maturity already accumulated at the start of the current growing season.
6	`plnt_pop`	plants/m^2	-	Plant population. Read but not used: the maximum LAI is always the `lai_pot` of `plants.plt`, with no population adjustment.
7	`yrs_init`	fraction	0 to 1	For trees and other perennials, the current age as a fraction of the years to maturity (`mat_yrs` in `plants.plt`). 0.5 for a tree with `mat_yrs` = 30 means a 15-year-old stand. Not used for annuals.
8	`rsd_init`	kg/ha	0 to 10 000	Initial surface residue from this plant.

Starting state

For each plant in the community, SWAT+ sets at the start of the run:

- Heat units to maturity, from `days_mat` in `plants.plt` and the weather generator temperatures (see plants.plt).
- Maturity stage: `phu_init` for annuals; `yrs_init` for perennials, whose year of growth is $\max(1, \lfloor \text{yrs_init} \times \text{mat_yrs} \rfloor)$ and whose lifetime heat unit fraction is $\text{yrs_init} + \text{phu_init}/\text{mat_yrs}$.
- Canopy height: $\text{yrs_init} \times h_{max}$ for perennials; for annuals $h_{max}\sqrt{f}$, where f is the fraction of maximum LAI that the LAI curve of `plants.plt` gives for `phu_init`.
- LAI from `lai_init` and biomass from `bm_init`. If the plant is growing, its root depth, root mass and seed mass are also set from the maturity stage.

- Surface residue from `rsd_init`, with carbon 42% of the residue mass, a C:N ratio of 10 and a C:P ratio of 100. The residue of all plants in the community is added together, so a two-crop rotation with 10 000 kg/ha each starts with 20 000 kg/ha.

In 62.0.1 a perennial that starts the run growing gets its lifetime heat unit fraction from `yrs_init` and `phu_init` before its roots are set. Earlier revisions left it at zero, which kept the starting root depth of perennials at the 25.4 mm minimum whatever their age.

A plant name that is not in `plants.plt` is not reported. The reader's check for a missing name can never trigger, and the plant silently takes the parameters of the first plant in `plants.plt`. Check plant names in `plant.ini` against `plants.plt` by hand.

Example

`refdata/Ames_sub1/plant.ini` in the SWAT+ repository, first communities, with the spacing reduced:

plant.ini:

pcom_name	plt_cnt	rot_yr_ini	plt_name	lc_status	lai_init	bm_init	phu_init	plnt_pop	yrs_init
cosy_comm	2	1							
			corn	n	0.00000	0.00000	0.00000	0.00000	0.00000
			soyb	n	0.00000	0.00000	0.00000	0.00000	0.00000
corn_comm	1	1							
			corn	n	0.00000	0.00000	0.00000	0.00000	0.00000
wetw_comm	1	1							
			wetw	y	2.00000	50000.00000	0.00000	0.00000	1.00000

`cosy_comm` is a corn and soybean rotation in which neither crop is growing at the start; the schedule plants them. Its two plant lines give 20 000 kg/ha of starting surface residue. `wetw_comm` is a wetland forest that starts growing, mature (`yrs_init` = 1), with LAI 2 and 50 000 kg/ha of biomass.

soil_plant.ini

Each line links an HRU to its soil initialization records. `hru-data.hru` names a record in its `soil_plant_init` column.

#	Column	Units	Description
1	`name`	-	Record name, up to 40 characters.
2	`sw_frac`	fraction	Initial soil water as a fraction of field capacity. Read but not used; initial soil water comes from `sw_init` in `parameters.bsn` (see soils.sol).
3	`nutrients`	-	Record in nutrients.sol.
4	`pest`	-	Record in `pest_hru.ini` (pesticides).
5	`path`	-	Record in `path_hru.ini` (pathogens).
6	`salt`	-	Record in `salt_hru.ini` (salts).
7	`hmet`	-	Record in `hmet_hru.ini` (heavy metals).
8	`cs`	-	Record in `cs_hru.ini` (constituents). Read only when the `event` column of `codes.bsn` is not 0.

The model reads column 6 as the salt record and column 7 as the heavy metal record. Some SWAT+ Editor versions write the header as `... path hmet salt`, the other way round. A salt record name placed in the column the Editor labels `salt` is then read as a heavy metal name, and the HRU uses the first `salt_hru.ini` record instead. Put the salt name in column 6 when salts are simulated.

Without a constituent column (the usual case), constituent runs use the first `cs_hru.ini` record for every HRU.

Name matching

When `hru-data.hru` is read, each HRU's `soil_plant_init` name is matched, and then the names in that record are matched against their files. When a name is not found:

- `nutrients`: the HRU uses the built-in values in the nutrients.sol table.
- `pest`, `path`, `salt`, `hmet`, `cs`: the HRU uses the first record of that file.
- `soil_plant_init` itself: nothing is reported (the check for a missing name can never trigger), and the HRU uses the first record of `soil_plant.ini`.

A project needs a `soil_plant.ini` file. If the file named in `file.cio` is missing, SWAT+ creates an empty file of that name and the soil initialization has no record to use.

Example

`refdata/0su_1hru/soil_plant.ini`:

```
soil_plant.ini: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                    sw_frac         nutrients              pest              path              hmet
soilplant1              0.00000          soilnut1              null              null              null
```

One record, `soilplant1`, with soil nutrients from `soilnut1` in `nutrients.sol` and no pesticide, pathogen, heavy metal or salt records. This file has the Editor's `hmet salt` header order.

nutrients.sol

Initial soil nitrogen and phosphorus and the humus properties of the static carbon model. Each record is one line. `soil_plant.ini` names a record in its `nutrients` column.

#	Column	Units	Built-in value	Description
1	`name`	-	`default`	Record name, up to 16 characters. A longer name in `soil_plant.ini` does not match.
2	`exp_co`	1/mm	0.001	Depth coefficient for the decrease of nitrate and labile P with depth. Values above 0.005 are replaced by 0.001.
3	`lab_p`	mg/kg	5.0	Labile (soluble) P concentration at the soil surface. 5.0 is used if 0.
4	`nitrate`	mg/kg	7.0	Nitrate N concentration at the soil surface. 7.0 is used if 0.
5	`fr_hum_act`	fraction	0.02	Fraction of humus in the active pool. 0.02 is used if 0. Static carbon model only.
6	`hum_c_n`	ratio	10.0	Humus C:N ratio (8 to 12). Static carbon model only; must not be 0.
7	`hum_c_p`	ratio	80.0	Humus C:P ratio (70 to 90). Static carbon model only; must not be 0.
8	`inorgp`	mg/kg	0.5	Inorganic P. Read but not used.
9	`watersol_p`	mg/kg	0.15	Water-soluble P. Read but not used.
10	`h3a_p`	mg/kg	0.25	H3A-extractable P. Read but not used.
11	`mehlich_p`	mg/kg	1.2	Mehlich-extractable P. Read but not used.
12	`bray_strong_p`	mg/kg	0.85	Strong Bray-extractable P. Read but not used.

The SWAT+ Editor also writes a `description` column, which is not read. The built-in values apply to HRUs whose `nutrients` name is `null` or not found, and to all HRUs when `nutrients.sol` is missing or named `null` in `file.cio`.

Initial mineral nitrogen and phosphorus

The concentrations at the surface decrease exponentially with the depth z (mm) to the bottom of each soil layer, and are converted to mass with the layer's bulk density ρ_b (Mg/m^3) and thickness h (mm):

$$NO3_{ly} = \text{nitrate}\; e^{-\text{exp_co}\, z}\, \frac{\rho_b\, h}{100}, \qquad P_{lab,ly} = \text{lab_p}\; e^{-\text{exp_co}\, z}\, \frac{\rho_b\, h}{100}$$

in kg/ha. The active mineral P pool is $P_{lab}\,(1 - PSP)/PSP$, with the phosphorus availability index PSP from `psp` in `parameters.bsn`, and the stable pool is four times the active pool. With the Vadas and White phosphorus model (`soil_p` = 1 in `codes.bsn`), PSP (0.1 to 0.7) and the stable to active ratio (1 to 7) are instead estimated from each layer's clay, organic carbon and labile P.

Initial organic pools

The organic carbon of each layer comes from the `carbon` column of `soils.sol`: $C_{ly} = 10\,000\, h\, \rho_b \times$ carbon/100 kg/ha, with organic N and P at C:N 10 and C:P 100 before the split into pools. The split depends on the `carbon` column of `codes.bsn`:

- Static carbon model (0): the active humus pool gets `fr_hum_act` of the organic matter and the stable pool the rest. The N and P of both pools are their carbon divided by `hum_c_n` and `hum_c_p`.
- CENTURY model (2): the share `init_seq` of `carbon.bsn` goes to the passive humus, slow humus and microbial pools, and the rest to metabolic and structural litter. Fixed C:N and C:P ratios are used; `fr_hum_act`, `hum_c_n` and `hum_c_p` are not. The slow humus pool follows the `mathers_method` switch of `carbon.bsn`.

In 62.0.1 the CENTURY pools are set up for carbon model 2 (earlier revisions tested for 1), the litter share is 1 − init_seq from `carbon.bsn` instead of a fixed 5%, and a non-lignin structural pool is also initialized.

Example

`refdata/Osu_1hru/nutrients.sol`:

```
nutrients.sol: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                    exp_co         lab_p       nitrate    fr_hum_act       hum_c_n       hum_c_p
soilnut1               0.00050       5.00000       7.00000       0.02000      10.00000      80.00000
```

Surface nitrate 7 mg/kg and labile P 5 mg/kg. With `exp_co` = 0.0005, the concentrations fall to $e^{-0.5} = 61\%$ of the surface value at 1000 mm.

om_water.ini

Initial water and water quality of channels, reservoirs and wetlands. Each line is one record; the `org_min` column of `initial.cha` (channels) or `initial.res` (reservoirs and wetlands) names a record.

#	Column	Units	Description
1	`name`	-	Record name, up to 16 characters.
2	`vol`	fraction	Initial water. Channels: fraction of bankfull depth; values above 1 put water on the flood plain. Reservoirs and wetlands: fraction of the principal spillway volume.
3	`sed`	mg/L	Suspended sediment.
4	`part_n`	mg/L	Organic N.
5	`part_p`	mg/L	Organic P.
6	`no3`	mg/L	Nitrate N.
7	`solp`	mg/L	Soluble mineral P.
8	`chl_a`	mg/L	Chlorophyll a.
9	`nh3`	mg/L	Ammonia N.
10	`no2`	mg/L	Nitrite N.
11	`cbn_bod`	mg/L	Carbonaceous biochemical oxygen demand.
12	`dis_ox`	mg/L	Dissolved oxygen.
13	`sand`	mg/L	Sand.
14	`silt`	mg/L	Silt.
15	`clay`	mg/L	Clay.
16	`sm_ag`	mg/L	Small aggregates.

#	Column	Units	Description
17	l_ag	mg/L	Large aggregates.
18	gvl	mg/L	Gravel.
19	tmp	°C	Water temperature.

The header names are those of the SWAT+ Editor; the reader does not use them. All values except vol and tmp are concentrations: SWAT+ converts them to masses (kg, or tonnes for sediment) with the initial water volume. A reservoir is filled only if it is already built at the start of the run (by its year and month operational); otherwise it starts empty.

If an org_min name is not found, the object uses the first record of om_water.ini. Projects with channels, reservoirs or wetlands therefore need the file. Aquifers do not use it in 62.0.1: the org_min column of initial.aqu is read but no aquifer value is set from it.

Example

refdata/Osu_1hru/om_water.ini, with the spacing reduced:

```
om_water.ini: added to match initial.cha org_min 'no_init' (channel water init)
name     flo  sed  orgn sedp no3  solp chl_a nh3  no2  cbn_bod dis_ox san  sil  cla  sag  lag  grv  tmp
no_init  0.0  0.0  0.0  0.0  0.0  0.0  0.0   0.0  0.0  0.0     0.0    0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0
```

One record, no_init, named by both initial.cha and initial.res in that project: channels and wetlands start empty. The header here uses the model's short names (flo, orgn, sedp, …) instead of the Editor's; only the order matters.

Related files

- landuse.lum: names a plant.ini community in its plnt_com column.
- hru-data.hru: names a soil_plant.ini record in its soil_plant_init column.
- plants.plt: plant parameters for every plt_name.
- management.sch: the schedule in which rot_yr_ini picks the starting year.
- soils.sol: soil layers, organic carbon and bulk density used with nutrients.sol.
- codes.bsn, parameters.bsn and carbon.bsn: carbon model, phosphorus model, psp and initial carbon pool fractions.
- initial.cha, initial.res and initial.aqu: name om_water.ini records.
- Pesticides, pathogens, heavy metals, salts and constituents: the other initialization files.
- Nutrient cycling and carbon.

Sources

- File names: type input_init and type input_soils in input_file_module.f90, read in readcio_read.f90
- plant.ini: reader readpcom.f90 (called from proc_db.f90); type plant_community_db and type plant_init_db in plant_data_module.f90; use in plant_init.f90 and pl_root_gro.f90
- soil_plant.ini: reader soil_plant_init.f90 (called from proc_read.f90); type soil_plant_initialize in hru_module.f90; name matching in hru_read.f90
- nutrients.sol: reader solt_db_read.f90; type soiltest_db in soil_data_module.f90; use in soil_nutcarb_init.f90 (called from main.f90.in) with carbon_bsn_read.f90
- om_water.ini: reader om_water_init.f90; type hyd_output, res_convert_mass and hyd_convert_conc_to_mass in hydrograph_module.f90; name matching in sd_channel_read.f90, res_read.f90, wet_initial.f90, aqu_read_init.f90; use in sd_hydsed_init.f90, res_initial.f90, wet_initial.f90
- Fixed-name salt and constituent files: salt_hru_read.f90, salt_aqu_read.f90, salt_cha_read.f90, cs_hru_read.f90, cs_aqu_read.f90, cs_cha_read.f90

File and column	Fortran variable
plant.ini pcom_name, plt_cnt, rot_yr_ini	pcomdb%name, %plants_com, %rot_yr_ini
plt_name, lc_status	pcomdb%pl%cpnm (index %db_num), %igro
lai_init, bm_init, phu_init	pcomdb%pl%lai, %bioms, %phuacc
plnt_pop, yrs_init, rsd_init	pcomdb%pl%pop, %fr_yrmat, %rsdin
soil_plant.ini name, sw_frac	sol_plt_ini%name, %sw_frac
nutrients, pest, path	sol_plt_ini%nutc, %pestc, %pathc (indices %nut, %pest, %path)
columns 6, 7, 8	sol_plt_ini%saltc, %hmetc, %csc (indices %salt, %hmet, %cs)
nutrients.sol columns 1 to 12	solt_db%name, %exp_co, %lab_p, %nitrate, %fr_hum_act, %hum_c_n, %hum_c_p, %inorgp, %watersol_p, %h3a_p, %mehlich_p, %bray_strong_p
om_water.ini name	om_init_name
om_water.ini columns 2 to 19	om_init_water%flo, %sed, %orgn, %sedp, %no3, %solp, %chla, %nh3, %no2, %cbod, %dox, %san, %sil, %cla, %sag, %lag, %grv, %temp

Purpose

Structural practices are field-scale conservation structures attached to a land use. Five files contain their parameter sets: tiledrain.str, septic.str, filterstrip.str, grassedww.str and bmpuser.str. They are named on the structural line of file.cio. Two databases on the hru_parm_db line belong with them: septic.sep (septic tank effluent quality) and urban.urb (urban land types). The saturated buffer file satbuffer.str is read from the run folder under that fixed name; it is not listed in file.cio.

Each record in a structural file has a name. A land use in landuse.lum points to records by name in its tile, sep, vfs, grww and bmp columns, and an urban land use points to urban.urb in its urban column. Every HRU that uses that land use gets the practice. Write null in a landuse.lum column for no practice. A name that is not found in the structural file is reported in diagnostics.out as <landuse> <name> not found (landuse-tiledr) (or landuse-septic, landuse-fstrip, landuse-grassww, landuse-bmpuser), and the HRU then gets no practice of that kind.

Practices can also be added during a run with the decision table actions tile_install, septic_install, fstrip_install, grassww_install, grassww_uninstall and user_def_bmp, which name a record in the matching file. See decision tables.

All files are optional. If a file is missing, or named null in file.cio, it is treated as empty.

File layout

All of these files share one layout. Line 1 is a title and line 2 is a column header; both are skipped. Each later line is one record, read as free-format values separated by spaces. The reader takes the values by position and ignores the header text, so every column must be present and in the order shown. Text after the last numeric column (the description column written by SWAT+ Editor) is ignored.

tiledrain.str

Subsurface (tile) drain parameter sets, referenced from the tile column of landuse.lum.

#	Column	Units	Typical range	Description
1	name	-	-	Name of the tile drain record.
2	dp	mm	0 to 6000	Depth of the drain tube below the soil surface. 0 means no drain: the HRU produces no tile flow.
3	t_fc	hours	0 to 100	Time to drain the soil to field capacity. Used only by the drawdown method (tiledrain = 0 in codes.bsn).

#	Column	Units	Typical range	Description
4	`lag`	hours	0 to 100	Drain tile lag time. Tile flow is held in a lag store that releases a fraction $1 - \exp(-24/\text{lag})$ each day.
5	`rad`	mm	3 to 40	Effective radius of the drains. DRAINMOD method only.
6	`dist`	mm	5000 to 100000	Distance between two drain tubes or tiles. DRAINMOD method only.
7	`drain`	mm/day	10 to 51	Drainage coefficient: the largest daily tile flow the drains can take. Used by both methods.
8	`pump`	mm/hr	0 to 10	Pump capacity for subirrigation. DRAINMOD method only.
9	`lat_ksat`	-	0.01 to 4	Multiplier on the saturated hydraulic conductivity of each soil layer (`k` in the soil file) to give its lateral conductivity. DRAINMOD method only.

Tile flow methods

Tile flow is computed only when the HRU's land use names a tile drain record with `dp` greater than 0, the soil temperature of the second layer is above 0 °C and the shallow water table is above the drains. The `tiledrain` column of `codes.bsn` selects the method, and its `wtable` column selects how the water table depth is found.

With `tiledrain` = 0 (drawdown method), the water above field capacity drains at a rate set by `t_fc`:

$$q_{tile} = \frac{h_{wt} - h_d}{h_{wt}} \, (SW - FC) \left(1 - \exp\left(\frac{-24}{t_{fc}}\right)\right)$$

where h_{wt} is the height of the water table above the bottom of the soil profile, h_d is the height of the drains above the bottom of the profile (profile depth minus `dp`), SW is the soil water content and FC the water content at field capacity, all in mm. If t_{fc} is below 1 hour the exponential factor is left out. The result is limited to `drain`.

With `tiledrain` = 1 (DRAINMOD method), the drainage flux comes from the Hooghoudt and Kirkham equations used in DRAINMOD, which use `rad`, `dist`, `lat_ksat`, `drain` and `pump`. When the water table is below the drains the same equations give a subirrigation flux, limited by `pump`.

In both methods the day's tile flow then passes through the lag store described under `lag`. If `lag` is 0.01 hours or less, the release fraction is zero and tile water stays in the lag store for the rest of the run.

`dist`, `rad` and `dp` enter the DRAINMOD equations together with the soil depth, all in millimetres, so `dist` must be given in millimetres. The SWAT+ Editor default record (example below) has `dist` = 30, which the model reads as 30 mm.

```
tiledrain.str: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                          dp          t_fc           lag           rad          dist         drain
mw24_1000           1000.00000      24.00000      96.00000     100.00000      30.00000      10.00000
```

A decision table can change the drain depth during a run with `tiledep_control` (drainage water management).

septic.str

On-site septic system parameter sets, adapted from Siegrist et al. (2005): system type, biozone geometry, biomass properties and reaction coefficients in the biozone. A record is meant to be referenced from the `sep` column of `landuse.lum`.

#	Column	Units	Description
1	`name`	-	Name of the septic system record.

#	Column	Units	Description
2	typ	-	Septic system type: the record number (row position, starting at 1) of the effluent type in septic.sep, given as an integer.
3	yr	year	Year the system starts operating. 0 means it operates from the start of the simulation.
4	operation	-	Operation flag: 0 not operating, 1 active, 2 failing. An active system becomes failing when the biozone clogs; a failing system returns to active after t_fail days.
5	residents	-	Number of permanent residents per house. A typical US value is 2.5.
6	area	m^2	Average drainfield area of one septic system. About 40 to 70 m^2 per person is usual; 100 m^2 for a household of 2.5.
7	t_fail	days	Time until a failing system is fixed (0 to 150).
8	dp_bioz	mm	Depth to the top of the biozone layer from the ground surface.
9	thk_bioz	mm	Thickness of the biozone layer, the thin soil layer under the effluent distribution chamber where pollutants are degraded by live bacteria.
10	cha_dist	km	Distance from the septic system to the stream.
11	sep_dens	per km^2	Number of septic systems per square kilometre.
12	bm_dens	kg/m^3	Density of biomass.
13	bod_decay	m^3/day	BOD decay rate coefficient in the biozone.
14	bod_conv	-	Ratio of bacterial growth to BOD degraded in the septic tank effluent.
15	fc_lin	-	Linear coefficient for field capacity in the biozone.
16	fc_exp	-	Exponential coefficient for field capacity in the biozone.
17	fecal_decay	m^3/day	Fecal coliform decay rate coefficient.
18	tds_conv	-	Conversion factor for plaque from total dissolved solids.
19	mort	-	Mortality rate coefficient.
20	resp	-	Respiration rate coefficient.
21	slough1	-	Linear coefficient for the rate of biomass sloughing.
22	slough2	-	Exponential coefficient for the rate of biomass sloughing.
23	nit	-	Nitrification rate coefficient.
24	denit	-	Denitrification rate coefficient.
25	p_sorp	L/kg	Linear P sorption distribution coefficient.
26	p_sorp_max	mg P/kg soil	Maximum P sorption capacity.
27	solp_slp	-	Slope of the linear effluent soluble P equation.
28	solp_int	-	Intercept of the linear effluent soluble P equation.

Septic simulation in 62.0.1

In revision 62.0.1 septic systems run, but receive no effluent:

- The sep column of landuse.lum assigns the named septic.str record to every HRU with that land use before the soil layers are set up. If operation is not 0, a biozone layer is added to the soil profile at dp_bioz with thickness thk_bioz, and from year yr the biozone routine runs on each day the biozone layer is above freezing. Unknown names are reported in diagnostics.out.
- The daily septic tank effluent volume is never computed and stays at zero. No effluent water reaches the biozone, and the effluent concentrations in septic.sep, which are multiplied by that volume, add no nitrogen, phosphorus or BOD to the soil. The biozone routine works only on the water and nutrients already in the layer. residents, area, cha_dist, sep_dens and the q_rate column of septic.sep are read but not used.
- The decision table action septic_install assigns a septic.str record to an HRU during the run. The biozone layer is placed only when the soils are set up at the start of the run, so an HRU that gets a septic system this way has no biozone layer.

septic.str: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4

name	typ	yr	operation	residents	area	t_fail	dp_bioz	t
standard	1	0	0	2.50000	100.00000	70	500.00000	50

septic.sep

Septic tank effluent quality database. Each record describes the effluent of one type of on-site wastewater system (from Siegrist et al., 2005; McCray et al., 2005). A `septic.str` record selects a row by its position through `typ`.

#	Column	Units	Description
1	`name`	-	Name of the effluent type (for example `gcon`, generic conventional system).
2	`q_rate`	m^3/capita/day	Septic tank effluent flow rate per person. McCray et al. (2005) give 0.227 as the US median. Not used.
3	`bod`	mg/L	7-day biological oxygen demand of the effluent. Typically 170 for a conventional system.
4	`tss`	mg/L	Total suspended solids in the effluent. Typically 75 for a conventional system.
5	`nh4_n`	mg N/L	Ammonium nitrogen in the effluent. Typically 60 (17 to 78) for a conventional system.
6	`no3_n`	mg N/L	Nitrate nitrogen in the effluent. 0 to 1.94 for a conventional system.
7	`no2_n`	mg N/L	Nitrite nitrogen in the effluent. Usually very low.
8	`org_n`	mg N/L	Organic nitrogen in the effluent. 9.4 to 15 for a conventional system.
9	`min_p`	mg P/L	Mineral phosphorus in the effluent.
10	`org_p`	mg P/L	Organic phosphorus in the effluent. Typically 1 for a conventional system.
11	`fcoli`	cfu/100 mL	Fecal coliform in the effluent. Typically 1E7 for a conventional system.

Values vary widely between system types; Table A-1 of Siegrist et al. (2005) lists them. The database distributed with SWAT+ Editor has 28 types, from generic conventional (`gcon`) and advanced (`gadv`) systems to sand filters, constructed wetlands, biofilters and textile filters.

`septic.sep: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4`

name	q_rate	bod	tss	nh4_n	no3_n	no2_n
gcon	0.22700	170.00000	75.00000	42.40000	0.00000	0.00000
gadv	0.22700	22.00000	14.00000	18.90000	9.60000	0.00000

filterstrip.str

Vegetative filter strip parameter sets, referenced from the `vfs` column of `landuse.lum`. A filter strip is a strip of dense vegetation at the field edge that intercepts runoff from upslope. It slows overland flow so particles settle, and it infiltrates part of the runoff. The model follows White and Arnold (2009). It reduces sediment, organic N, nitrate, particulate and soluble P, and pesticides in surface runoff leaving the HRU. The runoff volume itself is not changed.

#	Column	Units	Range used	Description
1	`name`	-	-	Name of the filter strip record.
2	`flag`	-	0 or 1	On/off switch. The filter strip is simulated only when `flag` is greater than 0.
3	`fld_vfs`	ratio	above 0, up to 300	Ratio of field area to filter strip area. Values of 30 to 60 are most common. Values above 300 are set to 300.

#	Column	Units	Range used	Description
4	con_vfs	fraction	0.1 to 0.95	Fraction of the field runoff that enters the most concentrated 10% of the strip. Runoff is rarely spread evenly: 10% of the strip often receives 25 to 75% of the runoff. A value of 0 or less is replaced by 0.5; other values are kept between 0.1 and 0.95.
5	cha_q	fraction	0 to 0.95	Fraction of the flow in the most concentrated 10% of the strip that is fully channelized. Channelized flow passes the strip without filtering or infiltration. Values above 0.95 are set to 0.95.

fld_vfs divides the field area to give the strip area, so it must be greater than 0 when the strip is switched on.

```
filterstrip.str: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                  flag       fld_vfs       con_vfs         cha_q  description
field_border             0       0.10000       0.00300       0.20000  Field_border
high_engineered          0       0.10000       0.00100       0.05000  Highly_engineered_low_channelized
```

Both records in this example have flag = 0, so a land use pointing to either one gets no filter strip.

grassedww.str

Grassed waterway parameter sets, referenced from the grww column of landuse.lum. A grassed waterway is a broad, shallow vegetated channel that takes runoff away from a field. The vegetation slows the flow, which protects the channel from scour and lets sediment and attached pollutants settle. SWAT+ simulates it as a trapezoidal channel with 8:1 side slopes.

#	Column	Units	Typical range	Default used when 0	Description
1	`name`	-	-	-	Name of the grassed waterway record.
2	`flag`	-	-	-	Read but not used. The waterway is simulated whenever the land use names a record.
3	`mann`	-	0.001 to 0.5	0.35	Manning's n of the waterway.
4	`sed_co`	-	0 to 1	0.005	Sediment transport coefficient, used to compute the transport capacity of the waterway.
5	`dp`	m	0 to 10	3/64 of `wd`	Depth of the waterway.
6	`wd`	m	0 to 1000	-	Top width of the waterway.
7	`len`	km	0 to 10000	square root of the HRU area	Length of the waterway.
8	`slp`	m/m	0 to 1	0.75 × HRU slope	Slope of the waterway.

Values of 0 or less in `mann`, `sed_co`, `dp`, `len` and `slp` are replaced by the defaults shown. If `dp` and `wd` together cannot form a trapezoid with 8:1 side slopes (the bottom width would be zero or negative), `dp` is reset to 3/64 of `wd`.

```
grassedww.str: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                  flag          mann        sed_co            dp            wd           len
grwway_high              0       0.05000       0.02000       1.00000       4.00000       0.50000       0.1
grwway_med               0       0.05000       0.02000       0.75000       3.00000       0.75000       0.0
grwway_low               0       0.05000       0.02000       0.50000       2.00000       1.00000       0.0
```

bmpuser.str

User-defined BMP removal efficiencies, referenced from the `bmp` column of `landuse.lum`. Many conservation practices are not modelled explicitly in SWAT+ but have published removal efficiencies. This record applies fixed reductions to the loads leaving the HRU. Surface runoff volume is not reduced, which keeps the water balance closed.

#	Column	Units	Range	Description
1	`name`	-	-	Name of the user BMP record.
2	`flag`	-	-	Read but not used. The BMP is applied whenever the land use names a record.
3	`sed_eff`	fraction	0 to 1	Sediment removal.
4	`ptlp_eff`	fraction	0 to 1	Particulate P removal (organic P and sorbed mineral P).
5	`solp_eff`	fraction	0 to 1	Soluble P removal in surface runoff.
6	`ptln_eff`	fraction	0 to 1	Particulate (organic) N removal.
7	`soln_eff`	fraction	0 to 1	Soluble N removal: nitrate in surface runoff and in lateral flow.
8	`bact_eff`	fraction	0 to 1	Bacteria removal. Read but not used.

Each load is multiplied by $(1 - \text{eff})$, so the efficiencies are fractions: 0.2 removes 20%. A value of 20 would give a negative load.

```
bmpuser.str: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                  flag        sed_eff       ptlp_eff       solp_eff       ptln_eff       soln_eff       bact
bmpusr1                  1        0.20000        0.20000        0.20000        0.20000        0.20000        0.2
```

satbuffer.str

Saturated buffer links. A saturated buffer diverts part of the tile flow from a source HRU into the soil of a receiving (buffer) HRU. Each record links one source to one receiver. The file is read only if a file named `satbuffer.str` is present in the run folder.

#	Column	Units	Description
1	`name`	-	Name of the buffer record.
2	`hru_src`	-	Number of the source HRU whose tile flow feeds the buffer.
3	`frac_src`	fraction	Fraction of the source HRU contributing tile flow to the buffer.
4	`flocon_dtbl`	-	Name of the flow control decision table (in the flow control decision table file) that sets how much tile flow goes into the buffer.
5	`hru_rcv`	-	Number of the receiving (buffer) HRU.
6	`lyr`	-	Soil layer of the receiving HRU that takes the diverted tile flow (0 = surface).

urban.urb

Urban land type database, referenced from the `urban` column of `landuse.lum`. The `urb_ro` column of `landuse.lum` selects the urban load method: `usgs_reg` (USGS regression equations) or `buildup_washoff` (build-up and wash-off of solids from impervious surfaces).

#	Column	Units	Built-in value	Description
1	`name`	-	-	Name of the urban land type.
2	`frac_imp`	fraction	0.05	Fraction of the HRU area that is impervious.
3	`frac_dc_imp`	fraction	0.05	Fraction of the HRU area that is directly connected impervious area.
4	`curb_den`	km/ha	0	Curb length density.
5	`urb_wash`	1/mm	0	Wash-off coefficient for removal of constituents from impervious surfaces.
6	`dirt_max`	kg/curb km	1000	Maximum amount of solids that can build up on impervious surfaces.
7	`t_halfmax`	days	1	Time for solids on impervious surfaces to build up to half of `dirt_max`.
8	`conc_totn`	mg N/kg sed	0	Total nitrogen concentration in suspended solids from impervious areas.
9	`conc_totp`	mg P/kg sed	0	Total phosphorus concentration in suspended solids from impervious areas.
10	`conc_no3n`	mg N/kg sed	0	Nitrate nitrogen concentration in suspended solids from impervious areas.
11	`urb_cn`	-	98	Curve number for impervious areas (moisture condition II). Read but not used: 62.0.1 uses a fixed curve number of 98 for impervious areas.

With the curve number method, surface runoff from an urban HRU is a weighted mix of runoff from the pervious part (the HRU's daily curve number) and from the directly connected impervious part (curve number 98), with `frac_dc_imp` as the weight. `frac_imp` is the weight for urban sediment and nutrient loads in both urban load methods.

```
urban.urb: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                  frac_imp   frac_dc_imp      curb_den      urb_wash      dirt_max     t_halfmax     c
urhd                   0.60000       0.44000       0.24000       0.18000     225.00000       0.75000     5
urmd                   0.38000       0.30000       0.24000       0.18000     225.00000       0.75000     5
```

References

- McCray, J.E., Kirkland, S.L., Siegrist, R.L. and Thyne, G.D. (2005). Model parameters for simulating fate and transport of on-site wastewater nutrients. Ground Water 43(4), 628–639.
- Siegrist, R.L., McCray, J., Weintraub, L., Chen, C., Bagdol, J., Lemonds, P., Van Cuyk, S., Lowe, K., Goldstein, R. and Rada, J. (2005). Quantifying site-scale processes and watershed-scale cumulative effects of decentralized wastewater systems. Project No. WU-HT-00-27, National Decentralized Water Resources Capacity Development Project, Washington University, St. Louis, MO.
- White, M.J. and Arnold, J.G. (2009). Development of a simplistic vegetative filter strip model for sediment and nutrient retention at the field scale. Hydrological Processes 23, 1602–1616. doi:10.1002/hyp.7291

Related files

- file.cio: the `structural` line names the five `.str` files; the `hru_parm_db` line names `urban.urb` and `septic.sep`.
- landuse.lum: points to records in all of these files.
- codes.bsn: `tiledrain` and `wtable` select the tile flow and water table methods.
- Decision tables: actions that install practices or change drain depth during a run.

Sources

- Readers, called from proc_db.f90:
  - sdr_read.f90 (`tiledrain.str`)
  - sep_read.f90 (`septic.str`)
  - septic_parm_read.f90 (`septic.sep`)
  - scen_read_filtstrip.f90 (`filterstrip.str`)
  - scen_read_grwway.f90 (`grassedww.str`)
  - scen_read_bmpuser.f90 (`bmpuser.str`)
  - sat_buff_read.f90 (`satbuffer.str`)
  - urban_parm_read.f90 (`urban.urb`)
- File name slots: `type input_structural` and `type input_parameter_databases` in input_file_module.f90
- Type definitions: `subsurface_drainage_parameters` and `saturated_buffer_parameters` in hru_module.f90; `septic_system` and `septic_db` in septic_data_module.f90; `filtstrip_operation`, `grwaterway_operation` and `bmpuser_operation` in mgt_operations_module.f90; `urban_db` in urban_data_module.f90
- Name lookup and assignment to HRUs: landuse_read.f90, hru_lum_init.f90, structure_init.f90, structure_set_parms.f90, plant_init.f90 (urban)
- Tile flow: swr_percmain.f90, swr_origtile.f90, swr_drains.f90, swr_substor.f90
- Septic: proc_hru.f90, soils_init.f90, sep_biozone.f90, hru_control.f90
- Filter strip, waterway, user BMP: smp_filter.f90, smp_grass_wway.f90, smp_bmpfixed.f90
- Urban: sq_daycn.f90, hru_urban.f90
- Decision table actions: actions.f90

File	Column	Fortran variable
tiledrain.str	dp, t_fc, lag, rad, dist, drain, pump, lat_ksat	sdr%depth, %time, %lag, %radius, %dist, %drain_co, %pumpcap, %latksat
septic.str	operation, residents, t_fail, dp_bioz, thk_bioz, cha_dist, sep_dens, bm_dens	sep%opt, %cap, %tfail, %z, %thk, %strm_dist, %density, %bd
septic.str	bod_decay, fc_lin, fc_exp, fecal_decay, tds_conv, mort, resp, slough1, slough2	sep%bod_dc, %fc1, %fc2, %fecal, %plq, %mrt, %rsp, %slg1, %slg2
septic.str	nit, denit, p_sorp, p_sorp_max, solp_slp, solp_int	sep%nitr, %denitr, %pdistrb, %psorpmax, %solpslp, %solpintc
septic.sep	name, q_rate, bod, tss, nh4_n, no3_n, no2_n, org_n, min_p, org_p, fcoli	sepdb%sepnm, %qs, %bodconcs, %tssconcs, %nh4concs, %no3concs, %no2concs, %orgnconcs, %minps, %orgps, %fcolis
filterstrip.str	flag, fld_vfs, con_vfs, cha_q	filtstrip_db%vfsi, %vfsratio, %vfscon, %vfsch
grassedww.str	flag, mann, sed_co, dp, wd, len, slp	grwaterway_db%grwat_i, %grwat_n, %grwat_spcon, %grwat_d, %grwat_w, %grwat_l, %grwat_s
bmpuser.str	flag, sed_eff, ptlp_eff, solp_eff, ptln_eff, soln_eff, bact_eff	bmpuser_db%bmp_flag, %bmp_sed, %bmp_pp, %bmp_sp, %bmp_pn, %bmp_sn, %bmp_bac
urban.urb	name, frac_imp, frac_dc_imp, curb_den, urb_wash, dirt_max, t_halfmax	urbdb%urbnm, %fimp, %fcimp, %curbden, %urbcoef, %dirtmx, %thalf
urban.urb	conc_totn, conc_totp, conc_no3n, urb_cn	urbdb%tnconc, %tpconc, %tno3conc, %urbcn2
landuse.lum	tile, sep, vfs, grww, bmp, urban	lum%tiledrain, %septic, %fstrip, %grasww, %bmpuser, %urb_lu

Purpose

SWAT+ simulates the pesticides listed in constituents.cs. It follows each one on plant foliage, in the soil layers, in surface runoff, lateral flow, tile flow and percolation, in channel water and bed sediment, in reservoirs and in aquifers. Four files describe them:

File	Named in	Content
pesticide.pes	hru_parm_db line of file.cio (default pesticide.pes)	Properties of each pesticide.
pest_hru.ini	init line of file.cio	Initial pesticide in soils and on plants.
pest_water.ini	init line of file.cio	Initial pesticide in channels, reservoirs and aquifers.
pest_metabolite.pes	fixed name, not in file.cio	Degradation products (daughters) of each parent pesticide.

Every pesticide named in constituents.cs must have a row in pesticide.pes with the same name. The database can contain many more pesticides than the run simulates. Pesticide applications in management.sch and decision tables name rows of pesticide.pes, and the application method comes from chem_app.ops.

pesticide.pes

Free format, whitespace-separated. Line 1 is a title and line 2 is the column header; both are skipped. Each later line is one pesticide with 17 values and a description, in the order below. The reader does not match columns by header name, so every value must be present and in this order.

#	Column	Units	Typical range	Description
1	name	-	-	Name of the pesticide, up to 16 characters. Must be unique.
2	soil_ads	(mg/kg)/(mg/L)	1 to 10^9	Soil adsorption coefficient normalized for soil organic carbon (K_{oc}).
3	frac_wash	fraction	0 to 1	Fraction of the pesticide on foliage washed off by a rainfall event.
4	hl_foliage	days	0 to 10,000	Half-life on foliage. 0 means no decay.
5	hl_soil	days	0 to 100,000	Half-life in the soil. 0 means no decay.
6	solub	mg/L	-	Solubility in water. Upper limit of the concentration in water leaving a soil layer.
7	aq_hlife	days	-	Half-life in channel, reservoir and aquifer water. 0 means no decay. Earlier documentation had aq_reac here, a reaction rate in 1/day; the column is now a half-life in days.
8	aq_volat	m/day	-	Volatilization coefficient in channel and reservoir water.
9	mol_wt	g/mol	-	Molecular weight. Used for the water and sediment mixing velocity and for the mass of metabolites.
10	aq_resus	m/day	-	Resuspension velocity of pesticide sorbed to bed sediment.
11	aq_settle	m/day	-	Settling velocity of pesticide sorbed to suspended sediment.
12	ben_act_dep	m	-	Depth of the active bed sediment layer.
13	ben_bury	m/day	-	Burial velocity in bed sediment.

#	Column	Units	Typical range	Description
14	`ben_hlife`	days	-	Half-life in bed sediment. 0 means no decay. Earlier documentation had `ben_reac` here, a reaction rate in 1/day; the column is now a half-life in days.
15	`pl_uptake`	-	0 and up	Transpiration stream concentration factor (TSCF): ratio of the concentration in the water taken up by roots to the dissolved concentration in the soil. 0 turns plant uptake off.
16	`harvest_sink`	0 or 1	0, 1	1: pesticide taken up by the plant leaves the system. 0: it stays in the plant and returns to the soil with the residue when the plant is harvested or killed.
17	`resistance_n`	fraction	0 to 1	Soil water used for the dissolved concentration in plant uptake. 0 uses the actual soil water, 1 uses field capacity, values in between blend the two.
18	`description`	-	-	Free text, one word.

Columns 15 to 17 are new in 62.0.1. Files written for earlier revisions have 14 values (name and 13 numbers) and then a description. With such a file the first 14 values of each row are read correctly, and the read of the row stops with an error when it meets the description where `pl_uptake` is expected. No message is written and the next row is read normally. The three new columns keep their built-in values (`pl_uptake` = 0, `harvest_sink` = 1, `resistance_n` = 1), so there is no plant uptake, which is the behaviour of earlier revisions. To use plant uptake, add the three columns before the description.

Half-lives and decay

Each day the pesticide in a compartment decays by first-order kinetics. The fraction left after one day is

$$f = \exp\left(-\frac{0.693}{t_{1/2}}\right)$$

where $t_{1/2}$ is the half-life of that compartment (`hl_foliage`, `hl_soil`, `aq_hlife` or `ben_hlife`). The half-life entered is a lumped value covering volatilization, photolysis, hydrolysis, biological degradation and other reactions. In channels the decay is scaled by the travel time through the reach, up to one day.

For most pesticides the foliar half-life is much shorter than the soil half-life because of volatilization and photodecomposition. If no foliar value is available, a common estimate is 0.25 to 0.5 times the soil half-life, lower for pesticides with a vapor pressure below 10^{-5} mm Hg, and at most 30 days.

Temperature-dependent decay

Column 7 of `parameters.bsn` switches temperature-dependent decay on. The SWAT+ Editor writes this column under the header `exp_sed`. When its value is 1, each half-life is taken as the value at 25 °C and adjusted with a Q10 of 2.58:

$$t_{1/2}(T) = t_{1/2,25}\ 2.58^{(25-T)/10}$$

The temperature T is the soil layer temperature for soil, the top soil layer temperature for foliage, and the water temperature for channel water and bed sediment. For aquifers the temperature is 0 °C in 62.0.1, so aquifer half-lives are about 10.7 times the 25 °C value. Reservoirs do not use the adjustment.

Any other value in that column gives the constant decay above. Check this column when a model is moved to 62.0.1: editor files commonly have 1.0 there, which switches the adjustment on.

Sorption and transport

The soil adsorption coefficient of a layer is

$$K_d = K_{oc} \frac{OC}{100}$$

where OC is the organic carbon content of the layer in percent. The pesticide in a layer is split between the soil and the water flowing through it according to K_d. The concentration in water leaving a layer is limited to `solub`. The concentration in surface runoff is the concentration in percolate times `pest_perc` of `parameters.bsn`.

On a day with at least 2.54 mm of precipitation, `frac_wash` of the pesticide on each plant's foliage is washed off to the top soil layer.

Plant uptake

When `pl_uptake` is above 0, growing plants take up pesticide from each soil layer with the water they transpire:

$$U = \text{TSCF} \frac{W_{up}}{\theta_{eff} + W_{up} + K_d \rho_b h} M$$

where W_{up} is the water taken up from the layer (mm), θ_{eff} is the soil water blended toward field capacity by `resistance_n`, ρ_b the bulk density, h the layer thickness (mm) and M the pesticide in the layer (kg/ha). Uptake is reported as `pl_uptake` in the pesticide output. With `harvest_sink` = 0, the pesticide kept in the plant returns to the soil when the plant is harvested or killed.

Metabolites

When a parent pesticide decays, a daughter listed in `pest_metabolite.pes` receives

$$M_{daughter} = \Delta M_{parent} \, f_{frac} \frac{MW_{daughter}}{MW_{parent}}$$

where f_{frac} is the daughter fraction for that compartment and MW the `mol_wt` values. Foliage uses `foliar_fr`, soil uses `soil_fr`, channel water uses `aq_fr` and channel bed sediment uses `ben_fr`. In 62.0.1 reservoirs (water and bed) and aquifers use `soil_fr`.

Calibration

These parameters can be changed during calibration through `calibration.cal`: `pst_koc`, `pst_washoff`, `pst_foliar_hlife`, `pst_soil_hlife`, `pst_solub`, `pst_aq_hlife`, `pst_aq_volat`, `pst_aq_settle`, `pst_aq_resus`, `pst_ben_hlife`, `pst_ben_bury` and `pst_ben_act_dep`. The daily decay factors are computed from the half-lives when `pesticide.pes` is read, so a calibration change to a half-life only has an effect when temperature-dependent decay is on.

pesticide.pes example

The first rows of `refdata/Osu_1hru/pesticide.pes` in the SWAT+ repository, as shipped:

```
pesticide.pes: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                  soil_ads     frac_wash    hl_foliage       hl_soil          solub       aq_hlife
245-tp              2600.00000       0.40000       5.00000      20.00000        2.50000        0.00700
2plus2                20.00000       0.95000      10.00000      21.00000   660000.00000        0.00700
```

The file was written for revision 60.5.4 and has no `pl_uptake`, `harvest_sink` or `resistance_n` columns. As described above, each row is read up to `ben_hlife`, the description is not stored, and the three new columns take their built-in values, so no pesticide is taken up by plants.

Silvex (`245-tp`) sorbs strongly (K_{oc} of 2600) and has a soil half-life of 20 days. Mecoprop (`2plus2`) is very soluble and hardly sorbs.

Every row has `aq_hlife` = 0.007 and `ben_hlife` = 0.05. These match the reaction coefficients in 1/day of the older database format, but 62.0.1 reads them as half-lives in days. A half-life of 0.007 days leaves a fraction $\exp(-0.693/0.007) \approx 10^{-43}$ after one day, and 0.05 days leaves about 10^{-6}. Osu_1hru has 1 in column 7 of `parameters.bsn`, so the half-lives are adjusted for temperature (except in reservoirs), which lengthens them at temperatures below 25 °C but not enough to change the result. Any of these pesticides that reached channel water, bed sediment, a reservoir or an aquifer would be almost entirely gone within a day. Osu_1hru has no `constituents.cs`, so in that model no pesticide is simulated and the file is read without effect.

pest_hru.ini

Initial pesticide in the soil and on plants of HRUs. An HRU selects a record by name in the `pest` column of `soil_plant.ini`.

The file is read in two passes that expect different layouts:

- The first pass counts records. It expects a title line, then for each record a header line, a name line and one line per pesticide.
- The second pass reads the values. It expects a title line and one header line, then for each record a name line and one line per pesticide.

A file with one record satisfies both. With more than one record the second pass reads the header line of record 2 as its name and the values that follow are shifted, so use one record.

Each pesticide line has three values, one line per pesticide in the order of `constituents.cs`:

Value	Units	Description
name	-	Pesticide name, for readability. Not checked.
soil	mg/kg	Concentration in the soil. The same concentration is given to every soil layer.
plant	kg/ha	Pesticide on the foliage. It is shared among the plants by leaf area; if there is no leaf area at the start, nothing is put on the foliage.

pest_water.ini

Initial pesticide in channels, reservoirs, wetlands and aquifers. Channels, reservoirs and wetlands select a record by name in their initial condition files (`initial.cha`, `initial.res`), and aquifers in `initial.aqu`.

Line 1 is a title and line 2 a header. Each record starts with a name line. The values follow as one list: a label, then the water value of each pesticide, then the bed sediment value of each pesticide, in the order of `constituents.cs`. Line breaks within the list are free, but the counting pass expects two lines per pesticide after the name line. For two pesticides:

```
pest_water.ini: initial pesticide in water
name
ch_init
atrazine  0.002
0.001
0.05
0.02
```

Here `atrazine` is the label, the water values are 0.002 and 0.001, and the bed sediment values 0.05 and 0.02.

Value	Units	Description
water	mg/L	Concentration in the water. Channels multiply it by the initial channel water volume. Aquifers read it as mg/kg of aquifer material over the aquifer depth with a bulk density of 2.0 t/m³.
benthic	mg/kg	Concentration in the active bed sediment layer. Channels multiply it by the volume of the active bed layer.

Reservoirs use both values as given, without a conversion by volume.

pest_metabolite.pes

The degradation products of each parent pesticide. SWAT+ reads it if a file of this name exists in the run directory.

Line 1 is a title and line 2 a header. For each parent there is one line with the parent name and the number of daughters, followed by one line per daughter:

Value	Units	Description
`name`	-	Daughter pesticide name. Must also be listed in `constituents.cs` and `pesticide.pes`.
`foliar_fr`	fraction	Fraction of the parent decayed on foliage that becomes this daughter.
`soil_fr`	fraction	Fraction of the parent decayed in the soil that becomes this daughter. Also used in reservoirs and aquifers.
`aq_fr`	fraction	Fraction of the parent decayed in channel water that becomes this daughter.
`ben_fr`	fraction	Fraction of the parent decayed in channel bed sediment that becomes this daughter.

The parent name must match a row of `pesticide.pes`. If it does not, its daughter lines are not skipped and the rest of the file is read out of step.

```
pest_metabolite.pes: metabolites
parent        num_metab
aatrex        1
deethyl_atr   0.2   0.3   0.1   0.1
```

Atrazine (`aatrex`) has one daughter. 30% of the atrazine decayed in the soil, adjusted by the ratio of molecular weights, becomes `deethyl_atr`.

Related files

- `constituents.cs`: the list of pesticides simulated.
- `soil_plant.ini`: links HRUs to `pest_hru.ini` records.
- `initial.cha`, `initial.res`, `initial.aqu`: link channels, reservoirs and aquifers to `pest_water.ini` records.
- `chem_app.ops` and `management.sch`: pesticide applications.
- `parameters.bsn`: `pest_perc` and the temperature-dependent decay switch in column 7.
- Pathogens and heavy metals use the same initial condition layout.

Sources

- Readers: `pest_parm_read.f90`, `pest_hru_aqu_read.f90`, `pest_cha_res_read.f90`, `pest_metabolite_read.f90`
- Types: `type pesticide_db`, `type pesticide_cp`, `type daughter_decay_fractions` in `pesticide_data_module.f90`; `type cs_soil_init_concentrations` and `type cs_water_init_concentrations` in `constituent_mass_module.f90`
- Pesticide list crosswalk: `constit_db_read.f90`
- Initial conditions: `pesticide_init.f90`, `sd_hydsed_init.f90`, `res_initial.f90`, `aqu_read_init_cs.f90`

- Landscape processes: pest_decay.f90, pest_washp.f90, pest_lch.f90, pest_pl_up.f90, mgt_harvbiomass.f90, mgt_harvtuber.f90
- Temperature adjustment: pest_tempdecay_module.f90
- Channels, reservoirs and aquifers: ch_rtpest.f90, res_pest.f90, aqu_1d_control.f90
- Switch in parameters.bsn: bsn_prm%temp_decay in basin_module.f90
- Calibration parameters: cal_parm_select.f90

Column	Fortran variable
name	pestdb%name
soil_ads	pestdb%koc
frac_wash	pestdb%washoff
hl_foliage	pestdb%foliar_hlife
hl_soil	pestdb%soil_hlife
aq_hlife, aq_volat, aq_resus, aq_settle	pestdb% same names
ben_act_dep, ben_bury, ben_hlife	pestdb% same names
pl_uptake, harvest_sink, resistance_n	pestdb% same names
description	pestdb%descrip
parameters.bsn column 7 (exp_sed)	bsn_prm%temp_decay
pest_hru.ini record	pest_soil_ini%name, %soil, %plt
pest_water.ini record	pest_init_name, pest_water_ini%water, %benthic
pest_metabolite.pes daughter	pestcp%daughter

Purpose

SWAT+ can follow pathogens (bacteria and other microorganisms) listed in constituents.cs. Three files describe them:

File	Named in	Content
pathogens.pth	hru_parm_db line of file.cio (default pathogens.pth)	Die-off, growth and transport properties of each pathogen.
path_hru.ini	init line of file.cio	Initial pathogens in soils and on plants.
path_water.ini	init line of file.cio	Initial pathogens in channels, reservoirs and aquifers.

Every pathogen named in constituents.cs should have a row in pathogens.pth with the same name.

The pathogen code in 62.0.1 does not yet work as a complete model. Several columns are read but not used, and the landscape and channel routines have the limitations listed in Behaviour in 62.0.1. Read that section before using pathogen results.

pathogens.pth

Free format, whitespace-separated. Line 1 is a title and line 2 is the column header; both are skipped. Each later line is one pathogen with the 18 values below in this order. The reader does not match columns by header name.

#	Column	Units	Used in 62.0.1	Description
1	name	-	yes	Pathogen name, up to 16 characters.
2	do_soln	1/day	yes	Die-off factor in soil solution at 20 °C.
3	gr_soln	1/day	yes	Growth factor in soil solution at 20 °C.
4	do_sorb	1/day	no	Die-off factor for pathogens sorbed to soil particles at 20 °C.

#	Column	Units	Used in 62.0.1	Description
5	`gr_sorb`	1/day	no	Growth factor for pathogens sorbed to soil particles at 20 °C.
6	`kd`	-	yes	Partition coefficient between solution and sorbed phases.
7	`t_adj`	-	yes	Temperature adjustment factor for die-off and growth.
8	`washoff`	fraction	yes	Fraction of the pathogens on foliage washed off by a rainfall event.
9	`do_plnt`	1/day	yes	Die-off factor on foliage at 20 °C.
10	`gr_plnt`	1/day	yes	Growth factor on foliage at 20 °C.
11	`fr_manure`	fraction	no	Fraction of manure containing active colony forming units.
12	`perco`	-	yes	Percolation coefficient: ratio of the concentration in percolate to the concentration in the surface layer solution.
13	`det_thrshd`	cfu/m²	no	Detection threshold below which soil pathogens would be set to zero.
14	`do_stream`	1/day	yes	Die-off factor in streams at 20 °C.
15	`gr_stream`	1/day	no	Growth factor in streams at 20 °C.
16	`do_res`	1/day	no	Die-off factor in reservoirs at 20 °C.
17	`gr_res`	1/day	no	Growth factor in reservoirs at 20 °C.
18	`conc_min`	cfu/m²	yes	Minimum pathogen level on foliage and in the soil surface layer.

Die-off and growth

On foliage and in the top soil layer the pathogen count N changes each day by the net die-off rate, adjusted for the mean air temperature T:

$$N_d = N_{d-1}\,\exp\!\left(-(k_{die} - k_{grow})\,\theta^{\,T-20}\right) - c_{min}$$

where k_{die} and k_{grow} are `do_plnt` and `gr_plnt` on foliage, or `do_soln` and `gr_soln` in the soil, θ is `t_adj` and c_{min} is `conc_min`. The result is not allowed to fall below `conc_min`.

On a day with at least 2.54 mm of precipitation, `washoff` of the pathogens on each plant's foliage moves to the top soil layer.

path_hru.ini

Initial pathogens in the soil and on plants of HRUs. An HRU selects a record by name in the `path` column of `soil_plant.ini`.

Line 1 is a title. Each record has a header line, a name line, and then the values as one list: a label, the soil value of each pathogen, then the plant value of each pathogen, in the order of `constituents.cs`. Line breaks within the list are free, but the list must take one line per pathogen so that the records are counted correctly. For two pathogens:

```
path_hru.ini: initial pathogens
name
hru_init
ecoli  1000.  500.
50.  10.
```

Here `ecoli` is the label, the soil values are 1000 and 500 and the plant values 50 and 10.

Value	Units	Description
soil	cfu/m²	Pathogens in the top soil layer.
plant	cfu/m²	Pathogens on the foliage.

path_water.ini

Initial pathogens in channels, reservoirs and aquifers. Reservoirs and channels select a record by name in their initial condition files (initial.res, initial.cha), and aquifers in initial.aqu.

Line 1 is a title and line 2 a header. The record has a name line and then the values as one list: a label, the water value of each pathogen, then the bed sediment value of each pathogen. The list must take two lines per pathogen. Use one record (see below).

Value	Units	Description
water	-	Pathogens in the water. The value is used as given, without conversion, so give it in the units of the pathogen loads in the model.
benthic	-	Pathogens in the bed sediment, used as given.

Behaviour in 62.0.1

- Columns `do_sorb`, `gr_sorb`, `fr_manure`, `det_thrshd`, `gr_stream`, `do_res` and `gr_res` are not used.
- The landscape routines (die-off, wash-off, runoff and percolation) do not look up each pathogen's row by name. They use the row of `pathogens.pth` whose position equals the position of the HRU's record in `path_hru.ini`, and use that one row for every pathogen.
- Each day the runoff step removes all pathogens left in the top soil layer after percolation and reports them as lost in surface runoff, whether or not there is runoff that day.
- Initial soil pathogens from `path_hru.ini` are set to zero again for any soil with more than one layer. Initial plant pathogens are written to the balance output but not put on the foliage.
- `path_water.ini` is read with a header line before each record in the reading pass and without one in the counting pass, so only a file with one record is read correctly.
- Channels take their initial pathogens from the `path_water.ini` record in the position of the channel's pesticide initial record, not its pathogen record. Aquifers take the name match from `path_water.ini` but the values from the `path_hru.ini` soil column of the record in the same position. Reservoirs use their `path_water.ini` record.
- The channel die-off step does not identify the channel being routed, so `do_stream` does not act on the routed water.

Related files

- constituents.cs: the list of pathogens simulated.
- soil_plant.ini: links HRUs to `path_hru.ini` records.
- initial.cha, initial.res, initial.aqu: link channels, reservoirs and aquifers to `path_water.ini` records.
- Pesticides and heavy metals use a similar initial condition layout.

Sources

- Readers: path_parm_read.f90, path_hru_aqu_read.f90, path_cha_res_read.f90
- Types: `type pathogen_db` in pathogen_data_module.f90; `type cs_soil_init_concentrations` and `type cs_water_init_concentrations` in constituent_mass_module.f90
- Pathogen list crosswalk: constit_db_read.f90; `soil_plant.ini` link: hru_read.f90
- Initial conditions: pathogen_init.f90, sd_hydsed_init.f90, res_initial.f90, aqu_read_init_cs.f90
- Landscape processes: path_ls_swrouting.f90, path_ls_runoff.f90, path_ls_process.f90, called from hru_control.f90
- Channel: ch_rtpath.f90; temperature function: theta.f90

Column	Fortran variable
name	path_db%pathnm
other pathogens.pth columns	path_db% same names
path_hru.ini record	path_soil_ini%name, %soil, %plt
path_water.ini record	path_init_name, path_water_ini%water, %benthic

Purpose

Heavy metals listed in constituents.cs can be routed through the network as loads. In 62.0.1 they enter the model only through export coefficients (exco_hmet.exc) and delivery ratios (dr_hmet.del) and are routed with the water. There is no metal property database, and soils, plants, channels and reservoirs have no metal processes.

file.cio has slots for four metal files:

File	Named in	Status in 62.0.1
metals.mtl	hru_parm_db line of file.cio	No reader. The file is not opened.
hmet_hru.ini	init line of file.cio	Read, but the values are not used.
hmet_water.ini	init line of file.cio	No reader. The file is not opened.
exco_hmet.exc, dr_hmet.del	export coefficient and delivery ratio lines of file.cio	Read. Metal loads from routing objects.

Set metals.mtl, hmet_hru.ini and hmet_water.ini to null in file.cio unless a later revision uses them. The hmet columns of initial.cha and initial.aqu name records of these files and have no effect.

hmet_hru.ini

Initial heavy metals in the soil and on plants of HRUs. The reader expects this layout:

- Line 1: title.
- For each record: a header line, a name line, then for each metal in the order of constituents.cs two lines, label soil and then label plant.

The counting pass expects one line per metal instead of two, so files with more than one record are miscounted. The values (soil and plant concentrations in mg/kg) are stored but no part of the model uses them.

Related files

- constituents.cs: the list of metals.
- file.cio: the file name slots.
- Pesticides and pathogens: the constituents with full process models.

Sources

- Reader: hmet_hru_aqu_read.f90, called from proc_read.f90
- Type: type cs_soil_init_concentrations (hmet_soil_ini) in constituent_mass_module.f90
- File name slots: in_parmdb%hmetcom_db, in_init%hmet_soil, in_init%hmet_water in input_file_module.f90
- Metal list: constit_db_read.f90
- Metal loads: exco_read_hmet.f90, dr_read_hmet.f90

Item	Fortran variable
hmet_hru.ini record name	hmet_soil_ini%name
soil value	hmet_soil_ini%soil
plant value	hmet_soil_ini%plt
metals.mtl slot	in_parmdb%hmetcom_db

Purpose

gwflow is a cell-based groundwater model. When it is active it replaces the lumped aquifers, and aquifer.aqu, initial.aqu and the aquifer connect file are not used. The aquifer is divided into cells, and each day SWAT+ solves a water balance for every cell with Darcy flow between neighbouring cells. Cells exchange water with channels and receive recharge from HRUs or landscape units. Optional processes add groundwater ET, groundwater flow into the soil profile, saturation excess flow, tile drainage, pumping, exchange with reservoirs, wetlands, floodplains and canals, recharge ponds, phreatophyte uptake, solute transport and heat transport.

The inputs are spread over a set of small files, most of them with the `.gw` extension, one file for each part of the model. The files are not listed in file.cio; SWAT+ looks for them by name in the run directory. Units are metres, days and square metres unless a column says otherwise.

Activation

Set `gwflow` to 1 in codes.bsn. At start-up SWAT+ then looks for `chancell.gw`:

- If `chancell.gw` exists, gwflow is active. Unless object.cnt already gives a gwflow object count, SWAT+ counts the channel-cell rows in `chancell.gw`, uses that count as the number of gwflow objects, removes the aquifer objects from the object total, and reads the channel-cell connections from `gwflow.con`.
- If `chancell.gw` is missing or empty, `gwflow` is reset to 0 and the run uses the lumped aquifers.

When gwflow is active SWAT+ writes a log of what it read and which options it switched on or off to `gwflow_record`. Check this file first when a gwflow run does not behave as expected.

When the count comes from `chancell.gw`, SWAT+ skips three lines before it starts counting, while the file has two lines before the first data row. The first channel-cell row is therefore not counted and the count is one less than the number of rows. `chancell.gw` is then read for that many rows, so the last row of the file is not used. Giving the gwflow count in object.cnt avoids this.

File overview

File	Required	Read when	Purpose
`chancell.gw`	yes	always	Channel-cell links, streambed zone, depth zone and observation flag
`gwflow.con`	yes	always	Connect file for the channel-cell objects
`codes.gw`	yes	always	Grid size, process switches, time step and output switches
`zones.gw`	yes	always	Aquifer and streambed parameters by zone
`cells.gw`	yes	always	Properties and initial state of every cell
`cellcon.gw`	yes	always	Neighbours of every cell
`outputs.gw`	yes	always	Observation wells and output times
`hrucell.gw`	one of the two	`conn_type` 1, or 2 without `lsucell.gw`	HRU-cell overlaps for recharge
`lsucell.gw`	one of the two	`conn_type` 2, or 1 without `hrucell.gw`	Landscape unit-cell overlaps for recharge
`chan_depth.gw`	no	file exists	Daily channel water depth per depth zone
`pumpex.gw`	no	`pumpex` is 1 and file exists	Specified pumping from cells
`hru_pump.gw`	no	file exists	HRUs to report irrigation pumping for
`tile.gw`	no	`tile` is 1 and file exists	Tile drain parameters and tile cells
`rescell.gw`	no	`res` is 1 and file exists	Reservoir-cell links
`gwflow.wetland`	no	`wet` is 1 and file exists	Wetland bed thickness
`floodplain.gw`	no	`fp` is 1 and file exists	Floodplain-cell links

File	Required	Read when	Purpose
gwflow_canal.con	no	canal is 1 and file exists	Canal-cell links
phreato.gw, phreato_cell.gw	no	phreato.gw exists	Phreatophyte ET curve and cells
tvheads.gw	no	file exists	Time-varying heads for boundary cells
solute.gw	no	solute is 1 and file exists	Solute transport settings and solute parameters
cell_sol.gw	no	solute transport on and file exists	Initial solute concentration per cell
minerals.gw	no	solute transport on, salts simulated, file exists	Salt mineral fractions per cell
ponds.gw, pond_cell.gw	no	ponds.gw exists	Recharge ponds and their cells
pond_div.gw	no	ponds active and file exists	Daily diversion into each pond
sw_group.gw	no	file exists	Groups of channel cells for exchange output
transit.gw	no	structured grid and file exists	Cells for transit-time output

When a switch in codes.gw is on and its file is missing, SWAT+ notes it in gwflow_record and the process is not simulated. For tile drainage, floodplain exchange and solute transport the switch is also turned off for the rest of the run.

Format conventions

- Values are separated by spaces. Column widths do not matter.
- Line 1 of every file is a title line and is skipped. Most tables have a column header on line 2, also skipped, and data from line 3. Files with a different layout are described in the section for that file.
- Cell numbers run from 1 to ncell, in the order of cells.gw, and every file that names a cell uses these numbers.
- Channel numbers are the numbers of the channels in the channel connect file (chandeg.con). Reservoir numbers follow the reservoir connect file.
- In cells.gw and zones.gw, an optional value may be given as null, which means it is not set.

codes.gw

Grid size, process switches and output switches. Line 1 is a title. Line 2 is a row of key names and line 3 is a row of values, one value under each key. SWAT+ pairs keys and values by position and recognises the keys by name, so the order of the columns does not matter and a missing key keeps its default (0, or empty for grid_type).

Key	Units	Description
grid_type	-	structured (regular grid) or unstructured. Both use the same cell numbering.
ncell	-	Number of cells. cells.gw and cellcon.gw must have this many rows.
cell_size	m	Cell size of a structured grid. Read but not used; cell areas come from cells.gw.
n_rows, n_cols	-	Rows and columns of a structured grid. Used only by the structured-grid sections of tile.gw and minerals.gw.
bc_type	-	Default boundary condition for boundary cells: 1 = specified head, 2 = no flow. A cell can override it in cells.gw.
conn_type	-	How recharge reaches the cells: 1 = from HRUs (hrucell.gw), 2 = from landscape units (lsucell.gw). Any other value stops the run.
gw_soil	0/1	Groundwater transfer into the soil profile when the water table rises into it. Only with HRU connection.
satx	0/1	Saturation excess flow: groundwater above the ground surface goes to the channel nearest to the cell.
pumpex	0/1	Specified pumping from pumpex.gw.

Key	Units	Description
tile	0/1	Tile drainage from tile.gw.
res	0/1	Exchange with reservoirs from rescell.gw.
wet	0/1	Exchange with wetlands. Only with HRU connection.
fp	0/1	Exchange with floodplains from floodplain.gw.
canal	0/1	Seepage from canals in gwflow_canal.con.
solute	0/1	Solute transport from solute.gw.
heat	0/1	Heat transport.
time_step	day	Flow time step. The day is split into int(1/time_step) steps, so use 1 or a fraction of a day such as 0.5.
write_day, write_mon, write_yr, write_aa	0/1	Write the gwflow water balance files for each interval.
river_thresh	m	Distance subtracted from every channel-bed elevation in chancell.gw before the channel exchange is calculated. 0 uses the elevations as given.

With LSU connection, SWAT+ turns gw_soil and wet off and says so in gwflow_record.

Output switches in print.prt

The write_* keys are defaults. print.prt can list gwflow rows, and a row that is present replaces the corresponding setting before any gwflow output file is opened:

print.prt row	Controls	Without the row
gwflow_wb	Water balance files, by interval (replaces write_day to write_aa)	write_* keys of codes.gw
gwflow_obs	Observation well files, by interval	all four intervals written
gwflow_pump	HRU irrigation pumping output (on if any interval is y)	on
gwflow_flux	Canal, pond, tile, channel-group and channel-observation output (on if the daily or yearly column is y)	on
gwflow_heat	Heat balance files: daily, yearly and average annual columns	same intervals as the water balance
gwflow_solute	Solute balance files, by interval	same intervals as the water balance

The daily channel hydrograph separation file gwflow_chan_hydsep_day.txt is written when daily water balance output is on. The files are described in the gwflow output reference.

zones.gw

Aquifer and streambed parameters, one row per zone. Line 1 is a title and line 2 a column header. The zone number of a row is its position in the file (the first data row is zone 1); the zone_id column is not read.

#	Column	Units	Description
1	zone_id	-	Zone number. Not read; the row position decides.
2	aquifer_K	m/day	Horizontal hydraulic conductivity of the aquifer.
3	aquifer_Sy	fraction	Specific yield of the aquifer.
4	streambed_K	m/day	Hydraulic conductivity of the streambed, for channel cells that use this zone.
5	streambed_thick	m	Streambed thickness, for channel cells that use this zone.
6	thermal_K	J/day/m/°C	Thermal conductivity of the aquifer, used by heat transport. Optional; null or missing means 0.

Cells name a zone for conductivity and a zone for specific yield in `cells.gw`. Channel cells name a zone for streambed properties in `chancell.gw`.

cells.gw

One row per cell, in cell order. Line 1 is a title and line 2 a column header. Row i must have `id` equal to i, or the run stops with a message in `gwflow_record`. The first 14 columns are required. Columns 15 to 23 are optional: a row can end after column 14, and `null` in any of them means the value is not set.

#	Column	Units	Description
1	`id`	-	Cell number, 1 to `ncell`, equal to the row number.
2	`name`	-	Cell name, used in output files.
3	`gis_id`	-	GIS identifier of the cell, written to output files.
4	`status`	-	0 = inactive, 1 = active, 2 = boundary. Boundary cells keep their head and are treated by `bc_type`.
5	`elev`	m	Ground surface elevation.
6	`thck`	m	Aquifer thickness. The aquifer bottom is `elev - thck`.
7	`K_zone`	-	Zone (row of `zones.gw`) for hydraulic conductivity. Also used for `thermal_K`.
8	`Sy_zone`	-	Zone (row of `zones.gw`) for specific yield.
9	`delay`	day	Recharge delay through the unsaturated zone.
10	`exdp`	m	Extinction depth for groundwater ET, measured down from the ground surface. 0 turns groundwater ET off for the cell.
11	`init`	m	Initial groundwater head (elevation). A value below the aquifer bottom is raised to the bottom.
12	`x`	m	x coordinate of the cell centre.
13	`y`	m	y coordinate of the cell centre.
14	`area`	m²	Cell area.
15	`strK`	m/day	Streambed conductivity for this cell, in place of the zone value.
16	`strthick`	m	Streambed thickness for this cell, in place of the zone value.
17	`bc_type`	-	Boundary condition for this cell, in place of the default in `codes.gw`.
18	`tile_depth`	m	Tile depth for this cell, in place of the value in `tile.gw`.
19	`tile_area`	m²	Tile drainage area for this cell, in place of the value in `tile.gw`.
20	`tile_K`	1/day	Tile conductance for this cell, in place of the value in `tile.gw`.
21	`row`	-	Grid row of a structured cell (0 for unstructured). Written to output only.
22	`col`	-	Grid column of a structured cell (0 for unstructured). Written to output only.
23	`init_temp`	°C	Initial groundwater temperature, used by heat transport.

The x and y coordinates are used to find, for each cell, the nearest channel cell (for saturation excess flow and tile drainage) and, for each boundary cell, the nearest active cell, which receives the recharge that falls on the boundary cell.

cellcon.gw

The neighbours of every cell, which define where lateral flow can happen. Line 1 is a title and line 2 a column header. One row per cell, in cell order; the first value must equal the row number.

#	Column	Description
1	`cell_id`	Cell number, equal to the row number.
2	`neighbor_tot`	Number of neighbours.
3+	`neighbor_ids`	`neighbor_tot` cell numbers.

Flow between a cell and a neighbour uses Darcy's law with the harmonic mean of the two conductivities. The face width is the square root of the smaller of the two cell areas. No flow passes to an inactive neighbour or to a boundary neighbour with `bc_type` 2.

outputs.gw

Output settings, one per row. Line 1 is a title and line 2 a column header. Each row starts with a category and a value; anything after the value (SWAT+ Editor writes a note) is ignored. Rows can be in any order.

`category`	`value`	Effect
`observation_cell`	cell number	Adds an observation well. Head, water table depth, temperature, nitrate and phosphorus at each well are written to `gwflow_obs_*.txt`. One row per well.
`head_output_time`	date as `YYYYDDD`	A date for a full head output. In 62.0.1 these dates are read but produce no output.
`detail_debug_cell`	cell number or 0	Read but not used.

chancell.gw

The cells crossed by channels, one row per channel-cell pair. Line 1 is a title and line 2 a column header. A cell crossed by two channels appears on two rows.

#	Column	Units	Description
1	`cell_id`	-	Cell crossed by the channel.
2	`elev_m`	m	Channel bed elevation in the cell.
3	`channel`	-	Channel number in `chandeg.con`.
4	`riv_length_m`	m	Length of the channel within the cell.
5	`zone`	-	Zone (row of `zones.gw`) for streambed conductivity and thickness.
6	`dep_zone`	-	Depth zone: column of `chan_depth.gw` that gives the daily water depth. Optional, default 0.
7	`obs`	0/1	1 writes daily channel exchange for this cell to `gwflow_chan_obs_flow_day.txt`. Optional, default 0.

Channel exchange

Exchange is calculated each day for every active channel cell with Darcy's law across the streambed. With bed elevation z_b (`elev_m` minus `river_thresh`), water depth d, stage $z_s = z_b + d$, groundwater head h, streambed conductivity K_b and thickness m_b, and flow area $A = w\,L$ (channel width times `riv_length_m`):

$$Q = \frac{K_b}{m_b}\,A \times \begin{cases} d & h < z_b \\ z_s - h & z_b < h < z_s \\ -(h - z_s) & h > z_s \end{cases}$$

A positive Q (m^3/day) is seepage from the channel into the aquifer and is limited to the water in the channel. A negative Q is groundwater discharge to the channel and is limited to the water stored in the cell. The width

w is the channel width from the channel data. The depth d is the channel depth from the channel data, unless `chan_depth.gw` is present, in which case it is the day's depth for the cell's depth zone.

gwflow.con

The connect file for the channel-cell objects. It has the usual connect-file layout (title line, header line, then one row per object); see connect files. Each row is one channel cell, with the cell number in the `CELL` column and one outflow to the channel (`sdc`, the channel number, `tot`, 1.0).

hrucell.gw

Overlaps between HRUs and cells, used to pass recharge from HRUs to cells and, with `gw_soil`, groundwater back into the soil. Line 1 is a title and line 2 a column header.

#	Column	Units	Description
1	`hru`	-	HRU number.
2	`area_m2`	m²	Total area of the HRU.
3	`cell_id`	-	Cell that overlaps the HRU.
4	`overlap_m2`	m²	Area of the overlap.

Rows must be grouped by HRU and the HRUs must be in increasing order, since SWAT+ reads the rows for HRU 1, then HRU 2, and so on. A row with an HRU number outside 1 to the number of HRUs is skipped. Each HRU's recharge is split between its cells by `overlap_m2 / area_m2`.

The HRU's recharge delay is taken from the `delay` of its cells in `cells.gw`. When an HRU overlaps several cells, the delay of the last cell listed for that HRU is used.

lsucell.gw

Overlaps between landscape units and cells, used in place of `hrucell.gw` with LSU connection. The layout differs from the other tables:

Line	Content
1	Title
2	Number of landscape units in the model
3	Number n of landscape units connected to cells
next n lines	One landscape unit number per line
next 2 lines	Headers, skipped
rest	One row per overlap: landscape unit number, landscape unit area (m²), cell number, overlap area (m²)

The overlap rows must be grouped by landscape unit in increasing order. With LSU connection every HRU uses a recharge delay based on the mean `delay` of all cells.

SWAT+ Editor writes `lsucell.gw` as a title followed by rows of cell, landscape unit and area, which is a different layout from the one the model reads.

chan_depth.gw

Daily channel water depth for each depth zone. Line 1 is a title and line 2 a column header. Then one row per simulated day, in order: day of year, year, and one depth (m) for each depth zone 1 to N, where N is the largest `dep_zone` in `chancell.gw`. One row is read per day in sequence; the day and year values are not checked.

pumpex.gw

Specified pumping from cells, read when `pumpex` is 1. Line 1 is a title and line 2 a column header. One row per pumping period.

#	Column	Units	Description
1	`name`	-	Label, not used.
2	`cell_id`	-	Pumped cell.
3	`rate`	m^3/day	Pumping rate during the period.
4	`yr_start`	year	First year of the period.
5	`dy_start`	day of year	First day of the period.
6	`yr_end`	year	Last year of the period.
7	`dy_end`	day of year	Last day of the period.

Consecutive rows with the same `cell_id` form one pump with several periods (up to 1000). Rows for one cell that are not next to each other create separate pumps. Each date is converted to a day count from the start of the simulation as (year − first year) × 365 + day, without leap days. A year before the first simulated year is counted as day 1.

hru_pump.gw

HRUs whose groundwater irrigation pumping is written each day to `gwflow_cell_wb_ppag_obs_day.txt`. Line 1 is a title, line 2 is the number of HRUs, and then one HRU number per line.

tile.gw

Tile drainage, read when `tile` is 1. Each value is on a separate line:

Line	Content
1	Title
2	Tile depth below the ground surface (m), for all cells
3	Tile drainage area (m^2), for all cells
4	Tile conductance (1/day), for all cells
5	Group flag: 1 if tile cell groups follow, 0 if not
groups	If the flag is 1: the number of groups, then for each group a name line, the number of cells, and one cell number per line
next	Header line, skipped
rest	Tile flag per cell (1 = tile drained, 0 = not). Unstructured grid: one value per line for all `ncell` cells. Structured grid: `n_rows` lines of `n_cols` values.

Columns 18 to 20 of `cells.gw` replace the depth, area and conductance for single cells. Each day, an active tile cell whose head h is above the tile elevation z_t (`elev - tile_depth`) drains

$$Q_{tile} = A_{tile}\, K_{tile}\, (h - z_t)$$

in m^3/day to the channel nearest to the cell. With groups, the daily tile flow of each group is written to `gwflow_tile_group_day.txt`.

The structured-grid form of the tile flags depends on grid arrays that 62.0.1 does not set up, so give the tile flags in the unstructured form. SWAT+ Editor writes the three tile values on one line followed by a list of tile cells, which is a different layout from the one the model reads.

rescell.gw

Reservoir-cell links, read when `res` is 1.

Line	Content
1, 2	Title and a second line, skipped
3	Reservoir bed thickness (m), for all reservoirs
4	Reservoir bed conductivity (m/day), for all reservoirs
5	Number of rows that follow
6	Header, skipped
rest	`cell_id`, `res_id`, `res_stage`

`res_id` is the reservoir number; a row with 0 is skipped. Exchange uses the ground elevation of the cell from `cells.gw` as the reservoir bed elevation. `res_stage` is read but not used.

gwflow.wetland

Wetland bed thickness, read when `wet` is 1 and HRUs are connected to cells. Lines 1 and 2 are skipped. Each later row has a wetland name and a bed thickness (m). The digits in the name give the HRU number, so `wet019` sets the bed thickness of the wetland in HRU 19. Wetlands not listed use 0.25 m.

SWAT+ Editor writes this information to a file named `wetland.gw`, which the model does not read, so wetlands then use 0.25 m.

floodplain.gw

Floodplain-cell links, read when `fp` is 1. Line 1 is a title, line 2 the number of rows, line 3 a header, then one row per cell: cell number, channel number, floodplain bed conductivity (m/day) and area of the cell inside the floodplain (m^2). The area is limited to the cell area.

gwflow_canal.con

Canal-cell links, read when `canal` is 1. The canal dimensions, bed properties and active days come from the canal records in `water_canal.wal`. Line 1 is a title. Each later row describes one canal:

```
canal_id  n  cell_1 length_1 elev_1  cell_2 length_2 elev_2  ...
```

with `n` triplets of cell number, canal length in the cell (m) and canal bed elevation in the cell (m). A canal with a diversion source is treated as a diversion canal. A canal with no diversion and a start day above 0 is treated as a canal fed from outside the model. Any other canal is linked to a channel through water allocation.

phreato.gw and phreato_cell.gw

Groundwater uptake by phreatophytes. Both files are needed when `phreato.gw` exists; there is no switch in `codes.gw`.

`phreato.gw` is the curve of uptake rate against water table depth. Line 1 is a title and line 2 a header, then one point per row: depth to the water table (m) and ET rate (m/day), in order of increasing depth.

`phreato_cell.gw` lists the cells with phreatophytes. Line 1 is a title and line 2 a header, then one row per cell: cell number and area covered by phreatophytes (m^2).

tvheads.gw

Line 1 is a title and line 2 a header. Each later row gives a cell number and one head per simulated year. In 62.0.1 the file is read but the heads are not applied, so boundary cells keep their initial heads.

solute.gw

Solute transport settings, read when `solute` is 1.

Line	Content
1, 2	Title and a second line, skipped
3	Number of transport time steps per flow time step
4	Longitudinal dispersivity (m)
5	Skipped (SWAT+ Editor writes the number of solutes here)
rest	One row per solute: name, retardation factor, reaction rate constant (1/day), concentration in canal water from outside the model (g/m^3)

The solutes and their order are set by the model: nitrate and phosphorus first, then the salt ions and then the other constituents listed in `constituents.cs`. The file must have one row per solute in that order; the name column is not checked. Further values on a row are ignored. SWAT+ Editor writes an initial concentration as a fifth value, which the model does not read; initial values come from `cell_sol.gw`.

The retardation factor divides the transport time step, so 1 means no sorption. The reaction term adds concentration × groundwater volume × rate constant (g/day) to the cell, so a negative rate constant gives first-order decay.

When constituents are simulated, SWAT+ also tries to read per-cell reaction groups and shale maps after `cell_sol.gw`. In 62.0.1 no file is open at that point, so gwflow solute transport cannot run together with the constituent module.

cell_sol.gw

Initial solute concentration in every cell. Line 1 is a title and line 2 a header, then one row per cell in cell order: cell number and one concentration (g/m^3) per solute, in the order of `solute.gw`. Without this file all concentrations start at 0.

minerals.gw

Salt mineral fractions per cell, read when solute transport is on, salt ions are simulated and the file exists. Line 1 is a title, line 2 the number of minerals, line 3 a header, then for an unstructured grid one row per cell with one fraction per mineral. The structured-grid form is read but the values are not stored. SWAT+ Editor does not write this file.

ponds.gw, pond_cell.gw and pond_div.gw

Recharge ponds. They are active when `ponds.gw` exists, and `pond_cell.gw` must then exist too.

`ponds.gw` has a title line, a header line and one row per pond:

#	Column	Units	Description
1	`pond_id`	-	Pond number, matched in `pond_cell.gw`.
2	`area`	m^2	Pond area.
3	`chan`	-	Channel that supplies the diverted water. 0 if none.
4	`canal`	-	Canal that supplies the diverted water, used when `chan` is 0. 0 if none.
5	`unl`	0/1	1 = the diverted water comes from outside the model, used when `chan` and `canal` are 0.
6	`bed_k`	m/day	Conductivity of the pond bed.
7	`wsta`	-	Weather station for rainfall and evaporation.
8	`evap_co`	-	Evaporation coefficient. Daily evaporation is `evap_co` × potential ET of the station × pond area.
9	`start_yr`	year	Year the pond starts operating.

#	Column	Units	Description
10	start_mo	month	Month the pond starts operating.
11	start_day	day	Day of the month the pond starts operating.
12+	unl_conc_1 …	g/m³	Concentration of each solute in water from outside the model (none when solute transport is off).

The start date is converted to a day count with 365-day years and fixed month lengths; a start year before the first simulated year means day 1.

pond_cell.gw has a title line, a header line and one row per pond-cell connection: pond number, cell number and connection area (m²). The rows for a pond give its cells.

pond_div.gw has a title line, a header line and one row per simulated day: year, month, day and the volume diverted into each pond that day (m³), in the order of ponds.gw. One row is read per day in sequence, so the file needs a row for every simulated day; the dates are not checked. Without the file, no water is diverted into the ponds.

sw_group.gw

Groups of channel cells whose daily channel exchange is summed and written to gwflow_gwsw_group_day.txt. Line 1 is a title and line 2 a header. Each later row gives a group number, the number of cells n in the group and n cell numbers.

transit.gw

Cells for groundwater transit-time output. Read only for a structured grid. Line 1 is a title, line 2 a header, line 3 the number of cells, then one cell number per line. SWAT+ Editor does not write this file.

Example

Start of a set of inputs for a structured grid of 1454 cells, with illustrative values.

codes.gw (abbreviated):

```
codes.gw: written by SWAT+ editor
     grid_type         ncell     cell_size        n_rows        n_cols   ...      time_step     write_aa  i
    structured          1454       5000.00            52            53   ...           1.00            1
```

zones.gw:

```
zones.gw: written by SWAT+ editor
   zone_id     aquifer_K    aquifer_Sy   streambed_K streambed_thick     thermal_K
         1       0.50000       0.20000       0.00500         0.50000       0.00000
         2       2.00000       0.20000       0.00500         0.50000       0.00000
```

cells.gw (first row, abbreviated):

```
 id  name      gis_id  status   elev   thck K_zone Sy_zone  delay  exdp   init ...  row  col init_temp
  1  cell0001       1       2 395.00  34.99      1       1   0.00  1.00 390.00 ...   52    1      null
```

Cell 1 is a boundary cell. Its aquifer bottom is at 395.00 - 34.99 = 360.01 m and it starts with a head of 390.00 m.

chancell.gw:

```
chancell.gw: written by SWAT+ editor
 cell_id   elev_m  channel  riv_length_m  zone  dep_zone  obs
      42   326.00       66       1853.55     1         0    1
      43   324.50       66        912.10     1         0    0
```

Channel 66 crosses cells 42 and 43, both with streambed properties from zone 1. With `river_thresh` at 5.00, the exchange at cell 42 uses a bed elevation of 321.00 m. Daily exchange at cell 42 is written to the channel observation output.

Related files

- `codes.bsn`: `gwflow` switches the module on.
- `object.cnt`: can give the number of gwflow objects directly.
- `print.prt`: gwflow output rows.
- `hru-data.hru` and connect files: the HRUs, landscape units, channels and reservoirs that cells connect to.
- `water_canal.wal`: canal data used by canal seepage.
- `constituents.cs` and salt inputs: the salts and constituents added to the gwflow solutes.
- gwflow outputs.

Sources

- Activation and object count: `basin_read_objs.f90`
- Readers: `gwflow_chan_read.f90` (`chancell.gw`, `chan_depth.gw` header) and `gwflow_read.f90` (all other `.gw` files and `gwflow_canal.con`), both called from `hyd_connect.f90`
- Wetland bed thickness: `wet_read_hyd.f90`
- print.prt rows: `basin_print_codes_read.f90`; override logic in `gwflow_read`; output files opened in `gwflow_output_init` in `gwflow_output.f90`
- Variables: `gwflow_module.f90`
- Daily solution: `gwflow_simulate.f90`, `gwflow_lateral.f90`, `gwflow_rech.f90`, `gwflow_gwet.f90`, `gwflow_channel_exch.f90`, `gwflow_tile.f90`, `gwflow_reservoir.f90`, `gwflow_wetland.f90`, `gwflow_floodplain.f90`, `gwflow_canal.f90`, `gwflow_phreatophyte.f90`, `gwflow_pond.f90`, `gwflow_solute.f90`, `gwflow_chem.f90`, `gwflow_heat.f90`

File and column	Fortran variable
`codes.gw` `n_rows`, `n_cols`	`grid_nrow`, `grid_ncol`
`codes.gw` `bc_type`	`bc_type_int`, copied to `bc_type_array`
`codes.gw` `gw_soil`, `satx`, `pumpex`, `tile`, `res`, `wet`, `fp`, `canal`, `solute`, `heat`	`gw_soil_flag`, `gw_satx_flag`, `gw_pumpex_flag`, `gw_tile_flag`, `gw_res_flag`, `gw_wet_flag`, `gw_fp_flag`, `gw_canal_flag`, `gw_solute_flag`, `gw_heat_flag`
`codes.gw` `time_step`	`gw_time_step`
`codes.gw` `write_day` to `write_aa`	`gwflag_day`, `gwflag_mon`, `gwflag_yr`, `gwflag_aa`
`codes.gw` `river_thresh`	`gw_bed_change`
`zones.gw` columns 2 to 6	`zones_aquK`, `zones_aquSy`, `zones_strK`, `zones_strbed`, `zones_Kt`
`cells.gw` `status`, `elev`, `thck`, `exdp`, `init`, `x`, `y`, `area`	`gw_state%stat`, `%elev`, `%thck`, `%exdp`, `%init`, `%xcrd`, `%ycrd`, `%area`
`cells.gw` `delay`	`delay`, converted to `gw_delay = exp(-1/delay)` per HRU
`cells.gw` columns 15 to 20	`cell_strK_over`, `cell_strthick_over`, `bc_type_array`, `cell_tile_depth_over`, `cell_tile_area_over`, `cell_tile_K_over`
`cellcon.gw`	`gw_state%ncon`, `cell_con%cell_id`
`outputs.gw`	`gw_obs_cells`, `gw_output_yr` and `gw_output_day`, `gw_cell_obs_ss`
`chancell.gw`	`gw_chan_id`, `gw_chan_elev`, `gw_chan_chan`, `gw_chan_len`, `gw_chan_zone`, `gw_chan_dpzn`, `gw_chan_obs`
`tile.gw` lines 2 to 5	`gw_tile_depth`, `gw_tile_drain_area`, `gw_tile_K`, `gw_tile_group_flag`
`rescell.gw` lines 3 and 4	`res_thick`, `res_K`
`solute.gw`	`num_ts_transport`, `gw_long_disp`, `gwsol_sorb`, `gwsol_rctn`, `canal_out_conc`
`ponds.gw`	`gw_pond_info`

Purpose

The salt module tracks eight salt ions in soil water, groundwater, channels, reservoirs and wetlands, together with five salt minerals in soils and aquifers that dissolve and precipitate. Salt enters from initial conditions, fertilizer

and manure, road salt, atmospheric deposition and point sources, and leaves with plant uptake, runoff, percolation and flow. High soil salinity can reduce plant growth.

The salt files are not listed in `file.cio`; SWAT+ looks for them by name in the run directory. The constituent module, which uses the same master file, is on the constituent inputs page.

Activation

Salt simulation is switched on by listing salt ions in the salt block of `constituents.cs`. With no salt ions listed, or without `constituents.cs`, the salt files are ignored.

The ions must be listed in this order, because several routines use the position of an ion and not its name:

Position	Ion
1	`so4` (sulfate)
2	`ca` (calcium)
3	`mg` (magnesium)
4	`na` (sodium)
5	`k` (potassium)
6	`cl` (chloride)
7	`co3` (carbonate)
8	`hco3` (bicarbonate)

Every file below that has one value per ion gives the values in this order. Files with mineral fractions give five values in this order: $CaCO_3$, $MgCO_3$, $CaSO_4$, $MgSO_4$, NaCl.

File overview

File	Content	Linked to objects by
`salt_hru.ini`	Initial ion concentrations and mineral fractions in HRU soils	`salt` column of `soil_plant.ini`
`salt_aqu.ini`	Initial ion concentrations and mineral fractions in aquifers	`salt` column of `initial.aqu_cs`
`salt_channel.ini`	Initial ion concentrations in channels	`salt` column of `initial.cha_cs`
`salt_res`	Initial ion concentrations in reservoirs and wetlands	`salt` column of `reservoir.res_cs` and `wetland.wet_cs`
`salt_irrigation`	Ion concentrations in irrigation water from outside the model	position, see below
`salt_fertilizer.frt`	Ion content of fertilizers and manures	position in the fertilizer list
`salt_road`	Road salt applied to the land surface	atmospheric deposition stations
`salt_atmo.cli`	Ion concentration in rainfall and dry deposition	atmospheric deposition stations, see climate inputs
`salt_recall.rec`	Ion loads from point sources	see recall inputs
`salt_plants`	Salt tolerance of each plant	position in `plants.plt`
`salt_uptake`	Daily ion uptake by each plant	position in `plants.plt`
`salt_urban`	Ion content of sediment from impervious areas	urban type name in `urban.urb`

Missing files

`salt_hru.ini`, `salt_aqu.ini` and `salt_channel.ini` are opened whether or not they exist. If one is missing, SWAT+ creates an empty file of that name in the run directory and reads no records from it. An empty file, whether created this way or supplied, is read the same way: SWAT+ finds no title line and no records.

The test projects `refdata/Ames_sub1` and `refdata/Osu_1hru` in the SWAT+ repository contain empty `salt_hru.ini`, `salt_aqu.ini` and `salt_channel.ini` files created by earlier runs. Both projects give `null` for the constituent file in `file.cio`, so no salt ions are simulated and the empty files have no effect on the results. If salt ions are listed in `constituents.cs`, an empty file behaves as follows:

- Empty `salt_hru.ini`: there is no record for the HRUs, but HRU setup still reads record 1, which does not exist. Give at least one record.
- Empty `salt_aqu.ini`: no aquifer finds a record, so aquifers start with no salt ions and no salt minerals.
- Empty `salt_channel.ini`: there is no record for the channels, but channel setup still reads record 1, which does not exist. Give at least one record.

Some files are needed as soon as salt ions are listed, because the setup reads from them for every object without checking that they were found:

- `salt_irrigation` is read for every HRU during setup.
- `salt_road` is read every day for every HRU when `atmodep.cli` is in use.

Provide these files whenever salts are simulated, with zero values if there is nothing to add.

salt_hru.ini

Initial salt in HRU soils. Lines 1 to 5 are skipped. Each record then takes three lines:

Line of record	Content
1	Record name
2	Soil values: 8 ion concentrations in soil water (mg/L), then 5 mineral fractions (percent)
3	Plant values: 13 numbers, read but not used

An HRU uses the record named in the `salt` column of its `soil_plant.ini` record. If the name is not found, the first record is used. Every soil layer starts with the same values. The ion mass in a layer is the concentration times the water stored in the layer at the start of the run.

salt_aqu.ini

Initial salt in aquifers. Lines 1 to 4 are skipped. Each later line is one record: name, 8 ion concentrations in groundwater (mg/L), and 5 mineral fractions (percent).

Aquifers find their record through `initial.aqu_cs`, which has a title line, a header line and one row per aquifer with the columns `name`, `pest`, `path`, `hmet`, `salt` and `cs`. The `salt` column names a record in `salt_aqu.ini`. For salts, aquifer n uses row n of `initial.aqu_cs`, whatever the row's name, so the rows must be in aquifer order. The ion mass is the concentration times the groundwater stored at the start of the run.

salt_channel.ini

Initial salt in channel water. Lines 1 and 2 are skipped. Each later line is one record: name and 8 ion concentrations (mg/L).

Channels find their record through `initial.cha_cs` (title, header, then rows with `name`, `pest`, `path`, `hmet`, `salt`, `cs`). A channel uses the `initial.cha_cs` row at the same position as the `initial.cha` row it selects by name; the `salt` column of that row names a record in `salt_channel.ini`. If the name is not found, the first record is used.

salt_res

Initial salt in reservoirs and wetlands. Lines 1 to 11 are skipped (two title lines, eight comment lines and a header). Each later line is one record: name and 8 ion concentrations (mg/L). The file is ignored when the reservoir nutrient file in `file.cio` is `null`.

Reservoirs find their record through `reservoir.res_cs` and wetlands through `wetland.wet_cs`. Both have two header lines and one row per reservoir (or wetland) data record with the columns `id`, `pest`, `weir`, `salt` and `cs`. `id` is the position of the record in `reservoir.res` or `wetland.wet`, and `salt` names a record in `salt_res`. A wetland without a matching record starts with no salt.

salt_irrigation

Ion concentrations in irrigation water taken from outside the model. Lines 1 and 2 are skipped. Each later line is one record: name and 8 ion concentrations (mg/L).

An HRU uses the record at the same position as the `salt_hru.ini` record it uses (record 3 of `salt_hru.ini` goes with record 3 of `salt_irrigation`); the name is not used. In 62.0.1 the transfer of salt with irrigation water is switched off in water allocation, so these concentrations do not change the salt balance.

salt_fertilizer.frt

Ion content of each fertilizer. Line 1 is a title and line 2 a header. Then one row per record of `fertilizer.frt`, in the same order and with the same number of rows:

#	Column	Units	Description
1	`name`	-	Fertilizer name. Not checked against `fertilizer.frt`.
2	`so4`	kg/kg	Sulfate per kg of material applied.
3	`ca`	kg/kg	Calcium per kg of material applied.
4	`mg`	kg/kg	Magnesium per kg of material applied.
5	`na`	kg/kg	Sodium per kg of material applied.
6	`k`	kg/kg	Potassium per kg of material applied.
7	`cl`	kg/kg	Chloride per kg of material applied.
8	`co3`	kg/kg	Carbonate per kg of material applied.
9	`hco3`	kg/kg	Bicarbonate per kg of material applied.

The salt added (kg/ha) is the amount applied (kg/ha) times the value in the table.

Where salt is added in 62.0.1:

- A manure operation in a management schedule or a `manure` action in a decision table adds salt, split between the top two soil layers by the surface fraction of the application method in `chem_app.ops`. The row used is the row at the position of the manure in `manure_db.frt`, and salt is added only when the manure's `org_min` entry matches a record in `manure_om.frt` other than the first.
- A fertilizer operation in a management schedule adds salt only when the HRU has standing water, and the salt goes into the ponded water.
- A fertilizer operation on dry soil adds no salt.

Salt from a fertilizer whose name in `fertilizer.frt` starts with `a` is reported as a soil amendment in the salt balance output; other fertilizers are reported as fertilizer.

salt_road

Road salt applied to the soil surface. The file is organised by the atmospheric deposition stations of `atmodep.cli` and uses the same time step as that file. Lines 1 to 4 are skipped. Then, for each station in the order of `atmodep.cli`, one line with the station name followed by one line per ion (8 lines), each starting with the ion name:

`atmodep.cli` time step	Values on each ion line	Daily application
`aa`	1 value, kg/ha per year	the value / 365, every day
`mo`	1 value per month of the deposition record, kg/ha per month	the month's value / days in the month

atmodep.cli time step	Values on each ion line	Daily application
yr	1 value per year of the deposition record, kg/ha per year	spread over the days with precipitation and a mean temperature below 0 °C, in proportion to the day's precipitation

Each HRU receives the road salt of the deposition station of its weather station, on its whole area, into the top soil layer. For the yr time step the precipitation and temperature used to spread the salt come from the weather station at the same position in the station list as the deposition station.

salt_plants

Salt tolerance of plants and switches for salt stress.

Line	Content
1, 2	Title and header, skipped
3	Conversion factor from total dissolved solids (mg/L) to electrical conductivity (dS/m)
4	Skipped
5	Salt stress switch: 0 = off, 1 = on
6	Soil type: 1 = gypsiferous ($CaSO_4$) soils, 2 = NaCl soils
7	Stress method: 1 = salt stress applied after the other stresses, 2 = salt stress included with the other stresses
8 to 11	Skipped
12	Header, skipped
13+	One row per plant in plants.plt, in the same order: name, threshold a (dS/m), slope b (% per dS/m)

The plant name is not checked; the row position decides.

Salt stress

When the switch is on, SWAT+ computes for each soil layer within the root zone the total dissolved solids of the soil water at saturation, after mineral dissolution, and converts it to the electrical conductivity of the saturation extract EC_e with the factor on line 3. The relative yield for that layer is

$$Y_r = \frac{100 - b\,(EC_e - a)}{100} \qquad \text{for } EC_e > a$$

limited to 0 to 1, and 1 when $EC_e \le a$. For gypsiferous soils 2 dS/m is added to a. A plant with $a = 0$ has no salt stress. The salt stress of the plant is the smallest Y_r over its root zone layers.

With stress method 1, plant growth is multiplied by the salt stress after the smallest of the water, temperature, nitrogen, phosphorus and aeration stresses has been applied. With method 2, the salt stress is one of the stresses and the smallest of them is used.

salt_uptake

Specified daily salt uptake by plants. Lines 1 to 3 are skipped. Then one row per plant in plants.plt, in the same order: name, and 8 uptake values (kg/ha per day). The plant name is not checked.

While a plant has roots, its daily uptake of each ion is split between the soil layers in proportion to the root mass in each layer and is limited to the ion mass in the layer. Without the file, plants take up no salt.

salt_urban

Ion content of sediment washed from impervious areas. Lines 1 and 2 are skipped. Each later line is an urban type name from urban.urb and 8 concentrations (mg of ion per kg of sediment). In 62.0.1 the values are read but not used.

Example

salt_plants in the layout SWAT+ Editor writes, with salt stress on and illustrative values:

```
salt_plants: written by SWAT+ editor
conversion factor from TDS --> EC (EC=TDS/factor)
640.0
plant salinity tolerance
1            flag (0 = off; 1 = on)
2            flag (1=caso4 soils; 2=nacl soils)
1            flag (1=salt stress applied after other stresses applied; 2=included with other stresses)
NOTE: a and b values are for nacl soils
NOTE: if a and b values = 0, no salt impact
a = EC_sat threshold for impact on crop yield
b = slope of salinity impact on crop yield
            name           a           b
            agrl        1.70       12.00
            corn        1.70       12.00
```

TDS is divided by 640 to give EC_e. For corn, with $EC_e = 4$ dS/m in a layer, the relative yield is $(100 - 12 \times (4 - 1.7))/100 = 0.72$. The rows after the header continue for every plant in plants.plt.

Related files

- constituents.cs: lists the salt ions and switches the salt module on.
- soil_plant.ini: names the salt_hru.ini record for each HRU.
- fertilizer.frt: sets the row order of salt_fertilizer.frt.
- atmodep.cli, salt_atmo.cli: deposition stations and salt deposition.
- salt_recall.rec: salt loads from point sources.
- gwflow inputs: salt ions become groundwater solutes when gwflow solute transport is on.
- constituent inputs: the selenium and boron module.

Sources

- Master list: constit_db_read.f90
- Readers: salt_hru_read.f90, salt_aqu_read.f90, salt_cha_read.f90, res_read_saltdb.f90, salt_irr_read.f90, salt_fert_read.f90, salt_roadsalt_read.f90, salt_plant_read.f90, salt_uptake_read.f90, salt_urban_read.f90
- Link files: aqu_read_init_cs.f90 (initial.aqu_cs), ch_read_init_cs.f90 (initial.cha_cs) with sd_channel_read.f90, res_read_salt_cs.f90, wet_read_salt_cs.f90, soil_plant_init.f90 and hru_read.f90
- Initialisation: salt_hru_init.f90, sd_hydsed_init.f90, res_initial.f90, wet_initial.f90
- Processes: salt_fert.f90, salt_fert_wet.f90 (called from mgt_sched.f90 and actions.f90), salt_irrig.f90, salt_roadsalt.f90, salt_uptake.f90, salt_chem_soil_single.f90, salt_chem_aqu.f90, salt stress in pl_waterup.f90 and pl_biomass_gro.f90
- Types and variables: constituent_mass_module.f90, salt_module.f90, salt_data_module.f90, res_salt_module.f90

File and column	Fortran variable
salt_hru.ini soil and plant lines	salt_soil_ini%soil, salt_soil_ini%plt
salt_aqu.ini	salt_aqu_ini%conc, salt_aqu_ini%frac

File and column	Fortran variable
salt_channel.ini	salt_cha_ini%conc
salt_res	res_salt_data%c_init
salt_irrigation	salt_water_irr%water, copied to cs_irr%saltc
salt_fertilizer.frt	fert_salt (type fert_db_salt)
salt_road	rdapp_salt%salt%road, %roadmo, %roadyr, %roadday
salt_plants lines 3, 5, 6, 7	salt_tds_ec, salt_tol_sim, salt_soil_type, salt_effect
salt_plants columns a, b	salt_stress_a, salt_stress_b
salt_uptake	salt_uptake_kg
salt_urban	salt_urban_conc
initial.aqu_cs	aqu_init_dat_c_cs
initial.cha_cs	ch_init_cs
reservoir.res_cs, wetland.wet_cs	res_dat_c_cs, wet_dat_c_cs

Purpose

constituents.cs lists the pesticides, pathogens, heavy metals, salt ions and other constituents that a run simulates. The last group, the other constituents, is the constituent module: selenate (SeO_4), selenite (SeO_3) and boron in soils, aquifers, channels, reservoirs and wetlands. The module simulates sorption, selenium reduction and selenium release from shale, and the constituents move with water like the other dissolved substances.

The salt ion files are described on the salt inputs page. Apart from constituents.cs, which is named in file.cio, the files are not listed in file.cio; SWAT+ looks for them by name in the run directory.

constituents.cs

The master list, named on the simulation line of file.cio (default constituents.cs). If the file is missing or named null, no pesticides, pathogens, salts or other constituents are simulated.

Line 1 is a title. Then come five groups, each as a count followed by that many names:

Group	Count	Names
1	number of pesticides	pesticide names, matched by name to pesticide.pes
2	number of pathogens	pathogen names, matched by name to pathogens.pth
3	number of heavy metals	metal names, not matched to any database (see metals)
4	number of salt ions	salt ion names, in the order given on the salt inputs page
5	number of other constituents	constituent names

Each count is read from a separate line and the names that follow are read as one list, starting on the next line, so they can be on one line or several. A count of 0 still needs its names line: SWAT+ skips one line after the count, so leave a blank line (or any text) there. A pesticide or pathogen name that is not in its database is kept in the list but gets no parameters.

The constituent module uses the position of each constituent and not its name. Fertilizer input, selenium reactions and the reservoir parameters assume this order:

Position	Constituent
1	seo4 (selenate)
2	seo3 (selenite)
3	boron

File overview

File	Content	Linked to objects by
cs_hru.ini	Initial dissolved and sorbed constituents in HRU soils	cs column of soil_plant.ini
cs_aqu.ini	Initial dissolved and sorbed constituents in aquifers	cs column of initial.aqu_cs
cs_channel.ini	Initial concentrations in channels	cs column of initial.cha_cs
cs_res	Reservoir and wetland parameters and initial concentrations	cs column of reservoir.res_cs and wetland.wet_cs
cs_irrigation	Concentrations in irrigation water from outside the model	position, see below
fertilizer.frt_cs	Constituent content of fertilizers and manures	position in the fertilizer list
cs_reactions	Reaction parameters for soils and aquifers	HRU and aquifer order
cs_uptake	Daily constituent uptake by each plant	position in plants.plt
cs_atmo.cli	Concentration in rainfall and dry deposition	atmospheric deposition stations, see climate inputs
cs_recall.rec	Loads from point sources	see recall inputs
cs_plants_boron	Boron tolerance of plants	read but not used
cs_urban	Constituent content of sediment from impervious areas	read but not used
cs_streamobs	Channels for daily concentration output	not read

Missing files

cs_hru.ini, cs_aqu.ini and cs_channel.ini are opened whether or not they exist. If one is missing, SWAT+ creates an empty file of that name in the run directory and reads no records from it. An empty file, whether created this way or supplied, is read the same way.

The test project refdata/Ames_sub1 in the SWAT+ repository contains empty cs_hru.ini, cs_aqu.ini and cs_channel.ini files created by earlier runs. It gives null for the constituent file in file.cio, so no constituents are simulated and the empty files have no effect on the results. If constituents are listed in constituents.cs, an empty file behaves as follows:

- Empty cs_hru.ini: there is no record for the HRUs, but HRU setup still reads record 1, which does not exist. Give at least one record.
- Empty cs_aqu.ini: when initial.aqu_cs is present, the aquifer search runs over the number of records in cs_irrigation and reads cs_aqu.ini records that do not exist. Give at least as many records as cs_irrigation has.
- Empty cs_channel.ini: there is no record for the channels, but channel setup still reads record 1, which does not exist. Give at least one record.

Two files are needed as soon as other constituents are listed in constituents.cs, because they are read for every HRU without a check that they were found:

- cs_irrigation is read for every HRU during setup.
- cs_uptake is read every day for every HRU with a growing plant.

Provide both whenever constituents are simulated, with zero values if there is nothing to add.

cs_hru.ini

Initial constituents in HRU soils. Lines 1 to 5 are skipped. Each record then takes three lines:

Line of record	Content
1	Record name
2	Soil values: one dissolved concentration (mg/L) per constituent, then one sorbed concentration (mg/kg of soil) per constituent
3	Plant values: the same number of values, read but not used

An HRU uses the record named in the `cs` column of its `soil_plant.ini` record; if the name is not found, the first record is used. `soil_plant.ini` has the `cs` column only when `event` in `codes.bsn` is not 0. With `event` at 0, the column is not read and every HRU uses the first record of `cs_hru.ini`. Every soil layer starts with the same values.

cs_aqu.ini

Initial constituents in aquifers. Lines 1 to 3 are skipped. Each later line is one record: name, one dissolved concentration (mg/L) per constituent, then one sorbed concentration (mg/kg) per constituent. The sorbed mass is calculated with an assumed aquifer bulk density of 2000 kg/m³.

Aquifers find their record through `initial.aqu_cs` (title, header, then rows with `name`, `pest`, `path`, `hmet`, `salt`, `cs`). Aquifer n uses row n of `initial.aqu_cs`, whatever the row's name, and the `cs` column of that row names a record in `cs_aqu.ini`. See salt inputs for the same link used by salts.

cs_channel.ini

Initial concentrations in channel water. Lines 1 and 2 are skipped. Each later line is one record: name and one concentration (mg/L) per constituent. Channels find their record through `initial.cha_cs` as described for `salt_channel.ini`, using its `cs` column.

cs_res

Constituent parameters and initial concentrations for reservoirs and wetlands. Lines 1 to 15 are skipped (two title lines, twelve comment lines and a header). The file is ignored when the reservoir nutrient file in `file.cio` is `null`. Each later line is one record:

#	Column	Units	Default	Description
1	`name`	-	-	Record name, named in the `cs` column of `reservoir.res_cs` or `wetland.wet_cs`.
2	`v_seo4`	m/day	0.001	Settling velocity of selenate.
3	`v_seo3`	m/day	0.001	Settling velocity of selenite.
4	`v_born`	m/day	0.001	Settling velocity of boron.
5	`k_seo4`	1/day	0.05	First-order loss rate of selenate.
6	`k_seo3`	1/day	0.03	First-order loss rate of selenite.
7	`k_born`	1/day	0.0	First-order loss rate of boron.
8	`theta_seo4`	-	1.08	Temperature factor for the selenate rate.
9	`theta_seo3`	-	1.08	Temperature factor for the selenite rate.
10	`theta_born`	-	1.08	Temperature factor for the boron rate.
11	`c_seo4`	mg/L	0.0	Initial selenate concentration.
12	`c_seo3`	mg/L	0.0	Initial selenite concentration.
13	`c_born`	mg/L	0.0	Initial boron concentration.

All 13 values must be given; a short line takes its remaining values from the next line. A reservoir without a `cs_res` record starts with no constituents and uses the default rates in the table. `reservoir.res_cs` and `wetland.wet_cs` are described under `salt_res`.

cs_irrigation

Concentrations in irrigation water from outside the model. Lines 1 and 2 are skipped. Each later line is one record: name and one concentration (mg/L) per constituent.

An HRU uses the record at the same position as the `cs_hru.ini` record it uses; the name is not used. In 62.0.1 the transfer of constituents with irrigation water is switched off in water allocation, so these concentrations do not change the constituent balance.

fertilizer.frt_cs

Constituent content of each fertilizer. Line 1 is a title and line 2 a header. Then one row per record of `fertilizer.frt`, in the same order and with the same number of rows:

#	Column	Units	Description
1	`name`	-	Fertilizer name. Not checked against `fertilizer.frt`.
2	`seo4`	kg/kg	Selenate per kg of material applied.
3	`seo3`	kg/kg	Selenite per kg of material applied.
4	`boron`	kg/kg	Boron per kg of material applied.

The mass added (kg/ha) is the amount applied (kg/ha) times the value in the table. Constituents are added in the same cases as salts (see `salt_fertilizer.frt`): with manure operations and decision-table `manure` actions whose manure has an `org_min` entry other than the first record of `manure_om.frt`, and with fertilizer operations on HRUs with standing water. For manure, the whole amount goes into the top soil layer.

cs_reactions

Reaction parameters for soils and aquifers, organised in reaction groups and shale formations. Every HRU and every aquifer must have a row.

Section	Content
Line 1, 2	Title and header, skipped
Line 3	Number of parameters n_p and number of reaction groups n_g
next n_p lines	One parameter per line, with one value per reaction group
next line	Header, skipped
next line	Number of shale formations n_s
next n_s lines	Per shale formation: S/Se ratio of the shale, rate constant for autotrophic reduction of oxygen (1/day), rate constant for autotrophic reduction of nitrate (1/day)
next line	Header, skipped
next lines	One line per HRU, in HRU order: HRU number, reaction group, n_s shale fractions (fraction of the HRU area over each shale formation)
next line	Header, skipped (only when the model has aquifers)
next lines	One line per aquifer, in aquifer order: aquifer number, reaction group, n_s shale fractions

The parameter lines are used in this order:

Line	Parameter	Units	Used for
1	Selenium reduction inhibition factor for nitrate	-	soils, aquifers
2	Oxygen concentration in soil water	mg/L	soils
3	Oxygen concentration in groundwater	mg/L	aquifers
4	Sorption coefficient K_d of selenate	L/kg	soils, aquifers

Line	Parameter	Units	Used for
5	Sorption coefficient K_d of selenite	L/kg	soils, aquifers
6	Sorption coefficient K_d of boron	L/kg	soils, aquifers
7	Rate constant for selenate reduction in soil	1/day	soils
8	Rate constant for selenate reduction in groundwater	1/day	aquifers

The first value of the HRU and aquifer lines is not checked; the line position decides. Further parameter lines are read but not used by soils and aquifers. The rate constant for selenite reduction is not read for soils or aquifers and stays 0, so selenite is not reduced there.

cs_uptake

Specified daily constituent uptake by plants. Lines 1 to 3 are skipped. Then one row per plant in `plants.plt`, in the same order: name and one uptake value (kg/ha per day) per constituent. The plant name is not checked. While a plant has roots, its daily uptake of each constituent is split between the soil layers in proportion to the root mass in each layer and is limited to the mass in the layer.

cs_plants_boron

Boron tolerance of plants. Line 1 is a title and line 2 is skipped, line 3 is a switch (0 = off, 1 = on), lines 4 to 7 are skipped, and then one row per plant in `plants.plt`: name, threshold a and slope b. In 62.0.1 the values are read but no routine uses them, so boron has no effect on plant growth.

cs_urban

Constituent content of sediment washed from impervious areas. Lines 1 and 2 are skipped. Each later line is an urban type name from `urban.urb` and one concentration (mg per kg of sediment) per constituent. In 62.0.1 the values are read but not used.

cs_streamobs

A list of channels for daily concentration and load output to `cs_streamobs_output`: a title line, the number of channels, then one channel number per line. In 62.0.1 the switch that would read it is never set, so the file is not read and the output is not written.

Example

`constituents.cs` for a run with one pesticide, the eight salt ions and the three constituents:

```
constituents.cs: constituent list
1
atrazine
0

0

8
so4 ca mg na k cl co3 hco3
3
seo4 seo3 boron
```

There are no pathogens or heavy metals, so their counts are 0 and each is followed by an empty names line.

A matching `cs_aqu.ini` record with 0.005 mg/L selenate in groundwater and 0.2 mg/kg sorbed selenate, values illustrative:

```
cs_aqu.ini: initial constituents in aquifers
seo4 seo3 boron: dissolved (mg/L), then sorbed (mg/kg)
name          seo4     seo3    boron   seo4_s   seo3_s  boron_s
aqu_cs1      0.005    0.001    0.100    0.200    0.050    1.000
```

Related files

- salt inputs: the salt ions listed in `constituents.cs`, and the link files `initial.aqu_cs`, `initial.cha_cs`, `reservoir.res_cs` and `wetland.wet_cs`.
- `file.cio`: names `constituents.cs`.
- `soil_plant.ini`: names the `cs_hru.ini` record for each HRU.
- `codes.bsn`: `event` decides whether `soil_plant.ini` has the `cs` column.
- `fertilizer.frt`: sets the row order of `fertilizer.frt_cs`.
- `pesticide.pes`, `pathogens.pth`, metals: the other groups in `constituents.cs`.
- `cs_atmo.cli` and `cs_recall.rec`: deposition and point-source loads.
- gwflow inputs: constituents become groundwater solutes when gwflow solute transport is on.

Sources

- Master list: `constit_db_read.f90`; type `cs_db_type` in `constituent_mass_module.f90`
- Readers: `cs_hru_read.f90`, `cs_aqu_read.f90`, `cs_cha_read.f90`, `res_read_csdb.f90`, `cs_irr_read.f90`, `cs_fert_read.f90`, `cs_reactions_read.f90`, `cs_uptake_read.f90`, `cs_plant_read.f90`, `cs_urban_read.f90`
- Link files and initialisation: `soil_plant_init.f90`, `cs_hru_init.f90`, `aqu_read_init_cs.f90`, `sd_channel_read.f90`, `sd_hydsed_init.f90`, `res_read_salt_cs.f90`
- Processes: `cs_fert.f90`, `cs_fert_wet.f90`, `cs_irrig.f90`, `cs_uptake.f90` (called from `pl_biomass_gro.f90`), `cs_sorb_hru.f90`, `cs_sorb_aqu.f90`, `se_reactions_soil.f90`, `se_reactions_aquifer.f90`, `res_cs.f90`
- Types and variables: `cs_module.f90`, `cs_data_module.f90`, `res_cs_module.f90`

File and column	Fortran variable
`constituents.cs` counts	`cs_db%num_pests`, `%num_paths`, `%num_metals`, `%num_salts`, `%num_cs`
`constituents.cs` names	`cs_db%pests`, `%paths`, `%metals`, `%salts`, `%cs`
`cs_hru.ini` soil and plant lines	`cs_soil_ini%soil`, `cs_soil_ini%plt`
`cs_aqu.ini`	`cs_aqu_ini%aqu`
`cs_channel.ini`	`cs_cha_ini%conc`
`cs_res`	`res_cs_data` (type `reservoir_cs_data`)
`cs_irrigation`	`cs_water_irr%water`, copied to `cs_irr%csc`
`fertilizer.frt_cs`	`fert_cs` (type `fert_db_cs`)
`cs_reactions` parameter lines 1 to 8	`rct(1:8, group)`, copied to `se_ino3`, `oxy_soil`, `oxy_aqu`, `kd_seo4`, `kd_seo3`, `kd_born`, `kseo4` of `cs_rct_soil` and `cs_rct_aqu`
`cs_reactions` shale lines	`rct_shale(:,1:3)`, copied to `sseratio`, `ko2a`, `kno3a`
`cs_uptake`	`cs_uptake_kg`
`cs_plants_boron`	`bor_tol_sim`, `bor_stress_a`, `bor_stress_b`
`cs_urban`	`cs_urban_conc`
`cs_streamobs`	`cs_str_obs`

Note

In revision 62.0.1 the water allocation calculation follows these inputs only for a network with a single place of use, other than an irrigation district, whose POD lines are listed in POD-number order. Other layouts can take the wrong amounts or stop the run, and several columns are read but have no effect. See Limitations in 62.0.1 before setting up a network.

Purpose

Water allocation moves water from where it is taken to where it is used: irrigation districts, towns, industry, treatment plants, reservoirs, canals and places outside the basin. Revision 62.0.1 describes the network with three kinds of object:

- A **place of use** (POU) is where water goes: an irrigation district, a water treatment plant, a water use, a water tower, a canal, a channel, a reservoir, an aquifer or a receiver outside the basin. Each POU has a daily demand (its duty).
- A **point of diversion** (POD) is where water is taken: a channel, reservoir, aquifer, canal, water tower, outside-basin source or an unlimited source. A POD can supply several POUs, and a POU can draw on several PODs.
- A **point of return** (POR) is where a POU sends water after it has been used or treated: back to a channel, reservoir or aquifer, on to another treatment plant or use, or out of the basin.

Two files define the network: `place_of_use.wro` lists every POU with its PODs and PORs, and `point_of_diver.wro` lists every POD with the POUs it serves. The `.wal` files describe the objects between them: treatment plants, water uses, water towers, canals, pipes, irrigation districts and outside-basin sources.

This structure replaces the `water_allocation.wro` transfer file of earlier revisions, whose columns in earlier documentation (`rule_typ`, `srcs`, `ob_typ`, `withdr`, `limit_mon`, `cha_ob`, `rcv_ob`, `rcv_num`) have no counterpart in 62.0.1. `water_allocation.wro`, `element.wro` and `water_rights.wro` are no longer read.

Files and activation

File	Name set by	Contents
`place_of_use.wro`	`water_rights` line of `file.cio`, first name	Places of use, with their PODs and PORs
`point_of_diver.wro`	`water_rights` line of `file.cio`, second name	Points of diversion and the POUs each serves
`water_hru_irr.wal`	fixed name	HRUs in each irrigation district
`wtps_wuses.wal`	fixed name	Numbers of treatment plants and water uses
`water_treat.wal`	fixed name	Water treatment plants
`water_use.wal`	fixed name	Water uses (domestic, industrial, commercial)
`outside_src.wal`	fixed name	Sources outside the basin
`om_treat.wal`, `om_use.wal`, `om_osrc.wal`	fixed names	Water quality sets for treated water, used water and outside sources
`water_tower.wal`	fixed name	Water towers (storage)
`water_canal.wal`	fixed name	Canals
`water_pipe.wal`	fixed name	Pipes
`outside_rcv.wal`	fixed name	Receivers outside the basin (not read)

The `water_rights` line of `file.cio` now takes two file names, the POU file and the POD file, with defaults `place_of_use.wro` and `point_of_diver.wro`. A `file.cio` written for an earlier revision has `water_allocation.wro element.wro water_rights.wro` on that line, and SWAT+ 62.0.1 then reads `water_allocation.wro` as the POU file and `element.wro` as the POD file. Update the line before running an older project; older file.cio files explains how the old line is read.

Water allocation runs when the POU file has at least one place of use. If a `.wro` file or `wtps_wuses.wal` is missing, or the line in `file.cio` says `null`, SWAT+ creates an empty file of that name in the run folder (a file called `null` in the second case) and reads nothing from it. The empty file does no harm.

All files have a title on line 1. The `.wro` files and most `.wal` files then give the number of records on line 2 and a column header on line 3. Records are read as free-format values separated by spaces, by position, so the header text is not checked.

Daily sequence

Each day, before any object is simulated:

1. The day's POD and POR totals are set to zero.
2. Outside-basin source PODs, water tower PODs and unlimited PODs give water to their POUs.
3. POUs with no PODs (a treatment plant or use fed only by returns from other POUs) treat or use the water they have stored and pass it to their PORs.
4. The demand of every POU is set, from `rate_max` or from its decision table, and split among its PODs by `frac` (or by a decision table).

During the day, natural PODs give water when their object is simulated: a channel POD after the channel is routed (the water comes out of the channel's outflow for the day), a reservoir POD after the day's inflow is added, an aquifer POD after recharge is added. A canal POD gives water when the canal receives water.

Each time a POD gives water, every POU whose PODs have all given water for the day receives it. Water delivered to a POU that is a treatment plant, use or canal is processed at once (treated, used, or routed through the canal), and the POU's outflow is passed to its PORs.

Because step 2 runs before step 4, outside-source, water tower and unlimited PODs work with the previous day's demand, and with zero demand on the first day.

place_of_use.wro

Line 1 is a title, line 2 the number of POUs, line 3 a header. Each POU then takes one line, followed by one line for each of its PODs and one line for each of its PORs.

POU line

#	Column	Units	Description
1	`num`	-	POU number, 1 to the number on line 2. The record is stored under this number.
2	`name`	-	POU name.
3	`typ`	-	POU type, from the table below.
4	`typ_num`	-	Number of the object of that type that receives the water (for example the channel number, or the record number in `water_treat.wal`). For `irr`, the record number in `water_hru_irr.wal`.
5	`pods`	-	Number of POD lines that follow. 0 for a POU fed only by returns from other POUs.
6	`pors`	-	Number of POR lines that follow.
7	`dtbl_mx`	-	Name of a flow control decision table (`flo_con.dtl`) that sets the daily demand, or `null`.
8	`rate_max`	m^3/s	Maximum demand rate. Without a decision table the POU demands rate_max × 86400 m^3 every day. With a decision table, its demand is capped at that amount.
9	`dtbl_pod_fr`	-	Name of a flow control decision table that sets the fraction taken from each POD, or `null` to use the `frac` column of the POD lines.
10	`dtbl_por_fr`	-	Name of a flow control decision table that sets the fraction sent to each POR, or `null` to use the `frac` column of the POR lines.

POU type	Water goes to
`irr`	The HRUs of an irrigation district in `water_hru_irr.wal`
`cha`	A channel (added to the channel's inflow)
`res`	A reservoir
`aqu`	An aquifer
`wtp`	A water treatment plant in `water_treat.wal`
`use`	A water use in `water_use.wal`
`wtow`	A water tower in `water_tower.wal`
`can`	A canal in `water_canal.wal`

POU type	Water goes to
`orcv`	A receiver outside the basin in `outside_rcv.wal`

POD lines

One line per POD, starting with the word `POD`.

#	Column	Units	Description
1	`kind`	-	The word `POD`.
2	`num`	-	Not used (overwritten by column 5).
3	`name`	-	POD name, for reference.
4	`typ`	-	POD type, for reference. The type used is the one in `point_of_diver.wro`.
5	`typ_num`	-	Object number, for reference.
6	`conv_typ`	-	Conveyance type, `pipe` or `pump`. Read but not used.
7	`conv_num`	-	Conveyance number. Read but not used.
8	`dtbl_min`	-	Decision table for the minimum level. Read but not used.
9	`const_min`	depends on POD type	Minimum level below which the POD gives no water. See the POD type table under point_of_diver.wro.
10	`dtbl_wdraw`	-	Name of a flow control decision table that manages the annual withdrawal limit, or `null`. When a name is given, the limit is not reset at the start of each year; the `duty_reset` decision table action resets it instead.
11	`ann_max`	m^3/s	Annual withdrawal limit for this POD and POU. Once the volume taken during the year reaches ann_max $\times$ 86400 m^3, the POD gives this POU no more water until the limit is reset.
12	`frac`	fraction	Fraction of the POU's daily demand taken from this POD. The fractions of a POU's PODs normally add up to 1.
13	`comp`	-	`y` to let this POD make up a shortfall when the POU's PODs together delivered less than its demand, `n` otherwise. Channel PODs cannot make up a shortfall.

POR lines

One line per POR, starting with the word `POR`.

#	Column	Units	Description
1	`kind`	-	The word `POR`.
2	`num`	-	Not used (overwritten by column 5).
3	`name`	-	POR name, for reference.
4	`typ`	-	Receiving object type: `cha`, `res`, `aqu`, `wtp`, `use`, `wtow`, `can` or `orcv`. `hru` is accepted but receives nothing.
5	`typ_num`	-	Number of the receiving object of that type.
6	`conv_typ`	-	Conveyance type. Read but not used.
7	`conv_num`	-	Conveyance number. Read but not used.
8	`dtbl_max`	-	Read but not used.
9	`const_max`	-	Read but not used.
10	`ann_max`	-	Read but not used.
11	`frac`	fraction	Fraction of the POU's outflow sent to this POR.

A water treatment plant POU sends each POR its `frac` of the treated water, and a water use POU sends each POR its `frac` of its effluent. For the other POU types, canals included, each POR receives its `frac` of the water withdrawn for the POU, in addition to the delivery to the POU itself.

point_of_diver.wro

Line 1 is a title, line 2 the number of PODs, line 3 a header. Each POD takes one line: five fixed columns, then six columns for each POU it serves.

#	Column	Units	Description
1	`num`	-	POD number.
2	`name`	-	POD name.
3	`typ`	-	POD type, from the table below.
4	`typ_num`	-	Number of the object of that type (channel number, reservoir number, record number in `outside_src.wal`, and so on).
5	`pous`	-	Number of POUs served.
6 to 11	`pou_num`, `pou_name`, `pod_num`, `pou_typ`, `pou_typ_num`, `right`	-	Repeated for each POU: the POU number in `place_of_use.wro`, its name, the position of this POD among that POU's POD lines, the POU type and object number, and the water right (`sr` senior, `jr` junior).

The `right` column is read but does not change the order of withdrawals in 62.0.1.

POD type	Water taken from	Meaning of `const_min`
`cha`	Channel outflow for the day	Minimum daily flow volume (m^3). No water is taken on days when the channel outflow is at or below it.
`res`	Reservoir storage	Fraction of the principal spillway volume. No water is taken when storage is at or below it.
`aqu`	Aquifer storage	Water table depth (m). Water is taken only while the water table is shallower than this depth.
`can`	Canal storage	Fraction of the current canal storage, so values up to 1 set no limit.
`wtow`	Water tower storage	Fraction of the current tower storage, so values below 1 set no limit.
`osrc`	Outside-basin source	Not used.
`unl`	Unlimited source with no water quality	Not used.
`hru`	Accepted, gives no water	-
`gwf`	Accepted, gives no water	-

The amount taken is the POD's duty (its `frac` of the POU demand), limited by the water the object has above the minimum. Withdrawals from channels, reservoirs, canals and towers take sediment and nutrients with the water in proportion. Aquifer withdrawals take water, nitrate and mineral P.

Two types do not follow this rule:

- An `osrc` POD gives its `frac` of the daily volume available from the outside source, whatever the POU demand. The available volume is the `flo` value of the `om_osrc.wal` record with the same number as the source. The water has no sediment or nutrients.
- A `unl` POD gives frac times its duty, which is $frac^2$ times the POU demand.

water_hru_irr.wal

Irrigation districts for `irr` POUs. Line 1 is a title, line 2 the number of districts, line 3 a header. Each district takes one line with its name and number of HRUs, followed by one line per HRU.

Line	Columns	Description
District	`name`, `num_hrus`	District name and the number of HRU lines that follow.
HRU	`hru`, `dtbl_lum`	HRU number and the name of the decision table in `lum.dtl` that sets its irrigation demand.

The demand of an irrigation POU is meant to be the sum of the demands of its HRUs, set by their decision tables. See Limitations in 62.0.1 for how 62.0.1 handles it.

wtps_wuses.wal

Sets up storage for treatment plants and water uses. It is needed whenever `water_treat.wal` or `water_use.wal` is used.

Line	Contents
1	Title
2	Number of treatment plants, number of water uses
3	Header (skipped)
4	Names of the treatment plants
5	Names of the water uses

The two numbers must match the number of records in `water_treat.wal` and `water_use.wal`. Lines 4 and 5 must both be present, even when one of the numbers is 0. The names are read but not used.

water_treat.wal

Water treatment plants. Line 1 is a title, line 2 the number of plants, line 3 a header. Each plant takes one line, followed by one line per treatment level.

Plant line:

#	Column	Units	Description
1	`name`	-	Plant name.
2	`stor_mx`	m^3	Maximum storage. Read but not used.
3	`lag_days`	days	Treatment time. Read but not used: treated water leaves the same day.
4	`loss_fr`	fraction	Water loss during treatment. Read but not used; the loss comes from the `flo` value of the quality set.
5	`num_treats`	-	Number of treatment level lines that follow.

Treatment level line:

#	Column	Description
1	`org_min_typ`	How the quality of the treated water is set: `const` (a record in `om_treat.wal`), `dtbl` (a decision table) or `recall` (a recall record).
2	`org_min_name`	Name of the `om_treat.wal` record, decision table or recall record.
3, 4	`pests_typ`, `pests_name`	Same pair for pesticides. Read but not used.
5, 6	`paths_typ`, `paths_name`	Same pair for pathogens. Read but not used.
7, 8	`salts_typ`, `salts_name`	Same pair for salt ions. Read but not used.
9, 10	`constit_typ`, `constit_name`	Same pair for other constituents. Read but not used.
11	`description`	Text.

If the project has pesticides, each treatment level line is followed by a header line and a line of pesticide concentrations in the treated water, one value per pesticide. If it has pathogens, a header line and a line of pathogen concentrations follow in the same way.

When a plant has more than one treatment level, the last level sets the quality of the outflow.

Treated outflow to each POR is

$$Q_{out} = V_{stor}\, f_{\text{om}}\, \text{frac}_{POR}$$

where V_{stor} is the water delivered to the plant that day (m^3), f_{om} is the `flo` value of the quality set (the fraction of water that leaves the plant after treatment losses) and $frac_{POR}$ is the POR's fraction. Sediment and nutrient loads are the quality set's concentrations times Q_{out}, limited to the loads that came into the plant. The difference between incoming and outgoing loads is reported as the amount removed by treatment.

water_use.wal

Water uses (domestic, industrial, commercial). Line 1 is a title, line 2 the number of uses, line 3 a header. Each use takes one line, followed by the optional pesticide and pathogen lines described for `water_treat.wal`.

#	Column	Units	Description
1	`name`	-	Use name.
2	`stor_mx`	m^3	Maximum storage. Read but not used.
3	`lag_days`	days	Read but not used.
4	`loss_fr`	fraction	Read but not used.
5 to 14	`org_min_typ`, `org_min_name`, … `constit_typ`, `constit_name`	-	Quality pairs as in a `water_treat.wal` treatment level. `org_min_name` names a record in `om_use.wal` when the type is `const`.
15	`description`	-	Text.

The outflow of a use is the water delivered that day times the `flo` value of its `om_use.wal` record (the fraction returned as effluent), with the record's concentrations. Each POR receives its `frac` of the outflow.

outside_src.wal

Sources outside the basin, used by `osrc` PODs. Line 1 is a title, line 2 the number of sources, line 3 a header. Each source takes one line, followed by the optional pesticide and pathogen lines described for `water_treat.wal`.

#	Column	Description
1	`name`	Source name.
2 to 11	`org_min_typ`, `org_min_name`, … `constit_typ`, `constit_name`	Quality pairs as in a `water_treat.wal` treatment level. `org_min_name` names a record in `om_osrc.wal` when the type is `const`.
12	`description`	Text.

om_treat.wal, om_use.wal and om_osrc.wal

Water quality sets for treated water, used water (effluent) and outside sources. Line 1 is a title, line 2 the number of records, line 3 a header. Each record is a name followed by 18 values.

#	Column	Units	Description
1	`name`	-	Record name.
2	`flo`	see text	In `om_treat.wal` and `om_use.wal`, the fraction of the incoming water that leaves the plant or use. In `om_osrc.wal`, the volume available from the source each day (m^3).
3	`sed`	mg/L	Sediment.
4	`orgn`	mg N/L	Organic nitrogen.
5	`sedp`	mg P/L	Organic phosphorus.
6	`no3`	mg N/L	Nitrate nitrogen.
7	`solp`	mg P/L	Soluble (mineral) phosphorus.
8	`chla`	mg/L	Chlorophyll a.
9	`nh3`	mg N/L	Ammonia nitrogen.
10	`no2`	mg N/L	Nitrite nitrogen.
11	`cbod`	mg/L	Carbonaceous biological oxygen demand.
12	`dox`	mg/L	Dissolved oxygen.

#	Column	Units	Description
13 to 18	`san`, `sil`, `cla`, `sag`, `lag`, `grv`	mg/L	Sand, silt, clay, small aggregates, large aggregates and gravel.
19	`temp`	°C	Water temperature.

water_tower.wal

Water towers: storage between a source and its users. Line 1 is a title, line 2 the number of towers, line 3 a header, then one line per tower.

#	Column	Units	Description
1	`id`	-	Tower number.
2	`name`	-	Tower name.
3	`stor_mx`	m³	Storage capacity. Used by the `wtow_vol` decision table condition and the `fill_wtow` action.
4	`ddown_days`	days	Read but not used.
5	`loss_fr`	fraction	Read but not used.

A tower is filled by a `wtow` POU and emptied by a `wtow` POD. Its water quality does not change in storage.

water_canal.wal

Canals. Line 1 is a title, line 2 the number of canals, line 3 a header, then one line per canal.

#	Column	Units	Description
1	`id`	-	Canal number.
2	`name`	-	Canal name.
3	`w_sta`	-	Nearby weather station. Read but not used.
4	`init`	-	Initial concentrations. Read but not used.
5	`dtbl`	-	Decision table for canal outflow. Must be `null` in 62.0.1: with any other name the canal releases no water.
6	`ddown_days`	days	Days to draw the storage down. Each day the canal releases its storage divided by `ddown_days`.
7	`w`	m	Bottom width.
8	`d`	m	Depth.
9	`l`	km	Length.
10	`s`	m/m	Slope. Read but not used.
11	`ss`	m/m	Side slope (horizontal to vertical) of the trapezoidal section.
12	`sat_con`	mm/day	Seepage rate from the canal bed to groundwater.
13	`evap_co`	fraction	Fraction of the canal storage kept after evaporation each day. 1 means no evaporation loss; 0 empties the canal.
14	`num_aqu`	-	Number of aquifers that receive seepage.
15 onward	`aqu_fr`	fraction	One value per aquifer: the fraction of the seepage that goes to aquifer 1, aquifer 2, and so on. The aquifers are always the first `num_aqu` aquifers of the model, in order.

The canal capacity is computed from its shape as $V_{max} = (w + ss\,d)\,d\,l \times 1000$ m³ (with l in km), and is used by the `fill_can` decision table action.

When a canal receives water, it first supplies any POUs for which it is a POD. Evaporation is then applied, and seepage of sat_con $\times\ d \times l$ m³ is divided among the aquifers by their fractions, with nitrate and soluble P in proportion. Finally the canal releases its storage divided by `ddown_days`. In 62.0.1 this release is removed from

the canal but is not passed on; the PORs of a canal POU receive their share of the water withdrawn for the canal instead.

When gwflow canal seepage is switched on, gwflow takes canal width, depth and `sat_con` from this file. It also expects a bed thickness, a diversion ID and start and end days for each canal, which `water_canal.wal` does not supply in 62.0.1, so gwflow treats every canal as connected to a channel.

water_pipe.wal

Pipes. In 62.0.1 pipes are read but have no effect: water moves from PODs to POUs and from POUs to PORs without conveyance losses.

Line 1 is a title, line 2 the number of pipes, line 3 a header. For each pipe the reader then takes a header line and two record lines: the first gives the number of aquifers, and the second is read as the full record. Both record lines have the same layout:

#	Column	Units	Description
1	`id`	-	Pipe number.
2	`name`	-	Pipe name.
3	`stor_mx`	m^3	Maximum storage.
4	`ddown_days`	days	Drawdown time.
5	`loss_fr`	fraction	Water lost along the pipe.
6	`num_aqu`	-	Number of aquifers that receive the loss.
7 onward	`aqu_fr`	fraction	One value per aquifer: fraction of the loss to that aquifer.

outside_rcv.wal

Receivers outside the basin, with columns `id`, `name` and `filename` (an output file for the receiver). SWAT+ 62.0.1 does not read this file, so `orcv` POUs and PORs have no receiving object.

Decision tables

Water allocation uses flow control decision tables (`flo_con.dtl`) and land use decision tables (`lum.dtl`). See decision tables for the file format.

Item	Kind	Effect
`wallo_dmd` with option `set_dmd`	action	Sets the POU demand to the action's constant (m^3).
`wallo_dmd` with option `irr_dmd`	action	Adds the current demand of the POU named by the action's object number.
`wallo_dmd` with option `fill_res`, `fill_wtow` or `fill_can`	action	Sets the demand to the volume needed to fill a reservoir (to a fraction of its principal or emergency volume), a water tower or a canal (to a fraction of capacity), times a second constant.
`duty_reset`	action	Sets a new annual withdrawal limit for one POD of one POU and resets the volume taken so far.
`irr_pou_dmd`	condition	Tests the current demand of a POU.
`wtow_vol`	condition	Tests water tower storage as a fraction of `stor_mx`.

Limitations in 62.0.1

Several parts of the daily calculation do not follow the input structure above:

- The duty used for a withdrawal is looked up with the POD's number and the position of the POU in that POD's list, in place of the POU's number and the POD's position in that POU. Results follow the input only when these coincide: POD number n is the n-th POD line of every POU it serves, and the k-th POU listed

for a POD in `point_of_diver.wro` is POU number k. A network with one POU whose POD lines are listed in POD-number order meets this. Other layouts take the wrong duty or fail.

- For an irrigation district (`irr` POU), the HRU demands from the district's decision tables are summed and then replaced by rate_max × 86400, so the POU always demands its maximum rate. Delivery to the district does not step through the listed HRUs.
- No decision table action sets the POD and POR fractions, so naming a table in `dtbl_pod_fr` or `dtbl_por_fr` sets all fractions to zero.
- For an aquifer POD, the water delivered is computed after the aquifer storage has been reduced, so it is smaller than the water removed from the aquifer by the factor $(1 - r)$, where r is the fraction of storage withdrawn.
- Projects with pesticides or pathogens fail when `water_treat.wal`, `water_use.wal` or `outside_src.wal` is read, because the storage for the treated-water concentrations is not set up.
- Water taken by start-of-day PODs (outside sources, towers, unlimited sources) is not counted in the day's delivered totals, which are reset when the day's demand is set.

Example

A town water use that draws on a channel and an aquifer and returns its effluent to another channel. The single POU lists its PODs in POD-number order, which keeps POD numbers and POU positions aligned as 62.0.1 requires.

`place_of_use.wro`:

```
place_of_use.wro: town supply
1
num  name   typ  typ_num  pods  pors  dtbl_mx  rate_max  dtbl_pod_fr  dtbl_por_fr
1    town   use  1        2     1     null     0.05      null         null
POD  1  river   cha  12  pipe  0  null  20000.0  null  20.0  0.7  n
POD  2  wells   aqu  3   pump  0  null  15.0     null  20.0  0.3  y
POR  1  outfall cha  14  pipe  0  null  0.0      0.0   1.0
```

`point_of_diver.wro`:

```
point_of_diver.wro: town supply
2
num  name   typ  typ_num  pous  pou_num  pou_name  pod_num  pou_typ  pou_typ_num  right
1    river  cha  12       1     1        town      1        use      1            sr
2    wells  aqu  3        1     1        town      2        use      1            sr
```

`wtps_wuses.wal`, `water_use.wal` and `om_use.wal`:

```
wtps_wuses.wal: town supply
0  1
treatment plant names on line 4, water use names on line 5
none
town
```

```
water_use.wal: town supply
1
name  stor_mx  lag_days  loss_fr  org_min_typ  org_min_name  pests_typ  pests_name  paths_typ  paths_name
town  0.0      0.0       0.0      const        town_eff      null       null        null       null
```

```
om_use.wal: town supply
1
name      flo  sed  orgn  sedp  no3  solp  chla  nh3  no2  cbod  dox  san  sil  cla  sag  lag  grv  temp
town_eff  0.8  20   2     0.5   10   1     0     5    0.1  20    4    0    0    0    0    0    0    18
```

The town demands $0.05 \times 86400 = 4320$ m³ a day: 70% (3024 m³) from channel 12 and 30% (1296 m³) from aquifer 3. The channel gives water only on days its outflow exceeds 20 000 m³, and the aquifer only while its water table is shallower than 15 m. The aquifer may make up a shortfall from the channel. Each POD may supply at most $20 \times 86400 = 1\,728\,000$ m³ a year to the town. 80% of the water delivered returns to channel 14 as effluent with the concentrations in `town_eff`.

Outputs

Set the `water_allo` row of print.prt to write `water_allo_day.txt` (and the monthly, yearly and average annual files) with each POU's demand and delivery, the withdrawal from each POD and the water sent to each POR. Treatment plants and water uses are reported in `wallo_treat_*.txt` and `wallo_use_*.txt`.

Related files

- file.cio: the `water_rights` line names the POU and POD files.
- Decision tables: flow control and land use tables used for demand, limits and irrigation.
- Channels, reservoirs and aquifers: PODs and receiving objects.
- print.prt: output switches.
- gwflow: canal seepage to the groundwater grid.

Sources

- File names and `file.cio` line: `type input_water_allocation` in input_file_module.f90, readcio_read.f90
- Read order: main.f90.in
- Readers: water_allocation_read.f90 (POU and POD files), water_hru_irr_read.f90, wallo_wtps_wuses.f90, water_treatment_read.f90, water_use_read.f90, water_osrc_read.f90, om_treat_read.f90, om_use_read.f90, om_osrc_read.f90, water_tower_read.f90, water_canal_read.f90, water_pipe_read.f90, water_orcv_read.f90 (not called)
- Types: water_allocation_module.f90; `hyd_output` in hydrograph_module.f90
- Daily calculation: wallo_start_day.f90 (called from time_control.f90), wallo_control.f90, wallo_withdraw.f90, wallo_pou_deliv.f90, wallo_return.f90, wallo_treatment.f90, wallo_use.f90, wallo_canal.f90, wallo_transfer.f90 (not called)
- POD hooks: sd_channel_control3.f90, res_control.f90, aqu_1d_control.f90, hru_control.f90
- Decision tables: actions.f90, conditions.f90
- Outputs: header_water_allocation.f90, wallo_pou_output.f90
- gwflow canals: gwflow_read.f90

File	Column	Fortran variable
`place_of_use.wro` POU line	`num`, `name`, `typ`, `typ_num`, `pods`, `pors`, `dtbl_mx`, `rate_max`, `dtbl_pod_fr`, `dtbl_por_fr`	`ipou`, `pou%name`, `%typ`, `%typ_num`, `%pods`, `%pors`, `%dtbl_mx`, `%rate_max`, `%dtbl_pod_fr`, `%dtbl_por_fr`
POD line	columns 2 to 13	`pou%pod%num`, `%name`, `%typ`, `%num`, `%conv_typ`, `%conv_num`, `%dtbl_min`, `%const_min`, `%dtbl_wdraw`, `%ann_max`, `%frac`, `%comp`
POR line	columns 2 to 11	`pou%por%num`, `%name`, `%typ`, `%num`, `%conv_typ`, `%conv_num`, `%dtbl_max`, `%const_max`, `%ann_max`, `%frac`
`point_of_diver.wro`	`num`, `name`, `typ`, `typ_num`, `pous`	`pod%num`, `%name`, `%typ`, `%typ_num`, `%pous`
`point_of_diver.wro` per POU	`pou_num`, `pou_name`, `pod_num`, `pou_typ`, `pou_typ_num`, `right`	`pod%pou%num`, `%name`, `%pod_num`, `%typ`, `%typ_num`, `%right`
`water_hru_irr.wal`	`name`, `num_hrus`, `hru`, `dtbl_lum`	`hruirr_db%name`, `%hrus`, `%hru_num`, `%dtbl_lum`
`water_treat.wal`	`name`, `stor_mx`, `lag_days`, `loss_fr`, `num_treats`	`wtp%name`, `%stor_mx`, `%lag_days`, `%loss_fr`, `%num_treats`
treatment level, `water_use.wal`, `outside_src.wal`	quality pairs	`%conc%org_min_typ`, `%org_min_name`, `%pests_typ`, `%pests_name`, `%paths_typ`, `%paths_name`, `%salts_typ`, `%salts_name`, `%constit_typ`, `%constit_name`
`om_*.wal`	18 values	`wtp_om_treat`, `wuse_om_efflu`, `osrc_om` (`type hyd_output`)
`water_tower.wal`	`stor_mx`, `ddown_days`, `loss_fr`	`wtow%stor_mx`, `%ddown_days`, `%loss_fr`
`water_canal.wal`	`w_sta` … `num_aqu`, `aqu_fr`	`canal%w_sta`, `%init`, `%dtbl`, `%ddown_days`, `%w`, `%d`, `%l`, `%s`, `%ss`, `%sat_con`, `%evap_co`, `%num_aqu`, `%aqu_loss_fr`
`water_pipe.wal`	`stor_mx` … `aqu_fr`	`pipe%stor_mx`, `%ddown_days`, `%loss_fr`, `%num_aqu`, `%aqu_loss_fr`

SWAT+ writes text files with results for each object type (HRU, landscape unit, channel, aquifer, reservoir and so on) and for the basin as a whole, by day, month, year and as an average over the simulation. Which files are written is set in print.prt: each row names a group of outputs and has a `y` or `n` for the daily, monthly, yearly and average annual files. A few files are written in every run.

The files go to the output folder named in file.cio, or to the run folder when none is named. Every run also writes files_out.out, a list of the output files it opened, which is the quickest way to see what a given print.prt produced.

Pages

Page	Content
Naming convention	How files are named and laid out, print timing, and the print.prt row behind every file.
Water balance	hru_wb, hru-lte_wb, lsunit_wb and basin_wb: precipitation, evapotranspiration, runoff, percolation, soil water and snow.
Channel, sediment, nutrient	channel_sd, channel_sdmorph, sd_chanbud, the sediment budget files ch_sedbud, chanbud_order and sedbud, their basin sums, and the legacy channel files.
Hydrographs	hydout, hydin, deposition and ru: water and loads passed between objects, and the hydcon.out connection list.
Recall	recall and basin_psc: loads released by point sources, inlets and diversions.
Nutrient balance and losses	hru_nb, hru_ls, lsunit_nb, lsunit_ls, basin_nb, basin_ls and hru-lte_ls: nitrogen and phosphorus balances and the sediment and nutrient losses from the landscape.
Aquifer, reservoir, plant	aquifer, reservoir, wetland, crop yields and the plant and weather (_pw) files.
Carbon	Soil, residue and plant carbon files.
gwflow	Outputs of the gridded groundwater module.
Checker and diagnostics	Files written in every run: checker.out, diagnostics.out, files_out.out and others.
Post-processing	The Python scripts in the test folder of the SWAT+ source.

Recent changes

- 62.0.1 adds the channel sediment budget files ch_sedbud_* (each channel), chanbud_order_* (each stream order) and sedbud_* (basin), all switched on by the channel_sd row.
- 62.0.0 added the carbon rows of print.prt (hru_cb_gl, hru_cb_trf, lsu_cb_gl and others) for the carbon output families; see Carbon.

Sources

- print.prt reader: basin_print_codes_read.f90
- Output folder: output_path_module.f90
- File opening: proc_open.f90, output_landscape_init.f90, header_write.f90, header_sd_channel.f90
- files_out.out: proc_bsn.f90

Filename pattern

Most time-step outputs are named

<object>_<group>_<interval>.txt

for example hru_wb_day.txt (HRU water balance, daily) or basin_sd_cha_aa.txt (basin sum of channel outputs, average annual). Some files have no group (aquifer_day.txt, reservoir_mon.txt, ru_yr.txt), and a few files written once per run have no interval (hru_cbn_lyr.txt, mgt_out.txt).

Interval	Meaning	print.prt column
_day	one row per object per day	daily
_mon	one row per object per month	monthly
_yr	one row per object per year	yearly
_aa	one row per object, average annual over the printed years	avann

The channel sediment budget files `ch_sedbud_*` and `chanbud_order_*` use `_d`, `_m`, `_y` and `_a` instead.

When `csvout` is `y` in `print.prt`, most time-step files also get a comma-separated copy with the same name and the extension `.csv`.

Output folder

Output files are written to the folder named on the output path line at the end of `file.cio`. SWAT+ creates the folder if it does not exist. When the line is missing, empty or `null`, outputs go to the folder SWAT+ runs in. `simulation.out`, `success.fin` and `erosion.txt` always go to the run folder.

File layout

Time-step files share one layout:

- Line 1: the project name from `file.cio` and the SWAT+ version banner.
- Line 2: column headers.
- Line 3: units (blank for columns without units).
- Lines 4 and on: data rows.

Rows with one object per row start with `jday`, `mon`, `day`, `yr`, `unit`, `gis_id` and `name`. Basin files use the same columns with `unit` and `gis_id` set to 1. `jday`, `mon`, `day` and `yr` are the date of the last day in the print step.

Print timing

The first blocks of `print.prt` set when rows are written:

- Daily rows are written from `day_start` of `yrc_start` to `day_end` of `yrc_end`, every `interval` days. A zero `day_start` means day 1, a zero `day_end` means day 366, and zero years mean the first and last simulated year.
- Monthly, yearly and average annual rows start after the first `nyskip` simulated years. Average annual values are averages over the remaining years.
- The average annual print intervals (`aa_int_cnt` and the end years that follow it) are read, but in 62.0.1 they are used only by the region output, which is not written, so they have no effect.

print.prt rows and files

Each row in the objects block of `print.prt` has a name and four `y`/`n` switches. When `use_obj_lbls` is `y`, rows are matched by name, may come in any order, and rows can be left out (they default to `n`). A name that appears twice stops the run with an error, and so does a name SWAT+ does not know. When `use_obj_lbls` is `n`, the rows are read by position and must follow the order of the table below up to `wetland_cs`; the carbon and gwflow rows can only be set in labelled mode.

Each `y` opens the file for that interval. The table shows the file stems; add `_day.txt`, `_mon.txt`, `_yr.txt` or `_aa.txt`.

`print.prt` row	Files	Described in
`basin_wb`	`basin_wb`	Water balance
`basin_nb`	`basin_nb`	Nutrient balance and losses
`basin_ls`	`basin_ls`	Nutrient balance and losses
`basin_pw`	`basin_pw`	Aquifer, reservoir, plant
`basin_aqu`	`basin_aqu`	Aquifer, reservoir, plant
`basin_res`	`basin_res`	Aquifer, reservoir, plant
`basin_cha`	`basin_cha` (legacy channels)	Channel and sediment
`basin_sd_cha`	`basin_sd_cha`, `basin_sd_chamorph`, `basin_sd_chanbud`	Channel and sediment

print.prt row	Files	Described in
basin_psc	basin_psc (recall objects summed)	Recall
region_wb, region_nb, region_ls, region_pw, region_aqu, region_res, region_sd_cha, region_psc	none in 62.0.1	
water_allo	water_allo, wallo_use, wallo_treat	Water allocation
lsunit_wb	lsunit_wb	Water balance
lsunit_nb, lsunit_ls, lsunit_pw	lsunit_nb, lsunit_ls, lsunit_pw	Nutrient balance and losses, Aquifer, reservoir, plant
hru_wb	hru_wb; also hru_path when constituents are simulated	Water balance
hru_nb	hru_nb, hru_ncycle	Nutrient balance and losses
hru_ls	hru_ls	Nutrient balance and losses
hru_pw	hru_pw	Aquifer, reservoir, plant
hru-lte_wb, hru-lte_nb, hru-lte_ls, hru-lte_pw	hru-lte_wb, hru-lte_ls, hru-lte_pw	Water balance, Nutrient balance and losses, Aquifer, reservoir, plant
channel	channel (legacy channels)	Channel and sediment
channel_sd	channel_sd, channel_sdmorph, sd_chanbud, sedbud, ch_sedbud, chanbud_order, channel_sd_subday.txt	Channel and sediment
aquifer	aquifer	Aquifer, reservoir, plant
reservoir	reservoir, wetland	Aquifer, reservoir, plant
recall	recall	Recall
hyd	hydin, hydout, deposition	Hydrographs
ru	ru (routing unit hydrographs)	Hydrographs
pest	hru_pest, channel_pest, reservoir_pest, aquifer_pest, basin_ls_pest, basin_ch_pest, basin_res_pest, basin_aqu_pest	
basin_salt, hru_salt, ru_salt, aqu_salt, channel_salt, res_salt, wetland_salt	basin_salt, hru_salt, rout_unit_salt, aquifer_salt, channel_salt, reservoir_salt, wetland_salt	Salts
basin_cs, hru_cs, ru_cs, aqu_cs, channel_cs, res_cs	basin_cs, hru_cs, rout_unit_cs, aquifer_cs, channel_cs; res_cs opens both reservoir_cs and wetland_cs	Constituents
wetland_cs	none (wetland constituent files follow res_cs)	
hru_cb, hru_cb_vars, hru_cb_gl, hru_cb_trf, hru_cb_lyr, hru_cb_cpool, hru_cb_npool, hru_cb_plt, hru_cb_flux, hru_cb_drv, hru_cb_dyn, hru_cb_snap, lsu_cb_gl, lsu_cb_trf, lsu_cb_plt	carbon files	Carbon
gwflow_wb, gwflow_flux, gwflow_heat, gwflow_solute, gwflow_obs, gwflow_pump	gwflow files	gwflow

The rows hru-lte_nb and region_* are read but open no files in 62.0.1.

Other switches

The line after `csvout` in print.prt has four more switches:

Switch	Values	Files
`crop_yld`	`a` (average annual), `y` (yearly), `b` (both), `n`	`crop_yld_aa.txt`, `crop_yld_yr.txt`, `basin_crop_yld_aa.txt`, `basin_crop_yld_yr.txt`; `y` and `b` also open `yield.out`
`mgtout`	`y`, `n`	`mgt_out.txt`, the log of management operations
`hydcon`	`y`, `n`	`hydcon.out`, the object connection list (see Hydrographs)
`fdcout`	`y`, `n`	`flow_duration_curve.out`

`cdfout` (netCDF output) is read but not used.

Files written every run

File	Content
`files_out.out`	List of every output file opened in the run, with a short tag.
`diagnostics.out`	Warnings about missing or unmatched inputs.
`checker.out`	Soil and HRU parameter summary.
`erosion.out`	USLE factors per HRU.
`area_calc.out`	Area checks for routing units and landscape units.
`lu_change_out.txt`	Land use and parameter changes made by decision tables.
`simulation.out`	Run log.
`success.fin`	Written when the run finishes normally.

See Checker and diagnostics for their contents.

Related files

- print.prt: the switches described on this page.
- file.cio: names the output folder.
- Output reference: overview of the output pages.

Sources

- `print.prt` reader: basin_print_codes_read.f90; switch type `basin_print_codes` in basin_module.f90
- Output folder: output_path_module.f90, read in readcio_read.f90
- Header writers called at startup: proc_open.f90, which calls output_landscape_init.f90, header_write.f90, header_sd_channel.f90 and the other `header_*.f90` routines
- Files written every run: proc_bsn.f90, proc_hru.f90, main.f90.in
- Print window: time_control.f90

`print.prt` item	Fortran variable
`nyskip`, `day_start`, `yrc_start`, `day_end`, `yrc_end`, `interval`	`pco%nyskip`, `pco%day_start`, `pco%yrc_start`, `pco%day_end`, `pco%yrc_end`, `pco%int_day`
`aa_int_cnt` and end years	`pco%aa_numint`, `pco%aa_yrs`
`csvout`, `use_obj_lbls`, `cdfout`	`pco%csvout`, `pco%use_obj_labels`, `pco%cdfout`
`crop_yld`, `mgtout`, `hydcon`, `fdcout`	`pco%crop_yld`, `pco%mgtout`, `pco%hydcon`, `pco%fdcout`
object row `<name>`	`pco%<group>%d`, `%m`, `%y`, `%a` (for example `hru_wb` -> `pco%wb_hru`, `channel_sd` -> `pco%sd_chan`, `basin_psc` -> `pco%recall_bsn`)

Purpose

The water balance files report the daily water budget of the land surface: precipitation and snow, evapotranspiration, surface runoff, lateral and tile flow, percolation, soil water and snowpack storage, and where the runoff goes. Every flux is a depth of water in mm over the area of the object, so objects of different size can be compared directly.

Four scopes are written, each switched on by a row in print.prt:

print.prt row	Files	One row per
hru_wb	hru_wb_day.txt, hru_wb_mon.txt, hru_wb_yr.txt, hru_wb_aa.txt	HRU
hru-lte_wb	hru-lte_wb_day.txt, hru-lte_wb_mon.txt, hru-lte_wb_yr.txt, hru-lte_wb_aa.txt	HRU-LTE
lsunit_wb	lsunit_wb_day.txt, lsunit_wb_mon.txt, lsunit_wb_yr.txt, lsunit_wb_aa.txt	landscape unit
basin_wb	basin_wb_day.txt, basin_wb_mon.txt, basin_wb_yr.txt, basin_wb_aa.txt	the whole basin (one row)

Each y in the daily, monthly, yearly or avann column of a row opens the matching file. With csvout set to y, a comma-separated .csv copy of each file is written as well.

print.prt also accepts a region_wb row, but 62.0.1 writes no region files, so the row has no effect. Routing units have no water balance file: the ru row writes routing unit hydrographs (ru_day.txt and so on; see Hydrograph outputs).

Format

- Line 1: the project name from file.cio and the SWAT+ version banner.
- Line 2: column headers.
- Line 3: units.
- Lines 4 and on: one row per object per print step.

The first seven columns are the same in every landscape output file:

#	Column	Description
1	jday	Day of the year of the last day in the print step (for example 31 for a January row of a monthly file).
2	mon	Month.
3	day	Day of the month.
4	yr	Calendar year.
5	unit	Sequence number of the object (HRU number, landscape unit number). The basin file always shows 1.
6	gis_id	GIS identifier of the object from its connection file. The basin file always shows 1.
7	name	Object name. The basin file shows the project name.

The remaining columns, in file order:

#	Column	Units	Description
8	`precip`	mm	Precipitation, rain and snow, before canopy interception.
9	`snofall`	mm	Precipitation that fell as snow.
10	`snomlt`	mm	Snowmelt.
11	`surq_gen`	mm	Surface runoff that leaves the HRU on the day, after the surface runoff lag.
12	`latq`	mm	Lateral flow out of the soil profile.
13	`wateryld`	mm	Water yield: `surq_gen` + `latq` + `qtile`.
14	`perc`	mm	Percolation out of the bottom of the soil profile.
15	`et`	mm	Actual evapotranspiration: `ecanopy` + `eplant` + `esoil`.
16	`ecanopy`	mm	Evaporation of water intercepted by the canopy.
17	`eplant`	mm	Plant transpiration.
18	`esoil`	mm	Soil evaporation, including evaporation of ponded water.
19	`surq_cont`	mm	Surface runoff generated on the HRU on the day, before the surface runoff lag. For an HRU with a wetland, this is the wetland outflow.
20	`cn`	-	Curve number used for the day.
21	`sw_init`	mm	Soil water content at the start of the print step.
22	`sw_final`	mm	Soil water content at the end of the print step.
23	`sw_ave`	mm	Average soil water content over the print step.
24	`sw_300`	mm	Soil water content of the top 300 mm of soil.
25	`sno_init`	mm	Snow water content at the start of the print step.
26	`sno_final`	mm	Snow water content at the end of the print step.
27	`snopack`	mm	Average snow water content over the print step.
28	`pet`	mm	Potential evapotranspiration.
29	`qtile`	mm	Tile drain flow.
30	`irr`	mm	Irrigation water applied.
31	`surq_runon`	mm	Surface runoff received from upslope objects.
32	`latq_runon`	mm	Lateral flow received from upslope objects.
33	`overbank`	mm	Overbank flood water received from channels.
34	`surq_cha`	mm	Surface runoff sent to channels.
35	`surq_res`	mm	Surface runoff sent to reservoirs.
36	`surq_ls`	mm	Surface runoff sent to other landscape objects.
37	`latq_cha`	mm	Lateral flow sent to channels.
38	`latq_res`	mm	Lateral flow sent to reservoirs.
39	`latq_ls`	mm	Lateral flow sent to other landscape objects.
40	`gwsoilq`	mm	Groundwater moved into the soil profile when the water table rises into it (gwflow only).
41	`satex`	mm	Saturation excess flow from a high water table (gwflow only).
42	`satex_chan`	mm	Saturation excess flow that reaches a channel (gwflow only).
43	`sw_change`	mm	Change in soil water content over the day.
44	`lagsurf`	mm	Surface runoff held back by the lag and still on its way to the channel.
45	`laglatq`	mm	Lateral flow held back by the lag and still on its way to the channel.
46	`lagsatex`	mm	Saturation excess flow held back by the lag and still on its way to the channel.
47	`wet_evap`	mm	Evaporation from the wetland water surface.
48	`wet_oflo`	mm	Wetland outflow (spill).
49	`wet_stor`	mm	Water stored in the wetland.
50	`plant_cov`	text	Plant community of the HRU's land use, from `landuse.lum`.
51	`mgt_ops`	text	Management schedule of the HRU's land use, from `landuse.lum`.

The columns `surq_cha` to `latq_ls` split the HRU's generated surface runoff and lateral flow by receiving object, using the fractions in the HRU's connection file. They are filled for HRUs only and stay zero in HRU-LTE rows.

Trailing columns

The header line lists `plant_cov` and `mgt_ops` in every file, but only the `hru_wb` files write them. The other files end after `wet_stor`, except `basin_wb_aa.txt`, which ends with the soft calibration label and the adjustment value: `Original Simulation` and `0.000` when no soft calibration is run. The label is two words, so a tool that splits on spaces finds one more value than the header has names.

Aggregation

- Daily rows are written for days inside the `print.prt` daily window (`day_start` of `yrc_start` to `day_end` of `yrc_end`), every `interval` days.
- Monthly, yearly and average annual rows start after the `nyskip` warm-up years.
- Fluxes (every column except the storages listed below) are summed over the month or year. The average annual value is the sum over the printed years divided by the number of printed years.
- `cn`, `sw_ave`, `sw_300`, `snopack` and `wet_stor` are averaged over the days in the step. In the average annual file they are averaged over all printed days.
- `sw_init` and `sno_init` give the value at the start of the step, and `sw_final` and `sno_final` the value at the end. In `*_aa.txt`, `sw_final` and `sno_final` are the values on the last day of the simulation.

Landscape unit and basin rows are area-weighted averages of their HRUs and HRU-LTEs, using the area fractions in `ls_unit.ele`.

Example

The header line of `hru_wb_aa.txt` as 62.0.1 writes it, split here over several lines:

```
 jday    mon    day     yr    unit  gis_id  name            precip     snofall      snomlt    surq_gen
        latq    wateryld        perc          et     ecanopy      eplant       esoil   surq_cont
          cn     sw_init    sw_final      sw_ave      sw_300    sno_init   sno_final     snopack
         pet       qtile         irr  surq_runon  latq_runon    overbank    surq_cha    surq_res
     surq_ls    latq_cha    latq_res     latq_ls     gwsoilq       satex  satex_chan   sw_change
     lagsurf     laglatq    lagsatex    wet_evap    wet_oflo    wet_stor   plant_cov       mgt_ops
```

To check an average annual HRU row, compare `precip` + `irr` + `surq_runon` + `latq_runon` with `et` + `wateryld` + `perc` plus the change in soil water and snow storage. A large gap usually points to wetland storage, gwflow exchange or water held in the runoff lag.

The reference outputs in `refdata/Ames_sub1` of the SWAT+ repository were written by revision 61.0.1, and their header has no `plant_cov` and `mgt_ops` names.

Related files

- `print.prt`: switches each file on or off and sets the print window.
- Output naming convention: every output file and the `print.prt` row that controls it.
- Channel and sediment outputs: where the water leaving the landscape goes next.
- Post-processing helpers: reading these files with Python.

Sources

- File opening and header writes: `output_landscape_init.f90`
- Record, header and units types, and the add and average operators: `output_waterbal`, `output_waterbal_header` and `output_waterbal_header_units` in `output_landscape_module.f90`
- Daily values: `hru_control.f90`, with the runoff lag in `sq_surfst.f90` and wetland outflow in `wetland_control.f90`
- Writers: `hru_output.f90`, `hru_lte_output.f90`, `lsu_output.f90`, `basin_output.f90`
- `print.prt` reader: `basin_print_codes_read.f90`
- Print window and step flags: `time_control.f90`

Column	Fortran variable
surq_gen	qday (after sq_surfst)
wateryld	qdr(j)
perc	sepbtm(j)
et	etday
ecanopy	canev
eplant	ep_day
esoil	es_day
surq_cont	surfq(j)
cn	cnday(j)
sw_ave	output_waterbal%sw
pet	pet_day
irr	irrig(j)%applied
surq_runon	ls_overq
latq_runon	latqrunon
overbank	hru(j)%wet_obank_in
gwsoilq	output_waterbal%gwsoil, from gwsoilq(j)
sw_change	output_waterbal%delsw
lagsurf, laglatq, lagsatex	surf_bs(1,j), bss(1,j), bss_ex(1,j)
wet_oflo	output_waterbal%wet_out
wet_stor	wet(j)%flo / (10 * area_ha)

Purpose

These files report what happens in the channel network: flow, sediment and nutrients entering, leaving and stored in each channel, channel geometry and erosion, and sediment budgets for each channel, each stream order and the whole basin.

SWAT+ routes channels defined in the channel files of the SWAT+ Editor (see Channels and channel-lte). Two rows of print.prt control all of the channel files:

print.prt row	Files opened
channel_sd	channel_sd_*, channel_sdmorph_*, sd_chanbud_*, and the sediment budget files ch_sedbud_*, chanbud_order_* and sedbud_*. With a sub-daily time step and daily set to y, also channel_sd_subday.txt.
basin_sd_cha	basin_sd_cha_*, basin_sd_chamorph_*, basin_sd_chanbud_*

Each y in the daily, monthly, yearly or avann column of a row opens the matching file. Most files use the suffixes _day, _mon, _yr and _aa. The ch_sedbud and chanbud_order files use _d, _m, _y and _a. With csvout set to y, a .csv copy of each file is written as well.

The channel files are opened only when the project has at least one channel, except sd_chanbud_*, which is opened whenever its switch is on.

The channel and basin_cha rows open the legacy channel files described at the end of this page.

Files at a glance

File	One row per	Content
channel_sd_*	channel	Water surface, storage, inflow and outflow of water, sediment and nutrients, and stream temperature.
channel_sdmorph_*	channel	Flow rates, sediment transport and erosion, and channel width, depth and slope.

File	One row per	Content
`sd_chanbud_*`	channel	Sediment and nutrient budget: inflow, outflow, deposition and erosion.
`ch_sedbud_*`	channel	Bank and bed erosion, floodplain deposition, channel size.
`chanbud_order_*`	stream order (1 to 12)	The `ch_sedbud` values summed over all channels of each stream order.
`sedbud_*`	basin (one row)	Basin sediment budget: upland yield, bank and bed erosion, floodplain, reservoir and wetland deposition.
`basin_sd_cha_*`	basin	`channel_sd` summed over all channels.
`basin_sd_chamorph_*`	basin	`channel_sdmorph` summed over all channels.
`basin_sd_chanbud_*`	basin	`sd_chanbud` summed over all channels.
`channel_sd_subday.txt`	channel and time step	Outflow for each sub-daily time step.

Leading columns

Files with one row per channel start with seven columns:

#	Column	Description
1	`jday`	Day of the year of the last day in the print step.
2	`mon`	Month.
3	`day`	Day of the month.
4	`yr`	Calendar year.
5	`unit`	Channel number.
6	`gis_id`	GIS identifier of the channel.
7	`name`	Channel name.

The basin files have the same seven columns with `unit` and `gis_id` set to 1 and the project name in `name`. In `ch_sedbud_*` the same three columns are labelled `ichan`, `ob` and `name`. `chanbud_order_*` has `iord` (the stream order) in place of all three, and `sedbud_*` has only the four date columns.

channel_sd files

After the leading columns come four values for the water surface, then three blocks of 18 values (storage, inflow, outflow) and a final stream temperature.

Column	Units	Description
`area`	ha	Water surface area of the channel.
`precip`	m^3	Precipitation falling on the water surface.
`evap`	m^3	Evaporation from the water surface.
`seep`	m^3	Seepage through the channel bed.

Each block has the same 18 quantities. The column names end in `_stor`, `_in` or `_out`:

Quantity	Storage units	Inflow and outflow units	Description
`flo`	m^3	m^3/s	Water: volume stored, or average flow rate.
`sed`	tons	tons	Sediment.
`orgn`	kg N	kg N	Organic nitrogen.
`sedp`	kg P	kg P	Organic (sediment-attached) phosphorus.
`no3`	kg N	kg N	Nitrate.
`solp`	kg P	kg P	Soluble mineral phosphorus.
`chla`	kg	kg	Chlorophyll a.

Quantity	Storage units	Inflow and outflow units	Description
nh3	kg N	kg N	Ammonium.
no2	kg N	kg N	Nitrite.
cbod	kg	kg	Carbonaceous biochemical oxygen demand.
dox	kg	kg	Dissolved oxygen.
san	tons	tons	Sand.
sil	tons	tons	Silt.
cla	tons	tons	Clay.
sag	tons	tons	Small aggregates.
lag	tons	tons	Large aggregates.
grv	tons	tons	Gravel.
null	°C	°C	Water temperature.

The 18th column of each block is headed `null` and has no unit in the units row. It is the water temperature of the stored water, the inflow and the outflow. The outflow temperature is the stream temperature from the heat balance calculation.

The last column, `water_temp` (°C), is not the heat balance temperature. It is $5 + 0.75\,\bar{T}$, where $\bar{T}$ is the mean air temperature of the day at the weather station of the channel that was routed last. All channels therefore show the same value for a given day. Use the `null` column of the outflow block for the stream temperature of each channel.

Aggregation

- The storage block is the storage at the end of the print step, also in the monthly, yearly and average annual files.
- Inflow and outflow rates (`flo_in`, `flo_out`) are averaged over the days of the step. All other inflow and outflow quantities are totals for the step; the average annual file gives the average yearly total.
- Water temperatures in the inflow and outflow blocks are flow-weighted averages.
- `precip`, `evap` and `seep` are totals for the step. `area` is the average daily area in the monthly and yearly files.

channel_sdmorph files

Column	Units	Description
flo_in	m^3/s	Inflow rate.
geo_bf	m^3/s	Base flow from the geomorphic aquifer method.
flo_out	m^3/s	Outflow rate.
peakr	m^3/s	Peak flow rate.
sed_in	tons	Sediment entering the channel.
sed_out	tons	Sediment leaving the channel.
washld	tons	Wash load (suspended sediment) leaving the channel.
bedld	tons	Sediment deposited in the channel.
dep	tons	Sediment deposited on the floodplain.
deg_btm	tons	Bed erosion.
deg_bank	tons	Bank erosion.
hc_sed	tons	Sediment from gully headcut erosion.
width	m	Bankfull top width at the end of the step.
depth	m	Bankfull depth at the end of the step.
slope	m/m	Channel slope at the end of the step.
deg_btm	m	Bed downcutting.
deg_bank	m	Bank widening.
hc_len	m	Headcut retreat.
flo_in_mm	mm	Inflow as a depth over the channel's drainage area.
aqu_in_mm	mm	Geomorphic base flow as a depth over the drainage area.
flo_out_mm	mm	Outflow as a depth over the drainage area.
sed_stor	tons	Sediment stored in the channel.
n_tot	kg N	Total nitrogen leaving the channel (organic N, nitrate, ammonium and nitrite).

Column	Units	Description
p_tot	kg P	Total phosphorus leaving the channel (organic and soluble P).
dep_bf	m	Flow depth at bankfull.
velav_bf	m/s	Average velocity at bankfull.

deg_btm and deg_bank each appear twice, first in tons and then in m; read them by position. The units row labels n_tot as kg_N, p_tot as kg_P and dep_bf as m/s; dep_bf is a depth in m.

In the monthly and yearly files, the flow rates, sed_stor, n_tot, p_tot, dep_bf and velav_bf are daily averages; the tons, mm and m erosion columns are totals; and width, depth and slope are the values at the end of the step. The average annual file gives daily averages for the first group and average yearly totals for the second.

sd_chanbud files

Column	Description
in_sed, out_sed	Sediment entering and leaving the channel.
fp_dep	Sediment deposited on the floodplain.
ch_dep	Sediment deposited in the channel.
bank_ero	Bank erosion.
bed_ero	Bed erosion.
in_no3, in_orgn, out_no3, out_orgn	Nitrate and organic nitrogen entering and leaving.
fp_no3	Nitrate lost on the floodplain.
bank_no3, bed_no3	Nitrate gained from bank and bed erosion.
fp_orgn, ch_orgn	Organic nitrogen deposited on the floodplain and in the channel.
bank_orgn, bed_orgn	Organic nitrogen gained from bank and bed erosion.
in_solp, in_orgp, out_solp, out_orgp	Soluble and organic phosphorus entering and leaving.
fp_solp	Soluble phosphorus lost on the floodplain.
bank_solp, bed_solp	Soluble phosphorus gained from bank and bed erosion.
fp_orgp, ch_orgp	Organic phosphorus deposited on the floodplain and in the channel.
bank_orgp, bed_orgp	Organic phosphorus gained from bank and bed erosion.
no3_orgn	Nitrate converted to organic nitrogen in the channel.
solp_orgp	Soluble phosphorus converted to organic phosphorus in the channel.

The sediment columns (in_sed to bed_ero) are in tons. The nitrogen and phosphorus columns are in kg, although the units row shows tons for every column. All values are totals for the print step; the average annual file gives the average yearly total.

Sediment budget files

These three file groups are new in 62.0.1. They use the same channel erosion and deposition values, summed in different ways. Their rows are written in free format (list-directed), so column widths vary.

ch_sedbud files (per channel)

Column	Units	Description
num	-	Number of days in the step on which the channel had inflow.
wid	m	Bankfull width, summed over those days.
dep	m	Bankfull depth, summed over those days.
fp_km2	km^2	Floodplain area, summed over those days. The floodplain is taken as five times the channel width over the channel length.
ebank_t	tons	Bank erosion.
ebtm_t	tons	Bed erosion.

Column	Units	Description
ebank_m	m	Bank erosion as a width.
ebtm_m	m	Bed erosion as a depth.
fp_t	tons	Floodplain deposition.
fp_mm	mm	Floodplain deposition as a depth over the floodplain, assuming a bulk density of 1 t/m³.

wid, dep and fp_km2 are summed over days, so divide them by num to get the average width, depth and floodplain area in the daily, monthly and yearly files. In the average annual file the sums are divided by the number of printed years but num is not, so the ratio is too small by that factor.

chanbud_order files (per stream order)

One row for each stream order from 1 to 12 in every step, with the ch_sedbud columns summed over all channels of that order. num is then the number of channel-days with inflow, and wid / num and dep / num give the average width and depth for the order. fp_mm is summed over channels and is not a depth for the order as a whole. The stream order of each channel comes from the order column of the channel database.

sedbud files (basin)

Column	Units	Description
upland_t	tons	Sediment yield from all HRUs.
ebank_t	tons	Bank erosion in all channels.
ebtm_t	tons	Bed erosion in all channels.
fp_t	tons	Floodplain deposition in all channels.
res_dep_t	tons	Net deposition in all reservoirs (sediment in minus sediment out).
wet_dep_t	tons	Net deposition in all wetlands.

Deposition inside the channels themselves is not part of this file; it is in the ch_dep column of sd_chanbud.

Print timing

The three sediment budget file groups start after the nyskip warm-up years, like the other files. Their daily files have a row for every day after that and do not follow the daily print window (day_start to day_end, interval) in print.prt.

Basin files

basin_sd_cha_*, basin_sd_chamorph_* and basin_sd_chanbud_* have the same columns as channel_sd_*, channel_sdmorph_* and sd_chanbud_*, with one row for the basin. Each value is the sum over all channels in the basin. Flows are summed along the whole network, so flo_out in basin_sd_cha is not the flow leaving the basin; use the outlet channel's row in channel_sd for that. basin_sd_cha has no water_temp column, and in basin_sd_chamorph the width, depth and slope columns are those of the last channel.

Sub-daily outflow

When the simulation uses a sub-daily time step and the channel_sd row has y in daily, channel_sd_subday.txt has one row per channel per time step: jday, mon, day, yr, unit, gis_id, tstep (time step number), name and flo_out. flo_out is the volume of water (m³) leaving the channel in that time step, although the header gives m³/s. The units row and the header of the .csv copy are those of the channel_sdmorph file.

Legacy channel files

The `channel` and `basin_cha` rows of print.prt open `channel_*.txt` and `basin_cha_*.txt` for legacy channel objects (`chan` in object.cnt). Their 59 columns cover flow, sediment, nutrients, pesticides, bacteria, metals, sediment size classes and erosion:

```
flo_in flo_out evap tloss sed_in sed_out sed_conc orgn_in orgn_out orgp_in orgp_out
no3_in no3_out nh4_in nh4_out no2_in no2_out solp_in solp_out chla_in chla_out
cbod_in cbod_out dis_in dis_out solpst_in solpst_out sorbpst_in sorbpst_out react
volat setlpst resuspst difus reactb bury sedpest bacp baclp met1 met2 met3 sand_in
sand_out silt_in silt_out clay_in clay_out smag_in smag_out lag_in lag_out grvl_in
grvl_out bnk_ero ch_deg ch_dep fp_dep tot_ssed
```

SWAT+ 62.0.1 does not route legacy channels (the routing call is disabled), so these files contain only zeros. Projects built with the SWAT+ Editor use the channels described above.

Example

The columns of every `chanbud_order_*` file:

```
jday  mon  day  yr  iord  num  wid  dep  fp_km2  ebank_t  ebtm_t  ebank_m  ebtm_m  fp_t  fp_mm
```

Take 40 first-order channels, each 5 m wide, that have flow on every day of a 10-year print period. In `chanbud_order_y.txt` the row with `iord` 1 has `num` = 40 × 365 = 14,600 and `wid` = 73,000, so `wid` / `num` gives the 5 m average width. In `chanbud_order_a.txt` the same row has `num` = 146,000 (the count over all ten years) and `wid` = 73,000 (the ten-year sum divided by 10), so the average width is 10 × `wid` / `num`.

The reference outputs in `refdata/Ames_sub1` of the SWAT+ repository were written by revision 61.0.1 and contain none of the sediment budget files.

Related files

- print.prt: switches each file on or off.
- Channels and channel-lte: channel inputs, including the stream `order` column.
- Water balance outputs: the landscape water that reaches the channels.
- Aquifer, reservoir, plant outputs: reservoir and wetland files.
- Output naming convention: all output files.

Sources

- File opening and headers: header_sd_channel.f90, header_write.f90, header_channel.f90
- Header and record types: sd_channel_module.f90, hydrograph_module.f90, water_body_module.f90, channel_module.f90, `basin_sediment_budget` in basin_module.f90
- Channel writers: sd_channel_output.f90, sd_chanmorph_output.f90, sd_chanbud_output.f90
- Sediment budget writers: ch_sedbud_output.f90, chord_sedbud_output.f90, basin_sedbud_output.f90
- Basin writers: basin_sdchannel_output.f90, basin_chanmorph_output.f90, basin_chanbud_output.f90, basin_channel_output.f90
- Values: sd_channel_control3.f90, sd_channel_sediment3.f90, ch_temp.f90; upland, reservoir and wetland terms in hru_control.f90, res_control.f90, wetland_control.f90
- Call order: command.f90

File	Header type	Record
`channel_sd_*`	`ch_wbod_hdr`, `hyd_stor_hdr`, `hyd_in_hdr`, `hyd_out_hdr`, `wtmp_hdr`	`ch_wat_*`, `ch_stor`, `ch_in_*`, `ch_out_*`, `wtemp`
`channel_sdmorph_*`	`sdch_hdr`	`chsd_*` (`sd_ch_output`)
`sd_chanbud_*`	`sdch_bud_hdr`	`ch_sed_bud*` (`channel_sediment_budget_output`)
`ch_sedbud_*`	`ch_sedbud_hdr`	`ch_morph*` (`channel_morphology_output`)
`chanbud_order_*`	`ch_bud_order_hdr`	`ch_morph_ord*`

File	Header type	Record
sedbud_*	ch_sed_bud_hdr	bsn_sedbud* (basin_sediment_budget)
channel_sd_subday.txt	sdch_hdr_subday	ob%hyd_flo(1,ii)

Column	Fortran variable
geo_bf	sd_ch_output%aqu_in
hc_len	sd_ch_output%hc_m
flo_out_mm	sd_ch_output%flo_mm
second deg_btm, deg_bank	deg_btm_m, deg_bank_m
null (each block)	hyd_output%temp
water_temp	module variable wtemp
ob (ch_sedbud)	ob(iob)%gis_id

Purpose

This page covers four groups of output files:

- aquifer storage and fluxes (aquifer_*.txt, basin_aqu_*.txt)
- reservoir storage and fluxes (reservoir_*.txt, basin_res_*.txt)
- crop yields (crop_yld_*.txt, basin_crop_yld_*.txt)
- plant growth, stress and weather (hru_pw_*.txt, hru-lte_pw_*.txt, lsunit_pw_*.txt, basin_pw_*.txt)

Each file is switched on in print.prt. The suffix gives the print interval: _day, _mon, _yr or _aa (average annual). When csvout is y, a .csv copy with the same columns is written beside each .txt file. The first seven columns of every file except the crop yield files are jday, mon, day, yr, unit, gis_id and name; see Naming convention.

Aquifer

Files and switches

File	print.prt row	Scope
aquifer_day.txt, aquifer_mon.txt, aquifer_yr.txt, aquifer_aa.txt	aquifer	one row per aquifer per interval
basin_aqu_day.txt, basin_aqu_mon.txt, basin_aqu_yr.txt, basin_aqu_aa.txt	basin_aqu	one row per interval for the whole basin

The aquifer files report the one-dimensional aquifers defined in aquifer.aqu and are opened only when the model has such aquifers. When the gwflow module replaces the aquifers, use the gwflow outputs instead.

Columns

Line 1 is the project name and version, line 2 the column header and line 3 the units. After the seven leading columns come 17 value columns:

#	Column	Units	Description
1	flo	mm	return flow leaving the aquifer
2	dep_wt	m	depth from the ground surface to the water table
3	stor	mm	water stored in the aquifer
4	rchrg	mm	recharge entering the aquifer from other objects

#	Column	Units	Description
5	`seep`	mm	deep seepage from the bottom of the aquifer
6	`revap`	mm	water removed by plant uptake and evaporation (revap)
7	`no3_st`	kg N/ha	nitrate-N stored in the aquifer
8	`minp`	kg P	mineral P sent out with the return flow (see the notes)
9	`orgn` (header)	%	organic carbon content of the aquifer (see the notes)
10	`orgp` (header)	kg N/ha	organic N in the aquifer (see the notes)
11	`no3_rchg`	kg N/ha	nitrate-N entering with recharge; `rchrgn` in earlier documentation
12	`no3_loss`	kg N/ha	nitrate-N lost in the aquifer (decay); `nloss` in earlier documentation
13	`no3_lat`	kg N/ha	nitrate-N leaving with the return flow; `no3gw` in earlier documentation
14	`no3_seep`	kg N/ha	nitrate-N leaving with deep seepage; `seepno3` in earlier documentation
15	`flo_cha`	mm	part of `flo` routed to channels
16	`flo_res`	mm	part of `flo` routed to reservoirs
17	`flo_ls`	mm	part of `flo` routed to other aquifers

Notes on individual columns:

- Columns 9 and 10 are mislabelled. The header says `orgn` (kg/ha_N) and `orgp` (kg/ha_P), but column 9 is the organic carbon content from the `carbon` column of `aquifer.aqu`, constant through the run, and column 10 is organic N, which 62.0.1 never updates, so it stays zero.
- `minp` is the mineral P concentration from the `sol_p` column of `aquifer.aqu` multiplied by the day's recharge volume. The value is a mass in kg for the whole aquifer; the units line gives kg/ha_P. It is computed from the recharge volume and leaves the aquifer with the return flow.
- `flo_cha`, `flo_res` and `flo_ls` split `flo` by the outflow fractions in the aquifer's connection file. Only outflows of type `tot` are counted, so the three columns add up to `flo` only when every outflow is of that type.

Aggregation

Fluxes are summed over the interval. Three state columns are averaged instead: the monthly file divides `stor`, `dep_wt` and `no3_st` by the number of days in the month, and the yearly file divides the sum of the monthly averages by 12. The average annual file divides the yearly values by the number of printed years. Column 9 is never summed: every interval reports the starting value.

The basin file is the area-weighted average of all aquifers, each weighted by its share of the basin area.

Example header

`basin_aqu_yr.txt` from the `Ames_sub1` reference dataset run with 62.0.1:

```
   jday   mon   day    yr   unit  gis_id  name                      flo          dep_wt            stor
                                                                     mm               m              mm
```

Reservoir

Files and switches

File	`print.prt` row	Scope
`reservoir_day.txt`, `reservoir_mon.txt`, `reservoir_yr.txt`, `reservoir_aa.txt`	`reservoir`	one row per reservoir per interval

File	print.prt row	Scope
basin_res_day.txt, basin_res_mon.txt, basin_res_yr.txt, basin_res_aa.txt	basin_res	all reservoirs added together

The reservoir files are opened only when the model has reservoirs.

Columns

After the seven leading columns come four blocks: 4 water body columns, then 18 storage, 18 inflow and 18 outflow columns.

Block	Columns	Units
Water body	area, precip, evap, seep	ha, m³, m³, m³
Storage at the end of the interval	flo_stor, sed_stor, orgn_stor, sedp_stor, no3_stor, solp_stor, chla_stor, nh3_stor, no2_stor, cbod_stor, dox_stor, san_stor, sil_stor, cla_stor, sag_stor, lag_stor, grv_stor, null	see below
Inflow during the interval	the same 18 names ending in _in	see below
Outflow during the interval	the same 18 names ending in _out	see below

Units of the 18 constituent columns, which repeat in the storage, inflow and outflow blocks:

Constituent	Units	Description
flo	m³	water volume
sed	t	sediment
orgn	kg N	organic N
sedp	kg P	organic (sediment-attached) P
no3	kg N	nitrate-N
solp	kg P	soluble mineral P
chla	kg	chlorophyll-a
nh3	kg N	ammonium-N
no2	kg N	nitrite-N
cbod	kg	carbonaceous biochemical oxygen demand
dox	kg	dissolved oxygen
san, sil, cla, sag, lag, grv	t	sand, silt, clay, small aggregates, large aggregates and gravel
null	°C	water temperature (the header gives null and no unit)

The inflow columns are flo_in, sed_in, orgn_in, sedp_in, no3_in, solp_in, chla_in, nh3_in, no2_in, cbod_in, dox_in, san_in, sil_in, cla_in, sag_in, lag_in, grv_in and null. The outflow columns are flo_out, sed_out, orgn_out, sedp_out, no3_out, solp_out, chla_out, nh3_out, no2_out, cbod_out, dox_out, san_out, sil_out, cla_out, sag_out, lag_out, grv_out and null (the last column of each block is headed null).

Inflow and outflow are volumes and masses for the interval, in m³, t and kg. The storage block is the reservoir content when the line is written, so in the monthly, yearly and average annual files it is the storage on the last day of the interval.

Aggregation

Inflow, outflow, precipitation, evaporation and seepage are summed over the interval, and the average annual file divides the totals by the number of printed years. The surface area is averaged over the days of the month, and the yearly value is the average of the monthly values.

reservoir_mon.csv has one extra line after the units line, RES reservoir_mon.csv, which the other reservoir files do not have.

Crop yield

Files and switches

The `crop_yld` column of `print.prt` controls all crop yield files: `y` gives yearly files, `a` average annual files, `b` both, and `n` none.

File	Written when `crop_yld` is	Content
`crop_yld_aa.txt`	`a` or `b`	one row per HRU and plant at the end of the run
`crop_yld_yr.txt`	`y` or `b`	header lines only (see below)
`basin_crop_yld_yr.txt`	`y` or `b`	one row per plant per year for the whole basin
`basin_crop_yld_aa.txt`	`y` or `b`	one row per plant at the end of the run

With `crop_yld = a`, only `crop_yld_aa.txt` is produced; the basin files need `y` or `b`. In 62.0.1 no data rows are written to `crop_yld_yr.txt` (or its `.csv`). For yearly yields use `basin_crop_yld_yr.txt`, or `yield.out` (switched on by `mgtout`) for every harvest.

HRU columns

`crop_yld_aa.txt` has a title line, a line `--YIELD (kg/ha)--`, the column header, and one row per plant in each HRU:

#	Column	Units	Description
1	`jday`	-	day of year of the last simulated day
2	`mon`	-	month
3	`day`	-	day of month
4	`yr`	-	year
5	`unit`	-	HRU number
6	`PLANTNM`	-	plant name from `plants.plt`
7	`MASS`	kg/ha	dry mass of the harvested yield
8	`C`	kg/ha	carbon in the yield
9	`N`	kg/ha	nitrogen in the yield
10	`P`	kg/ha	phosphorus in the yield

The values are the total yield over the whole run divided by the number of harvests of that plant, that is, the average yield per harvest. A plant that was never harvested reports zero. The `.csv` header lists a single `yield` column, while each data row contains the four values `MASS`, `C`, `N` and `P`.

Basin columns

#	Column	Units	Description
1	`year`	-	calendar year (the last year in the `_aa` file)
2	`plant_no`	-	plant number in the basin plant list
3	`plant_name`	-	plant name
4	`harv_area(ha)`	ha	area harvested; an HRU harvested twice in a year counts twice
5	`yld(t)`	t	total dry yield
6	`yld(t/ha)`	t/ha	`yld(t)` divided by `harv_area(ha)`

The yearly file skips the years before `nyskip`. In the average annual file, area and yield are divided by the number of printed years, and `yld(t/ha)` is total yield divided by total harvested area.

Example

`crop_yld_aa.txt` and `basin_crop_yld_aa.txt` from a two-year run of the `Ames_sub1` reference dataset with 62.0.1:

```
                                                                                --YIELD (kg/ha)--
  jday    mon    day     yr     unit PLANTNM                          MASS           C           N           P
   366     12     31   1976        1    corn                                    3692.158    1661.471
   366     12     31   1976        1    soyb                                       0.000       0.000

      year       plant_no   plant_name       harv_area(ha)     yld(t)          yld(t/ha)
       1976              1 corn                                  12.04350       47.48362       3.942675
       1976              2 soyb                                0.0000000E+00  0.0000000E+00  0.0000000
```

Plant and weather

Files and switches

File	print.prt row	Scope
`hru_pw_day.txt` to `hru_pw_aa.txt`	`hru_pw`	one row per HRU
`hru-lte_pw_day.txt` to `hru-lte_pw_aa.txt`	`hru-lte_pw`	one row per HRU-lte
`lsunit_pw_day.txt` to `lsunit_pw_aa.txt`	`lsunit_pw`	one row per landscape unit
`basin_pw_day.txt` to `basin_pw_aa.txt`	`basin_pw`	whole basin

All use the columns below. The HRU files add `plant_cov` and `mgt_ops` at the end of each row: the plant community and the management schedule of the HRU's land use.

Columns

#	Column	Units	Description
1	`lai`	m^2/m^2	leaf area index
2	`bioms`	kg/ha	total plant biomass
3	`yield`	kg/ha	harvested yield (dry mass)
4	`residue`	kg/ha	surface residue
5	`sol_tmp`	°C	temperature of soil layer 2
6	`strsw`	days	water (drought) stress days
7	`strsa`	days	aeration (excess water) stress days
8	`strstmp`	days	temperature stress days
9	`strsn`	days	nitrogen stress days
10	`strsp`	days	phosphorus stress days
11	`strss`	days	salinity stress days
12	`nplt`	kg N/ha	nitrogen in the plant biomass
13	`percn`	kg N/ha	nitrate-N leached below the soil profile
14	`pplnt`	kg P/ha	phosphorus in the plant biomass
15	`tmx`	°C	maximum air temperature
16	`tmn`	°C	minimum air temperature
17	`tmpav`	°C	mean air temperature
18	`solarad`	MJ/m^2	solar radiation
19	`wndspd`	m/s	wind speed
20	`rhum`	fraction	relative humidity
21	`phubas0`	°C	accumulated base-zero heat units
22	`lai_max`	m^2/m^2	maximum leaf area index (always 0 in 62.0.1)
23	`bm_max`	kg/ha	maximum total biomass during the interval
24	`bm_grow`	kg/ha	biomass growth
25	`c_gro`	kg C/ha	plant carbon growth

The units line writes `kgha` for columns 2 to 4 and 12 to 14 and `----` for the stress columns.

Aggregation

Columns	Monthly	Yearly	Average annual
`yield`, stress days, `percn`	sum	sum	yearly sum averaged over the printed years
`nplt`, `pplnt`	plant N and P at the end of the month	at the end of the year	at the end of the run
`bm_max`	maximum in the month	maximum in the year	maximum in the run
all other columns	daily average	daily average	daily average

`nplt` and `pplnt` are the N and P contents of the standing plants. They do not report uptake during the interval. After a harvest or kill they drop to zero, so yearly and average annual values are often zero for annual crops.

Example

`hru_pw_mon.txt`, `Ames_sub1` run with 62.0.1 (header, units and first data row):

jday	mon	day	yr	unit	gis_id	name	lai	bioms	yield	residue
							m**2/m**2	kgha	kgha	kgha
31	1	31	1975	1	1	hru0001	0.000	0.000	0.000	17278.244

Related files

- print.prt: switches for every file on this page.
- aquifer.aqu and reservoir.res: the objects reported here.
- Water balance outputs and Channel and sediment outputs.
- gwflow outputs: groundwater output when gwflow replaces the aquifers.

Sources

- Aquifer: aquifer_output.f90 (units 2520 to 2527), basin_aquifer_output.f90 (units 2090 to 2097), headers and arithmetic in aquifer_module.f90, file opening in header_aquifer.f90 and header_write.f90, values set in aqu_1d_control.f90 and aqu_initial.f90
- Reservoir: reservoir_output.f90 (units 2540 to 2547), basin_reservoir_output.f90, header_reservoir.f90, water body type in water_body_module.f90, constituent headers in hydrograph_module.f90
- Crop yield: hru_output.f90 (unit 4008), file opening in output_landscape_init.f90 (units 4008 to 4011) and header_yield.f90, basin rows in time_control.f90 (units 5100 and 5101)
- Plant and weather: hru_output.f90 (units 2040 to 2047), daily values in hru_control.f90, type, header and operators in output_landscape_module.f90
- `print.prt` rows: basin_print_codes_read.f90

Output column	Fortran variable
aquifer column 9 (`orgn` in the header)	`aqu_d%cbn` (from `aqu_dat%cbn`)
aquifer column 10 (`orgp` in the header)	`aqu_d%orgn`
aquifer `minp`	`aqu_d%minp`
reservoir water body block	`res_wat_d` (`type water_body`)
reservoir storage block	`res(j)` (`type hyd_output`)
reservoir inflow and outflow blocks	`res_in_d`, `res_out_d`
`crop_yld_aa` `MASS`, `C`, `N`, `P`	`pl_mass%yield_tot(ipl)` divided by `pcom%plcur(ipl)%harv_num`
plant and weather `nplt`, `pplnt`	`hpw_d%nplnt`, `hpw_d%pplnt`, both copied from `pl_mass%tot_com`
plant and weather `solarad`, `phubas0`	`hpw_d%solrad`, `hpw_d%phubase0`

Purpose

Two groups of output files report nutrients on the land:

- The nutrient balance files (`*_nb_*`) give the N and P added to, moved within and removed from the soil and plants: grazing and fertiliser inputs, fixation, atmospheric deposition, mineralisation and other transfers between soil pools, denitrification and plant uptake.
- The losses files (`*_ls_*`) give the sediment and nutrients that leave the land with surface runoff, lateral flow, tile flow and percolation.

All values are amounts per unit area summed over the print interval. Each group has a row in `print.prt` for each scope, with daily, monthly, yearly and average annual flags.

Files and switches

Scope	Nutrient balance row	Nutrient balance files	Losses row	Losses files
HRU	`hru_nb`	`hru_nb_{day,mon,yr,aa}.txt`	`hru_ls`	`hru_ls_{day,mon,yr,aa}.txt`
HRU-lte	`hru-lte_nb`	none (see below)	`hru-lte_ls`	`hru-lte_ls_{day,mon,yr,aa}.txt`
Landscape unit	`lsunit_nb`	`lsunit_nb_{day,mon,yr,aa}.txt`	`lsunit_ls`	`lsunit_ls_{day,mon,yr,aa}.txt`
Basin	`basin_nb`	`basin_nb_{day,mon,yr,aa}.txt`	`basin_ls`	`basin_ls_{day,mon,yr,aa}.txt`

Each file also gets a `.csv` copy when `csvout = y`. Notes on the switches:

- `print.prt` also accepts `hru-lte_nb`, `region_nb` and `region_ls` rows, but 62.0.1 writes no files for them.
- The `hru_nb` row also opens `hru_ncycle_{day,mon,yr,aa}.txt`, a nutrient cycling file with the columns `lab_min_p`, `act_sta_p`, `act_nit_n`, `act_sta_n`, `org_lab_p`, `rsd_hs_c`, `rsd_nitrorg_n` and `rsd_laborg_p`. In 62.0.1 these files contain only their header lines. The same transfers are in the `hru_nb` files.
- Landscape unit files are written only when the model defines landscape units in `ls_unit.def`.

Line 1 of every file is the project name and version, line 2 the column header and line 3 the units. The first seven columns are `jday`, `mon`, `day`, `yr`, `unit`, `gis_id` and `name`; see Naming convention.

Nutrient balance columns

#	Column	Units	Description
1	`grzn`	kg N/ha	nitrogen added to the soil in manure from grazing
2	`grzp`	kg P/ha	phosphorus added to the soil in manure from grazing
3	`lab_min_p`	kg P/ha	net phosphorus flow from the labile mineral pool to the active mineral pool; negative when the flow goes the other way
4	`act_sta_p`	kg P/ha	net phosphorus flow from the active mineral pool to the stable mineral pool; negative when the flow goes the other way
5	`fertn`	kg N/ha	nitrogen applied with fertiliser and manure (mineral and organic)
6	`fertp`	kg P/ha	phosphorus applied with fertiliser and manure (mineral and organic)
7	`fixn`	kg N/ha	nitrogen added to plant biomass by fixation
8	`denit`	kg N/ha	nitrogen lost from the nitrate pool by denitrification
9	`act_nit_n`	kg N/ha	nitrogen moved from the active organic pool to the nitrate pool (mineralisation)
10	`act_sta_n`	kg N/ha	nitrogen moved from the active organic pool to the stable organic pool
11	`org_lab_p`	kg P/ha	phosphorus moved from the organic pool to the labile pool (mineralisation)
12	`rsd_nitorg_n`	kg N/ha	nitrogen moved from fresh residue to the nitrate (80%) and active organic (20%) pools

#	Column	Units	Description
13	`rsd_laborg_p`	kg P/ha	phosphorus moved from fresh residue to the labile (80%) and organic (20%) pools
14	`no3atmo`	kg N/ha	nitrate added to the soil by atmospheric deposition, wet and dry
15	`nh4atmo`	kg N/ha	ammonium added to the soil by atmospheric deposition, wet and dry
16	`nuptake`	kg N/ha	nitrogen taken up by plants
17	`puptake`	kg P/ha	phosphorus taken up by plants
18	`gwsoiln`	kg N/ha	nitrate added to the soil from groundwater (gwflow module only)
19	`gwsoilp`	kg P/ha	phosphorus added to the soil from groundwater (gwflow module only)

The units line writes `kgha` for every column. The HRU files add `plant_cov` and `mgt_ops` at the end of each row: the plant community and management schedule of the HRU's land use. The landscape unit and basin files have the same two labels in their header but write no values under them.

Columns 9 to 13 (`act_nit_n` to `rsd_laborg_p`) are filled only with the static carbon model, `carbon = 0` in `codes.bsn`. With `carbon = 2` they stay at zero, and the corresponding flows are in the carbon outputs. `denit` and the mineral P columns 3 and 4 are filled under both settings.

Losses columns

#	Column	Units	Description
1	`sedyld`	t/ha	sediment yield leaving the land by water erosion
2	`sedorgn`	kg N/ha	organic nitrogen attached to sediment in surface runoff
3	`sedorgp`	kg P/ha	organic phosphorus attached to sediment in surface runoff
4	`surqno3`	kg N/ha	nitrate in surface runoff
5	`lat3no3`	kg N/ha	nitrate in lateral flow
6	`surqsolp`	kg P/ha	soluble phosphorus in surface runoff
7	`usle`	t/ha	soil loss predicted by the USLE (the units line says `tons`)
8	`sedminp`	kg P/ha	mineral phosphorus (active and stable) attached to sediment
9	`tileno3`	kg N/ha	nitrate in tile flow
10	`lchlabp`	kg P/ha	soluble (labile) phosphorus leached below the soil profile
11	`tilelabp`	kg P/ha	soluble (labile) phosphorus in tile flow
12	`satexn`	kg N/ha	nitrate in saturation excess runoff; always 0 in 62.0.1

The HRU files add `plant_cov`, `mgt_ops` and `percn` at the end of each row. `percn` (kg N/ha) is the nitrate leached below the soil profile, the same value as `percn` in the plant and weather files. The HRU-lte, landscape unit and basin files have these three labels in their header but write no values under them.

In the HRU-lte files only `sedyld` is calculated; the other columns are always zero.

Aggregation

Every column is a sum. The monthly and yearly files add up the daily values, and the average annual file divides the run total by the number of printed years. Nothing is averaged over days.

Landscape unit values are the sum of the HRU (and HRU-lte) values, each multiplied by its `RU_FRAC` in `ls_unit.ele`. Basin values are the sum of the HRU and HRU-lte values, each multiplied by its `BSN_FRAC` in `ls_unit.ele`. The fractions are used as given and are not checked: when they do not add up to 1, the basin and landscape unit values

are scaled by the same factor. In the `Ames_sub1` reference dataset the `BSN_FRAC` values add up to 2, so its basin values are twice the area-weighted HRU average.

Example

`hru_nb_yr.txt` and `hru_ls_yr.txt` from a two-year run of the `Ames_sub1` reference dataset with 62.0.1, header, units and first data row:

jday	mon	day	yr	unit	gis_id	name	grzn	grzp	lab_min_p	act_sta_p
							kgha	kgha	kgha	kgha
365	12	31	1975	1	1	hru0001	0.000	0.000	911.096	721.581

jday	mon	day	yr	unit	gis_id	name	sedyld	sedorgn	sedor
							tha	kgha	kg
365	12	31	1975	1	1	hru0001	0.033	0.172	0.

This dataset runs the dynamic carbon model (`carbon = 2`), so columns 9 to 13 of the nutrient balance are zero.

Related files

- print.prt: output switches.
- Water balance outputs: the runoff, lateral flow, tile flow and percolation volumes.
- Aquifer, reservoir, and plant outputs: plant N and P and leached nitrate.
- Carbon outputs: soil carbon and the N and P in the carbon pools.
- Nutrient cycling theory.

Sources

- Column types and headers: `type output_nutbal`, `type output_losses`, `output_nutbal_header`, `output_losses_header` and their units types in output_landscape_module.f90
- File opening: output_landscape_init.f90 (HRU units 2020 to 2037, `hru_ncycle` 3333 to 3340, basin 2060 to 2077, landscape unit 2150 to 2167, HRU-lte losses 2440 to 2447)
- Writers: hru_output.f90, hru_lte_output.f90, lsu_output.f90, basin_output.f90
- Daily values: hru_control.f90, hru_lte_control.f90, nut_pminrl.f90, nut_nminrl.f90, nut_nrain.f90, nut_solp.f90, pl_nup.f90, pl_pup.f90, ero_ysed.f90, pl_manure.f90, rsd_decomp.f90, cbn_zhang2.f90
- Weights: `BSN_FRAC` and `RU_FRAC` read in lsu_read_elements.f90
- `print.prt` rows: basin_print_codes_read.f90

Output column	Fortran variable
`grzn`, `grzp`	`hnb_d%grazn`, `hnb_d%grazp`
`lat3no3`	`hls_d%latno3`
`sedminp`	`hls_d%sedminp = sedminpa + sedminps`
`sedyld`	`sedyld / hru%area_ha`
`percn` (losses, HRU)	`hpw_d%percn` (`percn_aa = hpw_a%percn / yrs_prt` in the `_aa` file)
landscape unit rows	`runb_*`, `ruls_*` built with `lsu_elem%ru_frac`
basin rows	`bnb_*`, `bls_*` built with `lsu_elem%bsn_frac`

Purpose

SWAT+ moves water and its loads between objects (HRUs, routing units, channels, reservoirs, aquifers and so on) as hydrographs: a record of water volume, sediment, nutrients and temperature for the day. The files on this page report those hydrographs.

File	One row per	Content
`hydout_*`	object and outgoing connection	What each object sends to each receiving object.
`hydin_*`	object and incoming connection	What each object receives from each source object.
`deposition_*`	HRU or routing unit that receives inflow	Material deposited from incoming water.
`ru_*`	routing unit	Sum of the hydrographs of the routing unit's elements.
`hydcon.out`	object	The routing order and connections, written once at the start of the run.

`hydin`, `hydout` and `deposition` are switched on by the `hyd` row of `print.prt`, and `ru` by the `ru` row. Each `y` in the `daily`, `monthly`, `yearly` or `avann` column opens the `_day`, `_mon`, `_yr` or `_aa` file. `hydcon.out` is switched on by `hydcon` on the line after `csvout`. With `csvout` set to `y`, a `.csv` copy of each file is written as well.

The connections themselves are defined in the connection files (see connection files): each object lists the objects it sends water to, the hydrograph type it sends and the fraction.

Hydrograph columns

All four time-step file groups end with the same 18 columns:

Column	Units	Description
`flo`	m^3	Volume of water.
`sed`	tons	Sediment.
`orgn`	kg N	Organic nitrogen.
`sedp`	kg P	Organic (sediment-attached) phosphorus.
`no3`	kg N	Nitrate.
`solp`	kg P	Soluble mineral phosphorus.
`chla`	kg	Chlorophyll a.
`nh3`	kg N	Ammonium.
`no2`	kg N	Nitrite.
`cbod`	kg	Carbonaceous biochemical oxygen demand.
`dox`	kg	Dissolved oxygen.
`san`	tons	Sand.
`sil`	tons	Silt.
`cla`	tons	Clay.
`sag`	tons	Small aggregates.
`lag`	tons	Large aggregates.
`grv`	tons	Gravel.
`null`	°C	Water temperature.

`flo` is a volume: m^3 per day in the daily file, the total for the month or year in the monthly and yearly files, and the average yearly total in the average annual file. The units row of the files gives m^3/s for `flo`. To convert a daily value to an average rate, divide by 86,400.

All other loads are totals for the step in the same way. The last column is headed `null` and is the water temperature; monthly, yearly and average annual values are flow-weighted averages.

hydout files

#	Column	Description
1	`jday`	Day of the year of the last day in the print step.
2	`mon`	Month.
3	`day`	Day of the month.

#	Column	Description
4	`yr`	Calendar year.
5	`name`	Name of the source object.
6	`type`	Object type of the source object (`hru`, `hru_lte`, `ru`, `aqu`, `chandeg`, `res`, `recall`, `exco`, `dr`, `outlet`).
7	`objtyp`	Type code of the receiving object, as in the connection file (`hru`, `ru`, `aqu`, `sdc`, `res` and so on).
8	`typ_no`	Number of the receiving object within its type.
9	`hyd_typ`	Hydrograph type sent: `tot` (total), `rhg` (recharge), `sur` (surface runoff), `lat` (lateral flow) or `til` (tile flow).
10	`fraction`	Fraction of that hydrograph sent to this receiving object.
11 to 28	`flo` to `null`	The hydrograph sent, already multiplied by `fraction`.

An object with three outgoing connections has three rows per print step. The rows of all objects follow the routing order, so the rows of one step are not sorted by object name.

hydin files

The columns are those of `hydout`, seen from the receiving side:

#	Column	Description
5	`name`	Name of the receiving object.
6	`type`	Object type of the receiving object.
7	`objtyp`	Type code of the source object.
8	`typ_no`	Number of the source object within its type.
9	`hyd_typ`	Hydrograph type received.
10	`fraction`	Fraction of the source hydrograph received.
11 to 28	`flo` to `null`	The hydrograph received.

Rows are written for every incoming connection, but values are filled only when the receiving object is not an HRU, HRU-LTE or routing unit. For those three receiving types the incoming water is split into surface, lateral and tile parts and routed inside the object, and the `hydin` rows stay zero. Use the `hydout` rows of the source object instead.

`hydin_yr.csv` has one more column than the other `hydin` files: the object number is written after `type`, so the later columns are shifted by one against the header.

deposition files

`deposition_*` has `jday`, `mon`, `day`, `yr`, `name` and `type`, then the 18 hydrograph columns. It has a row for every HRU and routing unit that receives water from another object.

- For an HRU without a wetland, the `sed` column is the sediment deposited on the HRU from incoming surface runoff, in tons: the incoming sediment above the transport capacity of the runoff over the HRU. The other columns are zero.
- For an HRU with a wetland or rice paddy, incoming surface runoff and tile flow go into the wetland, and the row shows that inflow. In this case `flo` is in mm and the loads are per hectare.
- For a routing unit, the row shows the hydrograph of the last element processed in the routing unit. It does not measure deposition.

The deposition files are written from the start of the simulation, including the `nyskip` warm-up years, while the average annual value is divided by the number of printed years only.

ru files

`ru_*` has `jday`, `mon`, `day`, `yr`, `name` and `type`, then the 18 hydrograph columns, with one row per routing unit.

Each value is the sum, over the elements of the routing unit, of every hydrograph type the element produces. For an HRU element that is the total hydrograph plus its recharge, surface, lateral and tile hydrographs, so `flo` and the loads are larger than the routing unit's outflow. The outflow of a routing unit is in the `hydout` rows of the routing unit, with `hyd_typ` `tot` for the total flow.

hydcon.out

`hydcon.out` has no title or header line. It lists the objects in the order SWAT+ processes them, one line per object:

Field	Description
1	Sequence number in the routing order.
2	Object name.
3	Object type.
4	Number of the object's properties record in its data file.
5	Overbank connection pointer to landscape units (0 when none).
6	Number of outgoing connections.
7	Number of incoming connections.
8 and on	For each outgoing connection: the receiving object numbers, the receiving type code, its number within the type and the hydrograph type.

In 62.0.1 the receiving object numbers are written as the full list of all receiving objects for every outgoing connection, so lines for objects with more than one outgoing connection repeat that list.

Constituent hydrographs

print.prt and the source refer to hydrograph files for pesticides, pathogens, metals and salts (`hydin_pests_*`, `hydout_salts_*` and so on). SWAT+ 62.0.1 does not write them.

Print timing

`hydin`, `hydout` and `ru` start after the `nyskip` warm-up years. Daily rows follow the daily print window and `interval` in print.prt. `deposition` is written from the first simulated year, as described above.

Example

The column names of `hydout_day.txt` and `hydin_day.txt`:

```
jday  mon  day  yr  name  type  objtyp  typ_no  hyd_typ  fraction
flo  sed  orgn  sedp  no3  solp  chla  nh3  no2  cbod  dox  san  sil  cla  sag  lag  grv  null
```

A `hydout` row for an HRU sending 60% of its total flow to a routing unit has `type` `hru`, `objtyp` `ru`, `hyd_typ` `tot` and `fraction` 0.6, and its `flo` is 0.6 times the HRU's water yield in m^3.

Related files

- print.prt: the `hyd`, `ru` and `hydcon` switches.
- Connection files: the connections these files report.
- Recall outputs: point source and inlet hydrographs, in the same column layout.
- Channel and sediment outputs: channel inflow and outflow as rates.
- Output naming convention: all output files.

Sources

- File opening and headers: header_hyd.f90 (`hydin`, `hydout`, `deposition`, `hydcon.out`), header_write.f90 (`ru`)

- Header types: `hyd_header`, `hyd_header_time`, `hyd_header_obj`, `hyd_header_units`, `hyd_header_units2` in hydrograph_module.f90
- Writers: hydout_output.f90, hydin_output.f90, hyddep_output.f90, ru_output.f90, hyd_connect_out.f90
- Values: command.f90 (incoming and outgoing hydrographs), ru_control.f90, hru_control.f90, rls_routesurf.f90, hru_hyds.f90
- Hydrograph type codes: hyd_connect.f90

Column	Fortran variable
`type`	`ob(icmd)%typ`
`objtyp`, `typ_no`, `hyd_typ`, `fraction` (`hydout`)	`ob(icmd)%obtyp_out(iout)`, `obtypno_out`, `htyp_out`, `frac_out`
`objtyp`, `typ_no`, `hyd_typ`, `fraction` (`hydin`)	`ob(icmd)%obtyp_in(iin)`, `obtypno_in`, `htyp_in`, `frac_in`
hydrograph (`hydout`)	`ht1 = frac_out * ob(icmd)%hd(ihtyp_out)`
hydrograph (`hydin`)	`ob(icmd)%hin_d(iin)` and the `hin_m`, `hin_y`, `hin_a` sums
hydrograph (`deposition`)	`ht1` after `hru_control` or `ru_control`, summed into `ob(icmd)%hdep_m`, `hdep_y`, `hdep_a`
hydrograph (`ru`)	`ru_d(iru)` and the `ru_m`, `ru_y`, `ru_a` sums
`null`	`hyd_output%temp`

Purpose

A recall object adds a measured or externally simulated time series of water and loads to the model: a point source such as a treatment plant, an inlet from an area outside the model, or, with negative flows, a diversion. The recall output files show what each recall object released into the model.

print.prt row	Files	One row per
`recall`	`recall_day.txt`, `recall_mon.txt`, `recall_yr.txt`, `recall_aa.txt`	recall object
`basin_psc`	`basin_psc_day.txt`, `basin_psc_mon.txt`, `basin_psc_yr.txt`, `basin_psc_aa.txt`	basin (sum of all recall objects)

Each `y` in the `daily`, `monthly`, `yearly` or `avann` column of print.prt opens the matching file. With `csvout` set to `y`, a `.csv` copy of each file is written as well. The `basin_psc` files are written only when the project has at least one recall object.

Format

- Line 1: the project name from file.cio and the SWAT+ version banner.
- Line 2: column headers.
- Line 3: units.
- Lines 4 and on: data rows.

#	Column	Units	Description
1	`jday`	-	Day of the year of the last day in the print step.
2	`mon`	-	Month.
3	`day`	-	Day of the month.
4	`yr`	-	Calendar year.
5	`name`	-	Name of the recall object. In `basin_psc` files, the project name.

#	Column	Units	Description
6	`type`	-	Object type, always `recall`. `basin_psc` files have no `type` column; the header has a blank name in its place.
7	`flo`	m^3	Water released.
8	`sed`	tons	Sediment.
9	`orgn`	kg N	Organic nitrogen.
10	`sedp`	kg P	Organic (sediment-attached) phosphorus.
11	`no3`	kg N	Nitrate.
12	`solp`	kg P	Soluble mineral phosphorus.
13	`chla`	kg	Chlorophyll a.
14	`nh3`	kg N	Ammonium.
15	`no2`	kg N	Nitrite.
16	`cbod`	kg	Carbonaceous biochemical oxygen demand.
17	`dox`	kg	Dissolved oxygen.
18	`san`	tons	Sand.
19	`sil`	tons	Silt.
20	`cla`	tons	Clay.
21	`sag`	tons	Small aggregates.
22	`lag`	tons	Large aggregates.
23	`grv`	tons	Gravel.
24	`null`	°C	Water temperature.

The values are the recall record used on the day, unchanged: SWAT+ does not convert the numbers read from the recall time series. The model passes `flo` on to the receiving object as a daily volume in m^3, so that is how the column should be read, although the units row gives m^3/s. The `.csv` copies of the `recall` files use a different units row, which gives `flo` as m^3.

Values by recall time step

The time step of each recall object is set in its recall file:

- Daily: each day uses the record for that day. Outside the years covered by the time series, the values are zero.
- Monthly: the record for the month is used on every day of that month. A monthly row of `recall_mon.txt` is therefore the monthly record multiplied by the number of days in the month. Outside the covered years, the values are zero.
- Yearly: the record for the year is used on every day of that year, so a row of `recall_yr.txt` is 365 or 366 times the yearly record.
- Sub-daily: the daily row is the sum of the records for that day's time steps.

A negative `flo` in a daily recall time series is a withdrawal. The row shows the negative value as read. In addition, SWAT+ removes nitrate, ammonium, nitrite, organic nitrogen, soluble phosphorus and dissolved oxygen from the receiving channel's storage at the channel's current concentration. Those removed amounts are not in the recall files.

Aggregation

Monthly and yearly rows are totals over the step for all columns except `null`, which is a flow-weighted average. The average annual row is the average yearly total. Rows start after the `nyskip` warm-up years, and daily rows follow the daily print window and `interval` in `print.prt`.

`basin_psc` rows are sums over all recall objects, so withdrawals (negative flows) offset releases.

Example

The column names of `recall_day.txt`:

```
jday  mon  day  yr  name  type
flo  sed  orgn  sedp  no3  solp  chla  nh3  no2  cbod  dox  san  sil  cla  sag  lag  grv  null
```

A treatment plant entered as a monthly recall object with `flo` = 1,200 for March shows 1,200 on every March row of `recall_day.txt` and 37,200 in the March row of `recall_mon.txt`.

Related files

- Recall inputs: the recall objects and their time series.
- `print.prt`: the `recall` and `basin_psc` switches.
- Hydrograph outputs: the same column layout for water passed between objects; the `hydout` rows of a recall object show where its water goes.
- Output naming convention: all output files.

Sources

- File opening and headers: `header_write.f90`
- Header types: `hyd_header`, `hyd_header_time`, `rec_header_time`, `hyd_header_units`, `hyd_header_units3` in `hydrograph_module.f90`
- Writers: `recall_output.f90`, `basin_recall_output.f90`
- Daily record selection: `case ("recall")` in `command.f90`
- Reading the time series: `recall_read.f90`
- Nutrient removal for withdrawals: `recall_nut.f90`

Column	Fortran variable
hydrograph (`recall`)	`rec_d(irec) = ob(icmd)%hd(1)`, summed into `rec_m`, `rec_y`, `rec_a`
hydrograph (`basin_psc`)	`brec_d` (sum of `rec_d`), `brec_m`, `brec_y`, `brec_a`
`null`	`hyd_output%temp`

Purpose

The carbon output files report the carbon budget of the soil, residue and plants. Two kinds of file are available:

- Flat files with one row per HRU or landscape unit per interval: carbon gains and losses, soil carbon transformations and plant carbon.
- Per-layer files with one row per HRU per interval and one column per soil layer for each variable: soil carbon totals, carbon pools, N and P in the pools, carbon and N fluxes, process drivers, pool dynamics and soil properties.

Every family has a row in `print.prt` with the usual daily, monthly, yearly and average annual flags. The rows are recognised only when `print.prt` uses object labels (`use_obj_lbls = y`); in the fixed-order layout there is no place for them. A row name that SWAT+ does not know stops the run with an error.

Most columns are filled by the dynamic carbon model, which runs when `carbon = 2` in `codes.bsn`. Four per-layer families (`hru_cb_cpool`, `hru_cb_flux`, `hru_cb_drv`, `hru_cb_dyn`) are written only with `carbon = 2`. The other families are written for any setting, but with `carbon = 0` many of their columns stay at zero. See the carbon model theory for the pools and transformations.

Files and switches

`print.prt` row	Files	Rows	Layers
`hru_cb_gl`	`hru_carb_gl_{day,mon,yr,aa}`	HRU	no
`hru_cb_trf`	`hru_scf_{day,mon,yr,aa}`	HRU	no
`hru_cb_lyr`	`hru_cbn_lyr_{day,mon,yr,aa}`	HRU	yes
`hru_cb_cpool`	`hru_cpool_stat_{day,mon,yr,aa}`	HRU	yes, `carbon = 2` only
`hru_cb_npool`	`hru_n_p_pool_stat_{day,mon,yr,aa}`	HRU	yes

print.prt row	Files	Rows	Layers
hru_cb_plt	hru_plc_stat_{day,mon,yr,aa}	HRU	no
hru_cb_flux	hru_cflux_stat_{day,mon,yr,aa}	HRU	yes, carbon = 2 only
hru_cb_drv	hru_carb_drv_{day,mon,yr,aa}	HRU	yes, carbon = 2 only
hru_cb_dyn	hru_carb_dyn_{day,mon,yr,aa}	HRU	yes, carbon = 2 only
hru_cb_snap	hru_soil_snap_{day,mon,yr} and hru_soil_snap_tot	HRU	yes
lsu_cb_gl	lsu_carb_gl_{day,mon,yr,aa}	landscape unit	no
lsu_cb_trf	lsu_scf_{day,mon,yr,aa}	landscape unit	no
lsu_cb_plt	lsu_plc_stat_{day,mon,yr,aa}	landscape unit	no

Each file is written as .txt, plus .csv when csvout = y. The hru_cb_snap row writes _tot in place of _aa; see Soil snapshot. The landscape unit files are written only when the model defines landscape units in ls_unit.def.

Example print.prt rows (labelled layout):

objects	daily	monthly	yearly	avann
hru_cb_gl	n	y	y	y
hru_cb_lyr	n	n	y	y
hru_cb_snap	n	n	n	y

Value types and aggregation

The flat gain and loss files (hru_carb_gl, hru_scf, lsu_carb_gl, lsu_scf) report fluxes. Monthly and yearly files are sums over the interval, and the average annual file divides the run total by the number of printed years.

All other files report a state at the moment the row is written: the monthly file gives the value on the last day of the month, the yearly file the value on the last day of the year, and the average annual file the value at the end of the run. They are not averages. This also applies to the per-layer flux file hru_cflux_stat: its daily, monthly and yearly rows give the flux of the last day of the interval in kg/ha/day, and only the average annual file gives the run total divided by the number of printed years, in kg/ha/yr.

Flat HRU files

Carbon gains and losses

hru_carb_gl_* (row hru_cb_gl). One row per HRU. After the seven leading columns (jday, mon, day, yr, unit, gis_id, name) come 21 columns in kg C/ha, in three groups.

Soil:

Column	Description
sed_c	carbon removed with eroded sediment
surq_c	dissolved carbon in surface runoff
latq_c	dissolved organic carbon in lateral flow, all layers
perc_c	dissolved carbon in percolation below the profile
res_decay	carbon added to the soil by residue decay
man_app_c	carbon added with applied manure
man_graze_c	carbon added with manure from grazing animals
rsp_c	CO_2-C from soil respiration, whole profile
soil_emit_c	CO_2-C from burning soil carbon

Residue:

Column	Description
plant_surf_c	carbon added to surface residue by leaf drop and kill
plant_root_c	carbon added to soil residue by root death
rsd_surfdecay_c	carbon lost from surface residue by decay
rsd_rootdecay_c	carbon lost from root and incorporated residue by decay
harv_stov_c	carbon removed by residue harvest
rsd_emit_c	CO_2-C from burning surface residue

Plant:

Column	Description
npp_c	plant carbon growth (net primary production)
harv_abgr_c	carbon removed by grain and biomass harvest
harv_root_c	carbon removed by tuber (root) harvest
drop_c	carbon moved to residue by leaf drop and kill
grazeat_c	carbon eaten by grazing animals
plant_emit_c	CO_2-C from burning plants

Soil carbon transformations

hru_scf_* (row hru_cb_trf). One row per HRU with 13 columns in kg C/ha: meta_micr, str_micr, str_hs, co2_meta, co2_str, micr_hs, micr_hp, hs_micr, hs_hp, hp_micr, co2_micr, co2_hs, co2_hp. Each name gives the source and destination pool: meta metabolic litter, str structural litter, micr microbial biomass, hs slow humus, hp passive humus, and co2 carbon lost as CO_2.

In 62.0.1 these columns are reset to zero each day and never filled, so every value in hru_scf_* and lsu_scf_* is zero. For the same flows by layer, use hru_cflux_stat (cfmets1 for meta_micr, co2fs1 for co2_micr, and so on).

Plant carbon

hru_plc_stat_* (row hru_cb_plt). One row per HRU with seven columns in kg C/ha:

Column	Description
total_c	carbon in all plant parts
ab_gr_c	carbon in above-ground biomass
leaf_c	carbon in leaves
stem_c	carbon in stems
seed_c	carbon in seeds
root_c	carbon in roots
surf_rsd_c	carbon in residue

Per-layer HRU files

Layout

Each row starts with the seven leading columns, then a depth block depth_lyr1 to depth_lyrN giving the depth to the bottom of each soil layer in mm, then one block of N columns for each variable, named <variable>_lyr1 to <variable>_lyrN. There is no units line: line 1 is the project name and version, line 2 the column header, and the data start on line 3.

N is the same for every HRU. By default it is the largest number of soil layers among all HRUs, so the deepest profile is written in full. The optional file carbon_layers.prt sets N directly. In an HRU with fewer layers than N, the extra columns contain -99.

Soil carbon by layer

`hru_cbn_lyr_*` (row `hru_cb_lyr`). Line 1 ends with the note `total soil carbon (Mg/ha) by layer depth in mm`. The value columns are in Mg C/ha:

Columns	Description
`depth_lyr1` … `depth_lyrN`	depth to the bottom of each layer (mm)
`tot_300_sum`	total soil organic carbon in the top 300 mm
`tot_lyr1` … `tot_lyrN`	total soil organic carbon in each layer
`seq_300_sum`	sequestered (humus and microbial) carbon in the top 300 mm
`seq_lyr1` … `seq_lyrN`	sequestered carbon in each layer

For the 300 mm sums, the layer that crosses 300 mm is counted in proportion to its thickness above 300 mm. With `carbon = 0`, both totals are the sum of the active humus, stable humus and microbial carbon.

Carbon pools

`hru_cpool_stat_*` (row `hru_cb_cpool`, `carbon = 2` only). Ten variables per layer:

Variable	Units	Description
`residue_c`	kg C/ha	residue carbon in the layer
`structural_c`	kg C/ha	structural litter
`metabolic_c`	kg C/ha	metabolic litter
`hs_c`	kg C/ha	slow humus
`hp_c`	kg C/ha	passive humus
`microbial_c`	kg C/ha	microbial biomass
`lignin_c`	kg C/ha	lignin part of structural litter
`nonlignin_c`	kg C/ha	non-lignin part of structural litter
`root_mass`	kg/ha	root mass in the layer
`soil_water`	mm	water in the layer

N and P in the pools

`hru_n_p_pool_stat_*` (row `hru_cb_npool`). Eighteen variables per layer, in kg N/ha and kg P/ha: `tot_pool_n`, `residue_n`, `structural_n`, `metabolic_n`, `hs_n`, `hp_n`, `microbial_n`, `lignin_n`, `nonlignin_n`, then the same nine with `_p`. `tot_pool_n` and `tot_pool_p` add residue, structural and metabolic litter, slow and passive humus, microbial biomass, the dissolved organic pool and manure.

Carbon and N fluxes

`hru_cflux_stat_*` (row `hru_cb_flux`, `carbon = 2` only). Thirty-seven variables per layer. In the names, `s1` is microbial biomass, `s2` slow humus, `s3` passive humus, `met` metabolic litter and `str` structural litter, so `cfs1s2` is the carbon flow from microbial biomass to slow humus.

Variables	Units	Description
`cfmets1`, `cfstrs1`, `cfstrs2`	kg C/ha/day	carbon moved from litter to microbial biomass or slow humus
`efmets1`, `efstrs1`, `efstrs2`	kg N/ha/day	nitrogen moved with those flows
`immmets1`, `immstrs1`, `immstrs2`	kg N/ha/day	N immobilised by the litter flows
`mnrmets1`, `mnrstrs1`, `mnrstrs2`	kg N/ha/day	N mineralised by the litter flows
`co2fmet`, `co2fstr`	kg C/ha/day	CO_2-C from metabolic and structural litter
`cfs1s2`, `cfs1s3`, `cfs2s1`, `cfs2s3`, `cfs3s1`	kg C/ha/day	carbon moved between the humus and microbial pools
`efs1s2`, `efs1s3`, `efs2s1`, `efs2s3`, `efs3s1`	kg N/ha/day	nitrogen moved with those flows
`imms1s2`, `imms1s3`, `imms2s1`, `imms2s3`, `imms3s1`	kg N/ha/day	N immobilised by those flows
`mnrs1s2`, `mnrs1s3`, `mnrs2s1`, `mnrs2s3`, `mnrs3s1`	kg N/ha/day	N mineralised by those flows
`co2fs1`, `co2fs2`, `co2fs3`	kg C/ha/day	CO_2-C from microbial biomass, slow humus and passive humus

The average annual file gives yearly totals (kg/ha/yr); see Value types and aggregation.

Process drivers

`hru_carb_drv_*` (row `hru_cb_drv`, `carbon = 2` only). Fourteen variables per layer, all for the day the row is written:

Variable	Units	Description
`sut`	fraction	soil water control on decomposition
`tillagef`	-	tillage factor of the layer
`cons_bmix`	-	biological mixing coefficient of the layer
`tillagef_biomix`	-	part of the tillage factor from biological mixing
`tillagef_tillmix`	-	part of the tillage factor from tillage operations
`till_eff`	fraction	tillage effect on decomposition
`cdg`	fraction	soil temperature control on decomposition
`ox`	fraction	oxygen control on decomposition
`cs`	fraction	combined control on decomposition
`no3`	kg N/ha	nitrate available to the transformations
`nh4`	kg N/ha	ammonium available to the transformations
`co2_resp`	kg C/ha	CO_2-C respired in the layer
`soil_temp`	°C	soil temperature
`emix`	fraction	fraction of the layer mixed by tillage or biological mixing

Pool dynamics

`hru_carb_dyn_*` (row `hru_cb_dyn`, `carbon = 2` only). Twenty-one variables per layer:

Variables	Units	Description
`asp`, `abp`	fraction	share of decomposed slow humus and of decomposed microbial biomass sent to passive humus
`abco2`, `a1co2`, `asco2`, `apco2`	fraction	share of decomposed microbial biomass, metabolic and passive pools, slow humus and passive humus lost as CO_2
`ncbm`, `nchp`, `nchs`	ratio	N:C ratio of microbial biomass, passive humus and slow humus
`bmctp`, `bmntp`	kg/ha/day	potential C and N transformation of microbial biomass
`hsctp`, `hsntp`	kg/ha/day	potential C and N transformation of slow humus
`hpctp`, `hpntp`	kg/ha/day	potential C and N transformation of passive humus
`lmctp`, `lmntp`	kg/ha/day	potential C and N transformation of metabolic litter
`lsctp`, `lslctp`, `lslnctp`, `lsntp`	kg/ha/day	potential transformation of structural litter: C, lignin C, non-lignin C and N

Soil snapshot

`hru_soil_snap_*` (row `hru_cb_snap`). Thirteen soil properties per layer:

Variable	Units	Description
`bd`	Mg/m^3	bulk density
`awc`	mm/mm	available water capacity
`soil_k`	mm/h	saturated hydraulic conductivity
`tot_c`	kg C/ha	total soil organic carbon
`clay`, `silt`, `sand`, `rock`	%	particle size fractions and rock fragments
`alb`	fraction	albedo
`usle_k`	-	USLE soil erodibility
`ec`	dS/m	electrical conductivity
`caco3`	%	calcium carbonate content
`ph`	-	soil pH

The daily, monthly and yearly flags write `hru_soil_snap_day`, `_mon` and `_yr`. The average annual flag writes `hru_soil_snap_tot` instead, which has one row per HRU at the start of the run (marked `jday = 0`) and one at the end. Both rows give `tot_c` in kg C/ha, so the change in soil carbon in layer K over the run is `tot_c_lyrK` on the end row minus `tot_c_lyrK` on the start row. The start rows are written only when `carbon = 2`.

Repeated rows

In 62.0.1 the files of the `hru_cb_lyr`, `hru_cb_cpool`, `hru_cb_npool`, `hru_cb_plt`, `hru_cb_flux`, `hru_cb_drv` and `hru_cb_dyn` rows, and the daily, monthly and yearly files of `hru_cb_snap`, repeat their block of HRU rows once per HRU. In a model with 12 HRUs, every interval has 144 rows: the 12 HRU rows written 12 times with the same values. Keep the first occurrence of each `unit` within an interval when reading these files. `hru_soil_snap_tot`, `hru_carb_gl`, `hru_scf` and the landscape unit files are not affected.

Landscape unit files

The landscape unit files are area-weighted averages of the HRU values, each HRU weighted by its fraction of the landscape unit. Per-layer files have no landscape unit version because the HRUs in a unit can have different soil profiles.

File	Columns
`lsu_carb_gl_*`	the 21 columns of `hru_carb_gl`
`lsu_scf_*`	the 13 columns of `hru_scf` (all zero in 62.0.1)
`lsu_plc_stat_*`	one column, `lsu_plt_c`: carbon in all plant parts (kg C/ha)

`lsu_plt_c` is a state value: the monthly, yearly and average annual files give the value on the last day of the interval.

Layer count file

`carbon_layers.prt` is optional and is not listed in `file.cio`. When it is present in the run folder, it fixes N, the number of layer columns in every per-layer file. Use it to keep the same number of columns across runs, or to write only the top layers.

- Line 1: title, skipped.
- Line 2: header, skipped.
- Line 3: the number of layers, a whole number of 1 or more.

```
carbon_layers.prt: layers to expand in per-layer carbon output
n_layers
10
```

With `10`, every per-layer file has ten columns per variable, whatever the soils. A value below 1 is rejected with a message in `diagnostics.out`, and N then defaults to the largest layer count among the HRUs.

Legacy CSU carbon outputs

An older set of carbon files, kept for existing CSU workflows, runs beside the files above. SWAT+ Editor does not write the rows that switch them on; they are added to `print.prt` by hand. The code marks them for removal in revision 63.

`print.prt` row and flag	Files
`hru_cb` with `y` or `l` in the daily, monthly or yearly column	`hru_cbn_lyr.txt`, `hru_seq_lyr.txt`, `hru_n_p_pool_stat.txt`

print.prt row and flag	Files
as above, with `1` in any column	adds `hru_begsim_soil_prop.txt` and `hru_endsim_soil_prop.txt` (the start rows need `carbon = 2`)
as above, with `carbon = 2`	adds `hru_plc_stat.txt`, `hru_cflux_stat.txt`, `hru_cpool_stat.txt`
`hru_cb` with any flag other than `n`	`basin_carbon_all.txt` (header lines only when `hru_nb` has `y` in the average annual column)
`hru_cb_vars` with `y` or `1` in the daily, monthly or yearly column, and `carbon = 2`	`hru_carbvars.txt`, `hru_org_allo_vars.txt`, `hru_org_ratio_vars.txt`, `hru_org_trans_vars.txt`

These files have no interval suffix: one file collects the rows for every interval switched on. Each also gets a `.csv` copy when `csvout = y`. The legacy files contain the same quantities as the files above in a different layout. For example, legacy `hru_cbn_lyr.txt` and `hru_seq_lyr.txt` together contain the values that the new `hru_cbn_lyr_yr.txt` gives in one table.

The legacy files do not use the layer count file. The `cbn_diagnostics` setting from the old `carb_coefs.cbn` no longer exists; the `1` flag on the `hru_cb` row takes its place.

Example

Header and first row of `hru_carb_gl_yr.txt` from a two-year run of the `Ames_sub1` reference dataset with 62.0.1 (`carbon = 2`), shortened to the first eleven value columns:

```
      jday        mon        day         yr            unit              gis_id     name

       365         12         31       1975               1                   1 hru0001          0.702
```

Start of the first data row of `hru_cbn_lyr_yr.txt` from the same run, as values per column. Every soil in this dataset has 16 layers, so N = 16 and no column contains `-99`:

Column	Value
`unit`	1
`depth_lyr1` to `depth_lyr16`	10, 25, 50, 100, 150, 200, 300, 450, 600, 750, 900, 1050, 1200, 1500, 1800, 2000
`tot_300_sum`	44.06
`tot_lyr1`, `tot_lyr2`, `tot_lyr3`	2.28, 2.55, 4.24

The 12 HRU rows for 1975 appear 12 times each in this file (see Repeated rows).

Related files

- `print.prt`: output switches.
- `codes.bsn`: the `carbon` switch.
- `carbon.bsn` and `carbon_lyr.bsn`: carbon model inputs.
- Carbon model theory: pools and transformations.
- Naming convention: file suffixes and leading columns.

Sources

- `print.prt` rows: `basin_print_codes_read.f90`

- File opening and headers: output_landscape_init.f90, header types output_carb_gl_header and output_hscf_header in output_landscape_module.f90, variable lists and header and row helpers in carbon_module.f90
- Flat HRU files: hru_carbon_output.f90 (units 4520 to 4527 and 4550 to 4557)
- Per-layer files: soil_nutcarb_write.f90 and soil_carbvar_write.f90 (units 4530 to 4605), called from command.f90 and, for the end-of-run snapshot, hru_output.f90
- Landscape unit files: lsu_carbon_output.f90 (units 4750 to 4773), landscape unit count from lsu_read_elements.f90
- Layer count file: carbon_layers_read.f90
- Legacy files: carbon_legacy_module.f90 (units 8348 to 8387)

print.prt row	Flag variable
hru_cb_gl, hru_cb_trf, hru_cb_lyr, hru_cb_cpool, hru_cb_npool	pco%cb_gl_hru, pco%cb_trf_hru, pco%cb_lyr_hru, pco%cb_cpool_hru, pco%cb_npool_hru
hru_cb_plt, hru_cb_flux, hru_cb_drv, hru_cb_dyn, hru_cb_snap	pco%cb_plt_hru, pco%cb_flux_hru, pco%cb_drv_hru, pco%cb_dyn_hru, pco%cb_snap_hru
lsu_cb_gl, lsu_cb_trf, lsu_cb_plt	pco%cb_gl_lsu, pco%cb_trf_lsu, pco%cb_plt_lsu
hru_cb, hru_cb_vars (legacy)	pco%cb_hru, pco%cb_vars_hru

Output columns	Fortran source
hru_carb_gl	hsc_d, hrc_d, hpc_d (carbon_soil_gain_losses, carbon_residue_gain_losses, carbon_plant_gain_losses); res_decay is rsd_decay_c, man_graze_c is man_graz_c, the three *_emit_c columns are each type's emit_c
hru_scf	hscf_d (carbon_soil_transformations)
tot_lyrK, seq_lyrK, tot_300_sum, seq_300_sum	soil1%tot(k)%c, soil1%seq(k)%c, soil1%tot_300_c, soil1%seq_tot_300_c, all divided by 1000
hru_cflux_stat	soil1%org_flx_lr(k); _aa uses soil1%org_flx_cum_lr(k) divided by yrs_prt
hru_carb_drv	soil1%org_con_lr(k), soil%ly(k)%tillagef*, soil%ly(k)%bmix, soil%phys(k)%tmp, soil1%emix(k)
hru_carb_dyn	soil1%org_allo_lr(k), soil1%org_ratio_lr(k), soil1%org_tran_lr(k)
lsu_plt_c	sum of pl_mass%tot_com%c * lsu_elem%ru_frac

Purpose

The gwflow module simulates groundwater on a grid of cells and replaces the one-dimensional aquifers. It runs when gw_flow = 1 in codes.bsn and writes the output files described here. The input files are described on the gwflow input page.

The files fall into these groups:

- basin and cell water balance (gwflow_basin_wb_*, gwflow_cell_wb_*)
- heat and solute balances (gwflow_basin_heat_*, gwflow_basin_sol_*)
- observation wells (gwflow_obs_*)
- HRU pumping (gwflow_hru_pump_*)
- diagnostic files for canals, ponds, tiles and channels
- channel hydrograph separation (gwflow_chan_hydsep_day.txt)
- the cell definition and the run log, which are always written

Output switches

The write_day, write_mon, write_yr and write_aa flags in codes.gw (1 on, 0 off) switch the water balance files. The heat and solute balance files follow the same flags, and the hydrograph separation file follows write_day. Observation well, pumping and diagnostic output is on by default.

print.prt can override these settings with six rows, each with the usual daily, monthly, yearly and average annual flags. The rows are recognised only when print.prt uses object labels (use_obj_lbls = y). A row that is present replaces the default; a row that is absent leaves it as it is. All overrides are applied before any gwflow output file is opened, so a file that is switched off is not created.

print.prt row	Effect
gwflow_wb	each flag switches the water balance file for that interval; heat, solute and hydrograph separation files follow it unless their rows are present
gwflow_heat	each flag switches the heat balance file for that interval (there is no monthly heat file)
gwflow_solute	each flag switches the solute balance files for that interval
gwflow_obs	each flag switches the observation well file for that interval
gwflow_pump	any flag set to y switches on all HRU pumping files
gwflow_flux	the daily or yearly flag switches on the canal, pond, tile and channel diagnostic files

Example rows that keep only the monthly water balance and the daily observation wells:

objects	daily	monthly	yearly	avann
gwflow_wb	n	y	n	n
gwflow_obs	y	n	n	n
gwflow_pump	n	n	n	n
gwflow_flux	n	n	n	n

Basin water balance

gwflow_basin_wb_day.txt, _mon.txt, _yr.txt and _aa.txt give the water balance of the whole grid. Volumes are summed over all active cells and divided by the watershed area, so they are in mm over the watershed. Line 1 is a title, line 2 the column header and line 3 the units. The seven leading columns are jday, mon, day, yr, unit, gis_id and name (the project name).

The daily file has these columns after the leading ones:

Column	Units	Description
ts	day	gwflow time step
vbef	mm	groundwater volume at the start of the day
vaft	mm	groundwater volume at the end of the day
recharge	mm	recharge from the soil profile
gw_et	mm	groundwater evapotranspiration
gw_sw	mm	groundwater discharge to channels
sw_gw	mm	channel seepage to groundwater
sat_excess	mm	saturation excess flow to the land surface
soil	mm	groundwater moved into the soil profile
lateral	mm	net lateral flow between cells
boundary	mm	exchange across the grid boundary
pump_allo	mm	pumping for water allocation (irrigation)
pump_ext	mm	pumping specified in pumpex.gw
tile	mm	tile drain outflow
reservoir	mm	exchange with reservoirs
wetland	mm	exchange with wetlands
canal	mm	canal seepage
floodplain	mm	exchange with floodplains

Column	Units	Description
pond	mm	recharge pond seepage
phytorem	mm	phreatophyte transpiration
error	%	volume balance error
sat_frac	fraction	fraction of active cells with saturation excess flow that day
wt_depth	m	average depth to the water table (see the note below)
pump_def	mm	allocation pumping demand that could not be met

The monthly, yearly and average annual files replace ts, vbef and vaft with dvol (change in storage, mm) and drop error, sat_frac and wt_depth. They end with pump_def after phytorem.

The daily wt_depth is the average over the active cells. In the average annual file, dvol is the total change in storage over the run divided by the number of years, and every flux is the run total divided by the number of years.

Cell water balance

gwflow_cell_wb_day.txt, _mon.txt, _yr.txt and _aa.txt have one row per active cell per interval. Lines 1 and 2 are notes on units; line 3 is the column header.

Column	Units	Description
jday, mon, day, yr	-	date of the row
unit	-	cell number in the gwflow grid
gis_id	-	cell ID from the gis_id column of cells.gw (the gwflow cell shapefile ID)
name	-	cell name from cells.gw, such as cell0001 (first 12 characters)
head	m	groundwater head
wt_depth	m	land surface elevation of the cell minus the head
recharge	m^3/day	recharge
gw_et	m^3/day	groundwater evapotranspiration
gw_sw	m^3/day	discharge to channels
sw_gw	m^3/day	channel seepage
sat_excess	m^3/day	saturation excess flow
soil	m^3/day	transfer to the soil profile
lateral	m^3/day	net lateral flow
pump_allo	m^3/day	allocation pumping
pump_ext	m^3/day	specified pumping
tile	m^3/day	tile drain outflow
reservoir	m^3/day	reservoir exchange
wetland	m^3/day	wetland exchange
floodplain	m^3/day	floodplain exchange
canal	m^3/day	canal seepage
pond	m^3/day	recharge pond seepage
phytorem	m^3/day	phreatophyte transpiration

The monthly, yearly and average annual files give the average daily rate over the interval, so all fluxes stay in m^3/day.

wt_depth is measured from the cell's land surface elevation, which on a coarse grid represents the valley and not the streambed. Cells along channels can therefore show a deep water table while the head is close to the channel level. This comes from the grid resolution; the head itself is not affected.

Heat balance

When heat transport is on (heat = 1 in codes.gw), gwflow_basin_heat_day.txt, _yr.txt and _aa.txt are written. They follow the daily, yearly and average annual water balance switches, or the gwflow_heat row of print.prt when it is present; there is no monthly heat file. Values are in MJ.

The daily file has `ts`, `hbef` and `haft` (heat stored at the start and end of the day), then `recharge`, `gw_et`, `gw_sw`, `sw_gw`, `sat_excess`, `soil`, `lateral`, `dispersion`, `boundary`, `pump_allo`, `pump_ext`, `tile`, `reservoir`, `wetland`, `canal`, `floodplain`, `pond`, then `error` (%) and `tavg` (average groundwater temperature, °C). The yearly and average annual files start with `hdel` (change in heat stored) and end at `pond`. ## Solute balances {#output-reference-gwflow-solute-balances}

When solute transport is on (`solute = 1` in codes.gw), one file per solute and interval is written, named `gwflow_basin_sol_<solute>_<interval>.txt`. Nitrate (`no3`) and phosphorus (`p`) are always included; with salts in the model, `so4`, `ca`, `mg`, `na`, `k`, `cl`, `co3` and `hco3` are added; with constituents, `seo4` and `seo3`. Values are in kg. The interval files follow the water balance switches, or the `gwflow_solute` row of print.prt when it is present.

The daily file has `ts`, `mbef` and `maft` (solute mass at the start and end of the day), then `recharge`, `gw_sw`, `sw_gw`, `sat_excess`, `soil`, `advection`, `dispersion`, `react_in`, `react_out`, `mineral`, `sorption`, `pump_allo`, `pump_ext`, `tile`, `reservoir`, `wetland`, `canal`, `floodplain`, `pond` and `error` (%). The monthly, yearly and average annual files start with `delm` (change in mass) and end at `pond`.

Observation wells

Observation cells are listed as `observation_cell` lines in outputs.gw. The files `gwflow_obs_day.txt`, `_mon.txt`, `_yr.txt` and `_aa.txt` have one row per observation cell:

Column	Units	Description
`unit`	-	observation well number, in the order listed in outputs.gw
`gis_id`	-	cell number of the observation cell
`name`	-	`obs_0001`, `obs_0002`, …
`head`	m	groundwater head
`wt_depth`	m	land surface elevation minus head
`temperature`	°C	groundwater temperature; `-99` when heat transport is off
`no3`	mg/L	nitrate concentration; `-99` when solute transport is off
`p`	mg/L	phosphorus concentration; `-99` when solute transport is off

The monthly and yearly files give averages over the interval, and the average annual file the average over the run.

HRU pumping

When pumping for water allocation is simulated, `gwflow_hru_pump_day.txt`, `_mon.txt`, `_yr.txt` and `_aa.txt` report the groundwater pumped for each HRU in the `pump_allo` column. The daily file is in m^3/day and lists only HRUs that pumped that day; the monthly and yearly files give totals in m^3, and the average annual file m^3/yr.

When hru_pump.gw lists HRUs to observe, `gwflow_cell_wb_ppag_obs_day.txt` gives their daily pumping in m^3, one column per listed HRU. It is written under the same switch as the other pumping files.

Diagnostic files

These files are written when the related feature is in use and diagnostic output is on (`gwflow_flux`):

File	Content
`gwflow_canal_wb_day.txt`	daily canal water balance in m^3: `diversion`, `storage`, `pond_xfer`, `seepage`
`gwflow_canal_sol_day.txt`	daily canal solute balance in kg: `div_mass`, `storage`, `pond_mass`, `seep_mass` for each solute
`gwflow_pond_wb_day.txt`	daily recharge pond water balance: `area` (m^2), `storage`, `rain`, `div_added`, `evap`, `recharge`, `div_spec`, `div_unsat` (m^3)
`gwflow_pond_sol_day.txt`	daily pond solute balance: `area`, `storage`, `solute`, `mass`, `div_added`, `recharge` (kg)
`gwflow_pond_mass_day.txt`, `gwflow_pond_conc_day.txt`	daily solute mass (kg) and concentration (g/m^3), one column per pond

File	Content
`gwflow_tile_group_day.txt`	daily tile flow (m^3/s) for each group of tile cells
`gwflow_gwsw_group_day.txt`	daily groundwater and channel exchange (m^3) for each channel cell group in `sw_group.gw`
`gwflow_chan_obs_flow_day.txt`	daily groundwater and channel exchange (m^3) at the channel cells flagged in `chancell.gw`
`gwflow_chan_obs_no3_day.txt`	the same for nitrate (g), when solute transport is on

When `gwflow.wbgroups` is present, one daily water balance file per cell group is also written, named `gwflow_group_wb_day_` followed by the group number as two characters (a space and the digit for groups 1 to 9). Its columns follow the daily basin file, in m^3.

Hydrograph separation

`gwflow_chan_hydsep_day.txt` is written when daily water balance output is on. It gives, for each channel and day, the flow from each source in m^3/s: `surf_runoff`, `lateral`, `gw_discharge`, `stream_seep`, `satex_gw`, `satex_sw` and `tile_drain`.

Files always written

File	Content
`gwflow_record`	run log: which input files were found, which features are active, which `print.prt` rows changed the output switches, and error messages from reading
`gwflow_cell_definition.txt`	one line per cell: `cell_id`, `row`, `col`, `x_coord`, `y_coord`, `zone`, `status`, `area`

When `transit.gw` is present on a structured grid, `gwflow_transit_cell`, `gwflow_transit_chan` and (with tiles) `gwflow_transit_tile` give travel times in days.

The `head_output_time` lines in `outputs.gw` are still read but no longer produce output; cell heads are in the cell water balance files.

Related files

- gwflow input files
- `codes.bsn`: `gw_flow` switch
- `print.prt`: output rows
- Aquifer theory

Sources

- Output file setup and basin, cell, heat, solute and observation writers: `gwflow_output.f90` (`gwflow_output_init`, `gwflow_output_day`, the monthly, yearly and average annual routines, `gwflow_write_celldef`)
- `codes.gw` flags, `print.prt` overrides, pumping, diagnostic and hydrograph separation file setup: `gwflow_read.f90`
- Flags and unit numbers: `gwflow_module.f90`
- Daily simulation, pumping output and the call to the average annual output: `gwflow_simulate.f90`; pond and canal diagnostics in `gwflow_pond.f90` and `gwflow_canal_div.f90`
- `print.prt` rows: `basin_print_codes_read.f90`; `gwflow_record` opened in `basin_read_objs.f90`

Switch	Fortran variable
`write_day`, `write_mon`, `write_yr`, `write_aa` in `codes.gw`, or the `gwflow_wb` row	`gwflag_day`, `gwflag_mon`, `gwflag_yr`, `gwflag_aa`
`gwflow_heat` row (default: water balance flags)	`gwflag_heat_day`, `gwflag_heat_yr`, `gwflag_heat_aa`
`gwflow_solute` row (default: water balance flags)	`gwflag_solute_day`, `gwflag_solute_mon`, `gwflag_solute_yr`, `gwflag_solute_aa`
`gwflow_obs` row (default: all on)	`gwflag_obs_day`, `gwflag_obs_mon`, `gwflag_obs_yr`, `gwflag_obs_aa`, and `gwflag_obs` as their maximum
`gwflow_pump`, `gwflow_flux` rows (default: on)	`gwflag_pump`, `gwflag_flux`
hydrograph separation	`hydsep_flag`, set from `gwflag_day`
`heat`, `solute` in `codes.gw`	`gw_heat_flag`, `gw_solute_flag`

Cell file column	Fortran variable
`gis_id`, `name`	`cell_gis_id(i)`, `cell_name(i)` (written with `a12`)
`recharge` … `phytorem`	`gw_hyd_ss(i)%rech`, `gwet`, `gwsw`, `swgw`, `satx`, `soil`, `latl`, `ppag`, `ppex`, `tile`, `resv`, `wetl`, `fpln`, `canl`, `pond`, `phyt`

Purpose

SWAT+ writes the files on this page on every run; `print.prt` cannot switch them off. They show whether the run finished, which inputs SWAT+ could not match, the soil and HRU settings it used, and which output files it wrote. Check them before using any other output.

File	Content
`simulation.out`	run log: version banner, weather files read, one line per simulated day, completion message
`diagnostics.out`	input problems found while reading: names not found, files missing, values rejected
`checker.out`	one row per HRU with soil and hydrology settings after setup
`files_out.out`	list of output files opened in this run
`success.fin`	written only at the end of a run that finished
`area_calc.out`	drainage areas of routed objects, as given and as calculated
`erosion.out`	header lines only in 62.0.1
`erosion.txt`	empty in 62.0.1
`paddy_test.csv`	daily paddy rice water balance; column header only unless the model has paddy wetlands

With the gwflow module on, `gwflow_record` is also always written; see gwflow outputs.

simulation.out

The run log. It contains, in order:

- a banner with the revision, the compiler and its version, the build date and the operating system
- the date and time of the run
- one line for each weather input read (`reading from precipitation file`, `reading from temperature file`, and so on)
- one line per simulated day, giving the simulation name, month, day, year, the year number and the number of years, and the clock time
- at the end, `Execution successfully completed` and the date and time again

Each daily line starts with `Original Simulation` in a normal run. During soft calibration other names appear in that place.

```
                  SWAT+
              Revision 62.0.1
      Soil & Water Assessment Tool
IntelLLVM (2025.0.4), 2026-10-05 00:10:27, Linux
    Program reading . . . executing


 Date of Sim  10/20/2026 Time  12: 9:24
 reading from wgn file                                    Time  12: 9:24
 reading from wx station file                             Time  12: 9:24
 reading from precipitation file                          Time  12: 9:24
 reading from temperature file                            Time  12: 9:24
  ...
  Original Simulation            1   1  1975 Yr    1 of    2 Time  12: 9:24
  ...
  Original Simulation           12  31  1976 Yr    2 of    2 Time  12: 9:35

 Execution successfully completed

  Date of Sim  10/20/2026 Time  12: 9:35
```

If the completion message is missing, the run stopped early. The last daily line shows the last day that finished, so the problem occurred on the next day. Messages printed to the screen when the run stopped are not copied to this file.

diagnostics.out

Line 1 is `DIAGNOSTICS.OUT FILE`. Every later line reports a problem found while reading the inputs. About 60 places in the code write to this file. Most lines say that a name given in one file was not found in the file it points to; SWAT+ then uses a default or skips the setting and continues.

Common messages:

Message	Meaning
`<land use> <schedule> not found in management.sch`	a row of `landuse.lum` names a management schedule that is not in `management.sch`
`<name> not found (snow.sno)`, `(soils.sol)`, `(topography.hyd)`, and similar	an HRU or object points to a record that does not exist; the default record is used
`<file> file not found (tgage)`, `(pgage)`, and similar	a weather station names a climate file that is missing
`... could not be opened`	an input file listed in `file.cio` could not be opened
`... is required when carbon is enabled (codes.bsn carbon = 2)`	a carbon input file is missing while the dynamic carbon model is on
`carbon_layers.prt: n_layers must be >= 1 ...`	the optional layer count file has an invalid value

Example from a run of the `Ames_sub1` reference dataset with 62.0.1:

```
 DIAGNOSTICS.OUT FILE
 agrr_lum                          AGRR_rot                          not found in management.
 soyb_lum                          SOYB_rot                          not found in management.
 oats_lum                          OATS_rot                          not found in management.
 wwht_lum                          WWHT_rot                          not found in management.
 ames.tem                                    file not found (tgage)
```

A file with only the first line means no problems were found. Read the whole file after each run and decide for each line whether the default SWAT+ used is acceptable.

checker.out

One row per HRU, in HRU order, written after the HRUs are set up. Line 1 is the project name and version, line 2 the column header and line 3 the units; the last unit entry, `0=notile;1=tile;`, explains the `tiledrain` column.

#	Column	Units	Description
1	`sname`	-	soil name used by the HRU (no header label; the name is in the units line)
2	`hydgrp`	-	hydrologic soil group, A to D (no header label)
3	`zmx`	mm	maximum rooting depth of the soil
4	`usle_k`	-	USLE soil erodibility factor
5	`sumfc`	mm	water held at field capacity, whole profile
6	`sumul`	mm	water held at saturation, whole profile
7	`usle_p`	-	USLE support practice factor
8	`usle_ls`	-	USLE slope length and steepness factor
9	`esco`	-	soil evaporation compensation factor
10	`epco`	-	plant uptake compensation factor
11	`cn3_swf`	-	soil water factor for curve number condition III
12	`perco`	-	percolation coefficient
13	`latq_co`	-	lateral flow coefficient
14	`tiledrain`	0 or 1	1 if the HRU has tile drainage

The file has no HRU number or name column. Row K belongs to HRU K in `hru-data.hru`.

Example from a run of `Ames_sub1` with 62.0.1:

```
 demo                      SWAT+ 2026-10-05        MODULAR Rev 2026.62.0.1
                                        zmx       usle_k       sumfc       sumul      usle_p     usle_ls
 sname           hydgrp                (mm)                    (mm)        (mm)
soil_01-h1       B                2000.0000       0.0000    361.5003    616.0899      1.0000      0.3638
soil_02          B                2000.0000       0.0000    361.5004    449.0375      0.7500      0.3638
```

Look for zero, negative or extreme values: here `usle_k` is zero for both soils, so these HRUs produce no USLE erosion. Check whether that is intended.

files_out.out

Line 1 is `files_out.out - OUTPUT FILES WRITTEN`. Each later line names one output file opened in the run, after a tag for its type, such as `CHK`, `HRU`, `LSU`, `BASIN`, `BASIN_AQUIFER`, `BASIN_CROP_YLD`, `CROP`, `RES` or `DTBL`. `.csv` copies are listed separately.

```
files_out.out - OUTPUT FILES WRITTEN
CHK                       checker.out
HRU                       hru_carb_gl_mon.txt
HRU                       hru_carb_gl_mon.csv
```

Use it to confirm that every file you asked for in `print.prt` was opened. A file missing from the list was not switched on, or its object type is not in the model (for example, reservoir files in a model without reservoirs). Some files are not listed even when they are written, including the gwflow files and `reservoir_mon.csv`.

success.fin

Written at the end of a run that finished. It contains a blank line and `Execution successfully completed`. Calibration tools and scripts test for this file to tell whether a run finished. Delete it before starting a run if you rely on it, because SWAT+ does not remove an old copy at the start.

area_calc.out

Drainage areas of every routed object except HRUs and routing units: the object number, type and number within its type, the drainage area given in the connection file and the area SWAT+ calculated from the routing, followed by the same areas for each incoming object. Use it to check that the routing delivers the expected area to each channel and outlet. The file is empty when the model has only HRUs and routing units.

Other files

`erosion.out` contains three header lines and no data in 62.0.1. `erosion.txt` is opened at the start of each run and left empty.

`paddy_test.csv` is a temporary paddy rice output. It has a header line, and one line per day for each HRU that has a paddy wetland with water: date, HRU, precipitation, irrigation, seepage, PET, soil evaporation, plant transpiration, weir height, water depth, weir outflow, soil water, sediment concentration and yield, nitrate concentration and yield, LAI, salt concentration and soil water as a percentage of field capacity.

Checks after a run

1. `simulation.out` ends with `Execution successfully completed`, and `success.fin` exists.
2. `diagnostics.out` has only its first line, or every remaining line is understood and accepted.
3. `checker.out` has plausible values for every HRU, and `tiledrain` matches the design.
4. `files_out.out` lists every file requested in `print.prt`.

Related files

- Naming convention: file names and suffixes.
- Post-processing helpers: includes a parser for `checker.out`.
- `print.prt`: switches for the other output files.

Sources

- `simulation.out`, `success.fin`, `erosion.txt`, `paddy_test.csv`: `main.f90.in` (units 9003, 107, 888, 100100); weather lines in `proc_date_time.f90`; daily lines in `time_control.f90`; paddy rows in `hru_control.f90`
- `files_out.out`, `diagnostics.out`, `area_calc.out`: opened in `proc_bsn.f90` (units 9000, 9001, 9004); `area_calc.out` rows in `hyd_connect.f90`
- `checker.out`, `erosion.out`: `proc_hru.f90` (units 4000, 4001); header types `output_checker_header` and `output_checker_unit` in `hydrograph_module.f90`

`checker.out` column	Fortran variable
`sname`, `hydgrp`, `zmx`, `usle_k`, `sumfc`, `sumul`	`soil(j)%snam`, `%hydgrp`, `%zmx`, `%usle_k`, `%sumfc`, `%sumul`
`usle_p`, `usle_ls`	`hru(j)%lumv%usle_p`, `hru(j)%lumv%usle_ls`
`esco`, `epco`, `cn3_swf`, `perco`, `latq_co`	`hru(j)%hyd%esco`, `%epco`, `%cn3_swf`, `%perco`, `%latq_co`
`tiledrain`	`hru(j)%tiledrain`

Purpose

The `test` folder of the SWAT+ source has two Python scripts. They are not installed with the model; run them from a copy of the source or copy them next to your project.

- `spcheck.py` runs SWAT+ on a scenario and compares the numbers in two sets of output files. The build uses it for its regression test.
- `swat_output_to_csv_files.py` reads SWAT+ output files into pandas tables and can save them as tab-separated files.

spcheck.py

`spcheck.py` needs Python 3.10 or later and no extra packages. It has three commands.

run

`python spcheck.py run <executable> <scenario>`

Run from a folder that contains `build/`, `data/` and `runs/`. The command copies `data/<scenario>` to a new folder `runs/<scenario>/<NN>#<date and time>#<executable>`, where `NN` is the next two-digit run number (`01`, `02`, …), and runs `build/<executable>` there. If SWAT+ ends with an error, the run folder gets the suffix `-FAILED`.

compare

`python spcheck.py compare <scenario> <a> <b> [--abserr 1e-8] [--relerr 0.05] [--nlines N] [--nerrorlines N`

Compares two earlier runs of a scenario, given by their run numbers `a` and `b`. The files to compare are listed in `data/<scenario>/.testfiles.txt`, one name per line; lines starting with `#` are skipped. A failed run cannot be compared.

ctest

`python spcheck.py ctest <model_path> <data_dir> <tmp_dir> [--abserr 1e-8] [--relerr 0.05] [--nlines N] [--`

Copies the whole folder `<data_dir>`, including its output files, to `<tmp_dir>`, runs `<model_path>` there, and compares the files listed in `<data_dir>/.testfiles.txt` between the original and the new run. It exits with code 1 when it finds differences. Because the original outputs are copied too, a listed file that the new run does not write is compared with the copied original and passes.

Comparison rule

Both files are read line by line. On each line, only tokens that are decimal numbers are compared, by position; whole numbers (dates, object numbers) and text are skipped, and lines containing the version banner `MODULAR` are skipped. Two values f_1 (first run) and f_2 (second run) differ when

$$|f_1 - f_2| \geq e_{abs} + e_{rel}\,|f_2|$$

with e_{abs} = `--abserr` (default 10^{-8}) and e_{rel} = `--relerr` (default 0.05). For each differing line the script prints both lines, the field numbers, the two values and their absolute and relative error, and at the end the total number of differences.

The script assumes both files have the same layout. It stops at the end of the shorter file without reporting a difference in length. A line whose two versions have different numbers of decimal values is not reported, and it lowers the difference count by one. `--nlines` limits the number of lines checked per file, and `--nerrorlines` the number of differing lines reported per file.

Regression test in the build

`CMakeLists.txt` registers one test, `Ames_sub1`, which calls `spcheck.py ctest` with `--relerr 0.01` and `--abserr 1e-8`. The test reads its scenario from `<source>/data/Ames_sub1`; the reference data in the 62.0.1 source is in `refdata/Ames_sub1`. The `.testfiles.txt` of `Ames_sub1` lists 42 files; six of them (`hru_nut_carb_gl_aa.txt`, `hru_soilcarb_aa.txt`, `hru_rescarb_aa.txt`, `hru_plcarb_aa.txt`, `hru_soilc_stat.txt`, `hru_resc_stat.txt`) are no longer written by 62.0.1 and pass only because of the copy described above.

swat_output_to_csv_files.py

The script maps output file names to reading instructions for `pandas.read_csv` and loads each file into a table.

Use

```
cd <folder with SWAT+ outputs>
python /path/to/swat_output_to_csv_files.py
```

The script reads `files_out.out` from the current folder, then reads every file listed there, except `mgt_out.txt` and `yield.out`, and writes each one to `<name>.csv` in the same folder (for example `hru_wb_aa.txt.csv`). The `.csv` files are tab-separated.

From Python, one file can be read without writing anything:

```
import swat_output_to_csv_files as s
df = s.read_output("basin_wb_aa.txt", s.spec_dict, write_csv=False)
```

Supported files

Most files are read by skipping line 1 (title) and line 3 (units) and taking line 2 as the column names. `crop_yld_aa.txt` and `crop_yld_yr.txt` skip two lines. `lu_change_out.txt`, `basin_crop_yld_aa.txt` and `basin_crop_yld_yr.txt` keep line 3 as data. `checker.out` and `files_out.out` are read with fixed column names.

```
files_out.out  checker.out  hru_orgc.txt
hru_wb_aa.txt  hru_ncycle_aa.txt  hru_nb_aa.txt  hru_carbon_aa.txt  hru_nut_carb_gl_aa.txt  hru_ls_aa.txt
basin_wb_aa.txt  basin_nb_aa.txt  basin_carbon_aa.txt  basin_ls_aa.txt  basin_pw_aa.txt
crop_yld_yr.txt  crop_yld_aa.txt  lu_change_out.txt  basin_crop_yld_yr.txt  basin_crop_yld_aa.txt
hydout_aa.txt  hydin_aa.txt  deposition_aa.txt  wetland_aa.txt  basin_aqu_aa.txt  basin_res_aa.txt
recall_aa.txt  basin_cha_aa.txt  basin_sd_cha_aa.txt  basin_sd_chamorph_aa.txt  basin_psc_aa.txt  ru_aa.tx
```

Only average annual files are covered. `hru_orgc.txt` and `hru_nut_carb_gl_aa.txt` are not written by 62.0.1.

Limits with 62.0.1

- The script stops (exit code 1) at the first file listed in `files_out.out` that it has no instructions for, with the message `Swat+ output filename not found in read specification dictionary.` A 62.0.1 run usually lists such files (for example the carbon files, `sd_chanbud_aa.txt` or any daily file), so a full run stops part way. Reading single files with `read_output` works.
- It passes `delim_whitespace=True` to `pandas.read_csv`. pandas 2.2 warns about this option and pandas 3.0 no longer accepts it; with pandas 3.0 every file fails with the message `could not be parsed for some reason`.
- `basin_wb_aa.txt` ends with the two-word label `Original Simulation` and a number, which gives its data row more values than the header has names, so pandas moves the leading columns into the row index.
- `files_out.out` is read as two columns, tag and file name. With `csvout` on, some tags contain a space (`CHANBUD ORDER`), which breaks those lines.

Related files

- Output naming convention: output file names and layout.
- Checker and diagnostics: `files_out.out` and `checker.out`.
- Water balance outputs: the trailing label in `basin_wb_aa.txt`.

Sources

- `test/spcheck.py`
- `test/swat_output_to_csv_files.py`
- Regression test: `CMakeLists.txt` and `refdata/Ames_sub1/.testfiles.txt`
- `files_out.out`: `proc_bsn.f90`; removed carbon files noted in `output_landscape_init.f90`

This section is for people working on the SWAT+ Fortran source: building from a clone, running the test suite, and submitting changes back to `swat-model/swatplus`.

If you only need to run a prebuilt SWAT+ executable, see Install instead.

Pages

- Build with CMake. The build used on every platform: configuration, compilers (gfortran, `ifx`, `ifort`), compiler options, debug and release builds.
- Visual Studio on Windows. Visual Studio 2022 with the Intel oneAPI Fortran compiler: cloning, configuring, and debugging against an input dataset.
- VS Code Codespaces. Running a SWAT+ build and debug session inside a GitHub Codespaces virtual instance. Useful when you do not want to install a Fortran toolchain locally.
- Testing. The scenario regression test that compares model output with reference outputs, and how to run it at 62.0.1.
- Tagging and releases. How the version number comes from the Git tag and reaches the executable, and how a release is made.
- Coding conventions. Fortran 90/95 style rules for SWAT+ source contributions: portability, readability, reliability, arrays, I/O, and obsolescent features to avoid.
- Contributing. Reporting problems, pull requests, and contributing to this documentation.
- Code of conduct. The Contributor Covenant, applied to the SWAT+ project community.

SWAT+ is built out of source: all intermediate and final files go into a build folder, so the source tree stays clean and several builds (different compilers, debug and release) can exist side by side.

The build is defined in `CMakeLists.txt` at the root of the repository. It compiles every `.f90` file in `src/` into one executable.

Requirements

- CMake 3.22 or newer
- a Fortran compiler: gfortran, Intel `ifx`, or Intel `ifort` (`ifort` is no longer included in Intel oneAPI from the 2025 release on)
- Git, which the build uses to find the version number. Configuration stops if Git is not installed; in a copy of the source that is not a Git clone (a downloaded archive), the version becomes `unknown`.
- `make` or Ninja

Configure

From the `swatplus` folder:

```
cmake -B build -D CMAKE_BUILD_TYPE=Release
```

This writes the build system into `build/` without compiling anything. Give a build type: `Release` for an optimized executable, `Debug` for one with run-time checks and debugger symbols. Without a build type neither the optimization nor the debug flags are added.

To choose the compiler:

```
cmake -B build -D CMAKE_BUILD_TYPE=Release -D CMAKE_Fortran_COMPILER=gfortran
cmake -B build -D CMAKE_BUILD_TYPE=Release -D CMAKE_Fortran_COMPILER=ifx
```

On Windows, configure from the Intel oneAPI command prompt to use the Intel compilers, or use Ninja with gfortran:

```
cmake -B build -D CMAKE_GENERATOR_TOOLSET=fortran=ifx
cmake -B build -G "Ninja" -D CMAKE_Fortran_COMPILER=gfortran -D CMAKE_BUILD_TYPE=Release
```

`CMakePresets.json` also defines debug and release presets for gfortran (Linux, Windows, macOS), `ifx` and `ifort`; list them with `cmake --list-presets`.

Compile

```
cmake --build build -j 8
```

The executable is written to `build/` as

`swatplus-<version>-<compiler>-<system>_<arch>[-Rel|-Dbg][.exe]`

for example `swatplus-62.0.1-gnu-lin_x86_64-Rel`. The compiler part is `gnu`, `ifx` or `ifo` (Intel Fortran Classic), and the version comes from the Git tag (see Tagging and releases).

To start again from scratch, delete the build folder (`rm -rf build`, or `rd /s /q build` in Windows `cmd`). To remove only the compiled files and keep the configuration:

```
cmake --build build --target clean
```

Compiler options

Compiler	Always	Release	Debug
gfortran (Linux, macOS)	free form, no line length limit, implicit none, local variables set to zero, strict rounding	`-O`	`-Wall -fcheck=all -ffpe-trap=invalid,zero,overflow,unde -fsignaling-nans`
gfortran (Windows)	as above, plus `-fcheck=all` and the floating-point traps	`-O`	`-Wall`
`ifx`	free form, `-fpe0`, traceback	`-O`	`-warn all -O0 -check bounds`
`ifort`	free form, `-fpe0`, traceback	`-O`	`-warn all`

From 62.0.1 the gfortran run-time checks and floating-point traps are used only in Debug builds on Linux and macOS. Earlier releases also applied them to Release builds, and those executables could stop with an illegal instruction error on macOS when a system math routine raised a harmless floating-point flag.

Linking

- Linux, gfortran: linked statically, except on Arch, Arch-based and Fedora systems, which do not ship the static `libgfortran`; there the executable needs the gfortran run-time libraries at run time (`ldd <executable>` lists them).
- macOS, gfortran: the gfortran, quadmath and gcc run-time libraries are linked statically, so the executable runs on Macs without Homebrew's gcc installed. A fully static macOS executable is not possible.
- Intel compilers and Windows gfortran: linked statically.

Install

`cmake --install build` copies the executable to the root of the source folder, whatever `--prefix` is given. To put it somewhere else, copy the file from `build/` yourself.

Sources

- CMakeLists.txt: version from `git describe --tags` (lines 20 to 34), compiler flags (`fdialect`, `fdebug`, `frelease`) and static linking (lines 36 to 100), executable name (`SWATPLUS_EXE`), `install(TARGETS ${SWATPLUS_EXE} DESTINATION RUNTIME DESTINATION ${PROJECT_SOURCE_DIR})`.
- CMakePresets.json and doc/Building.md.

Visual Studio 2022 on Windows can build, run and debug SWAT+ with the Intel Fortran compiler. The steps are: install the tools, clone the repository, configure the CMake build, and run a debug session against an input dataset.

The screenshots for these steps are in the `doc/` folder of the SWAT+ source repository.

Required software

Install in this order. The Intel compiler requires Visual Studio to be present first.

1. Visual Studio Community 2022. During install, in the **Workloads** panel, check **Desktop development with C++**. That workload pulls in the CMake integration that SWAT+ needs.
2. Intel Fortran Compiler for Windows. Use the standalone installer. The current Intel oneAPI release ships `ifx`. The older `ifort` was deprecated in 2024 and is gone from the 2025 release. SWAT+ produces the same results with either Intel compiler.
3. Git for Windows. Visual Studio includes a copy of Git, but SWAT+ uses the standalone `git` during the CMake build to read the current tag for the version string.

Reboot after the three installs finish.

Optional: install Python (from the Microsoft Store) and the Python workload in Visual Studio if you want to run the integrated test suite from inside the IDE.

Starting Visual Studio

The Intel compiler only works with the Visual Studio CMake integration if Visual Studio is launched from the Intel oneAPI command prompt. Follow these steps every time.

1. Open the Windows Start Menu, type `intel`, and select **Intel oneAPI command prompt for Intel 64 for Visual Studio 2022**. Pin it if you use it often.
2. The terminal opens and prints the oneAPI initialization output:

   ```
   :: initializing oneAPI environment...
      Initializing Visual Studio command-line environment...
      Visual Studio version 17.13.0 environment configured.
      "C:\Program Files\Microsoft Visual Studio\2022\Community\"
      Visual Studio command-line environment initialized for: 'x64'
   :  compiler -- latest
   :  debugger -- latest
   :: oneAPI environment initialized ::
   ```

3. At the prompt, run:

   ```
   devenv
   ```

   This launches Visual Studio with the Intel environment variables set.

If you start Visual Studio by clicking the desktop icon directly, the Intel compiler will not be found and the build will fail.

Cloning the repository

Visual Studio can clone the SWAT+ repository and open it as a CMake project in one step.

1. On the Visual Studio start screen, click **Clone a Repository**.
2. Click the GitHub icon (lower left) to browse your repositories. Sign in if prompted. Select your fork of `swatplus` and click **Open**. Adjust the local path if needed.
3. Visual Studio clones the repository and loads it into Solution Explorer as a CMake project. It runs the CMake configure step against whichever Fortran compiler it finds (`ifx`, `ifort`, or `gfortran`). The 2025 Intel compiler only ships `ifx`, so any `ifort` configurations will fail on that version.

Configuring, compiling, running

Configure

SWAT+ is a CMake project. The Visual Studio toolbar exposes the available compiler and build-type combinations through a drop-down. Switching the selection triggers a reconfigure. The Output window prints the CMake settings used; check there which compiler was picked up.

To force a fresh configure, right-click `CMakeLists.txt` in Solution Explorer and select **Delete Cache and Reconfigure**.

Build

Use **Build > Build All** or **Build > Rebuild All**. The executable name encodes the compiler, architecture, and version. The version is read from the latest Git tag automatically. See Tagging and releases.

The Intel-built Windows executable is dynamically linked. To run it on a machine without Visual Studio, ship the Intel redistributable libraries alongside it.

Run and debug

You need to tell the project which folder contains the input dataset to run against.

1. Right-click `CMakeLists.txt` in Solution Explorer and select **Add Debug Configuration**. If Co-Pilot is hiding Solution Explorer, close that pane first.
2. In the dialog, pick **Default** and click **Select**. Visual Studio creates `launch.vs.json` if it does not exist and opens it for editing.

The default file looks like this:

```
{
  "version": "0.2.1",
  "defaults": {},
  "configurations": [
    {
      "type": "default",
      "project": "CMakeLists.txt",
      "projectTarget": "",
      "name": "CMakeLists.txt"
    }
  ]
}
```

Add a `currentDir` entry pointing at your input dataset. The path can be absolute or relative to the `swatplus` project root. The `name` field is what shows up in the toolbar.

```
{
  "version": "0.2.1",
  "defaults": {},
  "configurations": [
    {
      "type": "default",
      "project": "CMakeLists.txt",
      "projectTarget": "",
      "name": "Ames_testing",
      "currentDir": "C:/work/sp/Ames_testing"
    }
  ]
}
```

Do not change `project` or `type`. You can add more configurations by repeating **Add Debug Configuration** and editing the new block.

Pick your configuration from the toolbar drop-down and run **Debug > Start Debugging** or **Debug > Start Without Debugging**. Breakpoints, conditional watches, and step-through all work.

Resources

- Intel C++ and Fortran Compilers Redistributable Libraries by Version
- Intel Fortran Compiler
- Intel Fortran Compiler for oneAPI release notes
- Visual Studio documentation
- CMake

  Sources

  - `doc/VS-Win.md` and its screenshots in the source repository, with `CMakeSettings.json` for the Visual Studio configurations.

The SWAT+ repository ships a `.devcontainer/` folder that lets GitHub Codespaces start a Visual Studio Code instance in the browser with the build tools installed, so no local Fortran compiler is needed.

When a Codespaces instance starts, it:

- loads your fork into a remote working folder
- installs the Intel `ifx` and `gfortran` compilers
- configures VS Code to use the CMake build system
- installs the VS Code extensions needed to debug Fortran, and the `fortls` Fortran language server
- creates `/workdata/my_data` and copies the `refdata/Ames_sub1` dataset into it

In 62.0.1 the `/workdata/Ames_sub1` folder that the default `Ames_sub1` debug configuration points to is not created. To use that configuration, create the folder and copy `refdata/Ames_sub1` into it from the terminal:

```
mkdir -p workdata/Ames_sub1 && cp -r refdata/Ames_sub1/* workdata/Ames_sub1
```

The screenshots for these steps are in the `doc/` folder of the SWAT+ source repository.

Cost

GitHub gives every account 120 free CPU hours per month across all Codespaces instances. A 2-CPU instance running for 60 hours uses 120 hours of quota. Beyond that, you pay per active hour. A small per-hour storage fee applies whether the instance is running or stopped.

Instance states:

- Active: the instance is in use (interactive or compiling). After 30 minutes of inactivity it goes to Inactive. The timeout is configurable.
- Inactive: not running. Storage fee only.
- Shutdown: you stopped it manually. Same as Inactive billing-wise.
- Deleted: no further charges. A new instance has to be created from scratch to come back.

Starting an instance

1. Sign in to GitHub and go to your `swatplus` fork.
2. Open the hamburger menu (top left) and click **Codespaces**.
3. Click **New codespace** (top right).
4. Pick the repository, the branch, and the CPU count. Two CPUs is enough for SWAT+. Click **Create codespace**.

The instance takes a couple of minutes to come up. You should see an install progress bar in the lower right. When it disappears, a drop-down at the top of the editor asks which Fortran compiler to use. Pick one.

Building

Click the CMake icon in the left sidebar (the triangle-with-wrench) to open the CMake pane.

1. Under **Project Status > Configure**, check the Fortran compiler shown. To change it, hover and click the pencil icon, then pick another.
2. Right-click `CMakeLists.txt` in the CMake pane and select **Clean and Reconfigure**. Do this every time you switch compilers, and any time you add or remove a Fortran source file.
3. Right-click `CMakeLists.txt` again and select **Build All Projects**. The Output panel at the bottom should end with:

   ```
   [build] Build finished with exit code 0
   ```

Running and debugging

Click the debug icon in the left sidebar (the triangle with a bug). Pick `Ames_sub1` from the drop-down and click the green triangle.

After a fresh build, VS Code may ask which executable to run. Any of the debug-build options works. A release build is not debuggable.

Watch model output in the **Terminal** tab at the bottom. Set breakpoints or conditional breakpoints, or let it run. On a runtime exception, the debugger stops at the failure point and you can inspect variable values.

Using your own dataset

The `/workdata/my_data` folder starts with a copy of `Ames_sub1`. To use your own inputs, delete its contents and upload yours:

1. In the VS Code Explorer, right-click `/workdata/my_data` and select **Upload**.
2. Allow the cookie prompt if your browser shows one (GitHub needs it to open your local file picker).
3. Select all the files in your input dataset and upload.
4. Switch the debug drop-down to `my_data` and click the green triangle.

To add a third dataset, create a new folder under `/workdata/`, then edit `.vscode/launch.json`. The default file already has blocks for `Ames_sub1` and `my_data`. Copy one block, change `name` and `cwd`, and add a comma between blocks.

```
{
    "version": "0.2.0",
    "configurations": [
        {
            "name": "Ames_sub1",
            "type": "cppdbg",
            "request": "launch",
            "program": "${command:cmake.launchTargetPath}",
            "args": [],
            "stopAtEntry": false,
            "cwd": "${workspaceFolder}/workdata/Ames_sub1",
            "environment": [
                {
                    "name": "PATH",
                    "value": "${env:PATH}:${command:cmake.getLaunchTargetDirectory}"
                }
            ],
            "externalConsole": false,
            "MIMode": "gdb",
            "setupCommands": [
```

```
                {
                    "description": "Enable pretty-printing for gdb",
                    "text": "-enable-pretty-printing",
                    "ignoreFailures": true
                }
            ]
        },
        {
            "name": "walnut_creek",
            "type": "cppdbg",
            "request": "launch",
            "program": "${command:cmake.launchTargetPath}",
            "args": [],
            "stopAtEntry": false,
            "cwd": "${workspaceFolder}/workdata/walnut_creek",
            "environment": [
                {
                    "name": "PATH",
                    "value": "${env:PATH}:${command:cmake.getLaunchTargetDirectory}"
                }
            ],
            "externalConsole": false,
            "MIMode": "gdb",
            "setupCommands": [
                {
                    "description": "Enable pretty-printing for gdb",
                    "text": "-enable-pretty-printing",
                    "ignoreFailures": true
                }
            ]
        }
    ]
}
```

Stopping the instance

To stop billing for active usage, click **Codespaces** in the bottom-left status bar and select **Stop Current Codespace**. It takes a minute or two.

Deleting the instance

After stopping, go back to the browser tab where you created the instance (or open the Codespaces panel from your GitHub repository). Find the instance at the bottom, click the ellipsis on its row, and select **Delete**. Storage charges stop after deletion. A new instance has to be created from scratch the next time you want to work.

Sources

- `.devcontainer/devcontainer.json` and `.devcontainer/Dockerfile` at 62.0.1. `devcontainer.json` contains two `postCreateCommand` keys; only the last one (the `my_data` copy) is used.
- `doc/VSCode_Codespace.md` in the source repository, with the screenshots.

SWAT+ has a scenario regression test: it runs the model on a reference dataset and compares chosen output files with copies kept in the repository. Differences outside a tolerance point to a change in results, which is then either a bug or an intended change that needs new reference outputs.

Test script

`test/spcheck.py` (Python 3, standard library only) has three commands:

Command	Use
`ctest <executable> <reference folder> <work folder>`	copy the reference folder into the work folder, run the executable there, and compare the files listed in the folder's `.testfiles.txt` with the reference copies; exit with code 1 if any value differs
`run <executable> <scenario>`	run `build/<executable>` on `data/<scenario>` in a new numbered folder under `runs/<scenario>/`
`compare <scenario> <a> <b>`	compare two numbered runs of a scenario

Options `--abserr` and `--relerr` set the tolerances (defaults `1e-8` and `0.05`), and `--nlines` and `--nerrorlines` limit how much of each file is checked and reported.

Two numbers are equal when

$$|v_1 - v_2| \leq e_{abs} + e_{rel}\,|v_2|$$

Only real numbers are compared, value by value on matching lines; integers and text are skipped, as are the title lines with the build date (lines containing `MODULAR`).

Running the test at 62.0.1

`CMakeLists.txt` registers one test, `Ames_sub1`, with tolerances of `1e-8` absolute and 1 % relative, and reference data in `data/Ames_sub1`. At 62.0.1 the repository has no `data/` folder; the reference datasets are in `refdata/`. `ctest` therefore fails at once with `data dir not found`.

To run the comparison, call the script directly with the `refdata` folder:

```
cmake -B build -D CMAKE_BUILD_TYPE=Release
cmake --build build -j 8
python3 test/spcheck.py ctest build/swatplus-*-Rel refdata/Ames_sub1 build/test-runs --abserr
 ↪  1e-8 --relerr 0.01
```

Notes on the reference dataset:

- A full `Ames_sub1` run writes about 9 GB, mostly daily carbon output, and takes a few minutes. Make sure the work folder has the space.
- `.testfiles.txt` lists 42 files. With the shipped `print.prt` the model rewrites fewer than half of them; the rest (all `basin_*` files, `hydin_aa.txt`, `ru_aa.txt` and others) are copied from the reference folder and compared with themselves. The test therefore checks the HRU water and nutrient balances, plant and yield output and management output, and not channel, aquifer or basin output.

Adding a scenario

1. Put a complete input set in a new folder under `refdata/` (or `data/`).
2. Run the model and check the outputs you want to fix as the reference.
3. Commit the inputs and those outputs.
4. Add a `.testfiles.txt` naming the output files to compare, one per line (`#` starts a comment).
5. Add an `add_test` line to `CMakeLists.txt` pointing at the folder.

Continuous integration

`.github/workflows/test.yml` builds SWAT+ with Intel `ifx` on Ubuntu and runs `ctest` on pushes to branches whose name ends in `main`. With the `data/` path above, the test step fails. The release workflow (`build.yml`) builds the release executables for each tag and does not run the tests.

Sources

- test/spcheck.py: `ctest_all`, `cmp_file` and the `run/compare` commands.
- CMakeLists.txt: `set(ref_dir "${PROJECT_SOURCE_DIR}/data")`, tolerances `aerr = 1e-8`, `rerr = 0.01`, and `add_test(Ames_sub1 ...)`. The `Ithaca_sub6` test is commented out.
- refdata/Ames_sub1/.testfiles.txt.
- .github/workflows/test.yml and .github/workflows/build.yml.
- The 9 GB output size and the list of files rewritten are from a run of the 62.0.1 `gnu-lin_x86_64` release executable on 20 October 2026.

A SWAT+ version is a Git tag. No source file is edited to make a release: the build reads the tag and writes it into the executable's name and its start-up banner.

Version numbers

Current tags have three parts, revision, minor and patch, for example `62.0.0` and `62.0.1`. Older tags include `61.0`, `61.0.2.61` and `60.5.7`. List them with:

```
git tag --sort=-creatordate | head
```

At configure time CMake runs `git describe --tags`:

- on a tagged commit the version is the tag, for example `62.0.1`;
- on a later commit it is the tag, the number of commits since, and the abbreviated commit name, for example `62.0.1-3-g1a2b3c4`;
- outside a Git clone it is `unknown`.

A version can also be given by hand with `-D TAG=<version>` when configuring.

Version in the executable and outputs

The executable name, `swatplus-<version>-<compiler>-<system>_<arch>[-Rel|-Dbg]`, for example `swatplus-62.0.1-gnu-l`

The start-up banner, printed to the screen and to `simulation.out`:

```
                  SWAT+
             Revision 62.0.1
      Soil & Water Assessment Tool
GNU (13.3.0), 2026-10-19 22:51:37, Linux
    Program reading . . . executing
```

The title line of every output file, which contains the build date and `Rev <year>.<version>`:

```
 demo                        SWAT+ 2026-10-19        MODULAR Rev 2026.62.0.1
```

Building a tagged version

```
git checkout 62.0.1
cmake -B build -D CMAKE_BUILD_TYPE=Release
cmake --build build -j 8
```

A tag on GitHub can be moved after it is first published (the 62.0.1 tag was replaced on 19 October 2026). Run `git fetch --tags --force` before checking out a tag in an existing clone, so that the local tag matches the one on GitHub.

Making a release

1. Merge the changes for the release into `main`.
2. Tag the merge commit and push the tag:

   ```
   git tag 62.0.2
   git push origin 62.0.2
   ```

3. Pushing a tag of the form `x.y` or `x.y.z` starts the release workflow, which builds the executables for Windows, Linux and macOS with gfortran and the Intel compilers and attaches the zipped executables to a GitHub release.
4. Write the release notes on the GitHub release page, and archive the release on Zenodo (see Citing and license).
5. Tag the documentation repository with the same version, which publishes a matching version of this site.

Sources

- CMakeLists.txt: `if(NOT TAG)` block with `git describe --tags`, `SWATPLUS_EXE`, and `configure_file` of `src/main.f90.in` with `ISO`, `TODAY` and `YEAR` time stamps.
- src/main.f90.in: `prog = " SWAT+ @TODAY@ MODULAR Rev @YEAR@.@SWAT_VERSION@"` and the banner format. The generated `src/main.f90` is overwritten on every configure; edit `main.f90.in`.
- doc/Tagging.md.
- .github/workflows/build.yml: runs on tags matching `*.*` and `*.*.*`.
- Banner and title line: output of the 62.0.1 `gnu-lin_x86_64` release executable.

Conventions for writing SWAT+ Fortran source. The goal is portable, readable and reliable code that compiles cleanly on every supported compiler and operating system.

Each item is tagged with one of three levels:

- Required: mandatory. Pull requests that violate these will not be merged.
- Recommended: strongly preferred. Deviations need justification.
- Encouraged: optional. Apply for consistency where reasonable.

The list below is ordered by decreasing impact on code quality. Most subjective style items (indent width, capitalization) are in the lower tiers.

Interoperability and portability

Required

- Source must conform to the ISO Fortran 95 standard.
- No compiler-specific or platform-specific extensions.
- No use of compiler-dependent error specifier values (`IOSTAT`, `STAT`).
- The source must compile with `gfortran` from GCC, even if the production build uses the Intel compiler.
- `EXIT(N)` is the F90/95 standard exit. Prefer it to `STOP`. `STOP` does not always return an error code. If you need an error code passed back to a script, use `EXIT`, and keep its call sites in one central place.

Recommended

- For floating-point precision and integer size, use the F90/95 `KIND` feature. Do not depend on a vendor-supplied default precision flag.
- Do not use tab characters. Tabs are not part of the Fortran character set and break portability.

Encouraged

- For code that interacts with external frameworks, declare every variable with a `KIND` type to make integration easier.

Readability

Required

- Free-form source.
- Consistent indentation, at least two spaces per level.
- Organize source into modules.
- Do not use Fortran keywords (`DATA`, etc.) as variable names.
- Use meaningful names for variables and parameters. Recognized abbreviations are fine when full names would be unwieldy.
- Every externally-called function or subroutine has a header describing what it does, its arguments, and the author. Small internal routines can use a few descriptive comments instead.
- No magic numbers. Physical constants (pi, gas constants) go in `PARAMETER` statements, not hard-coded into expressions.
- Do not hard-code numeric literals in argument lists. The compiler's default precision for literals is not guaranteed.
- No `GOTO`. If you must use one, document why.

Recommended

- Name your loops and other constructs (subroutines, functions, modules, interfaces) where the name aids readability, especially in nested loops.
- Comment input, output, and local variables. Grouping comments across similar variables is fine when their names speak for themselves.
- Comment to mark major functional sections.
- Do not use Fortran statement names or intrinsic function names as symbols.
- Prefer named parameters over numeric literals: `REAL, PARAMETER :: PI = 3.14159, ONE = 1.0`.

Encouraged

- In new code, follow a consistent style. When editing existing code, match the style around you.
- Indent comments the same way as code.
- Group related procedures and data into modules.
- Keep lines to 132 characters or fewer.
- Use `<, >, <=, >=, ==, /=` instead of `.lt., .gt., .le., .ge., .eq., .ne.`.
- Name modules the same as the file that contains them. Avoid putting multiple modules in one file.
- Use blanks around operators, around `=`, and in declarations to separate syntax elements.
- Always use the `::` notation in declarations, even when there are no attributes.
- Line up attributes, variable names, and trailing comments vertically in declarations.
- Remove unused variables.
- Remove debug code once the bug is fixed.

Reliability

Required

- `IMPLICIT NONE` in every program, module, and procedure.
- Use `PRIVATE` in modules by default, then explicitly mark what should be `PUBLIC`. Exception: modules that exist only to expose public data (constants, for example) can stay fully public.
- Initialize every variable. Do not rely on the machine's default.
- Do not initialize a variable of one type with a value of another type.

Recommended

- Do not use `==` or `/=` on floating-point values. Compare against an explicit tolerance instead (an epsilon check).
- In mixed-type expressions and assignments, write the type conversion explicitly. Do not let the compiler do it implicitly.

- No `INCLUDE` files. Use `USE` with modules.
- Derived types belong in a separate module, together with the procedures that operate on them.
- Each procedure should do one thing.
- Module-level (global) public variables should contain only static or rarely-changing data.

Encouraged

- Always use parentheses to make expression evaluation order explicit.
- Prefer derived types (`%` field access) over loose collections of arrays.

Arrays

Required

- Subscript expressions must be integer.
- Do not assume any particular array passing mechanism when passing arrays as arguments.

Recommended

- Use array operations and intrinsics where they apply.
- Pass arrays as assumed shape.

Encouraged

- Declare `DIMENSION` on every non-scalar.

Dynamic memory and pointers

Required

- Prefer `ALLOCATABLE` arrays over pointers where possible. Reduces leak and fragmentation risk.
- Pointers are allowed when a subroutine needs to return a declared array to its caller.
- Initialize every pointer to `NULL()` in its declaration: `INTEGER, POINTER :: x => NULL()`.
- Prefer automatic arrays for transient allocations. `ALLOCATABLE` and `POINTER` arrays need explicit `DEALLOCATE`.

Recommended

- Always deallocate. Especially inside subroutines and loops.
- Check the `STAT` argument on `ALLOCATE` and `DEALLOCATE`.
- Do not repeatedly `ALLOCATE`, `DEALLOCATE`, then `ALLOCATE` a larger block. Heap fragmentation follows.

Encouraged

- Dynamic allocation is preferred over fixed-size declarations with worst-case dimensions.
- Use automatic arrays in subroutines for simplicity.

Looping

Required

- No `GOTO` for loop exit or continuation. Use `EXIT` or `CYCLE`.

Recommended

- No numbered `DO` loops (`DO 10 ... 10 CONTINUE`).

Functions and procedures

Required

- Avoid `SAVE`. Keep state in module variables instead.
- No `ENTRY` statements in functions.
- Functions must not return pointers.
- Do not name a user function the same as an intrinsic (`SUM`, etc.).
- A procedure that returns a single value should be a function. The value can be a derived type.
- Communicate through arguments or through the variables of the procedure's module.

Recommended

- Every dummy argument that is not a pointer has an `INTENT`.
- Avoid type-specific intrinsics (`AMAX`, `DMAX`). Use the generic (`MAX`).
- Do not declare static-dimensioned array arguments.
- Validate argument values where it matters.

Encouraged

- On an error, print a message that includes the routine name. You may terminate inside the routine, or pass an error flag back through the argument list.
- Use overloading when the same operation works on multiple types.
- Use modules or `CONTAINS` for explicit interfaces.
- Avoid external routines. They need separate interface blocks that drift out of sync.

I/O

Required

- External-file I/O statements include `err=`, `end=`, and `iostat=` as appropriate.
- All global variables, if any, are initialized at startup.

Recommended

- Avoid `NAMELIST` I/O.
- Use `WRITE` instead of `PRINT` for non-terminal I/O.
- Use character parameters or in-line format specifiers in `READ` and `WRITE`. Do not use labeled `FORMAT` statements.

Obsolescent features to avoid

Required

- No common blocks. Use modules.
- No assigned or computed `GO TO`. Use `CASE`.
- No arithmetic `IF`. Use a block `IF`.
- Use `REAL`, not `DOUBLE PRECISION`.
- Avoid `DATA`, `ASSIGN`, labeled `DO`, `BACKSPACE`, blank `COMMON`, `BLOCK DATA`.
- Do not branch to an `END IF` from outside its `IF` block.
- No non-integer `DO` loop counters.
- No Hollerith constants.
- No `PAUSE`.
- No multiple `RETURN`, no alternate `RETURN`.

Recommended

- No `EQUIVALENCE`. Use pointers or derived types.

Encouraged

- Do not implicitly reshape arrays when passing them. This was common in F77 and is forbidden by F90 but still possible through external routines without interface blocks.

Source files

Required

- Document every function interface: argument name, type, units, description, constraints, defaults.
- No `INCLUDE`. Use `USE`.
- Keep lines (including comments) under 80 columns where the language allows it.

Recommended

- Keep individual procedures under 300-500 effective lines.
- Separate statement blocks with blank lines or a marker character in column 1.
- Indent consistently.
- Module and subprogram names are lower case. The file is named after the contained module or subprogram, with `.f90`.

Encouraged

- Separate argument declarations from local declarations visually.
- Use descriptive unique names. Keep them under 12-15 characters.
- Indent continuation lines so a multi-line expression lines up readably.
- Comments start with a marker character (`!`). A standalone comment line starts the marker in column 1.

References

- SAS Coding Conventions
- WRF Coding Conventions
- USGS Coding Standards

  Sources

  - `doc/coding_conventions.md` in the source repository.

Code Of Conduct

The SWAT+ project follows the Contributor Covenant. Everyone taking part in the project, in issues, pull requests, reviews and discussions, is expected to follow it.

The full text, including how to report a violation, is in `CODE_OF_CONDUCT.md` in the source repository.

Sources

- `CODE_OF_CONDUCT.md` and `doc/code_of_conduct.md` at 62.0.1.

Repositories

- Model source: github.com/swat-model/swatplus, with issues, pull requests and releases.
- This documentation: github.com/swat-model/swatplus-documentation.

Questions about using the model are best asked through the SWAT support page.

Reporting a problem

Open an issue with:

- the SWAT+ version (shown in the banner at start-up and in the executable name);
- the operating system and, for your own builds, the compiler;
- a project that shows the problem, as small as possible (one of the `refdata` datasets with the changes that cause it is ideal);
- `simulation.out` and `diagnostics.out` from the run.

Contributing code

1. Fork `swat-model/swatplus` and create a branch from `main` with a descriptive name (`fix-aquifer-init`, `add-soil-temp-output`).
2. Build and run your change on the reference datasets before opening a pull request. See Build with CMake and Testing.
3. Open a pull request against `main`. Describe what the change does and why, which datasets you ran and how their outputs changed, and any change to input or output files.
4. Answer review comments on the same branch.

Reviewers look for:

- a clean build with gfortran and the Intel compilers;
- unchanged outputs for parts of the model the change does not touch;
- defaults for new parameters that keep existing results when the parameter is not set;
- input file changes matched by a SWAT+ Editor change or a note on how to update existing projects;
- code that follows the coding conventions.

Contributing to the documentation

The documentation is a MkDocs site in the `swatplus-documentation` repository. Each page states the SWAT+ commit it was checked against in its front matter (`verified_at_commit`), and every statement about model behaviour should come from the source at that commit. Page bodies are written for model users; source file names and variable names go in the collapsed Sources block at the end of each page (`??? quote "Sources"`).

To check a change locally:

```
pip install -r requirements.txt
mkdocs serve
```

The site is versioned with mike. Pushing a tag to the documentation repository, named after the SWAT+ version the pages describe (for example `62.0.1`), publishes that version and moves the `latest` alias to it. Earlier versions stay available from the version selector.

Sources

- CI: `.github/workflows/test.yml` and `.github/workflows/build.yml` in the source repository.
- Documentation build and deployment: `mkdocs.yml`, `requirements.txt` and `.github/workflows/deploy.yml` in the documentation repository.

SWAT+ can change model parameters from inside the model, using two kinds of calibration data.

Hard data are measured time series at a point, such as daily streamflow at a gauge, sediment or nutrient loads, or groundwater levels. Fitting the model to hard data is done by an external tool (SWAT+ Toolbox, SWAT-CUP, a script) that writes a parameter change list to `calibration.cal`, runs SWAT+, compares the output with the observations and repeats. SWAT+ applies the change list at start-up, before the simulation begins.

Soft data are estimates of individual processes that are not measured at the site, or only as long-term averages: the share of precipitation that leaves as surface runoff or evapotranspiration, the baseflow share of streamflow, or an average county crop yield (Seibert and McDonnell, 2002; Arnold et al., 2015). SWAT+ fits these itself. After the normal run it adjusts a few water balance and plant parameters one at a time, rerunning the simulation after each change, until the average annual values for each land-use group match the targets.

The usual order is soft calibration first, so that each part of the water balance is about right, then hard calibration against the gauge record, which should then need mostly adjustments to peaks and recessions.

Pages

- Hard calibration. How SWAT+ applies a parameter change list, change types and object types.
- Parameter change files. Layout of `cal_parms.cal` and `calibration.cal`, and every parameter name SWAT+ accepts.
- Conditions and regionalization. Applying a change only to objects that match a condition, such as a soil group, land use or calibration group.
- Soft calibration. The `codes.sft` switches, the targets in each `.sft` file, the adjustment steps and the output files.
- Sensitivity workflow. Running a sensitivity analysis with an external sampler that writes `calibration.cal`.

References

Arnold, J. G., Youssef, M. A., Yen, H., White, M. J., Sheshukov, A. Y., Sadeghi, A. M., Moriasi, D. N., Steiner, J. L., Amatya, D. M., Skaggs, R. W., Haney, E. B., Jeong, J., Arabi, M., and Gowda, P. H. (2015). Hydrological processes and model representation: impact of soft data on calibration. Transactions of the ASABE, 58(6), 1637–1660.

Seibert, J., and McDonnell, J. J. (2002). On the dialog between experimentalist and modeler in catchment hydrology: use of soft data for multicriteria model calibration. Water Resources Research, 38(11), 1241.

Sources

- Calibration start-up: `src/proc_cal.f90`, called from `src/main.f90.in` before the simulation.

- Hard calibration changes: src/cal_conditions.f90 and src/cal_parm_select.f90.
- Soft calibration: src/calsoft_control.f90, run from `main.f90.in` after the base simulation when `cal_soft = "y"`.
- Data types: src/calibration_data_module.f90.

Purpose

Soft calibration adjusts a small set of HRU and plant parameters until the simulated average annual water balance and crop yields match targets given for each land-use group. The targets are soft data: regional estimates such as the fraction of precipitation that leaves as surface runoff or evapotranspiration, the share of streamflow that is baseflow, or an average county yield. They are not time series.

Soft calibration runs inside the normal `swatplus` executable. After the base simulation ends, SWAT+ changes one parameter at a time and reruns the whole simulation period after each change. Each rerun is labelled on screen (`first esco adj`, `cn3_swf adj`, `epco adj`, and so on). A water balance calibration adds up to 17 full simulations in mode `a` and 20 in mode `b`, and plant calibration up to 13 more. The calibrated values are written to new files (see Output files); the model inputs are left unchanged.

In 62.0.1 two components work: the HRU water balance (`hyd` switch) and crop yield (`plnt` switch). The other switches are read, but they do not adjust anything. The status column of the switch table gives the detail for each.

Run sequence

1. At start-up SWAT+ reads the seven soft calibration files. A missing file, or a `null` name in file.cio, gives an empty table and that part is skipped. Without `codes.sft` soft calibration is off.
2. Any changes in calibration.cal are applied (see Hard calibration), so soft calibration starts from the changed values.
3. The base simulation runs as usual. At the end of each simulated year every HRU's annual water balance is added to its land-use group, and each harvest adds its yield to the plant targets.
4. If any switch in `codes.sft` is on, soft calibration runs next. It first sets the print.prt codes back to their defaults, which are mostly off, and turns on the average annual basin water balance. The warm-up period (`nyskip`) is kept. Most output files therefore contain the base run, and `basin_wb_aa.txt` gains a record for each calibration run.
5. Each adjustment step changes one parameter for every HRU in each land-use group that is outside its tolerance, restores the starting state of the model, and reruns the full period. The steps are listed under Adjustment steps.
6. The calibrated parameters are written out.

Input files

The `chg` line of file.cio names nine files. The first two are the hard calibration files described in Parameter change files. The last seven belong to soft calibration:

Position on `chg` line	Default name	Earlier name	Content
3	`codes.sft`	`codes.cal`	component switches
4	`wb_parms.sft`	`ls_parms.cal`	limits for the water balance parameters
5	`water_balance.sft`	`ls_regions.cal`	water balance targets by land-use group
6	`ch_sed_budget.sft`	`ch_orders.cal`	channel erosion targets
7	`ch_sed_parms.sft`	`ch_parms.cal`	limits for the channel parameters
8	`plant_parms.sft`	`pl_parms.cal`	plant parameter start values and limits
9	`plant_gro.sft`	`pl_regions.cal`	crop yield targets

A file is only read when its name is on the `chg` line. A project written by SWAT+ Editor 4.0.1 can contain the `.sft` files while their names on the `chg` line are `null`. In that case edits to the files have no effect.

All files are free format. Values are separated by blanks, and names must not contain blanks or slashes. The first line of every file is a title and is skipped.

codes.sft

- Line 1: title.
- Line 2: header.
- Line 3: eight one-letter switches in the order below.

#	Column	Values	Status in 62.0.1
1	`hyd`	`n`, `a`, `b`, `y`	Works. `a` and `b` select the mode; `y` behaves as `b`.
2	`landscape`	`n`, `y`	Not usable. It is meant for simplified (`hru-lte`) HRUs, but its target table is never set up.
3	`plnt`	`n`, `y`	Works, crop yield targets only.
4	`sed`	`n`, `y`	Not usable, see Upland sediment.
5	`nut`	`n`, `y`	Nothing is calibrated. Setting `y` starts the soft calibration runs and changes nothing.
6	`ch_sed`	`n`, `y`	Channel results are summed, but no parameter is adjusted.
7	`ch_nut`	`n`, `y`	Nothing is calibrated.
8	`res`	`n`, `y`	Nothing is calibrated.

Soft calibration runs when `hyd` is anything other than `n` or any other switch is `y`.

Mode `a` calibrates each water balance component against a separate ratio of precipitation. Mode `b` uses only the water yield ratio and the baseflow share (`wyr` and `bfr` in `water_balance.sft`) and derives the other targets from them. The steps for each mode are under Adjustment steps.

Example (SWAT+ Editor 4.0.1 layout with `hyd` set to `b`):

```
codes.sft: written by SWAT+ editor v4.0.1 on 2026-09-13 21:18 for SWAT+ rev. 62
hyd             landscape       plnt            sed             nut             ch_sed          ch_nut          res
b               n               n               n               n               n               n               n
```

water_balance.sft

- Line 1: title.
- Line 2: number of regions.
- Line 3: header.
- For each region: a line with the region name and the number of land-use rows. If that number is above zero, a header line follows, then one target row per land-use group.

Use a single region. The HRU list of a region is filled only when the region is named `basin` or the file has one region, and in both cases the region contains every HRU. A second region gets no HRUs and is not calibrated. HRU areas are taken from `ls_unit.ele` (the `bsn_frac` of element i is used for HRU i).

Target rows:

#	Column	Units	Description
1	`name`	none	Land-use group. Matched to the `cal_group` column of `landuse.lum`. The name `basin` matches every HRU.

#	Column	Units	Description
2	`surq_rto`	fraction of precipitation	Surface runoff / precipitation. Mode `a` target for `cn3_swf`. In mode `b` the value is replaced when the file is read by `wyr` × (1 − `bfr`).
3	`latq_rto`	fraction of precipitation	Lateral flow / precipitation. Mode `a` target for `latq_co`. In mode `b` it is replaced by 0.1 × `wyr` × `bfr`.
4	`perc_rto`	fraction of precipitation	Percolation below the soil profile / precipitation. Mode `a` target for `perco`. In mode `b` it is added to `wyr` to form the `esco` target.
5	`et_rto`	fraction of precipitation	Actual ET / precipitation. Mode `a` target for `esco` and `pet_co`. Not used in mode `b`.
6	`tileq_rto`	fraction	Read; not used as a target.
7	`pet`	mm	Read; not used.
8	`sed`	t/ha	Upland sediment yield target for the `sed` switch.
9	`wyr`	fraction of precipitation	Water yield / precipitation. Mode `b` target for `pet_co` and `esco`.
10	`bfr`	fraction of water yield	Share of water yield that is baseflow. Mode `b` only. Converted when the file is read to baseflow / precipitation (`wyr` × `bfr`), the target for `perco`.
11	`solp`	kg/ha	Read; not used.

The simulated values are built from the HRU annual water balance (the columns of `hru_wb_yr`), weighted by HRU area:

- precipitation: `precip`
- surface runoff: `surq_gen`
- lateral flow: `latq`
- percolation: `perc`
- ET: `et`
- water yield: `surq_gen + latq + perc + qtile`
- baseflow: `latq + perc + qtile`

An HRU is left out of these sums once it has received more than 10 mm of irrigation in total. Its parameters are still changed with the rest of its group. The ratios in a row are not checked to add up to 1.

Example written by SWAT+ Editor 4.0.1 for revision 62. The row contains mode `a` ratios. `wyr` and `bfr` are zero, so in mode `b` every target would be zero and nothing would be calibrated.

```
water_balance.sft: written by SWAT+ editor v4.0.1 on 2026-09-13 21:18 for SWAT+ rev. 62
1
name                  num
basin                   1
    name                 surq_rto      latq_rto      perc_rto        et_rto     tileq_rto           pet
    basin                 0.20000       0.10000       0.10000       0.50000       0.00000       0.90000
```

wb_parms.sft

- Line 1: title.
- Line 2: number of rows.
- Line 3: header.
- Then one row per parameter: `name chg_typ neg pos lo up`.

SWAT+ uses the rows by position and does not look at `name` or `chg_typ`. Keep the ten rows in the order below; renaming or reordering them changes which limits each step uses.

Row	name	Parameter changed	Used by	How it is changed
1	cn2	curve number of simplified HRUs	landscape switch; pos is also used by the sed switch	added
2	esco	esco in hydrology.hyd	modes a and b	added
3	latq_co	latq_co in hydrology.hyd	modes a and b	added
4	petco	pet_co in hydrology.hyd	modes a and b	multiplied
5	slope	HRU slope	sed switch	subtracted
6	tconc	time of concentration	sed switch	divided
7	etco	ET coefficient of simplified HRUs	landscape switch	added
8	perco	perco in hydrology.hyd	modes a and b	added
9	revapc	revap coefficient of simplified HRUs	landscape switch	added
10	cn3_swf	cn3_swf in hydrology.hyd, with the curve numbers recomputed	modes a and b	added

For the HRU water balance rows (2, 3, 4, 8, 10):

- `neg` and `pos` limit the adjustment made for one land-use group. For `petco` they limit the multiplier (Editor default 0.54 to 1.85). A group that reaches a limit is flagged, and some later runs skip it.
- `lo` and `up` bound the HRU parameter value after every change.

Mode `b` narrows the `petco` range for its first PET step. It halves `neg` and `pos` and moves `lo` and `up` halfway toward 1, then restores the full range for the second PET step. See Limits in 62.0.1 for what this does with the Editor defaults.

`cn3_swf` and `perco` are not changed on tile-drained HRUs. At start-up those HRUs are set to `cn3_swf` = 0.95 and `perco` = 0.1.

Example written by SWAT+ Editor 4.0.1 for revision 62:

```
wb_parms.sft: written by SWAT+ editor v4.0.1 on 2026-09-13 21:18 for SWAT+ rev. 62
10
name                      chg_typ          neg          pos           lo           up
cn2                        abschg     -6.00000      6.00000     35.00000     95.00000
esco                       abschg     -1.00000      1.00000      0.00000      1.00000
latq_co                    abschg     -1.00000      1.00000      0.00000      1.00000
petco                      abschg      0.54000      1.85000      0.70000      1.20000
slope                      pctchg    -25.00000     25.00000      0.00010      0.90000
tconc                      pctchg    -30.00000     30.00000      5.00000    960.00000
etco                       abschg     -0.40000      0.40000      0.80000      1.20000
perco                      abschg     -0.70000      0.70000      0.00100      1.00000
revapc                     abschg     -0.40000      0.40000      0.00000      0.40000
cn3_swf                    abschg     -1.20000      1.20000      0.00000      1.00000
```

plant_gro.sft

- Line 1: title.
- Line 2: number of regions.
- Line 3: header.
- For each region:
  - a line with `name`, the number of plants (`lum_num`), `nspu`, and then `nspu` HRU numbers;
  - if `lum_num` is above zero, a header line and one target row per plant.

With `nspu` = 0 the region covers every HRU. Otherwise `nspu` numbers follow, and a negative number closes a range: with `nspu` = 3, the list `1 -20 35` means HRUs 1 to 20 and HRU 35. Each HRU belongs to one plant region. `plant_parms.sft` is read second, so its region assignment is the one that applies.

Target rows:

#	Column	Units	Description
1	`name`	none	Plant name. Matched to the plant named in the harvest operation (`harv` or `hvkl` in a management schedule, `harvest` or `harvest_kill` in a decision table).
2	`yield`	t/ha	Target yield. Compared with the total harvested yield divided by the total harvested area over the whole run.
3	`npp`	t/ha	Read; not used.
4	`lai_mx`	none	Read; not used.
5	`wstress`	none	Read; not used.
6	`astress`	none	Read; not used.
7	`tstress`	none	Read; not used.

plant_parms.sft

- Line 1: title.
- Line 2: number of regions. Region i here is paired with region i of `plant_gro.sft`.
- Line 3: header.
- For each region:
  - a line with `name`, `lum_num`, `parms` and `nspu`;
  - if `lum_num` is above zero, a header line and `lum_num` × `parms` parameter rows.

Set `nspu` to 0 here, which puts every HRU in the region. With `nspu` above zero the region line is read a second time without the `parms` field, so the HRU list is misread.

Set `parms` to 4. The rows form four blocks of `lum_num` rows each, in this order: `pest_stress`, `epco`, `lai_pot`, `harv_idx`. Within every block the plants must be in the same order as in the matching `plant_gro.sft` region. SWAT+ finds a row by its position in the block and always runs all four steps.

Parameter rows:

#	Column	Description
1	`var`	`pest_stress`, `epco`, `lai_pot` or `harv_idx`
2	`name`	plant name
3	`init_val`	start value, applied when the file is read to every plant of that name in the region's HRUs. This happens whether or not `plnt` is on.
4	`chg_typ`	read; not used
5	`neg`	lower limit of the adjustment. For `epco` it is the lowest `epco` value the calibration may set.
6	`pos`	upper limit of the adjustment. For `epco` it is the highest `epco` value the calibration may set.
7	`lo`	lower bound on the plant's value
8	`up`	upper bound on the plant's value

ch_sed_budget.sft and ch_sed_parms.sft

These two files are read, but no channel parameter is adjusted.

`ch_sed_budget.sft` layout:

- Line 1: title.
- Line 2: number of regions.
- Line 3: header.
- For each region: `name`, the number of target rows (`ord_num`), `nspu` and `nspu` channel numbers (0 means all channels). A header line always follows, then `ord_num` rows of `name chw chd hc fpd`.

The row fields are channel widening (mm/yr), downcutting or accretion (mm/yr), headcut advance (m/yr) and floodplain accretion (mm/yr).

`ch_sed_parms.sft` has the same layout as `wb_parms.sft`. Only `name` and `chg_typ` of its first row are used, and only when writing the channel summary.

The channel results are only summed when `ch_sed` is the only switch on, and only target rows named `basin` collect them.

Adjustment steps

In every step:

- The tolerance is the relative difference |target − simulated| / target, taken over the average annual values of the land-use group.
- A step only reruns the simulation if at least one group is outside tolerance.
- The first run of a step moves the parameter by a fixed rate. Later runs use a secant estimate from the last two runs.

Water balance, mode a

Step	Parameter	Target compared	Tolerance	Runs	First-run change	`wb_parms.sft` row
1	`esco`	`et_rto` × precipitation vs simulated ET	1 %	1 + 2	+ (target − simulated, mm) / 500	2
2	`pet_co`	same as step 1	1 %	2	multiplied by a ratio near 1	4
3	`cn3_swf`	`surq_rto` × precipitation vs `surq_gen`	2 %	1 + 2	− (target − simulated, mm) / 100	10
4	`latq_co`	`latq_rto` × precipitation vs `latq`	10 % first run, 5 % after	1 + 1	+ 0.0025 × (target − simulated, mm)	3
5	`perco`	`perc_rto` × precipitation vs `perc`	5 %	1 + 3	+ 0.001 × (target − simulated, mm)	8
6	`cn3_swf`	as step 3	2 %	1 + 2	as step 3	10

For example, with `surq_rto` = 0.2, 800 mm of precipitation and 120 mm of simulated surface runoff, the target is 160 mm, the difference is 40 mm, and the first `cn3_swf` run lowers `cn3_swf` by 0.4.

Water balance, mode b

Step	Parameter	Target compared	Tolerance	Runs
1	`pet_co`, half range	`wyr` × precipitation vs simulated water yield	1 %	2
2	`esco`	(`wyr` + `perc_rto`) × precipitation vs simulated water yield	1 %	1 + 2
3	`pet_co`, full range	as step 1	1 %	2
4	`cn3_swf`	`wyr` × (1 − `bfr`) × precipitation vs `surq_gen`	2 %	1 + 2
5	`latq_co`	0.1 × `wyr` × `bfr` × precipitation vs `latq`	10 % first run, 5 % after	1 + 2
6	`perco`	`wyr` × `bfr` × precipitation vs `latq` + `perc` + `qtile`	5 %	1 + 3
7	`cn3_swf`	as step 4	2 %	1 + 2

Crop yield

Each plant's average harvested yield is compared with its `plant_gro.sft` target:

1. Aeration stress. If the yield is more than 20 % below target, HRUs growing that plant that have no tile drain and an accumulated aeration stress above 50 get `perco` = 0.95. One run.
2. `epco`, with a 3 % tolerance. The first run sets `epco` to 1, or to a low value when the yield is more than 10 % above target. Four more runs narrow the interval between 0 and 1. The value is kept within `neg` and `pos` of the `epco` block.
3. `pest_stress`, changed by the relative yield error. One run.
4. `lai_pot`, 1 LAI unit for every 20 % of yield difference, then secant. 1 + 2 runs.
5. `harv_idx`, half the relative yield difference, then secant. 1 + 2 runs.

Upland sediment

The `sed` switch is meant to fit the `sed` targets by shortening the time of concentration and lowering the HRU slope, with the USLE slope-length factor recomputed. In 62.0.1 it does not work:

- Its loops run over the number of channel regions instead of the number of water balance regions.
- Simulated sediment is only summed when `hyd` is also on.
- The secant runs always compute a change of zero.

Treat the `sed` switch as unavailable.

Output files

File	Written when	Content
`hydrology-cal.hyd`	`hyd` or `plnt` on	One row per HRU with the calibrated `hydrology.hyd` values, written at the end.
`plant_parms.cal`	`plnt` on	Same layout as `plant_parms.sft`, with `init_val` updated to the calibrated values and `nspu` set to 0.
`hru-out.cal`	any switch on	Header only, except with the `landscape` switch, which appends the simplified HRU parameters.
`hru-new.cal`	`hyd` on	Opened but left empty.

To use the results in later runs, point file.cio at the new files:

- Use plant_parms.cal as plant_parms.sft. Its init_val values are applied when the file is read, also with plnt = n.
- Use hydrology-cal.hyd as hydrology.hyd, after the edits in the warning below. Rows are written per HRU under the name of its hydrology.hyd record, so every HRU needs a separate record. The extra 16th column (perco_lim, labelled NOT_USED) is ignored when the file is read.

  Converting hydrology-cal.hyd back to an input

  Two edits are needed before hydrology-cal.hyd can replace hydrology.hyd:

  - LAT_TTIME contains the value after the start-up conversion, 1 − exp(−1 / days). Read back as an input it is taken as days and converted a second time, which makes lateral flow drain much faster. Put the original lat_ttime values back before using the file.
  - When hyd and plnt are both on, the file contains the title, header and HRU rows twice. Keep only the second set, which is written after plant calibration.

Limits in 62.0.1

- Only one water balance region works (see water_balance.sft).
- wb_parms.sft and plant_parms.sft rows are found by position, so the row order is part of the format.
- In mode b the first PET step halves neg and pos for petco. With the Editor defaults (0.54, 1.85) the upper limit becomes 0.925. When simulated water yield is too high the step should raise PET; it caps the multiplier at 0.925 and lowers PET instead.
- plant_parms.sft must use nspu = 0 and parms = 4.
- The landscape, sed, nut, ch_sed, ch_nut and res switches do not calibrate anything (see codes.sft).
- Annual sums include the warm-up years, but the averages divide by the printed years only. Target ratios are unaffected because precipitation is treated the same way. The fixed first-run rates, which are in mm, act on values scaled by the same factor.

Related pages

- Hard calibration. Changes from calibration.cal, applied before the base run.
- Parameter change files. cal_parms.cal and calibration.cal, the first two files on the chg line.
- landuse.lum. The cal_group column that water_balance.sft rows are matched against.
- hydrology.hyd. The parameters written to hydrology-cal.hyd.

  Sources

  - Readers, called from src/proc_cal.f90 in this order: pl_read_regions_cal.f90 (plant_gro.sft), pl_read_parms_cal.f90 (plant_parms.sft), calsoft_read_codes.f90 (codes.sft), lcu_read_softcal.f90 (water_balance.sft), ls_read_parms_cal.f90 (subroutine ls_read_lsparms_cal, wb_parms.sft), ch_read_orders_cal.f90 (ch_sed_budget.sft), ch_read_parms_cal.f90 (ch_sed_parms.sft).
  - Control: src/calsoft_control.f90, called from src/main.f90.in after the base simulation when cal_soft = "y".
  - Water balance: src/calsoft_hyd.f90 (mode a); src/calsoft_hyd_bfr.f90 and its steps calsoft_hyd_bfr_pet, _et, _surq, _latq, _perc (mode b).
  - Crop yield: src/calsoft_plant.f90, calsoft_plant_zero.f90, pl_write_parms_cal.f90. Yields are added to the targets at harvest in mgt_sched.f90 and actions.f90.
  - Upland sediment: src/calsoft_sed.f90. Simplified HRU water balance: caltsoft_hyd.f90.
  - Annual sums and averages: calsoft_sum_output.f90 (end of every simulated year, called from time_control.f90) and calsoft_ave_output.f90 (end of every run).
  - Tile-drained HRU settings: topohyd_init.f90. lat_ttime conversion: hydro_init.f90. Output files are opened in header_write.f90.
  - Types: src/calibration_data_module.f90. File names: type input_chg in src/input_file_module.f90.

File and column	Fortran variable
codes.sft hyd, landscape, plnt, sed, nut, ch_sed, ch_nut, res	cal_codes%hyd_hru, %hyd_hrul, %plt, %sed, %nut, %chsed, %chnut, %res
water_balance.sft target row	type soft_calib_ls_processes: name, srr, lfr, pcr, etr, tfr, pet, sed, wyr, bfr, solp
wb_parms.sft rows	ls_prms(1:10)
plant_gro.sft target row	type soft_calib_pl_processes
plant_parms.sft parameter row	type pl_parms_cal; row k of block b is prm(k + (b-1)*lum_num)
HRU land-use group	hru%lum_group_c, from landuse.lum cal_group
accumulated aeration stress	strsa

Output file	Unit
hydrology-cal.hyd	5001
plant_parms.cal	107
hru-out.cal	4999
hru-new.cal	5000

Purpose

Hard calibration fits the model to measured time series such as streamflow, sediment or nutrient loads at a gauge. SWAT+ does not search for parameter values itself. An external tool (SWAT+ Toolbox, SWAT-CUP, a script) picks a set of parameter changes, writes them to calibration.cal, runs SWAT+, compares the output with the observations, and repeats. Each run of SWAT+ applies the changes in calibration.cal once, at start-up, and then simulates the full period with the changed parameters.

The input files are never edited. A change exists only for the run that reads it, so the same TxtInOut folder can be reused for every trial.

Files

Two files, both named on the chg line of file.cio, drive hard calibration:

- cal_parms.cal lists every parameter that may be changed, the object type it belongs to, and the lowest and highest value it may take. SWAT+ Editor writes this file and it rarely needs editing.
- calibration.cal is the change list for one run. Each row names a parameter from cal_parms.cal, the type and size of the change, and optionally the objects to change and the conditions they must meet.

If either slot is null or the file is missing, no changes are applied. The column layouts and the full list of parameter names are on Parameter change files.

Run sequence

1. SWAT+ reads all input files and sets up every object (HRUs, soils, plants, channels, aquifers, reservoirs).
2. It reads cal_parms.cal and calibration.cal.
3. For each row of calibration.cal, in file order, it goes through the listed objects (all objects of that type when none are listed), checks the row's conditions for each object, and changes the parameter on every object that meets them.
4. The simulation runs once with the changed parameters.

Because the changes are applied after the inputs are read and before the first day is simulated, they act for the whole simulation period. The year and day columns of calibration.cal only limit changes to the precipitation and temperature records (see Climate changes).

Rows are applied in order, and a later row for the same parameter acts on the value left by the earlier row. Two pctchg rows of 10 for cn2 give a 21 % increase.

Change types

Type	New value	Example (current value 70, change 5)
`absval`	the change value replaces the current value	5
`abschg`	current value + change	75
`pctchg`	current value × (1 + change / 100)	73.5
`relchg`	current value × (1 + change)	420

After every change the result is limited to the range `abs_min` to `abs_max` given for that parameter in `cal_parms.cal`. Use one of the four types; any other text in the change type column gives an undefined value.

Object types

The `obj_typ` column of `cal_parms.cal` tells SWAT+ which objects a parameter belongs to. When a row of `calibration.cal` lists no objects, every object of that type is changed.

Type	Objects changed when none are listed	Example parameters
`hru`	every HRU	`cn2`, `esco`, `epco`, `perco`, `lat_ttime`, `canmx`
`sol`	every HRU, in the soil layers from `lyr1` to `lyr2`	`awc`, `k`, `bd`, `usle_k`
`plt`	see Plant parameters	`lai_pot`, `harv_idx`, `phu_mat`
`hlt`	every HRU of the simplified `hru-lte` type	`cn2_lte`, `awc_lte`
`aqu`	every aquifer	`alpha`, `revap_co`, `flo_min`
`sdc`	every channel	`chn`, `chk`, `bank_exp`
`swq`	every channel nutrient parameter set	`rs1`, `bc1`, `mumax`
`res`	every reservoir	`k_res`, `evrsv`, `nsed`
`rdt`	every reservoir decision table	`drawdown_days`, `withdraw_rate`
`bsn`	the basin (applied once)	`surlag`, `cdn`, `nperco`
`gwf`	every gwflow cell	`aquifer_K`, `aquifer_Sy`
`gwf_riv`	every channel connected to gwflow	`stream_K`, `stream_thk`
`gwf_sgl`	applied once	`stream_bed`, `reservoir_K`
`cli`	see Climate changes	`precip`, `temp`

Soil layers

For `sol` parameters, `lyr1` and `lyr2` in `calibration.cal` set the first and last soil layer to change. A `lyr1` of 0 starts at the top layer, and a `lyr2` of 0 or more than the number of layers ends at the bottom layer.

Reservoir decision tables

For `rdt` parameters (`drawdown_days`, `withdraw_rate`) the five integer columns after `conds` select which storage zones of the decision table to change. A non-zero value in the column switches that zone on:

Column	Zone
`lyr1`	exclusive flood control storage
`lyr2`	seasonal flood control storage, non-flood season
`year1`	seasonal flood control storage, flood season
`year2`	multiple use storage, non-flood season
`day1`	multiple use storage, second zone

Plant parameters

Plant parameters with object type `plt` are changed in the plant community of each listed HRU, for the plants named in the row's conditions. Only `phu_mat`, `lai_pot` and `harv_idx` are handled this way. Each row needs a condition that names the plant (see Conditions and regionalization); a row without one changes nothing.

Give the HRUs to change in the object list of the row. When the list is empty, SWAT+ goes through as many HRUs as there are plants in plants.plt, starting from HRU 1.

To change epco use object type hru. That changes the HRU value and sets the same value for every plant in the HRU's community. With object type plt, an epco row changes lai_pot instead.

The plant carbon parameters bio_e, hvsti, rdmx, t_opt and t_base are listed with object type plt in the SWAT+ Editor cal_parms.cal. In 62.0.1 rows for these parameters change nothing.

Climate changes

Precipitation and temperature records can be changed over a date window with the parameters precip and temp, giving object type cli in cal_parms.cal. The change is applied to every daily value from day day1 to day day2 of each year from year1 to year2 (calendar years, inside the simulation period).

List the records to change in the object list of the row. With no list, nothing is changed. List one record per row. When a row lists several records, the change is applied to every listed record once for each record in the list, so a pctchg of 10 on three records gives a 33 % increase on each. In 62.0.1 temp rows do not change the listed temperature record.

Unknown names

A row whose name is not in cal_parms.cal, or whose name SWAT+ does not recognise as a parameter, is skipped without a message. Check spelling against the parameter list if a change seems to have no effect.

Related pages

- Parameter change files. Column layout of cal_parms.cal and calibration.cal, and the parameter names.
- Conditions and regionalization. Limiting a change to objects that meet a condition.
- Sensitivity workflow. Driving many runs from an external sampler.
- Soft calibration. Fitting the average annual water balance and crop yields before hard calibration.

Sources

- Readers: src/cal_parm_read.f90 (cal_parms.cal) and src/cal_parmchg_read.f90 (calibration.cal), called from src/proc_cal.f90.
- Applying changes: src/cal_conditions.f90, which calls src/cal_parm_select.f90 for each object. The change operator is src/chg_par.f90.
- Order of calls: src/main.f90.in. proc_cal runs after proc_hru, proc_cha, proc_aqu and the reservoir and wetland setup, and before time_control.
- Types: type calibration_parameters and type update_parameters in src/calibration_data_module.f90.

Purpose

cal_parms.cal is the list of parameters that calibration may change: the name of each parameter, the type of object it belongs to, and the lowest and highest value it may take. calibration.cal is the change list for one run: each row picks a parameter from cal_parms.cal, says how to change it, and optionally limits the change to some objects or to objects that meet conditions. How SWAT+ applies the list is described on Hard calibration.

Both files are named on the chg line of file.cio, in the first two positions (defaults cal_parms.cal and calibration.cal). A null name or a missing file switches the change list off.

All values are free format and separated by blanks. Names must not contain blanks.

cal_parms.cal

- Line 1: title (skipped).
- Line 2: number of parameter rows.

- Line 3: column header (skipped).
- Then one row per parameter.

#	Column	Description
1	`name`	Parameter name, as listed under Parameter names.
2	`obj_typ`	Type of object the parameter belongs to: `hru`, `sol`, `plt`, `hlt`, `aqu`, `sdc`, `rte`, `swq`, `res`, `rdt`, `bsn`, `pst`, `gwf`, `gwf_riv`, `gwf_sgl` or `cli`. It decides which objects are changed when a row of `calibration.cal` lists none.
3	`abs_min`	Lowest value the parameter may take. Every change is limited to this value.
4	`abs_max`	Highest value the parameter may take. Every change is limited to this value.
5	`units`	Units, for information only.

SWAT+ Editor writes this file with suitable limits for each parameter, and it rarely needs editing. A narrower range for a calibration run belongs in the calibration tool; the range here is the physical limit.

Example from `refdata/Osu_1hru` in the SWAT+ repository:

```
cal_parms.cal: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
186
name                      obj_typ       abs_min       abs_max            units
cn2                           hru      35.00000      95.00000             null
cn3_swf                       hru       0.00000       1.00000             null
usle_p                        hru       0.00000       1.00000             null
ovn                           hru       0.01000      30.00000             null
elev                          hru       0.00000    5000.00000                m
slope                         hru       0.00010       0.90000              m/m
slope_len                     hru      10.00000     150.00000                m
lat_ttime                     hru       0.50000     180.00000             days
```

The count on line 2 must match the number of rows. Rows past the count are not read.

calibration.cal

- Line 1: title (skipped).
- Line 2: number of change rows.
- Line 3: column header (skipped).
- Then one row per change, each followed directly by its condition lines, if it has any.

Change rows

#	Column	Description
1	`name`	Parameter name. Must also be in `cal_parms.cal`.
2	`chg_typ`	`absval` (replace), `abschg` (add), `pctchg` (percent change) or `relchg` (relative change). See Change types.
3	`val`	Size of the change.
4	`conds`	Number of condition lines that follow this row. 0 for none.
5	`lyr1`	First soil layer to change, for `sol` parameters (0 = top layer). For reservoir decision tables, a zone switch.
6	`lyr2`	Last soil layer to change, for `sol` parameters (0 = bottom layer). For reservoir decision tables, a zone switch.
7	`year1`	First year of the window, for `precip` and `temp`. For reservoir decision tables, a zone switch.
8	`year2`	Last year of the window, for `precip` and `temp`. For reservoir decision tables, a zone switch.
9	`day1`	First day of the year in the window, for `precip` and `temp`. For reservoir decision tables, a zone switch.

#	Column	Description
10	`day2`	Last day of the year in the window, for `precip` and `temp`.
11	`obj_tot`	Number of entries in the object list that follows on the same line. 0 means every object of the parameter's type.
12…	object list	`obj_tot` object numbers.

The object numbers are the sequence numbers of the objects (the first HRU is 1, and so on). A negative number closes a range started by the number before it, so `obj_tot` = 3 with the list `1 -20 35` means objects 1 to 20 and object 35.

Condition lines

Each change row is followed by `conds` condition lines. A condition line has one of two forms:

- `range <var> <min> <max>`: the object's value of `<var>` must lie between `<min>` and `<max>`. Only `res_pvol` is recognised.
- `<var> <alt> <targ> <targc>`: a test against the text value `<targc>`. `<alt>` and `<targ>` must be present but are not used by the conditions that exist.

The variables and how they are tested are on Conditions and regionalization.

Example

`refdata/Osu_1hru/calibration.cal`:

```
  hru-new.cal developed from soft data calibration
    13
 NAME           CHG_TYP                  VAL   CONDS  LYR1  LYR2   YEAR1  YEAR2   DAY1   DAY2  OBJ_TOT
cn2         pctchg                 7.714000E+00  0          0          0          0          0          0
petco       pctchg                -5.273950E+01  0          0          0          0          0          0
latq_co     pctchg                -3.732030E+01  0          0          0          0          0          0
alpha       pctchg                 6.859000E+00  0          0          0          0          0          0
epco        pctchg                 6.534100E+01  0          0          0          0          0          0
awc         pctchg                -6.256550E+01  0          0          0          0          0          0
esco        pctchg                 3.305500E+01  0          0          0          0          0          0
perco       pctchg                 1.434000E+00  0          0          0          0          0          0
cn3_swf     pctchg                -7.710000E-02  0          0          0          0          0          0
canmx       pctchg                -6.680230E+01  0          0          0          0          0          0
lat_len     pctchg                -6.241200E+00  0          0          0          0          0          0
revap_co    pctchg                -5.854530E+01  0          0          0          0          0          0
surlag      pctchg                -2.125720E+01  0          0          0          0          0          0
```

Every row has no conditions and an empty object list, so each change applies to all objects of its type. The rows have one more zero than the header has columns; the reader stops after `obj_tot` when it is 0, so the extra value is ignored.

A row that changes `cn2` by +5 only on HRUs 1 to 20 and HRU 35, and only where the hydrologic soil group is `C`:

```
cn2         abschg      5.0     1   0    0    0    0    0    0    3    1  -20    35
hsg         =           0       C
```

Parameter names

The names SWAT+ 62.0.1 recognises are listed below with the object type to give in `cal_parms.cal` and the input the parameter comes from. A name that is in neither list is skipped without a message. Some recognised names have no effect at 62.0.1, or change a different value; the notes with each name say which.

HRU parameters (`hru`)

Name	Input	Notes
`cn2`	condition II curve number from `cntable.lum`	The curve numbers for the other moisture conditions are recomputed.
`cn3_swf`	`cn3_swf`, `hydrology.hyd`	Not changed on tile-drained HRUs.
`perco`	`perco`, `hydrology.hyd`	Not changed on tile-drained HRUs.
`esco`, `epco`	`esco`, `epco`, `hydrology.hyd`	`epco` is also set for every plant in the HRU.
`petco`	`pet_co`, `hydrology.hyd`	
`latq_co`, `lat_sed`	`latq_co`, `lat_sed`, `hydrology.hyd`	
`lat_ttime`	`lat_ttime`, `hydrology.hyd`	Use `absval` with a value in days. `abschg`, `pctchg` and `relchg` act on an internal coefficient and give wrong travel times.
`canmx`	`can_max`, `hydrology.hyd`	
`biomix`	`bio_mix`, `hydrology.hyd`	
`erorgn`, `erorgp`	`orgn_enrich`, `orgp_enrich`, `hydrology.hyd`	
`lat_orgn`, `lat_orgp`	`lat_orgn`, `lat_orgp`, `hydrology.hyd`	
`slope`, `slope_len`, `lat_len`, `dis_stream`	`slp`, `slp_len`, `lat_len`, `dist_cha`, `topography.hyd`	
`elev`	HRU elevation	
`ovn`	Manning's n for overland flow, `ovn_table.lum`	
`usle_p`	USLE support practice factor, `cons_practice.lum`	
`field_len`, `field_wid`, `field_ang`	`len`, `wd`, `ang`, `field.fld`	
`snofall_tmp`, `snomelt_tmp`, `snomelt_max`, `snomelt_min`, `snomelt_lag`	`fall_tmp`, `melt_tmp`, `melt_max`, `melt_min`, `tmp_lag`, `snow.sno`	
`tile_dep`, `tile_dtime`, `tile_lag`, `tile_rad`, `tile_dist`, `tile_drain`, `tile_pump`	`dp`, `t_fc`, `lag`, `rad`, `dist`, `drain`, `pump`, `tiledrain.str`	
`tile_latk`	`lat_ksat`, `tiledrain.str`	The result is written to the tile lag time (`lag`).
`cmn_h`, `nperco_h`, `pperco_h`, `phoskd_h`, `psp_h`, `nperco_lchtile_h`	HRU values of the basin nutrient parameters `cmn`, `nperco`, `pperco`, `phoskd`, `psp` and `nperco_lchtile`	Change one HRU's copy. The basin names below change the basin value.

Soil layer parameters (`sol`)

Changed in soil layers `lyr1` to `lyr2` of each HRU.

Name	Input	Notes
`z`	layer depth `dp`, `soils.sol`	
`bd`, `awc`, `k`	`bd`, `awc`, `soil_k`, `soils.sol`	Dependent water contents are recomputed after `awc` changes.
`clay`, `silt`, `sand`, `rock`	`clay`, `silt`, `sand`, `rock`, `soils.sol`	
`alb`, `usle_k`, `ec`, `cal`, `ph`	`alb`, `usle_k`, `ec`, `caco3`, `ph`, `soils.sol`	
`anion_excl`, `crk`	`anion_excl`, `perc_crk`, `soils.sol`	Whole profile values.
`cbn`	`carbon`, `soils.sol`	No effect: the soil carbon pools are set up after the change list is applied.
`lab_p`, `hum_c_n`, `hum_c_p`	`lab_p`, `hum_c_n`, `hum_c_p`, `nutrients.sol`	No effect, for the same reason.

Plant parameters (`plt`)

Changed for the plants named in the row's `plant` conditions, in the plant community of each listed HRU. See Plant parameters for how to write these rows.

Name	Input
`lai_pot`	`lai_pot`, `plants.plt`

Name	Input
harv_idx	harv_idx, plants.plt
phu_mat	heat units to maturity of the plant in the HRU's community, computed at start-up from the plant and the climate

bio_e, hvsti, rdmx, t_opt, t_base and usle_c are recognised names, but rows for them change nothing.

Basin parameters (bsn)

Changed once for the whole basin.

Name	Input
surlag, prf, evrch, percop, cdn, sdnco, tb_adj, n_updis, rsd_covco	parameters.bsn
init_seq, init_microb, init_slow, init_passive, koc_c, solc_ratio, manure_c_frac, bio_consol, till_consol, sfc_rsd_photodeg, n_act_frac, cnr_cap, cnr_ref, cpr_cap, cpr_ref, t_cbn_min, t_cbn_opt, t_cbn_max	carbon.bsn
bmix_a, bmix_b, bmix_c, tillmix_a, tillmix_b, tillmix_c	biological and tillage mixing curve coefficients, carbon.bsn

Basin rows for adj_pkr, ffcb, dorm_hr, cmn, nperco, pperco, phoskd, psp, nperco_lchtile, rsdco, msk_co1, msk_co2, msk_x, plaps, tlaps and evlai change nothing. SWAT+ copies these basin values to the HRUs, channels, plants and weather stations while it sets them up, before the change list is applied, and does not read the basin value again. For the nutrient parameters, use the HRU names (cmn_h, nperco_h, pperco_h, phoskd_h, psp_h, nperco_lchtile_h) instead.

The per-layer carbon rates of carbon_lyr.bsn (hp_rate, hs_rate, microb_rate, meta_rate, str_rate, microb_top_rate, hs_hp, a1co2, asco2, apco2, abco2) are recognised names, but rows for them change nothing.

Aquifer parameters (aqu)

Name	Input
flo_init_mm, dep_bot, dep_wt_init, no3_init, minp_init, cbn_init, flo_dist, bf_max, alpha, revap_co, deep_seep, sp_yld, hlife_n, flo_min, revap_min	gw_flo, dep_bot, dep_wt, no3_n, sol_p, carbon, flo_dist, bf_max, alpha_bf, revap, rchg_dp, spec_yld, hl_no3n, flo_min, revap_min, aquifer.aqu

Channel parameters (sdc or rte)

Name	Input
chw, chd, chs, chl, chn, chk, cov, d50, ch_clay, carbon, ch_bd, chss, bank_exp, vcr_coef, bankfull_flo, fps, fpn, hc_kh, hc_hgt, hc_ini, ch_n_conc, ch_p_conc, ch_p_bio, pk_rto, fp_inun_days, n_setl, p_setl, n_sol_part, p_sol_part, n_dep_enr, p_dep_enr, arc_len_fr, bed_exp, wash_bed_fr	channel geometry, hydraulics, sediment and nutrient parameters of the SWAT-DEG channel inputs

Channel nutrient parameters (swq)

Name	Input
mumax, rs1 to rs7, rk1 to rk6, bc1 to bc4	in-stream nutrient and algae rates, nutrients.cha (see channel inputs)

A rk5 row changes rs2 (the ben_disp column of nutrients.cha) instead.

Reservoir parameters (res)

Name	Input
esa, evol, psa, pvol	emergency and principal spillway surface areas and volumes, hydrology.res
k_res, evrsv	bottom hydraulic conductivity and evaporation coefficient, hydrology.res
nsed, res_d50, sed_stlr, velsetlr	normal sediment concentration, median particle size, settling rate and settling velocity, sediment.res
psetlr1, psetlr2, nsetlr1, nsetlr2, nsolr, psolr, soln_stl_fr, solp_stl_fr	nutrient settling rates, nutrients.res
vol, sed, orgn, orgp, no3, nh3, no2, solp	water, sediment and nutrient content at the start of the run, initial.res

Reservoir decision table parameters (rdt)

Name	Input
drawdown_days, withdraw_rate	release rules of the reservoir decision tables in res_rel.dtl, per storage zone (see Reservoir decision tables)

Pesticide parameters (pst)

List the pesticides to change (their order in pesticide.pes) in the object list. With an empty list nothing is changed.

Name	Input
pst_koc, pst_washoff, pst_foliar_hlife, pst_soil_hlife, pst_solub, pst_aq_hlife, pst_aq_volat, pst_aq_settle, pst_aq_resus, pst_ben_hlife, pst_ben_bury, pst_ben_act_dep	soil_ads, frac_wash, hl_foliage, hl_soil, solub, aq_hlife, aq_volat, aq_settle, aq_resus, ben_hlife, ben_bury, ben_act_dep, pesticide.pes

Simplified HRU parameters (hlt)

Name	Input
cn2_lte, awc_lte, etco_lte, tc_lte, soildep_lte, slope_lte, slopelen_lte, sy_lte, abf_lte, revapc_lte, percc_lte, sw_lte, gw_lte, gwflow_lte, gwdeep_lte, snow_lte, uslek_lte, uslec_lte, uslep_lte, uslels_lte	hru-lte.hru

awc_lte is given per mm of soil depth; SWAT+ multiplies the change and the upper limit by the soil depth.

gwflow parameters

These only act when gwflow is switched on in codes.bsn. Each name needs the object type shown.

Name	Object type	Objects changed	Input
aquifer_K, aquifer_Sy, aquifer_exdp	gwf	gwflow cells	aquifer hydraulic conductivity, specific yield and extinction depth, gwflow inputs

Name	Object type	Objects changed	Input
`tile_K`, `floodplain_K`	`gwf`	gwflow cells	tile drain and floodplain conductivity, only when tile or floodplain exchange is on
`aquifer_delay`	`hru`	HRUs	recharge delay from the soil to the water table, in days
`stream_K`, `stream_thk`	`gwf_riv`	channels	streambed conductivity and thickness of the cells connected to each channel
`stream_bed`	`gwf_sgl`	applied once	streambed elevation offset, one value for all channels
`reservoir_K`, `pond_bed_K`	`gwf_sgl`	applied once	reservoir and pond bed conductivity

Climate parameters (`cli`)

Name	Input
`precip`	daily precipitation of the listed `*.pcp` records, from `day1` to `day2` of each year from `year1` to `year2`
`temp`	daily temperature of the listed `*.tmp` records; changes nothing

See Climate changes before using these.

Related pages

- Hard calibration. When and how the change list is applied.
- Conditions and regionalization. Condition line variables.
- Soft calibration. The seven `.sft` files that follow these two on the `chg` line.

Sources

- Readers: `src/cal_parm_read.f90` and `src/cal_parmchg_read.f90`. Object list expansion: `src/define_unit_elements.f90`.
- Parameter names and the variables they change: `src/cal_parm_select.f90`. Object type dispatch: `src/cal_conditions.f90`.
- Types: `type calibration_parameters`, `type calibration_conditions` and `type update_parameters` in `src/calibration_data_module.f90`. File name slots: `in_chg%cal_parms` and `in_chg%cal_upd` in `src/input_file_module.f90`.

Column	Fortran variable
`cal_parms.cal` `name`, `obj_typ`, `abs_min`, `abs_max`, `units`	`cal_parms(i)%name`, `%ob_typ`, `%absmin`, `%absmax`, `%units`
`calibration.cal` `name` to `day2`	`cal_upd(i)%name`, `%chg_typ`, `%val`, `%conds`, `%lyr1`, `%lyr2`, `%year1`, `%year2`, `%day1`, `%day2`
`calibration.cal` `obj_tot` and list	`cal_upd(i)%num_tot`, expanded into `cal_upd(i)%num(:)`
condition `var alt targ targc`	`cal_upd(i)%cond(j)%var`, `%alt`, `%targ`, `%targc`
condition `range var min max`	`cal_upd(i)%cond(j)%var`, `cal_upd(i)%val1`, `cal_upd(i)%val2` (shared by all range lines of the row)

Purpose

A row of calibration.cal can be followed by condition lines. SWAT+ then changes the parameter only on the objects that meet every condition of the row. Conditions are checked once, at start-up, against the state of each object after the inputs are read.

Conditions and object lists are the two ways to calibrate parts of a watershed separately, for example one set of curve number changes for cropland and another for forest, or different groundwater parameters upstream and downstream of a gauge.

Condition lines

The number of condition lines is given in the **conds** column of the change row, and the lines follow the row directly. Each line has one of two forms.

Range condition

`range <variable> <min> <max>`

The object's value must be greater than **<min>** and less than **<max>**.

Text condition

`<variable> <operator> <number> <text>`

The object's value must equal **<text>**. All four values must be present. The operator and number are read but none of the existing conditions uses them, so any placeholder works (for example **=** and **0**).

When a row has several condition lines, an object must meet all of them.

Condition variables

Variable	Form	Objects	Condition met when
cal_group	text	HRU	the **cal_group** of the HRU's land use in landuse.lum equals the text
landuse	text	HRU	the HRU's land use name (the **lu_mgt** entry of hru-data.hru) equals the text
hsg	text	HRU	the hydrologic soil group of the HRU's soil (**hyd_grp** in soils.sol) equals the text, for example **C**
texture	text	HRU	the **texture** of the HRU's soil in soils.sol equals the text
plant	text	HRU	the first plant of the HRU's plant community is the named plant
pl_class	text		see below
res_typ	text	reservoir decision table	the decision table's name equals the text
res_pvol	range	reservoir	the principal spillway volume, in 10^4 m^3 (ha-m, as in hydrology.res), is between **<min>** and **<max>**

The HRU conditions apply to parameters whose object type is **hru**, **sol** or **plt**. They compare against the HRU with the same number as the object being changed, so with other object types (aquifers, channels) they test an unrelated HRU.

Text comparisons are exact and case sensitive, and an unknown variable name is ignored, so a misspelt condition lets every object through.

Notes on specific variables:

- **plant**: only the first plant in the community is checked. For a community of corn and soybean in that order, **plant = corn** matches and **plant = soyb** does not. For the plant parameters **lai_pot**, **harv_idx** and **phu_mat** the **plant** condition has a second role: it names the plant whose value is changed (see Plant parameters).

- `pl_class`: compares the class of the plant at the same position in `plants.plt` as the object number. It does not look at the HRU's plants, so it is not useful for selecting HRUs.
- `res_pvol`: both limits are taken from the last `range` line of the row.

Regional calibration

Calibration groups

The most direct way to calibrate regions or land-use groups separately is the `cal_group` column of `landuse.lum`:

1. Give each land use a group name in `cal_group`, for example `crop_north`, `crop_south`, `forest`. Land uses that share a name form one group.
2. Write one change row per group, each with a `cal_group` condition.

```
cn2      pctchg   -8.0   1   0  0  0  0  0  0   0
cal_group  =  0  crop_north
cn2      pctchg    4.0   1   0  0  0  0  0  0   0
cal_group  =  0  forest
```

The same `cal_group` names are used by soft calibration to match its water balance targets to HRUs.

Object lists

The other way is to list the objects in the row itself (`obj_tot` and the numbers after it), for example the HRUs, aquifers or channels upstream of a gauge. A negative number closes a range, so `3  1 -120 245` selects objects 1 to 120 and 245. Object lists work for every object type.

Object lists and conditions can be combined: the conditions are then checked only on the listed objects.

Scheduled land use updates

SWAT+ also reads a file named `scen_dtl.upd` from the project folder, if it exists. It lists decision tables from the scenario decision table file (`scen_lu.dtl` by default, named on the `decision_table` line of `file.cio`) that are checked for every HRU at the start of every simulated day. Their actions can change land use, management or other HRU settings during the run, for example to simulate land use change at a set date.

- Line 1: title.
- Line 2: number of tables.
- Line 3: header.
- One line per table: maximum number of times the table may act, table type, and table name.

The maximum and the type are read but not used in 62.0.1: every listed table is checked every day for every HRU.

This file is separate from `calibration.cal`. Its changes happen during the run, while `calibration.cal` changes happen before the run starts. See Decision tables for how tables are written.

Related pages

- Parameter change files. Layout of the change rows.
- Hard calibration. When and how changes are applied.
- landuse.lum. The `cal_group` column.

Sources

- Reading condition lines: `src/cal_parmchg_read.f90`.
- Evaluating conditions: `src/cal_conditions.f90`.
- Calibration group of an HRU: `src/hrudb_init.f90` copies `lum(ilu)%cal_group` to `hru(ihru)%cal_group`.
- `scen_dtl.upd`: `src/cal_cond_read.f90`; used in `src/time_control.f90`.
- Types: `type calibration_conditions`, `type update_parameters` and `type update_conditional` in `src/calibration_data_module.f90`.

Condition variable	Compared with
cal_group	hru(ielem)%cal_group
landuse	hru(ielem)%land_use_mgt_c
hsg	soil(ielem)%hydgrp
texture	soil(ielem)%texture
plant	pcom(ielem)%pl(1)
pl_class	pl_class(ielem)
res_typ	dtbl_res(ielem)%name
res_pvol	res_ob(ielem)%pvol, against 10000 * val1 and 10000 * val2

Purpose

A sensitivity analysis shows which parameters change a chosen model result the most, so that calibration can concentrate on them. SWAT+ does not include a sensitivity analysis. It is run by an external tool that writes calibration.cal, runs SWAT+, reads a result from the output files, and repeats for many parameter sets. Each run applies the changes in calibration.cal at start-up and leaves the input files untouched, so one project folder serves every run.

SWAT+ Toolbox does this from a graphical interface. The steps below are what any tool or script has to do.

Steps

1. Choose the parameters to test and a range for each. The names are those on Parameter names. The abs_min and abs_max values in cal_parms.cal are physical limits; a narrower range around reasonable values usually gives more useful results.
2. Generate the samples: use a sampling design that matches the analysis method: Sobol (Saltelli sampling), FAST, Morris (elementary effects), Latin hypercube with one-at-a-time changes (LH-OAT), and so on. Each sample is one set of parameter values. Libraries such as SALib (Python) or sensitivity (R) provide both the sampling and the analysis.
3. For each sample, write calibration.cal with one row per parameter, run SWAT+ in the project folder, and compute the result of interest: total or peak flow at an outlet, a performance measure against observations (NSE, KGE, percent bias), a sediment or nutrient load.
4. Compute the indices: pass the sample matrix and the vector of results to the analysis method: first-order and total-order indices for Sobol and FAST, mean and spread of elementary effects for Morris.

The runs are independent, so they can be spread over several processor cores, each in a separate copy of the project folder. Sample sizes of several hundred to a few thousand runs are common.

Writing calibration.cal

A sample with three parameters, written as relative and absolute changes:

```
sensitivity sample 17
3
NAME      CHG_TYP      VAL    CONDS  LYR1  LYR2  YEAR1  YEAR2  DAY1  DAY2  OBJ_TOT
cn2       pctchg     -6.2     0      0     0     0      0      0     0     0
esco      absval      0.71    0      0     0     0      0      0     0     0
alpha     absval      0.048   0      0     0     0      0      0     0     0
```

Points to watch:

- pctchg scales each object's current value, so HRUs keep their differences. absval sets every object to the same value.
- Every parameter name must also be in cal_parms.cal, and its limits there still apply. A sample value outside abs_min to abs_max is cut to the limit, which flattens the response near the edges.

- A misspelt name is skipped without a message. Check a first run by comparing two extreme samples before starting the full set.
- Some parameter names have no effect in 62.0.1, or work only with restrictions; see the notes on Parameter change files and Hard calibration before including them.

Reading results

Daily channel output is in `channel_sd_day.txt` (or `.csv`), with flow out of each channel in the `flo_out` column (m^3/s). Switch on only the output the analysis needs in `print.prt`; writing every file for thousands of runs costs a lot of time and disk space. The output files are described in the Output Reference.

Related pages

- Parameter change files. Format of `cal_parms.cal` and `calibration.cal`, and the parameter names.
- Hard calibration. How each run applies the change list.

Sources

- The change list is read by `src/cal_parmchg_read.f90` and applied by `src/cal_conditions.f90`, both from `src/proc_cal.f90` before the simulation starts.

These pages use the two small datasets in the `refdata` folder of the SWAT+ source repository and go through them file by file. They assume SWAT+ is installed and the Getting started pages have been read.

Pages

- Ames_sub1 walkthrough. Twelve HRUs near Ames, Iowa, simulated from 1975 to 2020, with no routing between objects. Each input group in turn, the run, and the outputs.
- Osu_1hru walkthrough. One rice paddy HRU in South Korea, simulated from 2010 to 2023, with a routing unit, an aquifer and a channel. Paddy management by decision tables, and the shipped files that 62.0.1 reads differently from their headers.
- Carbon notebook. The Jupyter notebook `jupyter_notebooks/carbon.ipynb`, which plots the soil carbon output of one HRU, and how to run it with Ames_sub1.
- Cookbook recipes. Short edits: change the simulation period, turn on an output, change the PET method, switch to Green-Ampt, add an HRU, switch on the carbon model.

Sources

- Datasets: `refdata/Ames_sub1` and `refdata/Osu_1hru` at 62.0.1.
- Notebook: `jupyter_notebooks/carbon.ipynb` at 62.0.1.

`Ames_sub1` is the reference dataset used to test SWAT+. It has twelve HRUs near Ames, Iowa, and 46 years of daily simulation (1975 to 2020), with no channels, aquifers or reservoirs. The folder is `refdata/Ames_sub1` in the source repository.

The sections below go through the input files in the order of `file.cio`, then run the model and read the outputs, as a second pass after the Quick start. Several inputs of this test dataset do not do what their names suggest. They are pointed out where they come up, because they are also the mistakes to look for in a new project.

file.cio

`file.cio` lists every other input file. Most lines in Ames_sub1 are `null`:

```
simulation      time.sim          print.prt         null              object.cnt        null
basin           codes.bsn         parameters.bsn    carbon.bsn
climate         weather-sta.cli   weather-wgn.cli   null              pcp.cli
 ↪  tmp.cli
connect         hru.con
```

```
hru               hru-data.hru
hydrology         hydrology.hyd     topography.hyd
structural        tiledrain.str     septic.str        filterstrip.str   grassedww.str
 ↪  bmpuser.str
hru_parm_db       plants.plt        fertilizer.frt    tillage.til       pesticide.pes     ...
 ↪  snow.sno
ops               harv.ops          graze.ops         irr.ops           chem_app.ops
 ↪  fire.ops   sweep.ops
lum               landuse.lum       management.sch    cntable.lum       cons_practice.lum
 ↪  ovn_table.lum
init              plant.ini         soil_plant.ini
soils             soils.sol         nutrients.sol
decision_table    lum.dtl
regions           ls_unit.ele       ...  aqu_catunit.ele
```

The only spatial objects are HRUs (`hru.con`). There is no routing: each HRU's water, sediment and nutrients leave the model, and the results are read per HRU.

Reference: file.cio.

Simulation settings

time.sim

```
day_start  yrc_start   day_end   yrc_end      step
        0       1975         0      2020         0
```

Day 0 at the start and end means 1 January of the first year and 31 December of the last. `step` 0 is a daily time step.

print.prt

```
nyskip      day_start  yrc_start  day_end   yrc_end   interval
0           0               1975         0     2020          1
...
objects                  daily       monthly        yearly         avann
hru_wb                       y             n             n             y
hru_nb                       y             n             n             y
hru_ls                       n             n             n             y
hru_pw                       y             n             n             y
hru_cb                       l             y             y             y
hru_cb_vars                  l             y             y             y
```

`nyskip` 0 means no warm-up years are left out of the outputs; for a real project set it to one to three years. Only HRU outputs are listed. Daily water balance, nutrient and plant output is on, and the carbon rows write daily output for each soil layer (`l`), which is what makes this run's output about 9 GB.

object.cnt

Twelve HRUs and no other objects.

References: time.sim, print.prt, object.cnt.

Basin settings

codes.bsn

```
pet_file  wq_file  pet  event  crack  swift_out  sed_det  rte_cha  deg_cha  wq_cha  nostress
 ↪  cn  c_fact  carbon  lapse  uhyd  sed_cha  tiledrain  wtable  soil_p  gampt  atmo_dep
 ↪  stor_max  i_fpwet  gw_flow  idc_till
null      null     2    0      0      0          0        0        0        0       0         2
 ↪  0       2       0      1     0        0          0       0       0      0         0
 ↪  0        0        3
```

- `pet` 2: potential evapotranspiration by the Hargreaves method.
- `cn` 2: read but not used in 62.0.1; the curve number always follows soil water.
- `carbon` 2: the soil carbon model is on, with its parameters from `carbon.bsn` and `carbon_lyr.bsn`.
- `gampt` 0: curve number runoff (Green and Ampt infiltration is off).

parameters.bsn and carbon.bsn

Basin-wide parameters and the carbon model coefficients. Ames_sub1 has `1.0` in column 7 of `parameters.bsn`, which switches on temperature-dependent pesticide decay.

References: codes.bsn, parameters.bsn, carbon.bsn.

Climate

`weather-sta.cli` defines one station:

```
name        wgn       pcp        tmp        slr   hmd   wnd   wnd_dir   atmo_dep
wgn_sta1    wgn_001   ames.pcp   ames.tem   sim   sim   sim   null      null
```

Precipitation comes from `ames.pcp`. Solar radiation, humidity and wind are generated (`sim`) from the monthly statistics in `weather-wgn.cli`.

Temperature does not come from a file. The station names `ames.tem`, but the temperature file list `tmp.cli` contains only `ames.tmp`, an empty file. The model cannot match the name, writes `ames.tem file not found (tgage)` to `diagnostics.out`, and generates daily temperatures from `weather-wgn.cli`. The measured record in `ames.tem` is never read. With Hargreaves PET, which depends on temperature, this affects ET as well.

`hru.con` gives each HRU the station name `wgn_01`, which is not the name in `weather-sta.cli` (`wgn_sta1`). With a single station the model uses it anyway, without a message; in a project with several stations, check that these names match.

References: weather-sta.cli, *.pcp, *.tmp, *.tem.

HRUs

hru-data.hru

Each row names the records an HRU uses:

```
id  name      topo          hydro     soil         lu_mgt            soil_plant_init  surf_stor
 ↪  snow     field
 1  hru0001   topohru0001   hyd0001   soil_01-h1   cosy_lum          soilplant1       null
 ↪  snow001  null
 2  hru0002   topohru0002   hyd0002   soil_02      mntill_corn_lum   soilplant1       null
 ↪  snow001  null
...
12  hru0012   topohru0012   hyd0012   soil_12      cosy_lum          soilplant1       null
 ↪  snow001  null
```

Eleven HRUs use `cosy_lum`, a corn and soybean rotation, and HRU 2 uses `mntill_corn_lum`, continuous corn with reduced tillage. Each HRU has a different soil.

hydrology.hyd and topography.hyd

HRU hydrology parameters (lateral flow travel time, `esco`, `epco`, `cn3_swf`, `perco`, `latq_co`, …) and slope, slope length and lateral flow length.

References: hru-data.hru, hydrology.hyd and topography.hyd.

Land use and management

`landuse.lum` links each land use to a management schedule:

Land use	Schedule in `landuse.lum`	Used by
`cosy_lum`	`mgt_01`	HRUs 1 and 3 to 12
`mntill_corn_lum`	`mgt_08`	HRU 2
`cosy_lum2`	`mgt_01A`	no HRU
`agrr_lum`, `soyb_lum`, `oats_lum`, `wwht_lum`	`AGRR_rot`, `SOYB_rot`, `OATS_rot`, `WWHT_rot`	no HRU; the schedules are not in `management.sch`

`management.sch` contains schedules `mgt_01` to `mgt_12` and `mgt_01A`, each a list of dated operations (tillage, planting, fertilizer, harvest). Two points about the shipped schedules at 62.0.1:

- The fertilizer name in the `fert` rows is one column to the right of where the model reads it, so the model finds no fertilizer and the rows apply nothing, without a message. The management page shows the expected layout.
- `lum.dtl` contains a decision table, `irr_year_irr`, meant to limit irrigation to 500 mm a year. At 62.0.1 the yearly total is not reset, so once 500 mm have been applied in total the table stops irrigation for the rest of the run.

References: landuse.lum, management.sch, *.dtl.

Databases and initial conditions

- `plants.plt`, `fertilizer.frt`, `tillage.til`, `pesticide.pes`, `urban.urb`, `septic.sep`, `snow.sno`: parameter databases.
- `harv.ops`, `graze.ops`, `irr.ops`, `chem_app.ops`, `fire.ops`, `sweep.ops`: parameters of each type of operation.
- `plant.ini`: the plant community each HRU starts with.
- `soil_plant.ini`: which nutrient record each HRU's soil starts with.
- `soils.sol` and `nutrients.sol`: soil layers and initial nutrient contents.

References: plants.plt, fertilizer.frt, tillage.til, *.ops, soils.sol, initial conditions.

Running the model

Check that there is about 10 GB of free space, or switch off the daily carbon output first (set the `daily` column of `hru_cb` and `hru_cb_vars` to `n` in `print.prt`). Then, from inside the folder:

```
path/to/swatplus-62.0.1-gnu-lin_x86_64-Rel
```

The screen shows the date being simulated:

```
  Original Simulation            1   14   1994 Yr    20 of    46 Time   12:37:36
```

The run took 4 minutes 20 seconds on a recent Linux workstation, most of it spent writing the daily carbon files.

Outputs

simulation.out

The last lines should be:

```
  Original Simulation           12  31  2020 Yr   46 of   46 Time  12:40:12

 Execution successfully completed
```

If the completion line is missing, the run stopped early; the last date shows where.

diagnostics.out

```
 DIAGNOSTICS.OUT FILE
 agrr_lum      AGRR_rot      not found in management.sch
 soyb_lum      SOYB_rot      not found in management.sch
 oats_lum      OATS_rot      not found in management.sch
 wwht_lum      WWHT_rot      not found in management.sch
 ames.tem                    file not found (tgage)
```

The first four lines are land uses whose schedules are missing; no HRU uses them, so they do not affect the results. The last line is the temperature file described under Climate.

Reference: Checker and diagnostics.

checker.out

One row per HRU with the main soil and hydrology values the model ended up with:

```
 sname        hydgrp       zmx    usle_k    sumfc     sumul   usle_p  usle_ls   esco   epco
 ↪  cn3_swf  perco  latq_co  tiledrain
soil_01-h1    B         2000.0    0.0000  361.500   616.090   1.0000   0.3638 0.9500 1.0000
 ↪  0.0000 0.5000   0.0100          0
soil_02       B         2000.0    0.0000  361.500   449.038   0.7500   0.3638 0.9500 1.0000
 ↪  0.0000 0.5000   0.0100          0
soil_03-h3    B         2000.0    0.0000  361.500   610.323   1.0000   0.3638 0.9500 1.0000
 ↪  0.0000 0.5000   0.0100          0
...
```

`usle_k` is 0 for every soil, so the HRUs produce no sediment. Checking this file for zeros and implausible values is a quick test of any new project.

hru_wb_aa.txt

The average annual water balance of each HRU. For HRU 1:

precip	snofall	surq_gen	latq	perc	et
952.6 mm	110.3 mm	34.7 mm	0.1 mm	0.0 mm	932.5 mm

Almost all precipitation leaves as ET and nothing percolates. For a real Iowa watershed, ET would be closer to two thirds of precipitation. Comparing the average annual water balance with regional estimates is the first check of any project, and soft calibration adjusts it.

Columns: Water balance outputs.

Other outputs

- `hru_nb_*`, `hru_ncycle_*`: nutrient balance and nitrogen cycle per HRU.
- `hru_ls_aa.txt`: sediment and nutrient losses per HRU.
- `hru_pw_*`: plant growth and stress per HRU.
- `crop_yld_yr.txt`, `crop_yld_aa.txt`: yields by HRU and crop.
- `mgt_out.txt`: every management operation performed, by date and HRU. Use it to check that the schedules do what was intended.
- `hru_carbvars.txt`, `hru_cflux_stat.txt`, `hru_cpool_stat.txt` and the other carbon files: see Carbon outputs.

The shipped folder also contains `basin_*` files and others that this run does not write. They are left from earlier model versions; see Quick start.

Changes to try

- Set `nyskip` to 2 in `print.prt` and compare `hru_wb_aa.txt`.
- Rename `ames.tmp` to `ames.tem` in `tmp.cli` so that the measured temperatures are used, and compare ET.
- Change the PET method in `codes.bsn` (`pet` 0 for Priestley-Taylor, 1 for Penman-Monteith); both need solar radiation, humidity and wind, which are generated here. See the cookbook.
- Follow the Carbon notebook to plot the carbon output.

Sources

- Dataset: `refdata/Ames_sub1` at 62.0.1.
- Run: the 62.0.1 `gnu-lin_x86_64` release executable, 20 October 2026. `diagnostics.out`, `checker.out`, `hru_wb_aa.txt` and the run time are from that run.
- Fertilizer column: the `fert` case of `src/mgt_sched.f90` uses `mgt%op1`, the fertilizer matched from `op_data1` (column 5); the Ames `fert` rows have `null` there and the name in `op_data2`. Details on management.sch.
- Station and file name lookups, and the irrigation decision table: see weather-sta.cli and decision tables.

`Osu_1hru` is the second reference dataset in the SWAT+ source repository. It has one HRU of 10 ha, a rice paddy in South Korea with weather from the Imsil station, simulated from 2010 to 2023. Unlike Ames_sub1, the HRU belongs to a routing unit that sends its water to an aquifer and a channel, so the routing, aquifer and channel outputs have values. The folder is `refdata/Osu_1hru` in the source repository.

The rice is managed almost entirely by decision tables: weir height, paddy irrigation, puddling, planting, fertilizer and harvest are actions in `lum.dtl`. This makes the dataset a small example of paddy management. Several shipped files are in older formats or are not read by 62.0.1, and the sections below say what the model does with each one.

file.cio

```
simulation        time.sim          print.prt         null              object.cnt        null
basin             codes.bsn         parameters.bsn    null
climate           weather-sta.cli   weather-wgn.cli   null              pcp.cli
 ↪  tmp.cli           slr.cli           null              wnd.cli           null
connect           hru.con           null              rout_unit.con     null
 ↪  aquifer.con       null              null              null        recall.con
 ↪  null              null              null              chandeg.con
channel           initial.cha       null              null              null
 ↪  nutrients.cha     channel-lte.cha   hyd-sed-lte.cha   null
```

```
reservoir          initial.res       null              null              sediment.res
 ↪  nutrients.res      weir.res              wetland.wet       hydrology.wet
routing_unit       rout_unit.def     rout_unit.ele     rout_unit.rtu     null
hru                hru-data.hru      null
...
recall             recall.rec
...
aquifer            initial.aqu       aquifer.aqu
...
water_rights       null              null              null
...
hydrology          hydrology.hyd     topography.hyd    field.fld
structural         tiledrain.str     septic.str        filterstrip.str   grassedww.str
 ↪  bmpuser.str
hru_parm_db        plants.plt        fertilizer.frt    tillage.til       pesticide.pes     null
 ↪  null               null              urban.urb         septic.sep        snow.sno
ops                harv.ops          graze.ops         irr.ops           chem_app.ops
 ↪  fire.ops           sweep.ops
lum                landuse.lum       management.sch    cntable.lum       cons_practice.lum
 ↪  ovn_table.lum
chg                cal_parms.cal     calibration.cal   null ...
init               plant.ini         soil_plant.ini    om_water.ini      null ...
soils              soils.sol         nutrients.sol     null
decision_table     lum.dtl           res_rel.dtl       null              null
regions            ls_unit.ele       ls_unit.def       null ...          aqu_catunit.ele   null
 ↪  ...
```

The `basin` line already has three slots, with `null` for the carbon file. The `water_rights` line has three `null` names where 62.0.1 reads two, and the extra `null` is ignored. Both lines are read correctly (see older file.cio files).

The `chg` line names `calibration.cal`. Its 13 rows change `cn2`, `petco`, `latq_co`, `alpha`, `epco`, `awc`, `esco`, `perco`, `cn3_swf`, `canmx`, `lat_len`, `revap_co` and `surlag` by a percentage for every object, and the model applies these changes at start-up (see hard calibration). The run below includes them.

Reference: file.cio.

Simulation settings

time.sim

```
day_start  yrc_start   day_end   yrc_end      step
        1       2010       365      2023         0
```

1 January 2010 to 31 December 2023, 14 years, with a daily time step.

print.prt

```
nyskip      day_start  yrc_start  day_end   yrc_end   interval
3           0          0          0         0         1
aa_int_cnt
0
csvout         dbout          cdfout
n              n              n
soilout        mgtout         hydcon         fdcout
n              y              n              n
objects                   daily       monthly        yearly         avann
basin_wb                     n             n              y              y
basin_nb                     n             n              n              y
```

```
...
hru_pw                          n             n             n             y
hru_cb                          n             n             n             y
hru-lte_wb                      n             n             n             y
...
pest                            n             n             n             y
```

`nyskip` 3 leaves the first three years (2010 to 2012) out of the outputs, so the average annual files cover 2013 to 2023. `basin_wb` writes yearly and average annual files, and every other row writes average annual files only.

An older SWAT+ Editor wrote this file, and 62.0.1 reads some of it differently from its headers:

- On the `csvout dbout cdfout` line, the second value is read as `use_obj_lbls`. It is `n`, so the object rows are read by position, in the fixed order listed on the print.prt page, and their names are not checked.
- On the `soilout mgtout hydcon fdcout` line, the first value is read as `crop_yld`. It is `n`, so no `crop_yld_*` files are written. `mgtout` is `y`, so `mgt_out.txt` is written.
- The fixed order has no `hru_cb` row. The `hru_cb` line is read as the `hru-lte_wb` row, and every later line is read one row early. All these lines have the same switches (`n n n y`), so in this file the shift changes nothing. The carbon rows can only be set by name, with `use_obj_lbls` = `y`.

object.cnt

```
name    ls_area  tot_area  obj  hru  lhru  rtu  mfl  aqu  cha  res  rec  exco  dlr  can  pmp
 ↪  out  lcha  aqu2d  hrd  wro
Osu          10        10    4    1     0    1    0    1    0    0    0     0    0    0    0
 ↪  0     1      0
```

Four objects: one HRU, one routing unit, one aquifer and one SWAT+ channel (`lcha`). The line ends before `hrd` and `wro`, which stay 0. `rec` is 0, so the recall object in `recall.con` is not built (see Other files).

References: time.sim, print.prt, object.cnt.

Basin settings

```
pet_file  wq_file  pet  event  crack  rtu_wq  sed_det  rte_cha  deg_cha  wq_cha  nostress  cn
 ↪  c_fact  carbon  lapse  uhyd  sed_cha  tiledrain  wtable  soil_p  gampt  atmo_dep  stor_max
 ↪  i_fpwet  gw_flow  idc_till
null      null     1    0      0      1       0        0        0        1       0         0
 ↪  0       0       0      1     0        0          0       0       0      a         0
 ↪  0        0        3
```

- `pet` 1: Penman-Monteith potential evapotranspiration, which uses solar radiation, humidity and wind.
- The sixth column has the old header `rtu_wq`, and 62.0.1 reads it as `swift_out`. Its value 1 makes every run write the SWIFT input files into a `SWIFT` folder at the end.
- `wq_cha` 1 is not used by 62.0.1.
- `carbon` 0: static soil carbon, so no carbon file is needed.
- `gampt` 0: curve number runoff.

References: codes.bsn, parameters.bsn.

Climate

`weather-sta.cli` defines one station:

```
name              wgn     pcp            tmp            slr            hmd   wnd
 ↪  wnd_dir  atmo_dep
s35610n127290e    Imsil   Imsilpcp.pcp   Imsiltmp.tmp   Imsilsol.slr   sim   Imsilwind.wnd
 ↪  null     null
```

Precipitation, temperature, solar radiation and wind come from the four Imsil files, each listed in its index file (pcp.cli, tmp.cli, slr.cli, wnd.cli). Relative humidity is generated (sim) from the monthly statistics of the Imsil record in weather-wgn.cli. The precipitation and temperature records run from 1981 to the end of 2023. The solar radiation and wind records end on day 304 of 2022, and the model generates those two variables for the rest of the run (see weather outside the observed record).

References: weather-sta.cli, *.pcp, *.tmp.

HRU, routing unit, aquifer and channel

hru.con has one HRU of 10 ha with no outflow connections (out_tot = 0):

```
id  name     gis_id  area  lat       lon        elev       hru  wst              cst  ovfl
 ↪  rule  out_tot
 1  hru0001       1    10  35.65311  127.36763  350.96629    1  s35610n127290e     0     0
 ↪  0        0
```

The HRU is the only element of routing unit rtu001 (rout_unit.def and rout_unit.ele). The routing unit collects its outflow and passes it on through rout_unit.con: surface, lateral and tile flow (tot) to channel 1, and percolation (rhg) to aquifer 1. The aquifer returns its flow to the same channel through aquifer.con. Channel 1 in chandeg.con has no outflow connection and is the basin outlet. The connection files page shows these three files.

aquifer.aqu has 21 records, aqu011 to aqu102 and aqu_deep010, all with the same parameters except the deep one. Only the first record is used: aquifer.con has one aquifer, and its aqu column points to row 1.

hru-data.hru names the records the HRU uses:

```
id  name      topo          hydro     soil        lu_mgt        soil_plant_init  surf_stor
 ↪  snow     field
 1  hru0001   topohru0001   hyd0001   PadHOEGOG   rice140_lum   soilplant1       paddy0001
 ↪  snow001  null
```

surf_stor = paddy0001 points to a record in wetland.wet whose hydrology record is paddy, which makes the HRU a paddy with ponded water and a weir. The wetland page explains the paddy records in this dataset. A misspelled name in any of the other columns would not be reported: the HRU would use the first record of that file (see name lookup).

References: *.con, hru-data.hru, aquifer.aqu, wetland.wet.

Land use and management

landuse.lum links rice140_lum to the plant community rice140_comm in plant.ini and to the schedule rice140_rot in management.sch, with curve number record rc_strow_g, conservation practice cross_slope and Manning's n record convtill_res. The plant community starts with no growing plant.

rice140_rot has one dated operation and seven decision tables:

```
rice140_rot    1    7
     plow
     weir60r
     weir0
     pad_irr_r
     puddle
     pl_hv_rice
     fert_paddy
manu      5          1      0     beef_salt   broadcast  12.9927
```

The dated operation applies 12.99 kg/ha of the manure beef_salt with broadcast on 1 May each year. The decision tables in lum.dtl do the rest:

Table	Action
plow	Tillage with fieldcultivat early in the year, once the base-zero heat units pass 0.05.
weir60r	Raises the paddy weir to 60 mm on a day between day 120 and day 130.
weir0	Lowers the weir to 0 on a day between day 270 and day 280.
pad_irr_r	From May, while the weir is up and the ponded water is shallower than 60 mm, irrigates with the ponding90 record of irr.ops.
puddle	After April, with the weir up, water ponded and less than 10 mm of rain that day, puddles the paddy with the med_eff record of puddle.ops.
pl_hv_rice	Plants rice140 with the weir up once the base-zero heat units pass 0.15, and harvests and kills it once it has reached 1.15 times its heat units.
fert_paddy	Applies 100 kg/ha of elem_n and 20 kg/ha of elem_p early in the year, and 36 kg/ha of elem_n once the rice has passed half its heat units.

Two shipped files that these tables use are not read as their authors intended:

- The plant action names the transplant record tr_rice140. That record is in transplant.plt, and 62.0.1 reads transplant records only from transplant.ops. The name is not found, no message is written, and the rice is planted from seed. mgt_out.txt still writes a TRANSPLANT line after each PLANT line. With the file renamed to transplant.ops, the model uses the record, and the planting and harvest dates change (see transplant.ops).
- puddle.ops has no title line, so the model reads its header line as the title and its first record, high_eff, as the header. The puddle table names med_eff, which is still read, so puddling works (see puddle.ops).

References: landuse.lum, management.sch, decision tables, *.ops.

Other files

- soils.sol, nutrients.sol, soil_plant.ini: soil PadHOEGOG and its initial nutrients. The soils page and the initial conditions page use Osu_1hru as their examples.
- channel-lte.cha, hyd-sed-lte.cha, initial.cha, nutrients.cha: the channel record used by chandeg.con. See SWAT+ channels.
- recall.con and recall.rec: rec is 0 in object.cnt, so no recall object takes part in the run. The model still reads recall.rec, because file.cio names it, and opens the data file RECYR. named there. That file is not in the folder, so the run leaves an empty file of that name behind. See recall.
- pesticide.pes: the half-life columns contain the old decay rates (0.007 and 0.05), which 62.0.1 reads as half-lives in days. No pesticides are simulated here, so they have no effect (see pesticide.pes).
- salt_hru.ini, salt_aqu.ini, salt_channel.ini: empty, and not used, because no salts are simulated (see salts).
- calibration.cal and cal_parms.cal: the parameter changes described under file.cio, in the format on the parameter change files page.

The shipped folder also contains output files left from runs with older revisions: mgt_out.txt, erosion.txt, fort.2222, the *.swf files at the top level and the SWIFT folder. A 62.0.1 run overwrites mgt_out.txt, erosion.txt and the files in SWIFT, and does not write the others.

Running the model

From inside the folder:

```
path/to/swatplus-62.0.1-gnu-lin_x86_64-Rel
```

With one HRU the run takes a few seconds and writes a few megabytes. The screen shows the date being simulated, and simulation.out ends with:

```
 Original Simulation           12  31  2023 Yr   14 of   14 Time  ...

 Execution successfully completed
```

Besides the output files, the run creates some empty files in the folder: `null`, `RECYR.`, `wtps_wuses.wal`, `cs_hru.ini`, `cs_aqu.ini` and `cs_channel.ini`. They come from readers that create a file when the one they look for is missing, and they do not affect the results.

Outputs

The values below are from a five-year run (2010 to 2014) of a copy of this folder, so the average annual values cover 2013 and 2014 only. A full run gives different numbers.

diagnostics.out

```
 DIAGNOSTICS.OUT FILE
```

No missing names or files are reported.

checker.out

```
sname       hydgrp      zmx   usle_k    sumfc     sumul  usle_p  usle_ls    esco    epco
 ↪  cn3_swf  perco  latq_co  tiledrain
PadHOEGOG   C         750.0   0.0000   98.114   230.336  0.7500   0.1045  0.9500  0.5000
 ↪  0.9500  0.9000   0.0100          0
```

`checker.out` is written before the changes in `calibration.cal` are applied, so `esco`, `epco`, `cn3_swf` and `perco` are the values from `hydrology.hyd`. `usle_k` is 0, so the HRU produces no USLE erosion. The sediment in the outputs comes from puddling, which sets the sediment concentration of the ponded water.

mgt_out.txt

The log of every operation and decision table action. The first season, shortened:

```
 1  2010   2  21  beef_salt   FERT
 1  2010   2  21  beef_salt   FERT
 1  2010   3  24  fldcult     TILLAGE
 1  2010   5   1  beef_salt   MANURE
 1  2010   5   4  paddy       RESET WEIR HEIGHT (m)   0.06
 1  2010   5   4  ponding     PADDY IRRIGATION
 1  2010   5   5  puddle      PUDDLE
 1  2010   5   8  rice140     PLANT
 1  2010   5   8  rice140     TRANSPLANT
 1  2010   7  21  beef_salt   FERT-WET
 1  2010  10   3  paddy       RESET WEIR HEIGHT (m)   0.00
 1  2010  10   8  rice140     HARV/KILL
```

The `FERT` lines from the `fert_paddy` table show the name `beef_salt`, the schedule's dated operation, in place of `elem_n` and `elem_p`. The amounts in the `op_var` columns (100, 20 and 36 kg/ha) are the ones applied. `FERT-WET` means the fertilizer went into the ponded water. Between planting and harvest the log also has a `PLANT_ALREADY_GROWING` line on every day the `pl_hv_rice` table finds the planting conditions met.

The 2010 rice is harvested on 8 October. In the later years of this run it does not reach 1.15 times its heat units before winter and is harvested the following spring, a few days before the next planting. Read this file to check that the decision tables do what was intended.

Water balance

`basin_wb_aa.txt` and `hru_wb_aa.txt` have the same values, because there is one HRU:

precip	irr	surq_gen	perc	et	wet_evap	wet_oflo
1321.9 mm	239.5 mm	830.8 mm	404.5 mm	333.3 mm	116.3 mm	830.1 mm

Most of the rain and irrigation leaves the paddy over the weir (`wet_oflo`), and this water is reported as surface runoff. `basin_wb_yr.txt` gives the same columns for each printed year.

Columns: Water balance outputs.

Routing, aquifer and channel

- `ru_aa.txt` and `hydout_aa.txt`: the routing unit sends about 83,000 m³ a year to the channel and about 40,000 m³ of recharge to the aquifer. The units line of these files says m³/s, but the values are volumes in m³ (see hydrographs).
- `aquifer_aa.txt`: 404.5 mm of recharge and 379.5 mm of return flow to the channel (`flo_cha`).
- `channel_sd_aa.txt`: the channel inflow is the routing unit and aquifer flow together, about 0.004 m³/s on average.

Columns: Aquifer, reservoir, plant, Channel, sediment, nutrient.

Plant growth and the paddy

- `hru_pw_aa.txt`: plant biomass, yield, stresses and weather for the HRU. With `crop_yld` set to `n` in `print.prt` there are no `crop_yld_*` files, so read the yield from this file.
- `wetland_aa.txt`: the ponded water of the paddy, with precipitation, evaporation, seepage and outflow in m³.
- `paddy_test.csv`: a daily table of the paddy (irrigation, seepage, weir height, water depth, sediment and nitrate). 62.0.1 writes it in every run.

Changes to try

- Rename `transplant.plt` to `transplant.ops` and compare the dates in `mgt_out.txt`.
- Set the first value on the `soilout` line of `print.prt` to `b` to get the `crop_yld_*` files.
- Set `pet` to 2 in `codes.bsn` for Hargreaves PET and compare `et` in `hru_wb_aa.txt`.
- Set the sixth value of `codes.bsn` to 0 to stop writing the `SWIFT` folder.
- Shorten the run in `time.sim`, or switch on daily `basin_wb` output, as shown in the cookbook.

Sources

- Dataset: `refdata/Osu_1hru` at 62.0.1.
- Run: a copy of the folder with `time.sim` shortened to 2010 to 2014 and the shipped output files removed, run with a 62.0.1 Linux build (Intel compiler) made from commit 97ca231 on 20 October 2026. That commit differs from the 62.0.1 tag only in gwflow, calibration and export coefficient code, which this dataset does not use. `diagnostics.out`, `checker.out`, `mgt_out.txt` and the output values are from that run. The transplant test used the same copy with `transplant.plt` copied to `transplant.ops`.
- `print.prt` reading: `basin_print_codes_read.f90` reads line 7 into `pco%csvout`, `pco%use_obj_labels`, `pco%cdfout` and line 9 into `pco%crop_yld`, `pco%mgtout`, `pco%hydcon`, `pco%fdcout`. In fixed order, the row after `hru_pw` is `pco%wb_sd` (`hru-lte_wb`), which receives the Osu `hru_cb` line.
- `object.cnt`: `basin_read_objs.f90` reads `bsn, sp_ob` in one list-directed read, and the short line ends the read at end of file.
- `codes.bsn`: `basin_read_cc.f90` reads the line into `bsn_cc`, whose sixth component is `swift_out`. The SWIFT files are written by `swift_output.f90`.
- `checker.out` and calibration: `checker.out` is written in `proc_hru.f90`, and `proc_cal` runs later in `main.f90.in`.
- Transplant lookup: `dtbl_lum_read.f90` matches the `plant` action's file pointer against `transpl%name`, which `plant_transplant_read.f90` fills from `transplant.ops` only. In the `plant` case of `actions.f90`, `mgt_transplant` is called only when `act_app` > 0, but the `TRANSPLANT` line is written whenever `pco%mgtout` = `y`.
- `fertilize` action log: the `fertilize` case of `actions.f90` writes `mgt%op_char` in the name column of `mgt_out.txt`, where `mgt` is the HRU's current schedule operation (`sched%mgt_ops(hru%cur_op)`, set in `mgt_operatn`).

- Recall: recall_read.f90 reads recall.rec whenever the file exists, independent of the rec count, and opens each record's data file without checking that it exists, which creates RECYR..
- Puddle file: mgt_read_puddle.f90 skips a title and a header line before the records.

The SWAT+ source repository has one analysis notebook, jupyter_notebooks/carbon.ipynb. It reads the carbon output of one HRU and draws time series of soil carbon, the carbon pools by soil layer, plant carbon and the carbon fluxes, with the HRU's management operations marked on each graph. It uses the legacy CSU carbon files, which have no interval suffix in their names (see legacy CSU carbon outputs). The newer carbon files, such as hru_cbn_lyr_day.txt, are not read by the notebook.

Files read

The notebook reads these files from one folder:

File	Use
hru-data.hru, landuse.lum	Soil, land use and schedule names of the chosen HRU.
mgt_out.txt	Management operations of the HRU, drawn as markers.
hru_cbn_lyr.txt	Total soil carbon by layer.
hru_seq_lyr.txt	Sequestered soil carbon, total and in the top 300 mm.
hru_plc_stat.txt	Plant carbon: above ground, leaf, stem, seed, root and surface residue.
hru_cpool_stat.txt	Residue, metabolic, structural, lignin, microbial, slow humus and passive humus carbon, root mass and soil water, by layer.
hru_org_trans_vars.txt	Potential transformation rates (bmctp, hsctp, hsntp, hpctp, lslctp, lslnctp) by layer.
hru_cflux_stat.txt	Carbon flows between pools and CO2 losses by layer.

The input files and the output files must be in the same folder, which is the case unless file.cio sets an output folder.

Switches

The legacy files are written only with these settings:

- carbon = 2 in codes.bsn, with carbon.bsn and carbon_lyr.bsn present (see carbon model). hru_plc_stat.txt, hru_cpool_stat.txt, hru_cflux_stat.txt and hru_org_trans_vars.txt are written only with carbon = 2.
- use_obj_lbls = y in print.prt, because the carbon rows are recognised only by name.
- A hru_cb row with y or l in the daily column, for the first five files in the table. l adds layer properties at the start and end of the run.
- A hru_cb_vars row with y or l in the daily column, for hru_org_trans_vars.txt.
- mgtout = y in print.prt, for mgt_out.txt.

The notebook uses the shortest interval in each file, and it is designed for daily output.

Ames_sub1 setup

The Ames_sub1 reference dataset at 62.0.1 already has all of these settings:

```
csvout        use_obj_lbls  cdfout
n             y             n
crop_yld      mgtout        hydcon        fdcout
b             y             n             n
objects                  daily       monthly        yearly         avann
...
hru_cb                     l             y             y             y
hru_cb_vars                l             y             y             y
```

The full run of 46 years writes about 9 GB, most of it daily carbon output. For the notebook, shorten the run first: with `time.sim` set to 1975 to 1976, the run takes a few seconds and the output folder is about 380 MB.

Running the notebook

1. Copy `refdata/Ames_sub1` to a new folder, shorten `time.sim`, and run SWAT+ in the copy (see Running the model).
2. Install the Python packages. The notebook uses `pandas` and `plotly`, and plotly needs `nbformat` to show figures in Jupyter:

   ```
   pip install jupyterlab pandas plotly nbformat
   ```

3. Open `carbon.ipynb` in JupyterLab or VS Code.
4. In the first code cell, under `# hru to run against.`, set:
   - `hru_id`: the HRU number to plot (1 by default).
   - `wdir` and `sdir`: the notebook reads from the folder `f"{wdir}/{sdir}"`. The values in the shipped notebook are the author's folders and must be changed, for example `wdir = "/home/me/runs"` and `sdir = "Ames_sub1"`.
   - `output_html_file`: `True` also saves every figure as an HTML file.
5. Run all cells.

The cell that reads a file does nothing when the file is missing, and the graph cells after it then work with the table of the previous file, or stop with an error if there is none. Turn on every switch listed above before running the notebook.

Graphs

Each section of the notebook has a cell that reads one file and one or more cells that draw Plotly figures from it. With daily output for HRU 1 of Ames_sub1, the notebook draws, in order:

1. Total soil carbon by layer (`hru_cbn_lyr.txt`), in Mg/ha per mm of layer thickness.
2. Sequestered carbon in the top 300 mm (`hru_seq_lyr.txt`), in Mg/ha.
3. Plant carbon components (`hru_plc_stat.txt`), in kg/ha.
4. Residue and soil carbon pools in the 10 mm top layer, and in the layer at a depth set in the cell (100 mm by default) (`hru_cpool_stat.txt`).
5. One graph by layer for each pool in `hru_cpool_stat.txt`: residue, metabolic, structural, lignin, microbial, slow humus, passive humus, live root mass and soil water content.
6. One graph by layer for each potential transformation rate in `hru_org_trans_vars.txt`.
7. One graph by layer for each carbon flow between pools (`cfmets1` to `cfs3s1`) and each CO2 loss (`co2fmet` to `co2fs3`) in `hru_cflux_stat.txt`.

Several graph cells have a `normalize` setting that divides each layer's value by the layer thickness, and a `show_mgt` setting that turns the management markers on or off. The markers are coloured by operation type: planting, harvest, tillage, fertilizer, manure, irrigation, pesticide and grazing.

Related pages

- Carbon outputs: the newer carbon files and the legacy files with their columns.
- carbon.bsn: carbon model parameters and initial pools.
- Soil carbon theory: what the pools and flows represent.
- Cookbook: carbon model: switching the carbon model on in another project.

Sources

- Notebook: `jupyter_notebooks/carbon.ipynb` at 62.0.1. The file names, the folder variables `wdir` and `sdir`, the `read_data` and `read_mgt_out` functions and the graph cells are in its code cells.

- Test: a copy of `refdata/Ames_sub1` with `time.sim` set to 1975 to 1976, run with a 62.0.1 Linux build (Intel compiler) made from commit 97ca231, which differs from the tag only in gwflow, calibration and export coefficient code. The run took about 10 seconds and wrote 379 MB. All code cells of the notebook were then run outside Jupyter with pandas 3.0.1 and plotly 6.6.0, with `wdir` pointed at the copy; every cell completed and 33 figures were drawn, with 731 daily values (1975 and 1976) in each data trace.
- Legacy file switches: `command.f90` calls `soil_nutcarb_write_legacy.f90` for `pco%cb_hru` and `soil_carbvar_write_legacy.f90` for `pco%cb_vars_hru` (only with `bsn_cc%cswat` = 2). The files are opened in `carbon_legacy_module.f90`. The `freq` column takes the values `day`, `mon` and `year`, which the notebook's reading cells filter on.
- Missing files: each reading cell tests `file_path.is_file()` and otherwise only sets `df_plot` to an empty table; the graph cells test `df.empty`, where `df` is the table from the last reading cell that found its file.

Six common edits to a SWAT+ project that already runs. Each recipe names the file and the values to change, and what to check after the run. The input reference pages linked from each recipe give the full rules.

Simulation period

Edit `time.sim`:

```
day_start  yrc_start   day_end   yrc_end      step
        0       1975         0      2020         0
```

- `yrc_start` and `yrc_end` are the first and last calendar years.
- `day_start` is the day of the year the run starts; 0 is read as 1, 1 January.
- `day_end` is the day of the year the run ends in the last year; 0 means 31 December.

To leave warm-up years out of the outputs, keep `time.sim` and set `nyskip` on line 3 of `print.prt` to the number of years to skip. The skipped years are still simulated.

Days outside the measured weather records are generated from `weather-wgn.cli` without a message, so a period longer than the records still runs. Compare the period with the first and last dates in the `.pcp` and `.tmp` files before reading the results.

References: time.sim, print.prt, weather outside the observed record.

Daily or monthly output

Each object row of `print.prt` has four switches, for daily, monthly, yearly and average annual files. Set the column to `y` to write that file, for example daily basin water balance:

```
objects                 daily       monthly        yearly         avann
basin_wb                    y             n             y             y
```

How the rows are read depends on the second value on line 7 (`use_obj_lbls`, labelled `dbout` in files from older Editors):

- `y`: rows are matched by name. A row can be added, removed or moved, and the carbon and gwflow rows can be used.
- `n`: rows are read by position in a fixed order. Do not add, remove or reorder rows: every later row would apply to the wrong output, without a message.

Daily output of many objects over a long period can take several gigabytes, so check the free disk space before a long run. After the run, `files_out.out` lists every output file that was opened.

References: print.prt, Output naming convention.

PET method

Edit `pet`, the third value in `codes.bsn`:

`pet`	Method	Weather it uses
0	Priestley-Taylor	temperature and solar radiation
1	Penman-Monteith	temperature, solar radiation, relative humidity and wind
2	Hargreaves	temperature
3	weather station PET	in 62.0.1 this gives Hargreaves PET (see below)

The PET method does not change which weather files are read. A variable that the station takes from the weather generator (`sim` in `weather-sta.cli`) is generated in every case, so Penman-Monteith with generated humidity and wind uses generated values. With `pet` = 3, 62.0.1 replaces the measured station PET with Hargreaves PET before the HRUs use it, so this code gives Hargreaves PET times `pet_co`.

After the run, compare `pet` and `et` in `hru_wb_aa.txt` or `basin_wb_aa.txt` with the earlier run.

References: PET method, weather-sta.cli.

Green-Ampt infiltration

Three files change:

1. `codes.bsn`: set `gampt`, the 21st value, to 1.
2. `time.sim`: set `step` to the number of time steps per day, for example 24 for hourly steps.
3. The precipitation files: give sub-daily records with the same number of values per day as `step`. With `step` above 1, 62.0.1 does not split a measured daily record over the time steps, so a daily `.pcp` file gives no rain in any time step.

Plant growth, soil water and nutrient cycling still run once a day.

After the run, compare `surq_gen` and `perc` in `basin_wb_aa.txt` with the curve number run.

References: runoff method, time step, *.pcp.

HRU addition

An HRU needs a row in several files. Copy the rows of an existing HRU and change them:

1. `hru.con`: add a row with the next `id`. Rows must be numbered 1, 2, 3 and so on in file order. Set `area`, `lat`, `elev`, the `hru` column (the row number of the new HRU in `hru-data.hru`) and `wst` (a station name in `weather-sta.cli`).
2. `hru-data.hru`: add a row whose `topo`, `hydro`, `soil`, `lu_mgt`, `soil_plant_init`, `snow` and `field` names exist in the corresponding files. A misspelled name is not reported: the HRU uses the first record of that file.
3. The HRU's outflow: either give it outflow connections in `hru.con` (`out_tot` and the groups after it), or add it as an element of a routing unit in `rout_unit.ele` and the element list in `rout_unit.def`, as in Osu_1hru. With neither, its water leaves the model.
4. `ls_unit.ele` and `ls_unit.def`, if the project has them: add the HRU as an element. The basin output files (`basin_wb` and the others) are added up from these elements with their `bsn_frac` values as given, so the `bsn_frac` values should add up to 1 after the change.
5. `object.cnt`: add 1 to `hru` and to `obj`.

After the run, `checker.out` has one row per HRU, in HRU order, with the soil and hydrology values the new HRU ended up with. Check `diagnostics.out` for names that were not found, and remember that the `topo`, `hydro`, `soil`, `lu_mgt`, `soil_plant_init` and `snow` names are not checked there.

References: *.con, hru-data.hru, object.cnt, print.prt.

Carbon model

The dynamic soil carbon model needs three changes:

1. `codes.bsn`: set `carbon`, the 14th value, to 2. The value 1 is reserved for C-FARM, which is not implemented, and runs the model with no organic matter cycling at all. Datasets from before revision 62 used 1 for the dynamic model and must be changed.
2. `file.cio`: name the carbon file as the third name on the `basin` line, for example `basin codes.bsn parameters.bsn carbon.bsn`.
3. Provide `carbon.bsn` and `carbon_lyr.bsn` in the project folder. `refdata/Ames_sub1` has both, and they can be copied as a start. SWAT+ stops with an error if either is missing. The old `carb_coefs.cbn` is no longer read.

For carbon output, `print.prt` must use `use_obj_lbls = y`, and the carbon rows are added by name. The newer rows (`hru_cb_gl`, `hru_cb_lyr`, `hru_cb_cpool` and others) each switch one family of files, and the legacy rows `hru_cb` and `hru_cb_vars` switch the files the carbon notebook reads. Daily per-layer carbon output is large: in Ames_sub1 it makes up most of the 9 GB of a full run, so start with a short period.

After the run, `files_out.out` lists the carbon files that were opened, and Carbon outputs describes their columns.

References: carbon model, carbon.bsn, older file.cio files, carb_coefs.cbn.

Sources

- Simulation period: `time_read.f90` sets `time%day_start` to 1 when it is 0 or less.
- Output rows: `basin_print_codes_read.f90` reads the second value of line 7 into `pco%use_obj_labels` and the object rows by position or by name accordingly.
- `codes.bsn`: `basin_read_cc.f90`. Column positions: `pet` 3 (`bsn_cc%pet`), `carbon` 14 (`bsn_cc%cswat`), `gampt` 21 (`bsn_cc%gampt`). PET with `pet` = 3 and sub-daily precipitation: see the Sources sections of codes.bsn and time.sim.
- HRUs: connection rows are read by `hyd_read_connect.f90`, HRU rows and their name lookups by `hru_read.f90`, counts by `basin_read_objs.f90`, and `checker.out` is written by `proc_hru.f90`. Basin output weighting by `bsn_frac`: see nutrient balance and losses.
- Carbon: the `basin` line of `file.cio` is read in `readcio_read.f90`, and `carbon.bsn` and `carbon_lyr.bsn` by `carbon_bsn_read.f90`.

The SWAT+ Fortran source has a generated API reference, built with FORD and published at

swat-model.github.io/swatplus

It is rebuilt for every release tag; on 20 October 2026 it shows "SWAT+ Model Sources (62.0.1)". The reference contains:

- every module, derived type, subroutine and function in `src/`;
- call and caller graphs;
- the source of each `.f90` file;
- the comment block at the top of each routine, searchable.

This site and the FORD reference

Use this site for input and output files, model theory, calibration and running the model. Use the FORD reference to find what a routine does, which variables it reads and writes, and where it is called from. The collapsed Sources block at the end of each page names the routines behind it, which are the starting points in FORD.

Local build

The FORD settings are in `ford.md.in` in the source repository; CMake fills in the version and writes `ford.md`:

```
cmake -B build -D TAG=$(git describe --tags --abbrev=0)
ford ford.md
```

The HTML is written to `build/doc`. FORD needs Graphviz for the call graphs.

Sources

- `ford.md.in` and `.github/workflows/doc.yml` (runs on tags matching `*.*` and `*.*.*`, builds with `ford ford.md` and publishes `build/doc` to GitHub Pages).

Purpose

These pages help when a SWAT+ run stops, completes with results that do not make sense, or takes too long. First runs and installation problems are covered in Getting started, and adjusting parameters to observations in Calibration.

Pages

- Common runtime errors: the messages SWAT+ prints when a run stops or a name in the inputs is not matched, the names it matches without a message, files it creates when they are missing, and inputs in an older format.
- Water balance issues: implausible precipitation, runoff, ET, percolation, lateral and tile flow, and basin totals that do not match the HRUs.
- Performance tips: build type, time step, output volume and the size of the carbon output files.

Checks after every run

1. `simulation.out` ends with `Execution successfully completed`, and `success.fin` exists. If not, the run stopped early: keep the screen output, since stop messages are not copied to the log files.
2. Read `diagnostics.out` to the end. Every line after the first is a name that was not matched, and SWAT+ used a default or left the feature off.
3. Check the names that are not reported when they do not match, above all the names in `hru-data.hru`, which fall back to the first record of each file. See Names matched without a message.
4. Read `checker.out`: the soil group, rooting depth, field capacity, `perco`, `latq_co` and tile flag of each HRU show whether the intended soil and parameters were used.
5. Compare the average annual water balance with regional estimates before calibrating.

The file layouts are in Checker and diagnostics. The revision of the executable is in the banner at the top of `simulation.out` (`Revision 62.0.1`); inputs written for an earlier revision can be misread without a message (see Old-format inputs).

Sources

- `diagnostics.out` and `files_out.out` are opened in `proc_bsn.f90`; the banner and the completion message are written by `main.f90.in`.
- Name lookup for HRUs: `hru_read.f90`.

Purpose

This page lists the messages SWAT+ 62.0.1 writes when a run stops or when it cannot match a name in the inputs, and the input problems that change the results without any message. For each one it gives the cause and the fix.

Message locations

Place	Content
the screen (standard output)	errors that stop the run, written just before it stops
`diagnostics.out`	names that could not be matched while the inputs were read; the run continues
`simulation.out`	the revision banner and one line per simulated day, ending with `Execution successfully completed`

Place	Content
gwflow_record	messages from the gwflow module, including the ones that stop the run

Messages printed on the screen are not copied to simulation.out. When the model is started by a script or another program, keep the screen output, for example with swatplus > run.log 2>&1. The layout of the log files is described in Checker and diagnostics.

In the messages below, <name> stands for a name from the inputs and <n> for a number. SWAT+ pads names with spaces, so the gaps in a real message are wider.

Errors that stop the run

Missing object.cnt

```
Cannot find object.cnt input file
```

The file named on the simulation line of file.cio (normally object.cnt) does not exist, or the name is null. Usually the model was started outside the project folder. Change into the folder that contains file.cio and start it again. Started in an empty folder, SWAT+ also leaves a few files behind (see Files the model creates).

Carbon input files

With carbon = 2 in codes.bsn, two files are required. SWAT+ stops with one of these messages, on the screen and in diagnostics.out:

```
ERROR: <name> is required when carbon is enabled (codes.bsn carbon = 2)
ERROR: <name> could not be opened
ERROR: <name> data/values line is missing or could not be parsed (expected 28 values)
ERROR: <name> provided <n> row(s); expected <n>
```

The first file is the one named in the third slot of the basin line of file.cio, normally carbon.bsn. The second has no slot: its name is the first name with _lyr added before the extension, normally carbon_lyr.bsn. Provide carbon.bsn with its 28 values and carbon_lyr.bsn with a row for each layer group, or set carbon to 0. The old carb_coefs.cbn is no longer read.

If the message names a file that is not a carbon file, such as ERROR: climate is required when carbon is enabled, the basin line of file.cio has only two names. See Old-format inputs.

print.prt rows

```
Error: The output object <name> in the input file print.prt is not a valid object.
       and cannot be processed
Error: <name> print object is duplicated in the input file print.prt.  Aborting
```

These checks apply when use_obj_lbls is y in print.prt, so that the rows are read by name. A row name that SWAT+ does not know, or a row given twice, stops the run. The valid names, and the files each one opens, are listed in print.prt rows and files.

Output folder

```
! ERROR: cannot create path specified in file.cio (out_path)
  > Windows-style path detected on Unix system: <name>
! ERROR: cannot create or access path specified in file.cio (out_path)
  > Path: <name>
```

The optional output folder line of file.cio names a folder SWAT+ cannot use. On Linux and macOS a folder name that contains : is taken as a Windows path and refused. The second message means the folder could not be created or opened, often because of file permissions. See Output folder.

gwflow input files

The gwflow module writes its messages to `gwflow_record` in the run folder, so the screen shows only that the program stopped. These messages stop the run:

Message in `gwflow_record`	Cause
`ERROR: gwflow.cells row has too few columns at i=<n>`	row *n* of `cells.gw` has fewer than the 14 required columns
`ERROR: gwflow.cells row out of order at i=<n> cell_id=<n>`	the cell IDs in `cells.gw` are not 1, 2, 3, … in file order
`ERROR: gwflow.cellcons row has too few columns at i=<n>`	row *n* of `cellcon.gw` has fewer than 2 columns
`ERROR: gwflow.cellcons row out of order at i=<n> cell_id=<n>`	the cell IDs in `cellcon.gw` are not in file order
`neither gwflow.hrucell or gwflow.lsucell found; stop simulation`	`conn_type` in `codes.gw` is neither 1 nor 2

The messages use older file names: `gwflow.cells` is `cells.gw` and `gwflow.cellcons` is `cellcon.gw`. With `conn_type` 1 or 2 and neither `hrucell.gw` nor `lsucell.gw` present, the run does not stop. See gwflow inputs.

Fortran run-time errors

Some readers do not check what they read. A missing line, a missing file or text where a number is expected then stops the run with a message from the Fortran run-time library instead of SWAT+. With gfortran builds it contains `Fortran runtime error`, and with Intel builds it starts with `forrtl: severe`. The message usually gives the file name or the unit number. Check that file for a missing or extra line or a wrong number of columns, and compare it with its page in the Input Reference.

Most readers do check, and do not stop. When a value in a row cannot be read, they stop reading that row without a message, and the columns after it keep their defaults. A row with a stray character or a missing value can therefore change the results silently.

Messages in diagnostics.out

Line 1 of `diagnostics.out` is `DIAGNOSTICS.OUT FILE`. Each later line is written while the inputs are read, and the run continues. From a run of the `Ames_sub1` reference dataset:

```
agrr_lum                              AGRR_rot                              not found in
↪ management.sch
ames.tem                                        file not found (tgage)
```

Land use names

Written for a row of `landuse.lum` that names a record that is not found. The line gives the land use name, the missing name and one of these endings:

Ending	Column	Result
`not found in plant.ini`	`plnt_com`	no plants at the start
`not found in management.sch`	`mgt`	no scheduled operations: no planting, fertilizer or harvest
`not found in cntable.lum`	`cn2`	built-in curve numbers 30, 55, 70 and 77 for soil groups A to D
`not found (landuse-consprac)`	`cons_prac`	P factor 1.0
`not found (landuse-tiledr)`	`tile`	no tile drain
`not found (landuse-septic)`	`sep`	no septic system
`not found (landuse-fstrip)`	`vfs`	no filter strip
`not found (landuse-grassww)`	`grww`	no grassed waterway

Ending	Column	Result
not found (landuse-bmpuser)	bmp	no user BMP

Correct the name in landuse.lum, or add the missing record to the file it points to. See Name matching. Ames_sub1 writes four not found in management.sch lines, so the HRUs with those land uses have no management.

Plant names

```
mgt schedule <n> op numb <n> <name> not found in plants.plt database
plant com <n> plant numb <n> <name> not found in plants.plt database
```

The first is written for a plnt line of management.sch whose plant name is not in plants.plt; the numbers are the position of the schedule in the file and of the line in the schedule. The second is written for a plant in plant.ini that is not in plants.plt. Plant names are case-sensitive.

Weather station names

```
<name> file not found (tgage)
```

A station in weather-sta.cli names a data file that is not listed in the matching index file. The ending gives the column: (pgage) precipitation and pcp.cli, (tgage) temperature and tmp.cli, (sgage) solar radiation, (hgage) humidity, (wgage) wind, (petgage) PET and (atmodep) atmospheric deposition. For the five weather variables, SWAT+ then generates that variable for the station on every day from its weather generator record. In Ames_sub1, weather-sta.cli names ames.tem while tmp.cli lists ames.tmp, so the temperatures are generated.

file not found (wgn) means the wgn name of the station is not in weather-wgn.cli. 62.0.1 has no record to fall back on in that case, and the generated weather of that station is undefined. Correct the name before running. See Unmatched names.

Reservoir, wetland and other names

Ending	Name that was not found
not found (res-init), (res-hyd), (res-release), (res-sed), (res-nut)	a record named in reservoir.res
not found (wet-init), (wet-hyd), (wet-release), (wet-sed), (wet-nut)	a record named in wetland.wet
not found (wetland.wet)	the surf_stor name of an HRU in hru-data.hru; the HRU has no surface storage
not found (field.fld)	the field name of an HRU; the HRU uses the built-in field size
not found (plants.plt)	the plant of an HRU in hru-lte.hru
entry in (hru-lte.hru) not found in (lum.dtl)	a decision table named in hru-lte.hru
not found (initial.cha), (hydrology.cha), (sediment.cha), (nutrients.cha)	a record named in the legacy channel.cha, which 62.0.1 reads but does not route

Carbon layer files

```
<name>: layer id <n> is out of range (1 to <n>); row ignored
carbon_layers.prt: n_layers must be >= 1; got <n>; default <n> kept
```

The first means a row of carbon_lyr.bsn has a layer number other than 1 or 2, and the row is skipped. The second means the optional layer count file has a value below 1, and the per-layer carbon outputs use the largest layer count of any HRU.

Names matched without a message

Several names are looked up without any report when they are not found. The run continues with the result in the table, so check these names by hand or in SWAT+ Editor.

Name	Result when it is not found
`topo`, `hydro`, `soil`, `lu_mgt`, `soil_plant_init` or `snow` in `hru-data.hru`	the HRU uses the first record of the target file. The reader has messages for these names, but they are never written in 62.0.1.
`wst` in a connection file such as `hru.con`	the object uses the first station in `weather-sta.cli`. The stations must also be listed in ascending order of name for the lookup to work. See Weather station lookup.
fertilizer name of a `fert` line in `management.sch`	nothing is applied. The name is read from `op_data1`; see the management.sch example.
harvest operation of a `harv` line	nothing is harvested
tillage name of a `till` line	no soil is mixed
transplant name of a `plnt` line or a decision table plant action	the plant starts from seed. In 62.0.1 the file is `transplant.ops`, and a file named `transplant.plt` is not read. See transplant.ops.
manure name of a `manu` line	the manure is found by name in `manure_db.frt`, but its make-up comes from the row of `manure_om.frt` in the same position, whatever `org_min` names. See manure_om.frt.
decision table named on an automatic operation line of `management.sch`	the table is not applied. See Automatic operations.
`urban` or `ov_mann` in `landuse.lum`	the HRU is not urban, or keeps Manning's n of 0.05

Files the model creates

For a number of input files, 62.0.1 opens the named file without checking that it exists, and the operating system then creates an empty file of that name. The run continues as if the file had no records. This happens for `time.sim` and `print.prt`, the place-of-use and point-of-diversion files of water allocation, `wtps_wuses.wal`, and the initial-condition files for pesticides, pathogens, metals, salts and constituents (`pest_hru.ini`, `salt_hru.ini`, `cs_aqu.ini` and others). For some of these slots a `null` name creates a file named `null`.

An empty file left behind changes the next run: an empty `time.sim`, for example, leaves the run without a valid simulation period. After a run that misbehaves, list the empty files in the project folder (`find . -size 0` on Linux and macOS) and restore or delete them. If in doubt, write the inputs again from SWAT+ Editor.

Old-format inputs

Files written for an earlier revision are read without a version check. These changes matter most for 62.0.1:

- **file.cio `basin` line.** It now has three names, the third being the carbon file. With only two, every later line of `file.cio` is read one line too low and all later file names are wrong, with no message. With `carbon = 2` the run stops with `ERROR: climate is required when carbon is enabled`. See Older file.cio files.
- **file.cio `water_rights` line.** It now has two names. An old line with `water_allocation.wro element.wro water_rights.wro` makes SWAT+ read `water_allocation.wro` as the place-of-use file.
- **transplant.plt.** Renamed to `transplant.ops`. A project that still has `transplant.plt` plants from seed.
- **pesticide.pes.** Three columns were added after the existing 13 values. Older rows are still read, and the new columns keep their defaults, which means no plant uptake. See pesticide.pes.
- **print.prt.** A file with `dbout` in place of `use_obj_lbls` is read by position, so an extra or missing row moves the switches of every later row to the wrong output group. See print.prt.
- **carb_coefs.cbn.** No longer read. Its settings are in `carbon.bsn` and `carbon_lyr.bsn`.

Most input files start with a title line and a column header line, and SWAT+ skips both. A file that starts with the header loses its first data row, with no message. In the reference data, `tillage.til` of `Ames_sub1` and `puddle.ops` of `Osu_1hru` have no title line, so their first entries are not read; see tillage.til and puddle.ops.

Runs that stop without a message

Full disk. When the disk fills, the run stops with a Fortran run-time error about writing a file, or with no message at all, and the last output files are cut short. Other programs that use the same disk start to fail as well. Daily output is the usual cause: a full 46-year run of the `Ames_sub1` reference dataset writes about 9 GB, almost all of it the daily carbon files switched on by its `hru_cb` and `hru_cb_vars` rows. Check the free space before long runs (`df -h .` on Linux and macOS) and switch off the daily outputs you do not need; see Output volume.

The last day. The last daily line in `simulation.out` is the last day that finished, so the problem occurred on the next day. Climate records that are shorter than the simulation do not stop the run: SWAT+ generates the missing days (see Weather outside the observed record).

A run that never gets past its first day. Earlier revisions could repeat the operations of a management schedule without end when every operation had the same month and day. 62.0.1 runs each operation at most once per pass on a given day, so a run that hangs at the start with 62.0.1 has another cause. A debug build with run-time checks usually stops with a message that points to it.

Related files

- Checker and diagnostics: the layout of `simulation.out`, `diagnostics.out`, `checker.out` and `files_out.out`.
- Water balance issues: runs that complete but give implausible results.
- Install and Quick start with Ames_sub1: first runs, `Permission denied` and the start-up banner.

Sources

- Stops: `basin_read_objs.f90` (`object.cnt`), `carbon_bsn_read.f90`, `basin_print_codes_read.f90` and `print_prt_error` in `basin_module.f90`, `output_path_module.f90`, `gwflow_read.f90`
- `diagnostics.out` is opened as unit 9001 in `proc_bsn.f90`. Writers: `landuse_read.f90`, `read_mgtops.f90`, `readpcom.f90`, `cli_staread.f90`, `hru_read.f90`, `hru_lte_read.f90`, `res_read.f90`, `wet_initial.f90`, `ch_read.f90`, `carbon_layers_read.f90`
- Weather fallback: `climate_control.f90`, `cli_precip_control.f90`, `cli_bounds_check.f90`, `cli_wgnread.f90`
- Same-date guard: `mgt_operatn.f90`

Purpose

When a run completes but `basin_wb_aa.txt` or `hru_wb_aa.txt` looks wrong, the cause is usually in the climate inputs, the curve numbers, the management, the soil and hydrology parameters, or the way the basin totals are weighted. This page goes through the water balance term by term. Many of the causes leave no message, so the checks below name the file and column to look at.

Before changing parameters, read `diagnostics.out` and `checker.out` (see Checker and diagnostics) and the list of names matched without a message. An HRU that silently uses the first record of `landuse.lum`, `soils.sol` or `hydrology.hyd` gives a water balance that no parameter change will fix.

Balance check

For an HRU, landscape unit or the basin, the average annual water balance should close:

$$P + I + Q_{\text{runon}} \approx ET + WYLD + PERC + \Delta SW + \Delta SNO$$

where P is `precip`, I is `irr`, Q_{runon} is `surq_runon` + `latq_runon`, ET is `et`, $WYLD$ is `wateryld` (`surq_gen` + `latq` + `qtile`), $PERC$ is `perc`, and ΔSW and ΔSNO are the changes in soil water and snow over the period.

The columns are described in Water balance outputs. A large gap points to wetland storage, gwflow exchange or water held in the surface runoff lag. A term that is far from what the climate and land cover suggest points to one of the sections below.

Precipitation

The `precip` column does not match the observed rainfall.

- **Weather station of the HRU.** Each HRU takes its station from the `wst` column of `hru.con`. A name that is not in `weather-sta.cli`, or a `weather-sta.cli` whose stations are not in ascending order of name, gives the HRU the first station in the file, with no message. See Weather station lookup.
- **Data file of the station.** A line `file not found (pgage)` in `diagnostics.out` means the station names a precipitation file that is not listed in `pcp.cli`, and SWAT+ generates the rainfall from the weather generator on every day. See Weather station names.
- **Simulation period.** Days before the start or after the end of the record, and days flagged as missing, are generated. See Weather outside the observed record.
- **Sub-daily runs.** With `step` above 1 in `time.sim`, a station with daily precipitation records gets no rainfall in its time steps. A sub-daily run needs sub-daily records or generated precipitation. See Time step.

`hru_pw` files and the `precip` column of `hru_wb` show what each HRU received, so compare a few HRUs with the station file before looking further.

Surface runoff

`surq_gen` is too high or too low.

- **Curve numbers.** The `cn2` column of `landuse.lum` names a record in `cntable.lum`, and the soil group in `soils.sol` picks one of its four values. A name that is not found is reported as `not found in cntable.lum`, and the HRU uses the built-in values 30, 55, 70 and 77 for groups A to D. The `hydgrp` column of `checker.out` shows the soil group each HRU uses.
- **Moisture adjustment.** `cn3_swf` in `hydrology.hyd` places the curve number for wet conditions between field capacity (0) and saturation (1). See Curve number soil water factor.
- **Runoff method.** `gampt` in `codes.bsn` selects the curve number method (0) or Green-Ampt (1), which is meant for sub-daily runs. See Runoff method.
- **Lag.** `surq_gen` is the runoff after the surface runoff lag and `surq_cont` the runoff generated on the day. A large difference between them in yearly totals means water is held in the lag.

Evapotranspiration

`et` is too high or too low.

- **PET method.** `pet` in `codes.bsn`: 0 Priestley-Taylor, 1 Penman-Monteith, 2 Hargreaves, 3 the PET file of the station. Penman-Monteith uses solar radiation, humidity and wind, which are generated when the station has no records for them. Compare the `pet` column of the water balance file with a regional estimate.
- **Compensation factors.** `esco` and `epco` in `hydrology.hyd` set how far down the profile soil evaporation and plant uptake can reach. `checker.out` lists the values each HRU uses. See Soil evaporation.
- **Plant cover.** If plants do not grow, `eplant` stays low and more water leaves as soil evaporation, runoff and percolation. Check biomass and LAI in the `hru_pw` files and yields in `crop_yld_aa.txt` (see Plant and weather).

Management

Crops do not grow, or grow without fertilizer or harvest.

- A land use whose `mgt` schedule is not in `management.sch` is reported as `not found in management.sch`, and its HRUs get no operations. The `Ames_sub1` reference dataset has four such land uses.

- Several operation names are not reported when they are not found: a fertilizer name gives no fertilizer, a harvest operation no harvest, a tillage name no mixing, and a transplant name a plant started from seed. In the `Ames_sub1` reference dataset, the fertilizer names of every `fert` line are in the wrong column, so the dataset applies no mineral fertilizer in 62.0.1. See the management.sch example.
- Set `mgtout` to `y` in `print.prt` to write `mgt_out.txt`, the log of every operation and decision table action that ran. An operation missing from the log did not happen.

Percolation

`perc` is close to zero.

`perco` in `hydrology.hyd` limits percolation out of the bottom of the soil profile, and the relation is strongly non-linear: 0.5 lets about 0.5 % of the potential percolation through, 0.9 about 3 %, and only values close to 1 allow free drainage. The value 0.5, used for every HRU in `Ames_sub1`, therefore leaves almost nothing to percolate. See Percolation from the soil profile.

Also check `sumfc` and `sumul` in `checker.out`, the water held at field capacity and saturation over the profile. Values that are the same for every HRU can mean the HRUs all use the first record of `soils.sol` because their soil names did not match.

Lateral and tile flow

`latq` or `qtile` is out of range.

- **Lateral flow.** `latq_co` and `lat_ttime` in `hydrology.hyd` scale and delay lateral flow. See Lateral flow.
- **Tile drains.** An HRU has a tile drain when the `tile` column of its land use names a record in `tiledrain.str`. The `tiledrain` column of `checker.out` shows 1 for those HRUs. A decision table `tile_install` action has no effect on an HRU whose land use had no tile record at the start. A tile `lag` of 0.01 h or less keeps tile water in the lag store, so `qtile` stays at zero. See Tile flow methods.
- **Septic systems.** In 62.0.1 a septic system adds no water or nutrients to the soil. See Septic simulation in 62.0.1.

Basin totals

The basin or landscape unit values do not match the HRU values.

Basin rows are sums of the HRU values weighted by the `BSN_FRAC` column of `ls_unit.ele`, and landscape unit rows by `RU_FRAC`. 62.0.1 uses these fractions as given, without scaling them to add up to 1. In the `Ames_sub1` reference dataset the 12 `BSN_FRAC` values add up to 2, so every basin water balance, nutrient balance, losses and plant file shows twice the area-weighted HRU average. Set each fraction to the HRU area divided by the basin area, and check that they add up to 1. The basin sum also assumes `ls_unit.ele` lists one element per HRU, in HRU order. See Aggregation and Nutrient balance and losses.

Routing unit hydrograph files (`ru_*`) add up every hydrograph type of every element: total flow plus the recharge, surface, lateral and tile parts. Their `flo` is therefore larger than the routing unit's outflow, which is in the `hydout` files. See ru files.

`flo` in the hydrograph files is a volume in m³ per print step, although the units row gives m³/s. Divide a daily value by 86,400 to compare it with a gauge record. See Hydrograph columns.

Related files

- Common runtime errors: messages and the names matched without a message.
- Water balance outputs: the columns and how they are aggregated.
- Ames_sub1 tutorial: a worked look at the reference dataset's water balance.
- Soft calibration: adjusting the water balance towards regional targets, once the inputs are right.

Sources

- Weather station and gage lookup: cli_staread.f90, climate_control.f90, cli_precip_control.f90
- HRU and land use name lookup: hru_read.f90, hru_module.f90, landuse_read.f90
- Operation name lookup and execution: read_mgtops.f90, mgt_sched.f90
- Percolation: perco conversion in topohyd_init.f90, use in swr_percmicro.f90
- Basin and landscape unit weighting: basin_output.f90, lsu_output.f90

Item	Fortran
BSN_FRAC, RU_FRAC in ls_unit.ele	lsu_elem%bsn_frac, lsu_elem%ru_frac
perco	hyd_db%perco, converted to hru%hyd%perco_lim
HRU name pointers	hru_db%dbs (type hru_databases, initialised to 1)

Purpose

The run time of SWAT+ depends on the length of the simulation, the number of objects (HRUs, channels, aquifers, reservoirs), the time step and the amount of output. The disk space a run needs depends almost entirely on the output switched on in print.prt. This page covers what can be changed.

One run per processor core

SWAT+ is a serial program: the source has no OpenMP or MPI code, and one run uses one processor core. To use several cores, run several scenarios or calibration samples at the same time, each in a separate copy of the project folder. Output file names are fixed, so two runs in the same folder overwrite each other's files.

Build type

The executables on the releases page are optimized builds (Rel in the file name; see Install). When building from source, set the build type:

```
cmake -B build -D CMAKE_BUILD_TYPE=Release
cmake --build build -j 8
```

-j 8 only speeds up the compilation. Without a build type, CMake adds no optimization flag and the executable name has no -Rel or -Dbg suffix. A Debug build adds run-time checks (array bounds and floating-point traps with gfortran) and runs much slower; use it to track down a crash and switch back afterwards. With gfortran on Windows, the 62.0.1 CMake setup adds the gfortran run-time checks to every build type, including Release. See Building from source.

Time step

step in time.sim sets the number of time steps per day. With a value above 1, precipitation, infiltration, surface runoff and channel and reservoir routing are calculated for every time step, while plant growth, soil water and nutrient cycling still run once a day. Run time grows with the number of steps. Use a sub-daily step only when the question needs it, for example Green-Ampt infiltration or sub-daily peak flows.

Output volume

Each y in the daily column of a print.prt row writes one row per object per day, so daily output is about 365 times the size of yearly output and 30 times the size of monthly output. Average annual output adds one row per object. Ways to reduce it:

- Switch daily output on only for the rows you will use, and use monthly, yearly or average annual output for the rest.
- Limit the daily rows to the period you need with the print period on line 3 of print.prt (day_start, yrc_start, day_end, yrc_end and interval), and skip warm-up years with nyskip. See Print period.

- Leave `csvout` at `n` unless you need the `.csv` files: with `y`, every output file is written twice, once as text and once as `.csv`.
- With gwflow, check the gwflow rows of `print.prt`. Several gwflow output groups are on unless a row switches them off; see gwflow outputs.

`files_out.out` lists every output file a run opened. Check it after a short first run to see what a long run will write.

Carbon output files

The carbon files are the largest outputs, because the per-layer files have a value for every soil layer and variable.

- **Legacy carbon rows.** The `hru_cb` and `hru_cb_vars` rows with `l` in the daily column write per-layer diagnostics for every HRU on every day. The `Ames_sub1` reference dataset has this setting, and a full 46-year run of its 12 HRUs writes about 9 GB, almost all of it these files. Set the daily column of these rows to `n` unless you need the daily layer values. See Legacy CSU carbon outputs.
- **Per-layer carbon rows.** In 62.0.1 the files of the `hru_cb_lyr`, `hru_cb_cpool`, `hru_cb_npool`, `hru_cb_plt`, `hru_cb_flux`, `hru_cb_drv`, `hru_cb_dyn` and `hru_cb_snap` rows repeat their block of HRU rows once per HRU. Their size grows with the square of the number of HRUs: 100 HRUs give 100 times the rows they should. Use yearly or average annual output for these rows in models with many HRUs. See Repeated rows.
- **Number of layers.** The per-layer files have as many columns per variable as the HRU with the most soil layers. An optional `carbon_layers.prt` file sets a smaller number, for example to write only the top three layers. See Layer count file.

Check the free disk space before a long run with daily output (`df -h .` on Linux and macOS). A disk that fills during a run stops it, often without a clear message; see Runs that stop without a message.

Object count and storage

Run time grows roughly in proportion to the number of HRUs, channels, aquifers and routing units. When testing inputs, use a short simulation period or a small part of the watershed, and run the full model once the inputs are settled.

SWAT+ reads its inputs at the start and writes the outputs during the run. With many daily outputs, a project folder on a local disk runs noticeably faster than one on a network share.

Summary

Change	Effect
`CMAKE_BUILD_TYPE=Release`	optimized code; a `Debug` build runs much slower
`step` 0 or 1 in `time.sim`	daily calculation, fastest
daily output only where needed	smaller files, faster runs
print period and `nyskip` in `print.prt`	fewer daily rows
`csvout = n`	half the output files
no `l` in the daily column of `hru_cb` and `hru_cb_vars`	avoids the largest carbon files
yearly or average annual per-layer carbon files	avoids the repeated HRU rows growing with the square of the HRU count
several runs in separate folders	uses several processor cores

Sources

- Build flags: `CMakeLists.txt` and `CMakePresets.json`
- Output switches: `basin_print_codes_read.f90`, file opening in `output_landscape_init.f90`
- Carbon writers and their calls: `command.f90`, `soil_nutcarb_write.f90`, `soil_carbvar_write.f90`, layer count in `carbon_layers_read.f90`